



CONFIDENTIAL

STi5512 SET TOP BOX BACKEND DECODER WITH INTEGRATED HOST PROCESSOR

PRELIMINARY DATA

■ High performance graphics system

- High resolution chroma mode (4:4:4) for RGB output
- 2 to 8 bits per pixel OSD options
- Link list control
- 4-bit mixing factor by region or 6-bit mixing for each CLUT entry (anti-aliasing)
- 8-bit Y, U and V resolution palette
- Extra YUV plane for background images or graphics
- 2D, paced BLT engine with "fill" function
- Anti-flicker and anti-flutter filters

■ Enhanced 32-bit VL-RISC CPU - 60 MHz clock

- Fast integer/bit operation and very high code density

■ High-performance memory/cache subsystem

- 2 Kbytes instruction cache, 2 Kbytes data cache or SRAM, 4 Kbytes SRAM
- 200 Mbytes/s maximum bandwidth

■ Combined video and audio decoder core

- Video decoder fully supports MPEG-2 MP@ML. Letter box (16:9 and 14:9), 2:1, 3:1 and 4:1 downsizing
- Memory reduction - PAL MP@ML in 12 Mbits
- Audio decoder supports layers 1 and 2 of MPEG 1, and an AC-3 interface to an external decoder
- Digital YCrCb output in 4:2:2 format

■ PAL/NTSC/SECAM encoder

- Macrovision™ 7.01, teletext and closed caption
- Outputs RGB with 10-bit DACs and CVBS, Y, C and component output (YUV) with 10-bit DACs
- Separate OSD control for RGB and CVBS outputs
- Genlock support

■ High performance SDRAM memory interface

- Supports two 16- or one 64-Mbit 100 MHz SDRAMs
- Accessible by MPEG decoder, PTI, DMAs and CPU
- High bandwidth access from CPU allows high performance OSD operations

■ Programmable external memory interface (EMI)

- Glueless interface to SDRAM

■ Programmable transport interface

- Parallel or serial input
- Supports DVB bit-streams
- More than 32 PIDs supported
- DVB descrambler
- 32 SI/PSI filters of 8 bytes

■ Vectored interrupts - 8 prioritized levels

■ Interfaces and DMA engines

- 2 SmartCard interfaces, 2 UARTs, 2 I²C/SPI controllers, 3 PWM outputs, 4 timers, 3 capture timers
- Block move DMA
- Teletext interface, input from external source
- IEEE1284 port, or IEEE1394 A/V link layer interface

■ Low power controller/real time clock/watchdog

■ JTAG test access port

■ Professional toolset support

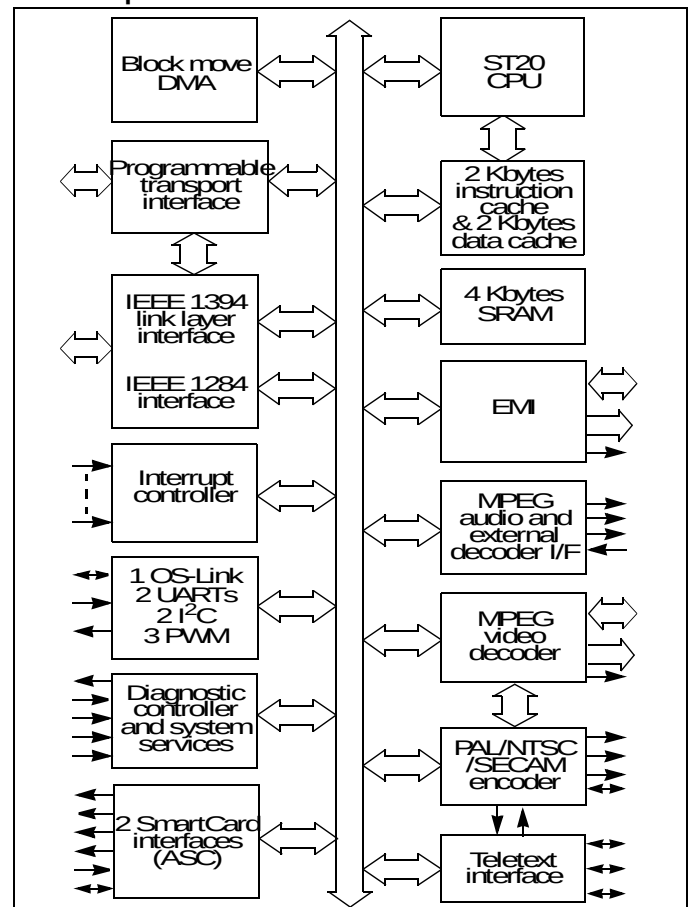
- ANSI C compiler and libraries
- Advanced debugging tools

■ Non-intrusive debug controller

- Hardware breakpoints
- Real-time trace

APPLICATIONS

■ Set top boxes to DVB standards



1	Introduction	8
1.1	STi5512 options and order codes	8
1.2	Differences between the STi5512 and the STi5510	9
2	Architecture overview	10
2.1	STi5512 functional modules	12
3	Summary specification.....	17
4	Pin list	19
4.1	PIO pins and alternative functions	23
5	Package specifications	25
5.1	Thermal guidelines	25
5.2	Pin-out	25
5.3	Package dimensions	28
6	Central processing unit.....	29
6.1	Registers	29
6.2	Processes and concurrency	30
6.3	Priority	31
6.4	Process communications	32
6.5	Timers	32
6.6	Traps and exceptions	33
7	Instruction set	38
7.1	Instruction cycles	38
7.2	Instruction characteristics	39
7.3	Instruction set tables	40
8	Interrupt system.....	47
8.1	Interrupt vector table	48
8.2	Interrupt handlers	48
8.3	Interrupt latency	50
8.4	Pre-emption and interrupt priority	50
8.5	Restrictions on interrupt handlers	50
8.6	Interrupt level controller	50
8.7	Interrupt assignments	51
9	Memory map.....	52
9.1	Mapping	52
9.2	System memory use	55

10	Memory	56
10.1	External memory	56
10.2	On-chip SRAM memory	56
10.3	Caching	57
11	External memory interface	63
11.1	Overview	63
11.2	Common pin functions	65
11.3	DRAM device type pin functions and access cycles	67
11.4	SRAM/peripheral device type pin functions and access cycles	75
11.5	SDRAM device type pin functions and access cycles	80
11.6	EMI configuration	94
11.7	EMI default configuration	95
12	System services	96
12.1	Shared pins	96
12.2	Hard and soft reset	96
12.3	Bootstrap	97
12.4	Support for debugging	98
13	Diagnostic controller	100
13.1	Diagnostic hardware	100
13.2	Access features	101
13.3	Software debugging features	101
13.4	Peeking and poking the host from the target	104
14	Test access port	105
15	Serial link interface (OS-Link)	106
15.1	OS-Link protocol	106
15.2	OS-Link speed	106
15.3	OS-Link connections	106
16	Data flow	108
16.1	On-chip modules	108
16.2	Video data flow	109
16.3	Audio data flow	110
17	Link layer interface (LLI)	112
17.1	Introduction	112
17.2	Data streams	113
17.3	Stream multiplexing	113
17.4	Serial and parallel transport streams	115

17.5	Sync byte detector	116
18	Programmable transport interface (PTI)	118
18.1	Introduction	118
18.2	PTI functions	119
18.3	Operation	119
18.4	External interfaces	124
18.5	PTI timer module	124
18.6	Address map	125
18.7	DMA	125
18.8	Input interface	126
18.9	Section filter	126
18.10	Shared memory	128
19	MPEG video decoder	129
19.1	Decoder operation	129
19.2	Resets	130
19.3	Bit buffer and start code detection (video)	130
19.4	Video decoding pipeline control	131
19.5	Quantization table loading	133
19.6	Memory mapping of data	134
19.7	Using picture pointers	138
19.8	The video pipeline	138
19.9	PES parser overview	141
20	Sub-picture decoder	144
20.1	Introduction	144
20.2	Buffer management and pointers	145
20.3	Subpicture decoder operation	145
20.4	Sub-picture display	147
21	Display planes	149
21.1	Overview	149
21.2	Background color	151
21.3	Still picture plane	151
21.4	MPEG video plane	153
21.5	On-screen display (OSD)	167
21.6	Sub-picture or cursor plane	182
21.7	Mixing display planes	182
22	2D block move	185
22.1	Copying blocks of data	185

23	Teletext interface	187
23.1	Teletext interface pins	187
23.2	Teletext data output	187
23.3	Teletext data input	189
23.4	Teletext interrupt control	189
24	Digital encoder	190
24.1	Introduction	190
24.2	Video timing	190
24.3	Reset procedure	195
24.4	Slave modes	196
24.5	Input demultiplexor	202
24.6	Sub-carrier generation	202
24.7	Burst insertion (PAL and NTSC)	203
24.8	Subcarrier insertion (SECAM)	204
24.9	Luminance encoding	205
24.10	Chrominance encoding	207
24.11	Composite video signal generation	208
24.12	RGB and UV encoding	210
24.13	Closed captioning	211
24.14	CGMS encoding	212
24.15	WSS encoding	213
24.16	VPS encoding	213
24.17	Teletext encoding	213
24.18	Line skip and line insert capability	216
24.19	CVBS, S-VHS, RGB and UV outputs	216
24.20	DENC registers	217
25	Triple video DACs	219
25.1	Description	219
25.2	Input codes for video application	220
25.3	Video output voltage level	220
25.4	Video specifications and DAC setup	221
25.5	Output-stage adaptation and amplification	221
26	MPEG audio decoder with AC-3 interface	222
26.1	PCM output	222
26.2	Audio decoder control	224
26.3	AC-3 interface	229
27	Clocks and power-down mode	231
27.1	CPU and related clocks	232
27.2	Video and audio clocks	232
27.3	Real-time counter	234

27.4	Watchdog counter	234
27.5	Power-down mode	234
28	Block move DMA.....	236
28.1	Moving blocks of data	236
29	PWM and counter module.....	237
29.1	External interface.	237
29.2	PWM outputs	237
29.3	Capture inputs	238
29.4	Compare (programmable timer) facilities	238
29.5	Capture/compare counter, prescaling and clocking	238
30	Asynchronous serial controller.....	239
30.1	Control	239
30.2	Data frames	240
30.3	Transmission	241
30.4	Reception	242
30.5	Baud rate generation	244
30.6	Interrupt control	246
30.7	SmartCard operation	249
31	SmartCard interface.....	251
31.1	External interface	251
31.2	SmartCard clock generator	252
32	Synchronous serial controller.....	253
32.1	Introduction	253
32.2	Basic operation	254
32.3	I2C operation	262
33	Parallel input/output.....	269
33.1	PIO registers	269
34	IEEE 1284 port (PC parallel port).....	270
34.1	IEEE 1284 port pins	271
34.2	IEEE 1284 mode	272
34.3	Transport stream mode	274
35	Electrical specifications.....	278
35.1	Absolute maximum ratings	278
35.2	Operating conditions	278
35.3	DC specifications	279

35.4	DAC specifications	279
36	Timing specifications	280
36.1	SDRAM	280
36.2	PCM/AC-3 decoder interface	285
36.3	EMI timings	287
36.4	PIO timings	289
36.5	OS-Link timings	290
36.6	Reset and analyze timings	291
36.7	Clock timings	293
36.8	TAP timings	294
36.9	Transport stream timings	295
36.10	Teletext timings	297
36.11	IEEE 1284 timings	299
37	Revision history	300
37.1	7155560A to 7145560B	300
37.2	STi5512-STi5510	301

1 Introduction

The STi5512 is the latest in the OMEGA family of integrated multimedia decoder engines for DVB set top box systems. Compared to the STi5510, it has an enhanced External Memory Interface which supports SDRAM, without the need for glue logic. The device has increased performance with a 60MHz CPU clock, a digital YCrCb output, support for Genlock, and new display modes. The STi5512 can also be used as a replacement part for the STi5510 as it has the same architecture and is pin-compatible, however, some software modifications are required. STi5512 mode is selected by setting pin **YC0** low, and STi5510 mode is selected by setting pin **YC0** high.

The STi5512 is available with any combination of ICAM and Macrovision options, and different part numbers are used to identify the options as described in the section below. Throughout this manual, where the information is generic to all Macrovision and ICAM options, the device is referred to as the STi5512 only.

The [Revision history on page 300](#) describes the changes made to the STi5510 Data Sheet version 7112463A to create this STi5512 Data Sheet. The changes listed in the Revision history are hypertext-linked into the document, where they are also identified by change bars. Section 1.2 on page 9 lists the differences between the STi5512 and STi5510.

1.1 STi5512 options and order codes

The STi5512 can be supplied with ICAM and/or macrovision options. These options can be provided by STMicroelectronics depending on the market opportunity; please consult with your ST sales office for availability.

The part number is coded according to these options, the package type and the silicon revision, as follows

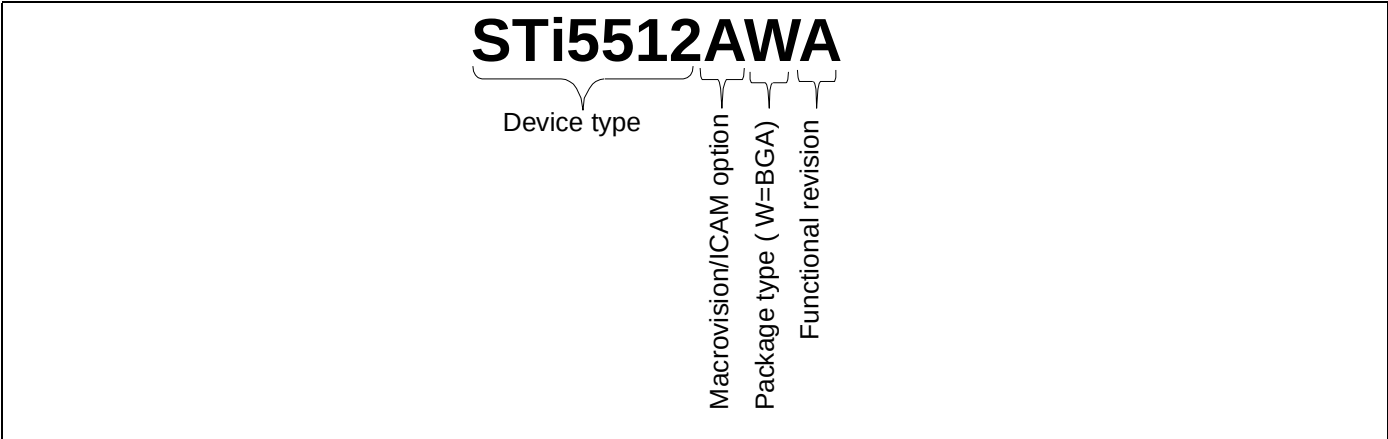


Figure 1 Part coding definition

The table below indicates the coding of the Macrovision/ICAM options.

	ICAM OFF	ICAM ON
Macrovision OFF	STi5512A	STi5512N
Macrovision ON	STi5512M	STi5512S

Table 1 STi5512 ICAM and Macrovision options

1.2 Differences between the STi5512 and the STi5510

- Increased performance:
 - 60 MHz CPU and system clock rate
 - 72 Mbits/s transport stream input processing rate. However, when processing more than 11 matching scrambled sections per packet for more than 2 consecutive packets the maximum data rate drops to 60 Mbps.
 - UART hardware RTS/CTS and ability to use FIFO's in SmartCard mode (T=0 protocol) with auto-repeat transmit of characters when NAKed from the SmartCard, and precise record of characters in error which have been NAKed automatically on receive.
- EMI supports SDRAM.
- Support for 16 and 64 Mbit JEDEC or PC100 SDRAMs on the EMI.
- LMC (local memory interface) supports two 16 or one 64 Mbit JEDEC or PC100 SDRAMs with x16 bit configuration. The ST20 system and the MPEG decoder and display are able to access ALL 64 Mbits of memory. The caching of data in this area allows any of the 8 Mbytes of memory to be cached.
- A Genlock system locks to an external analogue source and switch between STi5512 generated OSD and picture in RGB. In this way, a glueless interface to an external synchronization stripper is possible.
- Digital YCrCb Output in 4:2:2 format.
- Support for boot bank width population options on customers boards (16 or 32 bit) by optionally applying address shift based on bank width after reset.
- External micro support by tristate all of the EMI data, address and strobe signals on a external DMA request.
- DENC enhancements:
 - Support for placement of teletext, WSS and VPS on any line.
 - VBI insertion in square pixel mode.
 - Vertical pan/scan resolution of 2 lines on the video plane; horizontal pan/scan resolution of 1/8 pixel on the still plane.
 - Programmable TDL storage format.
- Two additional display modes for full resolution source video using frame-based filtering for luma and chroma
 - Luma/chroma downsampling for 16:9 Letterbox display.
 - Chroma upsampling for full resolution display.

2 Architecture overview

A block diagram of a DVB digital set top receiver is shown in Figure 2 below.

The STi5512 includes the following hardware modules:

- Transport demultiplexor
- System microcontroller
- MPEG video decoder
- MPEG audio decoder
- PAL/NTSC/SECAM encoder

The STi5512 directly interfaces to external memory and peripherals with no extra glue logic, keeping the system cost to a minimum. The STi5512 architectural block diagram is shown below.

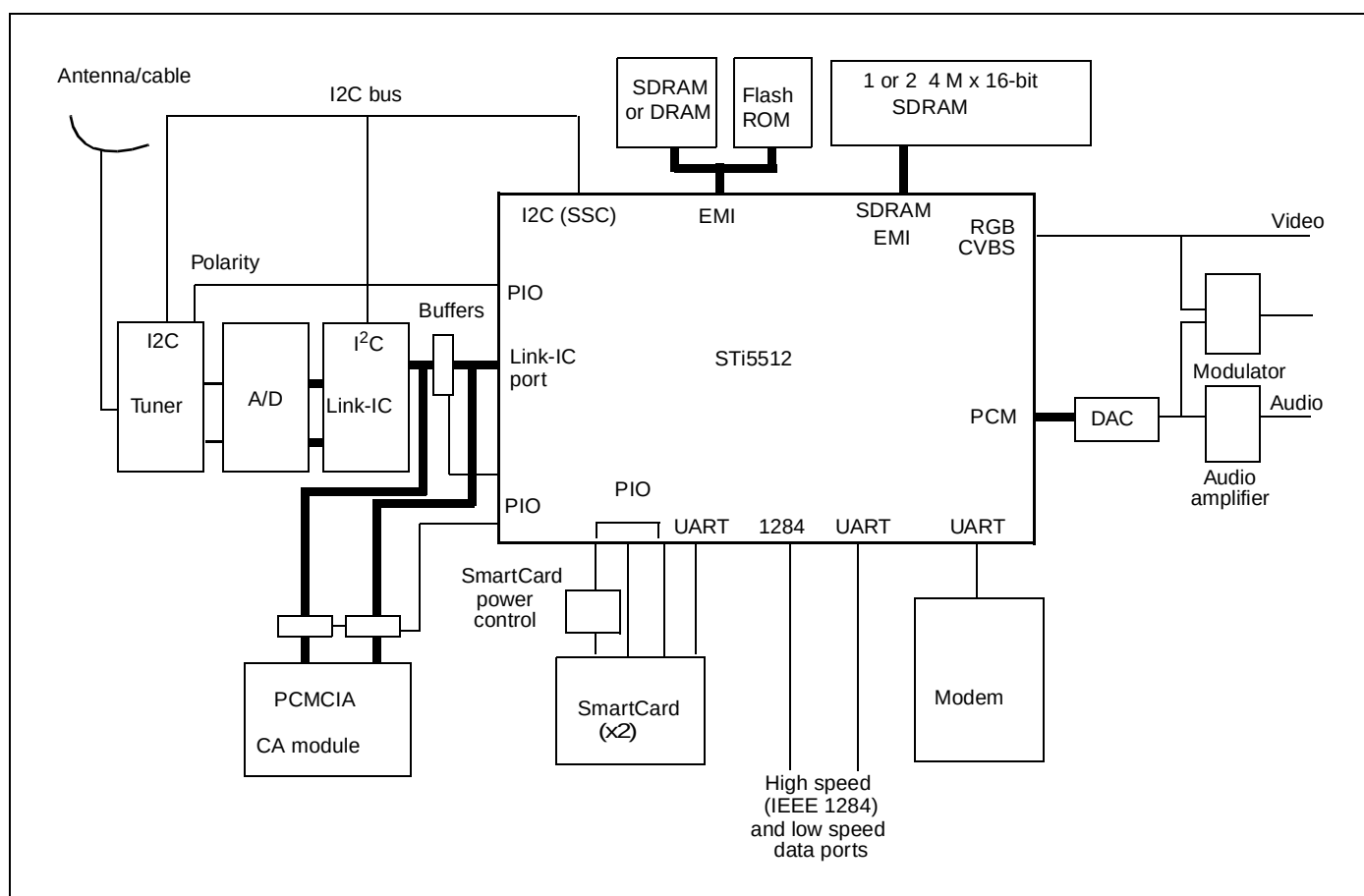


Figure 2 Digital set top box block diagram

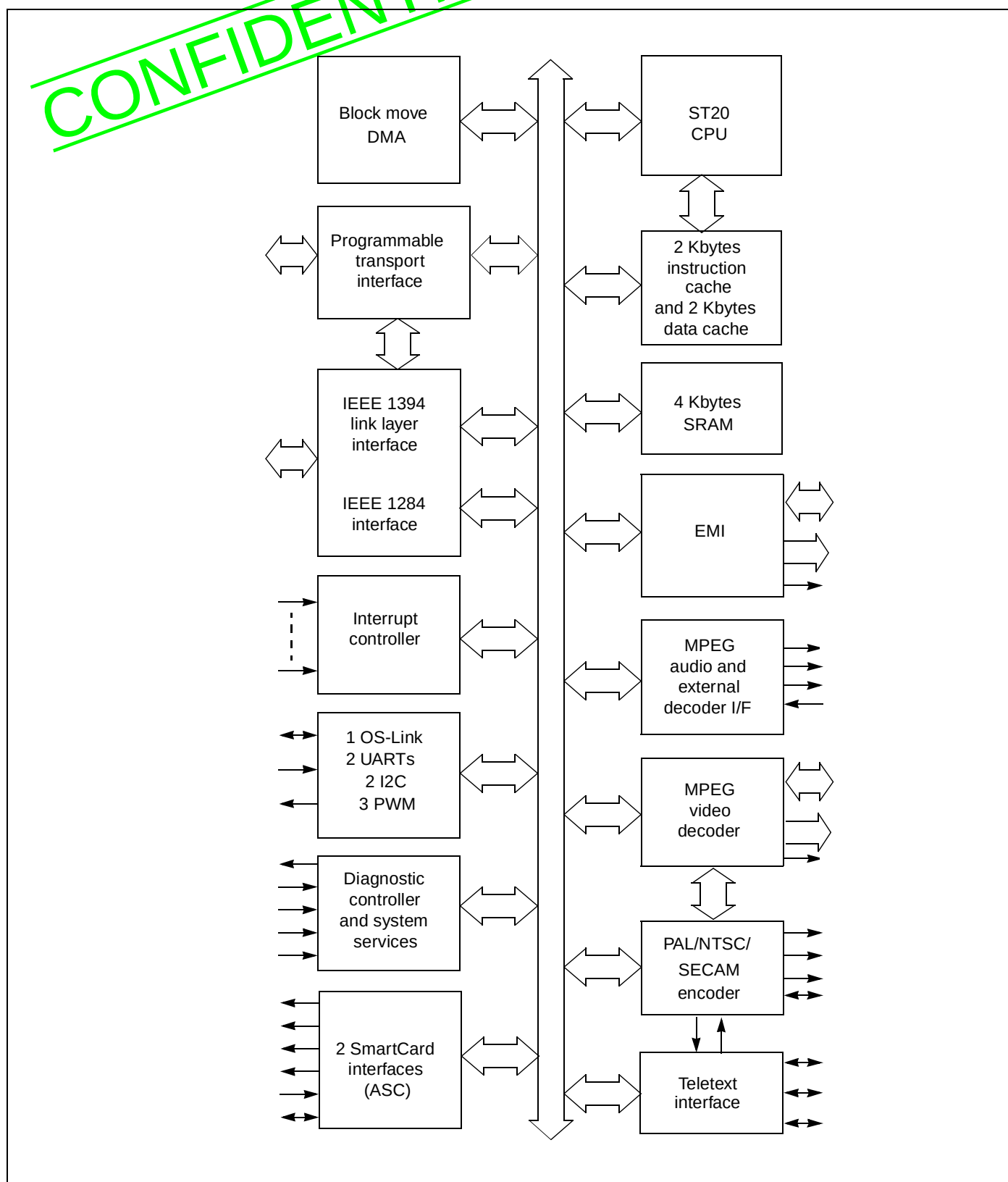


Figure 3 STi5512 architecture block diagram

2.1 STi5512 functional modules

[STi5512 architecture block diagram on page 11](#) shows the subsystem modules that comprise the STi5512. These modules are outlined below.

Processor

The Central Processing Unit (CPU) on the STi5512 is the ST20-C2 32-bit processor core running at the CPU clock rate. It contains instruction processing logic, instruction and data pointers and an operand register. It directly accesses the high speed on-chip caches and SRAM, which can store data or programs. The processor can also access memory via the External Memory Interface (EMI) and SDRAM interface.

MPEG-2 video and MPEG-1 audio decoder subsystems

This subsystem takes the MPEG compressed data streams and decompresses them, outputting digital YUV data in the case of the video decoder, and stereo PCM samples in the case of the audio decoder. The decoded video is fed to the display subsystem. An interface is provided to output an audio bit-stream for decoding by an external MPEG or AC-3 decoder to support multi-channel (surround) audio.

The video decoder implemented on the STi5512 uses a patented memory reduction/bandwidth reduction scheme to offer the user the best compromise between bandwidth and memory size. The algorithm is lossless and uses “on-the-fly” decoding to reduce the memory requirements to two frame buffers in memory size reduction mode. When used in bandwidth reduction mode, the memory usage is the normal three buffers, but the bandwidth required by the decoder is significantly reduced over a classical implementation.

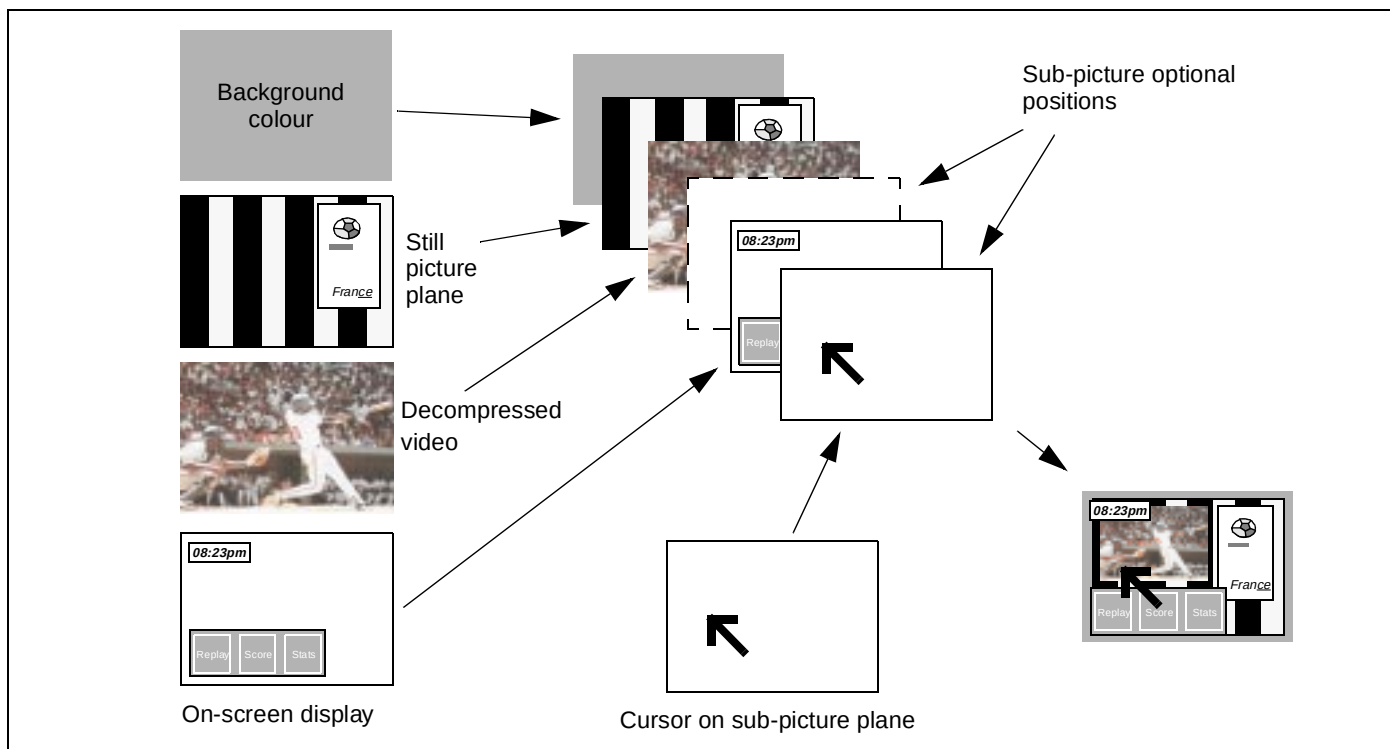


Figure 4 Display planes

The SDRAM interface includes all the signals necessary for control of the memory. Refresh is handled automatically by the decoder. The SDRAM interface supports two 16-Mbit or one 64-Mbit 100 MHz SDRAMs. The memory is used to hold the bit buffer, store decoded pictures and provide the display buffer. It also holds the user-defined on-screen display (OSD) bitmaps and can be used by the CPU for private storage of data. For the decoding of PAL MP@ML sequences, 12 Mbits of SDRAM are required.

Graphics and display subsystem

The display unit is part of the MPEG video decoder. It can overlay several display planes, as shown in [Figure 4 on page 12](#).

It takes the decompressed MPEG video data and performs the following functions:

- Optionally perform horizontal resampling of both luminance and chrominance data
- Reconstruct vertical data to create 4:2:2 sample format
- Generate on-screen display bit map for superposition onto picture output
- Allow sub-picture decoder output to be mixed onto picture output
- Optionally perform anti-flicker and anti-flutter filtering
- Optionally perform vertical resampling of both luminance and chrominance data
- Still picture plane with graphics capability
- 2D block move to copy rectangular sections of the display
- 4:4:4 chroma resolution for RGB output.
- 4:2:2 chroma resolution for CVBS output
- Separate control on OSD between CVBS and YC on one hand and RGB and YUV on the other

The sub-picture decoder can also be used as a hardware cursor unit. The priority of the sub-picture is first raised by programming a register so it is above all the other display planes. A cursor can be defined using an optionally compressed (run-length encoded) bitmap stored in external SDRAM. The bitmap can be any size up to a full screen. Per-pixel alpha-blending factors can be defined for each cursor to provide anti-aliasing with the background. The cursor is then moved around using register writes into X and Y coordinate registers.

The digital video data is fed to the PAL/NTSC/SECAM encoder subsystem.

PAL/NTSC/SECAM encoder

Integrated into this subsystem is all the digital processing and the digital to analog convertors required to process the digital video output from the MPEG video decoder and produce RGB, YUV, YC and CVBS analog outputs. The output of the teletext interface is filtered and re-inserted into the blanking interval in this subsystem.

Programmable Transport Interface

The transport demultiplexing function is performed in a programmable hardware module, the programmable transport interface (PTI). Its operation is as described below.

The input interface may select between either a LinkIC stream or an IEEE 1394 controller as the source for the transport stream. Data packets from the input interface are input into a FIFO while the PID is checked to see if it is currently selected for processing or is to be discarded. A selected packet is parsed by the module to determine its type and to extract data from it. If the packet is encrypted using the DVB Standard the correct key is written into the DVB decryption core in the transport module and the packet is decrypted.

After parsing and descrambling the packet, the data is either transferred to buffers in external memory or directly to the MPEG audio and video decoders. If the audio and video data is buffered then the data can be transferred by DMA from the buffer to the MPEG decoders.

DVB standard sections are filtered by a set from 32 possible 8-byte filters to look for a match. Matching sections are then transferred to memory buffers for processing by software.

Error conditions, system time clock recovery, and control of the hardware module are handled by software running on the ST20.

Memory subsystem

The STi5512 on-chip memory system provides 200 Mbytes/s internal data bandwidth, supporting pipelined 2-cycle internal memory access at 20 ns cycle times at 60 MHz. The STi5512 memory subsystem consists of instruction and data caches, SRAM and an external memory interface (EMI).

The STi5512 product has 4 Kbytes of on-chip SRAM. The advantage of this is the ability to store on-chip any time-critical code, such as interrupt routines, software kernels or device drivers, and even frequently used data.

The instruction and data caches are direct mapped with a write-back system for the data cache. The caches support burst accesses to the external memories for refill and write-back which are effective for increasing performance with page-mode DRAM memories. The data cache may also be configured as an additional 2 Kbytes of internal SRAM.

The STi5512 EMI controls access to the external memory and peripherals including the DMA data ports. It can access a 16 Mbyte physical address space in each of the three memory banks, or greater if DRAM is used. It provides sustained transfer rates of up to 80 Mbytes/s for SRAM, and up to 40 Mbytes/s using page-mode DRAM.

The 32-bit programmable EMI supports ROM, SRAM and DRAM/SDRAM for the ST20. SDRAM is supported at the CPU clock rate. The STi5512 supports boot bank width ROM/Flash population options using the address shift mechanism.

The SDRAM interface supports the use of two 16 Mbits or one 64 Mbits of external 100 MHz SDRAM. This memory is used to store the display data generated by the MPEG decoder and the CPU and read by the display unit.

Interrupt subsystem

The STi5512 interrupt subsystem supports eight prioritized interrupt levels. Four external interrupt pins are provided. Level assignment logic allows any of the internal or external interrupts to be assigned to any interrupt level. Interrupt level sharing is supported for level-sensitive interrupts.

Serial communications

To facilitate the connection of this system to a modem for a pay-per-view type system and other peripherals, two UARTs (ASC2s) are included in the device. The UARTs provide an asynchronous serial interface and can be programmed to support a range of baud rates and data formats, for example, data size, stop bits and parity. The UARTs are buffered with 16 byte FIFOs for transmit and receive data.

Two synchronous serial communications (SSC2) interfaces are provided on the device. These can be used to control, via an I²C or SPI bus, the tuner, Link-IC, E²PROM (if used) and the remote control devices in the application.

Block move engine

High performance block data transfer can be performed as a memory to memory DMA operation using the block move module.

IEEE 1284 interface

An 8-bit wide parallel interface (conforming to the IEEE 1284 standard) supports a high speed data input/output port to and from the set top receiver. The interface has a dedicated DMA controller to transfer data between memory and the port with little CPU overhead.

The STi5512 has a new mode called 1284_master_mode. In this mode, five pairs of 1284 interface pins swap functionality and direction. The control signal for this mode comes from bit 15 of the EMIconfigpadlogic signal. When low, the 1284 is in slave mode, when high, it is in master mode.

The five pairs of 1284 interface pins are

1284notSelectIn	1284notFault
1284notInIt	1284Select
1284notAutoFd	1284PError
1284HostLogicH	1284Busy
1284notStrobe	1284notAck

IEEE 1394 Link Layer Interface

The Link Layer Interface (LLI) facilitates the transfer of MPEG (or other) data from PID streams selected in the PTI to an external IEEE 1394 A/V Link layer controller. The LLI is bidirectional and so may also receive MPEG and other data which can be fed back into the PTI for demultiplexing or descrambling or both. The external link layer controller must support the IEC 61883 A/V digital interface specification and all its time-stamping requirements.

SmartCard interfaces

The SmartCard interfaces support SmartCards that are compliant with ISO7816-3 and use asynchronous protocol. Each interface is implemented with a UART (ASC), dedicated programmable clock generator, and eight bits of parallel IO port.

PWM and counter module

This module includes three separate pulse width modulator (PWM) generators using a shared counter, plus four timer-compare and three capture channels sharing a second counter. The counters can be clocked from a pre-scaled clock, using the 27 MHz **ClockIn** for the PWM counter and the system clock for the capture/compare counter. The event on which the timer value is captured is also programmable. The PWM counter is 8-bit with 8-bit registers to set the output high time. The capture/compare counter and the compare and capture registers are 32-bit.

Parallel IO module

Forty bits of parallel IO are provided. Each bit is programmable as an output or an input. The output can be configured as a totem pole or open drain driver. Input compare logic is provided which can generate an interrupt on any change of any input bit.

Many pins of the STi5512 device are multi-functional, and can either be configured as PIO or connected to an internal peripheral signal such as a UART or SSC.

Teletext

The teletext connects to the internal digital encoder using a request and data protocol. It translates teletext data to and from memory. It has two modes of operation; teletext data in and teletext data out.

In teletext data out mode, the teletext interface uses DMA to retrieve teletext data from memory, and serializes the data for transmission to the composite video encoder.

In teletext data in mode, teletext data is extracted from the composite video signal and is fed into the teletext interface as a serial stream. The teletext interface assembles the data and uses DMA to pass this data to memory.

Low power controller module

The low power controller module provides low power modes and a watchdog timer.

Diagnostic controller

The Diagnostic Controller Unit (DCU) is accessed via the JTAG test access port. It is the main access for communication with a host for development, including loading code and debugging. It provides:

- Bootstrapping and debugging during development
- Hardware breakpoint and watchpoint facilities
- Real-time tracing
- External LSA triggering support

OS-Link serial port

The OS-Link serial port can provide communication with a host for development, including loading code and debugging. This is provided for compatibility with previous silicon and software products.

System services module

The STi5512 system services module includes:

- A Phase Locked Loop (PLL) which accepts 27 MHz input and generates all the internal high frequency clocks needed for the CPU and the OS-Link
- A second PLL which is used to generate all the MPEG decoder and display clocks
- A Test Access Port which is JTAG compatible
- Reset controller

3 Summary specification

Video decoder	
Bit streams accepted	MPEG-1 video (ISO/IEC 11172-2), MPEG-2 video (ISO/IEC 13818-2) MPEG-2 Packetized elementary stream (PES) format as defined by ISO/IEC 13818-1 MPEG-1 ISO/IEC 11172-1 packets
MPEG-2 Profiles /Levels Supported	Main Profile @ Main Level (MP@ML), Main Profile @ Low Level (MP @ LL) Simple Profile @ Main Level (SP @ ML)
Maximum Picture Size	Width: 4080, Number of macroblocks: 16,383
Motion Vector Range	MPEG-1: -1024 to 1023 (full pel), -512 to 511.5 (half pel) horizontal and vertical MPEG-2: -1024 to 1023.5 horizontal and vertical
Compressed Data Input	8-bit asynchronous data port Peak input rate: 28.5Mbyte/s (228Mbit/s) Maximum sustained input rate (with 55MHz primary clock):100Mbit/s
SDRAM Interface	External SDRAM used for storage of picture buffers, bit buffer and on-screen display definitions 16-bit data bus, 1 or 2 banks, Refresh handled by decoder Configurations supported: SDRAM: 1M x 16, (1 bank) and SDRAM: 2M x 16, (2 banks)
Start Code Detection	Automatic detection of start codes of picture layer and above to enable microcontroller to access header data. Counters provided for time-stamp tracking.
Decoding pipeline	Instruction register set up each picture defines pipeline operation. Double-buffered quantization matrices enable loading of new tables concurrently with decoding.
Error concealment	Automatic concealment of errors detected by VLD and decoding pipeline by macroblock copy.
Display	
Video clock	27MHz nominal
Video output	External pel clock Horizontal/vertical synchronization provided by internal DENC or external source Interlaced output. 3:2 pull-down operation supported On chip up/down-sampling with anti-aliasing filter Vertical chroma reconstruction or luma filtering up to 4-tap filter
Pan and scan vectors	Horizontal: Maximum vector size: 2047 pels, resolution: 1/8 pel Vertical: Maximum vector size: 1022 lines, resolution: 16 lines
On-screen display (OSD)	Bitmap separately definable for each field can be superimposed on final picture output. OSD defined as rectangular regions, each with unique palette defining 4, 16 or 256 colors (including transparency). Each region has a blending factor, which can be selectively applied to each color in the palette. Number of regions limited by memory space allocated to OSD. Regions definitions can be organized as a linked list. Block move facility available for reduction of microcontroller loading.

Table 2 Summary specification

Audio decoder	
Bit streams accepted	MPEG-1 audio ISO/IEC 11172-3 audio elementary stream MPEG-2 packetized elementary stream (PES) format as defined by ISO/IEC 13818-1 MPEG-1 ISO/IEC 11172-1 packets Audio PCM data (for decoder bypass)
Performance	ISO/IEC 11172-3 Layers I and II All MPEG input bit rates supported with sampling rates of 32kHz, 44.1kHz and 48kHz, free format at 32kHz and 48kHz sampling rates Decodes in single channel, dual channel, stereo, or joint stereo modes
Audio clock	27MHz nominal
Compressed data input	Byte-mode input. Burst rate up to 28.5Mbyte/s
PCM output	16 or 18-bit PCM output, I2S and other popular formats supported
Error concealment	Automatic error concealment on CRC or synchronization error detection
General	
Support for A/V sync	PTS/DTS extraction from MPEG packet layers with automatic association

Table 2 Summary specification

4 Pin list

Signal names are prefixed by **not** if they are active low; otherwise they are active high.

Pin	Number	Function
VDD	13	Power supply
GND	16	Ground
VClamp1-3 ¹	3	Power supply for clamp diodes
VDDA0-1	2	Analog power supply for PAL/NTSC/SECAM encoder
VSSA0-1	2	Analog ground for PAL/NTSC/SECAM encoder
RTCVDD	1	Real time clock supply
VDD_VPLL	1	Analog power supply for video PLL
VSS_VPLL	1	Analog ground for video PLL

Table 3 Power supply pins

1. The VClamp pins are a power supply bus used to diode clamp the voltage on 5V tolerant digital input or output pins to. The voltage on the digital signal pin is then clamped to within $V_{I\max}$ (5.5V) if the applied voltage is increased above 5V.

If the device is to be interfaced to 3.3V logic signals only, then the VClamp pins can be connected to the STi5512 3.3V VDD power supply. However if any pin is to be interfaced to a 5V logic signal, then the VClamp pins must be connected to the 5V power supply (the 5V logic device power supply). Note in this case the 5V power supply must be capable of sinking the clamp current of transient signals above 5V.

In the latter case it is important to ensure the correct power supply ramp sequence. The VClamp power supply must be applied before or at the same time as the VDD 3.3V power supply. This is to ensure that during power supply power up and power down, $V_{Clamp} > VDD + 0.5V$.

Pin	In/Out	Function
R_OUT ¹	out	Red output
G_OUT ¹	out	Green output
B_OUT ²	out	Blue output
C_OUT ¹	out	Chroma output
CV_OUT ¹	out	Composite video output
Y_OUT ¹	out	Luma output
I_REF_DAC_RGB ¹	in	DAC current reference
I_REF_DAC_YCC ¹	in	DAC current reference
V_REF_DAC_RGB ¹	in	DAC voltage reference
V_REF_DAC_YCC ¹	in	DAC voltage reference
OSD_ENABLE	in/out	OSD enable
notHSYNC	in/out	Horizontal sync
ODD_OR_EVEN	in/out	Vertical sync
YC0-7 ²	output	Digital YUV output
CFC	input	DENC color burst phase and frequency control. This pin can be used in non-scart based Genlock applications, see Figure 130 on page 203 and associated text. If it is not used, this pin must be grounded.

Table 4 Video output interface pins

1. The digital encoder video outputs are analogue signals and are not 5V tolerant. The same applies to the video DAC voltage and current reference pins.
2. The **YC0** pin is tri-stated during reset and then sampled at the end of the reset to determine whether the EMI pins are in STi5510 or STi5512 mode. If the **YC0** pin is sampled high (i.e at VDD) then the STi5510 mode is selected for the EMI pins and a low value selects STi5512 mode. For details see [External memory interface on page 63](#). In STi5512 mode the address shift for bank 3 is dependent on the boot bank width and the strobe pins are tri-stated when the EMI bus is granted to an external DMA device. External 10K Ω pull-up or pull-down resistors should be fitted to the **YC0** according to the functionality desired. If this pin is left *not* connected, the pin will be pulled high by an internal pull-up and will default the EMI pins to STi5510 mode.

Pin	In/Out	Function
SCLK/A_C_STB	out	Serial clock or AC-3 data strobe
PCM_DATA/A_C_DATA	out	PCM data out or AC-3 data out
PCMCLK	in/out	PCM clock
LRCLK/A-WORD_CLK	out	Left/right clock or AC-3 word clock
notA_C_REQ	in	AC-3 data request
notA_PTS_STB	in	AC-3 audio PTS strobe

Table 5 AC-3/MPEG1 audio output interface pins

Pin	In/Out	Function
Interrupt0-1	in	Interrupt

Table 6 External interrupt pins

Pin	In/Out	Function
ClockIn	in	System input clock - PLL or TimesOneMode
SpeedSelect0-1	in	PLL speed selector
notRST	in	System reset
CPUAnalyse / TrigIn	in	Error analysis / External trigger input to DCU
CPUReset	in	Soft reset for analyzing from OS-Link
ErrorOut / TrigOut ¹	in/out, out	Error indicator / Signal to trigger external debug circuitry (e.g. LSA)

Table 7 System services pins

1. This pin is tri-stated during reset and then sampled at the end of the reset to determine whether the OS-Link is active and to determine the function of the shared **CPUAnalyse / TrigIn** and the **ErrorOut / TrigOut**, as described in the *System Services* chapter. If the **ErrorOut** pin is sampled high (i.e at VDD) then the DCU signals (**TrigIn** and **TrigOut**) are selected and a low value indicates OS-Link signals (i.e. **CPUAnalyse**, **ErrorOut**) are to be used. External 10K Ω pull-up or pull-down resistors should be fitted to the **ErrorOut** according to the functionality desired.

Pin	In/Out	Function
MemAddr2-23	out	Address bus
MemData0-31	in/out	Data bus. MemData0 is the least significant bit (LSB) and MemData31 is the most significant bit (MSB).
MemRdnotWr	out	ReadnotWrite strobe
MemReq	in	Direct memory access request
MemGrant	out	Direct memory access granted
MemWait	in	Memory cycle extender
notMemCAS0,2	out	CAS strobes for SDRAM/DRAM in Banks 0 and 1
notMemCAS1	out	CAS strobe for DRAM or SDRAM clock
notMemCAS3	out	CAS strobe for DRAM or sub-bank chip select for bank 3
notMemRAS0	out	RAS strobe for SDRAM/DRAM in Bank 0, chip select for Bank0 or RAS strobe for lowest DRAM sub-bank in Bank0
notMemRAS1	out	RAS strobe for highest DRAM sub-bank in Bank0 or SDRAM Chip select signal for highest sub-bank of Bank0
notMemRAS2	out	RAS strobe for SDRAM/DRAM in bank 1, chip select for Bank1 or RAS strobe for lowest DRAM sub-bank in Bank1
notMemRAS3	out	RAS strobe for highest DRAM sub-bank in Bank1 or SDRAM Chip select signal for Bank1
notMemCSROM	out	Chip select strobe for ROM in bank3.
notSDRAMCS0	out	SDRAM Chip select signal for Bank0 or lowest sub-bank of Bank0
notMemOE	out	Output enable strobe - banks 0-3.
notMemBE0-3	out	Byte enable strobes - banks 0-3.
notMemCS2	out	Chip select strobe for memory in bank 2.
BootSource0-1	in	Boot from ROM or from link.
ProcClockOut	out	Processor clock.

Table 8 STi5512 External memory interface pins

Pin	In/Out	Function
AD0-12	out	SDRAM address bus
DQ0-15	in/out	SDRAM data bus (lower byte)
notSDCS0	out	SDRAM chip select for first SDRAM
notSDCS1/AD13	out	SDRAM chip select for second SDRAM or AD13
notSDCAS	out	SDRAM CAS
notSDRAS	out	SDRAM RAS
notSDWE	out	SDRAM write enable
MEMCLKIN	in	SDRAM memory clock input
MEMCLKOUT	out	SDRAM memory clock output
DQML	out	DQ mask enable (lower)
DQMU	out	DQ mask enable (upper)

Table 9 Shared SDRAM interface pins

Pin	In/Out	Function
LPClockIn ¹	in	Low power input clock
LPClockOsc ¹	in/out	Low power clock oscillator
AUX_CLK_OUT	out	Auxiliary clock for general use

Table 10 Low power controller and real time clock pins

1. The low power clock pins are not 5V tolerant.

Pin	In/Out	Function
PIO0[0-7]	in/out	Parallel input/output pin or alternative function (see Table 17)
PIO1[0-7]	in/out	Parallel input/output pin or alternative function (see Table 17)
PIO2[0-7]	in/out	Parallel input/output pin or alternative function (see Table 17)
PIO3[0-7]	in/out	Parallel input/output pin or alternative function (see Table 17)
PIO4[0-7]	in/out	Parallel input/output pin or alternative function (see Table 17)

Table 11 PIO pins

Pin	In/Out	Function
LinkIn	in	Serial data input channel
LinkOut	out	Serial data output channel

Table 12 OS-Link pins

Pin	In/Out	Function
TSInByteClk	in	Link IC byte clock
TSInByteClkValid	in	Link IC byte clock valid edge
TSInData0-7	in	Link IC data
TSInError	in	Link IC packet error
TSInPacketClk	in	Link IC packet strobe

Table 13 Transport stream input pins

Pin	In/Out	Function
TtxtEvennotOdd	in	Teletext even not odd vertical sync signal
TtxtHsync	in	The HSYNC signal input when the teletext interface is operating in the input mode

Table 14 Teletext interface

The teletext clock and data inputs are shared PIO pins, as shown in [Table 17](#).

High speed data port pins have a dual function, and can be used either to interface to an external IEEE 1394 link layer controller or provide an IEEE 1284 parallel port interface.

Pin	In/Out	Function
1284Data0-7 / AVData0-7	in/out	IEEE 1284 port data or AV data
1284notSelectIn	in	IEEE 1284 port control signals or AV signals
1284notInit / AVPacketTag3	in	
1284notFault / AVPacketTag2	out	
1284notAutoFd / AVPacketTag1	in	
1284Select / AVPacketTag0	out	
1284PErrror / AVByteClkValid	out, in/out	
1284Busy / AVPacketClk	out, in/out	
1284notAck / AVByteClk	out	
1284notStrobe/AVPacketError	in	

Table 15 High-speed data port pins

Pin	In/Out	Function
TDI	in	Test data input
TDO	out	Test data output
TMS	in	Test mode select
TCK	in	Test clock
notTRST	in	Test logic reset

Table 16 TAP pins

4.1 PIO pins and alternative functions

To improve flexibility and to allow the STi5512 to fit into different set-top box application architectures, the input and output signals from some of the peripherals are not directly connected to the pins of the device. Instead they are assigned to the alternative function inputs and outputs of a PIO port bit. This scheme allows these pins to be configured as general purpose PIO if the associated peripheral input or output is not required in that particular application.

Peripheral inputs connected to the alternative function input of a PIO bit are permanently connected to the input pin. The output signal from a peripheral is only connected when the PIO bit is configured into either push-pull or open drain driver alternative function mode.

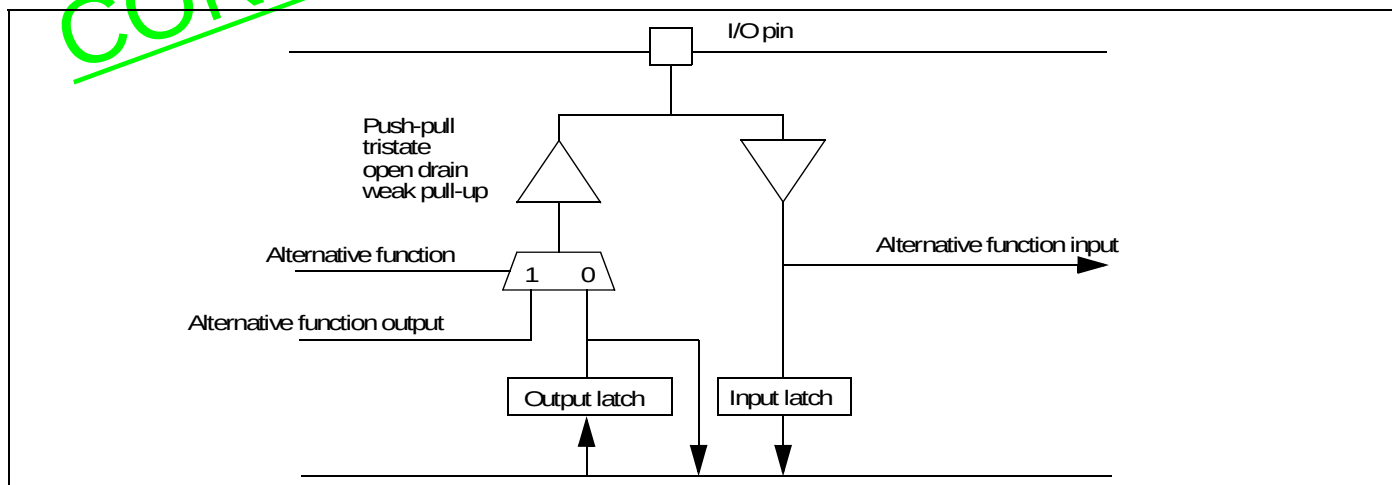


Figure 5 I/O port pin

Table 17 shows the assignment of the alternative functions to the PIO bits. Parentheses () in the table indicate suggested or possible pin usages as a PIO, not an alternative function connection.

Port bit	Alternative function of PIO pins				
	PIO port 0	PIO port 1	PIO port 2	PIO port 3	PIO port 4
0	ASC0TxD or Sc1DataOut	SSC0 MTSR	ASC2TxD or Sc0DataOut	SSC1 MTSR	ASC3TxD
1	ASC0TRxD or Sc1DataIn	SSC0 MRST	ASC2RxD or Sc0DataIn	SSC1 MRST	ASC3RxD
2	Sc1ClkGenExtClk	SSC0 SClk	Sc0ClkGenExtClk	SSC1 SClk	TtxtClockIn
3	Sc1Clk	PWMOut0	Sc0Clk	CaptureIn0	1284PeriphLogicH/ASC3 CTS
4	(Sc1RST)	PWMOut1	(Sc0RST)	CaptureIn1	1284HostLogicH/ASC3 RTS
5	(Sc1CmdVcc)	ASC1TxD	(Sc0CmdVcc)	CaptureIn2	Interrupt2
6	(Sc1CmdVpp) Sc1Dir	ASC1RxD	(Sc0CmdVpp) Sc2Dir	CompareOut2	Interrupt3
7	(Sc1Detect)	PWMOut2	(Sc0Detect)	1284InnotOut	TtxtData

Table 17 Alternative function of PIO pins

5 Package specifications

The STi5512 is packaged in a 256-pin ball grid array (BGA) with 16 additional central ground balls.

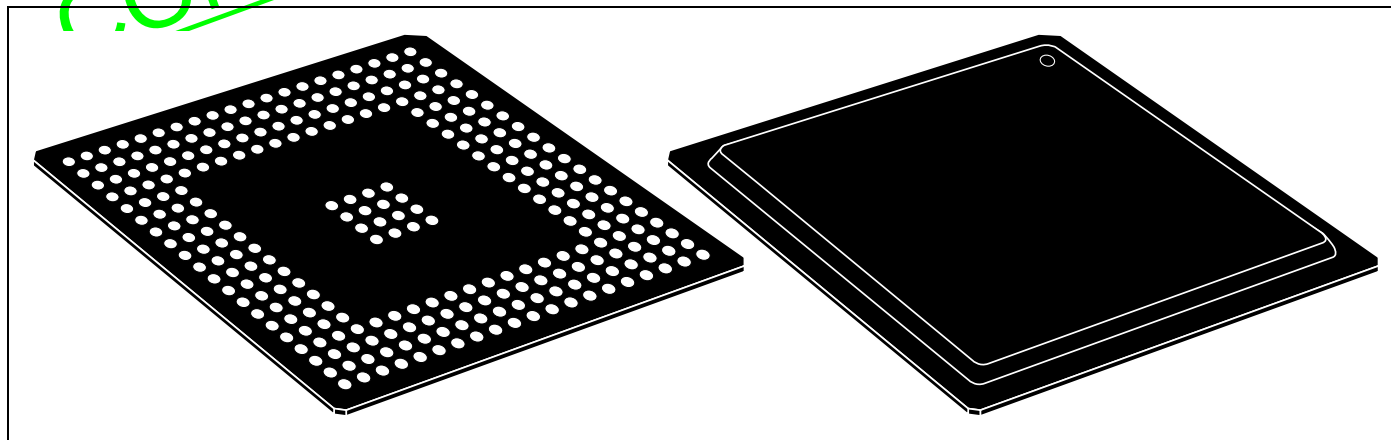


Figure 6 256-pin BGA package

5.1 Thermal guidelines

Guidelines for PGA board design with optimum thermal management are given in a technical note entitled "PBGA Thermal Management Guidelines for Board Design" number DVD-Doc-081.

5.2 Pin-out

The following pages give the allocation of pins to the package, shown from the top looking down.

Note NC indicates that the pin is reserved; do not connect these pins.

	1	2	3	4	5	6	7	8	9	10
A	PIO4[3]/ 1284Periph LogicH/ ASC3 CTS	PIO4[6]/ Interrupt3	Interrupt1	PIO4[1]/ ASC3RxD	MemData 31	MemData 28	MemData 25	Vclamp3	MemData 19	MemData 15
B	PIO4[4]/ 1284Host LogicH/ ASC3 RTS	PIO4[5]/ Interrupt2	Interrupt0	PIO4[0]/ ASC3TxD	MemData 30	MemData 27	MemData 24	MemData 21	MemData 18	MemData 14
C	1284notInit/ AVPacket Tag3	1284not SelectIn	PIO3[7]/ 1284Innot Out	PIO1[6]/ ASC1 RxD	MemData 29	MemData 26	MemData 23	MemData 20	MemData 17	MemData 13
D	1284Select/ AVPacket Tag0	1284not AutoFd/AV PacketTag1	1284not Fault/AV PacketTag2	YC7	PIO1[5]/ ASC1 TxD	VDD	MemData 22	YC0	MemData 16	VDD
E	1284Data0/ AVData0	1284notAck/ AVByteClk	1284Busy/ AVPacket Clk	1284PErrors/ AVByteClk Valid						
F	1284Data1/ AVData1	1284Data2/ AVData2	1284Data3/ AVData3	VDD						
G	Vclamp1	1284Data4/ AVData4	1284Data5/ AVData5	1284Data6/ AVData6						
H	1284Data7/ AVData7	1284not Strobe/AV PacketError	TCK	TtxtEven notOdd						
J	notTRST	TMS	TDO	TDI						
K	LPClock In	LPClockOsc	RTCVDD	VDD						
L	notRST	PIO4[2]/ TtxtClockIn	PIO4[7]/ TtxtData	OSD_ ENABLE						
M	LRCLK/A- WORD_ CLK	notHSYNC	ODD_OR_E VEN	TtxtHsync						
N	PCM_DATA/ A_C_DATA	notA_C_REQ	PCMCLK	notA_PTS_ STB						
P	V_REF_DA C_YCC	I_REF_DAC_ YCC	SCLK/ A_C_STB	VDD						
R	VSSA1	CV_OUT	C_OUT	Y_OUT						
T	V_REF_DA C_RGB	I_REF_DAC_ RGB	VDDA1	CFC						
U	VSSA0	B_OUT	R_OUT	G_OUT	VDD	PIO0[6]/ Sc1Dir	YC6	PIO3[1]/ SSC1 MRST	VDD	PIO3[5]/ Capture In2
V	VDDA0	PIO2[0]/ ASC2TxD	PIO2[2]/ Sc0ClkGen ExtClk	PIO0[0]/ ASC0TxD	PIO0[3]/ Sc1Clk	PIO0[7]/ (Sc1 Detect)	PIO1[1]/ SSC0 MRST	PIO3[2]/ SSC1 SClk	PIO1[7]/ PWMOOut2	PIO3[6]/ Compare Out2
W	PIO2[1]/ ASC2RxD	PIO2[3]/ Sc0Clk	PIO2[5]/ Sc0Cmd Vcc	PIO0[1]/ ASC0RxD	PIO0[4]/ (Sc1RST)	Vclamp2	PIO1[2]/ SSC0 SClk	PIO1[3]/ PWMOOut0	PIO3[3]/ CaptureIn 0	ErrorOut/ TrigOut
Y	PIO2[4]/ (Sc0RST)	PIO2[6]/ Sc0Dir	PIO2[7]/ Sc0Detect	PIO0[2]/ Sc1ClkGenE xtClk	PIO0[5]/ (Sc1Cmd Vcc)	PIO1[0]/ SSC0 MTSR	PIO3[0]/ SSC1 MTSR	PIO1[4]/ PWMOOut1	PIO3[4]/ CaptureIn 1	CPU Analyse/ TrigIn

GND	GND
GND	GND
GND	GND
GND	GND

11	12	13	14	15	16	17	18	19	20									
MemData12	MemData8	MemData5	MemData1	MemAddr22	MemAddr18	MemAddr15	MemAddr11	MemAddr8	MemAddr7	A								
MemData11	MemData7	MemData4	MemData0	MemAddr21	MemAddr17	MemAddr14	MemAddr10	MemAddr9	MemAddr6	B								
MemData10	MemData6	MemData3	MemAddr23	MemAddr20	MemAddr16	MemAddr13	MemAddr3	MemAddr4	MemAddr5	C								
MemData9	YC1	MemData2	VDD	MemAddr19	YC2	MemAddr12	notMemCAS2	notMemCAS3	MemAddr2	D								
<table><tr><td>GND</td><td>GND</td></tr><tr><td>GND</td><td>GND</td></tr><tr><td>GND</td><td>GND</td></tr><tr><td>GND</td><td>GND</td></tr></table>						GND	GND	GND	GND	GND	GND	GND	GND	VDD	notMemRAS3	notMemCAS0	notMemCAS1	E
						GND	GND											
						GND	GND											
						GND	GND											
						GND	GND											
						notMemRAS2	notMemCSROM	notMemRAS0	notMemRAS1	F								
						notSDRAMCS0	notMemBE2	notMemBE3	MemRdnotWr	G								
						ProcClockout	notMemCS2	notMemBE0	notMemBE1	H								
						VDD	MemWait	BootSource0	BootSource1	J								
						SpeedSelect1	MemGrant	MemReq	notMemOE	K								
YC3	TSInError	TSInByteClk	SpeedSelect0	L														
TSInData6	TSInData7	TSInPacketClk	TSInByteClkValid	M														
VDD	TSInData3	TSInData4	TSInData5	N														
vss_vpll	TSInData0	TSInData1	TSInData2	P														
YC4	DQ15	aux_clk_out	vdd_vpll	R														
DQ11	DQ12	DQ13	DQ14	T														
YC5	ClockIn	VDD	AD6	AD12	MEMCLKOUT	VDD	DQ8	DQ9	DQ10	U								
CPUReset	AD0	AD3	AD7	AD10	notSDWE	notSDCS0	DQ0	DQ6	DQ7	V								
LinkOut	AD1	AD4	AD8	AD11	notSDCAS	notSDCS1/AD13	DQ1	DQ3	DQ5	W								
LinkIn	AD2	AD5	AD9	MEMCLKIN	notSDRAS	DQMU	DQML	DQ2	DQ4	Y								

5.3 Package dimensions

Table 18: Package dimensions on page 28 gives the values of the dimensions marked in Figure 7.

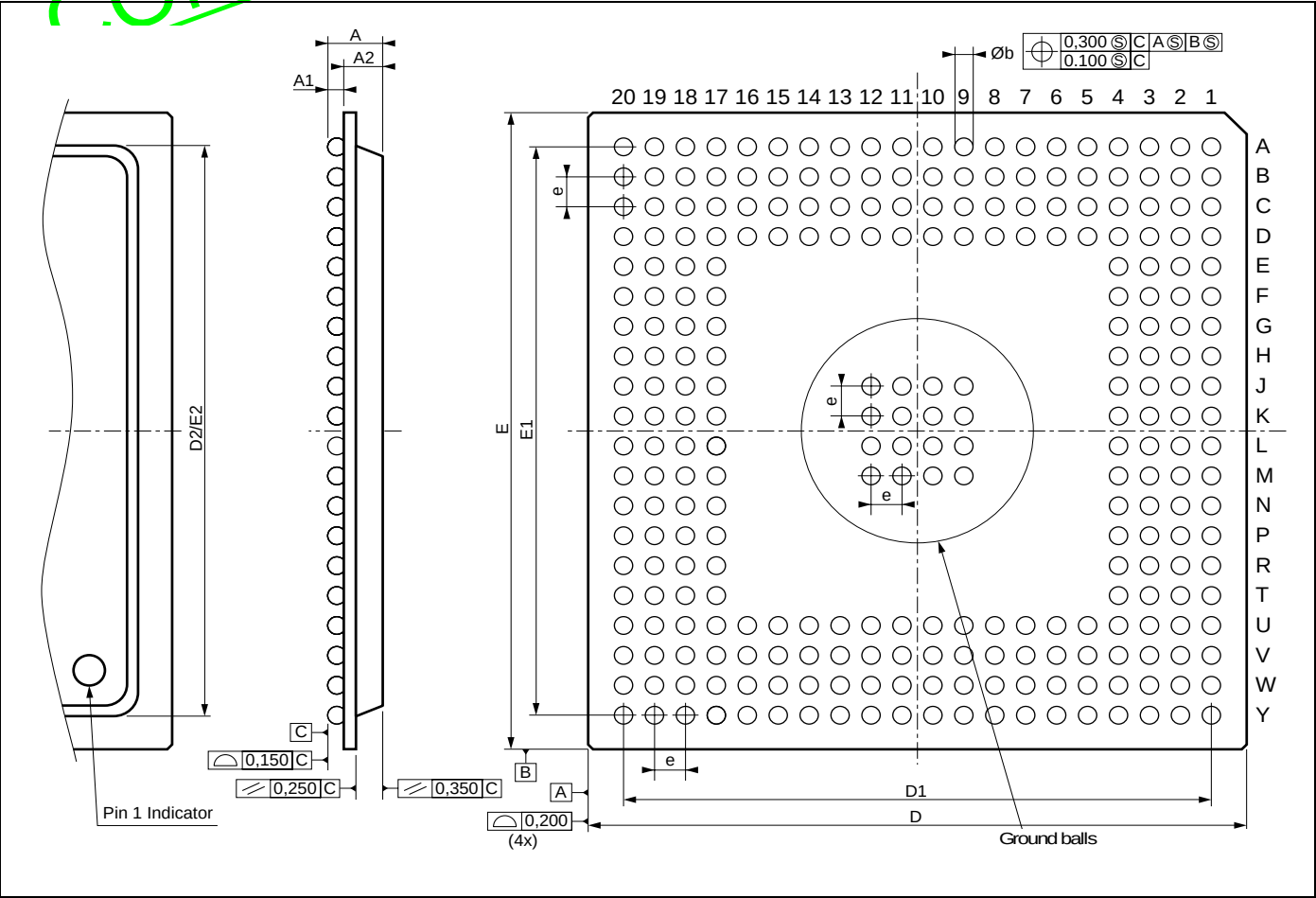


Figure 7 Package dimension

Dimension	Millimeters			Inches		
	Minimum	Typical	Maximum	Minimum	Typical	Maximum
A	2.125		2.595	0.083		0.102
A1	0.50		0.70	0.020		0.027
A2	1.625		1.895	0.064		0.074
b	0.60		0.90	0.024		0.035
D	26.82	27.00	27.18	1.055	1.063	1.070
D1		24.13 basic			0.951 basic	
D2	23.90		24.10	0.941		0.949
e		1.27 basic			0.050 basic	
E	26.82	27.00	27.18	1.055	1.063	1.070
E1		24.16 basic			0.951 basic	
E2	23.90		24.10	0.941		0.949

Table 18 Package dimensions



6 Central processing unit

The Central Processing Unit (CPU) is the ST20 32-bit processor core. It contains instruction processing logic, instruction and data pointers, and an operand register. It can directly access the high speed on-chip memory, which can store data or programs. Where larger amounts of memory are required, the processor can access memory via the External Memory Interface (EMI). The processor provides high performance:

- Fast integer multiply - 4 cycle multiply
- Fast bit shift - single cycle barrel shifter
- Byte and part-word handling
- Scheduling and interrupt support
- 64-bit integer arithmetic support

The scheduler provides a single level of pre-emption. In addition, multi-level pre-emption is provided by the interrupt subsystem. Additionally, there is a per-priority trap handler to improve the support for arithmetic errors and illegal instructions, refer to [Section 6.6: Traps and exceptions](#) on page 33.

6.1 Registers

The CPU contains six registers which are used in the execution of a sequential integer process. The six registers are:

- The workspace pointer (**Wptr**) which points to an area of store where local data is kept
- The instruction pointer (**Iptr**) which points to the next instruction to be executed
- The status register (**Status**)
- The **Areg**, **Breg** and **Creg** registers which form an evaluation stack

The **Areg**, **Breg** and **Creg** registers are the sources and destinations for most arithmetic and logical operations. Loading a value into the stack pushes **Breg** into **Creg**, and **Areg** into **Breg**, before loading **Areg**. Storing a value from **Areg**, pops **Breg** into **Areg** and **Creg** into **Breg**. **Creg** is left undefined.

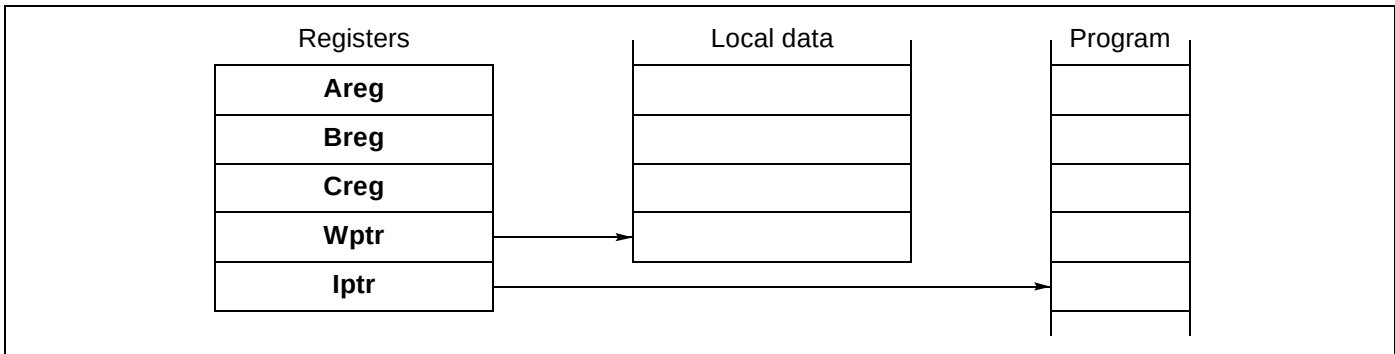


Figure 8 Registers used in sequential integer processes

Expressions are evaluated on the evaluation stack, and instructions refer to the stack implicitly. For example, the *add* instruction adds the top two values in the stack and places the result on the top of the stack. The use of a stack removes the need for instructions to explicitly specify the location of their operands. No hardware mechanism is provided to detect that more than three values have been loaded onto the stack; it is easy for the compiler to ensure that this never happens.

Note that a location in memory can be accessed relative to the workspace pointer, enabling the workspace to be of any size. The use of shadow registers provides fast, simple and clean context switching.

6.2 Processes and concurrency

This section describes the default behavior of the CPU and it should be noted that the user can alter this behavior, for example by disabling timeslicing or installing a user scheduler.

A process starts, performs a number of actions, and then either stops without completing or terminates complete. Typically, a process is a sequence of instructions. The CPU can run several processes in parallel (concurrently). Processes may be assigned either high or low priority, and there may be any number of each.

The processor has a microcoded scheduler which enables any number of concurrent processes to be executed together, sharing the processor time. This removes the need for a software kernel, although kernels can still be written if desired. At any time, a process may be:

- active
- being executed,
- interrupted by a higher priority process,
- on a list waiting to be executed.
- inactive
- waiting to input,
- waiting to output,
- waiting until a specified time.

The scheduler operates in such a way that inactive processes do not consume any processor time. Each active high priority process executes until it becomes inactive. The scheduler allocates a portion of the processor's time to each active low priority process in turn (see [Section 6.3: Priority](#) on page 31). Active processes waiting to be executed are held in two linked lists of process work spaces, one of high priority processes and one of low priority processes. Each list is implemented using two registers, one of which points to the first process in the list, the other to the last. In the linked process list shown in [Figure 9](#) below, process *S* is executing and *P*, *Q* and *R* are active, awaiting execution. Only the low priority process queue registers are shown; the high priority process ones behave in a similar manner.

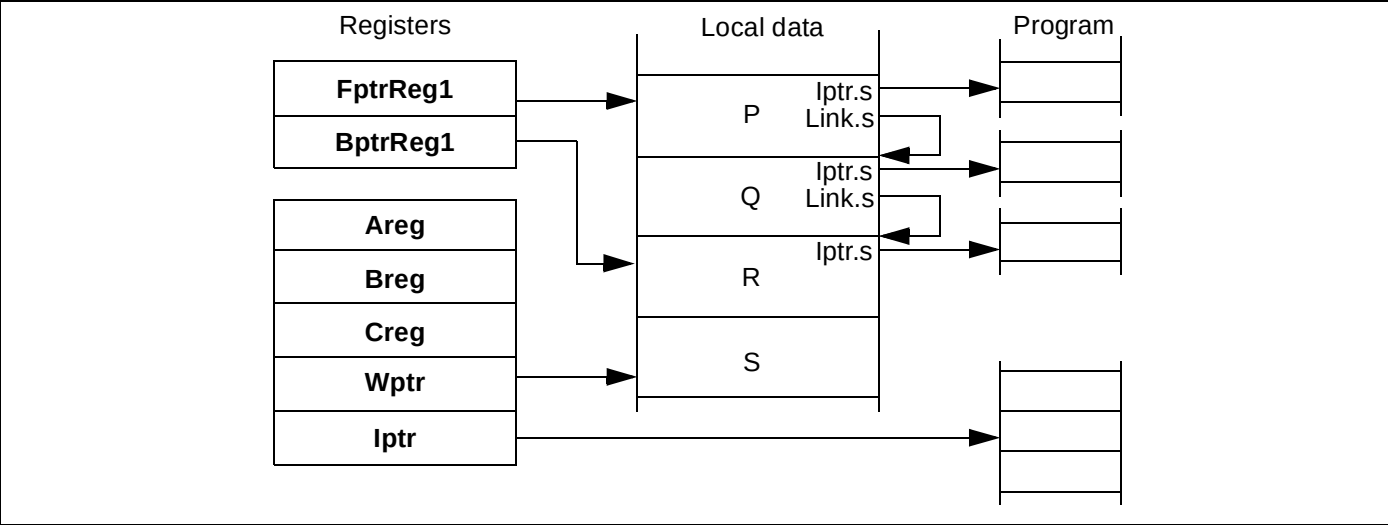


Figure 9 Linked process list

Function	High priority	Low priority
Pointer to front of active process list	FptrReg0	FptrReg1
Pointer to back of active process list	BptrReg0	BptrReg1

Table 19 Priority queue control registers

Each process runs until it has completed its action or is descheduled. In order for several processes to operate in parallel, a low priority process is only permitted to execute for a maximum of two timeslice periods. After this, the



machine deschedules the current process at the next timeslicing point, adds it to the end of the low priority scheduling list and instead executes the next active process. The timeslice period is 1ms.

There are only certain instructions at which a process may be descheduled. These are known as descheduling points. A process may only be timesliced at certain descheduling points. These are known as timeslicing points and are defined in such a way that the operand stack is always empty. This removes the need for saving the operand stack when timeslicing. As a result, an expression evaluation can be guaranteed to execute without the process being timesliced part way through.

Whenever a process is unable to proceed, its instruction pointer is saved in the process workspace and the next process taken from the list.

The processor core provides a number of special instructions to support the process model, including *startp* (start process) and *endp* (end process). When a main process executes a parallel construct, *startp* is used to create the necessary additional concurrent processes. A *startp* instruction creates a new process by adding a new workspace to the end of the scheduling list, enabling the new concurrent process to be executed together with the ones already being executed. When a process is made active it is always added to the end of the list, and thus cannot pre-empt processes already on the same list.

The correct termination of a parallel construct is assured by use of the *endp* instruction. This uses a data structure that includes a counter of the parallel construct components which have still to terminate. The counter is initialized to the number of components before the processes are started. Each component ends with an *endp* instruction which decrements and tests the counter. For all but the last component, the counter is non zero and the component is descheduled. For the last component, the counter is zero and the main process continues.

6.3 Priority

The following section describes 'default' behavior of the CPU and it should be noted that the user can alter this behavior, for example, by disabling timeslicing and priority interrupts.

The processor can execute processes at one of two priority levels, one level for urgent (high priority) processes, one for less urgent (low priority) processes. A high priority process will always execute in preference to a low priority process if both are able to do so.

High priority processes are expected to execute for a short time. If one or more high priority processes are active, then the first on the queue is selected and executes until it has to wait for a communication, a timer input, or until it completes processing.

If no process at high priority is active, but one or more processes at low priority are active, then one is selected. Low priority processes are periodically timesliced to provide an even distribution of processor time between tasks which use a lot of computation.

If there are n low priority processes, then the maximum latency from the time at which a low priority process becomes active to the time when it starts processing is the order of $2n$ timeslice periods. It is then able to execute for between one and two timeslice periods, less any time taken by high priority processes. This assumes that no process monopolizes the time of the CPU; i.e. it has frequent timeslicing points.

The specific condition for a high priority process to start execution is that the CPU is idle or running at low priority and the high priority queue is non-empty.

If a high priority process becomes able to run while a low priority process is executing, the low priority process is temporarily stopped and the high priority process is executed. The state of the low priority process is saved into 'shadow' registers and the high priority process is executed. When no further high priority processes are able to run, the state of the interrupted low priority process is re-loaded from the shadow registers and the interrupted low priority process continues executing. Instructions are provided on the processor core to allow a high priority process to store the shadow registers to memory and to load them from memory. Instructions are also provided to allow a process to exchange an alternative process queue for either priority process queue. These instructions allow extensions to be made to the scheduler for custom run-time kernels.

A low priority process may be interrupted after it has completed execution of any instruction. In addition, to minimize the time taken for an interrupting high priority process to start executing, the potentially time consuming instructions are interruptible. Also some instructions may be aborted, and are restarted when the process next becomes active (see [Instruction Set on page 38](#)).

6.4 Process communications

Communication between processes takes place over channels, and is implemented in hardware. Communication is point-to-point, synchronized and unbuffered. As a result, a channel needs no process queue, no message queue and no message buffer. A channel between two processes executing on the same CPU is implemented by a single word in memory; a channel between processes executing on different processors is implemented by point-to-point links. The processor provides a number of operations to support message passing, the most important being *in* (input message) and *out* (output message).

The *in* and *out* instructions use the address of the channel to determine whether the channel is internal or external. This means that the same instruction sequence can be used for both hard and soft channels, allowing a process to be written and compiled without knowledge of where its channels are implemented.

Communication takes place when both the inputting and outputting processes are ready. Consequently, the process which first becomes ready must wait until the second one is also ready. The inputting and outputting processes only become active when the communication has completed. A process performs an input or output by loading the evaluation stack with, a pointer to a message, the address of a channel, and a count of the number of bytes to be transferred, and then executing an *in* or *out* instruction.

6.5 Timers

There are two 32-bit hardware timer clocks which 'tick' periodically. These are independent of any on-chip peripheral real time clock. The timers provide accurate process timing, allowing processes to deschedule themselves until a specific time.

One timer is accessible only to high priority processes and is incremented approximately every microsecond, cycling completely in approximately 4295 seconds. The other is accessible only to low priority processes and runs 64 times slower, giving 15625 ticks per second. It has a full period of approximately 76 hours.

Actual timer speeds are derived from the processor speed **ProcClockOut** and are given in the *Clocks* chapter. The periods may be calculated as follows:

$$\text{High_priority_clock_period} = 1\mu\text{s} \times \text{Nominal_speed} / \text{ProcClockOut_speed}$$

$$\text{Low_priority_clock_period} = \text{High_priority_clock_period} \times 64$$

Register	Function
ClockReg0	Current value of high priority (level 0) process clock.
ClockReg1	Current value of low priority (level 1) process clock.
TnextReg0	Indicates time of earliest event on high priority (level 0) timer queue.
TnextReg1	Indicates time of earliest event on low priority (level 1) timer queue.
TptrReg0	High priority timer queue.
TptrReg1	Low priority timer queue.

Table 20 Timer registers

The current value of the processor clock can be read by executing a *ldtimer* (load timer) instruction. A process can arrange to perform a *tin* (timer input), in which case it will become ready to execute after a specified time has been reached. The *tin* instruction requires a time to be specified. If this time is in the 'past' then the instruction has no effect. If the time is in the 'future' then the process is descheduled. When the specified time is reached the process becomes active. In addition, the *ldclock* (load clock), *stclock* (store clock) instructions allow total control over the clock value and the *clockenb* (clock enable), *clockdis* (clock disable) instructions allow each clock to be individually stopped and re-started.

Figure 10 shows two processes waiting on the timer queue, one waiting for time 21, the other for time 31.

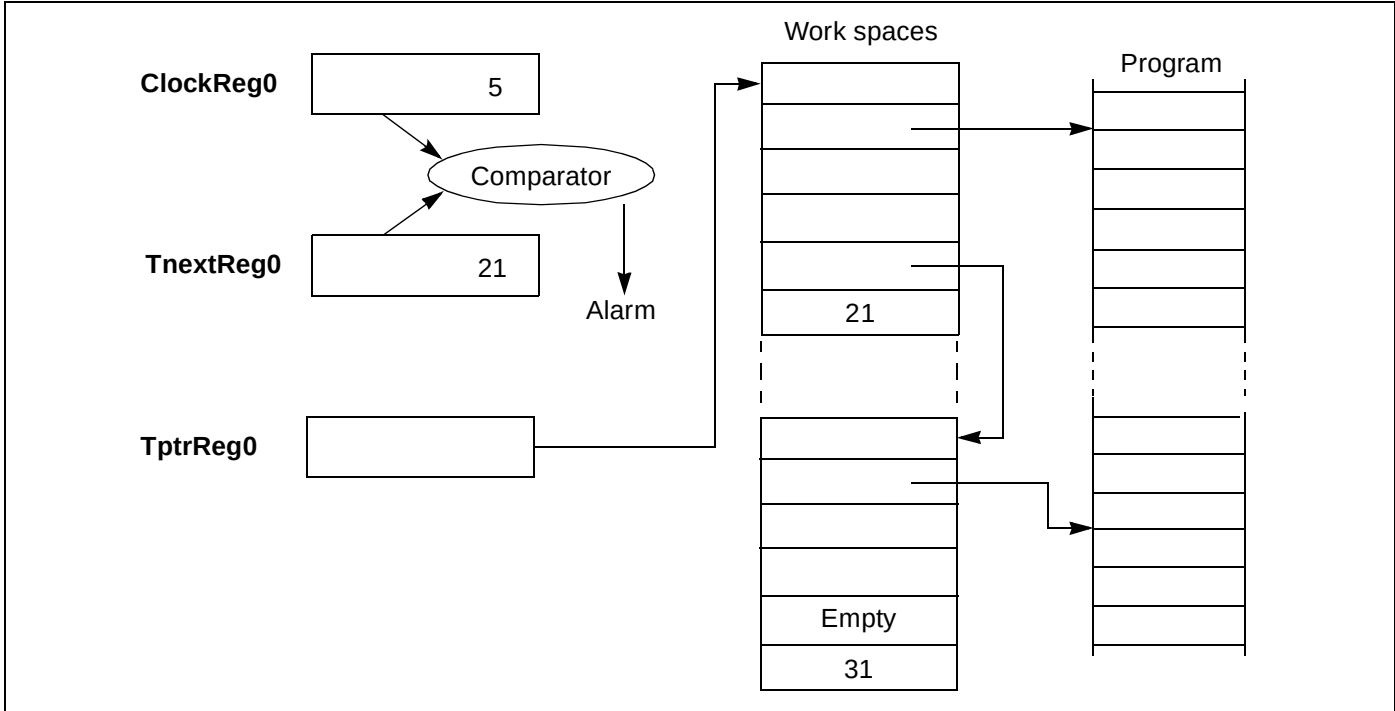


Figure 10 Timer registers

6.6 Traps and exceptions

A software error, such as arithmetic overflow or array bounds violation, can cause an error flag to be set in the CPU. The flag is directly connected to the **ErrorOut** pin. Both the flag and the pin can be ignored, or the CPU stopped. Stopping the CPU on an error means that the error cannot cause further corruption. As well as containing the error in this way it is possible to determine the state of the CPU and its memory at the time the error occurred. This is particularly useful for postmortem debugging where the debugger can be used to examine the state and history of the processor leading up to and causing the error condition. In addition, if a trap handler process is installed, a variety of traps/exceptions can be trapped and handled by software. A user supplied trap handler routine can be provided for each high/low process priority level. The handler is started when a trap occurs and is given the reason for the trap. The trap handler is not re-entrant and must not cause a trap itself within the same group. All traps can be individually masked.

6.6.1 Trap groups

The trap mechanism is arranged on a per priority basis. For each priority there is a handler for each group of traps, as shown in Figure 11.

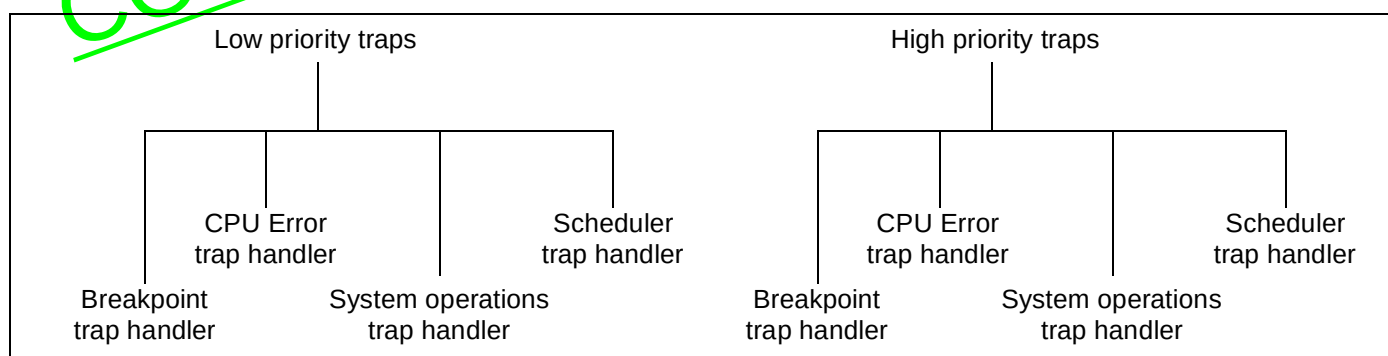


Figure 11 Trap arrangement

There are four groups of traps, as detailed below.

- **Breakpoint:** This group consists of the *Breakpoint* trap. The breakpoint instruction (*j0*) calls the breakpoint routine via the trap mechanism.
- **Errors:** The traps in this group are *IntegerError* and *Overflow*. *Overflow* represents arithmetic overflow, such as arithmetic results which do not fit in the result word. *IntegerError* represents errors caused when data is erroneous, for example when a range checking instruction finds that data is out of range.
- **System operations:** This group consists of the *LoadTrap*, *StoreTrap* and *IllegalOpcode* traps. The *IllegalOpcode* trap is signalled when an attempt is made to execute an illegal instruction. The *LoadTrap* and *StoreTrap* traps allow a kernel to intercept attempts by a monitored process to change or examine trap handlers or trapped process information. It enables a user program to signal to a kernel that it wishes to install a new trap handler.
- **Scheduler:** The scheduler trap group consists of the *ExternalChannel*, *InternalChannel*, *Timer*, *TimeSlice*, *Run*, *Signal*, *ProcessInterrupt* and *QueueEmpty* traps. The *ProcessInterrupt* trap signals that the machine has performed a priority interrupt from low to high. The *QueueEmpty* trap indicates that there is no further executable work to perform. The other traps in this group indicate that the hardware scheduler wants to schedule a process on a process queue, with the different traps enabling the different sources of this to be monitored.

The scheduler traps enable a software scheduler kernel to use the hardware scheduler to implement a multi-priority software scheduler.

Note that scheduler traps are different from other traps as they are caused by the micro-scheduler rather than by an executing process.

Trap groups encoding is shown in Table 21 below. These codes are used to identify trap groups to various instructions.

Trap group	Code
Breakpoint	0
CPU errors	1
System operations	2
Scheduler	3

Table 21 Trap group codes

In addition to the trap groups mentioned above, the **CauseError** flag in the **Status** register is used to signal when a trap condition has been activated by the *causeerror* instruction. It can be used to indicate when trap conditions have occurred due to the user setting them, rather than by the system.

6.6.2 Events that can cause traps

Table 22 summarizes the events that can cause traps, and gives the encoding of bits in the trap **Status** and **Enable** words.

Trap cause	Status/Enable codes	Trap group	Comments
Breakpoint	0	0	When a process executes the breakpoint instruction (<i>j0</i>) then it traps to its trap handler.
IntegerError	1	1	Integer error other than integer overflow - e.g. explicitly checked or explicitly set error.
Overflow	2	1	Integer overflow or integer division by zero.
IllegalOpcode	3	2	Attempt to execute an illegal instruction. This is signalled when <i>opr</i> is executed with an invalid operand.
LoadTrap	4	2	When the trap descriptor is read with the <i>ldtraph</i> instruction or when the trapped process status is read with the <i>ldtrapped</i> instruction.
StoreTrap	5	2	When the trap descriptor is written with the <i>sttraph</i> instruction or when the trapped process status is written with the <i>sttrapped</i> instruction.
InternalChannel	6	3	Scheduler trap from internal channel.
ExternalChannel	7	3	Scheduler trap from external channel.
Timer	8	3	Scheduler trap from timer alarm.
Timeslice	9	3	Scheduler trap from timeslice.
Run	10	3	Scheduler trap from <i>runp</i> (run process) or <i>startp</i> (start process).
Signal	11	3	Scheduler trap from <i>signal</i> .
ProcessInterrupt	12	3	Start executing a process at a new priority level.
QueueEmpty	13	3	Caused by no process active at a priority level.
CauseError	15 (Status only)	Any, encoded 0-3	Signals that the <i>causeerror</i> instruction set the trap flag.

Table 22 Trap causes and Status/Enable codes

6.6.3 Trap handlers

For each trap handler there is a trap handler structure and a trapped process structure. Both the trap handler structure and the trapped process structure are in memory and can be accessed via instructions.

The trap handler structure specifies what should happen when a trap condition is present, see [Table 23](#).

The trapped process structure saves some of the state of the process that was running when the trap was taken, see [Table 24](#).

In addition, for each priority, there is an **Enables** register and a **Status** register. The **Enables** register contains flags to enable each cause of trap. The **Status** register contains flags to indicate which trap conditions have been detected. The **Enables** and **Status** register bit encodings are given in [Table 22 on page 35](#).

Register	Comments	Location
Iptra	Iptra of trap handler process.	Base + 3
Wptr	Wptr of trap handler process. A null Wptr indicates that a trap handler has not been installed.	Base + 2
Status	Contains the Status register that the trap handler starts with.	Base + 1
Enables	A word which encodes the trap enable and global interrupt masks, which will be ANDed with the existing masks to allow the trap handler to disable various events while it runs.	Base + 0

Table 23 Trap handler structure

Register	Comments	Location
Iptra	Points to the instruction after the one that caused the trap condition.	Base + 3
Wptr	Wptr of the process that was running when the trap was taken.	Base + 2
Status	The relevant trap bit is set, see Table 21 on page 34 for trap codes.	Base + 1
Enables	Interrupt enables.	Base + 0

Table 24 Trapped process structure

A trap will be taken at an interruptible point if a trap is set and the corresponding trap enable bit is set in the **Enables** register. If the trap is not enabled then nothing is done with the trap condition. If the trap is enabled then the corresponding bit is set in the **Status** register to indicate the trap condition has occurred.

When a process takes a trap the processor saves the existing **Iptra**, **Wptr**, **Status** and **Enables** in the trapped process structure. It then loads **Iptra**, **Wptr** and **Status** from the equivalent trap handler structure and ANDs the value in **Enables** with the value in the structure. This allows the user to disable various events while in the handler, in particular a trap handler must disable all the traps of its trap group to avoid the possibility of a handler trapping to itself.

The trap handler then executes. The values in the trapped process structure can be examined using the *ldtrapped* instruction. When the trap handler has completed its operation it returns to the trapped process via the *tret* (trap return) instruction. This reloads the values saved in the trapped process structure and clears the trap flag in **Status**.

Note that when a trap handler is started, **Areg**, **Breg** and **Creg** are not saved. The trap handler must save the **Areg**, **Breg**, **Creg** registers using *stl* (store local).

6.6.4 Trap instructions

Trap handlers and trapped processes can be set up and examined via the *ldtraph*, *sttraph*, *ldtrapped* and *sttrapped* instructions. Table 25 describes the instructions that may be used when dealing with traps.

Instruction	Meaning	Use
ldtraph	load trap handler	Load the trap handler from memory to the trap handler descriptor.
sttraph	store trap handler	Store an existing trap handler descriptor to memory.
ldtrapped	load trapped	Load replacement trapped process status from memory.
sttrapped	store trapped	Store trapped process status to memory.
trapebn	trap enable	Enable traps.
trapdis	trap disable	Disable traps.
tret	trap return	Used to return from a trap handler.
causeerror	cause error	Program can simulate the occurrence of an error.

Table 25 Instructions which may be used when dealing with traps

The first four instructions transfer data to/from the trap handler structures or trapped process structures from/to an area in memory. In these instructions **Areg** contains the trap group code (see Table 21 on page 34) and **Breg** points to the 4 word area of memory used as the source or destination of the transfer. In addition **Creg** contains the priority of the handler to be installed/examined in the case of *ldtraph* or *sttraph*. *ldtrapped* and *sttrapped* apply only to the current priority.

If the *LoadTrap* trap is enabled then *ldtraph* and *ldtrapped* do not perform the transfer but set the **LoadTrap** trap flag. If the *StoreTrap* trap is enabled then *sttraph* and *sttrapped* do not perform the transfer but set the **StoreTrap** trap flag.

The trap enable masks are encoded by an array of bits (see Table 22 on page 35) which are set to indicate which traps are enabled. This array of bits is stored in the lower half-word of the **Enables** register. There is an **Enables** register for each priority. Traps are enabled or disabled by loading a mask into **Areg** with bits set to indicate which traps are to be affected and the priority to affect in **Breg**. Executing *trapebn* ORs the mask supplied in **Areg** with the trap enables mask in the **Enables** register for the priority in **Breg**. Executing *trapdis* negates the mask supplied in **Areg** and ANDs it with the trap enables mask in the **Enables** register for the priority in **Breg**. Both instructions return the previous value of the trap enables mask in **Areg**.

6.6.5 Restrictions on trap handlers

There are various restrictions that must be placed on trap handlers to ensure that they work correctly.

- Trap handlers must not deschedule or timeslice. Trap handlers alter the **Enables** masks, therefore they must not allow other processes to execute until they have completed.
- Trap handlers must have their **Enable** masks set to mask all traps in their trap group to avoid the possibility of a trap handler trapping to itself.
- Trap handlers must terminate via the *tret* (trap return) instruction. The only exception to this is that a scheduler kernel may use *restart* to return to a previously shadowed process.

7 Instruction set

This chapter provides information on the ST20-C2 instruction set. It contains tables listing all the instructions, and where applicable provides details of the number of processor cycles taken by an instruction.

The instruction set has been designed for simple and efficient compilation of high-level languages. All instructions have the same format, designed to give a compact representation of the operations occurring most frequently in programs.

Each instruction consists of a single byte divided into two 4-bit parts. The four most significant bits (MSB) of the byte are a function code and the four least significant bits (LSB) are a data value, as shown in [Figure 12](#).

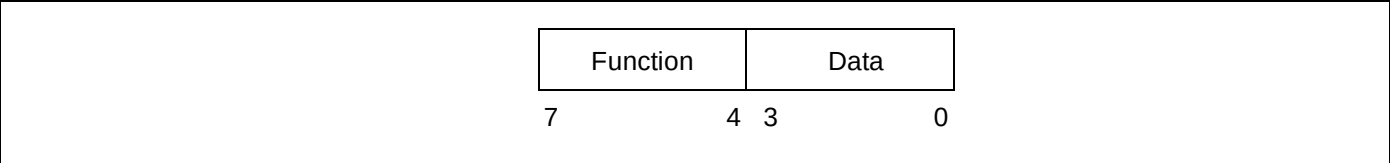


Figure 12 Instruction format

For further information on the instruction set refer to the *ST20C2 Instruction Set Manual* (document number 72-TRN-273).

7.1 Instruction cycles

Timing information is available for some instructions. However, it should be noted that many instructions have ranges of timings which are data dependent.

Where included, timing information is based on the number of clock cycles assuming any memory accesses are to 2 cycle internal memory and no other subsystem is using memory. Actual time will be dependent on the speed of external memory and memory bus availability.

Note that the actual time can be increased by:

- The instruction requiring a value on the register stack from the final memory read in the previous instruction – the current instruction will stall until the value becomes available.
- The first memory operation in the current instruction can be delayed while a preceding memory operation completes - any two memory operations can be in progress at any time, any further operation will stall until the first completes.
- Memory operations in current instructions can be delayed by access by instruction fetch or subsystems to the memory interface.
- There can be a delay between instructions while the instruction fetch unit fetches and partially decodes the next instruction – this will be the case whenever an instruction causes the instruction flow to jump.

Note that the instruction timings given refer to 'standard' behavior and may be different if, for example, traps are set by the instruction.



7.2 Instruction characteristics

Table 28 on page 40 gives the basic function code of each of the primary instructions. Where the operand is less than 16, a single byte encodes the complete instruction. If the operand is greater than 15, one prefix instruction (*prefix*) is required for each additional four bits of the operand. If the operand is negative the first prefix instruction will be *nprefix*. Examples of *prefix* and *nprefix* coding are given in Table 26 below.

Mnemonic	Function code	Memory code
ldc #3	#4	#43
ldc #35		
is coded as		
prefix #3	#2	#23
ldc #5	#4	#45
ldc #987		
is coded as		
prefix #9	#2	#29
prefix #8	#2	#28
ldc #7	#4	#47
ldc -31 (<i>ldc #FFFFFFE1</i>)		
is coded as		
nprefix #1	#6	#61
ldc #1	#4	#41

Table 26 Prefix coding

Any instruction which is not in the instruction set tables is an invalid instruction and is flagged illegal, returning an error code to the trap handler, if loaded and enabled.

The **Notes** column of the tables indicates the features of an instruction as described in Table 27.

Ident	Feature
E	Instruction can set an <i>IntegerError</i> trap
L	Instruction can cause a <i>LoadTrap</i> trap
S	Instruction can cause a <i>StoreTrap</i> trap
O	Instruction can cause an <i>Overflow</i> trap
I	Interruptible instruction
A	Instruction can be aborted and later restarted.
D	Instruction can deschedule
T	Instruction can timeslice

Table 27 Instruction features

7.3 Instruction set tables

Function code	Memory code	Mnemonic	Processor cycles	Name	Notes
0	0X	j	5	jump	D, T
1	1X	ldlp	1	load local pointer	
2	2X	pfix	0 to 1	prefix	
3	3X	ldnl	2	load non-local	
4	4X	ldc	1	load constant	
5	5X	ldnlp	1	load non-local pointer	
6	6X	nfix	0 to 1	negative prefix	
7	7X	ldl	1	load local	
8	8X	adc	1	add constant	O
9	9X	call	8	call	
A	AX	cj	1 or 5	conditional jump	
B	BX	ajw	2	adjust workspace	
C	CX	eqc	1	equals constant	
D	DX	stl	1	store local	
E	EX	stnl	2	store non-local	
F	FX	opr	0	operate	

Table 28 Primary functions

Memory code	Mnemonic	Processor cycles	Name	Notes
22FA	testpranal	2	test processor analyzing	
23FE	saveh	3	save high priority queue registers	
23FD	savel	3	save low priority queue registers	
21F8	sthf	1	store high priority front pointer	
25F0	sthb	1	store high priority back pointer	
21FC	stlf	1	store low priority front pointer	
21F7	stlb	1	store low priority back pointer	
25F4	sttimer	2	store timer	
2127FC	lddevid	1	load device identity	
27FE	ldmemstartval	1	load value of MemStart address	

Table 29 Processor initialization operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
24F6	and	1	and	
24FB	or	1	or	
23FB	xor	1	exclusive or	
23F2	not	1	bitwise not	
24F1	shl	1	shift left	
24F0	shr	1	shift right	
F5	add	1	add	A, O
FC	sub	1	subtract	A, O
25F3	mul	4	multiply	A, O
27F2	fmul	6	fractional multiply	A, O
22FC	div	5 to 37	divide	A, O
21FF	rem	5 to 40	remainder	A, O
F9	gt	1	greater than	A
25FF	gtu	1	greater than unsigned	A
F4	diff	1	difference	
25F2	sum	1	sum	
F8	prod	4	product	A
26F8	satadd	2	saturating add	A
26F9	satsub	2	saturating subtract	A
26FA	satmul	5	saturating multiply	A

Table 30 Arithmetic/logical operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
21F6	ladd	2	long add	A, O
23F8	lsub	2	long subtract	A, O
23F7	lsum	2	long sum	
24FF	ldiff	2	long diff	
23F1	lmul	5 to 6	long multiply	A
21FA	ldiv	5 to 39	long divide	A, O
23F6	lshl	2	long shift left	A
23F5	lshr	2	long shift right	A
21F9	norm	2 to 5	normalize	A
26F4	slmul	5	signed long multiply	A, O
26F5	sulmul	5	signed times unsigned long multiply	A, O

Table 31 Long arithmetic operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
F0	rev	1	reverse	
23FA	xword	4	extend to word	A
25F6	cword	3	check word	A, E
21FD	xdbl	2	extend to double	
24FC	csngl	3	check single	A, E
24F2	mint	1	minimum integer	
25FA	dup	1	duplicate top of stack	
27F9	pop	1	pop processor stack	
68FD	reboot	1	reboot	

Table 32 General operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
F2	bsub	1	byte subscript	
FA	wsb	1	word subscript	
28F1	wsubdb	1	form double word subscript	
23F4	bcnt	1	byte count	
23FF	wcnt	1	word count	
F1	lb	1	load byte	
23FB	sb	2	store byte	
24FA	move		move message	I

Table 33 Indexing/array operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
22F2	ldtimer	1	load timer	
22FB	tin		timer input	I
24FE	talt	3	timer alt start	
25F1	taltwt		timer alt wait	D, I
24F7	enbt	2 to 8	enable timer	
22FE	dist		disable timer	I

Table 34 Timer handling operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
F7	in		input message	D
FB	out		output message	D
FF	outword		output word	D
FE	outbyte		output byte	D
24F3	alt	2	alt start	
24F4	altwt	4 to 7	alt wait	D
24F5	altend	9	alt end	
24F9	enbs	1 to 2	enable skip	
23F0	diss	1	disable skip	
21F2	resetch	3	reset channel	
24F8	enbc	2 to 5	enable channel	
22FF	disc	2 to 7	disable channel	

Table 35 Input and output operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
22F0	ret	3	return	
21FB	ldpi	1	load pointer to instruction	
23FC	gajw	3	general adjust workspace	
F6	gcall	6	general call	
22F1	lend	5 to 8	loop end	T

Table 36 Control operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
FD	startp	5	start process	
F3	endp	4 to 6	end process	D
23F9	runp	3	run process	
21F5	stopp	2	stop process	
21FE	ldpri	1	load current priority	

Table 37 Scheduling operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
21F3	csub0	2	check subscript from 0	A, E
24FD	cnt1	3	check count from 1	A, E
22F9	testerr	2	test error false and clear	
21F0	seterr	2	set error	
25F5	stoperr	2 to 3	stop on error (no error)	D
25F7	clrhalterr	1	clear halt-on-error	
25F8	sethalterr	1	set halt-on-error	
25F9	testhalterr	2	test halt-on-error	

Table 38 Error handling operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
25FB	move2dinit	3	initialize data for 2D block move	
25FC	move2dall		2D block copy	I
25FD	move2dnonzero		2D block copy non-zero bytes	I
25FE	move2dzero		2D block copy zero bytes	I

Table 39 2D block move operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
27F4	crcword	36	calculate crc on word	A
27F5	crcbyte	12	calculate crc on byte	A
27F6	bitcnt	3	count bits set in word	A
27F7	bitrevword	2	reverse bits in word	
27F8	bitrevnbits	2	reverse bottom n bits in word	A

Table 40 CRC and bit operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
27F3	cflerr	3	check floating point error	E
29FC	fptesterr	1	load value true (FPU not present)	
26F3	unpacksn	10	unpack single length floating point number	A
26FD	roundsn	7	round single length floating point number	A
26FC	postnormsn	9	post-normalize correction of single length floating point number	A
27F1	ldinf	1	load single length infinity	

Table 41 Floating point support operation codes

Memory code	Mnemonic	Processor cycles	Name	Notes
2CF7	clr	3	check in range	A, E
2CFC	ciru	3	check in range unsigned	A, E
2BFA	cb	3	check byte	A, E
2BFB	cbu	2	check byte unsigned	A, E
2FFA	cs	3	check sixteen	A, E
2FFB	csu	2	check sixteen unsigned	A, E
2FF8	xsword	3	sign extend sixteen to word	A
2BF8	xbword	3	sign extend byte to word	A

Table 42 Range checking and conversion instructions

Memory code	Mnemonic	Processor cycles	Name	Notes
2CF1	ssub	1	sixteen subscript	
2CFA	ls	1	load sixteen	
2CF8	ss	2	store sixteen	
2BF9	lbx	1	load byte and sign extend	
2FF9	lsx	1	load sixteen and sign extend	

Table 43 Indexing/array instructions

Memory code	Mnemonic	Processor cycles	Name	Notes
2FF0	devlb	3	device load byte	A
2FF2	devls	3	device load sixteen	A
2FF4	devlw	3	device load word	A
62F4	devmove		device move	I
2FF1	devsb	3	device store byte	A
2FF3	devss	3	device store sixteen	A
2FF5	devsw	3	device store word	A

Table 44 Device access instructions

Memory code	Mnemonic	Processor cycles	Name	Notes
60F5	wait	5 to 11	wait	D
60F4	signal	7 to 12	signal	

Table 45 Semaphore instructions

Memory code	Mnemonic	Processor cycles	Name	Notes
60F0	swapqueue	4	swap scheduler queue	
60F1	swaptimer	5	swap timer queue	
60F2	insertqueue	3 to 4	insert at front of scheduler queue	
60F3	timeslice	3 to 4	timeslice	
60FC	ldshadow	6 to 31	load shadow registers	A
60FD	stshadow	6 to 17	store shadow registers	A
62FE	restart	20	restart	
62FF	causeerror	7 to 8	cause error	
61FF	iret	3 to 11	interrupt return	
2BF0	settimeslice	2	set timeslicing status	
2CF4	intdis	2	interrupt disable	
2CF5	intenb	2	interrupt enable	
2CFD	gintdis	5	global interrupt disable	
2CFE	gintenb	5	global interrupt enable	

Table 46 Scheduling support instructions

Memory code	Mnemonic	Processor cycles	Name	Notes
26FE	ldtraph	12	load trap handler	L
2CF6	ldtrapped	12	load trapped process status	L
2CFB	sttrapped	12	store trapped process status	S
26FF	sttraph	12	store trap handler	S
60F7	trapenb	4	trap enable	
60F6	trapdis	4	trap disable	
60FB	tret	8 to 10	trap return	

Table 47 Trap handler instructions

Memory code	Mnemonic	Processor cycles	Name	Notes
68FC	ldprodid	1	load product identity	
63F0	nop	1	no operation	

Table 48 Processor initialization and no operation instructions

Memory code	Mnemonic	Processor cycles	Name	Notes
64FF	clockenb	2	clock enable	
64FE	clockdis	2	clock disable	
64FD	ldclock	2	load clock	
64FC	stclock	2	store clock	

Table 49 Clock instructions

8 Interrupt system

The interrupt system allows an on-chip module or external interrupt pin to interrupt the currently running process in order to run an interrupt handling process.

An interrupt may be signalled by one of the following:

- a signal on an external **Interrupt** pin
- a signal from an internal peripheral or subsystem
- software asserting an interrupt in the **Pending** register

Interrupts are implemented using an on-chip interrupt controller peripheral and an on-chip interrupt level controller. The interrupt level controller (described in [Interrupt level controller on page 50](#)) multiplexes incoming interrupts onto the eight programmable interrupt inputs of the interrupt controller. This multiplexing is controllable by software.

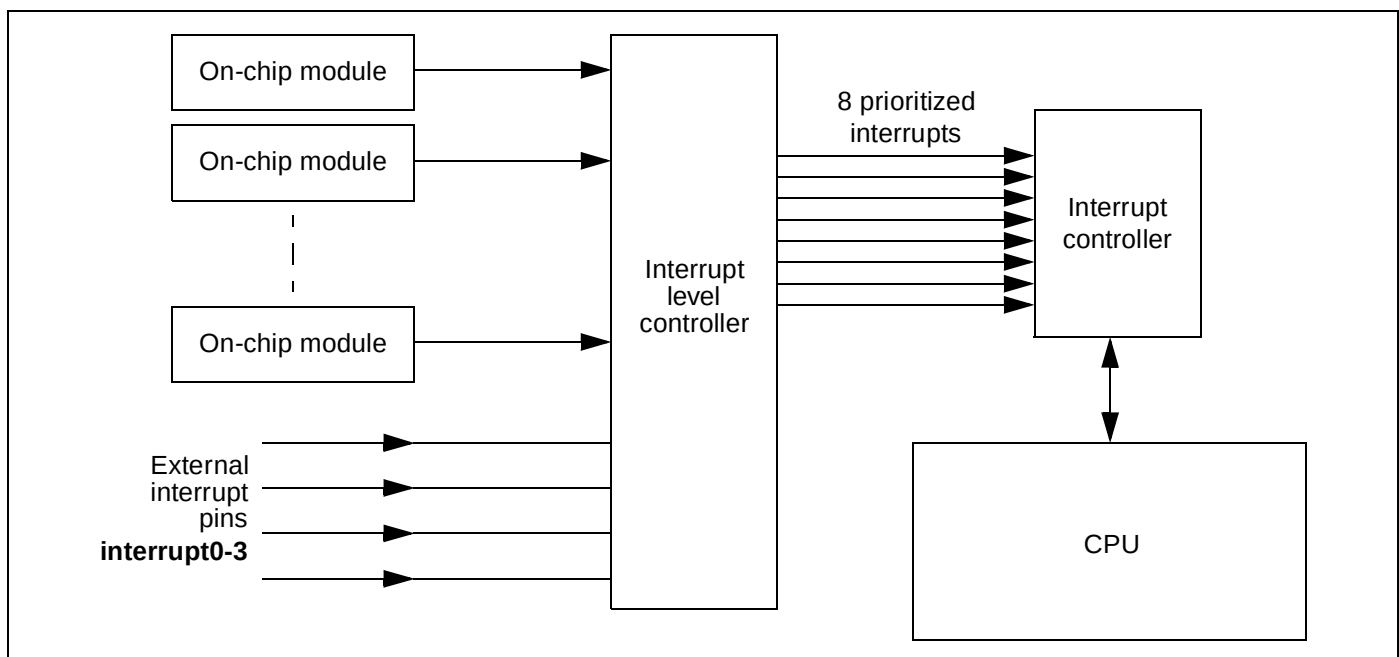


Figure 13 Interrupt system

The interrupt controller supports eight prioritized interrupts as inputs, and manages the pending interrupts. This allows nested pre-emptive interrupts for real-time system design.

All interrupts are at a higher priority than the low priority process queue. Each interrupt can be programmed to be at a lower or higher priority than the high priority process queue, by writing to the priority bit in the **HandlerWptr** registers. Interrupts which are specified as higher priority must be contiguous from the highest numbered interrupt downwards.

For example, if 4 interrupts are programmed as higher priority and 4 as lower priority the higher priority interrupts must be **Interrupt7:4** and the lower priority interrupts **Interrupt3:0**.

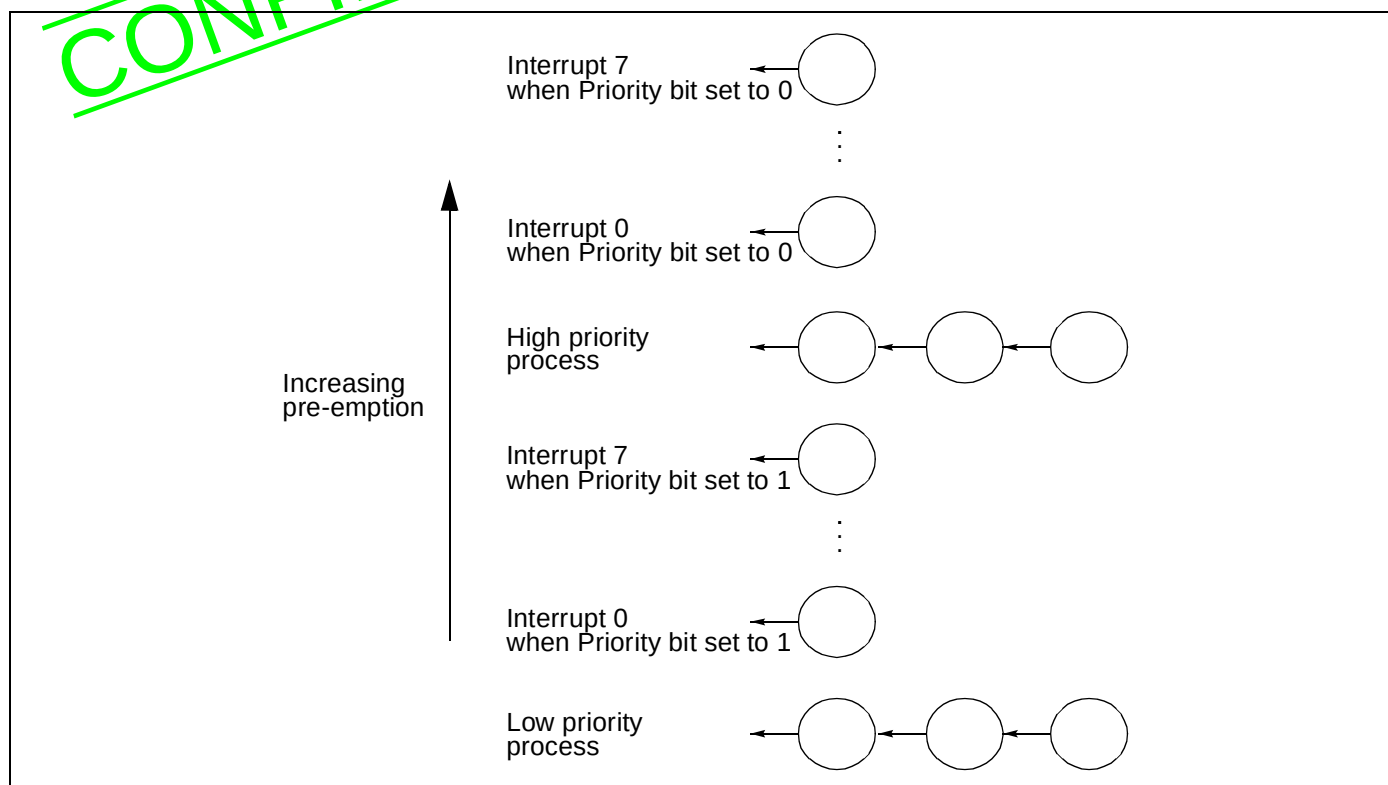


Figure 14 Interrupt priority

8.1 Interrupt vector table

The interrupt controller contains a table of pointers to interrupt handlers. Each interrupt handler is represented by its work space pointer (**HandlerWptr**). The table contains a work space pointer for each level of interrupt.

The **HandlerWptr** gives access to the code, data and interrupt save space of the interrupt handler. The position of the **HandlerWptr** in the interrupt table implies the priority of the interrupt.

Run-time library support is provided for setting and programming the vector table.

8.2 Interrupt handlers

At any interruptible point in its execution the CPU can receive an interrupt request from the interrupt controller. The CPU immediately acknowledges the request.

In response to receiving an interrupt the CPU performs a procedure call to the process in the vector table. The state of the interrupted process is stored in the work space of the interrupt handler as shown in [Figure 15](#) below. Each interrupt level has its own work space.

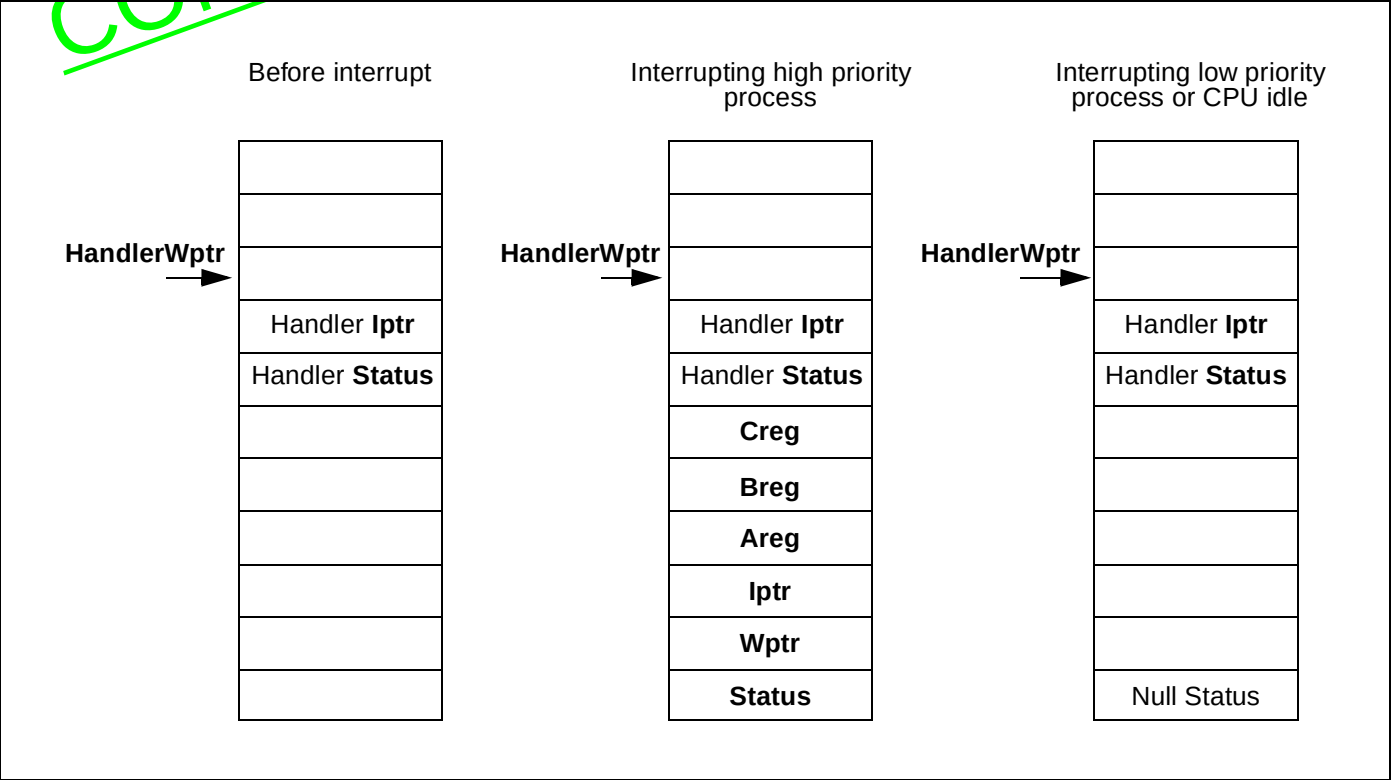


Figure 15 State of interrupted process

The interrupt routine is initialized with space below **HandlerWptr**. The **Iptra** and **Status** word for the routine are stored there permanently. This should be programmed before the **HandlerWptr** is written into the vector table.

The behavior of the interrupt differs depending on the priority of the CPU when the interrupt occurs. If an interrupt occurs when the CPU is running at high priority, and the interrupt is set at a higher priority than the high priority process queue, the CPU saves the current process state (**Areg**, **Breg**, **Creg**, **Wptr**, **Iptra** and **Status**) into the workspace of the interrupt handler. The value **HandlerWptr**, which is stored in the interrupt controller, points to the top of this work space. The values of **Iptra** and **Status** to be used by the interrupt handler are loaded from this work space and starts executing the handler. The value of **Wptr** is then set to the bottom of this save area.

If an interrupt occurs when the CPU is running at high priority, and the interrupt is set at a lower priority than the high priority process queue, no action is taken and the interrupt waits in a queue until the high priority process queue is empty (see [Section 8.4: Pre-emption and interrupt priority](#) on page 50).

Interrupts always take priority over low priority processes. If an interrupt occurs when the CPU was idle or running at low priority, the **Status** is saved. This indicates that no valid process is running (**Null Status**). The interrupted processes (low priority process) state is stored in shadow registers. This state can be accessed via the *ldshadow* (load shadow registers) and *stshadow* (store shadow registers) instructions. The interrupt handler is then run at high priority.

When the interrupt routine has completed it must adjust **Wptr** to the value at the start of the handler code and then execute the *iret* (interrupt return) instruction. This restores the interrupted state from the interrupt handler structure and signals to the interrupt controller that the interrupt has completed. The processor will then continue from where it was before being interrupted.

8.3 Interrupt latency

The interrupt latency is dependent on the data being accessed and the position of the interrupt handler and the interrupted process. This allows systems to be designed with the best trade-off use of fast internal memory and interrupt latency.

8.4 Pre-emption and interrupt priority

Each interrupt channel has an implied priority fixed by its place in the interrupt vector table. All interrupts will cause scheduled processes of any priority to be suspended and the interrupt handler started. Once an interrupt has been sent from the controller to the CPU the controller keeps a record of the current executing interrupt priority. This is only cleared when the interrupt handler executes a return from interrupt (*iret*) instruction. Interrupts of a lower priority arriving will be blocked by the interrupt controller until the interrupt priority has descended to such a level that the routine will execute. An interrupt of a higher priority than the currently executing handler will be passed to the CPU and cause the current handler to be suspended until the higher priority interrupt is serviced.

In this way interrupts can be nested and a higher priority interrupt will always pre-empt a lower priority one. Deep nesting and placing frequent interrupts at high priority can result in a system where low priority interrupts are never serviced or the controller and CPU time are consumed in nesting interrupt priorities and not executing the interrupt handlers.

8.5 Restrictions on interrupt handlers

There are various restrictions that must be placed on interrupt handlers to ensure that they interact correctly with the rest of the process model implemented in the CPU.

- Interrupt handlers must not deschedule.
- Interrupt handlers must not execute communication instructions. However they may communicate with other processes through shared variables using the semaphore *signal* to synchronize.
- Interrupt handlers must not perform 2D block move instructions.
- Interrupt handlers must not cause program traps. However they may be trapped by a scheduler trap.

8.6 Interrupt level controller

The interrupt level controller multiplexes twenty three incoming interrupt signals onto the eight interrupt inputs of the interrupt controller. In this way, it gives programmable control of the priority of the interrupts and extends the number of possible interrupts to twenty three.

There are twenty three interrupt signals to be handled by the interrupt subsystem. They may be generated by other on-chip subsystems or be received from external pins. Software assigns signal *n* to one of the 8 inputs to the interrupt controller by writing the priority of the required input in the register **Int n Priority**.

Thus each input of the interrupt controller responds to zero or more of the twenty three system interrupts. The interrupt level controller asserts interrupt output *p* when one or more of the input interrupts with programmed priority equal to *p* are high. It is level sensitive.

Where two or more system interrupts are assigned to one interrupt handler, the routine is able to ascertain the source of an interrupt by doing a device read from the **InputInterrupts** register and examining the bits that correspond to the system interrupts assigned to that handler.

8.7 Interrupt assignments

The interrupts from the internal peripherals and external pins on the STi5512 are assigned as shown in the table below.

Interrupt	Peripheral	Pin	Notes
0	Port 0		Compare function on PIO port.
1	Port 1		Compare function on PIO port.
2	Port 2		Compare function on PIO port.
3	Port 3		Compare function on PIO port.
4	Port 4		Compare function on PIO port.
5	SSC0		OR of signals SSC0TIR, SSC0RIR, SSC0EIR.
6	SSC1		OR of signals SSC1TIR, SSC1RIR, SSC1EIR.
7	ASC3		OR of signals ASC3TIR, ASC3TBIR, ASC3RIR, ASC3EIR.
8	ASC2		OR of signals ASC2TIR, ASC2TBIR, ASC2RIR, ASC2EIR.
9	ASC1		OR of signals ASC1TIR, ASC1TBIR, ASC1RIR, ASC1EIR.
10	ASC0		OR of signals ASC0TIR, ASC0TBIR, ASC0RIR, ASC0EIR.
11	PWM and Capture		OR of signals PWM0Int, PWM1Int, Capture0Int, Capture1Int.
12	P1284		IEEE1284 parallel port DMA complete.
13	Teletext		Teletext DMA complete.
14	PTI interrupt		
15	Block Move engine		Block move complete.
16			Reserved.
17		Interrupt0	
18		Interrupt1	
19		Interrupt2	Alternative function for PIO port 4 pin 5.
20		Interrupt3	Alternative function for PIO port 4 pin 6.
21	Video decoder		
22	Audio decoder		

Table 50 Interrupt assignments

9 Memory map

9.1 Mapping

The STi5512 has a 32-bit signed twos-complement address space with an address range from **MinInt** (0x80000000) at the bottom to **MaxInt** (0x7FFFFFFF) at the top. A byte of memory is addressed by a 30-bit word address plus a 2-bit byte-selector identifier in the word. A word of memory is addressed by a 30-bit word address with the byte-selector set to zero.

Memory is divided into areas with different memory characteristics and intended purposes. Some areas are dedicated to a specific purpose, either because they contain memory-mapped devices or because they are reserved by the system.

Figure 16 shows the memory map arrangement, and Table 51 on page 53 gives the mapping details.

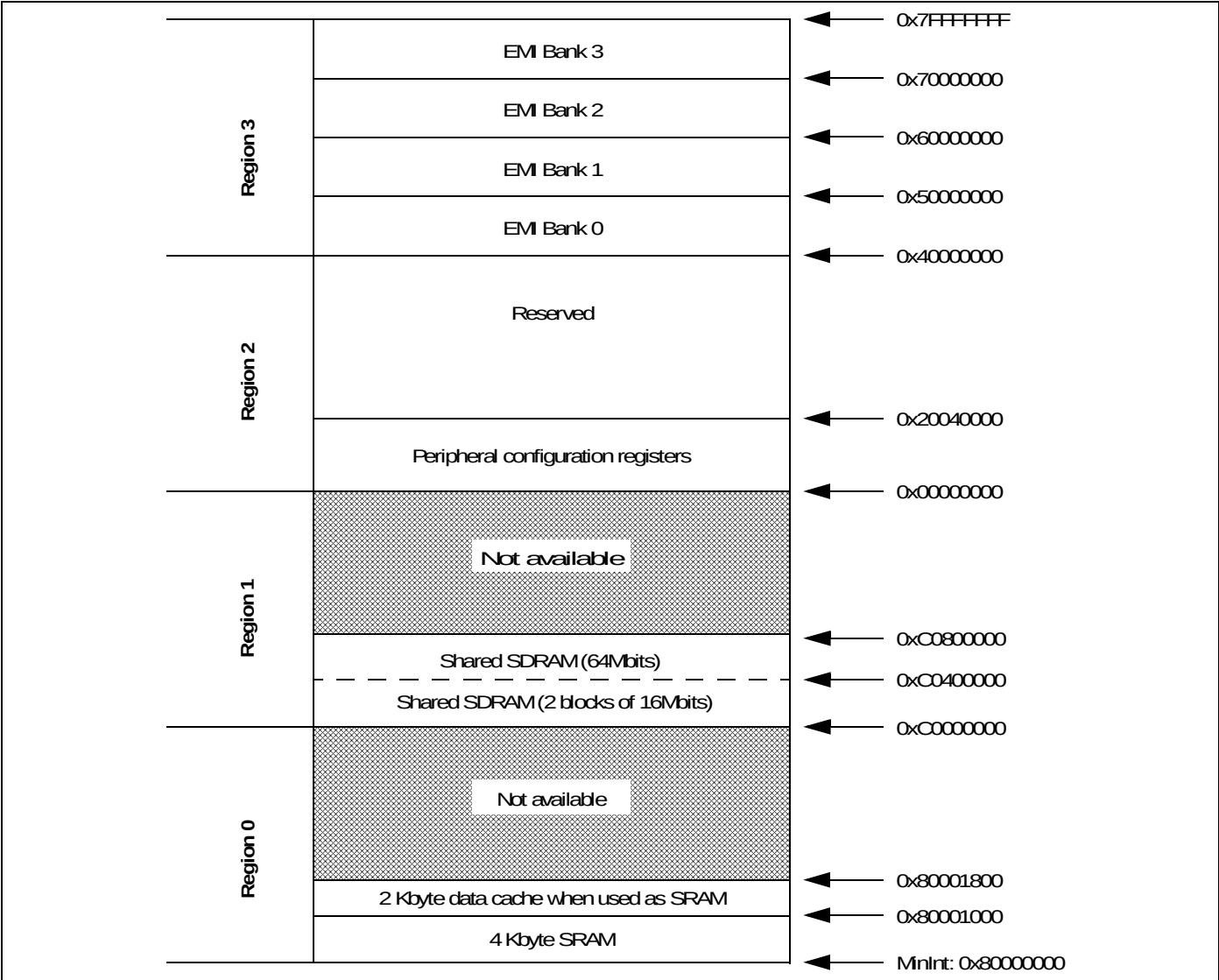


Figure 16 Memory map

Memory is normally accessed by the load, store, block move and channel instructions. These will use data cache if it is enabled, and do not guarantee the order of accesses to different addresses. The device access instructions listed in



Table 44: *Device access instructions on page 45* should be used when there is a need to bypass the data cache in a cacheable area, or if there is a need to know when a write occurs to an external device or memory area.

The address space is divided up for different uses as follows:

- Region 0: The bottom 4 Kbytes (or 6 Kbytes if the data cache is not used) is occupied by on-chip SRAM.
- Region 1: The 8 Mbyte area from 0xC0000000 to 0xC07FFFFFFF is for SDRAM, which is shared with the MPEG decoders. Register bit MY64 of the **CFG_DRC** register selects either of the following memory modes:
 - 16Mb SDRAM memory mode (1 or 2 units), address space 0xC0000000 to 0xC03FFFFFFF
 - 64Mb SDRAM memory mode, address space 0xC0000000 to 0xC07FFFFFFF
- Region 2: The area from 0x00000000 to 0x3FFFFFFF (region 2) is dedicated to memory-mapped or command-mapped on-chip peripheral registers.
- Region 3: 0x40000000 to 0x7FFFFFFF is for external memory and peripherals, accessed through the EMI.

Label	Address (byte)		Use
	Start	Finish	
BootEntry	0x7FFFFFFE		Boot entry point
	0x70000000	0x7FFFFFFF	EMI bank 3, normally used for boot ROM. DRAM/SDRAM not supported.
	0x60000000	0x6FFFFFFF	EMI bank 2: DRAM/SDRAM not supported
	0x50000000	0x5FFFFFFF	EMI bank 1: DRAM or SDRAM supported
	0x40000000	0x4FFFFFFF	EMI bank 0: DRAM or SDRAM supported
	0x20040000	0x3FFFFFFF	Reserved
	0x20030000	0x2003FFFF	Programmable transport interface (PTI) (64 Kbytes) ¹
	0x20028000	0x2002FFFF	Reserved
	0x20027000	0x20027FFF	IEEE1394 link layer interface (LLI) ¹
	0x20026000	0x20026FFF	Block move DMA controller ¹
	0x20025000	0x20025FFF	IEEE1284 parallel port ¹
	0x20024000	0x20024FFF	Teletext interface ¹
	0x20012000	0x20023FFF	Reserved
	0x20011000	0x20011FFF	Interrupt level controller ¹
	0x20010000	0x20010FFF	PIO port 4 controller ¹
	0x2000F000	0x2000FFFF	PIO port 3 controller ¹
	0x2000E000	0x2000EFFF	PIO port 2 controller ¹
	0x2000D000	0x2000DFFF	PIO port 1 controller ¹
	0x2000C000	0x2000CFFF	PIO port 0 controller ¹
	0x2000B000	0x2000BFFF	PWM and counter controller ¹
	0x2000A000	0x2000AFFF	Synchronous serial controller (SSC) 1 ¹
	0x20009000	0x20009FFF	Synchronous serial controller (SSC) 0 ¹
	0x20008000	0x20008FFF	SmartCard interface 1 ¹
	0x20007000	0x20007FFF	SmartCard interface 0 ¹

Table 51 STi5512 memory map

Label	Address (byte)		Use
	Start	Finish	
	0x20006000	0x20006FFF	Asynchronous serial controller (ASC) 3 ¹
	0x20005000	0x20005FFF	Asynchronous serial controller (ASC) 2 (SmartCard 0) ¹
	0x20004000	0x20004FFF	Asynchronous serial controller (ASC) 1 ¹
	0x20003000	0x20003FFF	Asynchronous serial controller (ASC) 0 (SmartCard 1) ¹
	0x20001000	0x20002FFF	Reserved
	0x20000400	0x20000FFF	Low power controller ¹
	0x20000000	0x200003FF	Interrupt controller ¹
	0x00005002	0x1FFFFFFF	Reserved
	0x00005000	0x00005001	MPEG control register
	0x00004000	0x00004FFF	Cache configuration ¹
	0x00003000	0x00003FFF	Diagnostic controller unit (DCU) ¹
	0x00002000	0x00002FFF	External memory interface (EMI) ¹
	0x00000000	0x00001FFF	Audio, video, DENC and sub-picture
	0xC0400000	0xFFFFFFFF	Reserved
	0xC0000000	0xC03FFFFFFF	SMI SDRAM: Video/Audio, OSD memory, CPU user code, data and stack.
	0x80001800	0xBFFFFFFF	Reserved
	0x80001000	0x800017FF	Internal SRAM if the data cache is not enabled. User code, data and stack.
MemStart	0x80000140	0x80000FFF	Internal SRAM: < 4 Kbytes user code, data and stack
	0x80000130	0x8000013F	Low priority Scheduler trapped process
	0x80000120	0x8000012F	Low priority Scheduler trap handler
	0x80000110	0x8000011F	Low priority SystemOperations trapped process
	0x80000100	0x8000010F	Low priority SystemOperations trap handler
	0x800000F0	0x800000FF	Low priority Error trapped process
	0x800000E0	0x800000EF	Low priority Error trap handler
	0x800000D0	0x800000DF	Low priority Breakpoint trapped process
	0x800000C0	0x800000CF	Low priority Breakpoint trap handler
	0x800000B0	0x800000BF	High priority Scheduler trapped process
	0x800000A0	0x800000AF	High priority Scheduler trap handler
	0x80000090	0x8000009F	High priority SystemOperations trapped process
	0x80000080	0x8000008F	High priority SystemOperations trap handler
	0x80000070	0x8000007F	High priority Error trapped process
	0x80000060	0x8000006F	High priority Error trap handler
	0x80000050	0x8000005F	High priority Breakpoint trapped process
	0x80000040	0x8000004F	High priority Breakpoint trap handler
TrapBase	0x80000014	0x8000003F	Reserved.
	0x80000010	0x80000013	Link0 (boot) input channel
	0x80000004	0x8000000F	Reserved
	0x80000000	0x80000003	Link0 output channel

Table 51 STi5512 memory map

1. Registers accessed via CPU device accesses

9.2 System memory use

Certain sections of the address space are reserved for system use as follows:

- The locations below the address **MemStart** at the bottom of memory are dedicated to processor use. The address of **MemStart** is returned by the *ldmemstartval* instruction.
- When booting from ROM, the system boots from the predefined location **BootEntry** (0x7FFFFFFE) at the top of memory.

Areas of memory reserved for processor use should not be accessed directly. Special instructions are provided for manipulating these areas. The special address **MemStart** marks the base of user memory space.

Peek and *poke* use the two words above **MemStart**, i.e. memory locations 0x80000140 to 0x80000147. These words should not be used by the application if it is to be debugged via the OS-Link. The use of peek and poke is described in [Chapter 12](#).

9.2.1 Subsystem channels memory

Each channel based DMA subsystem is allocated a word of storage below **MemStart**. This is used by the processor to store information about the state of that channel. This information should not normally be examined directly, although debugging kernels may need to do so.

Interrupting DMA subsystems do not have a channel word allocated and rely on interrupts to perform synchronization with the processes running on the processor. Note : on a STi5512 device, there is only one channel based DMA subsystem, the OS-Link.

9.2.2 Boot channel

The subsystem channel which is a link input channel, is identified as a 'boot channel'. When the processor is reset and is set to boot from link, it waits for boot commands on this channel. In the case of STi5512 this is the OS link channel **Link0**.

9.2.2.1 Memory for trap handlers

The area of memory reserved for trap handlers is broken down hierarchically:

- Each high/low process priority has a set of trap handlers
- Each set of trap handlers has a handler for each of the four trap groups
- Each trap group handler has a trap handler structure and a trapped process structure
- Each of the structures contains four words

The contents of these addresses can be accessed via *ldtraph*, *sttraph*, *ldtrapped* and *sttrapped* instructions.

9.2.3 Boot ROM

When the processor boots from ROM, it jumps to a boot program held in ROM with an entry point 2 bytes from the top of memory at 0x7FFFFFFE. These 2 bytes are used to encode a negative jump of up to 256 bytes down in the ROM program. For large ROM programs it may then be necessary to encode a longer negative jump to reach the start of the routine.

10 Memory

10.1 External memory

EMI accessible memory

The EMI decodes region 3 of the address space into four banks, into which different external memories and peripherals can be mapped. Two of the banks (banks 0,1) support either DRAM or SDRAM and one bank is normally used for boot ROM (bank 3).

- The locations 0x40000000 to 0x5FFFFFFF are generally used for DRAM or SDRAM, but may be used for any external memory or peripherals.
- The locations 0x60000000 to 0x6FFFFFFF may be used for any external memory or peripherals except DRAM/SDRAM.
- The locations 0x70000000 to 0x7FFFFFFF may be used for any external memory or peripherals except DRAM/SDRAM, but are generally used for boot ROM. When booting from ROM, the system boots from the predefined location **BootEntry** (0x7FFFFFFE) at the top of memory space.

Accessing some areas of memory causes special access characteristics (strokes etc.) to be generated depending on the way the EMI is programmed.

The EMI provides address decoding, address and data buses, timing strobes, enabling signals and refresh where appropriate. In addition, some internal registers are provided which are used to:

- control the cacheability status of some memory addresses for the DCACHE access,
- configure the DCACHE as 2K SRAM block,
- enable the ICACHE,
- invalidate/flush the CACHES,
- provide status information,
- lock some configuration registers.

MPEG SDRAM (SMI - Shared Memory Interface)

The shared MPEG SDRAM memory interface occupies up to the first 64 Mbits of region 1, and is shared between the CPU and the MPEG decoders and other A/V processes. OSD and still image bitmaps, for example, are stored in this memory. The SMI only supports SDRAM memory, either 1 x 16Mbit device, 2 x 16Mbit devices or 1 x 64Mbit device. For details of the SMI SDRAM interface configuration and set-up, refer to the register manual.

10.2 On-chip SRAM memory

This internal memory module, known as on-chip memory, contains 6 Kbytes of SRAM, which is mapped into the lowest 6 Kbytes of memory space from **MinInt** (0x80000000) extending upwards, as shown in [Figure 16: Memory map on page 52](#).

Part of the lowest 4 Kbytes of memory is committed to system use; see [System memory use on page 55](#) for details. The remainder of the lowest 4 Kbytes of memory is uncommitted and can be used to store on-chip data, stack or code for time-critical routines.

The upper 2 Kbytes of the on-chip memory is also uncommitted SRAM, and is contiguous with the lower 2 Kbytes. However, it can be configured to be the data cache, as described below, in which case it is not available as SRAM.

Locations between 0x80001800 (or 0x80001000 if data cache is used) and 0xBFFFFFFF should not be addressed.

10.3 Caching

Cache can be used to reduce the average access delay imposed on the CPU when it accesses a memory location to read or write. Some locations should not be cached, for example those to which other modules have direct memory access (DMA) and the CPU also requires access to those locations. This is because DMA operations always bypass the data cache and so cache incoherence problems will occur. It is therefore recommended to configure such memory locations as being non-cacheable.

The STi5512 cache subsystem provides:

- 2 Kbytes of direct-mapped write-back data cache
- 2 Kbytes of direct-mapped read-only instruction cache

The cache configuration is held in memory-mapped registers. The registers must be accessed using the device access instructions.

Device access instructions can also be used to force access to external memory as they bypass the cache. Device writes do not change the value in the cache. These instructions can be used to solve any cache coherency issues, for example where data is being DMA copied from a cached location. However care must be taken in using device access instructions to access memory (rather than device registers) as cache coherency problems may occur rather than being solved, if the CPU also performs normal (therefore cached) memory accesses to these locations.

Registers are provided to configure areas of memory to be cacheable or non-cacheable for data access, as described in [Cacheable and non-cacheable memory locations on page 60](#).

Note that the correct cache initialization sequences, described in [Cache initialization on page 58](#), must be used before the caches are enabled.

10.3.1 Outline of operation

The cache is four 32-bit words (16 bytes) wide and 128 lines (2 KBytes, 512 words) high. It is direct-mapped (sometimes called *one way set associative*). This is shown in [Figure 17](#) below.

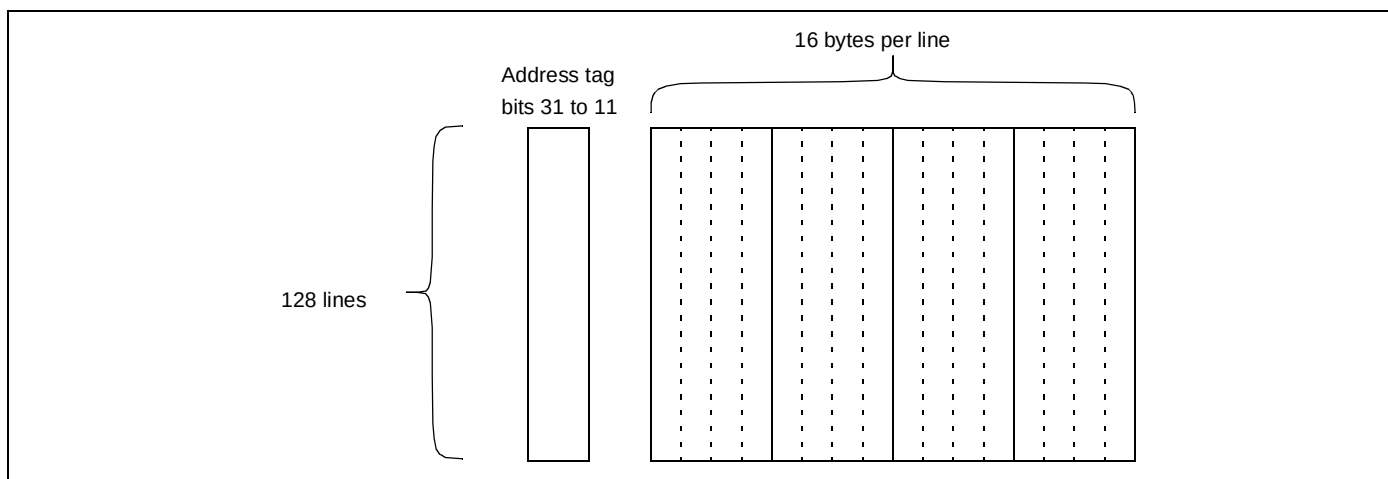


Figure 17 Kbyte data or instruction cache2

Each line of the cache can only store data from specific four-word sections of memory at 2 Kbyte intervals, with the bottom line of the cache coinciding with the 4 words just above each 2 Kbyte boundary. Thus the line number of the cache pinpoints the four-word section of memory within a 2 Kbyte block, i.e. bits 4 to 10 of the address. The 21 most significant bits of the address selects the 2 Kbyte block. These 21 bits are stored in 128 tag registers, with one tag

register corresponding to each cache line. The significance of the parts of the address when using the cache are shown in the figure below.

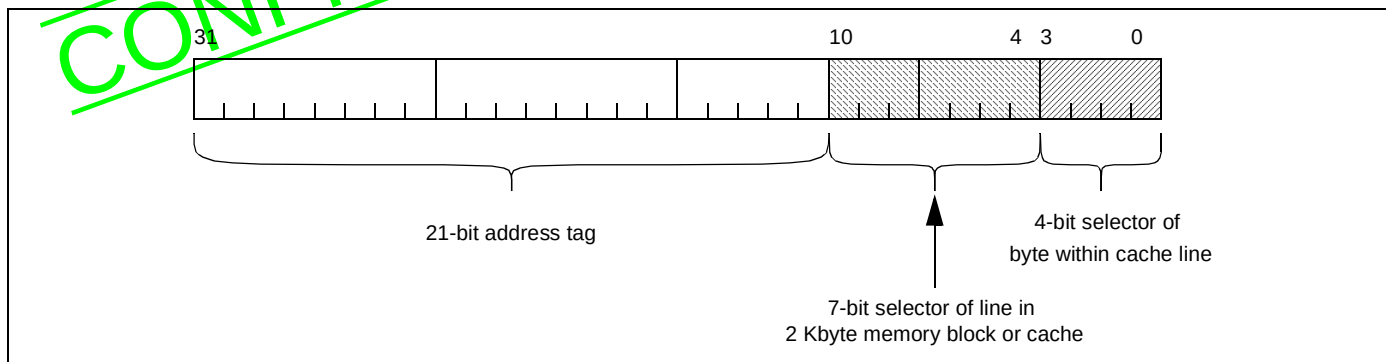


Figure 18 Address fields when using cache

If a request is made to access a cacheable memory location, and a copy of that location is held in cache, then the access is said to have made a cache hit. A hit is identified by comparing the address bits 11 to 31 with the address tag for the cache line given by the address bits 4 to 10. If the cache is hit, then the access is completed by the cache subsystem. If the cache is missed, the appropriate cache line is written back to memory, and if necessary the new location in memory is read into that cache line. All cache reads and writes to memory are complete lines because of the efficiency of accessing the memory in burst mode.

10.3.2 Cache initialization

Before the caches are enabled, they must be correctly initialized. To do this the cache must first be invalidated before it is accessed. To ensure this occurs, the invalidate bit of each cache must be set with the cache disabled and then the enable bit set to enable the cache.

This sequence has the effect of forcing a cache to be invalid, which initializes the cache state before any other accesses are considered by the cache.

10.3.3 Cache subsystem control

The cache subsystem registers control cache functions such as flushing and invalidation, and are used to mark sections of memory space as cacheable or not cacheable. Registers should be accessed using the device access instructions.

The **CAC_CacheControlLock** must be 0 before some registers can be changed. After changing registers, the **CAC_CacheControlLock** should be set to 1. Once this lock is set it cannot be cleared except by a reset. It is not recommended to change the cache configuration other than at reset.

10.3.4 Data cache

It is possible to select either data cache or an extra 2 Kbyte of on-chip SRAM. This is done by writing to the **CAC_DCachNotSRAM** register. The default is to enable the extra on-chip SRAM.

Set the **CAC_DCachNotSRAM** bit to 1 to select data cache mode and enable the cache. Do not access locations 0x80001000 to 0x800017FF when using the data cache. The cache invalidate bit should be set before enabling the cache. Invalidating a cache marks every line as not containing valid data. This is done by setting the **CAC_InvalidateDCache** register to 1. This register is automatically reset to 0 on completion of the task, but it cannot be directly read by the CPU as the register is write only. Therefore bit 2 is provided in the **CAC_CacheStatus** register to indicate when the invalidate operation has completed.

Note that for the data cache, setting the **CAC_InvalidateDCache** register before the cache is enabled does not actually cause the invalidation sequence to begin. This only occurs when the cache is enabled when writing to the **CAC_DCachNotSRAM** register. So the initialize sequence of operations should be:

- write 1 to the **CAC_InvalidateDCache** register;
- write 1 to the **CAC_DCachNotSRAM** register as an “atomic” operation.
Where, “atomic” means than no other CPU software processes / tasks, trap handlers or interrupt handlers can execute in the time between the write to invalidate register and the write to cache enable register. Note: this will normally be the case as the cache initialization should be performed during the initialization stage of the application code, before any operating system (STLite/OS20) or interrupt handlers have been installed.
- wait for the invalidate sequence (128 CPU clock cycles) to finish.
This can done by a software delay loop and/ or by polling bit 2 of the cache status register.

Note it is recommended to ensure that the CPU does not attempt to execute any code or access any data in cacheable memory locations before the invalidate sequence has finished. Therefore if the cache initialization is done by the CPU (rather than by poking the registers via the TAP and DCU, before booting the application code from the software toolset) the CPU software (code and data) that waits for the invalidate sequence to finish should reside in non-cacheable memory locations, so that the cache is not accessed while it is still invalidating. If this is not ensured it is possible for data corruption to occur.

It is not recommended to change selection from data cache to SRAM during operation. However, if it is necessary to do so, it is essential to flush the cache to maintain memory integrity before making the change.

Flushing the cache means forcing a write-back to memory of every dirty line in the cache. A dirty line is a line of cache that has been written to since it was loaded or last written back. Only the data cache can be flushed; the instruction cache never needs flushing since it is read only.

To flush the data cache, set the **CAC_FlushingDCache** register to 1. It is automatically reset to 0 on completion of the task, but cannot be read directly by the CPU as the register is write only. Therefore to check that the flush operation has finished it is necessary to poll bit 4 of the cache status register.

Ensure that the cache-flush software function is “atomic” and operates as follows:

- writes to the flush register;
- waits for the flush to finish;
- writes to the **CAC_DCachNotSRAM** register as an “atomic” operation.

Where, “atomic” means than no other software processes / tasks, trap handlers or interrupt handlers can execute in the time between writing to the flush register and the DCache being changed back to SRAM mode.

Note: the CPU code function and associated data used to perform this sequence must be placed in non-cacheable memory locations, so the cache is not accessed while it is still flushing.

10.3.5 Instruction cache

To use the instruction cache it must be first be invalidated by writing 1 to the **CAC_InvalidateICache** register. Invalidating a cache marks every line as not containing valid data. This is done by setting the **CAC_InvalidateICache** register to 1. This register is automatically reset to 0 on completion of the task (128 CPU cycles). However this register can not be read by the CPU, as it is write only, therefore bit 3 (Invalidating ICache) in the **CAC_CacheStatus** register can be read instead to check that the invalidate operation has completed. The instruction cache must be allowed to complete the invalidate operation before the cache is enabled. Note: unlike the data cache the instruction cache actually starts the invalidation sequence when the **CAC_InvalidateICache** is written to and so can be allowed to complete before enabling the cache by writing to the **CAC_EnableICache** register.

The instruction cache can then be enabled by writing 1 to the **CAC_EnableICache** register; the default condition is disabled, no instruction cache.

The **CAC_CacheStatus** register is read only, and shows the current state of the caches.

The cache configuration can be locked by writing a 1 to the **CAC_CacheControlLock** register bit. Reset of this flag is only performed by a hardware reset. This bit should be set to 1 after all the cache configuration registers have been written.

10.3.6 Cacheable and non-cacheable memory locations

Note: The cacheability control registers for both region1 and region 3 should be programmed before the caches are enabled. In this way the cacheable and non-cacheable memory areas are already defined when the caches are enabled, see [Cache initialization on page 58](#). Do not dynamically change these registers after cache initialization.

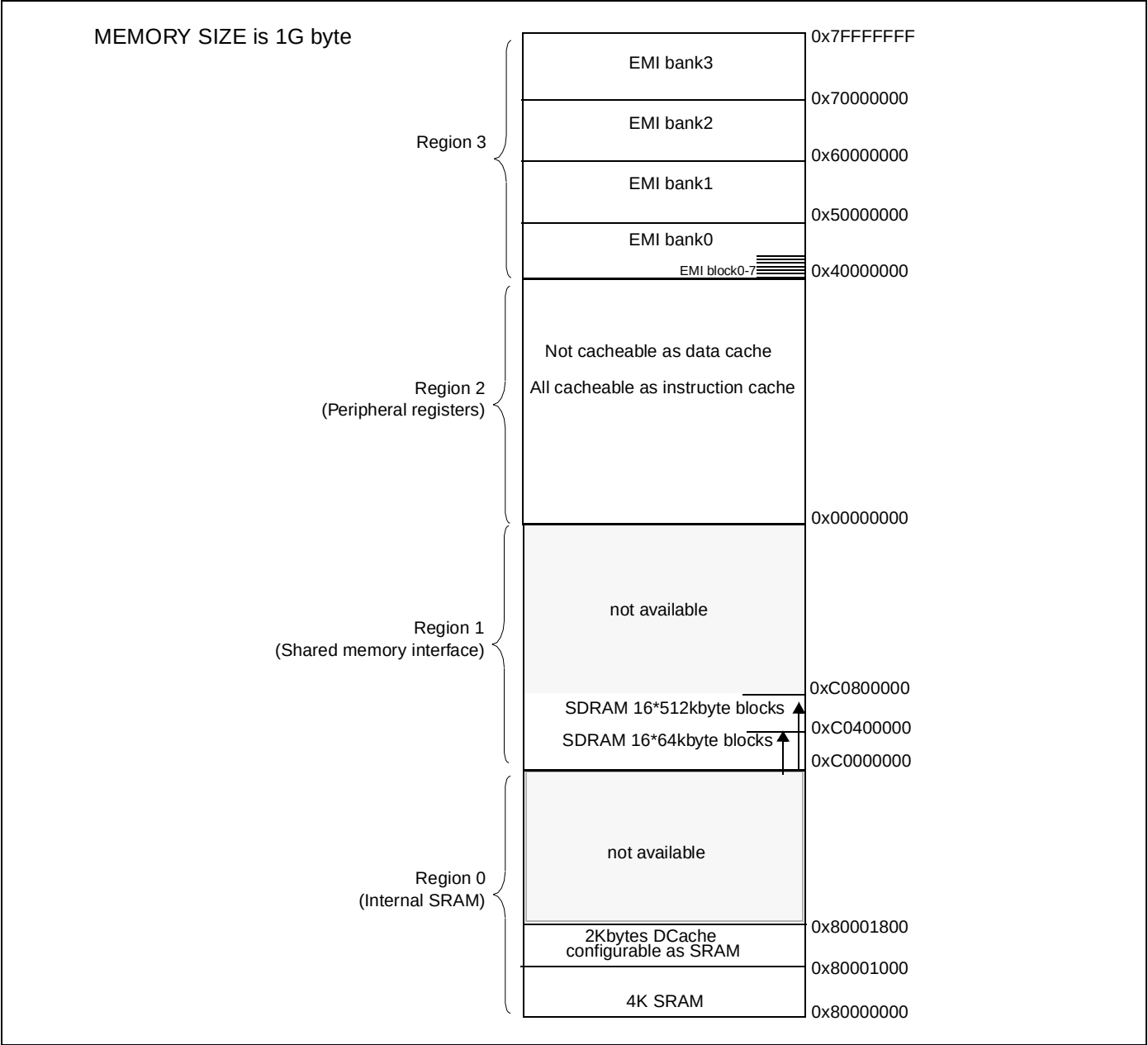


Figure 19 Memory cacheability map



Region 0

REGION 0 is used only to access internal SRAM and is, therefore, never cacheable for either ICACHE or DCACHE.

There are always 4K of SRAM. Because the DCACHE can be configured to be SRAM, there can be another 2K SRAM (for a total amount of 6K SRAM). This is controlled by the **CAC_DCachNotSRAM** register, which must be configured to '0' for SRAM, or to '1' for DCACHE.

Region 1

This region is mostly non-cacheable for data access. It can be programmed to be cacheable (default is non-cacheable) in either 16 blocks of 64K bytes or 16 blocks of 512K bytes. These blocks are contiguous and are placed starting from the address 0xC0000000. Each one of these blocks may be selected individually as cacheable or non-cacheable for DCACHE memory accesses. This is controlled by two CAC registers (CAC_CacheControl0 and 1), where each bit controls the cacheability characteristics of a particular 64K/512K block. To select between 64K byte or 512K byte block size, bit 8 of **CAC_CacheControl0** register is used. After reset, all blocks are marked non-cacheable and the size of each block is set to 64K byte. The registers must then be programmed to mark certain blocks cacheable.

The remainder of REGION1 is non-cacheable for DCACHE.

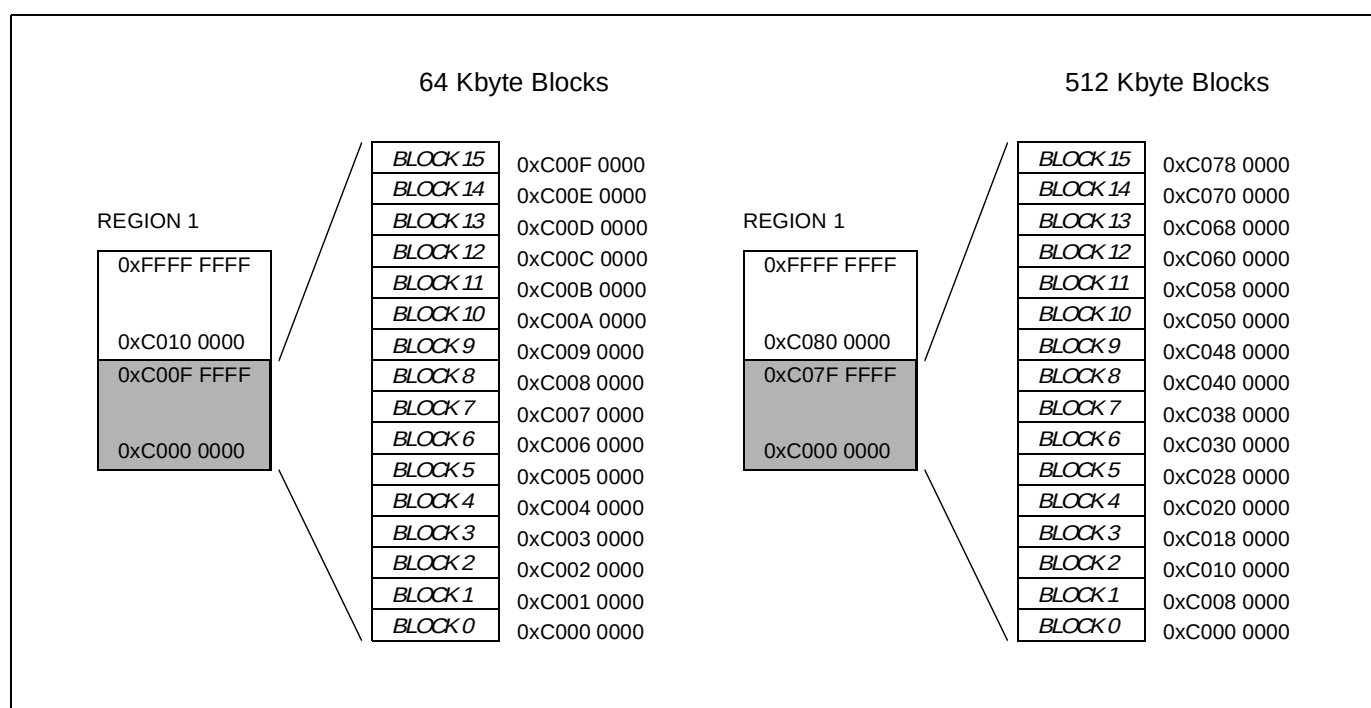


Figure 20 Region1 cacheable area with block sizes of 64K byte and 512Kbyte.

Region 2

Always non-cacheable for DCACHE, but is all cacheable for ICACHE.

Region 3

This region is split into 4 EMI banks, 0-3. For ICACHE they are all cacheable. For DCACHE they can be set as cacheable or non-cacheable by the 4-bit register **CAC_EMIBank0-3**, the default is non-cacheable. EMI bank 0 is split into two parts; the upper part is cacheable in the same way as EMI banks 1-3 and the lower part, containing 8 x 64Kbyte blocks, can be set non-cacheable. These blocks, located between 0x40000000 and 0x4007FFFF, can be set cacheable by the 8-bit register **CAC_EMIBlock0**.

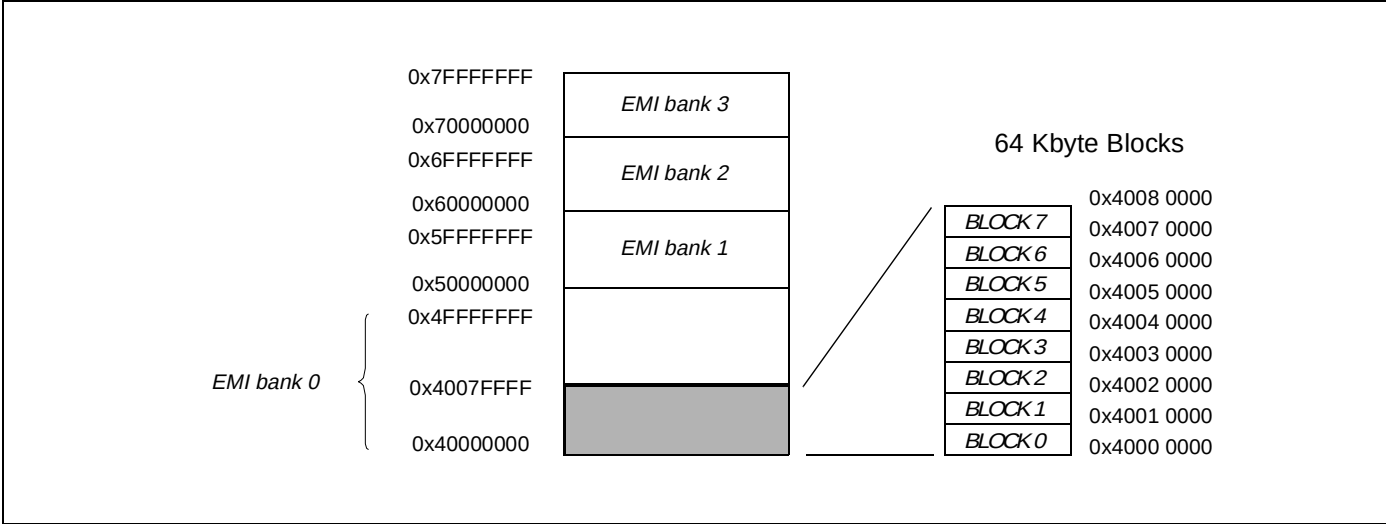


Figure 21 Region 3

11 External memory interface

11.1 Overview

The External Memory Interface (EMI) controls the movement of data between the STi5512 CPU and off-chip memory. It supports memory subsystems with minimal (often zero) external support logic, for up to 4 banks of external memory.

The EMI is similar to that of the STi5510 and for DRAM and SRAM/peripheral device type modes, is backwards compatible with the STi5510. However, the STi5512 also supports SDRAM memory, and address shifting in bank3 (FLASH/ROM bank) depending on the boot-from-ROM data-bus width size (set by the BootSource0-1 pins). The STi5512 can be booted in 5510 mode so that the EMI strobes are compatible with the STi5510, in this mode they remain in an inactive high state when the EMI is requested by an external DMA controller (using the MemReq pin). Alternatively, the device can be booted in 5512 mode where all the strobes are tristated. 5512 mode allows the EMI to interface easily to an external microprocessor, such as the ST40.

The EMI can access a 16 Mbyte physical address space (greater if SDRAM/DRAM is used) in four general-purpose memory banks. The EMI supports the memory subsystems required in most set-top receiver applications, with zero external support logic, including 8, 16 and 32-bit DRAM devices or 16 and 32 bit wide SDRAM devices.

The memory map is divided into four equal regions (EMI banks) which may be independently configured to accommodate one of SDRAM, DRAM, SRAM, ROM, FLASH and most peripheral device types (see the figure below). A bank can only accommodate one type of device but different device types can be placed in different banks to provide glueless support for mixed memory systems. At the most, two banks may be configured as a SDRAM/DRAM (one in bank 0 and the other in bank 1) though these banks may be sub-decoded to provide glueless connection to several devices.

Note It is not possible to have both SDRAM and DRAM in different banks at the same time on the STi5512.

The interface can be configured for a wide variety of timing and decode functions through the configuration registers.

The timing of each of the four memory banks can be set separately, with different device types being placed in each bank (up to two SDRAM/DRAM banks are allowed) with no external hardware support. Banks can be configured to contain 8, 16 or 32 bit wide devices, and board population options for different widths can be supported using the bank width dependent address shifting feature.

The external memory interface efficiently supports dynamic memory and other devices such as static memory and IO devices. This is possible because the required wave-forms can be programmed by configuration registers (see [EMI configuration on page 94](#)).

Memory is byte addressed, with words aligned on four-byte boundaries and half-words on two-byte boundaries.

During read cycles, byte-level addressing is performed internally by the STi5512. The EMI can read bytes, half-words or words. It always reads the size of the bank.

During write cycles the STi5512 uses the notMemBE0-3 strobes to address bytes. If a particular byte is not to be written, then the corresponding data outputs are tristated. Writes can be less than the size of the bank.

The internally generated address is indicated on pins MemAddr2-23. The least significant bit of the data bus is always MemData0. The most significant bit is adjusted dynamically to suit the required external bus size.

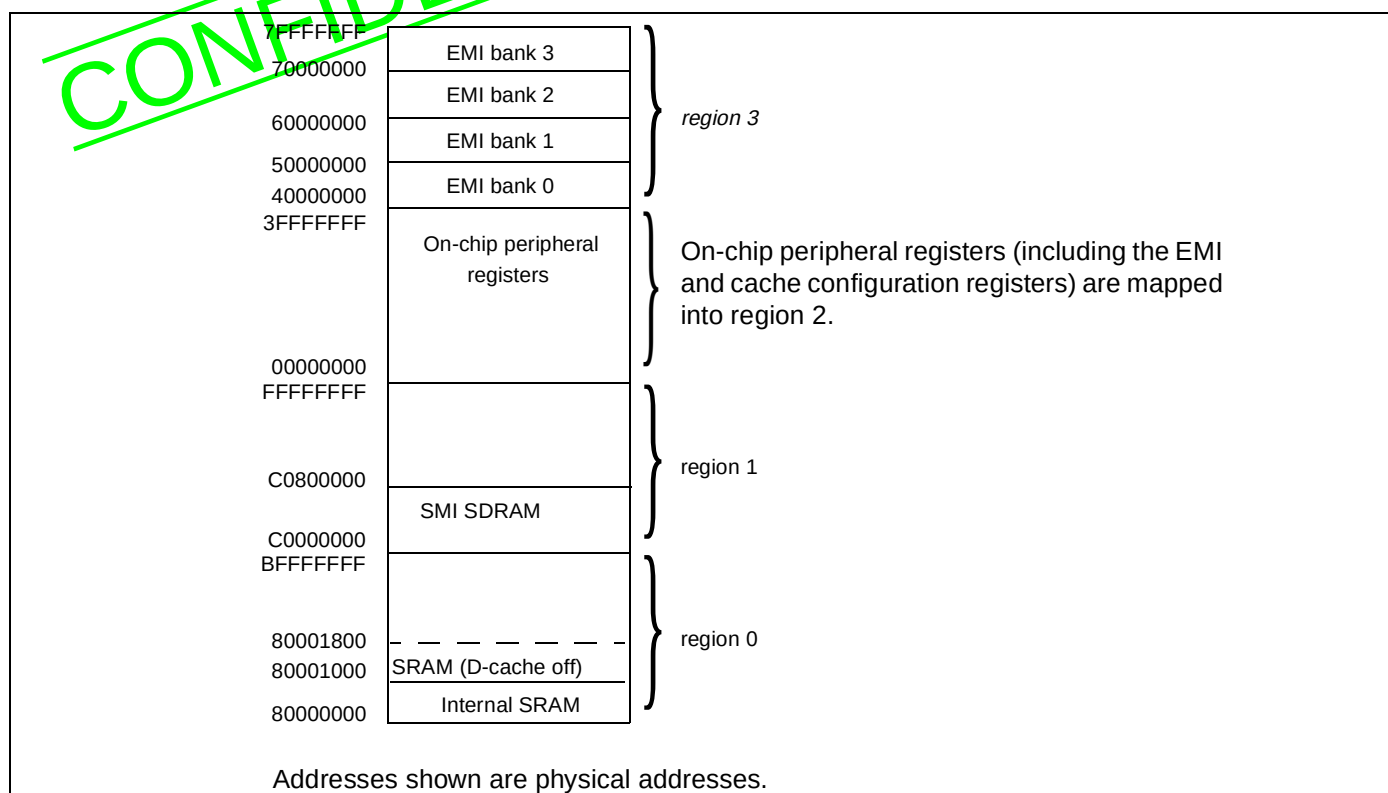


Figure 22 Memory allocation

The EMI can support three distinct types of memory, called device types; DRAM type with a multiplexed row and column address which is used to support fast page mode DRAM, SDRAM type with a multiplexed row and column address and SRAM or peripheral type which is used to support SRAM, peripherals and EPROM or Flash ROM.

EMI banks 0 and 1 support either type, while banks 2 and 3 only support SRAM or peripheral type. Support for word, byte and part-word addressing is provided.

As the banks are of a fixed size, range checking of addresses is not possible. This means that software tools must be aware of the physical external memory capacity.

In this chapter a *cycle* is one ST20 CPU clock cycle and a *phase* is one half of the duration of one ST20 CPU clock cycle.

Note The functions of the external memory interface pins change depending upon which device type the EMI bank has been configured for, either SDRAM, DRAM or SRAM/peripheral.
A signal name prefixed by **not** indicates active low.

11.2 Common pin functions

This section describes the functions of pins that do not change depending upon the device type (DRAM, SDRAM or SRAM/peripheral) of the EMI bank being accessed. The table is sorted, alphabetically, by pin name.

Pin	Function										
BootSource0-1	These two pins determine whether the device will boot from link or from ROM. When the <i>BootSource0-1</i> pins are both held low the STi5512 will boot from its link. If either or both pins are high, the STi5512 will boot from ROM as shown in the table below. Boot code is run from a slow external ROM placed in bank 3 (at the top of memory). The <i>BootSource0-1</i> pins also encode the size of bank 3. This overrides the value in the configuration registers for the PortSize for bank 3										
	<table><tr><th>BootSource1-0</th><th>Function</th></tr><tr><td>0:0</td><td>Boot from link. The STi5512 loads bootstrap down the OS-Link and executes from MemStart.</td></tr><tr><td>0:1</td><td>Boot from ROM. Port size of bank 3 hardwired to 32-bits.</td></tr><tr><td>1:0</td><td>Boot from ROM. Port size of bank 3 hardwired to 16-bits.</td></tr><tr><td>1:1</td><td>Boot from ROM. Port size of bank 3 hardwired to 8-bits.</td></tr></table>	BootSource1-0	Function	0:0	Boot from link. The STi5512 loads bootstrap down the OS-Link and executes from MemStart .	0:1	Boot from ROM. Port size of bank 3 hardwired to 32-bits.	1:0	Boot from ROM. Port size of bank 3 hardwired to 16-bits.	1:1	Boot from ROM. Port size of bank 3 hardwired to 8-bits.
	BootSource1-0	Function									
	0:0	Boot from link. The STi5512 loads bootstrap down the OS-Link and executes from MemStart .									
	0:1	Boot from ROM. Port size of bank 3 hardwired to 32-bits.									
	1:0	Boot from ROM. Port size of bank 3 hardwired to 16-bits.									
	1:1	Boot from ROM. Port size of bank 3 hardwired to 8-bits.									
If the STi5512 is set to boot from OS-Link or is being booted by the DCU then the bootstrap must execute from internal memory until the EMI has been configured. If this is not possible then the EMI must be completely configured using <i>poke</i> commands before loading the bootstrap into external memory and executing it.											
Note that the STi5512 will only boot from OS-Link if it is in OS-Link mode rather than DCU mode. This is determined by the value on the ErrorOut/TrigOut pin after reset. For more details see the pin list section.											
MemData0-31	The data bus transfers 32, 16 or 8-bit data items depending on the bus width configuration. The least significant bit of the data bus is always <i>MemData0</i> . The most significant bit varies with bus width. It will be <i>MemData31</i> for 32-bit data items, <i>MemData15</i> for 16-bit data items, and <i>MemData7</i> for 8-bit data items.										

Table 52 Common pin functions

Pin	Function																																
MemReq, MemGranted	Direct memory access (DMA) can be requested at any time by driving the synchronous MemReq signal high. If the device is booted in backwards compatible 5510 mode, only the address and data buses are tristated after the current memory access or refresh cycle terminates, with the strobes going to an inactive high state. If the device is booted in 5512 mode then the address and data buses are tristated along with all the EMI strobe pins. During the DMA transfer, strobes are left inactive for 5510 mode and tristate for 5512 mode. If a DMA is active for longer than one programmed refresh interval, then external logic becomes responsible for providing refresh. The <i>MemGranted</i> signal is set high to signal that the bus request has been granted. When the DMA transfer is complete the external device sets the <i>MemReq</i> signal low and the EMI will respond by setting the MemGranted signal low and starting any pending access from an internal device. The table below lists the EMI pin states while MemGranted is asserted. <table><tr><th rowspan="2">Pin name</th><th colspan="2">Pin state</th></tr><tr><th>5510 mode</th><th>5512 mode</th></tr><tr><td>MemAddr2-23</td><td>tristate</td><td>tristate</td></tr><tr><td>MemData0-31</td><td>tristate</td><td>tristate</td></tr><tr><td>notMemBE0-3</td><td>inactive high</td><td>tristate</td></tr><tr><td>notMemRAS0-3</td><td>inactive high</td><td>tristate</td></tr><tr><td>notMemCAS0-3</td><td>inactive high</td><td>tristate</td></tr><tr><td>notMemCSROM</td><td>inactive high</td><td>tristate</td></tr><tr><td>notMemOE</td><td>inactive high</td><td>tristate</td></tr><tr><td>MemRdnotWr</td><td>high</td><td>tristate</td></tr><tr><td>notMemCS2</td><td>inactive high</td><td>tristate</td></tr></table>	Pin name	Pin state		5510 mode	5512 mode	MemAddr2-23	tristate	tristate	MemData0-31	tristate	tristate	notMemBE0-3	inactive high	tristate	notMemRAS0-3	inactive high	tristate	notMemCAS0-3	inactive high	tristate	notMemCSROM	inactive high	tristate	notMemOE	inactive high	tristate	MemRdnotWr	high	tristate	notMemCS2	inactive high	tristate
Pin name	Pin state																																
	5510 mode	5512 mode																															
MemAddr2-23	tristate	tristate																															
MemData0-31	tristate	tristate																															
notMemBE0-3	inactive high	tristate																															
notMemRAS0-3	inactive high	tristate																															
notMemCAS0-3	inactive high	tristate																															
notMemCSROM	inactive high	tristate																															
notMemOE	inactive high	tristate																															
MemRdnotWr	high	tristate																															
notMemCS2	inactive high	tristate																															
ProcClockOut	This is a reference signal for external bus cycles which oscillates at the ST20 CPU clock frequency. The CPU speed is selected by the SpeedSelect0-1 pins. Normally, the 60MHz CPU speed will be selected, however, the 50MHz speed may be selected for backwards compatibility with the STi5510. The EMI is clocked at the same frequency as ProcClockOut. The main use of ProcClockOut is to clock external logic which may be generating an input into the STi5512 in order to synchronize the signal to the internal CPU clock. For example an EMI wait state generator circuit.																																

Table 52 Common pin functions

11.3 DRAM device type pin functions and access cycles

11.3.1 Pin descriptions

When an EMI bank (either bank 0 or 1) is programmed as DeviceType DRAM in the register EMI_ConfigData0, it sets all the other EMI ConfigData0-3 register bits for that bank to be in DRAM format. For this case the EMI pin functions (alphabetically ordered) are described in this section and the external access cycles are described in the next section.

Pin	Function																							
MemAddr2-23	The address bus is operated in multiplexed mode. When a bank is configured to contain DRAM, then the internally generated 32-bit address is multiplexed as row and column addresses through the external address bus.																							
MemRdnotWr	This signal indicates whether the current cycle is a read or a write cycle. During writes, the signal is asserted low at the beginning of the access (i.e. at the start of RAS _{Time} for a non page mode access) and de-asserted high at the end of the access (end of CAS _{Time}), unless the next access is a page mode write. If the access is a page mode read followed by a page mode write MemRdnotWr is asserted low at the start of the write CAS _{Time} , after the programmed bus release time. At all other times this signal is held high.																							
MemWait	The MemWait input pin is ignored for banks configured as DRAM.																							
notMemBE0-3	The EMI uses word addressing and four byte-enable strobes are provided. The use of the byte enable pins depends on the bus width. The 32-bit wide memory is defined as an array of 4 byte words with 30 address bits selecting a 4 byte word. Each byte of this array is addressable with the byte enable pins notMemBE0-3 selecting a byte within a word. The 16-bit wide memory is defined as an array of 2 byte words with 31 address bits selecting a 2 byte word and notMemBE0-1 selecting a byte within the word. The 8-bit wide memory is defined as an array of 1 byte words with 32 address bits selecting a word. For 16-bit and 8-bit wide memory, the lower order address bits (A1 and A0) are multiplexed onto the unused byte-enable pins to give an address bus 31 or 32 bits wide respectively. The function of the byte enables notMemBE0-3 for different bank size configurations is given below. Other bus masters must not drive the same data pins during a write. <table><tr><th rowspan="2">notMemBE3-0</th><th colspan="3">External port size</th></tr><tr><th>32-bit</th><th>16-bit</th><th>8-bit</th></tr><tr><td>notMemBE3</td><td>enables MemData24-31</td><td>becomes A1</td><td>becomes A1</td></tr><tr><td>notMemBE2</td><td>enables MemData16-23</td><td>undefined</td><td>becomes A0</td></tr><tr><td>notMemBE1</td><td>enables MemData8-15</td><td>enables MemData8-15</td><td>undefined</td></tr><tr><td>notMemBE0</td><td>enables MemData0-7</td><td>enables MemData0-7</td><td>enables MemData0-7</td></tr></table> For banks configured as DRAM, the notMemBE strobes have a fixed timing and are valid from the start of CAS_{Time} to one phase before the end of CAS_{Time}.	notMemBE3-0	External port size			32-bit	16-bit	8-bit	notMemBE3	enables MemData24-31	becomes A1	becomes A1	notMemBE2	enables MemData16-23	undefined	becomes A0	notMemBE1	enables MemData8-15	enables MemData8-15	undefined	notMemBE0	enables MemData0-7	enables MemData0-7	enables MemData0-7
notMemBE3-0	External port size																							
	32-bit	16-bit	8-bit																					
notMemBE3	enables MemData24-31	becomes A1	becomes A1																					
notMemBE2	enables MemData16-23	undefined	becomes A0																					
notMemBE1	enables MemData8-15	enables MemData8-15	undefined																					
notMemBE0	enables MemData0-7	enables MemData0-7	enables MemData0-7																					

Table 53 DRAM device type pin list

CONFIDENTIAL

Pin	Function																	
notMemCAS0-3	The four notMemCAS strobes are only used for banks 0 and 1 when programmed as DRAM device types. The CAS strobes can be programmed on a per bank basis to be in one of two modes: - Bank mode in which only one CAS strobe is used for the entire bank and sub-banks (if any). - Byte mode in which each CAS strobe is used as a byte decoded CAS strobe and can be used across both banks (and any sub-banks). The byte mode is used to support 16 or 32-bit wide DRAMs or DRAM modules. It provides multiple CAS strobes, one for each byte, and a single write signal to allow byte write operations. The alternative type DRAMs can be interfaced using bank mode. This mode has multiple write signals, one for each byte, and a single CAS to allow byte write operations on banks constructed with 1, 4, or 8-bit wide DRAMs. The selection of bank or byte mode can be independently programmed for banks 0 and 1. CAS strobes in bank mode If banks 0 and 1 are set to device type DRAM with bank mode selected then notMemCAS0 is the sole CAS strobe for bank 0 and notMemCAS2 is the sole CAS strobe for bank 1. Unused CAS strobes remain inactive during an access. This changes if only one bank of DRAM is programmed in the EMI, in either bank 0 or bank 1. In this case only notMemCAS0 is used for both banks, that is, notMemCAS0 will be used as the bank mode CAS strobe for the DRAM in bank 1. CAS strobes in byte mode For banks containing DRAM, which require byte decoded CAS strobes, one programmable CAS strobe is allocated to each byte. Each of the CAS strobes in this mode will have the timing programmed into the CAS timing configuration registers of the bank being accessed if they are active during that cycle. Byte mode CAS strobes are active during an access if the byte corresponding to the strobe is being accessed. During refresh cycles, all CAS strobes will go low at the start of the cycle and remain low until the end of the cycle. The table below shows how the CAS strobes are used in byte mode where the strobes are common to both banks and any sub-banks. Only the CAS strobes which enable the bytes being accessed will be active during an access cycle. <table><tr><th rowspan="2">CAS strobe (byte mode)</th><th colspan="2">Bank 0 and Bank 1</th></tr><tr><th>32-bit</th><th>16-bit</th></tr><tr><td>notMemCAS3</td><td>enables MemData24-31</td><td>Inactive</td></tr><tr><td>notMemCAS2</td><td>enables MemData16-23</td><td>Inactive</td></tr><tr><td>notMemCAS1</td><td>enables MemData8-15</td><td>enables MemData8-15</td></tr><tr><td>notMemCAS0</td><td>enables MemData0-7</td><td>enables MemData0-7</td></tr></table>	CAS strobe (byte mode)	Bank 0 and Bank 1		32-bit	16-bit	notMemCAS3	enables MemData24-31	Inactive	notMemCAS2	enables MemData16-23	Inactive	notMemCAS1	enables MemData8-15	enables MemData8-15	notMemCAS0	enables MemData0-7	enables MemData0-7
CAS strobe (byte mode)	Bank 0 and Bank 1																	
	32-bit	16-bit																
notMemCAS3	enables MemData24-31	Inactive																
notMemCAS2	enables MemData16-23	Inactive																
notMemCAS1	enables MemData8-15	enables MemData8-15																
notMemCAS0	enables MemData0-7	enables MemData0-7																

(continued on next page)

Table 53 DRAM device type pin list

Pin	Function																																																												
(continuing)	Mixing bank and byte mode																																																												
notMemCAS0-3	For full flexibility any permutation of bankwidth (16 / 32) and CAS mode (byte / bank) is supported for both banks 0 and 1. The following table gives a full listing of the active strobes for all such permutations. <table><tr><th>Bank Configuration</th><th>notMemCAS0</th><th>notMemCAS1</th><th>notMemCAS2</th><th>notMemCAS3</th></tr><tr><td colspan="5">***** One DRAM in bank 0 or 1 *****</td></tr><tr><td>bank mode</td><td>active</td><td>unused</td><td>unused</td><td>unused</td></tr><tr><td>byte mode, portsize 16 bit</td><td>active MemData0-7</td><td>active MemData8-15</td><td>unused</td><td>unused</td></tr><tr><td>byte mode, portsize 32 bit</td><td>active MemData0-7</td><td>active MemData8-15</td><td>active MemData16-23</td><td>active MemData24-31</td></tr><tr><td colspan="5">***** Two DRAMs in bank 0 and 1 *****</td></tr><tr><td>DRAM in bank 0 bank mode</td><td>active</td><td>unused</td><td>unused</td><td>unused</td></tr><tr><td>DRAM in bank 0 byte mode, portsize 16 bit</td><td>active MemData0-7</td><td>active MemData8-15</td><td>unused</td><td>unused</td></tr><tr><td>DRAM in bank 0 byte mode, portsize 32 bit</td><td>active MemData0-7</td><td>active MemData8-15</td><td>active MemData16-23</td><td>active MemData24-31</td></tr><tr><td>DRAM in bank 1 bank mode</td><td>unused</td><td>unused</td><td>active</td><td>unused</td></tr><tr><td>DRAM in bank 1 byte mode, portsize 16 bit</td><td>active MemData0-7</td><td>active MemData8-15</td><td>unused</td><td>unused</td></tr><tr><td>DRAM in bank 1 byte mode, portsize 32 bit</td><td>active MemData0-7</td><td>active MemData8-15</td><td>active MemData16-23</td><td>active MemData24-31</td></tr></table> For banks which do not contain DRAM the relevant notMemCAS pin is inactive.	Bank Configuration	notMemCAS0	notMemCAS1	notMemCAS2	notMemCAS3	***** One DRAM in bank 0 or 1 *****					bank mode	active	unused	unused	unused	byte mode, portsize 16 bit	active MemData0-7	active MemData8-15	unused	unused	byte mode, portsize 32 bit	active MemData0-7	active MemData8-15	active MemData16-23	active MemData24-31	***** Two DRAMs in bank 0 and 1 *****					DRAM in bank 0 bank mode	active	unused	unused	unused	DRAM in bank 0 byte mode, portsize 16 bit	active MemData0-7	active MemData8-15	unused	unused	DRAM in bank 0 byte mode, portsize 32 bit	active MemData0-7	active MemData8-15	active MemData16-23	active MemData24-31	DRAM in bank 1 bank mode	unused	unused	active	unused	DRAM in bank 1 byte mode, portsize 16 bit	active MemData0-7	active MemData8-15	unused	unused	DRAM in bank 1 byte mode, portsize 32 bit	active MemData0-7	active MemData8-15	active MemData16-23	active MemData24-31
Bank Configuration	notMemCAS0	notMemCAS1	notMemCAS2	notMemCAS3																																																									
***** One DRAM in bank 0 or 1 *****																																																													
bank mode	active	unused	unused	unused																																																									
byte mode, portsize 16 bit	active MemData0-7	active MemData8-15	unused	unused																																																									
byte mode, portsize 32 bit	active MemData0-7	active MemData8-15	active MemData16-23	active MemData24-31																																																									
***** Two DRAMs in bank 0 and 1 *****																																																													
DRAM in bank 0 bank mode	active	unused	unused	unused																																																									
DRAM in bank 0 byte mode, portsize 16 bit	active MemData0-7	active MemData8-15	unused	unused																																																									
DRAM in bank 0 byte mode, portsize 32 bit	active MemData0-7	active MemData8-15	active MemData16-23	active MemData24-31																																																									
DRAM in bank 1 bank mode	unused	unused	active	unused																																																									
DRAM in bank 1 byte mode, portsize 16 bit	active MemData0-7	active MemData8-15	unused	unused																																																									
DRAM in bank 1 byte mode, portsize 32 bit	active MemData0-7	active MemData8-15	active MemData16-23	active MemData24-31																																																									
notMemOE	If the access is to a bank configured as DRAM then the notMemOE strobe is active only during a read access when it is asserted low CAS₀Time after the start of CASTime (on the programmed falling edge of the CAS strobe of the bank), and de-asserted high at the end of CASTime.																																																												

Table 53 DRAM device type pin list

CONFIDENTIAL

Pin	Function																					
notMemRAS0-3	EMI banks 0 and 1 are both capable of supporting DRAM devices. Furthermore, each of these banks may be sub-decoded into two sub-banks. The strobes notMemRAS0-3 are used as the DRAM RAS strobes to these banks or sub-banks. The four signals do not necessarily correspond to individual banks. For example, one bank may be sub decoded. The function of the RAS pins changes depending if one or two banks of DRAM have been programmed. This is shown in the two tables below. <table><tr><th>Sub-Banks</th><th>notMemRAS description</th></tr><tr><td>0</td><td>only notMemRAS0 strobe is active for DRAM in bank 0 or 1</td></tr><tr><td>2</td><td>lower sub-bank RAS strobe is notMemRAS0 upper sub-bank RAS strobe is notMemRAS1</td></tr></table> <table><tr><th>Sub-Banks in Bank 0</th><th>Sub-Banks in Bank 1</th><th>notMemRAS Description</th></tr><tr><td>0</td><td>0</td><td>notMemRAS0 strobe is active for bank 0 notMemRAS2 strobe is active for bank 1</td></tr><tr><td>2</td><td>0</td><td>notMemRAS0 strobe is active for lower sub-bank of bank 0 notMemRAS1 strobe is active for upper sub-bank of bank 0 notMemRAS2 strobe is active for bank 1</td></tr><tr><td>0</td><td>2</td><td>notMemRAS0 strobe is active for bank 0 notMemRAS2 strobe is active for lower sub-bank of bank 1 notMemRAS3 strobe is active for upper sub-bank of bank 1</td></tr><tr><td>2</td><td>2</td><td>notMemRAS0 strobe is active for lower sub-bank of bank 0 notMemRAS1 strobe is active for upper sub-bank of bank 0 notMemRAS2 strobe is active for lower sub-bank of bank 1 notMemRAS3 strobe is active for upper sub-bank of bank 1</td></tr></table>	Sub-Banks	notMemRAS description	0	only notMemRAS0 strobe is active for DRAM in bank 0 or 1	2	lower sub-bank RAS strobe is notMemRAS0 upper sub-bank RAS strobe is notMemRAS1	Sub-Banks in Bank 0	Sub-Banks in Bank 1	notMemRAS Description	0	0	notMemRAS0 strobe is active for bank 0 notMemRAS2 strobe is active for bank 1	2	0	notMemRAS0 strobe is active for lower sub-bank of bank 0 notMemRAS1 strobe is active for upper sub-bank of bank 0 notMemRAS2 strobe is active for bank 1	0	2	notMemRAS0 strobe is active for bank 0 notMemRAS2 strobe is active for lower sub-bank of bank 1 notMemRAS3 strobe is active for upper sub-bank of bank 1	2	2	notMemRAS0 strobe is active for lower sub-bank of bank 0 notMemRAS1 strobe is active for upper sub-bank of bank 0 notMemRAS2 strobe is active for lower sub-bank of bank 1 notMemRAS3 strobe is active for upper sub-bank of bank 1
Sub-Banks	notMemRAS description																					
0	only notMemRAS0 strobe is active for DRAM in bank 0 or 1																					
2	lower sub-bank RAS strobe is notMemRAS0 upper sub-bank RAS strobe is notMemRAS1																					
Sub-Banks in Bank 0	Sub-Banks in Bank 1	notMemRAS Description																				
0	0	notMemRAS0 strobe is active for bank 0 notMemRAS2 strobe is active for bank 1																				
2	0	notMemRAS0 strobe is active for lower sub-bank of bank 0 notMemRAS1 strobe is active for upper sub-bank of bank 0 notMemRAS2 strobe is active for bank 1																				
0	2	notMemRAS0 strobe is active for bank 0 notMemRAS2 strobe is active for lower sub-bank of bank 1 notMemRAS3 strobe is active for upper sub-bank of bank 1																				
2	2	notMemRAS0 strobe is active for lower sub-bank of bank 0 notMemRAS1 strobe is active for upper sub-bank of bank 0 notMemRAS2 strobe is active for lower sub-bank of bank 1 notMemRAS3 strobe is active for upper sub-bank of bank 1																				

Table 53 DRAM device type pin list

11.3.2 Access cycles

DRAM access cycles are supported in banks 0 and 1 only when these are set to device type DRAM.

A DRAM memory access cycle consists of a number of defined periods or times, as shown in the figure below. All of the named times shown in this diagram together with other parameters such as RAS address shift and page size are programmable to suit a wide variety of DRAM types.

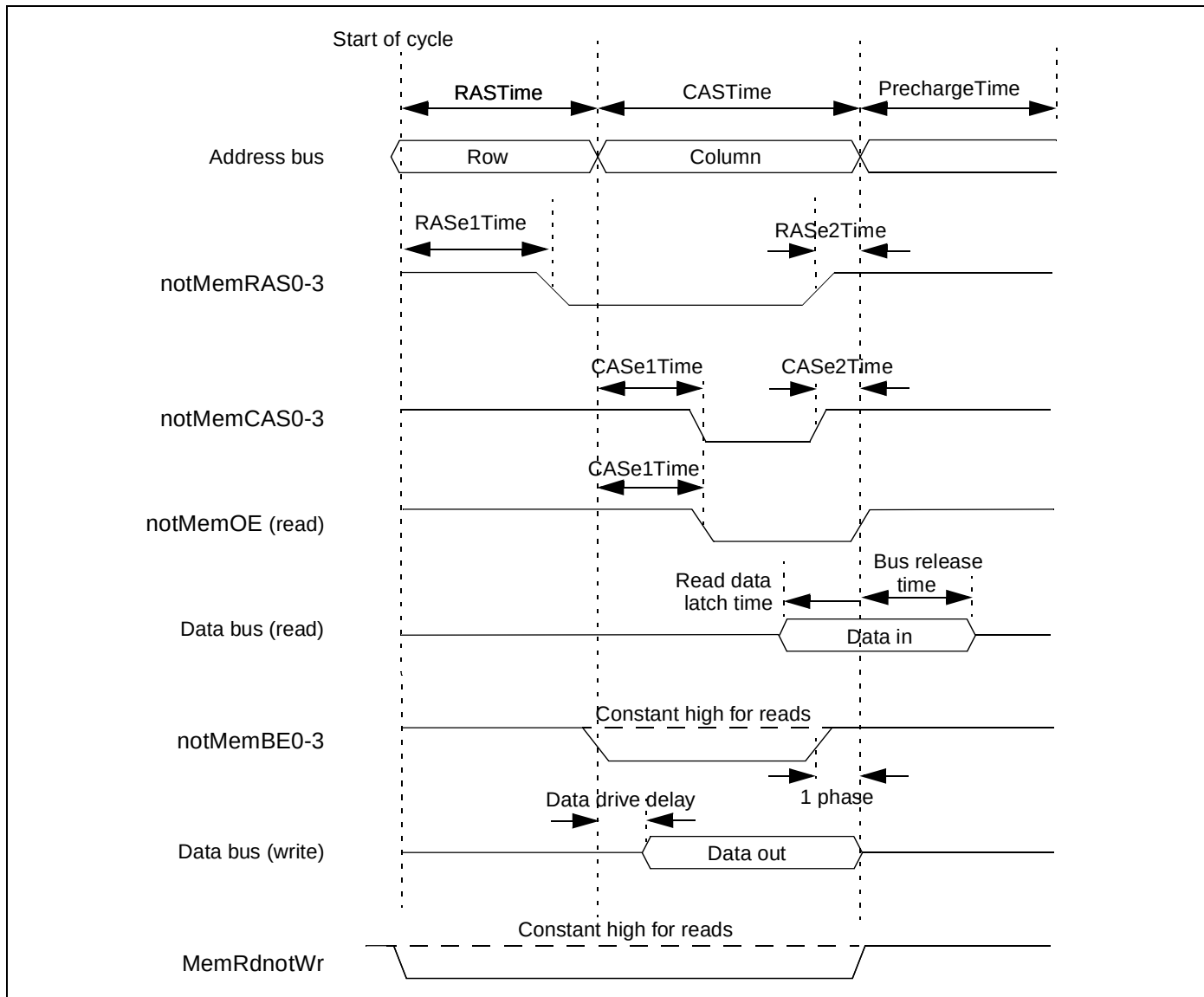


Figure 23 DRAM memory cycle

RASTime and CASTime are consecutive and CASTime can be followed by concurrent PrechargeTime and BusReleaseTime.

For device type DRAM, RASTime is used for RAS address latching, CASTime for CAS address latching and the PrechargeTime/BusReleaseTime for RAS precharge and output driver tristate times. For consecutive access to the same bank of DRAM, RASTime will only occur when there is a page miss. The next access will not commence until the PrechargeTime for a previous access to the same bank has been completed. During the RASTime a transition can only be programmed on the RAS strobes.

During the CASTime the CAS strobes and either the byte-enable or notMemOE strobes are active. The address is output on the address bus without being RAS shifted. Write data is valid during CASTime. Read data is latched into the interface at the point defined by the LatchPoint bit in the register EMI_ConfigData3 for the bank being accessed.

The PrechargeTime and BusReleaseTime commence concurrently at the end of the CASTime. A PrechargeTime will occur and the active notMemRAS strobe will be taken high if:

- the next access is to the same bank but to a different row address.
- the next access is to a different bank.

The BusReleaseTime running concurrently with the PrechargeTime will occur if:

- the current cycle is a read and the next cycle is a write.
- the current cycle is a read and the next cycle is a read from a different bank.

The BusReleaseTime is provided to allow an accessed device to float to a high impedance state.

Page mode

DRAM pages are delineated using the RASBits configuration parameter. These bits are used as an address mask for comparison with the previous dram address. If an access is requested by an internal subsystem of the STi5512 to a DRAM bank while a DRAM access is in progress, the new address is compared to the current access address. If the row addresses are the same, the access may proceed as a page mode access. There is no specific configuration bit to select pagemode DRAM. If all the RASBits are set to 0, then no page hits will be caused and normal DRAM RAS/CAS cycles will always be produced.

A page mode access does not include the RASTime period. The notMemRAS strobe is not taken high before commencing the page mode access. If the current access is a read and the page mode access is due to be a write, a BusReleaseTime is inserted as shown in figure below. The notMemRAS strobe is held low during this period.

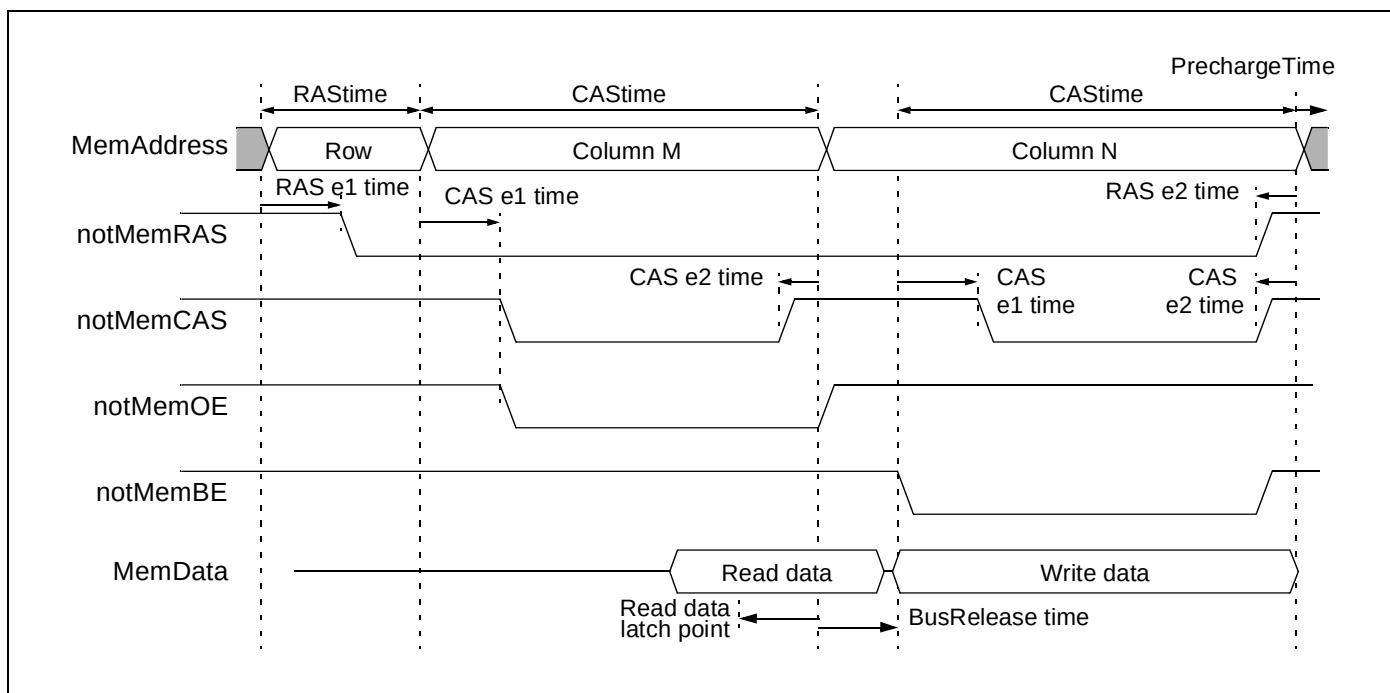


Figure 24 Read followed by page mode write

When setting the RASBits, care must be taken to consider the port size. Also, if the bank has been sub-decoded, the sub-bank selection address bits must be included in the comparison, therefore, the RASBits corresponding to these addresses must be set.

For example, if the DRAM bank is composed of 2 x 256k * 16 devices, the sub-bank selection address bit is A₁₉, so the RASBits corresponding to address bits A₁₉-A₁₀ must be set.

When page mode is active, the RASe2time must be programmed to zero.

SubBank	PortSize	SubBank Size	SubBank selection address	RAS strobe selection	
				1 DRAM Bank (0 or 1)	2 DRAMs Banks (0 and 1)
2	8 bit	256K	Address<18>	lower sub-bank = notMemRAS0	Bank 0: lower sub-bank = notMemRAS0 upper sub-bank = notMemRAS1 Bank 1: lower sub-bank = notMemRAS2 upper sub-bank = notMemRAS3
		1M	Address<20>	upper sub-bank = notMemRAS1	
		4M	Address<22>		
		16M	Address<24>		
	16 bit	256K	Address<19>		
		1M	Address<21>		
		4M	Address<23>		
		16M	Address<25>		
	32 bit	256K	Address<20>		
		1M	Address<22>		
		4M	Address<24>		
		16M	Address<26>		

Table 54 Address Sub-Bank decoding for only 1 DRAM Bank (0 or 1) and for 2 DRAM Banks (0 and 1)

Refresh

DRAM banks are periodically refreshed at intervals specified by the RefreshInterval configuration parameter.

The notMemCAS strobe(s) is taken low at the beginning of the refresh time. The position of the RAS falling edge (RASedge) is programmable and the minimum width of the CAS pulse is the sum of the RAS_{Time} and CAS_{Time} values specified for random access. If there is more than one bank of DRAM the refresh configuration will then be taken from the lowest numbered bank configured as DRAM.

All sub banks are refreshed in the same access and a cycle is inserted between each bank and/or sub-bank in order to spread current peaks. If no DRAM has been programmed for a bank then no transitions occur on the relevant RAS or

CAS strobes and all unused RAS and CAS strobes (i.e. strobes not used due to the choice of bank/byte mode, sub-banks and bankwidth) will remain inactive during a refresh cycle.

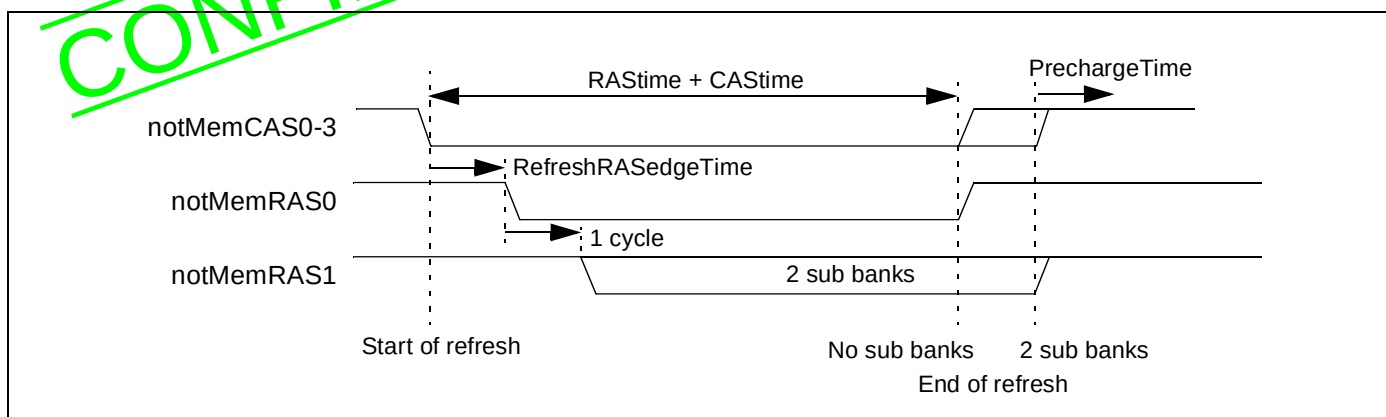


Figure 25 Generic refresh access for one DRAM bank

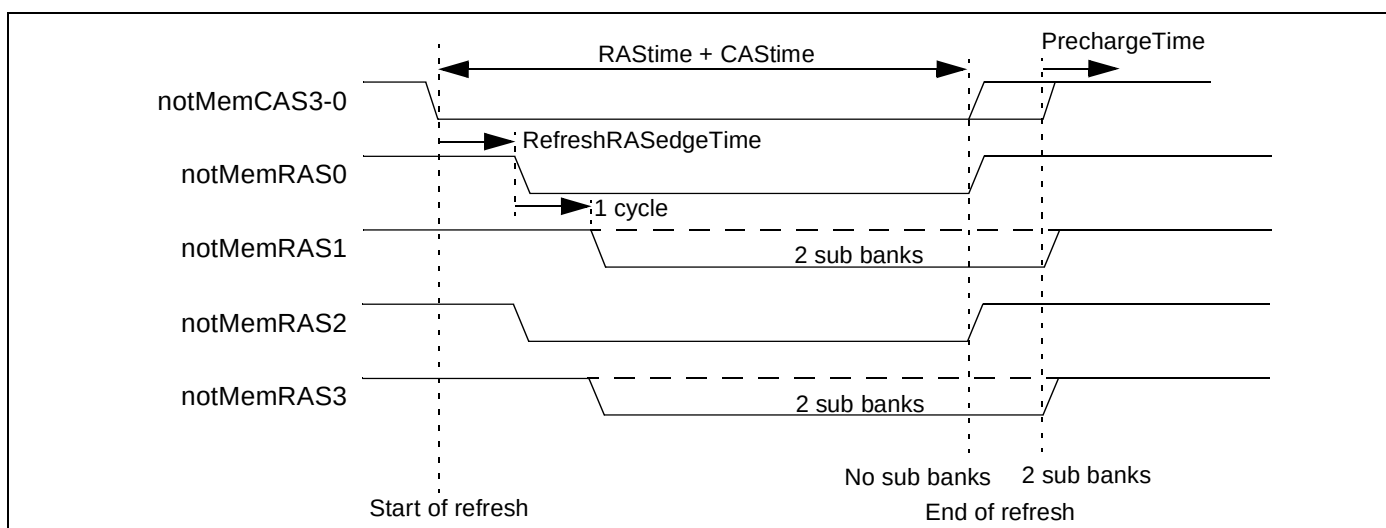


Figure 26 Generic refresh access for two DRAM banks

Name	Programmable value
PrechargeTime	1 - 8 cycles
RefreshInterval	(1 - 16) * 128 cycles
RefreshRASedgeTime	1 or 2 cycles after start of refresh

Table 55 Refresh parameters

The EMI ensures that notMemCAS and notMemRAS are both high for the required time before every refresh cycle by inserting a PrechargeTime in the last bank being accessed and ensuring all PrechargeTimes are complete.

Refreshes cannot take place until after a DRAMinitialize command in the ConfigCommand register is performed.

11.4 SRAM/peripheral device type pin functions and access cycles

These are described in the following sub-sections. The second sub-section describes Bank-width based address shifting and boot-bank address shifting.

11.4.1 Pin descriptions

When an EMI bank (either bank 0,1,2 or 3) is programmed as device type SRAM/peripheral in the EMI_ConfigData0 register it sets all the other EMI ConfigData0-3 register bits for that bank to be in SRAM/peripheral format. The following table describes the pin functions for this device type in alphabetical order.

Pin	Function
MemAddr2-23	The address bus is operated in non-multiplexed mode. When a bank is configured to contain SRAM then the internally generated 32-bit address is output through the external address bus. Internal addresses 0-1 are not output on the address pins unless the data bus width address bus shifting feature is used for 8 or 16 bit wide data banks. This feature is only allowed in SRAM/peripheral mode.
MemRdnotWr	This signal indicates whether the current cycle is a read or a write cycle. During writes, the signal is asserted low at the beginning of the access (at the start of CStime) and de-asserted high at the end of the access (end of CStime). At all other times this signal must be held high.

Table 56 SRAM/peripheral device type pin list

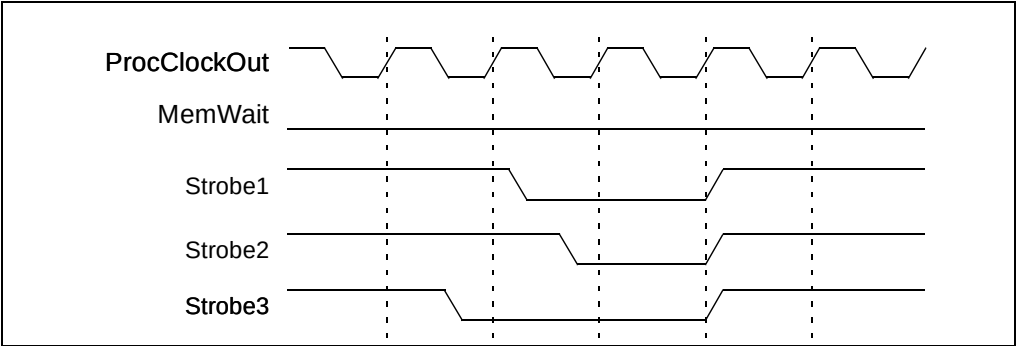
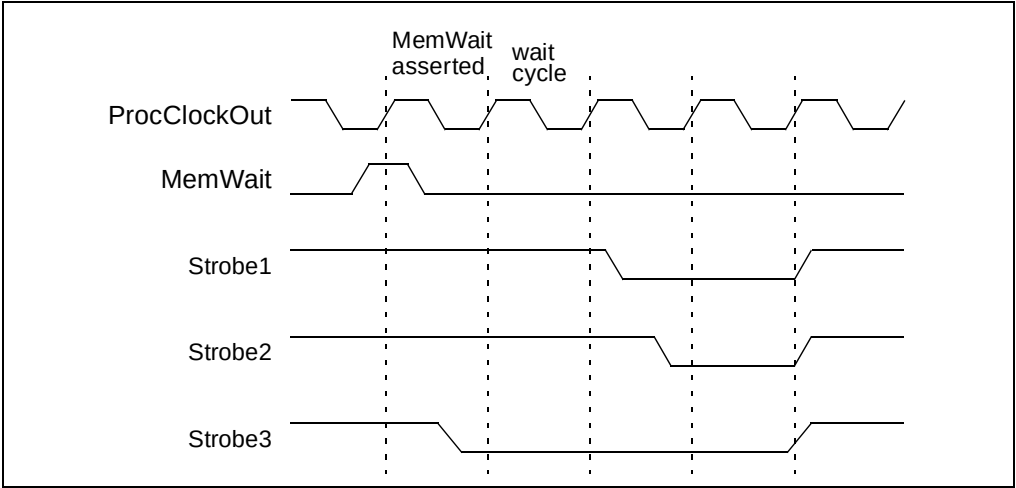
Pin	Function
MemWait	The MemWait pin extends external cycles by enabling variable access times, for example, for shared SRAM memory access. The MemWait signal can be treated as asynchronous or synchronous to the clock ProcClockOut, depending on the state of bit 5 in the register EMI_ConfigPadLogic. Setting this bit high selects MemWait to be synchronous and setting it low selects it to be asynchronous. Wait states are generated by taking MemWait high. The strobes retain their states during the cycle after the one in which MemWait was asserted until it is de-asserted. When it is de-asserted the access continues as programmed by the EMI configuration. If MemWait is synchronous then wait states can be inserted at precise times during the access. If it is asynchronous then it does not require any external synchronization but cannot accurately insert wait states during an access. The selection of the asynchronous or synchronous MemWait signal is the same for all banks. MemWait is only sampled during SRAM/peripheral accesses. The following description and diagrams are for a synchronous configuration. MemWait has the effect of freezing the states of the strobes for the duration of the cycles in which it was sampled high. Any strobe transitions occurring on the sampling edge or on the falling edge immediately after this will not be inhibited, however, transitions on the rising and falling edges of the following cycle will not occur. Figure 27 and Figure 28 show the extension of the external memory cycle and the delay of the strobe transitions.  Figure 27 Strobe activity without MemWait  Figure 28 Strobe activity with MemWai The asynchronous MemWait does not need to meet setup and hold times of the ProcClockOut signal rising edges. It uses an extra clock edge to synchronize the signal before it is sampled in the EMI. Apart from this extra cycle of latency the response to the two types of MemWait is the same.

Table 56 SRAM/peripheral device type pin list



CONFIDENTIAL

Pin	Function																							
notMemBE0-3	The EMI uses word addressing and four byte-enable strobes. Use of the byte enable pins depends on the bus width as follows: 32-bit wide memory is defined as an array of 4 byte words with 30 address bits selecting a 4 byte word. Each byte of this array is addressable with the byte enable pins notMemBE0-3 selecting a byte within a word. 16-bit wide memory is defined as an array of 2 byte words with 31 address bits selecting a 2 byte word and notMemBE0-1 selecting a byte within the word. 8-bit wide memory is defined as an array of 1 byte words with 32 address bits selecting a word. For 16-bit and 8-bit wide memory, the lower order address bits (A1 and A0) are multiplexed onto the unused byte-enable pins. This gives an address bus 31 bits wide using A1 and 32 bits wide using A0. Though for SRAM/peripheral banks the preferred method of obtaining address bits A1 or A1 and A0 is to use the data port width address shifting feature, as described later in this section. The function of the byte enables notMemBE0-3 for different bank size configurations is given in the table below. Other bus masters must not drive these same data pins during a write. <table><tr><th rowspan="2">Pin</th><th colspan="3">External port size</th></tr><tr><th>32-bit</th><th>16-bit</th><th>8-bit</th></tr><tr><td>notMemBE3</td><td>enables MemData24-31</td><td>becomes A1</td><td>becomes A1</td></tr><tr><td>notMemBE2</td><td>enables MemData16-23</td><td>undefined</td><td>becomes A0</td></tr><tr><td>notMemBE1</td><td>enables MemData8-15</td><td>enables MemData8-15</td><td>undefined</td></tr><tr><td>notMemBE0</td><td>enables MemData0-7</td><td>enables MemData0-7</td><td>enables MemData0-7</td></tr></table> For banks configured as SRAM, the notMemBE strobe pins are used as data enable strobes and all have the same timing as each other. The strobes may be configured to be active on read cycles, write cycles, or both read and write cycles. The timing of the rising and falling edges of the strobes can be programmed independently for read and write accesses by the registers EMI_ConfigData1 and EMI_ConfigData2 for that bank.	Pin	External port size			32-bit	16-bit	8-bit	notMemBE3	enables MemData24-31	becomes A1	becomes A1	notMemBE2	enables MemData16-23	undefined	becomes A0	notMemBE1	enables MemData8-15	enables MemData8-15	undefined	notMemBE0	enables MemData0-7	enables MemData0-7	enables MemData0-7
Pin	External port size																							
	32-bit	16-bit	8-bit																					
notMemBE3	enables MemData24-31	becomes A1	becomes A1																					
notMemBE2	enables MemData16-23	undefined	becomes A0																					
notMemBE1	enables MemData8-15	enables MemData8-15	undefined																					
notMemBE0	enables MemData0-7	enables MemData0-7	enables MemData0-7																					
notMemCAS0-3	For banks configured as SRAM/peripheral, the notMemCAS0-3 strobe pins are inactive.																							
notMemCS2	This strobe acts as the chip select strobe (notMemCS signal) for bank2 of the EMI. Bank2 can only be configured as SRAM/peripheral.																							
notMemCS-ROM	This strobe acts as the chip select strobe (notMemCS signal) for bank3 of the EMI. Bank3 can only be configured as SRAM/peripheral. This bank usually, though not necessarily, contains the system ROM/FLASH.																							
notMemOE	This strobe is fully programmable and will behave according to the values in the registers EMI_ConfigData for that bank. It can be configured to be active on read, write, or read and write accesses. The timing of the rising and falling edge of the strobe can be programmed independently for read and write accesses by the EMI_ConfigData1 and EMI_ConfigData2 registers for the bank.																							
notMemRAS0-3	The four RAS strobe pins have a different function for a bank programmed to be SRAM/peripheral from that for DRAM. If banks 0 or 1 are programmed as device type SRAM/peripheral, then the notMemRAS strobes will be used as chip selects for those banks as shown in the table below <table><tr><th>Pin</th><th>Function for Bank in SRAM/Peripheral Mode</th></tr><tr><td>notMemRAS0</td><td>used as chip select (notMemCS signal) for bank 0, not used in banks 1, 2 and 3</td></tr><tr><td>notMemRAS1</td><td>not used</td></tr><tr><td>notMemRAS2</td><td>used as chip select (notMemCS signal) for bank 1, not used in banks 0, 2 and 3</td></tr><tr><td>notMemRAS3</td><td>not used</td></tr></table>	Pin	Function for Bank in SRAM/Peripheral Mode	notMemRAS0	used as chip select (notMemCS signal) for bank 0, not used in banks 1, 2 and 3	notMemRAS1	not used	notMemRAS2	used as chip select (notMemCS signal) for bank 1, not used in banks 0, 2 and 3	notMemRAS3	not used													
Pin	Function for Bank in SRAM/Peripheral Mode																							
notMemRAS0	used as chip select (notMemCS signal) for bank 0, not used in banks 1, 2 and 3																							
notMemRAS1	not used																							
notMemRAS2	used as chip select (notMemCS signal) for bank 1, not used in banks 0, 2 and 3																							
notMemRAS3	not used																							

Table 56 SRAM/peripheral device type pin list

11.4.2 Address shifting

Bank-width based address shifting

The shifts can only be enabled in banks programmed to the device type SRAM or peripheral,

Address shifting can be enabled on a per bank basis to allow population options on boards with various bank data bus widths to be handled more easily. The shift amount is dependent on the width of the bank. Shifting is enabled or disabled by 4 bits, one for each bank of the EMI padlogic register ConfigPadLogic0-3, where bit 0 refers to bank 0 and so on. The table below shows the addresses presented on the MemAddr2-23 pins for different configurations. The addresses presented on the notMemBE0-1 signals for 16 or 8 bit banks are not affected by the shifting.

Bank device type	configpadlogic0-3	Bank Data Portsize	MemAddr2-23
DRAM/SDRAM	0 = shift disabled 1 = not allowed	X	Address2-23 during CAS _{time}
SRAM or Peripheral	0 = shift disabled 1 = shift enabled	32 bit	Address2-23
		16 bit	Address1-22
		8 bit	Address0-21

Table 57 DRAM, SDRAM and SRAM/peripheral address-shifting

Bank-width based boot-bank address shifting

When booting the device from ROM, the address-shift feature cannot be used for the ROM bank (bank 3) because the ConfigPadLogic register cannot be written to from the code in the ROM before the first access from the ROM after reset. Therefore, a hardware override to the ConfigPadLogic register bit for bank 3 of the EMI is provided for booting the device in 5512 mode.

Note The **STi5512** can support a “**5512 mode**” or a “**5510 mode**” which is entered when the chip comes out of reset. The mode is determined by sampling the YC0 pin of the digital video output port. If this is low (by holding the pin low with a 10KΩ pull down resistor) then **5512 mode** is selected, high means **5510 mode**.

The 5512 mode controls whether the automatic boot bank (bank 3) address shifting is active or not.

Automatic boot bank (bank 3) address shifting is active only in 5512 mode. In this mode the ConfigPadLogic register bit for enabling address shifting (bit 3) on bank 3 is ignored, and is overridden by the setting of the BootSource0-1 pins. This means that the address shifting, for example, on a 16 bit wide FLASH on bank 3 can be enabled by selecting boot from 16 bit ROM on the BootSource pins, and does not need any EMI configuration to enable shifting.

The table below shows how the address shifting is controlled for bank 3 only, in 5512 mode.

BootSource<1:0>	shifted Memaddr2-23> pins
01 (32 bit)	mem_address<2:23>
10 (16 bit)	mem_address<1:22>
11 (8 bit)	mem_address<0:21>

Table 58 Boot-bank address-shifting in 5512 mode for bank 3 only

11.4.3 Access cycles

A generic peripheral (e.g. SRAM, EPROM, FLASH, etc.) type of access is provided which is suitable for direct interfacing to a wide variety of SRAM, ROM, FLASH and other peripheral devices. No internal sub-decoding is provided with banks in this configuration. All of the named times shown in the figure below together with other parameters such as *bank size* and *bank size dependent shifts* can be programmed for a wide variety of device types.

For details of the configuration of the EMI see [EMI configuration on page 94](#).

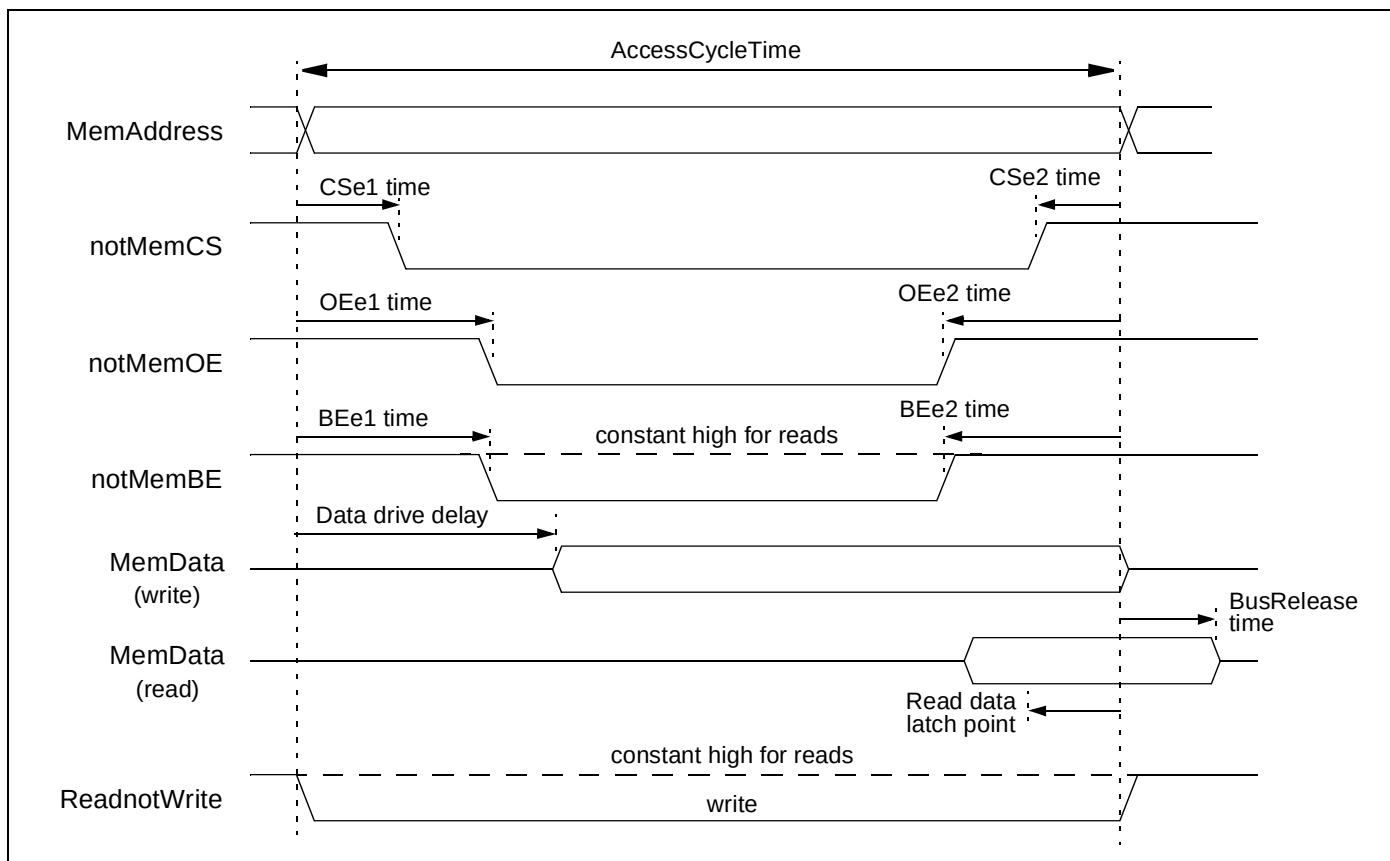


Figure 29 Generic peripheral access

11.5 SDRAM device type pin functions and access cycles

These are described in the following two sections. The first section describing the pins also contains descriptions of Multiple SDRAM banks and sub-bank decoding.

11.5.1 Pin descriptions

For the device type SDRAM the EMI pin functions are listed alphabetically in the following table.

Pin	Function
MemAddr2-23	The address bus is operated in multiplexed mode. When a bank is configured for SDRAM, then the internally generated 32-bit address is multiplexed as row and column addresses through the external address bus. MemAddr2-23 is driven with the ROW address during an activate command and the COLUMN address (the full address bus without being shifted) during a read or write operation. Two address lines have a special meaning to the SDRAM device: - A10 address line is used as a mode bit during precharge, read and write commands. It is sometimes called the auto precharge (AP) signal. - The top address line on the memory device is called the bank select (BS) and must be consistent for opening and accessing the memory page. As the top address bit changes depending on memory size, all of the address lines above AP hold their row address throughout read and write commands. The actual address signal used for the mode selection will be controlled using the bank data bus width PortSize EMI configuration register settings, because the address lines effectively shift as the bus width alters.
MemData0-31	The function of the data bus pins is the same as for the common MemData pin function described before, but the timing is somewhat different, due to the burst nature of the SDRAM. For writes, data is driven for each cycle of the burst access, either immediately after start of access cycle or one phase later (DataDriveDelay configuration parameter). For reads, the data will be latched for each cycle of the read a number of cycles after the command is sent to the SDRAM (CASlatency parameter).
MemRdnotWr	This pin is used as the ReadnotWrite (not_we) signal and indicates that the current cycle is a read cycle and is normally high. It is asserted low for the precharge and write commands. Additionally it will be low for the SDRAM mode register initialization cycle.
notMemBE0-3	The byte enable strobe pins are shared between all banks and are used by banks programmed for SDRAM as the DQM strobe signals. They are asserted low during read and write accesses to enable which bytes/ words are accessed. DQM is a multiple function signal defined as data mask for both reads and writes. During reads DQM performs synchronous output enable. During writes DQM performs write data masking. The DQM latency is different for reads than it is for writes. For reads, DQM latency is defined as the difference between the clock when DQM is asserted and the clock when the output bus has been forced to High-Z and its value is always 2 clock cycles. For writes, DQM latency is defined as the difference between the clock when DQM is asserted and the clock when the write input data is inhibited and its value is always 0. notMemBE[i] will only be active if the corresponding byte is to be accessed. For single byte accesses only one will be active. The behavior of these signals is dependent on the data port width of the bank, programmed by the PortSize configuration bits.
notMemCAS0-3	Only notMemCAS0 and notMemCAS2 are used as CAS strobes for SDRAM banks. notMemCAS0 is used when there is only one bank of SDRAM programmed in the EMI, in either bank 0 or 1. notMemCAS2 is used as the CAS strobe for bank 1 when there are two banks of SDRAM programmed in the EMI. These strobes will be low in a read, write, refresh and SDRAM mode register initialization cycle. notMemCAS1 is used as the SDRAM clock output. This must be enabled by programming bit 10 of the EMI_ConfigPadlogic register to be 1. notMemCAS3 is not used for SDRAM banks.

Table 59 SDRAM device type pin list

CONFIDENTIAL

Pin	Function															
notMemRAS0-3	These pins will be low for a bank activate, precharge, refresh cycle and SDRAM mode register initialization cycle. notMemRAS0 will be activated for SDRAM accesses in EMI bank 0 (or in EMI bank 1 when there is only one SDRAM device programmed in the EMI) and notMemRAS2 will be activated for accesses in bank 1 if EMI bank 0 is also configured as SDRAM device type. notMemRAS0 is used as the SDRAM sub-bank chip select for either bank 0 or 1 when there is only 1 bank of SDRAM programmed in the EMI, or as the sub-bank chip select for bank 0, when two banks of SDRAM are programmed in the EMI. In the latter case of 2 SDRAM banks, it is not possible to sub-decode bank 1, only bank 0 can be sub-decoded. This is summarized in the table below. <table><tr><th>RAS strobe</th><th>Function for 1 bank of SDRAM in EMI</th><th>Function for 2 banks of SDRAM in EMI</th></tr><tr><td>notMemRAS0</td><td>RAS strobe for SDRAM in bank 0 or 1</td><td>RAS strobe for SDRAM in bank 0</td></tr><tr><td>notMemRAS1</td><td>SDRAM chip select for highest sub-bank of bank 0 or 1</td><td>SDRAM chip select for highest sub-bank of bank 0</td></tr><tr><td>notMemRAS2</td><td>not used</td><td>RAS strobe for SDRAM in bank 1</td></tr><tr><td>notMemRAS3</td><td>not used</td><td>SDRAM chip select for bank 1</td></tr></table>	RAS strobe	Function for 1 bank of SDRAM in EMI	Function for 2 banks of SDRAM in EMI	notMemRAS0	RAS strobe for SDRAM in bank 0 or 1	RAS strobe for SDRAM in bank 0	notMemRAS1	SDRAM chip select for highest sub-bank of bank 0 or 1	SDRAM chip select for highest sub-bank of bank 0	notMemRAS2	not used	RAS strobe for SDRAM in bank 1	notMemRAS3	not used	SDRAM chip select for bank 1
RAS strobe	Function for 1 bank of SDRAM in EMI	Function for 2 banks of SDRAM in EMI														
notMemRAS0	RAS strobe for SDRAM in bank 0 or 1	RAS strobe for SDRAM in bank 0														
notMemRAS1	SDRAM chip select for highest sub-bank of bank 0 or 1	SDRAM chip select for highest sub-bank of bank 0														
notMemRAS2	not used	RAS strobe for SDRAM in bank 1														
notMemRAS3	not used	SDRAM chip select for bank 1														
notSDRAMCS0	This pin is the dedicated chip select signal for SDRAM banks. It's function depends on whether one or two banks of SDRAM have been programmed in the EMI. It is the chip select or lower sub-bank chip select for bank 0 as described below.															

Table 59 SDRAM device type pin list

Multiple SDRAM banks and sub-bank decoding

Up to two banks of SDRAM can be used in one EMI bank if sub-decoding is used. No sub-decoding is allowed in bank 1 if two banks (0 and 1) of SDRAM are programmed in the EMI. However, sub-decoding is allowed if there is only one bank of SDRAM programmed in the EMI. Banks are selected by the notSDRAMCS chip select signals.

The table below summarizes how notSDRAMCS is used when one of banks 0 and 1 are SDRAM and when both banks are SDRAM.

1 SDRAM bank	SubBanks		notSDRAMCS signal
	0		only notSDRAMCS0 is used
	2		lower sub-bank: notSDRAMCS0 is used upper sub-bank: notMemRAS1 is used
2 SDRAM banks	SubBanks in bank 0	SubBanks in bank 1	
	0	0	notSDRAMCS0 is active for bank 0 notMemRAS3 is active for bank 1
	2	0	lower sub-bank: notSDRAMCS0 is used upper sub-bank: notMemRAS1 is used notMemRAS3 is active for bank 1

Table 60 notSDRAMCS usage with one SDRAM bank and with two SDRAM banks

The SDRAM bank may be sub-decoded into two sub-banks using the SubBanks configuration parameter in the EMI_ConfigData0 register for the bank. The size and timing of each sub-bank is identical.

The address bits used to select the lower or upper sub-bank strobe depends on the size of the sub-banks and the port size of the SDRAM bank. This is shown in [Table 61](#). The size of the sub-banks is programmed by the SubBankSize parameter in the EMI_ConfigData0 register.

Note The **SubBankSize** is the total amount of memory cells contained in one SDRAM memory device.

SubBanks	SubBank Size	Sub-bank selection address	chip select strobe selection
2	16Mbit	A ₂₁	0 = notSDRAMCS0 1 = notMemRAS1
	32 Mbit	A ₂₂	
	64 Mbit	A ₂₃	
	128 Mbit	A ₂₄	

Table 61 Sub-bank decoding

An address line connected to the SDRAM is used as a precharge mode signal. It is always Address pin 10 on the SDRAM but which address line this is from the EMI will change depending on the port width

Port Size	AP bit
8 bit	mem_addr<10>
16 bit	mem_addr<11>
32 bit	mem_addr<12>

Table 62 AP bit table

The bank select signal for an SDRAM device is normally connected to the address line(s) above the top row address.

The actual EMI address pins used for the bank select signal are fixed and only depend upon the internal organization of the SDRAM memory device which is used. The EMI can be configured to support either two or four internal banks by programing the DRAM Banks field (bit 4) of the register EMI_ConfigData3. This is shown below in [Table 63](#).

Number of SDRAM internal banks	SDRAM Memory Size	SDRAM bank selection address (internal address)	EMI BS address pins
2	16 Mbit	A ₂₀	mem_addr<15> for BS0
	32 Mbit	A ₂₁	
	64 Mbit	A ₂₂	
	128 Mbit	A ₂₃	
4	16 Mbit	A ₂₀ - A ₁₉	mem_addr<16:15> for BS[1:0]
	32 Mbit	A ₂₁ - A ₂₀	
	64 Mbit	A ₂₂ - A ₂₁	
	128 Mbit	A ₂₃ - A ₂₂	

Table 63 SDRAM internal bank selection address and BS mapping to EMI mem_addr pins

11.5.2 SDRAM access

The figure below describes a typical read and write accesses to an SDRAM. This example shows a bank activation, due to a page miss, then two write accesses in the same bank are performed in page mode.

A precharge is then done in anticipation of another bank activation command. The notMemBE signals are used as a data mask, so that only the correct bytes are updated.

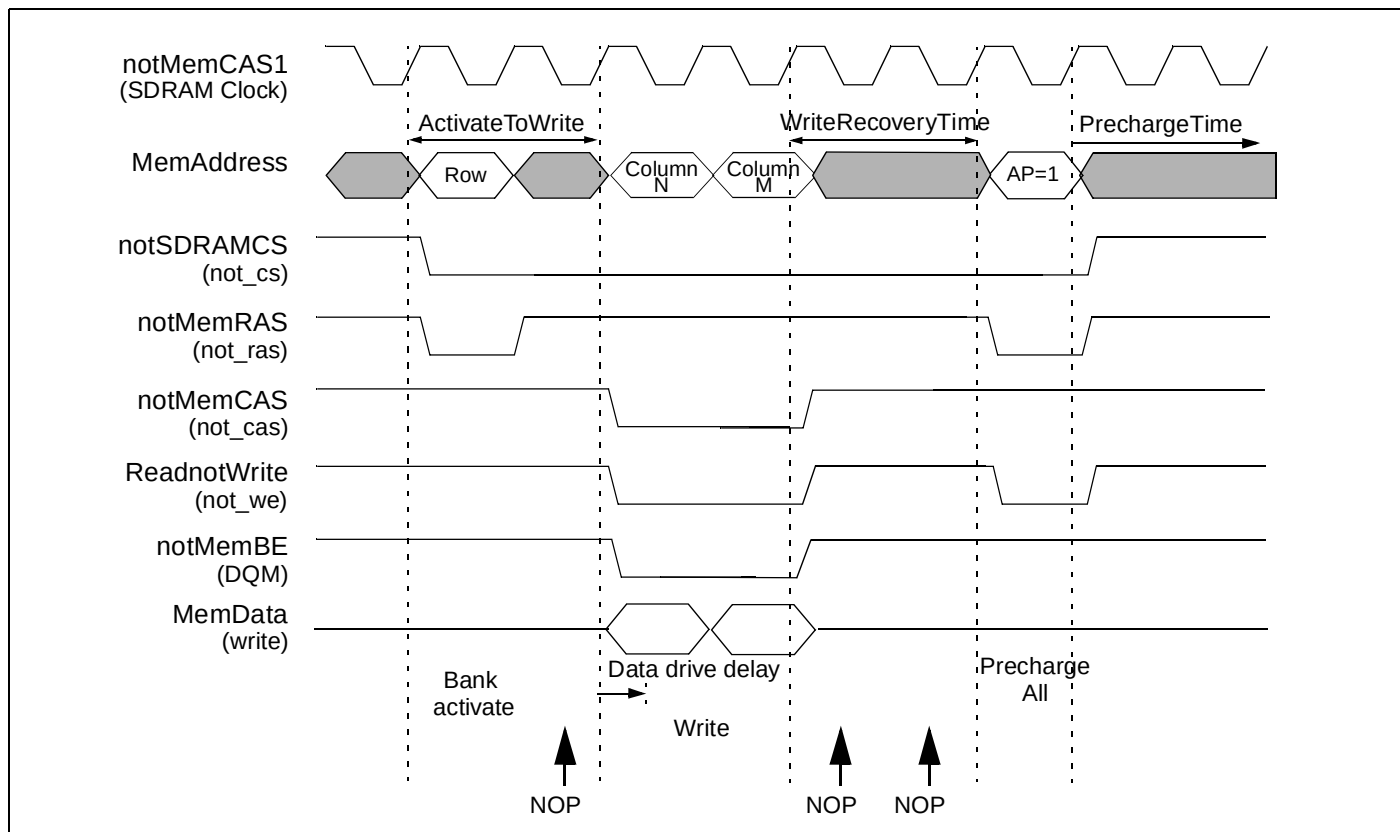


Figure 30 SDRAM write accesses

The following two figures show a bank activation, due to a page miss, then two read accesses in the same bank are performed in page mode. A precharge is then done in anticipation of another bank activation command. If only part of the SDRAM word (32 bit) is to be read then the notMemBE signals are used as a data output enable.

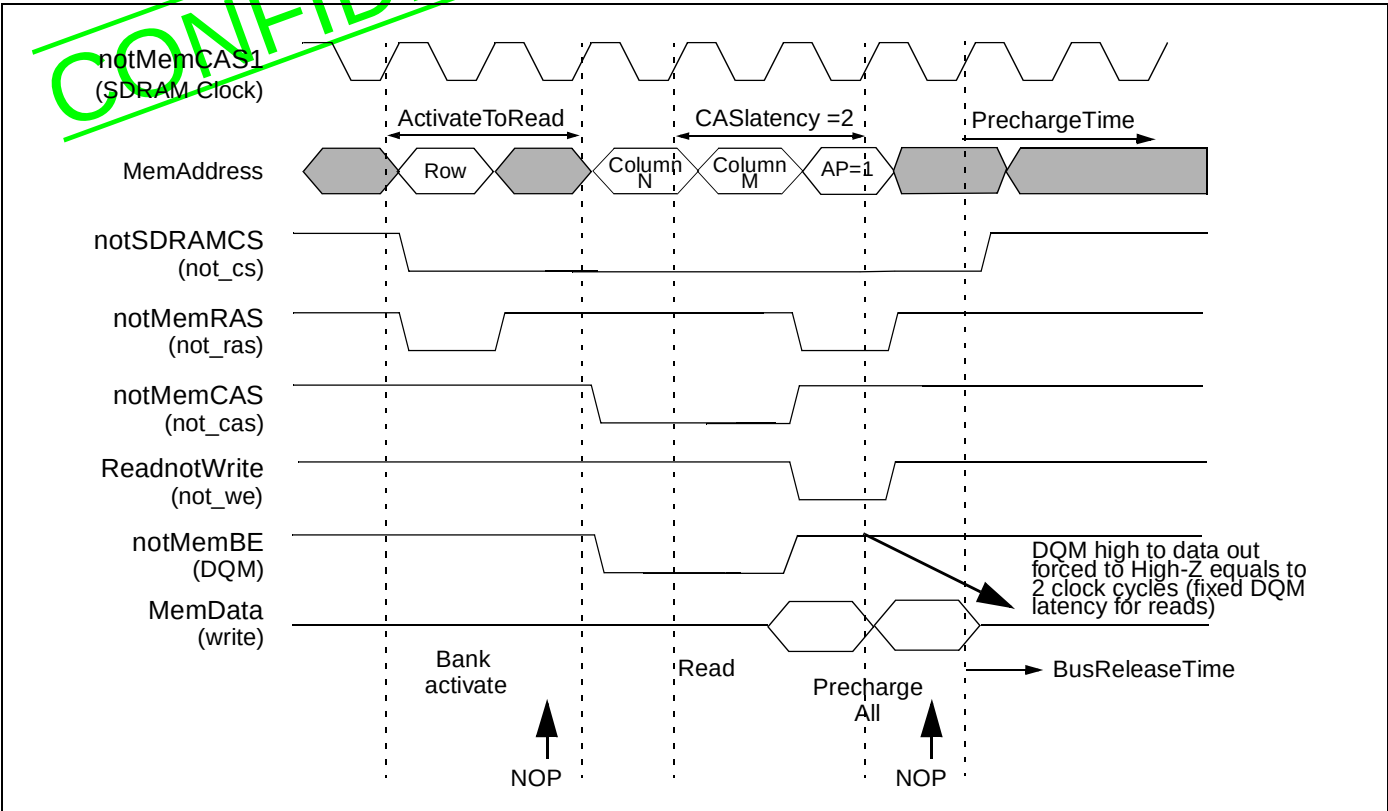


Figure 31 SDRAM read accesses with CASlatency = 2 cycles

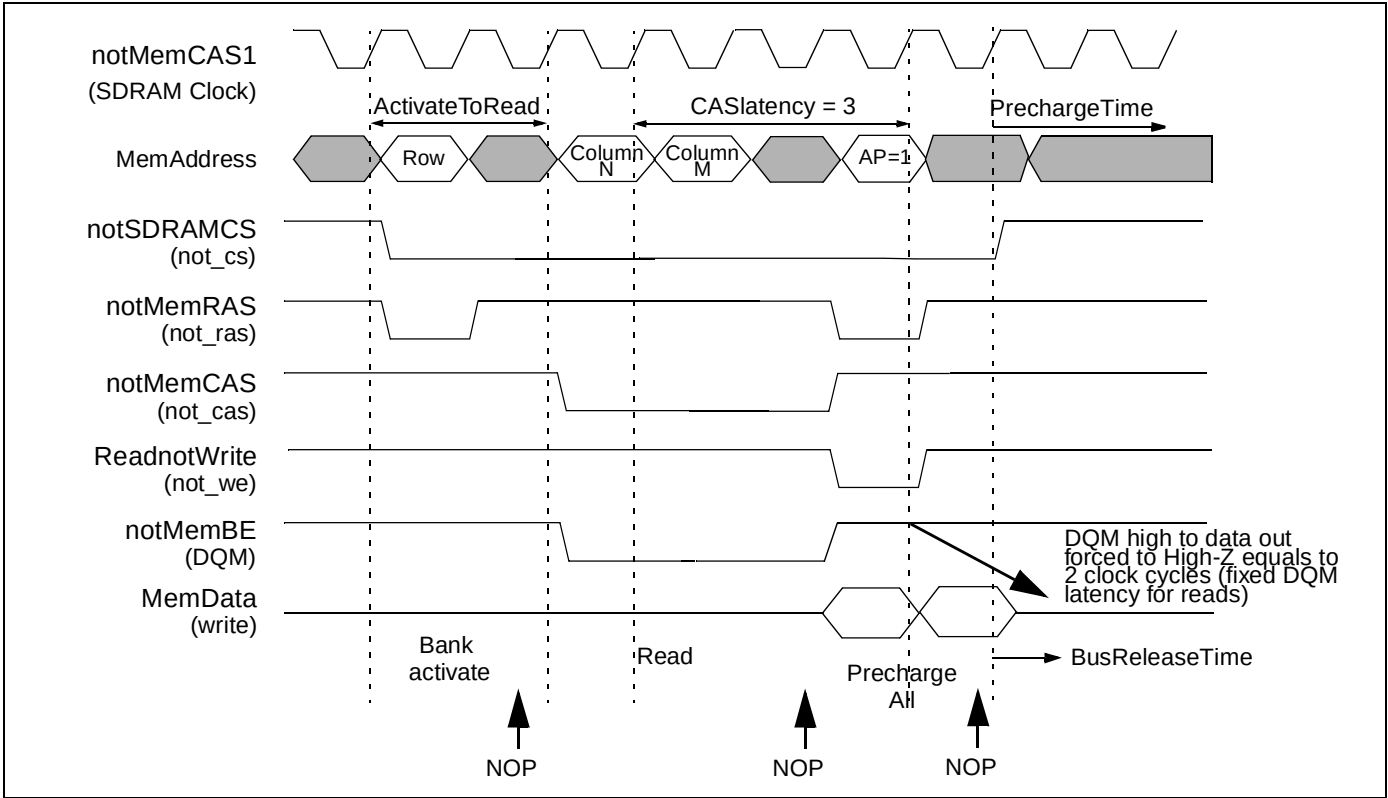


Figure 32 SDRAM read accesses with CASlatency = 3 cycles

SDRAM controller states

The following diagram shows how the EMI controls an SDRAM. These states are a subset of the commands implemented by an SDRAM. The diagram is just to describe the functionality and does not explain the actual implementation - i.e. these states may not all be implemented by one block within the EMI. For instance the refresh controller will take over from the SDRAM controller when it decides it is time for a refresh operation.

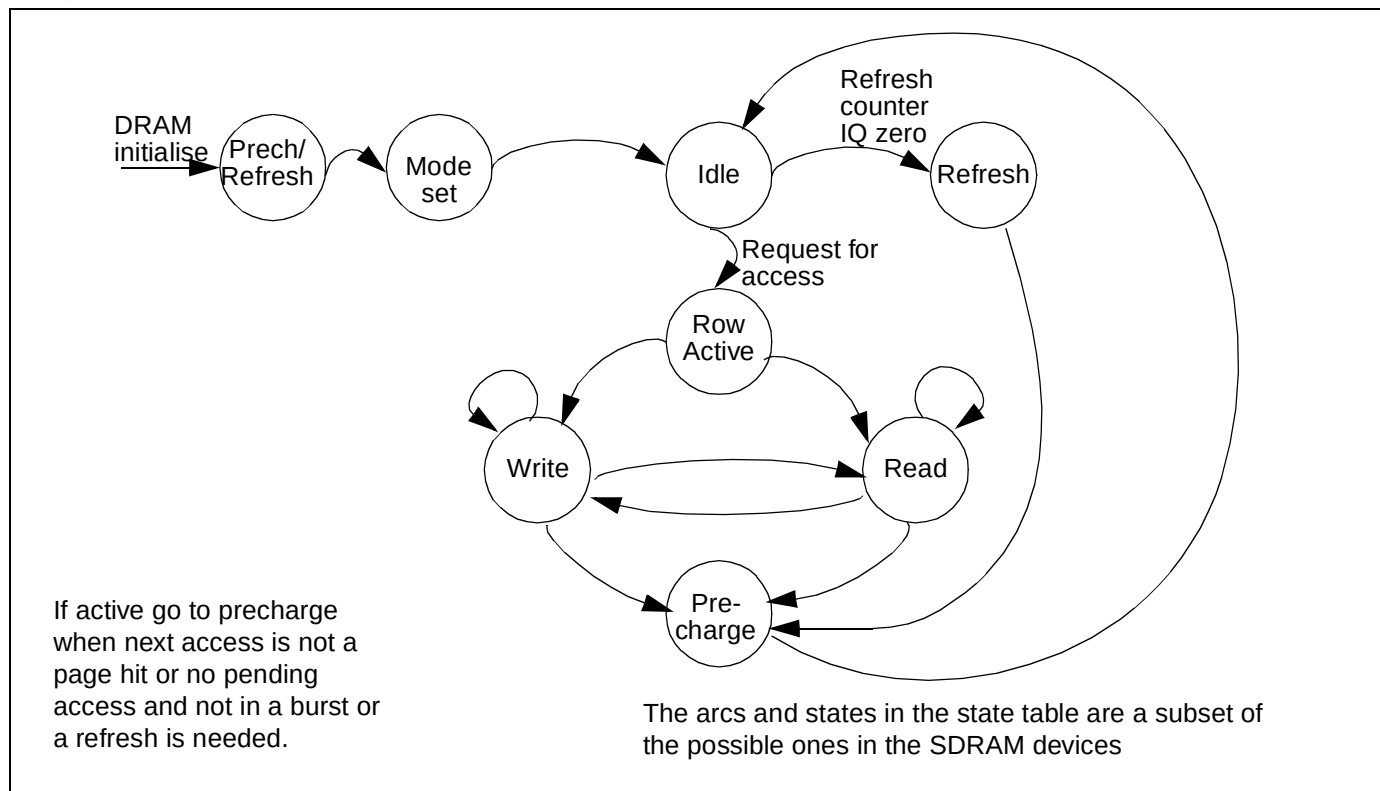


Figure 33 SDRAM controller states

Supported SDRAM commands

The EMI supports the following SDRAM commands. The relevant notSDRAMCS signal will be active for all these operations.

Command	Strobes state			Mode bits		Address	Notes
	not_ras	not_cas	not_we	BS	A10 AP		
Precharge all	0	1	0	X	1	X	Used to close pages after a burst in simple page mode.
Activate	0	1	1	1/0	1/0	Row addr	Opens a page (Row active)
Write	1	0	0	1/0	0	Col addr	Write words (uses byte enables to mask bytes).
Read	1	0	1	1/0	0	Col addr	Read words
Mode set	0	0	0	1/0	1/0	Mode data	Done once only at start up.
Refresh	0	0	1	X	X	X	CBR style refresh (Auto Refresh)
No operation	1	1	1	X	X	X	Does nothing (same as having CS inactive)

Table 64 SDRAM commands with notSDRAMCS active

The mode bits will change depending on port size, because notmemBE2 and notmemBE3 become address lines in 8/16 bit mode. The auto precharge AP bit is always A10 on the SDRAM, but the bank select (BS) bit on the device is dependent on memory size. It is A11 on a 16Mbit device. See [Table 63 on page 82](#) for further details.

Supported operations for a single bank of SDRAM

Using the above set of commands, the current implementation of the EMI will support the following truth table:

SDRAM current state	Command	Notes
IDLE	DESL	DESL means notSDRAMCS not active
	ACTIVATE	The bank specified by the address pins and ROW address is activated
	REFRESH	SDRAM enters in refresh mode
	MODE SET	SDRAM enters in mode register set cycle
ROW ACTIVE	NOP	When the number of cycles between the activate of the page and the read/write command is > 1 cycle.
	READ	A read operation starts
	WRITE	A write operation starts
READ	READ	This command will be issued for each read access in the same ROW (page mode).
	NOP	This command will be performed in case of a write access will follow the current read access and the write access is in the same ROW (page mode) or before a PRECHARGE ALL command when CASlatency is > 2 clock cycles. If in the cycle in which the NOP is issued a new request comes and it is in the same ROW, a READ or a WRITE command will be generated.
	PRECHARGE ALL	This command will happen each time there is either a new access (Read or Write) but to a different ROW address (page miss) or an EMI bank switch or no new pending requests from the EMI buffer or a pending refresh or a DMA access. The command will be issued in the cycle after the current READ only if CASlatency is < 3 clock cycles.
WRITE	WRITE	This command will be issued for each write access in the same ROW (page mode).
	READ	This command will happen when a read access will follow a write one and it's in the same ROW (page mode).
	NOP	This command will happen each time there is either a new access (Read or Write) but to a different ROW address (page miss) or an EMI bank switch or no new pending requests from the EMI buffer or a pending refresh or a DMA access. The command will be followed by a PRECHARGE ALL after the WriteRecoveryTime is expired. If in the cycle in which the NOP is issued a new request comes and it is in the same ROW, a READ or a WRITE command will be generated.
PRECHARGE ALL	DESL	DESL means notSDRAMCS not active
	NOP	This command will be performed until DESL command.
REFRESH	DESL	DESL means notSDRAMCS not active

Table 65 Supported commands for a single SDRAM access

Timing parameters

The number of cycles required for each SDRAM command depend on the clock frequency supplied to the device. The EMI is designed to work at a number of speeds so it is important to be able to vary the values. For instance when interfacing to a 100MHz device the following parameters change with clock frequency. Customers may wish to interface to a lower cost 83MHz device in which case additional cycles will need to be inserted.

Parameter	100 MHz SDRAM		83 MHz SDRAM	
	50MHz SDRAM/CPU	60MHz SDRAM/CPU	50MHz SDRAM/CPU	60MHz SDRAM/CPU
ModeSetDelay	1	2	1	2
RefreshTime	5	6	6	8
ActivateToRead	2	2	2	3
ActivateToWrite	1	2	1	2
CASlatency	2	2	2	2
PrechargeTime	2	2	2	3

Table 66 Command cycle times when using a 100MHz or an 83MHz SDRAM

SDRAM access examples

The following 13 figures show timing examples of typical SDRAM accesses.

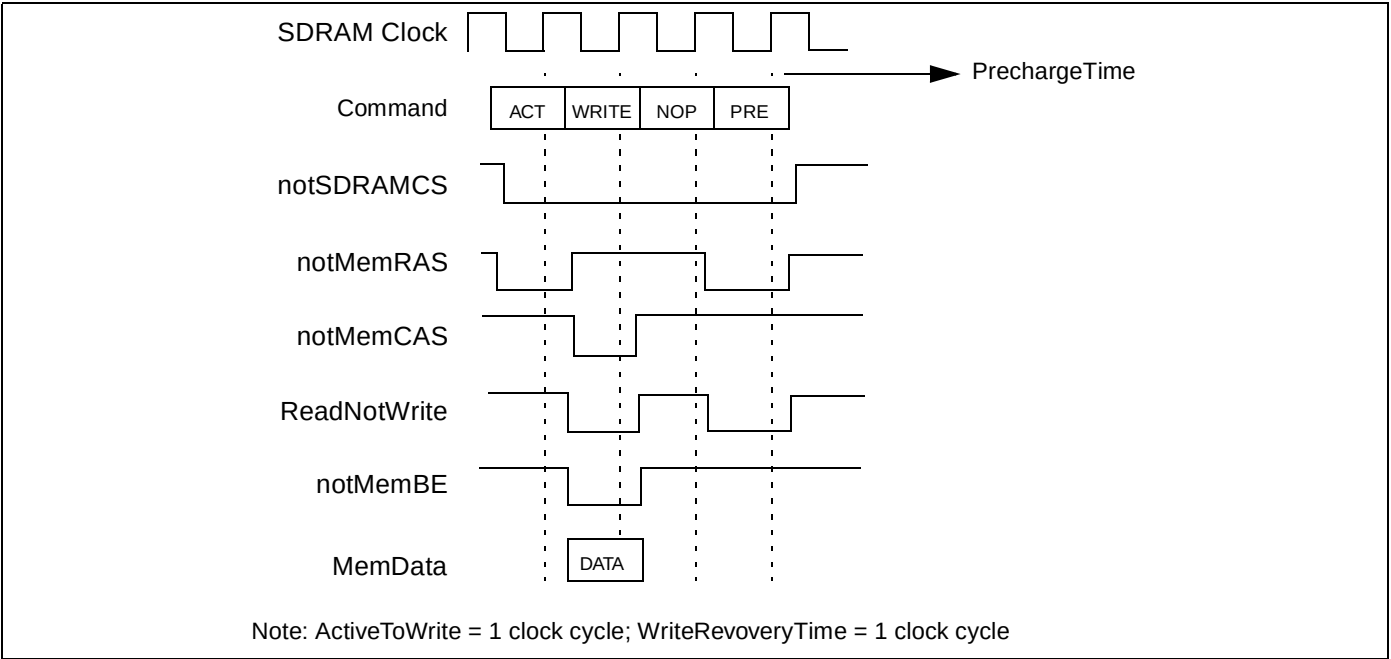


Figure 34 single WRITE

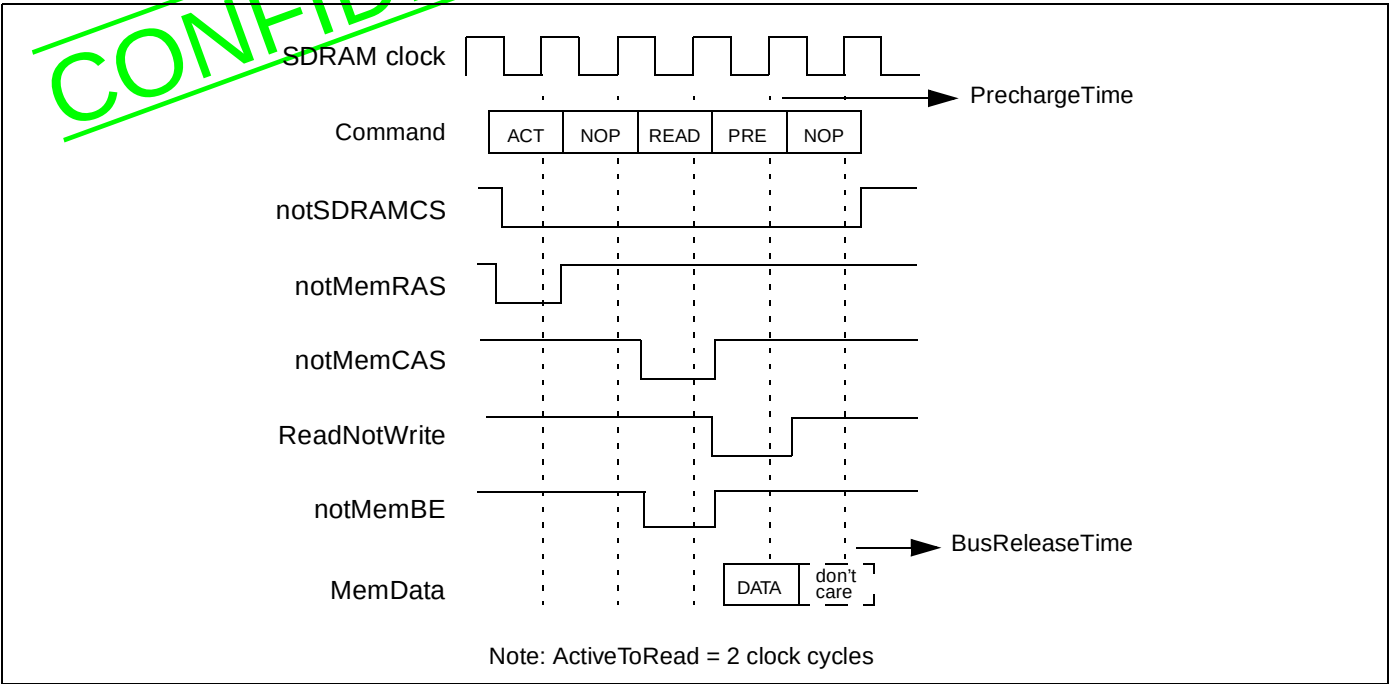


Figure 35 Single READ and CASLatency = 1 cycle

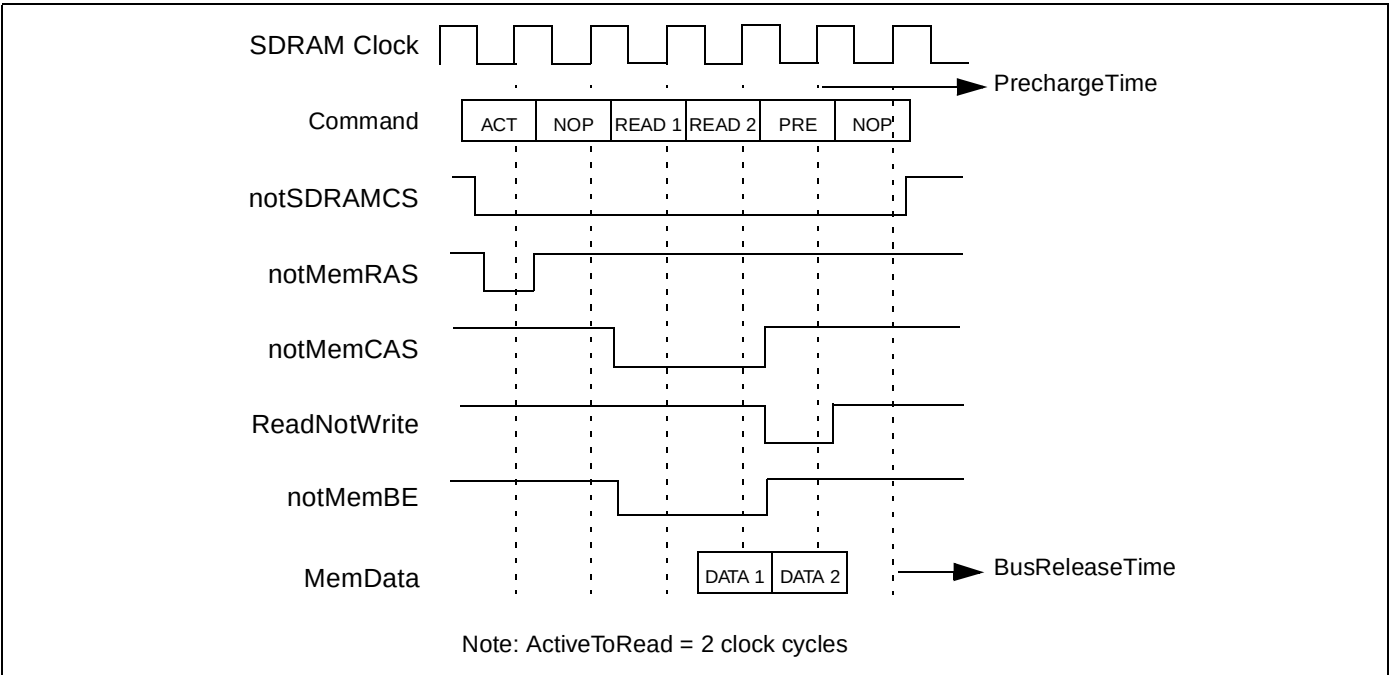


Figure 36 Pagemode READs and CASLatency = 1 cycle

CONFIDENTIAL

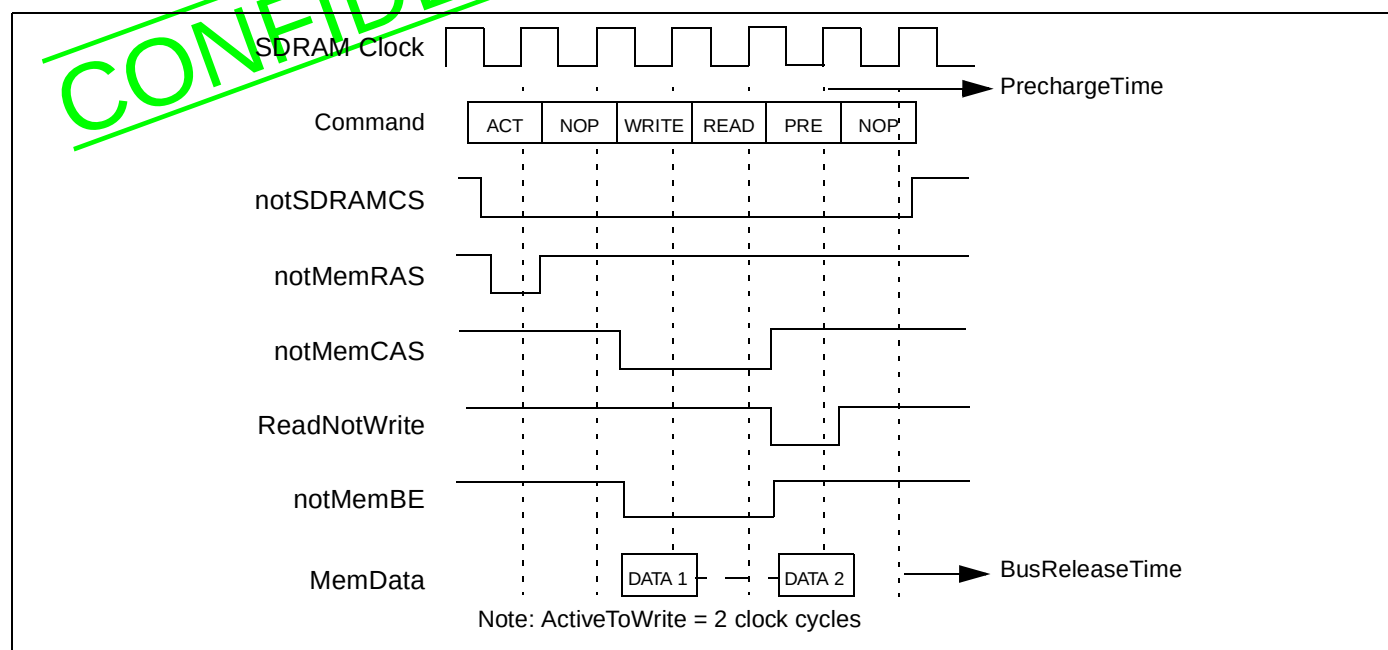


Figure 37 pagemode READ after WRITE and CASLatency = 1 cycle

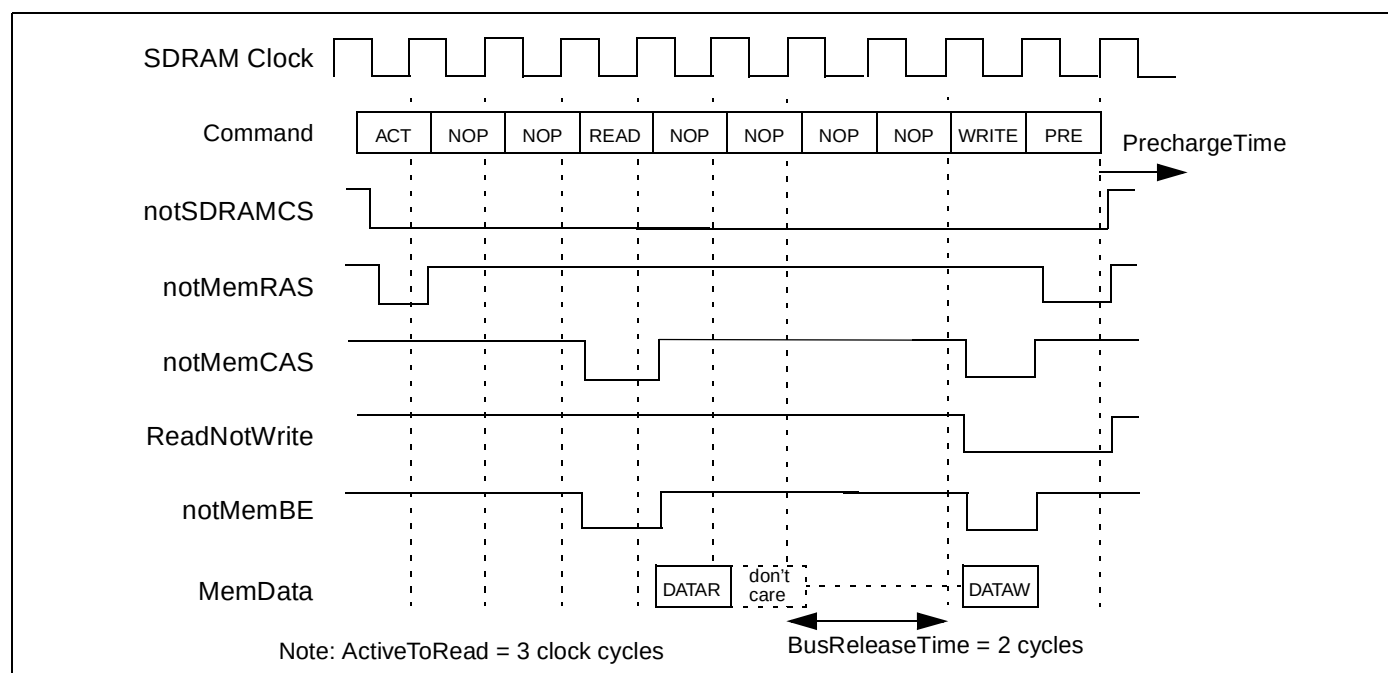


Figure 38 pagemode WRITE after READ and CASLatency = 1 cycle

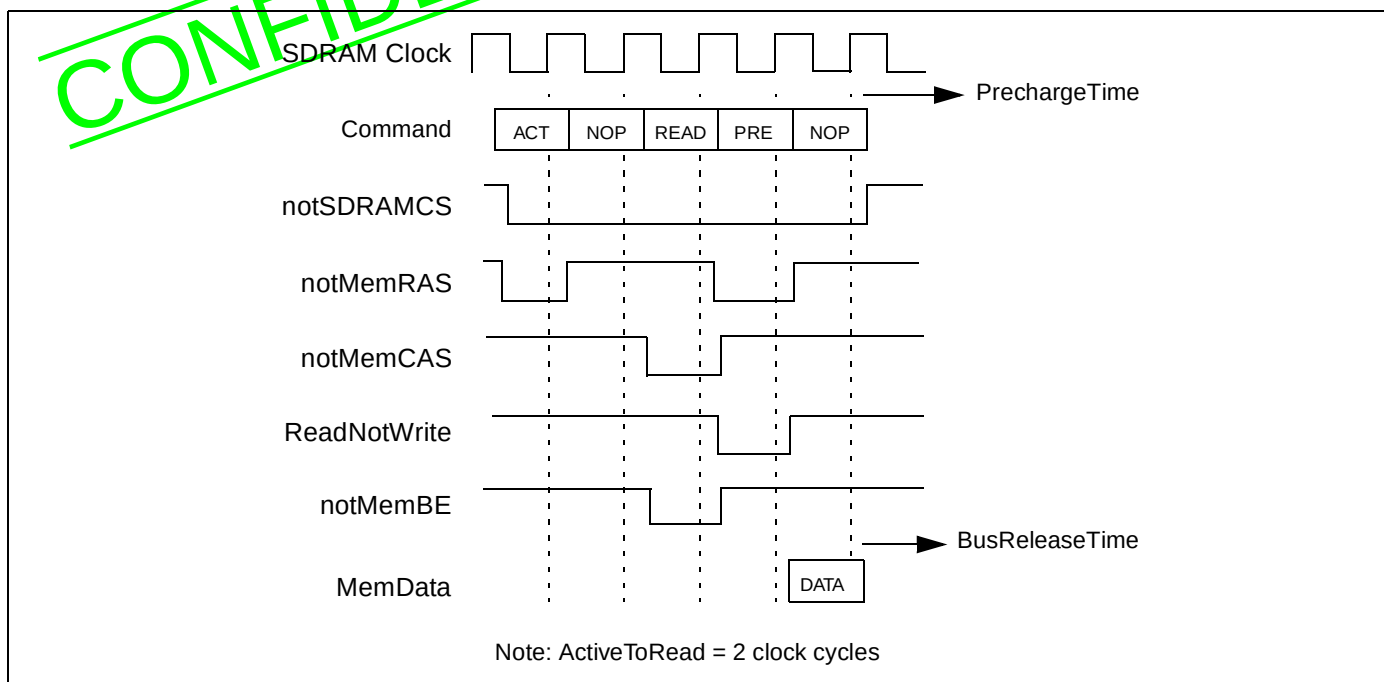
CONFIDENTIAL

Figure 39 single READ and CASLatency = 2 cycles

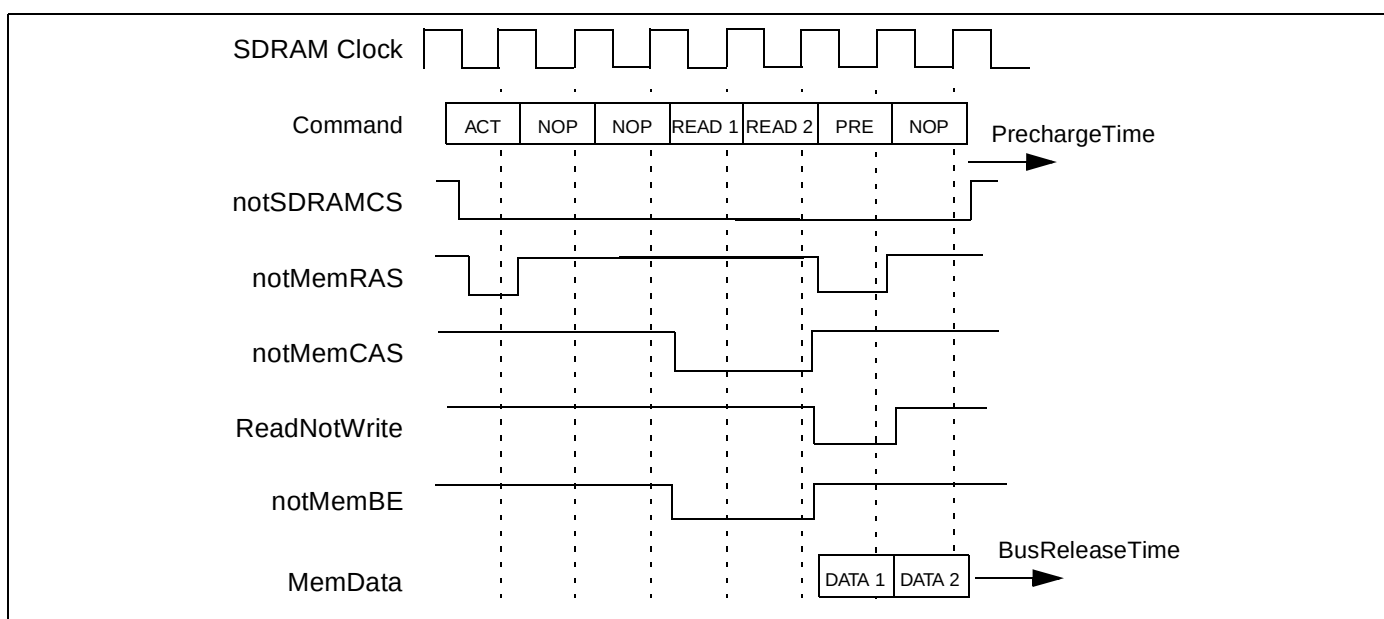


Figure 40 pagemode READ and CASLatency = 2 cycles

I

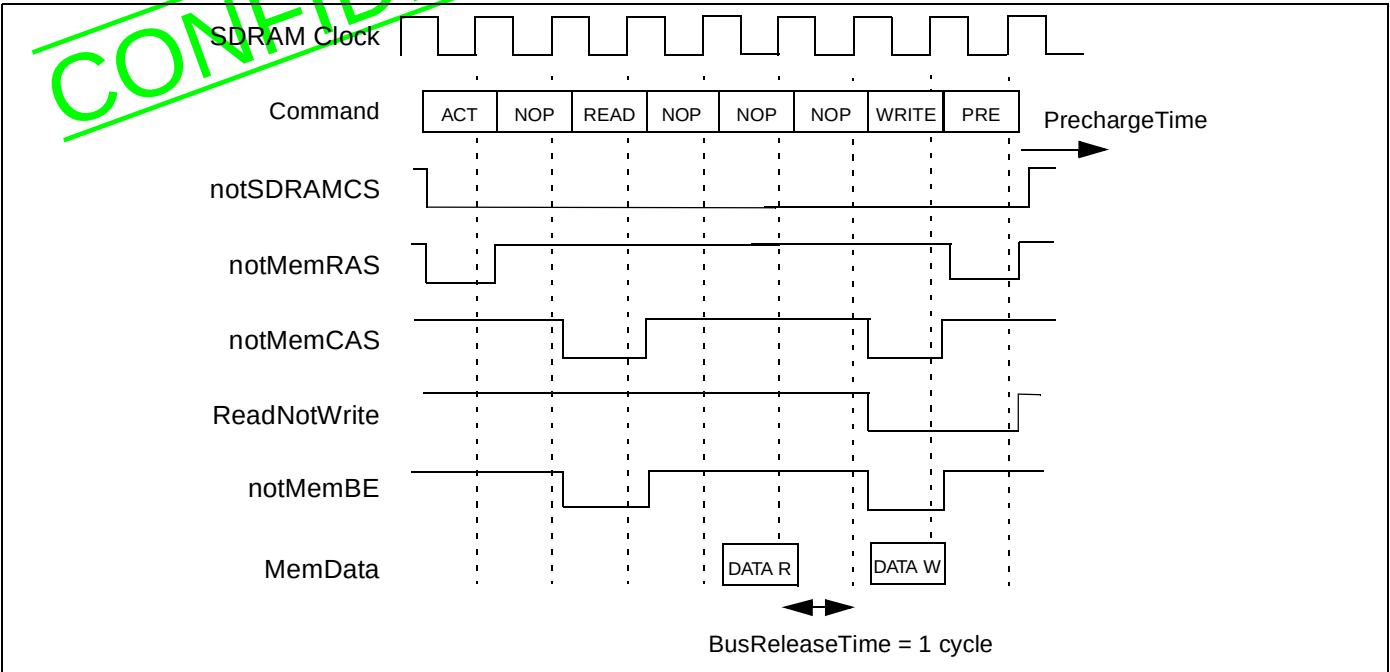


Figure 41 pagemode WRITE after READ and CASLatency = 2 cycles

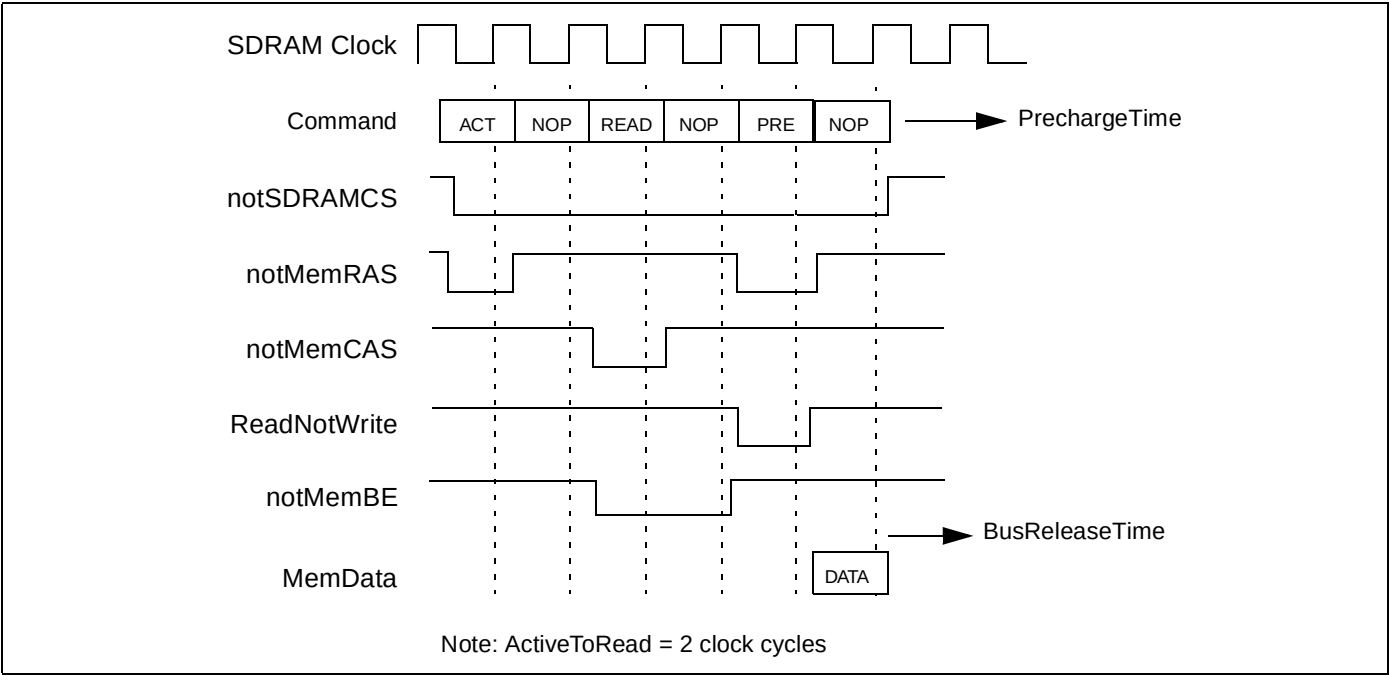


Figure 42 single READ and CASLatency = 3 cycles

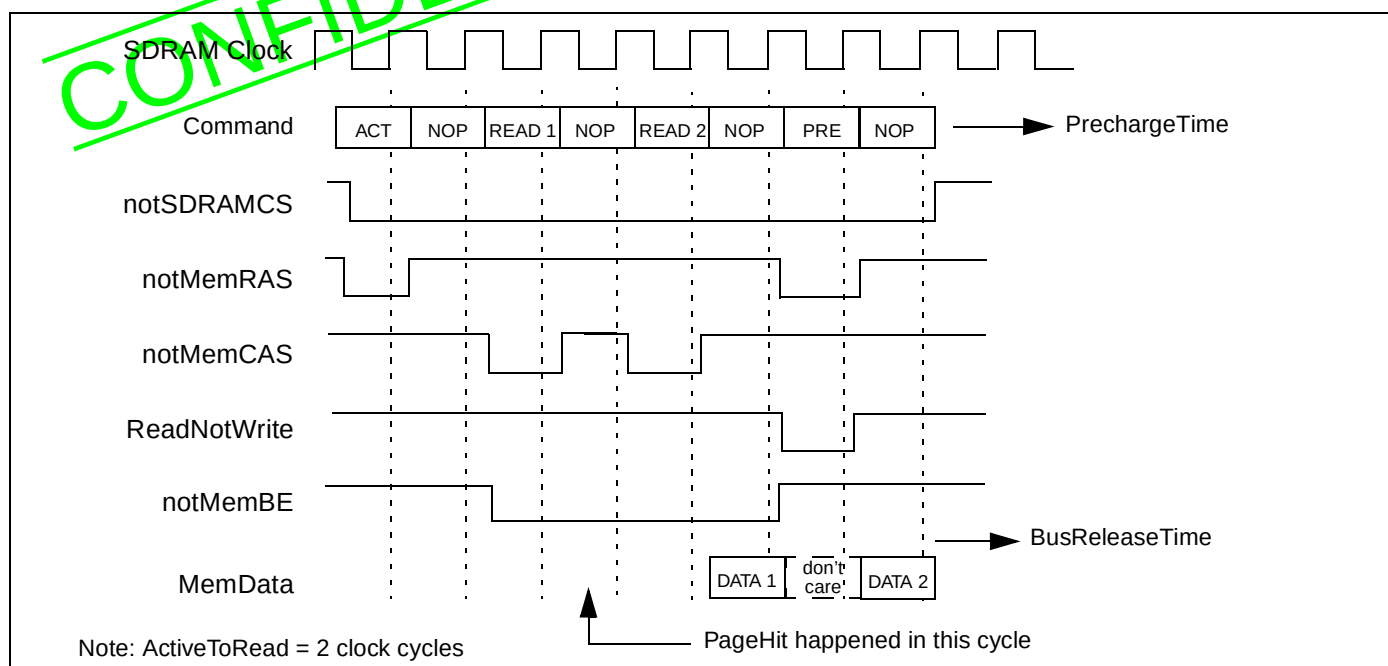


Figure 43 pagemode READ and CASLatency = 3 cycles. Pagehit during the NOP

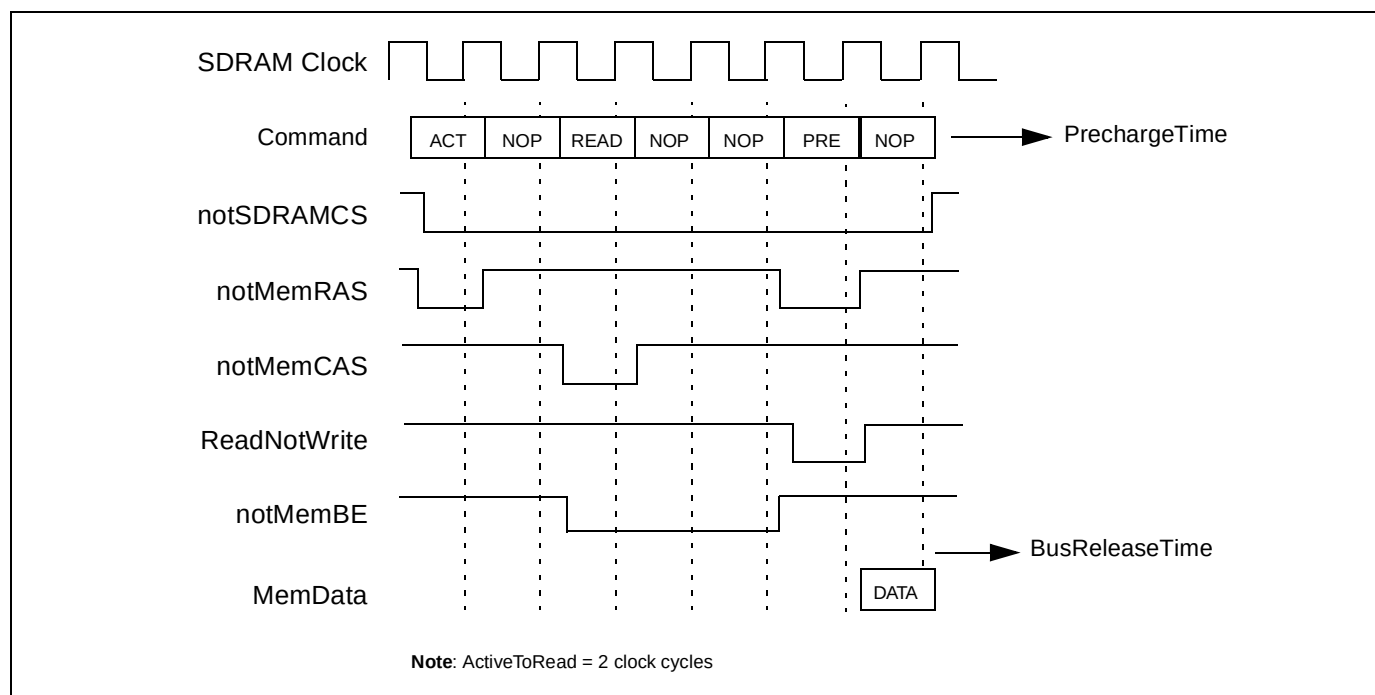


Figure 44 single READ and CASLatency = 4 cycles

SDRAM refresh cycle

The SDRAM bank is periodically refreshed at intervals specified by the RefreshInterval configuration parameter. All sub banks to be refreshed in the same access. After the last refresh command is issued, a wait period allows for RefreshTime + PrechargeTime cycles before starting a new access with a Bank Activate command.

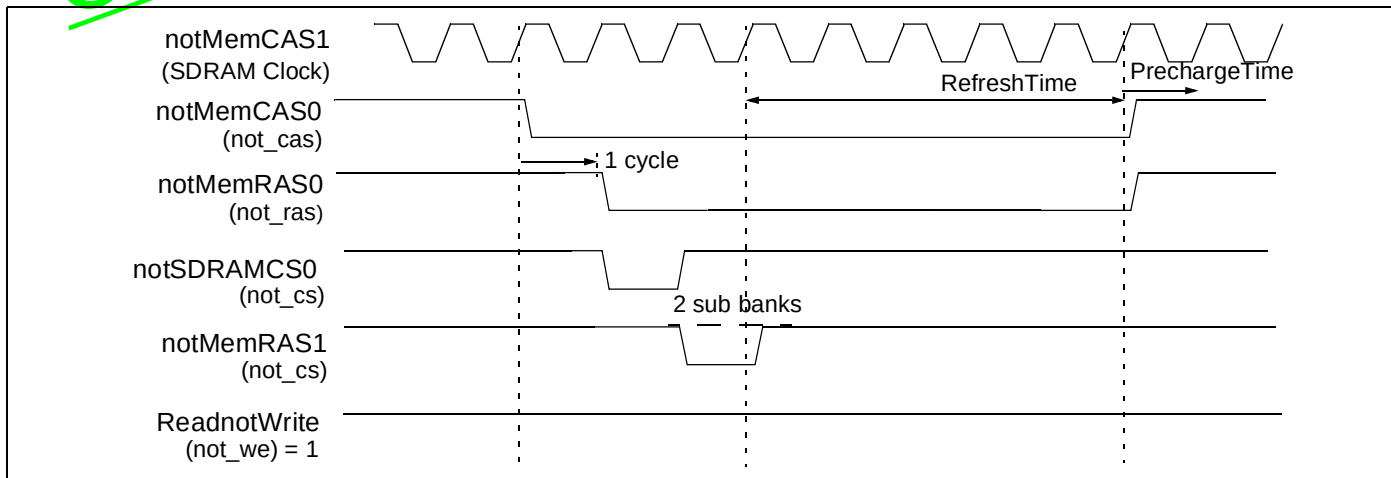


Figure 45 Generic refresh access for SDRAM bank (case one SDRAM bank)

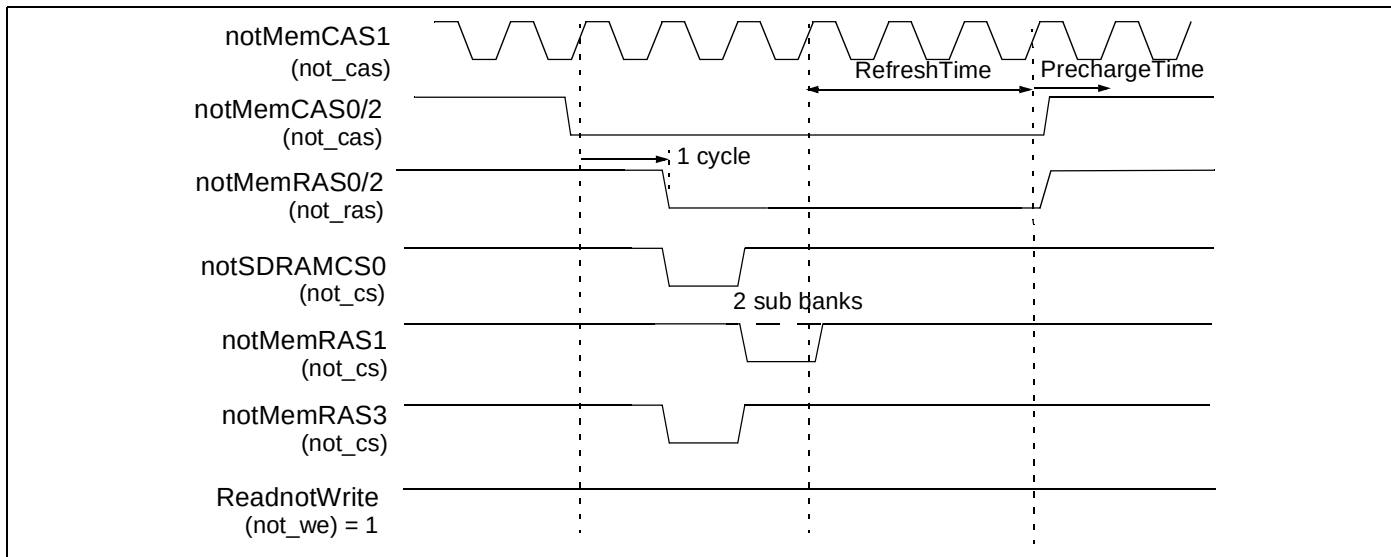


Figure 46 Generic refresh access for SDRAM bank (case two SDRAM banks)

Name	Programmable value	40MHz	60MHz
PrechargeTime	1 - 8 cycles	25 - 200ns	16 to 128ns
RefreshInterval	(1 - 16) * 128 cycles	3 - 48us	2 to 32us
RefreshTime	1 - 16 cycles	25 to 400ns	16 to 256ns

Table 67 Strobe timing parameters for refresh accesses

The precharge time commences after the end of the refresh access. The SDRAM bank is precharged before the refresh access starts.

Since there can be more than one bank of SDRAM, there is a set of refresh-specific configuration parameters for each of those banks. The refresh configuration is then taken from the lowest numbered bank configured as SDRAM, from bank 0 to bank 3.

11.6 EMI configuration

The EMI configuration is held in memory-mapped registers. Each EMI bank has several parameters which can be configured. The parameters define the structure of the external address space and how it is allocated to the four banks and the timing of the strobe edges for the four banks.

Each EMI bank has 64 bits of configuration data, which is held in four 16-bit configuration registers. In addition, there is an EMI_ConfigLock register for each bank, an EMI_ConfigStatus register, an EMI_DRAMInitialize register and an EMI_ConfigPadlogic register. To ensure good configuration, each of the four banks must be configured after reset and then have their configuration locked by writing to the EMI_ConfigLock register before any accessing an external bank. The registers may be accessed independently of EMI activity.

Note *EMI_ConfigPadlogic register bit 15 is used for 1284 Slave/Master Mode selection by the IEEE 1284 port.*

Register	EMI base offset	Width	Type	Description
ConfigData0Bank0	#00	16	R/W	EMI bank 0 configuration data register 0
ConfigData1Bank0	#04	16	R/W	EMI bank 0 configuration data register 1
ConfigData2Bank0	#08	16	R/W	EMI bank 0 configuration data register 2
ConfigData3Bank0	#0C	16	R/W	EMI bank 0 configuration data register 3
ConfigData0Bank1	#10	16	R/W	EMI bank 1 configuration data register 0
ConfigData1Bank1	#14	16	R/W	EMI bank 1 configuration data register 1
ConfigData2Bank1	#18	16	R/W	EMI bank 1 configuration data register 2
ConfigData3Bank1	#1C	16	R/W	EMI bank 1 configuration data register 3
ConfigData0Bank2	#20	16	R/W	EMI bank 2 configuration data register 0
ConfigData1Bank2	#24	16	R/W	EMI bank 2 configuration data register 1
ConfigData2Bank2	#28	16	R/W	EMI bank 2 configuration data register 2
ConfigData3Bank2	#2C	16	R/W	EMI bank 2 configuration data register 3
ConfigData0Bank3	#30	16	R/W	EMI bank 3 configuration data register 0
ConfigData1Bank3	#34	16	R/W	EMI bank 3 configuration data register 1
ConfigData2Bank3	#38	16	R/W	EMI bank 3 configuration data register 2
ConfigData3Bank3	#3C	16	R/W	EMI bank 3 configuration data register 3
ConfigLockBank0	#40	1	W	Write protection bit. When set makes EMI_ConfigData0-3 for Bank 0 read only.
ConfigLockBank1	#44	1	W	Write protection bit. When set makes EMI_ConfigData0-3 for Bank 1 read only.
ConfigLockBank2	#48	1	W	Write protection bit. When set makes EMI_ConfigData0-3 for Bank 2 read only.
ConfigLockBank3	#4C	1	W	Write protection bit. When set makes EMI_ConfigData0-3 for Bank 3 read only.
ConfigStatus	#50	8	R	Status information
SDRAMModeReg0	#58	16	W	SDRAM mode register for a single SDRAM bank, or SDRAM bank 0 when there are two SDRAM banks.
SDRAMModeReg1	#5C	16	W	SDRAM mode register for SDRAM bank 1 when there are two SDRAM banks.
DRAMInitialize	#60	1	W	Initialize any DRAM/SDRAM in the system.
SDRAMNOPGen	#64	1	W	When an SDRAM is in the system, it generates NOP commands during initialization phase until a DRAMInitialize is issued.
ConfigPadlogic	#70	16	R/W	Padlogic configuration data register.

Table 68 EMI configuration registers

11.7 EMI default configuration

The default configuration is loaded into all four banks on reset. It should allow the EMI to read data from a slow ROM memory (in bank 3) before the EMI is configured by the software. The following parameters are set.

Parameter	Default value
DataDriveDelay	101: (5 phases)
BusReleaseTime	10: (2 cycles)
CSactive	01: (active during read only)
OEactive	01: (active during read only)
BEactive	00: (inactive)
Portsize	Value of the signal on boot source pins
DeviceType	001: (peripheral)
AccessTimeRead	1000: (8+2=10 cycles)
CSe1TimeRead	00: (0 phases)
CSe2TimeRead	00: (0 phases)
OEe1TimeRead	00: (0 phases)
OEe2TimeRead	00: (0 phases)
LatchPoint	00: (end of access cycle)

Table 69 EMI default configuration

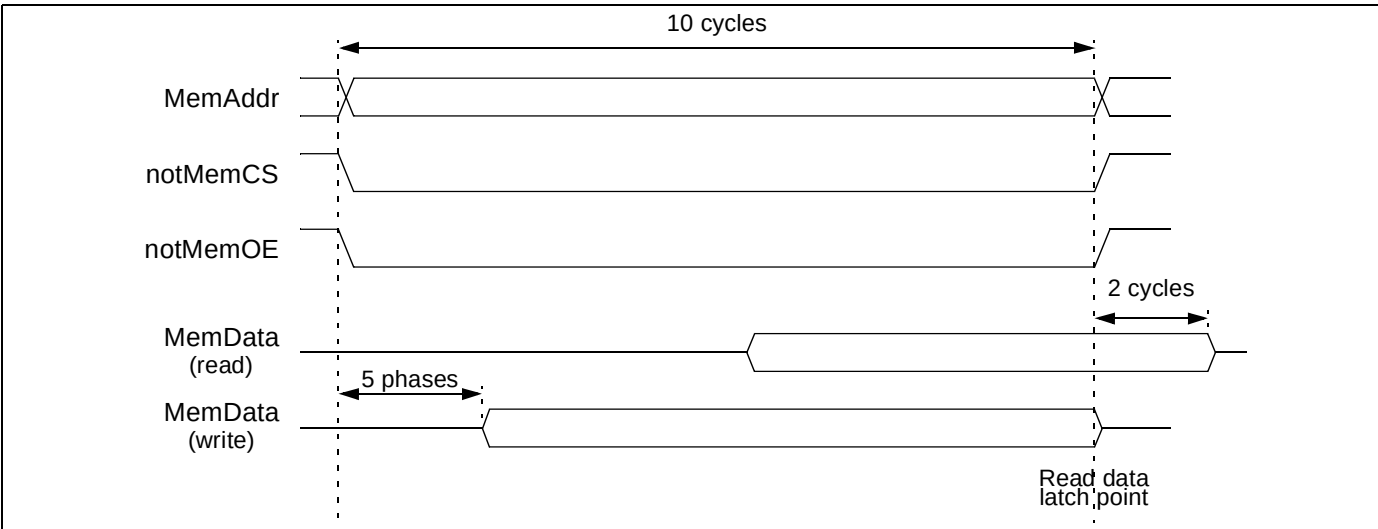


Figure 47 EMI default configuration

SDRAMModeReg0/1			
	latency mode	burst type	burst length
15:7	6:4	3	2:0
000000000	010	0	010

Table 70 EMI default configuration for SDRAMModeReg0/1 registers

12 System services

The system services module includes all of the necessary logic to initialize the device and support debugging. Device initialization and debugging can also be done with the diagnostic controller unit (DCU); see the [Diagnostic controller on page 100](#).

12.1 Shared pins

The **CPUAnalyse** and **ErrorOut** system service pins are shared with the DCU **Trigin** and **Trigout** pins. The **ErrorOut** pin is sampled after reset to decide how these pins are to be used. If sampled low the pins will be used as **CPUAnalyse** and **ErrorOut**, as used by system services; if sampled high the pins will be used as **Trigin** and **Trigout** as used by the DCU. Generally, in a system, the **ErrorOut** pin will have a pull-down or pull-up resistor (~10K) attached, depending on the functionality required.

12.2 Hard and soft reset

The STi5512 has 2 reset pins: **notRST** and **CPUReset**.

Power-on (hard) reset

The ST20 reset is asynchronous and does not require any clocks to initialize its state. The MPEG reset is a mixture of asynchronous and synchronous resets, and requires several internal clock cycles (ClockIn) to initialize the device state. The PLLs take time to start-up and stabilize after all the following conditions are met:

- VDD is in the normal operating range
- Clockin stabilized within normal VIH/VIL and frequency limits

For power-on reset, ensure that the **notRST** is asserted (low) *before* the clocks and power supply are stable (see [Table 137: Power-on \(hard\) reset timings on page 291](#)). However, **notRST** can be reasserted at any time after board power-up when the clocks and power supply are stable.

When **notRST** is asserted (regardless of any other inputs), all modules are asynchronously forced into their power-on reset state. The following conditions must be met before **notRST** can be de-asserted (high) and the device bootstrap started:

- The **LPClockIn** clock source must be active (see [Real-time counter on page 234](#)).
- The clocks and power should be stable: the PLL needs to be started and running, for the MPEG part at least 160 cycles of clockin (27Mhz) must have been applied
- The **notTRST** TAP reset pin has been asserted. Note that the notTRST pin is normally held to ground with a 10K pull down resistor, to meet this condition. Boundary scan testers can then still drive the TAP interface, if required.

Bootstrap code can either be in off-chip ROM or can be received on the OS-Link or through the DCU, as described in [Bootstrap on page 97](#).

Soft reset

During power-on reset, the entire chip is affected, including the clock control logic, which takes a relatively long time to reset. An alternative “soft” reset is provided which does not affect the clocks and takes much less time. This form of reset must only be used when the system is up and running, and not on power-up. Soft reset is invoked by taking **CPUReset** high while **CPUAnalyse** is held low and **notRST** is held high.

12.3 Bootstrap

The STi5512 can be bootstrapped from external ROM, from the OS-Link or from the diagnostic controller (DCU).

12.3.1 Booting from the DCU

The STi5512 can be booted from the DCU at any time by setting up the test access port (TAP) to do so. The procedure is explained in the *Diagnostic controller* chapter. When booting from DCU the **ErrorOut** pin must be pulled high on the board so that the shared **ErrorOut** and **CPUAnalyse** pins are used in the DCU mode (see [Shared pins on page 96](#)).

If the user does not set up the TAP to boot from DCU then the STi5512 will either boot from ROM or the OS-Link as soon as it comes out of reset.

12.3.2 BootSource0-1 pins

When not booting from DCU, the **BootSource0-1** pins determine whether the STi5512 will boot from the OS-Link or from ROM as it comes out of reset. In the latter case they also define the width of the ROM. The meanings of the different combinations of settings of these pins are given in [Table 71](#) below.

BootSource0	BootSource1	Effect
0	0	Boot from OS-Link.
1	0	Boot from ROM. Port size of bank 3 hard wired to 32-bits.
0	1	Boot from ROM. Port size of bank 3 hard wired to 16-bits.
1	1	Boot from ROM. Port size of bank 3 hard wired to 8-bits.

Table 71 BootSource0-1 pin settings

12.3.3 Booting from ROM

If either or both **BootSource** pins are high, the STi5512 will boot from ROM. Boot code is run from a slow external ROM placed in bank 3 at the top of memory. The ROM width is given by the **BootSource0-1** pins, as in [Table 71](#). When booting from ROM, the value in the configuration registers for the **PortSize** for bank 3 is disregarded.

When booting from ROM, the STi5512 starts to execute code from the top two bytes in external memory, at address #7FFFFFFE, which should contain a backward jump to a bootstrap program in ROM.

12.3.4 Booting from OS-Link

If both **BootSource** pins are low then the STi5512 will boot from the OS-Link. The bootstrap code can be loaded from a host using the OS-Link. External memory cannot be accessed until the EMI has been configured, so normally the bootstrap and configuration code is kept small enough to execute from internal memory.

When booting from link the **ErrorOut** pin must be pulled high on the board so that the shared **ErrorOut** and **CPUAnalyse** pins are used in the OS-Link mode (see [Shared pins on page 96](#)).

When booting from a link, the STi5512 will wait for the first bootstrap message to arrive on the link. The first byte received down the link is the control byte.

If the control byte is greater than 1 (i.e. 2 to 255), it is taken as the length in bytes of the boot code to be loaded down the link. The bytes following the control byte are then placed in internal memory, starting at location **MemStart**. Following reception of the last byte, the STi5512 will start executing code at **MemStart**. The memory space immediately above the loaded code is used as work space.

A byte arriving on the bootstrapping link after reception of the last bootstrap byte is retained, and no acknowledge is sent until a process inputs from the link.

Peek and poke

Any location or locations in internal or external memory can be interrogated and altered while the STi5512 is waiting for a bootstrap from link. This facility can be used to test memory, initialize external peripherals or load code.

When booting from link, if the first byte (the control byte) received down the link is greater than 1, it is taken as the length in bytes of the boot code to be loaded down the link, as described above.

If the control byte is 0, then eight more bytes are expected on the link. The first four byte word is interpreted as an internal or external memory address at which to *poke* (write) the second four byte word.

If the control byte is 1, the next four bytes are interpreted as the address from which to *peek* (read) a word of data; the word is sent back down the output channel of the bootstrap link.

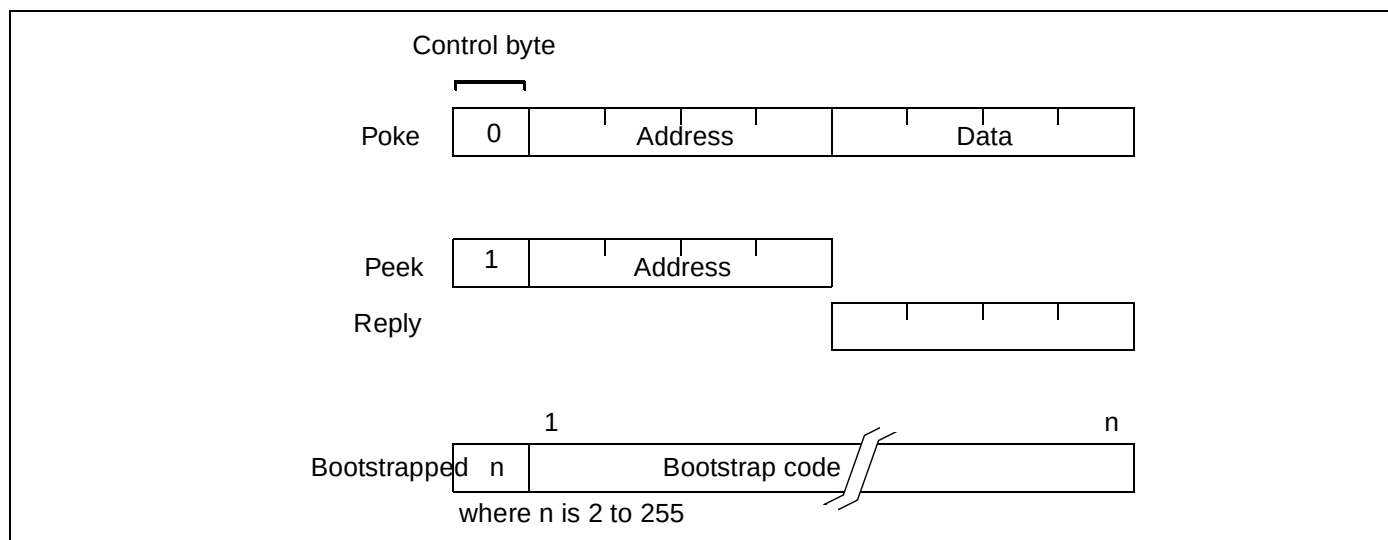


Figure 48 Peek, poke and bootstrap

Peeks and *pokes* in the address range #20000000 to #3FFFFFFF access the internal peripheral device registers. Therefore they can be used to configure the EMI before booting. Addresses which overlap the internal peripheral addresses (#20000000 to 3FFFFFFF) cannot be accessed via the link.

Following a *peek* or *poke*, the STi5512 returns to its previously held state. Any number of accesses may be made in this way until the control byte is greater than 1, when the STi5512 will commence reading its bootstrap program.

Peek and *poke* use the two words above **MemStart**, i.e. memory locations #80000140 to #80000147. These words should not be used by the application if it is to be debugged via the OS-Link.

12.4 Support for debugging

The systems services module provides debugging support, which can be used in conjunction with the OS-Link. Debugging may also be performed using the DCU, as described in the chapter entitled *Diagnostic controller*.

System services debugging support uses the following pins:

- CPUReset
- CPUAnalyse
- ErrorOut

12.4.1 Analyze

The analyze function is provided for post-mortem debugging from a OS-Link. It enables the CPU to be rebooted while still preserving memory data and configuration as well as CPU status.

If **CPUAnalyze** is taken high when the STi5512 is running, the CPU will halt at the next descheduling point. **CPUReset** may then be asserted. When **CPUReset** goes low again **CPUAnalyze** has to be taken low. As soon as **CPUAnalyze** is taken low the STi5512 will start its boot sequence but the previous memory data and configuration as well as several of the previous status flags will be maintained and available for examination. Refreshing of external DRAM continues.

An input OS-Link will continue with outstanding transfers. An output OS-Link will not make another access to memory for data but will transmit those bytes already in the link buffer. Providing there is no delay in link acknowledgment, the link will be inactive within a few microseconds of the **CPUReset** pulse.

If **CPUAnalyze** is taken low without **CPUReset** going high, the processor state and operation are undefined.

12.4.2 Errors

Software errors, such as arithmetic overflow or array bounds violation, can cause an error flag to be set. This flag is directly connected to the **ErrorOut** pin. The STi5512 can be set to ignore the error flag in order to optimize the performance of a proven program. If error checks are removed, any unexpected error then occurring will have an arbitrary undefined effect.

User-defined trap handlers to deal with software errors are supported, as described in [Central processing unit on page 29](#).

The STi5512 can also be set to halt on error by setting the **HaltOnError** flag. This is done by executing the *sethalterr* instruction. Halting prevents further corruption and allows postmortem debugging. In the event of a processor halting in this way, the links will finish any outstanding transfers before shutting down. If **CPUAnalyze** is asserted, then all inputs on OS-Links continue but outputs will not make another access to memory for data. Memory refresh will continue.

If a high priority process pre-empts a low priority process, the status of the **Error** and **HaltOnError** flags is saved for the duration of the high priority process and restored at the conclusion of it. The status of both flags is transmitted to the high priority process. Either flag can be altered in the process without upsetting the error status of any complex operation being carried out by the preempted low priority process.

13 Diagnostic controller

The ST20 Diagnostic Controller Unit (DCU) provides a means for booting the CPU, and for the control and monitoring of all systems on the chip, via the standard IEEE 1194.1 Test Access Port (see [Test access port on page 105](#)). The DCU includes on-chip hardware with ICE (In Circuit Emulation) and LSA (Logic State Analyzer) features to facilitate verification and debugging of software running on the on-chip CPU in real time. It is an independent hardware module with a private link from the host to support real-time diagnostics.

13.1 Diagnostic hardware

The on-chip diagnostic controller assists in debugging, while reducing or eliminating the intrusion into the target code space, the CPU utilization, and impact on the application. As shown in the figure below, the DCU and TAP provide a means of connecting a diagnostic host to a target board with a suitable JTAG port connector and interface.

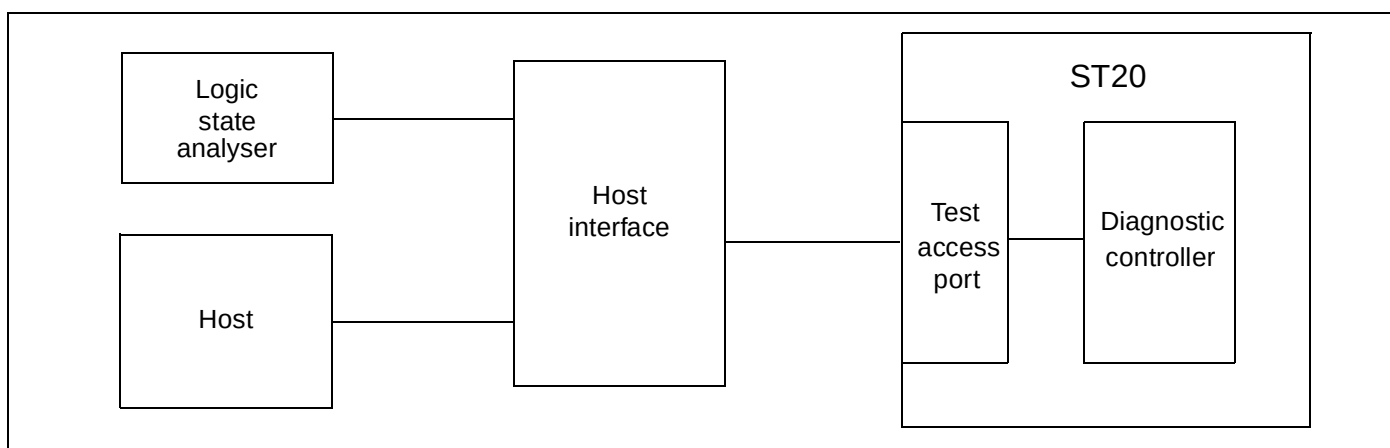


Figure 49 Debugging hardware

The diagnostic controller provides the following facilities for debugging from a host:

- control of target CPU and subsystems including CPU boot
- hardware breakpoint, watchpoint, datawatch and single instruction step
- complex trigger sequencing and choice of subsequent actions
- non-intrusive jump trace and instruction pointer profiling
- access to the memory of the target while the device is powered up, regardless of the state of the CPU
- full debugging of ROM code

When running multi-tasking code on the target, one or more processes can be single-stepped or stopped while others continue running in real time. In this case, the running threads can be interrupted by incoming hardware interrupts, with a low latency.

The host can communicate with the DCU via a private link, using the 5 standard test pins.

Target software also has access to the diagnostic facilities and access through the DCU to the host memory.

A logic state analyzer can be connected to the **TriggerIn** and **TriggerOut** pins. The response to **TriggerIn** and the events that cause a **TriggerOut** signal can be controlled by the host or by target software.

The diagnostic controller provides debugging facilities with much less impact on the software and target performance. In particular it gives:

- non-intrusive attachment to the host system
- no intrusion into the performance of the CPU or any subsystems
- no intrusion into the code space, so the application builder does not need to add a debugging kernel
- no intrusion into any on-chip functional modules, including any communications facilities
- no functional external connection pins are used

The connections between the diagnostic controller and other on-chip modules and external hardware may vary between ST20 variants.

13.2 Access features

13.2.1 Access to target memory and peripheral registers from host

Full read and write access to the entire on-chip and external memory space and the register space is available via the TAP. This is independent of the state of the CPU.

13.2.2 Access from target CPU process

The CPU itself can program its own diagnostic controller. Further access may be explicitly prevented by the lock mechanism so that the application being debugged cannot interfere with the breakpoint and watchpoint settings. When the breakpoint or watchpoint match occurs, then the diagnostic controller may release the lock according to settings in the control register.

13.2.3 Access to host memory from target

If the target CPU accesses any address in the top half of the DCU memory space, then these accesses are mapped on to host memory via the TAP as target initiated peek and poke messages. Peek accesses and poke accesses are specifically enabled by separate property bits.

13.3 Software debugging features

13.3.1 Control of the target CPU including boot

Various state information about the target CPU may be monitored and the CPU may be controlled from the diagnostic controller via the TAP. The control of the CPU extends to stalling, forcing a trap and booting.

13.3.2 Non-intrusive IPTR profiling

A copy of the **IPTR** is visible as a read-only register in the diagnostic controller. This register may be read at any time. Reading this register is not intrusive on the CPU or its memory space.

13.3.3 Events

Support is provided by the diagnostic controller to trigger actions when certain predefined events occur.

Breakpoint

The function of the breakpoint is to break before the instruction is executed, but only if it really was going to be executed. A 32-bit comparator is used to compare the breakpoint register against the instruction pointer of the next instruction to be executed. The matched instruction is not executed and the CPU state, including all CPU registers, is defined as at the start of the instruction. The previous instruction is run to completion.

Breakpoint range

The function of a breakpoint range is equivalent to any single breakpoint but where the breakpoint address can be anywhere within a range of addresses bounded by lower and upper register values.

Watchpoint

The function of a watchpoint is to trigger after a memory access is made to an address within the range specified by a pair of 32-bit registers. The CPU pipeline architecture allows for the CPU to continue execution of instructions without necessarily waiting for a write access to complete. So, by the time a watchpoint violation has been detected, the CPU may have executed a number of instructions after the instruction which caused the violation. If the subsequent action is to stall the CPU or to take a hardware trap, then the last instruction executed before the stall or trap may not be the instruction which caused the violation.

Datawatch

The function of a datawatch is to trigger after a data value specified in one 32-bit register is written to a memory word address specified in another 32-bit register. The subsequent action is equivalent to a watchpoint.

Choice of subsequent actions

Following a watchpoint match, or any other condition detectable by the diagnostic controller, the subsequent action may be programmed to be one of the following:

- stall the CPU, i.e. inhibit further instructions from being executed by the CPU
- wait until the end of the current instruction, then signal a hardware trap
- signal an immediate hardware trap
- continue without intrusion

In addition, the diagnostic controller may take any combination of the following actions:

- signal on **TriggerOut** to a logic state analyzer
- send a *triggered* message via the TAP to the host
- unlock access by the target CPU

13.3.4 Hardware single instruction step

The function of single stepping one CPU instruction is performed by using a breakpoint range over the code to be single stepped. The DCU includes a mechanism to prevent the breakpoint trap handler single-stepping itself. By selecting an inverse range, the effect of single stepping one high level instruction can be achieved.

13.3.5 Jump trace

Jump tracing monitors code jumps, where a jump is any change in execution flow from the stream of consecutive instructions stored in memory. A jump may be caused by a program instruction, an interrupt or a trap.

When the jump occurs, a 32-bit DCU register is loaded with the origin of the jump. This value points to the instruction which would have been executed next if the jump had not occurred. The CPU may not have completed the instruction prior to the change in flow. The diagnostic controller can be set to trace the origin of each jump, the destination, or both.

The DCU copies the details of each jump to a rolling trace buffer in memory. The trace buffer may be located in host memory, but using target memory will have less impact on performance. The tracing facility has two modes:

- Low intrusion. In this mode the DCU uses dead memory cycles to write the trace into the buffer. This means that the CPU is not delayed, but some trace information may be lost.
- Complete trace. In this mode, the CPU is stalled on every jump to ensure the data can be written to the buffer. This means that no trace information is lost, but the CPU performance is affected.

13.3.6 Logic state analyzer (LSA) support

Two signals, **TriggerIn** and **TriggerOut**, are provided to support diagnostics with an external LSA. The action by the DCU on receiving a **TriggerIn** signal is programmable. The selection of internal events which trigger a **TriggerOut** signal is also programmable.

13.3.7 Trigger combinations and sequences

Complex trigger conditions can be programmed. For example:

- the 5th time that breakpoint 3 is encountered
- enable a watchpoint when a breakpoint occurs

There is no software intrusion imposed by this mechanism

13.3.8 Controlling the diagnostic controller

This section gives a summary of host communications with the diagnostic controller.

The diagnostic controller has direct access to:

- the instruction pointer
- a selection of CPU state control signals
- the memory bus
- memory-mapped peripheral configuration registers

This access does not depend on the state of the CPU. Access to non-memory-mapped peripheral configuration registers is via the CPU, and for this the CPU must be active and running the appropriate handler.

The host can give two commands to the diagnostic controller: *peek* and *poke*. *Peek* reads memory locations or configuration registers, and *poke* writes to memory locations or configuration registers. The diagnostic controller responds to a *peek* command with a *peeked* message, giving the contents of the peeked addresses.

The diagnostic controller has registers, which are accessed from the host using *peek* and *poke* commands. The registers are used to control breakpoints, watchpoints, datawatch, tracing and other facilities.

The target CPU can also access these registers using the normal load and store instructions, so the target software running on the CPU can program its own diagnostic controller. A lock is provided to prevent CPU access, which can be released by the diagnostic controller when a breakpoint or watchpoint match occurs.

In addition, the target CPU can *peek* and *poke* the host via the diagnostic controller by reading or writing addresses in the top half of the memory space of the diagnostic controller. This facility can be disabled.

Various different types of CPU events can be selected as *trigger events*. When an trigger event occurs, the diagnostic controller can send a *triggered* message.

The four types of message are summarized in Table 72. The messages are distinguished by the two least significant bits of the message header byte.

Message type	Direction	Bit 1	Bit 0	Meaning
poke	Command.	0	0	Write to one or more addresses.
peek	Command.	0	1	Read from one or more addresses.
peeked	Opposite to <i>peek</i> command.	1	0	The result of a <i>peek</i> command.
triggered	DCU to host.	1	1	A trigger event has occurred.

Table 72 Types of diagnostic controller message

Messages may be initiated from either the host or the target. Target initiated messages, which constitute asynchronous or unsolicited messages, can be enabled by a property bit.

Messages are composed of a header byte followed by zero or more data bytes, depending on the type of message. The formats for the four message types are shown in the figure below.

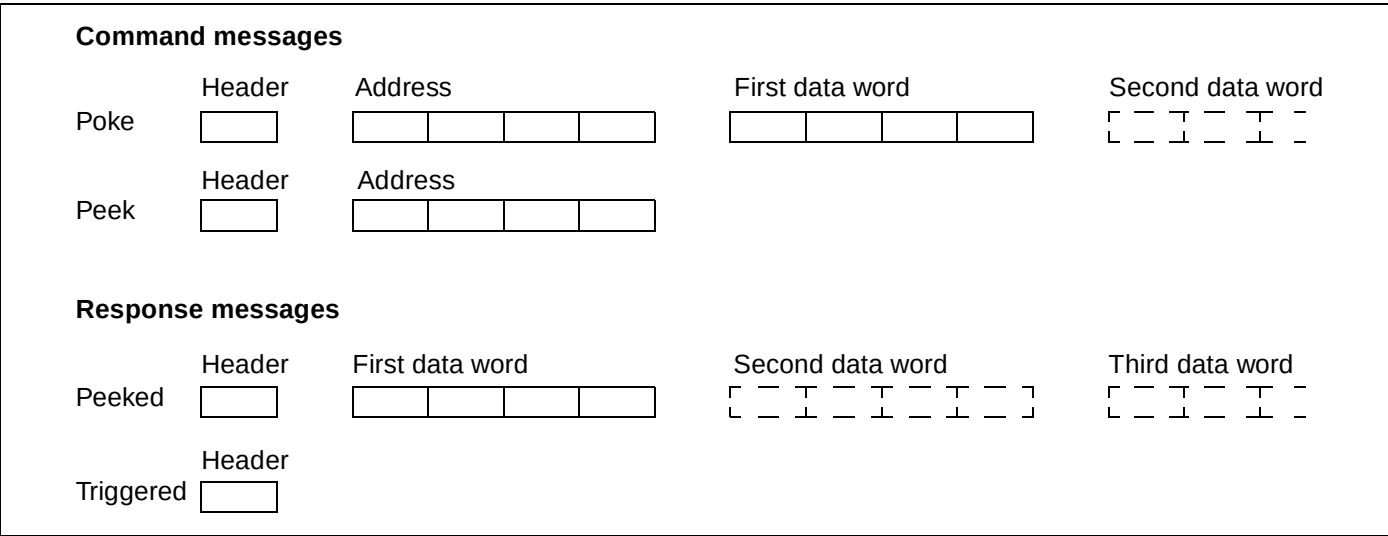


Figure 50 Message formats

13.4 Peeking and poking the host from the target

The target CPU can *peek* and *poke* the host via the diagnostic controller. This is done by reading or writing a single word to a block of addresses within the DCU register block. The DCU will then send a *peek* or *poke* message to the host. After a host *peek*, the target CPU will wait until the host responds with a *peeked* message, which the DCU returns to the CPU as memory read data.

Peeking and poking the host from the target can be enabled or disabled. After reset, these bits are cleared, so peek and poke from the target are disabled.

14 Test access port

The STi5512 Test Access Port (TAP) conforms to IEEE standard 1149.1.

The TAP has pins as listed in the table below. **TDO** can be over-driven to the power rails, and **TCK** can be stopped in either logic state. None of the TAP pins has an internal pull-up.

Pin	In/Out	Function
TDI	in	Test data input.
TDO	out	Test data output.
TMS	in	Test mode select.
TCK	in	Test clock.
notTRST	in	Test logic reset.

Table 73 STi5512 TAP pins

The instruction register is 5 bits long with no parity. The pattern “00001” is loaded into the register during the *Capture-IR* state.

There are four defined public instructions, see the table below. All other instruction codes are reserved.

Instruction code ¹					Instruction	Selected register
0	0	0	0	0	EXTEST	Boundary-Scan
0	0	0	1	0	IDCODE	Identification
0	0	0	1	1	SAMPLE/PRELOAD	Boundary-Scan
1	1	1	1	1	BYPASS	Bypass

Table 74 Instruction codes

1. MSB... LSB; LSB closest to **TDO**.

There are three test data registers; **Bypass**, **Boundary-Scan** and **Identification**. These registers operate according to 1149.1. The operation of the **Boundary-Scan** register is defined in the BSDL description.

The identification code is #MD4CD041, where M is the 4-bit silicon revision number.

bit 31				bit 0 ¹			
Mask rev	²	ST20 family	Variant	STMicroelectronics manufacturers id		³	
M				0	4	1	

Table 75 Identification code

- Closest to **TDO**.
- 0 indicates STMicroelectronics part, 1 indicates customer part.
- Defined as 1 in IEEE 1149.1 standard.

15 Serial link interface (OS-Link)

The STi5512 has an OS-Link based serial communications subsystem. The OS-Link is used to provide serial data transfer and its main function is for booting the device and debugging during software development.

The OS-Link is a serial communications engine consisting of two signal wires, one in each direction. OS-Links use an asynchronous bit-serial (byte-stream) protocol, each bit received is sampled five times, hence the term *oversampled links* (OS-Links). The OS-Link provides a pair of channels, one input and one output channel.

The OS-Link is used for the following purposes:

- Bootstrapping - the program which is executed at power up or after reset can reside in ROM in the address space, or can be loaded via the OS-Link directly into memory.
- Diagnostics - diagnostic and debug software can be downloaded over the link connected to a PC or other diagnostic equipment, and the system performance and functionality can be monitored.

15.1 OS-Link protocol

The quiescent state of a link output is low. Each data byte is transmitted as a high start bit followed by a one bit followed by eight data bits followed by a low stop bit (see [Figure 51](#) below). The least significant bit of data is transmitted first. After transmitting a data byte the sender waits for the acknowledge, which consists of a high start bit followed by a zero bit. The acknowledge signifies both that a process was able to receive the acknowledged data byte and that the receiving link is able to receive another byte. The sending link reschedules the sending process only after the acknowledge for the final byte of the message has been received. The link allows an acknowledge to be sent before the data has been fully received.

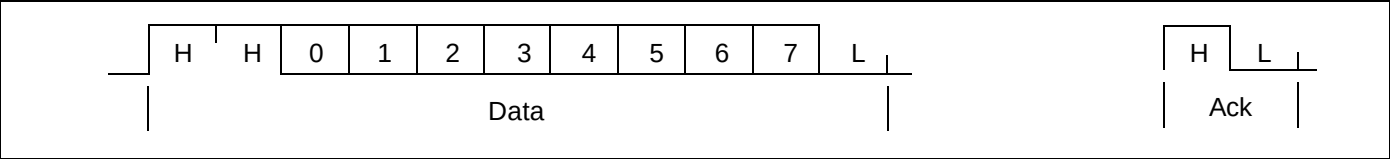


Figure 51 OS-Link data and acknowledge formats

15.2 OS-Link speed

The OS-Link data rate is 19.98 Mbits/s, but it will operate correctly when connected to 20 Mbits/s OS-Links.

15.3 OS-Link connections

Links are TTL compatible and intended to be used in electrically quiet environments, between devices on a single printed circuit board or between two boards via a backplane. Direct connection may be made between devices separated by a distance of less than 300 mm.



For longer distances a matched 100 ohm transmission line should be used with series matching resistors (RM), see [Figure 53](#). The value of RM to match a 100Ω transmission line is 75Ω. When this is done the line delay is less than 0.4 bit time to ensure that the reflection returns before the next data bit is sent.



Figure 52 OS-Links directly connected

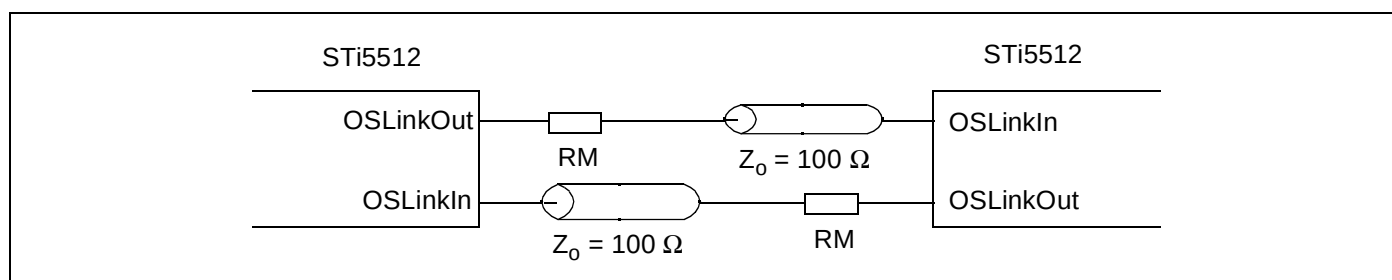


Figure 53 OS-Links connected by transmission line

16 Data flow

This chapter describes the normal data flow through the STi5512 from the incoming transport stream to the outgoing analog video and PCM audio. It shows how the picture and sound modules of the part are used together. The individual modules are described in the appropriate chapters.

16.1 On-chip modules

The STi5512 reads in an MPEG-2 transport stream, demultiplexes it, decodes the audio and video elementary streams and creates a video picture and audio PCM.

Demultiplexing extracts the video and audio MPEG streams plus other PES data such as teletext and DVB subtitles. Hardware modules are provided on-chip for decoding the MPEG video and audio. The data before decoding is called compressed data (CD), and digital video data after decoding is called pixel data.

The on-chip modules processing the compressed data streams from the incoming transport stream to the decoders are shown in the figure below.

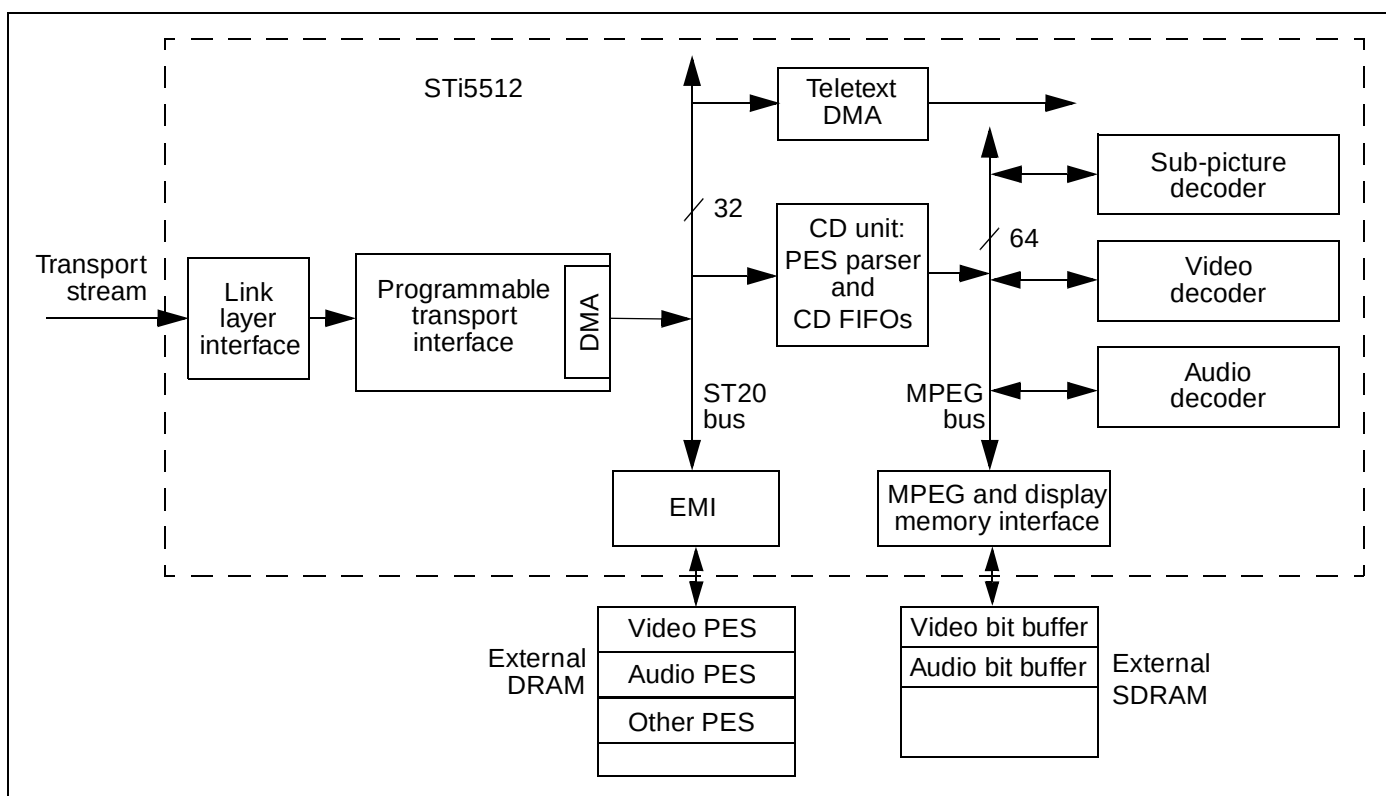


Figure 54 Compressed data modules

The on-chip modules processing the decoded data from the decoders to the video and audio output are shown in the figure below.

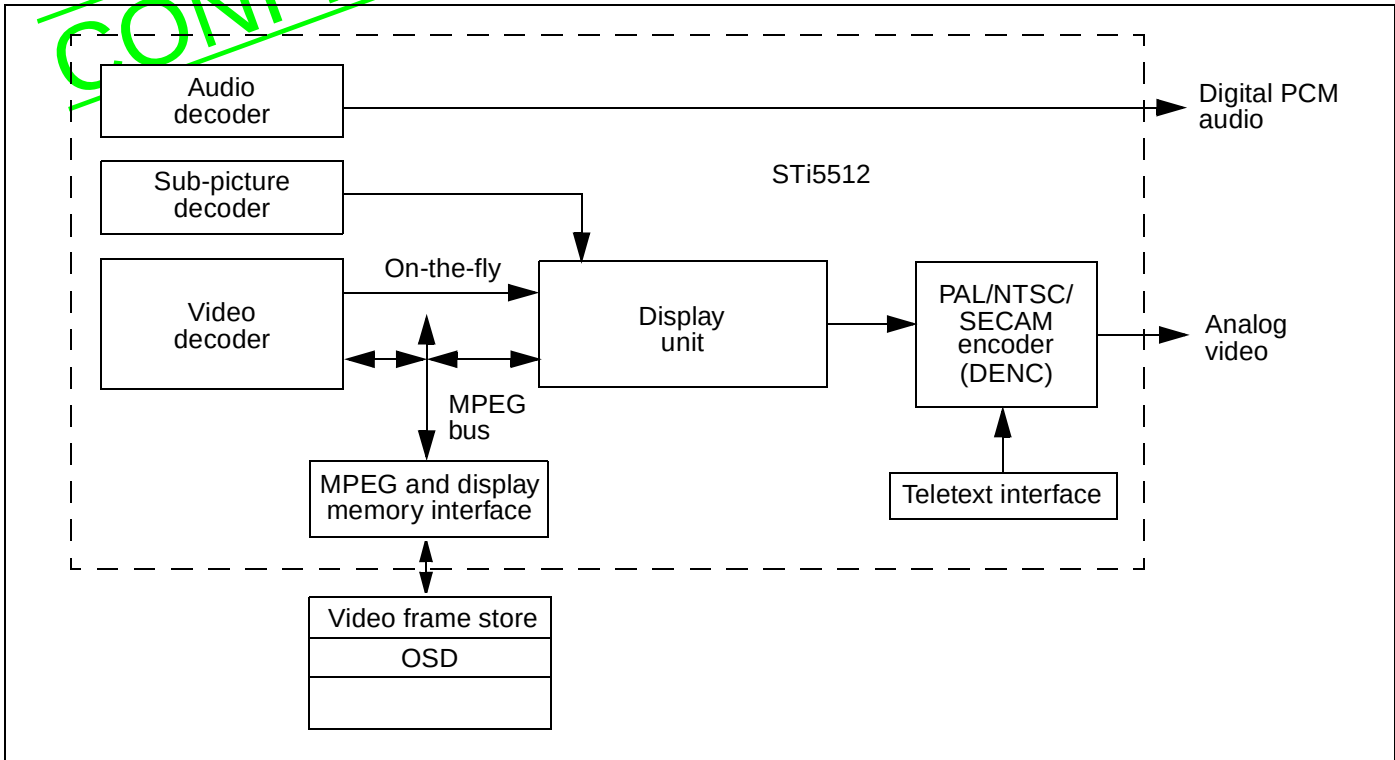


Figure 55 Decoded data modules

16.2 Video data flow

The data flow for MPEG-2 video streams is summarized in the figure below. Rectangular boxes represent processing modules, and rounded boxes represent buffers and FIFOs.

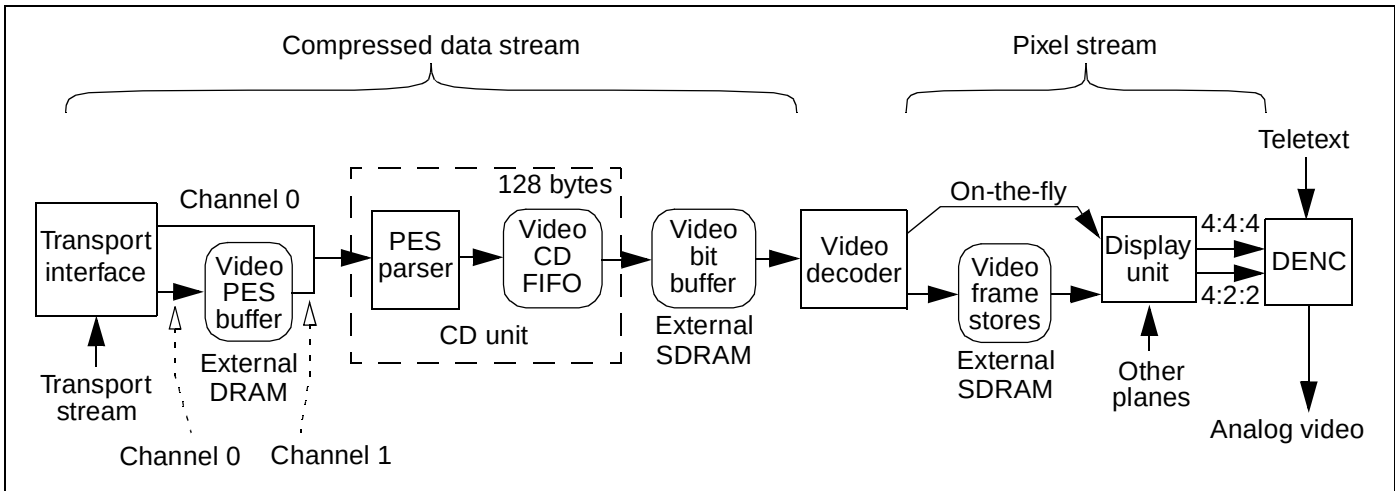


Figure 56 MPEG-2 video data flow

The compressed data is read in as a transport stream by the link layer interface (LLI). The stream may be read from the IEEE 1394 AV link layer interface or the channel decoder interface. The transport stream is normally passed directly into the programmable transport interface (PTI), as described in [Programmable transport interface \(PTI\) on page 118](#).

The PTI demultiplexes the stream, extracting the required packets. Any selected video stream PES packets are sent on by a high speed DMA engine included in the PTI module. This four-channel DMA engine requires very little intervention by the CPU.

The stream can be either:

- written into a circular video PES buffer in external DRAM by DMA channel 0 and then read into the CD unit by DMA channel 1 or
- sent directly to the CD unit by DMA channel 0.

Video data enters the CD unit through the PES parser, which passes the data to the video CD FIFO. The video CD FIFO holds 128 bytes and writes 512-bit bursts into the video bit buffer in external SDRAM. The video decoder (described in [MPEG video decoder on page 129](#)) reads 1024-bit bursts from the video bit buffer. It decodes the compressed bit stream and produces a pixel stream. I-frames and P-frames must be written into video frame stores, while B-frames may be either written into a frame store or sent 'on-the-fly' directly to the display unit.

The display unit is described in [Display planes on page 149](#). It converts the blocks of pixels into rows and performs filtering and pan/scan. It then mixes the video with the other display planes and sends two pixel streams to the on-chip PAL/NTSC/SECAM encoder (DENC). One pixel stream is in 4:4:4 format and is generally used for TV display, while the other is in 4:2:2 format, and is generally used for output to a VCR. The DENC is described in [Digital encoder on page 190](#). It converts the pixel streams into analog signals for output from the device. The 4:4:4 pixel stream is converted into YUV and RGB signals, and the 4:2:2 pixel stream is converted into CVBS and YC signals. Teletext can be inserted into the output signals.

16.3 Audio data flow

The data flow for audio streams is summarized in [Figure 57](#) below. Rectangular boxes represent processing modules, and rounded boxes represent buffers and FIFOs.

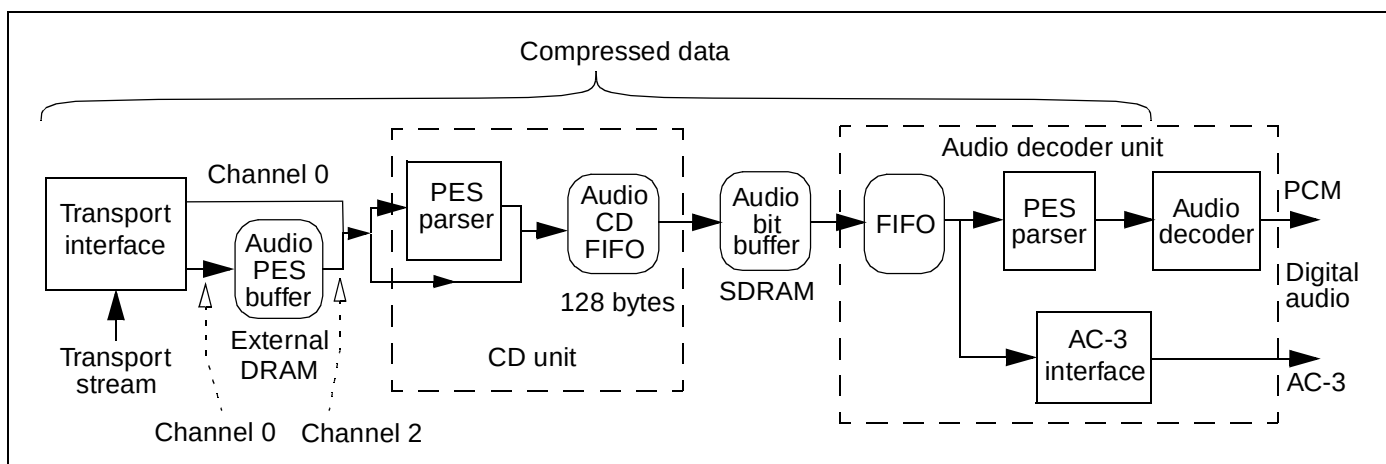


Figure 57 Audio data flow

As for video, the compressed audio data is read in as a transport stream by the transport interface. The transport interface demultiplexes the stream and extracts the required audio PES packets. These are either:

- sent directly to the CD unit by DMA channel 0 or
- written into a circular audio PES buffer in external DRAM by DMA channel 0 and then read into the CD unit by DMA channel 2.

If the audio data is in the form of MPEG-2 packetized PES, then the data must be sent directly to the audio CD FIFO, in which case it will not pass through the PES parser in the CD unit. Otherwise, the audio data is interleaved with the video data, which will be sent on the video route through the PES parser. The PES parser will separate out any audio packets

and route them to the audio CD FIFO. The audio CD FIFO writes 512-bit bursts into the audio bit buffer in external SDRAM. The CD unit is described in [Chapter 19: MPEG video decoder](#) on page 129. The audio decoder (described in [Chapter 26: MPEG audio decoder with AC-3 interface](#) on page 222) unit includes its own FIFO and PES parser. It reads from the bit buffer into its FIFO, and then the data may be either:

- passed to the PES parser, which passes the data on to the audio decoder for decoding into PCM data for output or
- passed to an AC-3 interface to send to an external Dolby AC-3 decoder.

17 Link layer interface (LLI)

17.1 Introduction

The LLI block operates as a multiplexer, selecting data streams from a number of sources and routing them to the correct subsystem and pins. It has an input port for transport streams, called **TSIn**, and a bidirectional high speed data port, called the LLI port.

The LLI port can be configured as an IEEE 1284 interface or an interface to an external IEEE 1394 Link Layer Controller. The LLI pins are shared between these functions. In this chapter, the port may be referred to as the IEEE 1394 port or the IEEE 1284 port depending on the context.

The LLI also supplies a locally generated byte clock derived from the system clock, with a frequency down to 4.2 MHz.

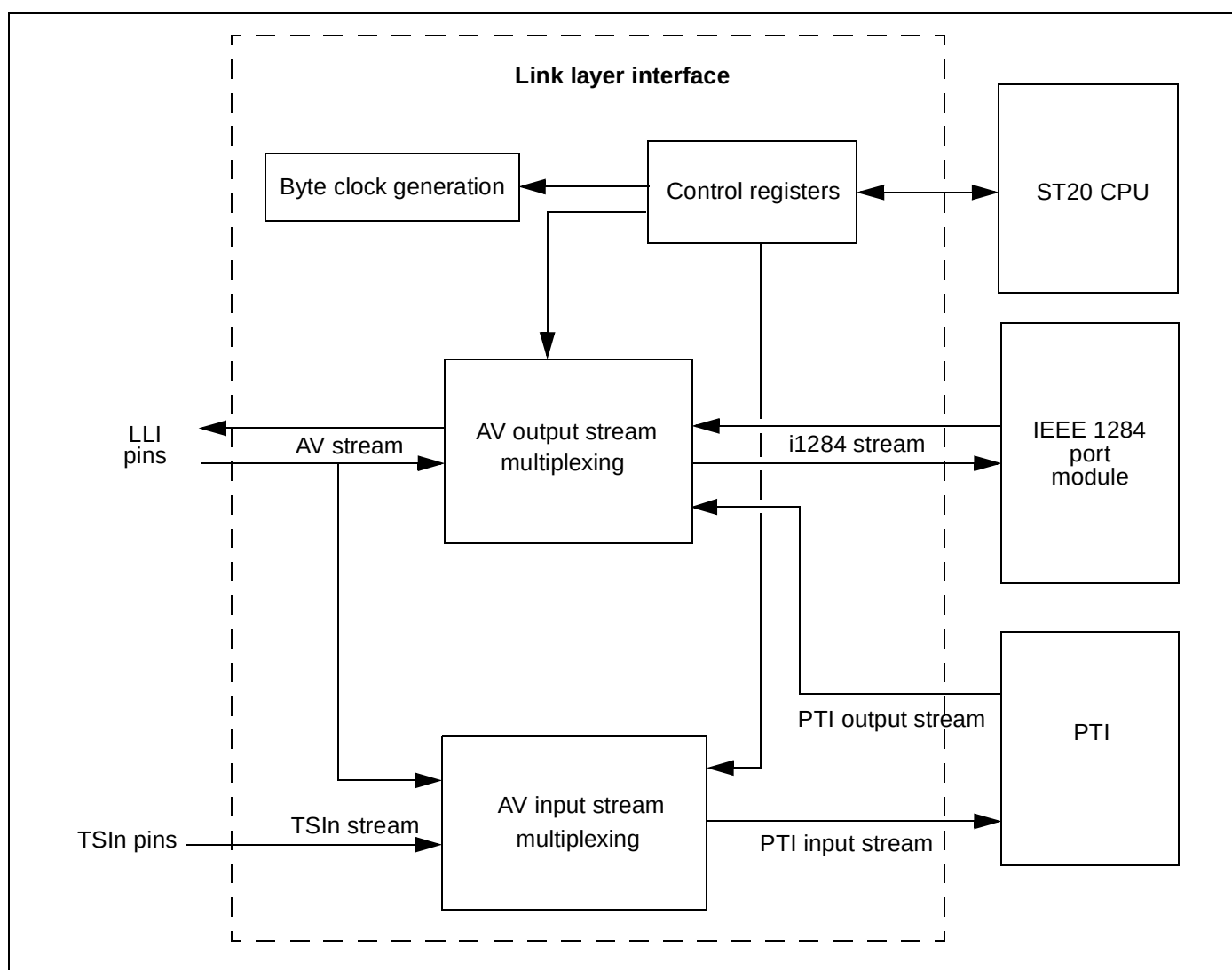


Figure 58 Architecture of the LLI

17.2 Data streams

The LLI multiplexes several data streams. These streams are described in the remainder of this section.

AV stream

The AV stream is the data passing between the IEEE 1394 pins and the AV input and output stream multiplexers. It may be an input or output stream, and may be clocked by either the external TSIn stream clock or by a locally generated byte clock. Depending on the settings of the **LLIControl** register bits, the AV stream may be:

- An outgoing stream from the PTI output port to the IEEE 1394 pins,
- An incoming transport stream on the IEEE 1394 pins going to the PTI input,
- A stream of IEEE 1284 signals from the IEEE 1284 port module to the IEEE 1284 pins,
- A stream of IEEE 1284 signals from the IEEE 1284 pins to the IEEE 1284 port module,
- A transport stream from the IEEE 1284 port module.

The input and output AV stream multiplexers are independent and the AV stream input is taken directly from the IEEE 1394 pins, so that some loop-back modes are possible. For example, a transport stream output from the IEEE 1284 port module can be input to the PTI.

PTI input stream

The PTI input stream is the stream from the LLI to the PTI.

PTI output stream

The PTI output stream is the transport stream from the PTI block to the AV output multiplexer. It consists of the same signal group as the input stream, with the addition of **PacketTag0-3** signals but without the **PacketError** signal.

i1284 stream

The i1284 stream is defined as the transport or IEEE 1284 format information stream from the IEEE 1284 port module.

TSIn stream

The TSIn stream is defined as the stream from the **TSIn** pins to the AV input multiplexer. The direction of the **TSIn** pins is always input. This stream can be serial or parallel.

17.3 Stream multiplexing

The LLI can operate in two basic modes, namely 1284 mode and AV mode.

17.3.1 1284 mode

In 1284 mode the signals from the IEEE 1284 port pins are directly routed to the IEEE 1284 port module. The IEEE 1284 port module then has full control of the IEEE 1284 pins and their direction. 1284 mode is selected by setting the **AVnot1284** bit in the **LLIControl** register to 0. In this mode, the setting of the **AVOutnotIn** bit has no effect.

The PTI output stream is disconnected when the LLI is in 1284 mode. The PTI input stream can be either an AV stream from the IEEE 1284 port module or a TSIn stream from the **TSIn** pins:

- If **AVnotTS** is 1, the transport stream from the IEEE 1284 port module is fed to the PTI. The IEEE 1284 port module must be programmed to output a transport stream to the pins. The outgoing transport stream is looped back via the pins and sent to the PTI as an AV input stream.
- If **AVnotTS** is 0, the transport input from the **TSIn** pins is fed to the PTI. The IEEE 1284 port operates in normal IEEE 1284 mode.

17.3.2 AV mode

AV mode is selected by setting the **AVnot1284** bit in the **LLIControl** register to 1. In AV mode, the IEEE 1394 port pins are routed either to the PTI output or PTI input stream. When operating in AV mode, the direction of the IEEE 1394 port pins is selected by the **AVOutnotIn** bit in the **LLIControl** register:

- When **AVOutnotIn** is 1, the PTI output stream is routed to the IEEE 1394 port pins. The pins will be configured in output mode.
- When **AVOutnotIn** is 0, the IEEE 1394 port pins receive the transport stream from the external IEEE 1394 device. If **AVnotTS** equals 1, the transport stream from the external IEEE 1394 port will be routed to the PTI input stream. If **AVnotTS** equals 0, the **TSIn** stream will be routed to the PTI input stream.

17.3.3 IEEE 1394 pin functions

The function and direction of data flow through the IEEE 1394 pins depends on the setting of the bits in the **LLIControl** register. Table 76 shows the functions of each of the IEEE 1394 pins with different settings of the **LLIControl** register bits.

External pin name	Function and pin direction					
	AVnot1284=0 (1284 mode)		AVnot1284=1 (AV mode)			
			AVOutnotIn=0		AVOutnotIn=1	
1284Data0-7	1284Data0-7	I/O	AVInputData0-7	I ¹	AVOutputData0-7	O
1284notSelectIn	1284notSelectIn	I	No function	I	Gnd	O
1284notInit	1284notInit	I	No function	I	AVPacketTag3	O
1284notFault	1284notFault	O	No function	I	AVPacketTag2	O
1284notAutoFd	1284notAutoFd	I	No function	I	AVPacketTag1	O
1284Select	1284Select	O	No function	I	AVPacketTag0	O
1284PErr	1284PErr	O	AVInputByteClkValid	I ¹	AVOutputByteClkValid	O
1284Busy	1284Busy	O	AVInputPacketClk	I ¹	AVOutputPacketClk	O
1284notAck	1284notAck	O	AVByteClk	O ²	AVByteClk	O ²
1284notStrobe	1284notStrobe	I	AVInputPacketError	I ¹	Gnd	O

Table 76 Pin functions

¹ These signals are only fed into the PTI inputs when the **LLIControl** register bit **AVnotTS** is set to 1.

² The **ByteClk** output on this pin is selected by the **LLIControl** register bit **LocalnotTSClock**. Setting this register bit to 0 selects the clock input on the **TSByteClock** pin to be output on this pin. Setting the register bit to 1 selects the local byte clock generator signal to be output on this pin.

17.3.4 Byte clock

The IEEE 1394 byte clock is always output. It can either be generated locally or routed from the **TSIn** input byte clock, according to the setting of the **LocalnotTSClock** bit in the **LLIControl** register. The internal byte clock is free-running and is programmed to have a period which is a fixed multiple of the system clock period, as defined by the byte clock ratio register.

Changing the clock source during operation may lead to glitches occurring on the clock. Changes should therefore only be performed before initialization of any external IEEE 1394 LLI devices which use the clock. The local byte clock is automatically selected when inputting serial streams to the **TSIn** pins, as described in Section 17.4.

17.4 Serial and parallel transport streams

The stream coming into the **TSIn** pins can be either in serial or parallel format. The format is selected by the **SernotPar** bit of the **LLIControl** register. Any transport stream from or to the IEEE 1394 port pins is always parallel. The function of the **TSIn** pins depends on whether the transport mode is serial or parallel. [Table 77](#) shows the functions of the **TSIn** pins in each mode.

External pin	Parallel mode (SernotPar = 0)	Serial mode (SernotPar = 1)
TSInByteClk	ByteClk	BitClk
TSInByteClkValid	ByteClkValid	BitClkValid
TSInData<7:0>	Data<7:0>	Data<7>
TSInError	Error	BitError
TSInPacketClk	PacketClk	PacketClk

Table 77 Usage of transport pins in parallel and serial mode

17.4.1 Parallel streams

The meanings of parallel transport input signals are given in [Table 78](#).

Signal	Description
Data0-7	8-byte transport data. Strobed on a valid edge of ByteClk .
ByteClk	Transport clock. A byte clock rising edge is considered valid if ByteClkValid is high.
ByteClkValid	High when the byte clock is valid.
PacketClk	This signal indicates the first byte in a packet. When PacketClk is strobed low on a ByteClk rising edge (valid or not), the next valid ByteClk edge which strobes PacketClk high will clock in the first byte in the packet.
Error	This signal will be high on the first byte of a packet if the current packet is not valid. For transport output streams this signal will be kept low.

Table 78 Transport parallel input stream signals

17.4.2 Serial streams

The function of the **TSIn** pins in serial mode is as defined in [Table 79](#).

External pin	Pin function	Description
TSInData7	Data7	Transport data. Strobed on a valid edge of BitClk . The transport data enters the module MSB first.
TSInByteClk	BitClk	Transport clock. A BitClk rising edge is considered valid if BitClkValid is high on that clock edge.
TSInByteClkValid	BitClkValid	This signal validates the incoming BitClk .
TSInPacket Clk	PacketClk	This signal indicates the first bit in the packet. When PacketClk is strobed low on a BitClk edge (valid or not), the next valid BitClk edge which strobes PacketClk high will clock in the first bit in the packet.
TSInError	Error	This signal will be high on the first bit of a packet if the current packet is not valid.

Table 79 Transport serial input stream signals

When inputting serial streams, data gets converted to parallel internally. The locally generated **ByteClk** is used to enter the parallelized stream into the PTI. The frequency of the byte clock has to be programmed so that the resulting byte rate is higher than the input bit rate of the serial stream.

17.4.3 Error and packet clock

For streams which do not support a packet clock, the **PacketClk** signal should be tied low. In this case the **Error** signal will be ignored and should be tied low.

17.5 Sync byte detector

The LLI is equipped with a synchronization mechanism, which byte aligns the transport stream and can supply a packet clock to the PTI if there is no input packet clock.

Synchronization is enabled by writing a non-zero value to the **LLISyncLock** register and disabled by writing zero. By default the synchronization is disabled. When synchronization is disabled, the input packet clock is used.

It is only possible to parse a transport stream if the start of each packet is known. Many devices will supply this information in the form of a packet clock, which indicates a byte at the start of a packet. However if a packet clock is not supplied it can be derived from the data stream.

The first byte in every transport packet is a Sync byte which has the value 0x47. Every transport packet is 188 bytes long for DVB, but packet length is programmable by means of the **LLISyncPeriod** register. The synchronizer will search the stream for Sync bytes. When it is "confident" that it has found the start of packets it will output data, along with a derived packet clock. The confidence level is programmable using two registers - **LLISyncLock** and **LLISyncDrop**. These parameters control the locking to the Sync byte and losing the synchronization respectively.

Initially the detector will search every bit position in the serial input for the Sync byte, while throwing all data away. Once one is found the detector will lock onto it. In the payload of the packet it is obviously possible to get Sync byte emulation. It is therefore necessary to gain some degree of confidence that the byte found was indeed a Sync byte and not an emulated one. This is done by checking for a Sync byte at the assumed position of the header in successive packets. If **LLISyncLock** successive Sync bytes are found valid data is then output starting with that Sync. If however a non Sync byte is found before this occurs, the Sync detector will return to searching every bit position.

If valid data is being output and **LLISyncDrop** successive packet headers are found not to be Sync bytes, then the detector will stop outputting valid data and recommence the above procedure, searching every bit position for Sync bytes while throwing the data away.

The value of **LLISyncLock** is a degree of confidence that the correct Sync byte has been found and **LLISyncDrop** will be a function of the error rate in the input stream. A value of zero for **LLISyncLock** indicates that no Sync detection is needed and the input packet clock should be used.

The behavior of the Sync byte detector is shown in [Figure 59 on page 117](#).

To allow the input interface to be more flexible, the packet length is programmable by the CPU in the range 0 to 255. The value of the packet length will be stored in **LLISyncPeriod**.

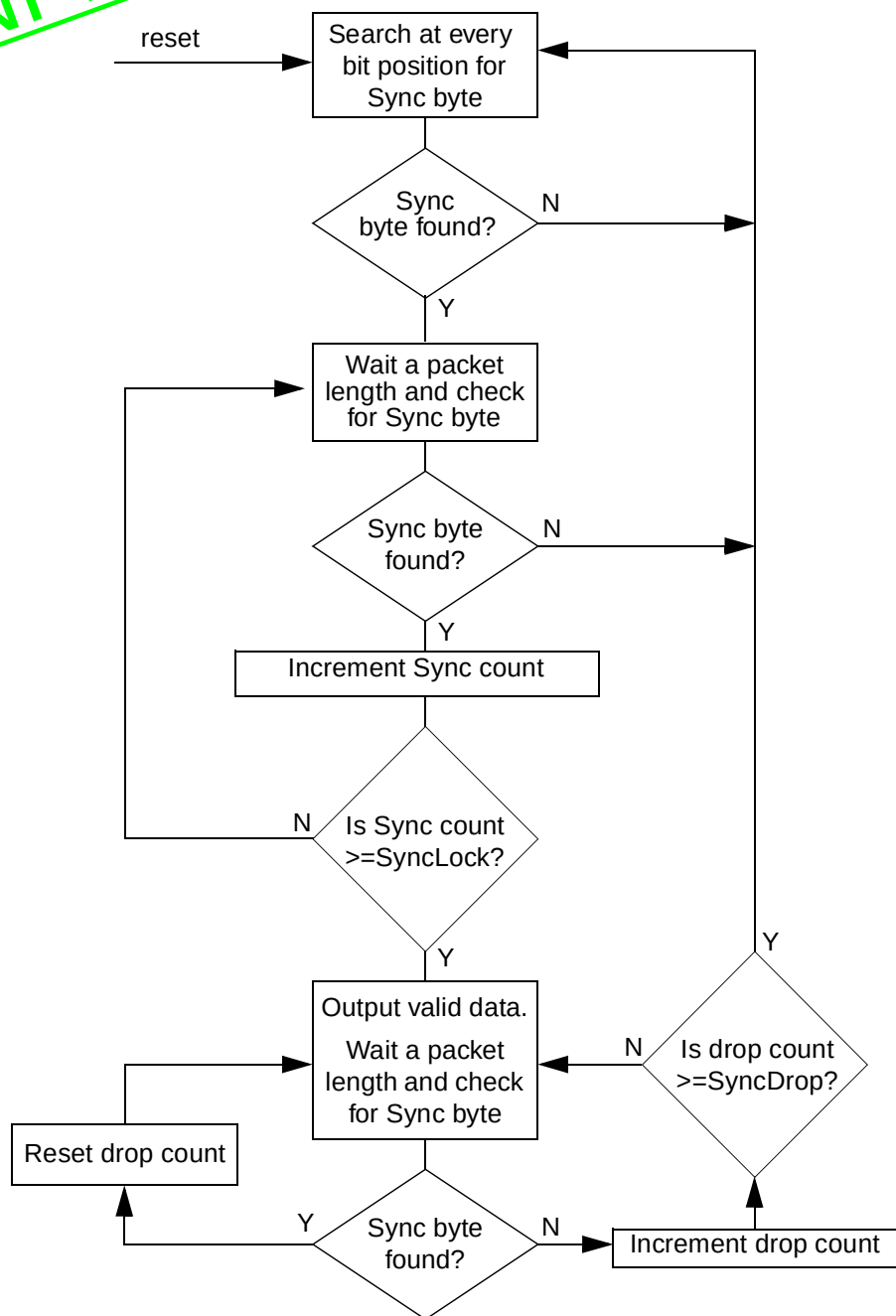
CONFIDENTIAL

Figure 59 Flow diagram of Sync detection

18 Programmable transport interface (PTI)

18.1 Introduction

The PTI module parses, descrambles and demultiplexes the transport stream, using a mixture of hardware and software running on an application specific processor called the Transport Controller (TC). The TC gives the PTI the level of flexibility normally associated with software based demultiplexing of transport streams without the overhead of this processing being placed on the ST20 CPU.

The PTI is configured and programmed by registers and two blocks of static shared memory contained within the PTI, one block containing instructions and the other data. The data block contains structures shared with the ST20 CPU plus structures private to the TC. Code for the TC can be downloaded into the PTI instruction memory by a PTI software driver running on the ST20.

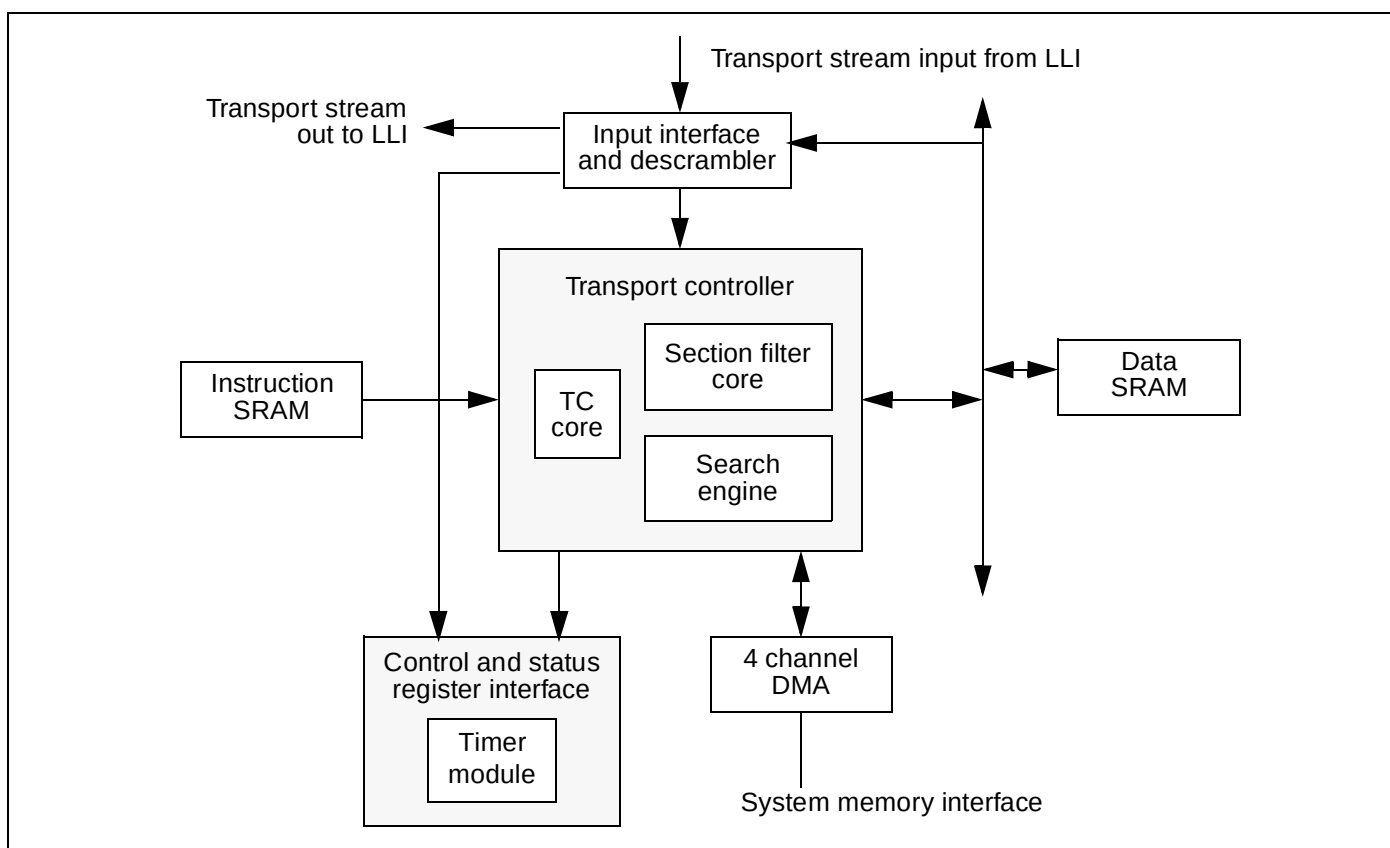


Figure 60 Programmable transport interface structure

The functionality of the PTI is therefore defined by a combination of the PTI hardware, the software running on the TC, and the software driver running on the ST20. This arrangement allows great flexibility by changing the code to be run. Many parameters of the code can be modified to change the behavior and features of the PTI. The TC code and PTI driver software are provided by STMicroelectronics. Versions of these software components are available, which support the following:

- Generic MPEG2/DVB transport stream parsing, descrambling and demultiplexing.

Specific details of the data structures and mechanisms used to communicate between the TC and the PTI driver running on the ST20 are contained in the documentation for these software components.

The remainder of this chapter is a description of typical features that can be implemented by the PTI and a detailed description of the hardware registers and memory areas within the PTI visible to the ST20 CPU software.

18.2 PTI functions

The PTI provides great flexibility, since many features can be implemented in either hardware, software or a combination of the two. What follows is a description of the features of the PTI when running the first version of the generic DVB code.

The programmable transport interface hardware performs the following functions:

- Serial and Parallel interface for transport stream input.
- Support for incoming MPEG2 transport streams with a data rate up to 60 Mbps¹.
- Framing of transport packets (sync byte detection).
- Descrambling to DVB standard - transport or PES level.
- Section filtering - 32 filters of 8 bytes in hardware.
- CRC checking of sections.
- DMA and buffering of streams in circular buffers in memory. This behavior can be changed with TC software.
- DMA of two of the streams to the audio and video MPEG decoder compressed data FIFOs either directly or via buffers in memory.

Software can control the following functions:

- PID filtering of more than 32 PIDs.
- Two descrambling keys per stream.
- Adaptation field parsing - PCR detection and time stamping.
- Special purpose section filters.
- Demultiplexing of transport stream by PID.
- Communication to ST20 CPU of buffer state.

In addition to these transport device functions, the interface can copy the entire transport stream or transport packets with selected PIDs from the transport stream through to the PTI output stream interface. Different PID groups or packet elements can be tagged using the **PacketTag0-3** signals which are output with the transport stream. This allows the input or output of transport streams using other interface standards. On the STi5512, this is used to provide a bidirectional transport stream interface to an external IEEE 1394 AV link layer device connected to the shared IEEE 1284/1394 and transport stream I/O pins.

Details of how to control these features are contained in the PTI Application Programming Interface (API) and the PTI software documentation for the particular version of the PTI code.

18.3 Operation

The programmable transport interface (PTI) performs the transport parsing and processing functions as outlined above without intervention of the ST20 CPU. The block is controlled by the ST20 and communicates with it via a shared data SRAM block local to the PTI, registers in the PTI, data structures in the ST20 memory written by the PTI and an interrupt from the PTI.

1. When processing more than 11 matching scrambled sections per packet for more than 2 consecutive packets the maximum data rate will drop to 50 Mbps.

The shared data SRAM is used to hold the main data structures for the PTI including:

- PID values;
- Descrambler keys for each PID;
- Control bits for each PID to set up DMA parameters, to mark the PCR PID, to control section CRC checking, and to mark PIDs which need copying to the selective transport output interface;
- PID state information, such as a transport/PES level descrambling flag, partial sections for filtering, partial section CRC values, and current continuity count values;
- Descriptors and pointers to the circular buffers where the streams from each PID are sent;
- The last adaptation field and its time stamp from the local system clock.

Registers are provided to allow the ST20 CPU to initialize and control the block and to provide interrupt status and control. These registers are described in detail in [Address map on page 125](#).

18.3.1 Initialization

After reset of the STi5512, the TC in the PTI is halted and the PTI block will remain idle. It stays in this state until:

- The TC code has been loaded into the instruction SRAM by the ST20;
- Initialization has been performed as described below;
- The TC enabled by setting the **TCEnable** bit of the **TCMode** register high.

There are a number of initialization steps that must be performed before the TC can be enabled.

- The data SRAM must be initialized with any data structures required by the TC software.
- The interrupt status registers must be cleared.
- The **IIFIFOEnable** register bit must be set high to enable the input FIFO.

18.3.2 Typical operation

When the TC is running, the software waits for a transport packet to arrive. Having detected the start of a packet, the TC code then sets up the PTI hardware search engine to perform PID filtering. The search engine searches a contiguous block of PID values within data SRAM for a match with the PID in the incoming transport packet header. If the packet is to be rejected, the software discards the packet and waits for the next packet to arrive. If the packet is one to be processed, the TC examines the data structure for the PID in the data SRAM to determine what other processing is required. The type of transport packet is recorded in this data structure by the ST20. The PTI driver can set variables in this data structure to configure the TC software to perform various tasks. Typical tasks might be:

- Descrambling at the transport or the PES level with a key pair;
- Section filtering, with a set of filters and section CRC checking for streams containing sections;
- Directing the output stream either to a circular buffer in memory or to a compressed data FIFO of an audio or video decoder;
- Enabling or disabling a stream.

Having examined the PID data structures, the TC sets up the rest of the hardware in the PTI to perform the required descrambling and DMA operations before starting to parse the rest of the packet. Processing will vary depending on the contents of the transport packet, which includes:

- PES data,
- Section data,
- Adaptation fields,
- Continuity count fields.

Typical processing for different packet types and fields is described in the rest of this section.

PES data

Transport packets which contain PES data and are not rejected by PID filtering, are descrambled if required. The PES data is DMA transferred either into a circular buffer or to a decoder compressed data FIFO. The DMA features of the PTI can be used to buffer a PES stream in memory and then transfer the data to a decoder without the CPU being involved. Optionally an interrupt can be generated to the ST20 when the buffer for a PES stream has data added to it and the state of the buffer changes from empty to non-empty. An interrupt will be raised and an error flag set in the data SRAM if the buffer overflows. In such cases, the most recent data will be lost.

Section data

Transport packets which contain section data and are not rejected by PID filtering, are descrambled if required and subjected to section filtering on each section or partial section in the packet.

The PTI contains a hardware section filter which implements 32 filters of 8 bytes each. These 8 bytes are used to filter against the first and fourth to the tenth bytes at the front of each section. Any subset of the 32 filters, including all or none, can be applied to any PID. When a section passes the filtering, the complete section is written to the ST20 memory space, either to a circular buffer or to defined locations for a set of sections. Sections may be split between several transport packets.

The hardware also performs the detection of the section syntax indicator bit and will perform CRC checking on a complete section. If the CRC check fails, the TC software will remove the incorrect section from the section buffer and the current PID will be discarded until a packet with a unit start indicator arrives, since the section length for the corrupted section could have been in error.

The TC software uses a bit in the interrupt status registers to raise an interrupt for the ST20 signalling a buffer having a section placed in it.

There are no restrictions on any of:

- The alignment of the sections in transport packets,
- The lengths of sections other than those in the MPEG-2 standard,
- The numbers of sections in a packet when filtering standard sections.

Section filtering can be implemented by a mixture of TC code with the hardware section filter. Alternatively, it may be preferred to perform filtering purely in TC code to implement a small number of longer or special purpose filters. In this case there may be some restrictions on the minimum length of a section or the number per transport packet, to ensure that the processing can be performed within the period of one transport packet interval.

Adaptation fields

Typically only the Program Counter Reference (PCR) would be extracted from this field although the TC software could extract other data.

If a PID is flagged as the source for PCR values then any adaptation field in a transport packet with this PID containing a PCR will have the PCR value extracted and stored in the data SRAM. The value is stored with a time stamp, which is the time when the transport packet arrived, as given by the System Time Clock (STC) counter value. An interrupt is raised to the ST20 and the interrupt bit itself is used as a handshake for the processing of the PCR by the ST20. Until the bit is cleared, no more PCRs will be captured.

The STC counter is clocked by the 27 MHz input clock to the STi5512 and can be initialized by the ST20 CPU.

Continuity count field

The TC software uses this field to check for missing transport packets. If a continuity count error is detected, the software will discard any partial units of data, such as a partially complete section, and search for a new data unit starting point.

18.3.3 DMA operations

The PTI contains a four channel DMA controller which can be programmed by the TC and the ST20 CPU. The channels are used to write or read data to or from circular buffers defined by a base and a top pointer. For each channel there is also a read and a write byte pointer.

The TC uses DMA channel 0, a write only DMA, to send the data from a transport packet to the ST20 memory space, writing the data into either a circular buffer or onto a fixed location which can be mapped onto the compressed data FIFO of an audio or video decoder. Since the TC can reload the registers of channel 0 at the start of processing each transport packet, this allows payloads from transport packets with different PIDs to be output to different buffers. It also enables the TC code to support output buffers with data structures other than circular buffers.

At the end of each transport packet, the DMA write pointer of the corresponding buffer data structure in the PTI data SRAM is updated. If the transport packet did not contain the end of a complete data unit such as a section, a temporary write pointer variable is used. This is done so that the ST20 process only sees a complete unit of data to be processed. When the data unit is complete, the write pointer used by the ST20 process is updated and an interrupt is set to signal to the ST20 process that data is in that buffer. This mechanism of updating the write pointers and interrupting, is not used in the special case that the buffers are being transferred by DMA to an audio, video or other decoder.

Channels 1 to 3 can be used to read from circular buffers and write the data to an audio or video decoder, or any other device, at a set of fixed locations in response to a request signal. The request signals **notCDReq1-3** are internal signals coming from the on-chip decoders. Each time the request signal is active and there is data to be read from the buffer (i.e. the read pointer is not equal to the write pointer) a programmable number of bytes can be transferred. This is followed by a hold-off time in which the request is not sampled, allowing the request signal time to become valid again.

These three channels can have the write pointers updated by either the TC software after data has been placed in the corresponding buffer by DMA channel 0, or by the ST20 CPU if this is writing data into the buffer that DMA channels 1-3 are reading from. Again, variables in the PID data structures in the PTI data SRAM are used to indicate to the TC software which buffers are being read by DMA channels 1-3 and therefore when to update the write pointers of channels.

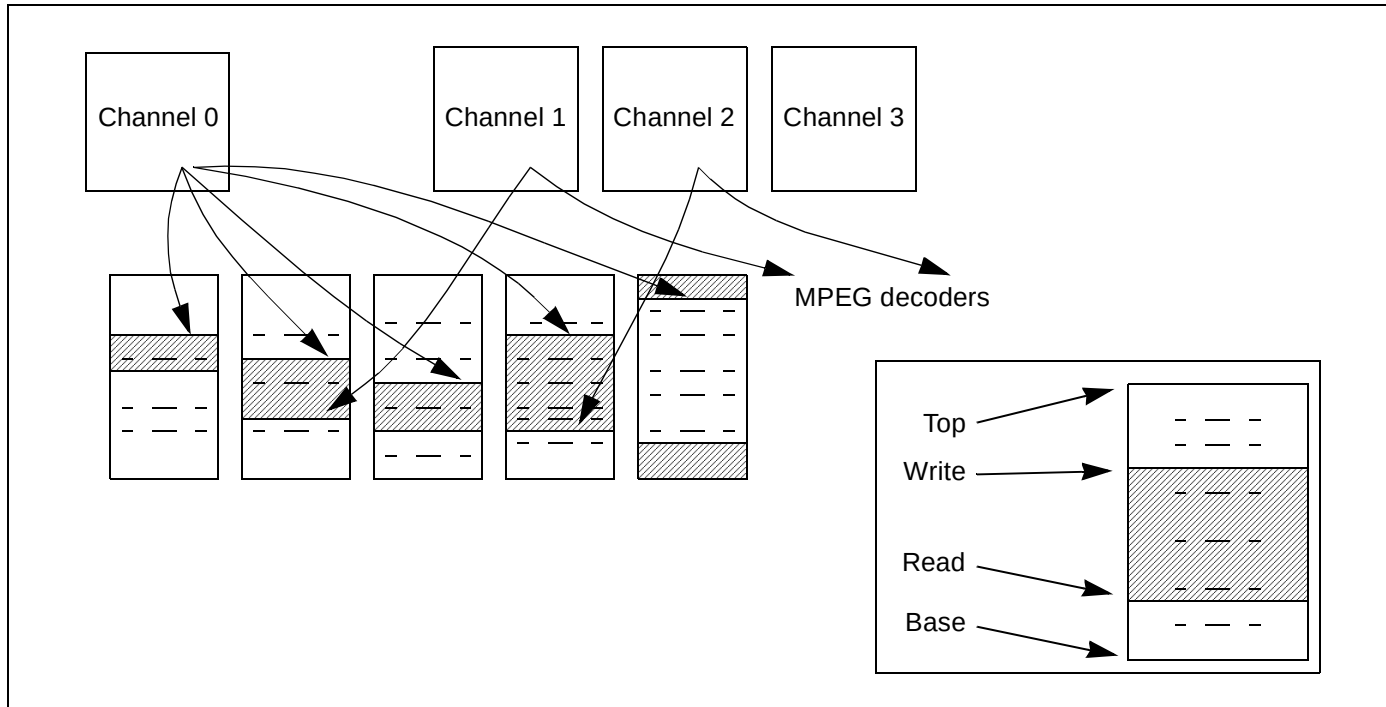


Figure 61 DMA channels

18.3.4 Interrupt handling

The PTI may interrupt the ST20 under a number of conditions, for example when a DMA operation block changes a buffer from being empty to non-empty or in the case of an error condition. The interrupt is generated by the TC writing into one of the 64 interrupt status register bits. These bits are ORed together to produce one interrupt and fed to the interrupt controller via the interrupt level controller. The ST20 CPU can then use this interrupt to schedule a process to deal with this condition.

Any of the 64 interrupt status bits can be disabled from producing an interrupt by resetting the corresponding **InterruptEnable** bit. Using this mechanism, an ST20 process can process a buffer until the process believes the buffer to be empty (by reading the write pointer for that buffer held in the data SRAM). The process will then clear the status bit which corresponds to that buffer by writing a 1 to the corresponding interrupt acknowledge register bit.

The detection of the empty condition on a buffer and the acknowledge of an interrupt does not lock out the TC from writing the write pointer after the ST20 has checked, and setting the interrupt status bit before the ST20 has acknowledged. It is therefore necessary to re-confirm the buffer state before waiting on a semaphore for that buffer. Re-checking the write pointer avoids data being left in the buffer until the next data arrives and the TC sets the interrupt again. If the buffer is still empty then the ST20 process will enable the interrupt by setting the correct interrupt enable bit before making the process wait on a semaphore.

Figure 62 shows the TC and the ST20 processes and the mechanism described above. Note that at any given time the processes can be at any point with respect to each other during the critical regions of code and there is no implied time for each step of a process only an ordering of steps.

After the buffer has been refilled the TC will set the interrupt status bit causing the PTI interrupt handler to be run. When the interrupt handler finds the buffer process semaphore status bit is set then the interrupt handler will signal to the semaphore to restart the process and disable that interrupt bit. Therefore, the process itself disables the interrupt at the PTI level, and only enables it when it is about to “sleep”.

An error condition would be handled in a similar manner.

The TC sets a given interrupt by writing a ‘1’ in the appropriate bit position in the appropriate **SetInterrupt** register. Any bits where a ‘0’ is written are not changed. The ST20 may reset interrupts by writing a ‘1’ in the appropriate bit position in the appropriate **PTIIntAck** register. Again, any bits where a ‘0’ is written are not changed. Interrupts may be enabled by the ST20 by writing ‘1’ or disabled by writing ‘0’ into the appropriate bit position in the appropriate **PTIIntEnable** register. The ST20 CPU can determine which interrupts are currently set by reading the appropriate **PTIIntStatus** register.

The association of interrupt bits with particular conditions and events is determined by the TC code and the corresponding PTI driver running on the ST20 CPU.

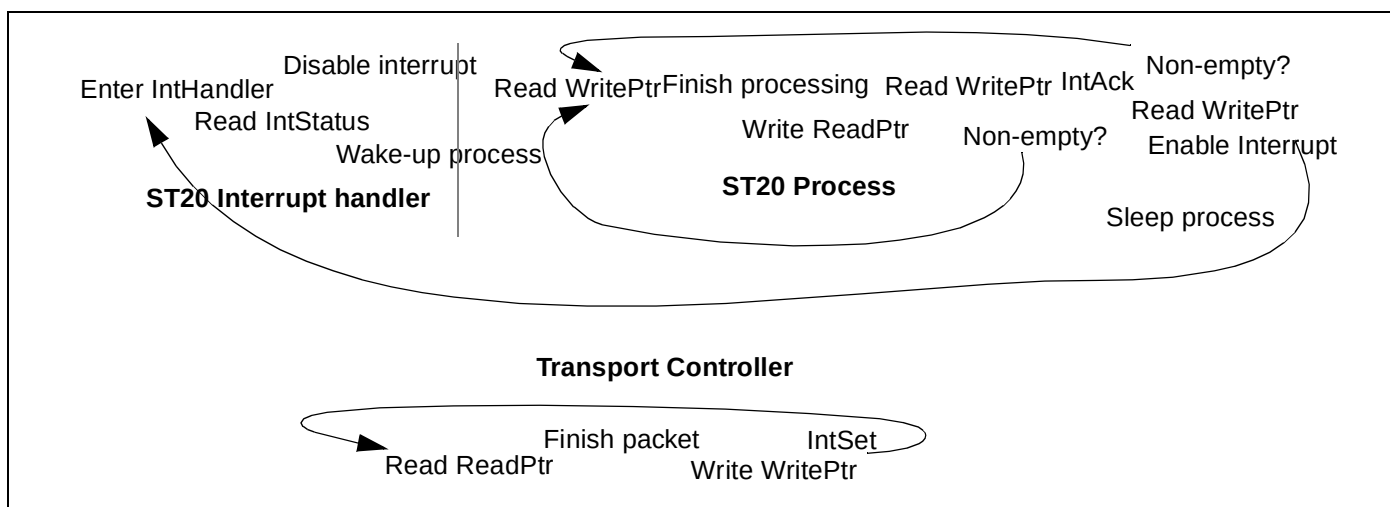


Figure 62 PTI buffer interrupt handling

18.3.5 Output of transport streams to an external interface

Concurrently with the parsing, processing, and DMA transfer of the transport packets, some or all of the transport stream can be copied to an external interface. In the case of the STi5512, the interface is designed for an external IEEE 1394 Audio/Video Link Layer controller. The ST20 software can tag certain or all PIDs via the data structures in the data SRAM in the PTI to be output to the link layer interface, either with or without descrambling. In the case that the output data is descrambled, the transport or PES level scrambling control bits are reset to indicate unscrambled data. It is possible to output scrambled data and input descrambled data to memory at the same time.

Other modifications of the transport stream, including modifications of tables and substitution of packets, are possible by suitable programming of the TC.

Groups of PIDs, or parts of transport packets can be flagged using the **PacketTag0-3** signals that can be output by the PTI with the transport stream. These signals are controlled by the TC software and can be updated for each byte output.

18.4 External interfaces

The external interfaces of the PTI are connected to the pins of the STi5512 via the Link Layer Interface module. This provides multiplexing of streams from the shared IEEE 1394 or IEEE 1284 ports and the Link IC interface pins, with some options on the sources of byte clocks for transport streams being input or output.

18.5 PTI timer module

The timer module in the PTI contains the System Time Clock (STC) and three time stamp registers.

- Packet start time register;
- Audio PTS (Presentation Time Stamp) latch register (33 bits);
- Video PTS latch register (33 bits).

The System Time Clock value is latched into one of these three registers according to the triggers shown in [Table 80](#).

Trigger	Action
Rising edge of packet clock	STC is latched into the packet start time register.
Beginning of Audio Frame output	STC is latched into the audio PTS register.
VSYNC	STC is latched into the video PTS register.

Table 80 Timer module register update triggers

The packet start time register can be read by the TC and used to determine the arrival time of a PCR (Program Clock Reference). The CPU cannot directly access this register. TC software can store this arrival time together with the PCR in shared data SRAM so it can be read by the CPU.

The audio and video PTS registers, **PTIAudPTS** and **PTIVidPTS** can be read directly by the CPU and used by driver software to synchronize the audio and video. The time stamps are 33 bit values, so the most significant bit is held in a separate word.

18.6 Address map

The address map of the PTI registers is given in [Table 81](#). The registers and shared memory are in a block in the peripheral space of the ST20 address space, with base address *PTIBaseAddress*. The value of *PTIBaseAddress* is given in the *STi5512 Register manual*.

All these registers and shared memory areas must be accessed using device access instructions. Each of these locations can only be written or read as a word; half word and byte accesses are not supported. Locations marked as reserved should not be written to or read.

Register	Offset from the PTIBaseAddress	Bits	R/W	Function
Configuration registers	0x0000 - 0x0054	22×32	R/W	PTI configuration registers
	0x0058 - 0x00FFC			Reserved. Do not read or write.
DMA registers	0x1000 - 0x1FFC	25×32	RW	DMA registers.
IIF registers	0x2000 - 0x3FFC	32×32	RW	Input interface registers.
Section filter core	0x4000 - 0x41FC	128×32	RW	Filters and masks in the section filter.
	0x4200 - 0x5FFC			Reserved. Do not read or write.
Transport controller	0x6000 - 0x7FFC	9×32	R	Internal TC registers, read for debug.
Data shared memory	0x8000 - 0x8BFC	768×32	RW	Data memory.
	0x8C00 - 0xBFFC			Reserved. Do not read or write.
Instruction shared memory	0xC000 - 0xCFFC	1024×32	RW	Instruction memory.
	0xD000 - 0xFFFC			Reserved. Do not read or write.

Table 81 PTI registers and shared memory

18.7 DMA

[Figure 63 on page 126](#) shows how the buffer pointer registers are used for the DMA channels. The base register points to the base word of the buffer, and must be 16-byte aligned, so bits 0 to 3 must be zero. The top register points to the top byte of the buffer. If a circular buffer is being used then this address must be one byte below a 16-byte aligned address, so bits 0 to 3 must be 1. The buffer for channel 0 only can be reduced to a single address by setting the top register equal to the base register. In this case, the data will be written to a fixed address defined by the write pointer and the write pointer will not be updated.

The read and write buffers point to the next word to be read or written respectively.

DMA channel 0 is used by the TC to write data from the payload of a transport packet into the ST20 memory space. The channel 0 registers are fetched from the PID state structure in the data shared memory at the start of processing a transport packet. Data is transferred by DMA, the write pointer is updated by the DMA hardware and then the TC copies the updated write pointer back to the PID structure. The ST20 CPU would not normally read or write the DMA channel 0 registers but would instead write to the PID structure in data shared memory to set up the pointers before the TC was started.

The DMA buffer pointer registers should be initialized by the ST20 CPU software. For each DMA channel, the read and write pointers should be initialized to the same value, so that the buffer is empty. The base and top pointers should be initialized to point to the beginning and end of the buffers.

In the case of channels 1 to 3, these buffers will have data read from them for transfer to the decoders. During operation of the DMA channels 1 to 3, the read and write pointers are examined by the hardware to determine if there is data in the buffers to be transferred. If there is data in the buffer and the request line for that channel is active then the data will be transferred and the read pointer updated. If channel 0 DMA is writing into the buffer then the write pointer will be updated by the TC; otherwise the ST20 CPU will update the write pointer after adding data to a buffer.

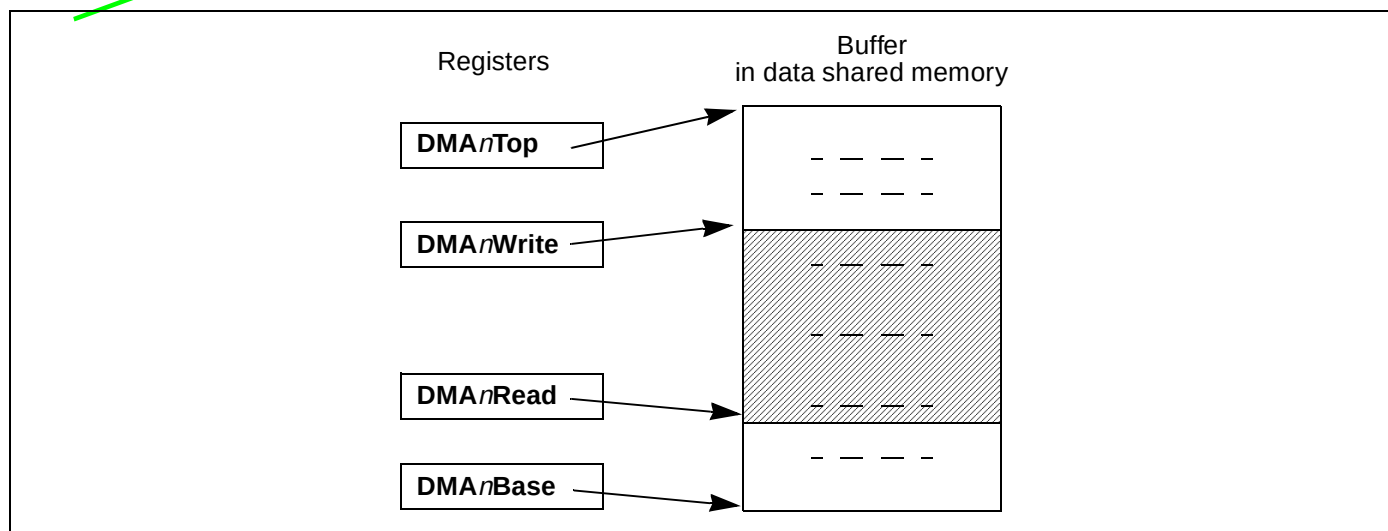


Figure 63 Buffer pointer registers

18.8 Input interface

Table 82 lists the registers used to control the input interface (IIF). The alternative output allows parts of the incoming transport stream to be routed out to an external device.

Register	Offset from PTIBaseAddress	Bits	Access	Description
IIFFIFOCOUNT	0x2000	8	R	The number of bytes in the input FIFO.
IIFAltFIFOCOUNT	0x2004	8	R	The number of bytes in the alternative output FIFO.
IIFFIFOEnable	0x2008	8	R/W	Allow data to be written into the input FIFO.
IIFAltLatency	0x2010	8	W	The delay between a packet arriving and being available at the alternative output.
IIFSyncLock	0x2014	4	W	The number of correct SYNC bytes to lock.
IIFSyncDrop	0x2018	4	W	The number of wrong SYNC bytes to lose lock.

Table 82 Input interface registers

18.9 Section filter

The section filter in the PTI hardware and software parses the section information in an MPEG-2 transport stream packet. Sections that pass the filter are transferred via the channel 0 DMA to ST20 memory. These sections have a fixed format and are defined by the MPEG-2 systems specification¹.

The data sections can arrive at a faster rate than the system can process them, so a filter selects only those sections that are required and thus reduces the required processing rate. In addition, the sections that are used to construct tables are repeated regularly, so it is possible to build up an information table by capturing a proportion of them using

1. Generic Coding Of Moving Pictures And Associated Audio: Systems, Recommendation H.222.0, ISO/IEC 13818-1

one set of values in the filters, and then capturing the remainder of the table by setting the filters up to select the missing sections.

The hardware filter system looks for a match to a total of 32 filters of 8 bytes each. Each bit of each of the filters may be individually masked, so that no comparison is performed on that bit of the filter. In addition to the filtering operation the PTI can perform CRC checking on the sections which match a filter. The CRC checking can be enabled for any of:

- all sections,
- no sections,
- only those sections that have the section syntax indicator bit set via bits in the PID data structure read by the TC code.

A filter mask word in each PID data structure specifies which set of filters is applied to sections in a transport packet with that PID.

The section filter is programmable by means of a set of registers. Each 8-byte section filter entry is composed of four 32-bit words in memory, with each group of four words aligned on a 4-word boundary. Within the 4-word group, the section filter is composed of two 32-bit words dedicated to the storage of filter data, plus two 32-bit words dedicated to the storage of masking information. An overall view of the section filter data is shown in the figure below, as it appears in the memory map.

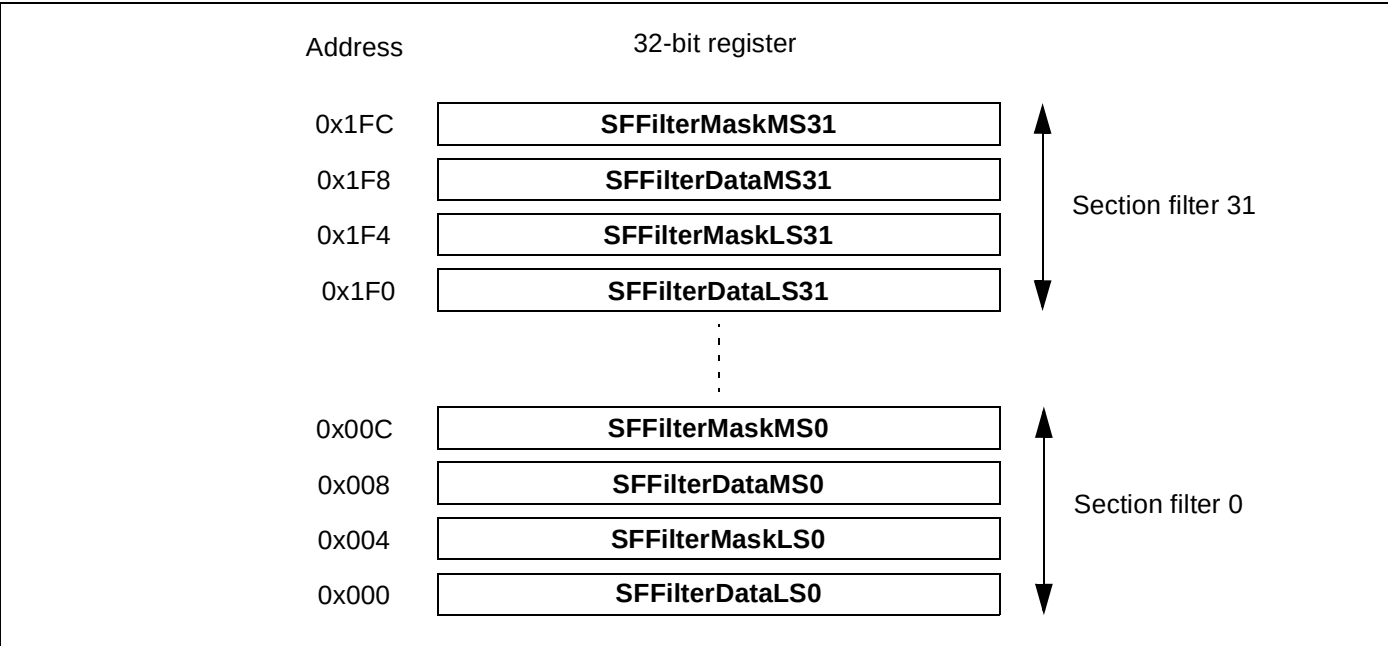


Figure 64 Section filter memory map

Only one word of a filter can be updated at a time, so updates of the section filters should be carried out when one of the following is true:

- a false match will not matter;
- the filter is not in use for any PIDs;
- the PIDs that use the filter have been disabled.

The section filter registers are in a block in the peripheral space in the ST20 memory map. The address of the base of the block is called *SFBaseAddress*, which has the value given by:

$$SFBaseAddress = PTIBaseAddress + 0x4000$$

Filtering is performed by testing whether:

$$packet \text{ BITAND } FilterMask = FilterData \text{ BITAND } FilterMask$$

If this is true then the packet passes through the filter.

18.10 Shared memory

18.10.1 Data memory

The PTI contains 3 Kbytes of data memory of which is accessed as 32-bit words by the ST20 CPU and as 16-bit words by the TC. It is located at addresses in the range 0x8000 to 0x8BFC above the *PTIBaseAddress*. This memory is used to hold the private data structures of the TC and data shared between the two processors.

18.10.2 Instruction memory

The 4 Kbyte instruction memory holds the instructions that the TC executes. It is accessed as 32-bits wide by both the TC and the ST20 CPU, and is loaded with code by the ST20 before enabling the TC. The ST20 cannot access the instruction memory while the TC is executing. It is located at addresses 0xC000 to 0xCFFC above the *PTIBaseAddress*.

19 MPEG video decoder

This chapter describes the STi5512 MPEG video decoder. The video decoder decompresses a MPEG 2 bit-stream and constructs a picture. The display functions are described in [Display planes on page 149](#) and the registers to control the decoder are described in the *STi5512 Register Manual*.

19.1 Decoder operation

The video decoder is a picture decoder; it decodes a whole picture and then stops until instructed to decode the next picture present in the video bit-stream.

Normally, the decoding of a new picture commences in response to the start of display of a new picture. The registers whose contents can change from picture to picture are double-banked and are updated automatically when decoding starts. The bit-stream is read from the bit buffer into the variable-length decoder (VLD), and picture reconstruction can commence. Any predictors required are fetched from the appropriate area of the external memory, and the reconstructed picture is written back into the area of this memory assigned to the decoded picture.

While a picture is being decoded the start code detector is used to locate the start of the next picture header, which the CPU then reads in order to set up the double-banked registers for the decoding of the next picture.

All of these tasks can be synchronized using interrupts generated on start code hits and vertical sync signals.

19.1.1 Start code search

The video decoder is able to decode in its entirety a video bit-stream from the slice layer downwards. The higher layers (i.e. picture and upwards) are decoded by the driver in order to extract the information needed for decoding and set up the appropriate video decoder registers and quantization tables. Since the header information is byte-aligned and requires minimal interpretation, this task represents only a small load on the CPU.

The start code detector parses the bit-stream stored in the bit buffer and locates start codes corresponding to picture layer and above. When one of these start codes has been found, the start code detector stops and raises an interrupt.

The CPU is then able to read the header data following the start code. The start code detector starts automatically whenever the decoding of a new picture starts and on user command. In normal operation, start code parsing is performed one picture in advance of decoding.

19.1.2 Operation in bandwidth reduction mode

In bandwidth reduction mode the decoder requires the use of three frame buffers in external memory. This is the normal mode of operation, where I, P and B-frames are decoded into and displayed from frame buffers in external memory. This mode is highly optimal in terms of memory bandwidth usage.

19.1.3 Operation in reduced memory mode

In reduced memory mode the decoder requires the use of only two frame buffers in external memory. The bidirectional frames are decoded and displayed on-the-fly.

As the display is interlaced the B-frames must be decoded twice, once for each field. This requires the decoder to be able to loop back in the bit-buffer and re-decode an image.

This is controlled by the driver in conjunction with the start code detector and is described further in [Decoder / display sequencing on page 132](#).

Note Before performing a pipe-reset when decoding a B-picture on-the-fly, quit "fly" mode by resetting register bit VID_DCF [FLY].

19.2 Resets

Hard reset is a global signal and is described in the *System Services* chapter.

The following types of soft reset can be used for the video decoder:

- A *total soft reset* is generated by setting and resetting bit **VID_CTL.SRS** and register **AUD_RES**. They must be set for a duration of at least 540ns.
- The audio, video and sub-picture subsystems may be individually *soft reset* by setting and resetting **VID_SRA** and **AUD_RES**, **VID_SRV** and **SPD_SPR** respectively.
- A *pipeline reset* is generated by setting and resetting bit **VID_CTL.PRS**. It must be set for a duration of at least 40ns.

After a soft reset, all processes concerning decoding and bit buffer control are reset. Any data remaining in the bit buffer, the compressed data FIFO and the start code detector FIFO are lost.

The interrupt unit is reset. All registers maintain their contents and the display process is not disturbed. A soft reset would normally be used when the decoding of the current bit-stream must be terminated and it is required to restart on a new sequence.

After a hard or a soft reset or a video soft reset, the first task performed by the pipeline when it has been enabled will always be a search for the beginning of a new sequence. The bit buffer data is flushed until the first picture start code following a sequence start code is detected by the pipeline, at which time it stops. At this point normal picture decoding behavior is resumed. After a hard or a soft reset, the first search performed by the start code detector in response to the first DSYNC will always be a search for a sequence start code, after which it stops. After this, the start code detector operates normally.

A pipeline reset terminates the decoding of the current picture. The remaining bits of the picture are flushed from the bit buffer until the next picture start code is detected by the pipeline. At this point normal behavior is resumed, i.e. the pipeline waits for the next picture decoding instruction. No other part of the circuit is affected by a pipeline reset. A pipeline reset would normally be used as part of a manual error recovery procedure. A pipeline reset has no effect if the decoding pipeline is in its idle state.

19.3 Bit buffer and start code detection (video)

19.3.1 Bit buffer

The transfer of compressed data is carried out using the DMA engines of the PTI. The PTI is described in [Programmable transport interface \(PTI\) on page 118](#). Compressed data can be taken from any memory space visible to the CPU and transferred to the relevant elementary stream decoder.

19.3.2 Start code detection

The start code detector operates in parallel with the decoding pipeline. The purpose of this unit is to allow external access to the header data which follows start codes in the input bit-stream. Compressed data is read twice from the bit buffer- once into the pipeline, and once into the start code detector through the 128-byte header FIFO. The transfer of data into the header FIFO does not affect the bit buffer level; only the data transfer into the pipeline can reduce the bit buffer level.

Start code detection is initiated in two ways:

- Automatically whenever the internal event DSYNC occurs. DSYNC is derived from VSYNC as described in [Decoding task control on page 138](#). A DSYNC is generated every time the pipeline starts a new picture decoding task.
- By software writing to the **VID_HDS** register with bit **VID_HDS.HDS** set.

When start code detection has been started, data is read continuously from the bit buffer into the header FIFO and parsed by the start code detector, which receives the FIFO output data. When a start code is detected, the data scanning stops and the status bit **VID_STA.SCH** becomes 1. When a start code has been detected, it can be identified by reading the **VID_HDF** register. The start code detector detects all start codes other than the codes from 0x00000102 through to 0x000001AF. The first slice start code 0x00000101 can be optionally detected to help driver development.

The register **VID_HDF** should always be read twice to return a 16-bit value. The most significant byte is read first. After detection of a start code, **VID_HDF** will return one of the 16-bit values shown below.

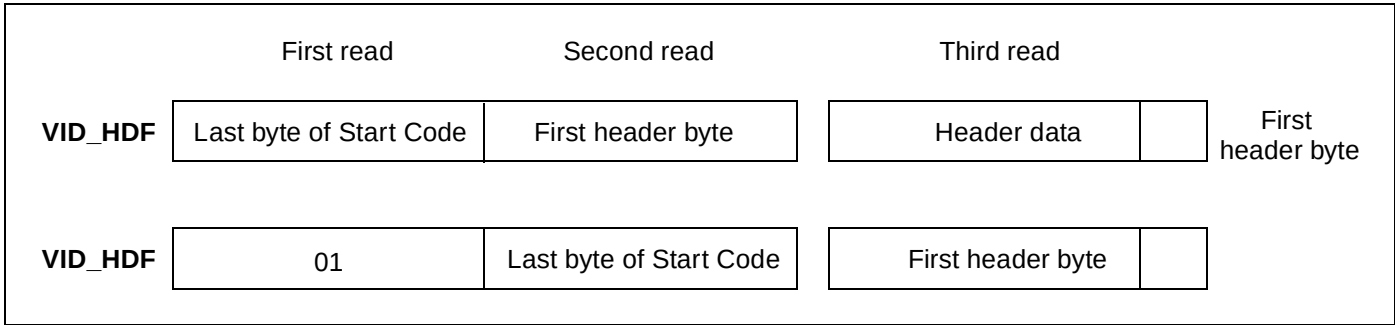


Figure 65 States of VID_HDF after detection of a start code

The first step is to examine the first byte read from **VID_HDF**. If this contains 0x01, then the start code can be identified by a second read at the same address. If the first byte is not 0x01 then it must be the last byte of the start code and the second byte is the first byte of the header data. In both cases subsequent reads from **VID_HDF** will give access to the header data which follows the start code.

Scanning for start codes will recommence on the next DSYNC or a write to **VID_HDS.HDS**. *Whenever a start code has been detected, the VID_HDF register must be read in order for the start code detector to restart correctly.* The number of reads before a manual or automatic (DSYNC) restart must always be even.

The first start code search after a hard or soft reset will be a search for a sequence header start code; all other start codes will be ignored. When this start code has been read, all subsequent searches will look for any start codes other than slice start codes.

The two status bits **VID_STA.HFE** (header FIFO empty) and **VID_STA.HFF** (header FIFO full) indicate the state of the header FIFO. Reading from HDF must never be performed if **VID_STA.HFE** is 1. **VID_STA.HFF** is set whenever the header FIFO contains at least 66 bytes.

The start code detector can also be programmed to stop on the first slice of the picture. This allows the use of the start code search even after reception of the picture start code. All header data that is not used by the application can then be skipped without risk, in order to jump to the next picture start code.

This mode is enabled by setting bit **VID_HDS.SOS**. To differentiate between first slice start code (00 00 01 01) and other start codes, it is possible to detect at which position (MSB or LSB) the Last Byte of Start code is positioned in the **VID_HDF** register. Register bit **VID_HDS.SCM** when set indicates that the Last Byte of Start code is held by the MSB of **VID_HDF**; it is zero otherwise.

19.4 Video decoding pipeline control

The pipeline is the core of the decoder. It is that part of the circuit which converts the compressed bit-stream data for each picture into a decoded (or reconstructed) picture. These pictures can be frame or field pictures. The operation of the pipeline is controlled picture-by-picture. The decoding of a new picture can potentially start on every VSYNC, but usually the rate of decoding is faster than the VSYNC rate.

The pipeline is controlled by the pipeline controller. When the pipeline controller starts the decoding pipeline a DSYNC signal is issued and **VID_STA.PSD** is set.

This signal is also sent to the start code detector. When the pipeline has completed its decoding operation, a completion signal is sent to the pipeline controller, which is then able to launch another decoding operation, either immediately or when the next VSYNC occurs.

The pipeline controller interprets certain bits of the decoding instruction, which must be set up by the user before the start of each new task. The remaining bits of the instruction define the decoding task itself.

The pipeline receives its compressed data from the bit buffer. This data is first processed by the variable length decoder (VLD) which regenerates the run/level coded DCT coefficients and the motion vectors (if present) for each macroblock. The picture data is reconstructed by passing the run/level data through the inverse quantizer and inverse DCT blocks.

This is then added to the predictors which have been fetched from the memory taking into account the macroblock prediction modes and motion vectors.

Finally, the decoded picture is written back into the memory, from where it can be accessed by the display unit for output.

The pipeline is also able to skip through picture data for various reasons. The different possibilities are:

- Skip to Next Sequence. This occurs unconditionally on the first instruction execution after a hard or soft reset (see [Resets on page 130](#)). Compressed data is skipped until the first picture start code following a sequence start code is found. The pipeline then indicates task completion and waits for a new instruction.
- Skip to Next Picture. This occurs either after a pipeline reset (see [Resets on page 130](#)) or when the decoding instruction specifies that one or two pictures should be skipped (see [Decoding task control on page 138](#)). In the first case compressed data is skipped until the next picture start code is found, after which the pipeline indicates task completion and waits for a new instruction. In the second case, after the skipping operation the decoding of the following picture is started immediately.
- Skip to Next Slice. This occurs after automatic error concealment (see [Error recovery and missing macroblock concealment on page 140](#)). Compressed data is skipped until the next slice start code in the picture is found, after which normal decoding resumes.

Before starting to decode a sequence, certain static parameters must be set up. These are:

- MPEG-1 or MPEG-2 mode selection. Bit **VID_PPR2.MP2** must be set for an MPEG-2 sequence, reset for an MPEG-1 sequence.
- Decoded picture size. Register **VID_DFW** must be set up with the picture width in macroblocks, and register **VID_DFS** must be set up with the number of macroblocks in the picture.

Decoding is enabled by setting bit **VID_CTL.EDC**.

19.4.1 Decoder / display sequencing

The decoder has two main modes of operation:

- Bandwidth reduction mode (normal mode)
- Memory reduction mode

The modes can be selected on a frame by frame basis. The user can decide for each frame how it is decoded by setting or clearing the bit **OTF** in the **VID_PPR1** and **FLY** in **VID_DCF** for each picture. The bit **FNF** in **VID_DCF** must also be correctly set depending on the type of picture to be decoded. If a picture is decoded on-the-fly then, in general, it has to be decoded twice to allow the display of both fields. To decode the picture twice, the bit **DC2** in **VID_TRF** has to be set during the first decode so that the pipeline will loop back to the start of the frame as soon as the first decode is complete.

In normal usage the decoder will be configured in one mode or the other at the start of decode. This is important because the phasing of the decoding and display processes is not the same in the two modes. Figure 66 below, shows the required phasing for each of the decode modes. It can clearly be seen that there is a full-field phase difference between the two display sequences this makes switching between the two modes of decode during decoding difficult.

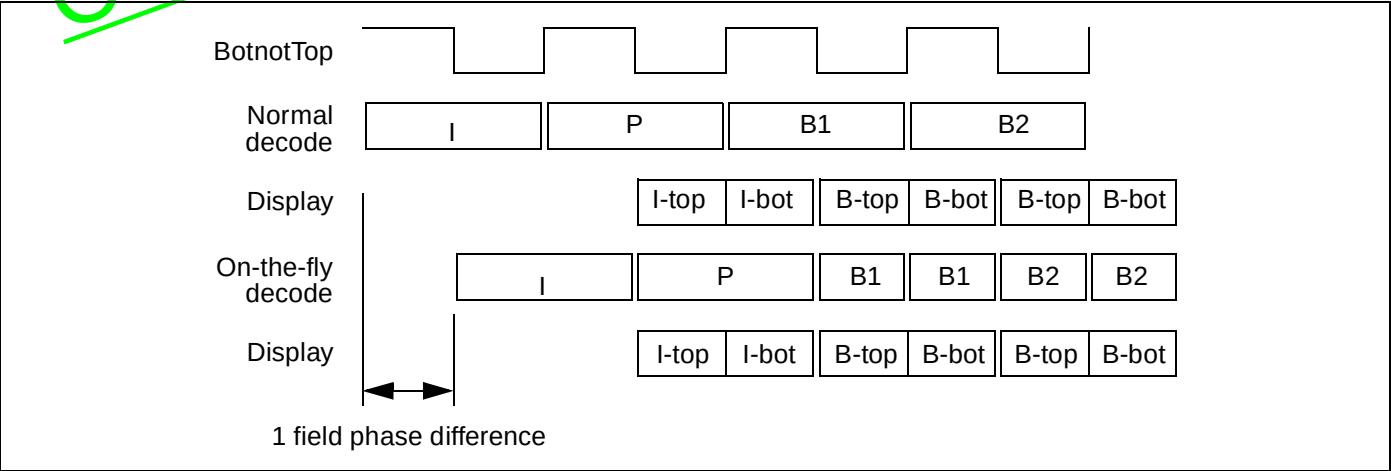


Figure 66 Phasing required for decoder modes

If memory reduction mode is selected then the main difference in the decoder control sequencing comes in the decoding of the B-frames. When the start code detector stops on a picture and the picture type is identified as a B-frame then the current position of the bit-buffer read pointer must be stored so that the pipeline can loop back to the beginning of the B-frame to re-decode the picture for display of the second field. The current position of the bit-buffer pointer can be stored by setting then resetting bit 0 in the **VID_LDP** register while the start code detector is aligned at the start of the picture required to be re-decoded. At the same time that this is carried out the temporal reference of the image to be re-decoded must be programmed in **VID_TRF**. This is important because the resolution of the jump back is large and may jump back to a preceding picture therefore the temporal reference is required to make sure the correct picture is re-decoded.

19.5 Quantization table loading

The two quantization matrices (intra and non-intra) used by the inverse quantizer must be initialized by the user. There are no built-in quantization matrices. Therefore, they must be loaded either with default matrices or with those extracted from the bit-stream by the ST20.

The quantization tables are double-buffered. This enables one or both tables to be updated without disturbing the decoding task in progress.

The video decoder maintains two bits which record whether one or both of the tables have been modified. A modified table is automatically brought into operation at the start of the next decoding operation, i.e. when the next DSYNC occurs.

After a hard reset, the same pair of tables is always selected. The data previously loaded into the tables is not affected. Other types of reset have no effect on the quantization tables.

The quantization tables are written at the address held in the register **VID_QMW**. Bit **VID_HDS.QMI** is used to select the Intra or Non-Intra quantization table; when it is set, the Intra table is selected; when clear the Non Intra table is selected.

19.6 Memory mapping of data

19.6.1 Video decoder memory (SDRAM) addressing

The locations in an SDRAM are addressed row by row, bank A then bank B, as shown below.

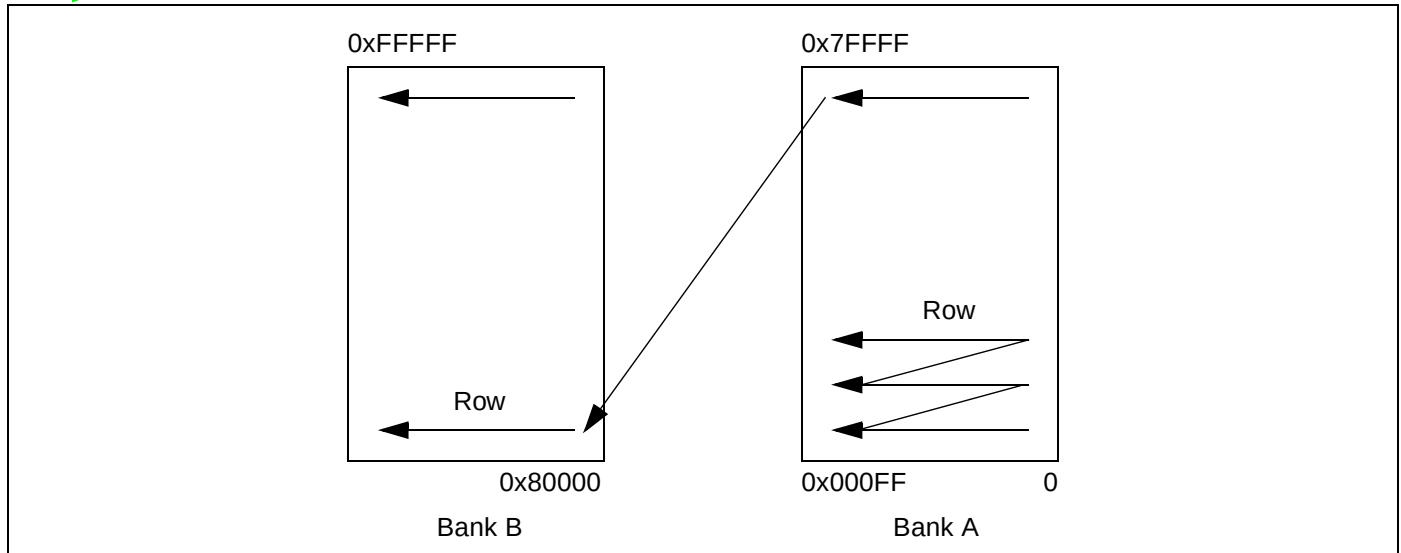


Figure 67 Standard addressing in a SDRAM (16-bit words)

19.6.2 32-bit word addressing for the CPU

The CPU accesses the SDRAM by using a 21-bit address for each 32-bit word. It is the task of the SDRAM memory controller to re-map the logical address space of the CPU onto the SDRAM address space.

The logical address map seen by the CPU is different from the one described in [Video decoder memory \(SDRAM\) addressing on page 134](#). For each row, both banks are used. The addresses seen by the CPU through the SDRAM interface are counted in the following order:

- Bank A, row0
- Bank B, row0
- Bank A, row1
- Bank B, row1
- etc...

When using a second SDRAM chip, addresses continue in a similar way, starting from the next address above the first SDRAM. A maximum of two SDRAM chips is supported, as shown below.

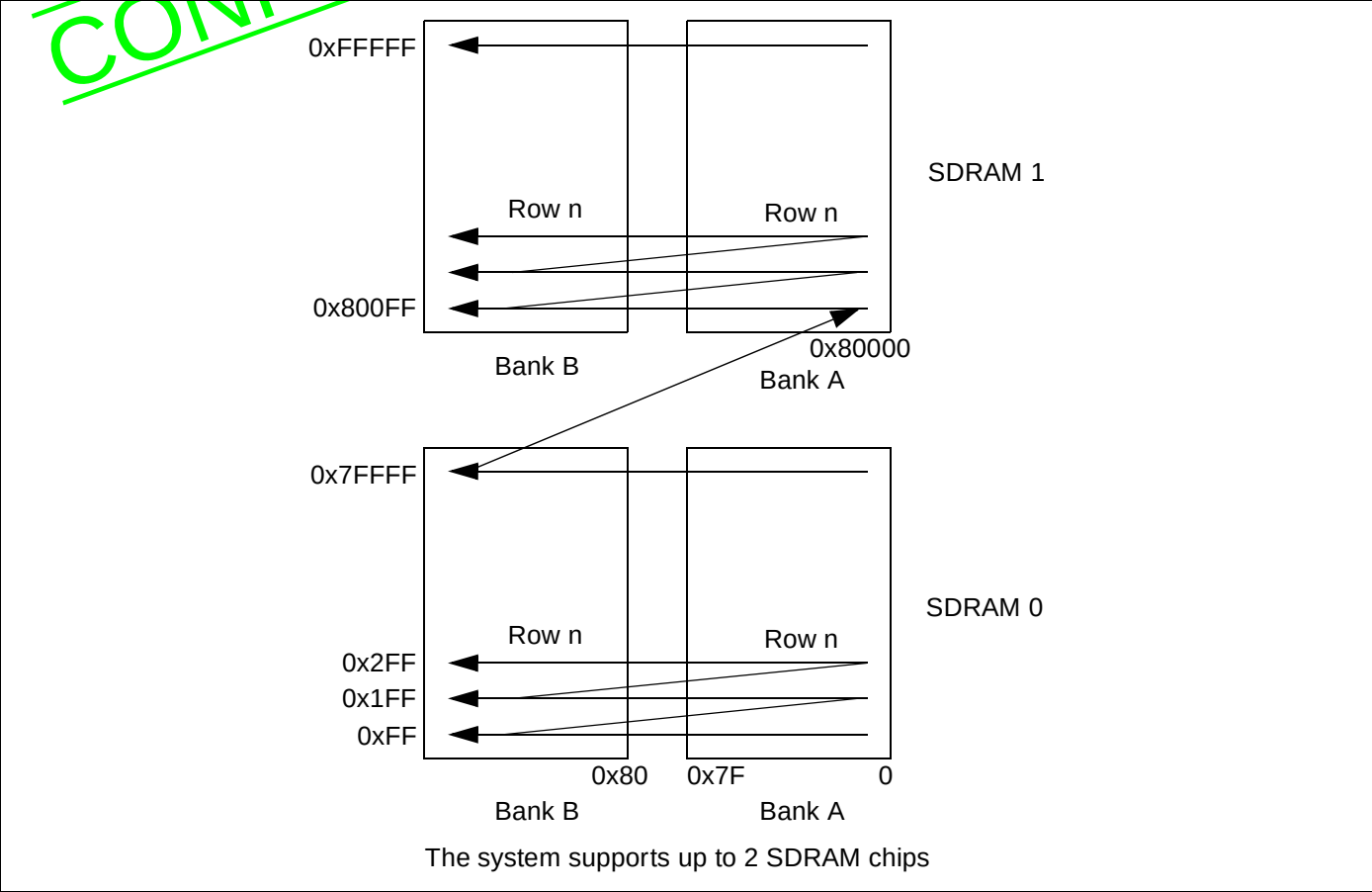


Figure 68 32-bit word addressing, as seen by the CPU

19.6.3 64-bit word addressing for FIFO processes

The video decoder uses circular buffers mapped into external SDRAM which act as software FIFOs. The processes pertaining to these circular buffers are managed with a 64-bit granularity. The memory mapping for these buffers is similar to that of the CPU and is shown in [Figure 69](#) below.

When using a second SDRAM chip, addresses continue in a similar way, starting from the next address above the first SDRAM. A maximum of two SDRAM chips is supported.

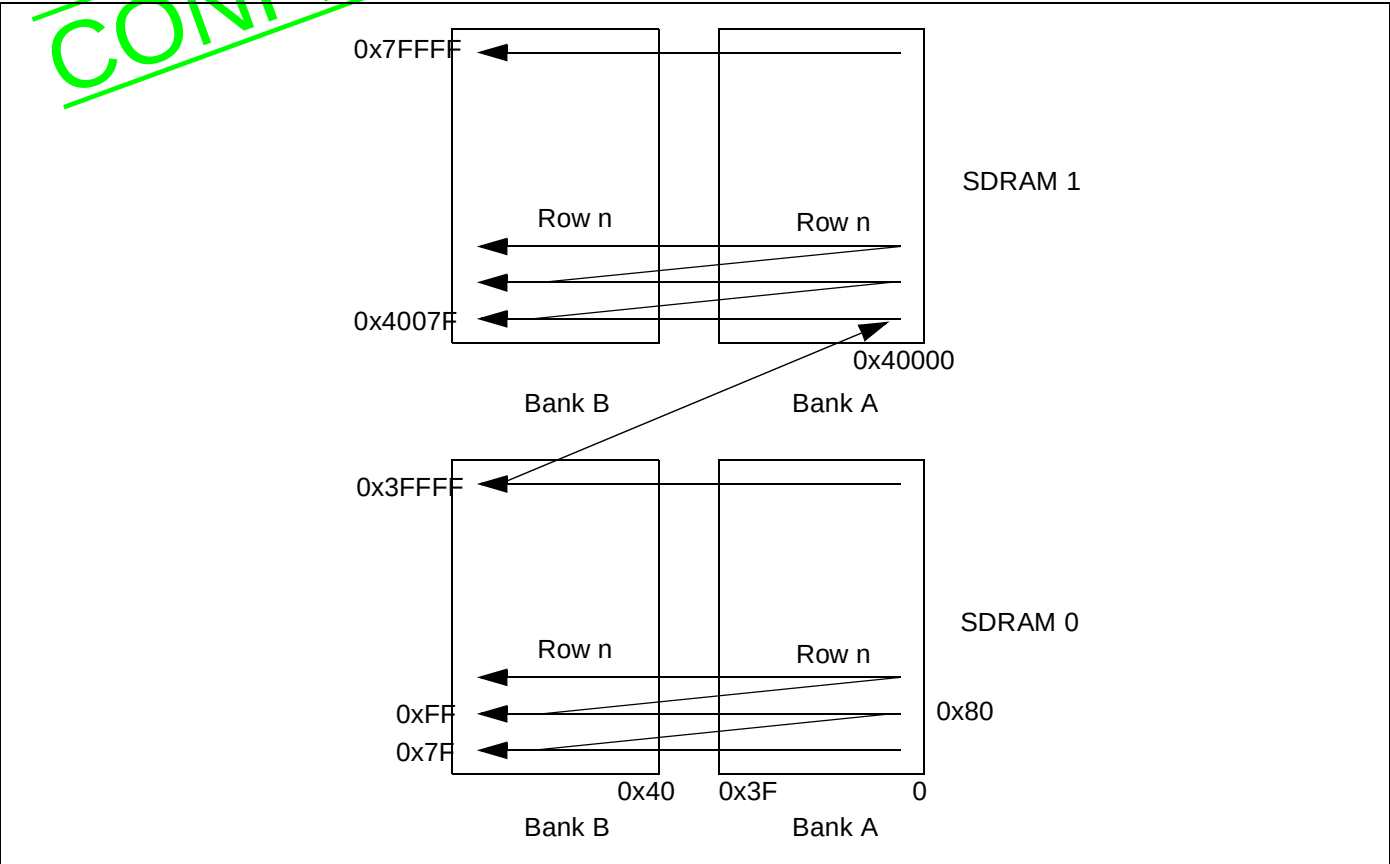


Figure 69 64-bit word addressing for FIFO processes

19.6.4 Memory segments

The circular buffer start and end pointers are programmed by the user in segments, where each segment is 256 bytes. The values in the configuration registers are numbers of segments. For example a value of 4 means 4 x 256 bytes = 1kbyte or 128 x 64-bit words. This would result in a pointer pointing to a 64-bit word address of 128 (0x80). This address would be physically mapped to the first word in the second row of bank A of SDRAM 0, as shown below.

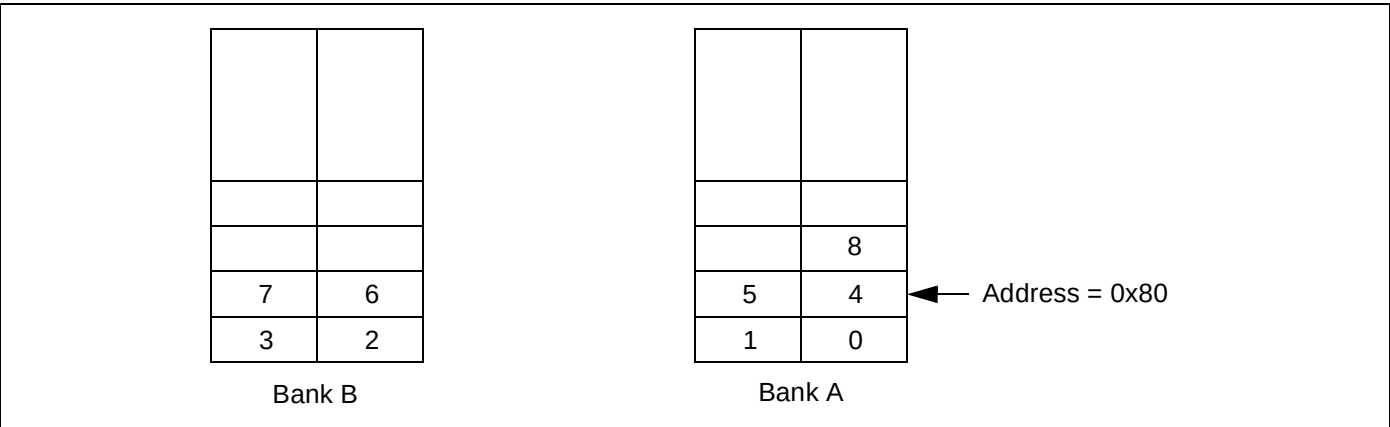


Figure 70 SDRAM segments as seen by the user



19.6.5 Arrangement of pixel pairs inside a luma SDRAM row

Every SDRAM row in a luma frame contains 256 16-bit words and can store up to two luma macroblocks. Every 16-bit word contains a pair of horizontally adjacent luma pixels. The row itself stores a pair of horizontally adjacent luma macroblocks. The pixel pairs are arranged in line order; the first 16 words store the first line of pixels for the two macroblocks, the next 16 words the second line and so on, as shown below.

16-bit word addresses in SDRAM row	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
Arrangement of luma pixel pairs	Y = 2 pixels	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
⋮	Macroblock 0								Macroblock 1							
16-bit word addresses in SDRAM row	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF
Arrangement of luma pixel pairs	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y

Figure 71 Arrangement of pixel pairs in a luma SDRAM row

19.6.6 Arrangement of pixel pairs inside a chroma SDRAM row.

Every SDRAM row in a chroma frame contains 256 16-bit words and can store up to four chroma macroblocks. Every 16-bit word contains a pair of horizontally adjacent 8-bit chroma pixels. The row stores pixel pairs in line order for macroblocks 0 and 1 and then macroblocks 2 and 3. The Cb and Cr words are interleaved two by two in the linear addressing order, as shown below.

16-bit word addresses in SDRAM row	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
Arrangement of luma pixel pairs	Cb = 2 pixels	Cb	Cr	Cr	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr
⋮	Macroblock 0								Macroblock 1							
16-bit word addresses in SDRAM row	70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F
Arrangement of luma pixel pairs	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr
16-bit word addresses in SDRAM row	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
Arrangement of luma pixel pairs	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr
⋮	Macroblock 2								Macroblock 3							
16-bit word addresses in SDRAM row	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF
Arrangement of luma pixel pairs	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr	Cb	Cb	Cr	Cr

Figure 72 Arrangement of pixel pairs in a chroma SDRAM row

19.7 Using picture pointers

Before the decoding of each picture the following frame buffer pointers must be set up:

- **VID_RFC, VID_RFP** - reconstructed frame pointers for chroma and luma,
- **VID_FFC, VID_FFP** - forward prediction frame pointers for chroma and luma,
- **VID_BFC, VID_BFP** - backward prediction frame pointers for chroma and luma.

A fourth pair of pointers, **VID_DFC, VID_DFP**, the displayed frame pointers.

VID_RFP and **VID_RFC** define the memory buffer to which the decoded picture is written. **VID_FFP, VID_FFC, VID_BFP** and **VID_BFC** define the areas in memory from which the predictors are fetched.

The rules governing the use of the prediction frame pointers are given below.

Pictures are always stored as frames of interleaved lines, and thus to access a field (top or bottom), the starting address of the frame must be defined.

P-Frame picture (frame, field or dual-prime prediction)

VID_FFP and **VID_FFC** are set to the address of the predictor frame (in which the two predictor fields lie). **VID_BFP** and **VID_BFC** are not used.

B-Frame picture (frame or field prediction)

VID_FFP and **VID_FFC** are set to the address of the forward predictor frame (in which the two predictor fields lie). **VID_BFP** and **VID_BFC** are set to the address of the backward predictor frame (in which the two predictor fields lie).

P-Field picture (field, 16 x 8 or dual-prime prediction)

When decoding either field, **VID_FFP** and **VID_FFC** are set to the address of the previous decoded I or P frame. **VID_BFP** and **VID_BFC** are not used.

B-Field picture (field or 16 x 8 prediction)

VID_FFP and **VID_FFC** are set to the address of the frame in which the two forward predictor fields lie. **VID_BFP** and **VID_BFC** are set to the address of the frame in which the two backward predictor fields lie.

I-Pictures

For I-picture decoding, no predictors are necessary, but **VID_FFP** and **VID_FFC** must be set to the address of the last decoded I- or P-picture for use by the automatic error concealment function.

19.8 The video pipeline

19.8.1 Decoding task control

A task is a single picture decoding operation. A task is specified by the task description or instruction, which is set up before the decoding of each picture. A task commences when the internal signal DSYNC is generated. A task completes (the decoder becomes idle) when the picture is entirely reconstructed in the memory and the picture header of the following picture is detected by the pipeline. The instruction is double buffered, so that during execution of a decoding task, the instruction for the next task can be set up by the CPU. When the next instruction is activated, a DSYNC can be generated, and the next decoding task started. The buffering mechanism is illustrated in [Figure 73](#) below. Note that some instruction bits are latched by VSYNC, others by a signal from the pipeline controller "new instruction".

The instruction is written into registers **VID_PPR1** and **VID_PPR2**. If a new instruction is not written, the task descriptor will be the same as the previous one.

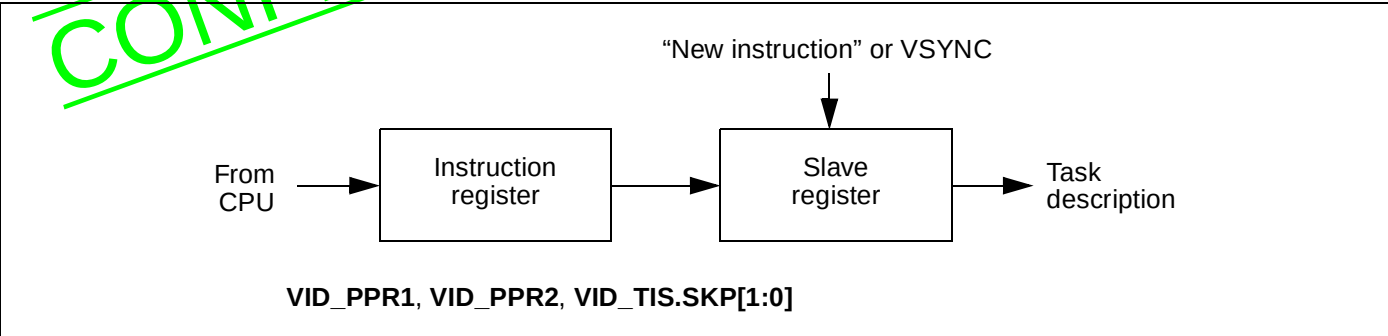


Figure 73 Instruction buffering

Normally, it is a VSYNC that starts the execution of a new instruction, and thus the generation of DSYNC. If however, a VSYNC occurs before task completion (i.e. before the pipeline becomes idle), the start of the next task will be delayed until the present one is completed. In this way the decoding of a picture can be allowed to extend beyond the nominal period allotted to it, usually one or two VSYNC periods.

Three status bits (and thus interrupts) are associated with pipeline control:

- **VID_STA.PSD** indicates the occurrence of a DSYNC.
- **VID_STA.PII** indicates that the pipeline is idle.
- **VID_STA.DEI** indicates that the decoder is idle, i.e. the pipeline is idle and the next picture start code has been found.

The operation of the pipeline controller is shown in the state diagram of [Figure 74](#) below. The meanings of the abbreviations for the state transitions are given in [Table 83](#).

Abbreviation	Meaning
ERC	Automatic error concealment.
EXE.FIS	Both VID_TIS.EXE and VID_TIS.FIS are set.
EXE.Vsync	Bit VID_TIS.EXE set when external VSYNC occurs.
DEI	Decoder idle interrupt generated.
PSC	Picture start code.
PSD	Pipeline start decode interrupt generated.
SEQ	Sequence start code
Skip and decode	VID_TIS = EXE SKP[01] and VSYNC occurred.
Skip and stop	VID_TIS = EXE SKP[11] and VSYNC occurred.
Skip twice and decode	VID_TIS = EXE SKP[10] and VSYNC occurred.

Table 83 State transition abbreviations

The concealment macroblocks are accessed using the pointers **VID_FFP** and **VID_BFP**. Lost macroblocks in the first row are copied directly from the previous pictures (i.e. as P-macroblocks with zero motion vectors). If an intra picture is coded with concealment motion vectors, these will be used. If not, then the concealment will be a simple copy from the previous picture using zero vectors. Even in intra pictures, the pointer **VID_FFP** must be set up.

The table below shows the rules that are used for fetching concealment macroblocks.

Picture type	Macroblock type	Fetch rule
I-picture	I-macroblock without vectors	Copy with zero motion.
	I-macroblock with vectors	Copy as forward predicted macroblock.
P-picture	I-macroblock without vectors	Copy with zero motion.
	I-macroblock with vectors	Copy as forward predicted macroblock.
	P-macroblock	Copy using stored vector.
	P-field-macroblock	Copy in field mode using both vectors.
	Skipped macroblock	Copy with zero vector.
	Dual-prime macroblock	Copy using stored vector.
B-picture	I-macroblock without vectors	Copy with zero motion.
	I-macroblock with vectors	Copy as forward predicted macroblock.
	Forward macroblock	Copy using stored vector.
	Backward macroblock	Copy using stored backward vector.
	Bidirectional macroblock	Only the forward vectors are stored, concealed as forward macroblock.
	Skipped macroblock	Copy in frame mode using the same mode and vectors as the previous macroblock.

Table 84 Rules for fetching concealment macroblocks

If an error is detected in the bit-stream before it enters the parser, then an error start code can be inserted into the bit-stream in order to initiate concealment. However, when doing this there are certain restrictions on the placement of the error start code in order to avoid emulation of other start codes. An Application Note is available on this topic.

Overflow or underflow error

An overflow error occurs whenever the pipeline reconstructs more macroblocks than are defined by the decoded picture size, **VID_DFS**. This can occur when the input data to the decoder contains undetected errors. This condition is signalled by bit **VID_STA.SER**. Decoding is automatically halted when this error is detected. In order to restart decoding a pipeline reset must be performed.

An underflow error occurs whenever the pipeline reconstructs less macroblocks than are defined by the decoded picture size, **VID_DFS**. This condition is signalled by bit **VID_STA.PDE**. Decoding is automatically halted when this error occurs. In order to restart decoding a pipeline reset must be performed.

Soft reset

The effect of this last resort action is described in [Resets on page 130](#).

19.9 PES parser overview

The PES parser is situated between the ST20arbiter/bus and the compressed data FIFOs of the video/audio core. It has a 100 Mbits/sec (max burst) bit rate, and allows the following input streams:

- Packetized PES (MPEG-2), ISO 13818-1
- MPEG-1 system layer (ISO 11172-1)

The MPEG2 PES & MPEG1 system parser accepts PES streams in the same way that pure audio or video streams are accepted.

For packetized elementary stream data which is demultiplexed from a transport stream (MPEG-2), the data stream consists of concatenated, incomplete packets of audio and video PES. To handle this configuration, the STi5512 contains two separate parsers: one for the audio (audio PES parser in audio decoder) and one for the video (MPEG2 PES & MPEG1 system parser).

As the audio or video data is input, it is demultiplexed by each parser and the audio / video streams are placed in their respective buffers. For program stream data or MPEG-1 systems stream data, the audio and video packets are complete so that a single parser (MPEG2 PES & MPEG1 system parser) can be used. The packets are internally separated into video and audio streams. If required, the two parsers can still be used but the packets must be separated by the ST20 (recommended mode). See the figure below.

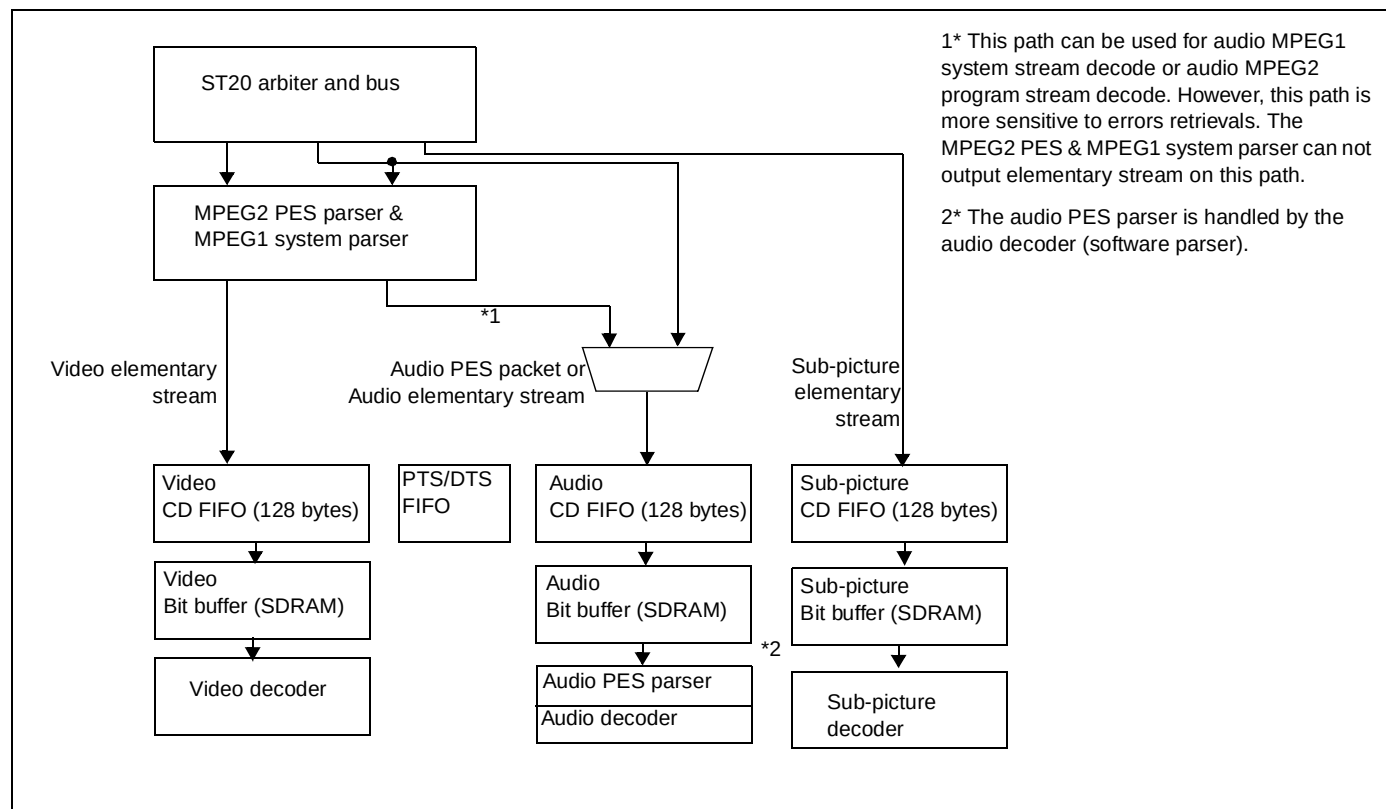


Figure 75 System parser internal architecture

When the MPEG2 PES & MPEG1 system parser is configured to accept MPEG-2 PES audio/video packets (mode 3), the parser extracts audio & video bit-streams in accordance with the programmed stream ID. For the audio stream this is contained in PES_CF1; for the video stream in PES_CF2. Any audio or video packets which are not selected for decode (because their stream IDs do not match the programmed values) are discarded. The audio PES are output on path 1 (see Figure 75).

When used for decoding program streams or MPEG-1 system streams, the audio, video and system level data are automatically separated internally to the MPEG2 PES & MPEG1 system parser. Time stamp association is supported by the decoder.

During parsing, decode or display time stamps (DTs or PTs, selected by PES_CF1.SDT) are stored in an internal FIFO. When the image corresponding to these time stamps is decoded (or, in the case of video, about to be decoded) the corresponding time stamp is made available and a flag or interrupt is given.

Functional Modes

The parsers are enabled by setting register PES_CF2.SS for the video parser and STREAMSEL/DECODESEL registers for the audio parser. Depending on the required mode, one or both of the parsers are required.

Four different modes can be configured with the two mode bits of register PES_CF2[7:6]:

- Mode 0: Automatic configuration. The parser examines the incoming stream and self-configures for decode. The mode selected can be read back from PES_TM2[1].
- Mode 1: MPEG-1 system stream decode. Single data strobe input format. The audio elementary stream is extracted by the MPEG-2 PES PARSE & MPEG-1 SYSTEM PARSE block and sent to the CD audio FIFO.
- Mode 2: MPEG-2 PES decode. Twin data strobe input format. The video PES stream and the audio PES stream are sent separately and respectively to MPEG-2 PES PARSE & MPEG-1 SYSTEM PARSE block and audio CD FIFO. This is the most common way to enter data into the circuit.
- Mode 3: MPEG-2 whole PES Audio/Video Packets. Single data strobe input format. This is used to decode MPEG-2 program streams. The audio PES are output on path 1 (see [Figure 75](#)) extracted by the MPEG-2 PES PARSE & MPEG-1 SYSTEM PARSE block and send to the CD audio FIFO.

The video parser is reset by setting PES_CF2.SS to 0.

20 Sub-picture decoder

20.1 Introduction

A hardware sub-picture decoder is integrated in the STi5512. The sub-picture bit-buffer that contains sub-picture units (SPU) is integrated in SDRAM external memory and has a programmable size. Its position and size can be randomly chosen in multiples of 2 Kbytes. The sub-picture bit buffer is set up at power up reset. During player operation, its size and location are constant. Compressed data is input into the bit-buffer using a DMA or by a CPU write. Once control is given to the sub-picture decoder it is autonomous until stopped by software control. The sub-picture decoder can decode complete sub-picture units consisting of a sub-picture unit header, compressed pixel data and the display control sequence table without any interaction from the CPU.

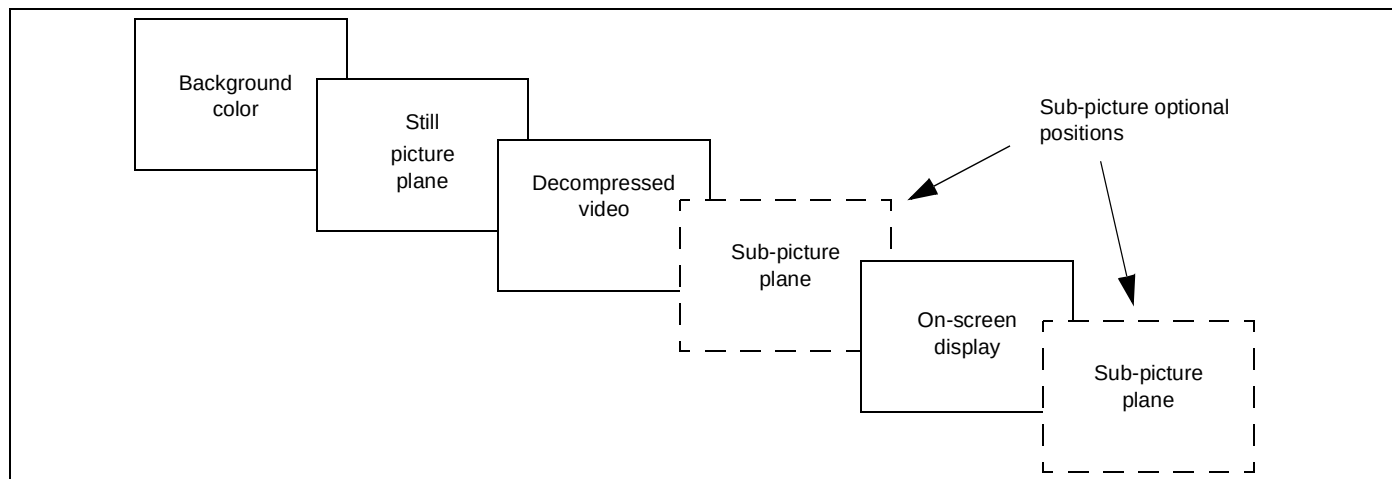


Figure 76 Display planes

The sub-picture decoder can also be used as a hardware cursor unit. The priority of the sub-picture is first raised by programming a register so it is in front of all the other display planes. A cursor can be defined using an optionally compressed (run-length encoded) bitmap stored in external SDRAM. The bitmap can be any size up to a full screen. Per-pixel alpha-blending factors can be defined for each cursor to provide anti-aliasing with the background. The cursor is then moved around using register writes into X and Y coordinate registers. Figure 77 shows the architecture of the sub-picture decoder.

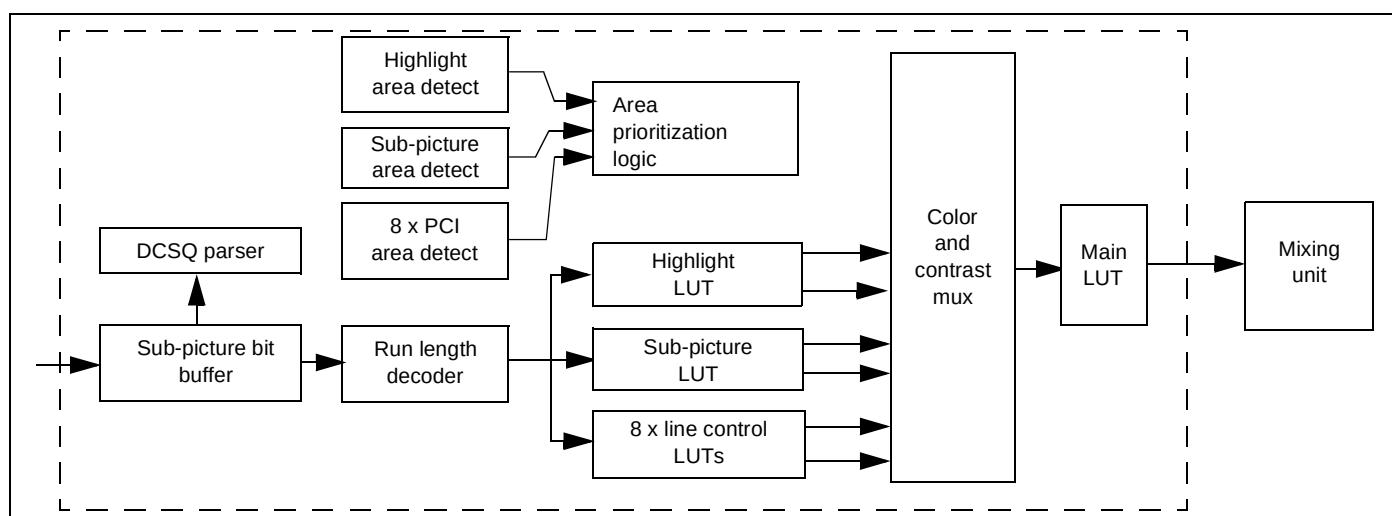


Figure 77 Sub-picture unit architecture

20.2 Buffer management and pointers

There are four programmable registers to control the sub-picture bit buffer read and write processes, as shown in Figure 78 below:

- Bit buffer base address (**VID_SPB**). This is an offset relative to the ST20 SDRAM base address. It is programmed in units of 2 Kbytes.
- Bit buffer end address (**VID_SPE**). This address is an offset relative to the ST20 SDRAM base address. It is programmed in units of 2 Kbytes.
- Bit buffer read pointer (**VID_SPRead**). It is set by software for each sub-picture unit. This is done before control is given to the sub-picture hardware decoder. This register is double buffered. The shadow register is updated with each field VSYNC event. This pointer is an offset relative to the ST20 SDRAM base address. It is programmed in units of 64-bit words (see Figure 77).
- Bit buffer write pointer (**VID_SPWrite**). It is set by the ST20 before transferring each sub-picture unit into the bit buffer. This pointer is an offset relative to the ST20 SDRAM base address. It is programmed in units of 64 bit words.

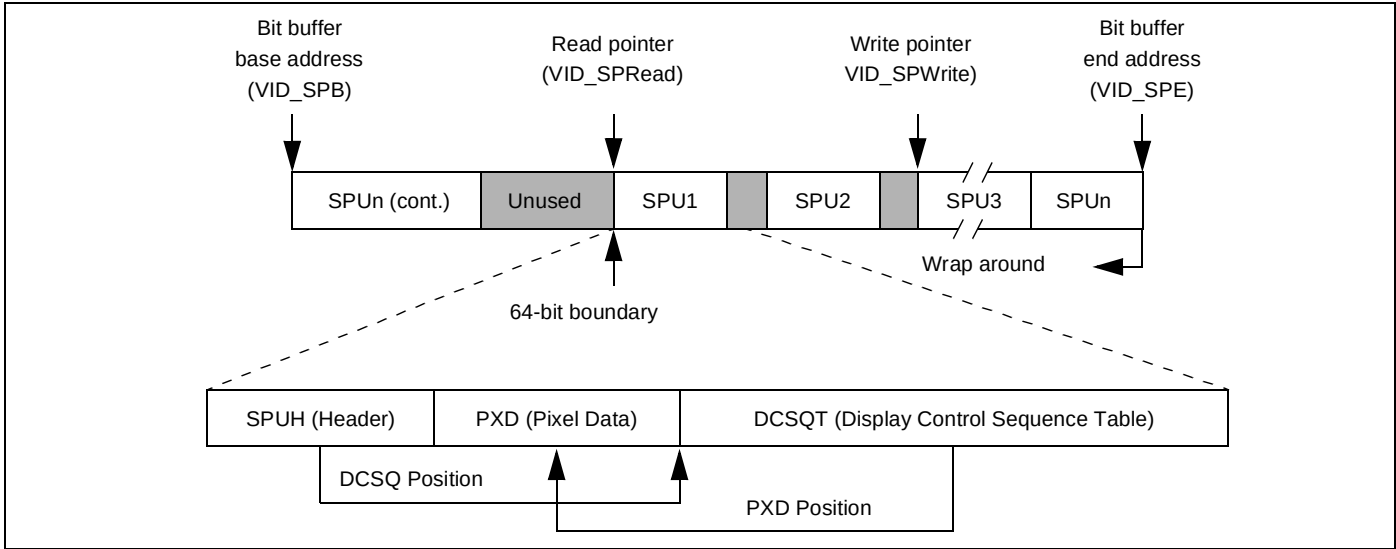


Figure 78 Buffer management

20.3 Subpicture decoder operation

Each sub-picture unit data buffer start position is programmed using the register **VID_SPWrite**. Subsequently the sub-picture header, the pixel data, the display control sequences are sent via fifos to the sub-picture decoder. Write into fifos is done by DMA or by CPU write. Only data belonging to the sub-picture unit (SPUH, PXD, DCSQT) are transferred into the sub-picture bit buffer. Sub-picture pack headers are removed by the software demultiplexor. The decoder reads the header of the first packet (see Figure 79) and jumps to the first display control pointer.

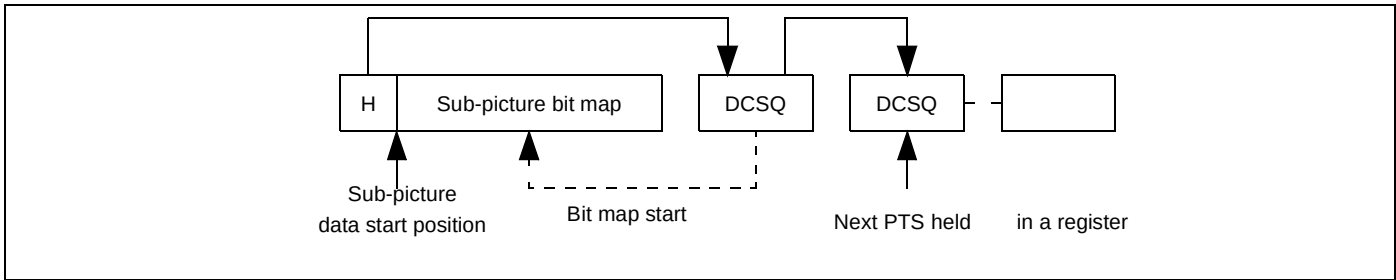


Figure 79 Sub-picture unit structure

The instructions found in the DCSQ packets enable the sub-picture unit to program the palettes and set mixing factors etc. for each region. The DCSQ packets also contain a time stamp, which indicates which image the sub-picture information refers to. This information is related to a local time for this sub-picture unit. The micro should enable a given sub-picture unit at the right global time via some registers: data buffer start position, start sub-picture unit status bit.

Register bits **SPD_CTL1.bot_top** can be used to set which field the subpicture commands are executed on: any field, top field or bottom field. The default is "any field".

The overall control of the sub-picture decoder is performed by software.

The final information in the DCSQ packet is the region size (rectangle) and the relative position, in bytes, of the bit-map start.

A key point here is that the sub-picture decoder must read beyond the end of the DCSQ packet in order to verify the next PTS. With this information held in a register, the sub-picture decoder knows, in advance, when to change the DCSQ or bit-map information. The sub-picture unit simply executes the same DCSQ until the image corresponding to the next time-stamp is reached.

This is done at the beginning of every field so that the sub-picture decoder can load all the relevant information from DCSQ before the first sub-picture pixel is required.

The sub-picture region declaration is held in registers in the decoder so that the sub-picture decoder is turned on and off at the correct position on the screen (see [Figure 80](#) below). The bit-map start pointer indicates where, in the bit map data, to start decoding. When the correct image, corresponding to the local time stamp contained in the DCSQ, should be displayed the sub-picture controller enables the sub-picture decode for that image.

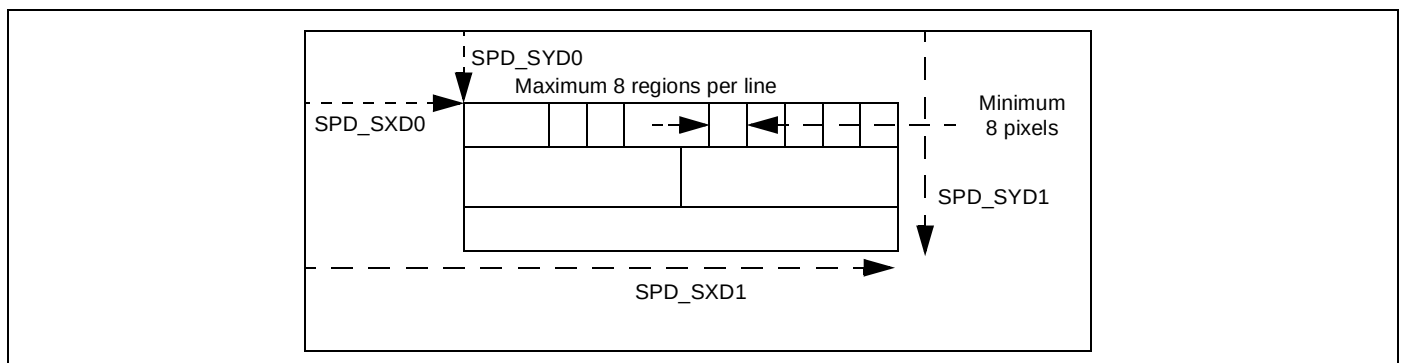


Figure 80 Sub-picture region declaration

A pause mode is defined in the sub-picture decoder. As explained previously, the sub-picture decoder is autonomous within a sub-picture unit.

This means that the DCSQ switching is timed automatically using an internal 90kHz clock. During video trick modes, where the video stream may be frozen or slowed down the same thing should be possible with the sub-picture decoder in order to maintain the synchronization between the two streams.

A pause mode is implemented for the sub-picture decoder which stops the 90kHz counter and therefore pauses the sub-picture decoder. This is controlled using the **P** field in the **SPD_CTL1** register and is synchronized to the VSYNC signal. This control bit can therefore be used as a pause and a single step control bit.

The sub-picture decoder registers are put together in the sub-picture memory map except:

- sub-picture software reset (register **SPD_SPR**),
- sub-picture pause mode (**SPD_CTL1.P** bit),
- sub-picture FIFO full (bit 18 of **VID_ITS** and **VID_STA** register).

20.4 Sub-picture display

20.4.1 Look-up tables

There are 11 look-up tables inside the sub-picture decoder:

- 1 highlight LUT (2 bits to 4 bits mapping)
- 1 sub-picture LUT (2 bits to 4 bits mapping)
- 8 PCI LUTs (2 bits to 4 bits mapping)
- 1 main LUT (4 bits to 24 bits mapping)

The sub-picture and PCI LUTs are automatically supplied by the decoder itself (sub-picture commands contained in the SPU). The highlight and main LUTs need to be loaded by the ST20 (**SPD_HCN**, **SPD_HCOL**, **SPD_LUT** registers).

The output of the sub-picture main LUT is mixed with the other planes. The contrast value between these two sources is set by the SET_CONTR DCSQ command, by the PCINFs of a CHG_COLCON command or by a highlight color information (the highlight LUT has the highest priority, followed by the PCI LUTs. The sub-picture LUT has the lowest priority). The mixed video is a 24 bits Y, C_r, C_b video where:

- $Y_{MIXED} = [Y_{PLANES} \times (16 - k) + Y_{SUBP} \times k] / 16$
- $Cr_{MIXED} = [Cr_{PLANES} \times (16 - k) + Cr_{SUBP} \times k] / 16$
- $Cb_{MIXED} = [Cb_{PLANES} \times (16 - k) + Cb_{SUBP} \times k] / 16$
- $k = 0$ if contrast value from high light, sub-picture, PCI LUTs = 0
- $k = \text{contrast value} + 1$ if contrast value > 0

20.4.2 Sub-picture areas

The active sub-picture decoding area can be 720 x 576 or 720 x 480 pixels. In order to align the sub-picture decoding area with the video decoding area, the upper left corner of the active sub-picture decoding area has to be set by software, using the **SPD_XD0** and **SPD_YD0** registers.

*Note The minimum values for **SPD_XD0** and **SPD_YD0** are 3 and 2 respectively. If the programmed values are less than this, no subpicture is displayed.*

The same semantics have been defined as for the video decoder, as shown in [Figure 81](#) below. The active sub-picture display area is defined in a similar manner, using the **SPD_SXD0**, **SPD_SYD0**, **SPD_SXD1** and **SPD_SYD1** registers.

The highlight area is defined by the **SPD_HLS119X**, **SPD_HLSY**, **SPD_HLEX**, **SPD_HLEY** registers and is set by software.

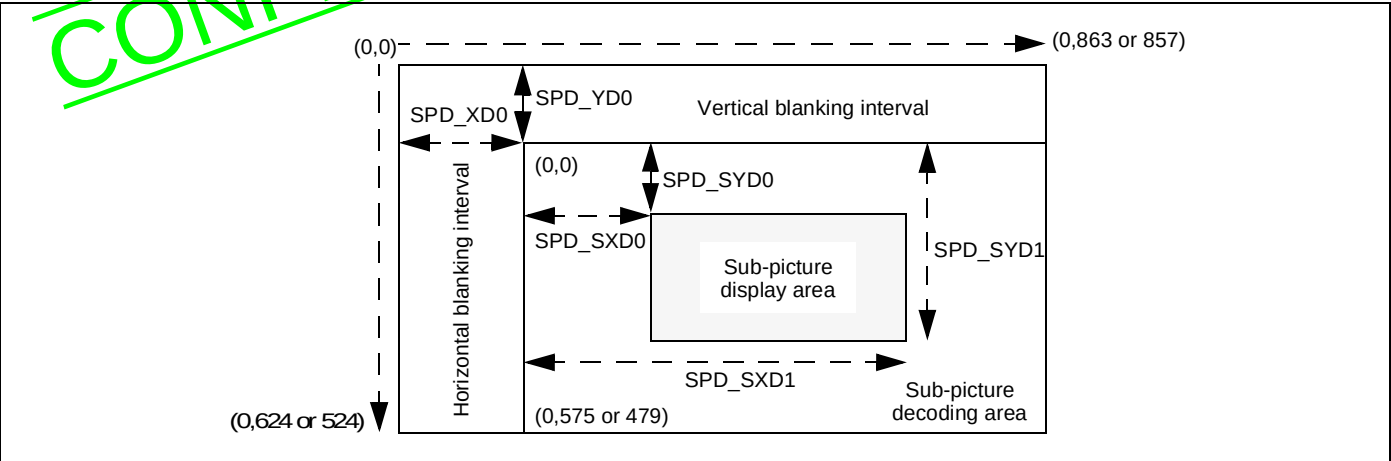


Figure 81 Sub-picture areas

21 Display planes

21.1 Overview

The graphics and display subsystem reads, processes, overlays and mixes pixel data stored in various buffers in SDRAM, and produces a combined image for display on a TV. The buffers are called the display planes.

The subsystem assumes five display planes as follows:

- Background color (see [Background color on page 151](#))
- Still picture and graphics plane, 4:2:2 YCbCr (see [Still picture plane on page 151](#))
- MPEG video plane (see [MPEG video plane on page 153](#))
- On-screen display plane (OSD) (see [On-screen display \(OSD\) on page 167](#))
- Sub-picture or cursor plane (see [Sub-picture or cursor plane on page 182](#))

The display planes are shown below.

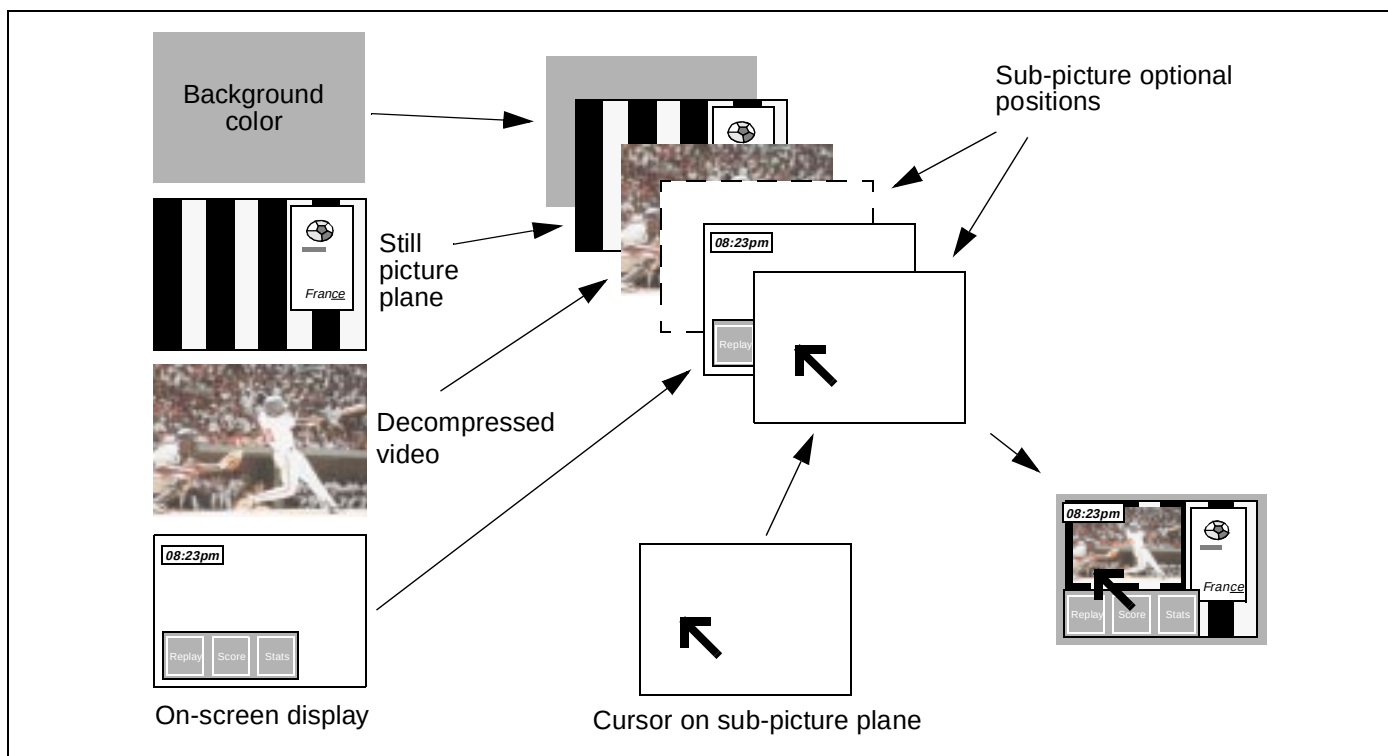


Figure 82 Display planes

The display planes are normally overlaid in the order shown above, with the background color at the back and the sub-picture used as a cursor plane at the front. The position of the sub-picture plane can be programmed by register bit **VID_OUT.SPO** as:

- The most forward layer as shown above. In this case it can be used as a cursor plane or a second on-screen display plane.
- Behind the OSD plane, in front of the MPEG video. In this case it can be used as a second on-screen display plane.

Figure 83 is a simplified block diagram of the graphics and display subsystem.

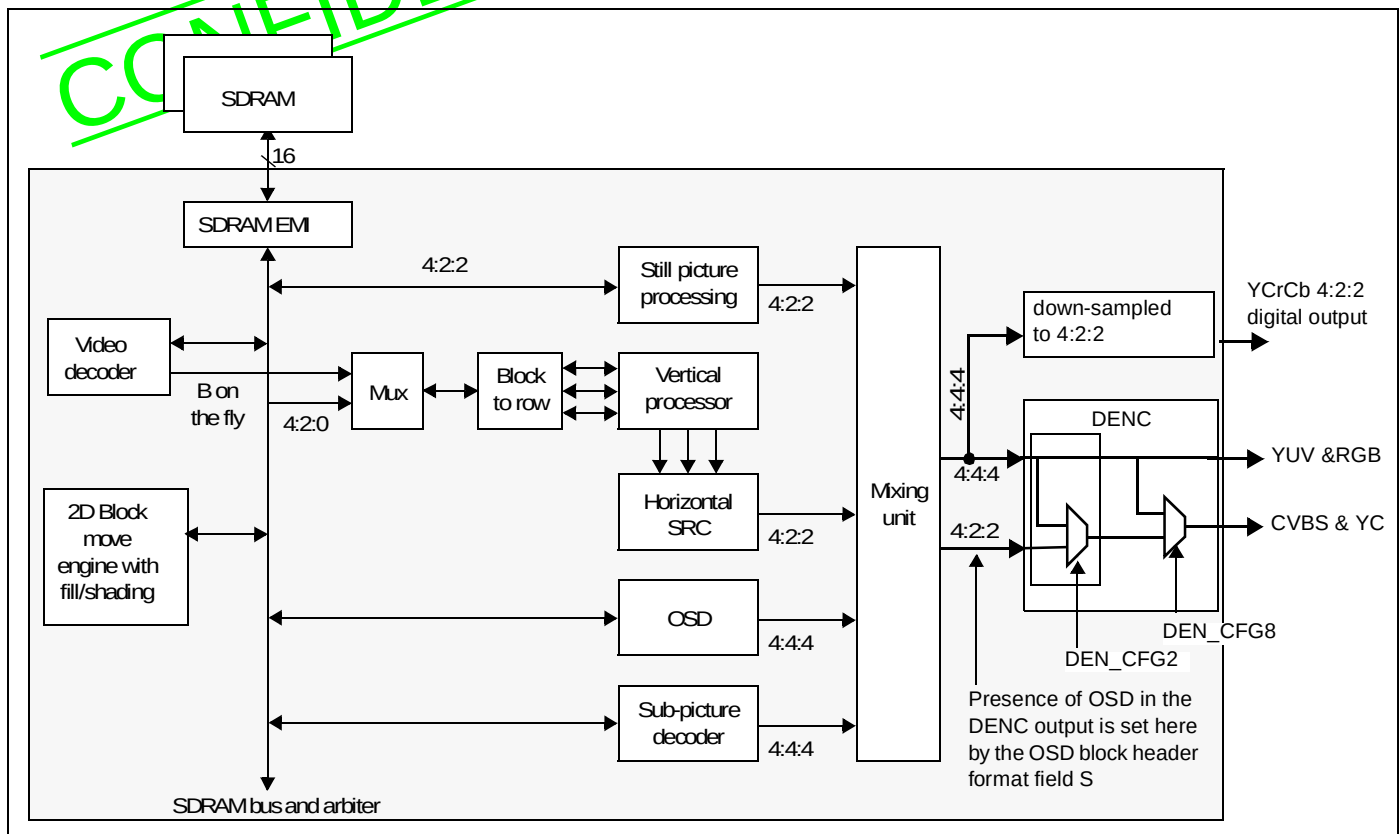


Figure 83 Graphics and display subsystem

The planes can be blended together using the mixing unit, as described in [Mixing display planes on page 182](#).

The mixing unit has two outputs as illustrated in the figure above:

- A 4:2:2 output that is input to the DENC to generate CVBS and YC output. This format is generally used for VCR recording. The OSD and sub-picture display planes can be disabled from this output as described in [4:2:2 Output control on page 184](#).
- A 4:4:4 output, used in either of the following ways:
 - As an input to the DENC to generate the YUV and RGB signals. This is generally used for TV display, and includes ALL of the available display planes.
 - As a digital signal that bypasses the DENC and is output to the YUV pins "YC0 to YC7". This signal is downsampled to 4:2:2 format by removing the chroma. It is used, for example, for connection to an external graphics device.

The routing of the 4:4:2 and 4:4:4 signals from the mixing unit, through the DENC to the DACs, is controlled by the DEN_CFG2 and DEN_CFG8 registers. When the YCrCb 4:4:2 signal is used, the presence of OSD in the DENC output is selected or deselected by the OSD block header format field S (see [Table 97: OSD block header format on page 176](#)). When the YCrCb 4:4:4 signal is used, OSD is always present in the DENC output.

21.2 Background color

The background color is always the back plane of video + still picture, with all the other display planes on top. The color of the background is defined using the three registers **BCK_Y**, **BCK_U** and **BCK_V**.

21.3 Still picture plane

The *still picture plane* is sometimes referred to as the *third display layer* (TDL).

The still picture plane can hold a picture stored in memory as a video backdrop. The picture can be, for example, a JPEG or MPEG still image. The display priority has the still picture backdrop as the second layer from the bottom, above the background color. The MPEG video, the sub-picture and then the OSD are above the still picture plane. A typical example using three of the layers is shown in [Figure 84](#) below.

The still picture plane can be faded in and out of the background color and the mixed with the video plane.

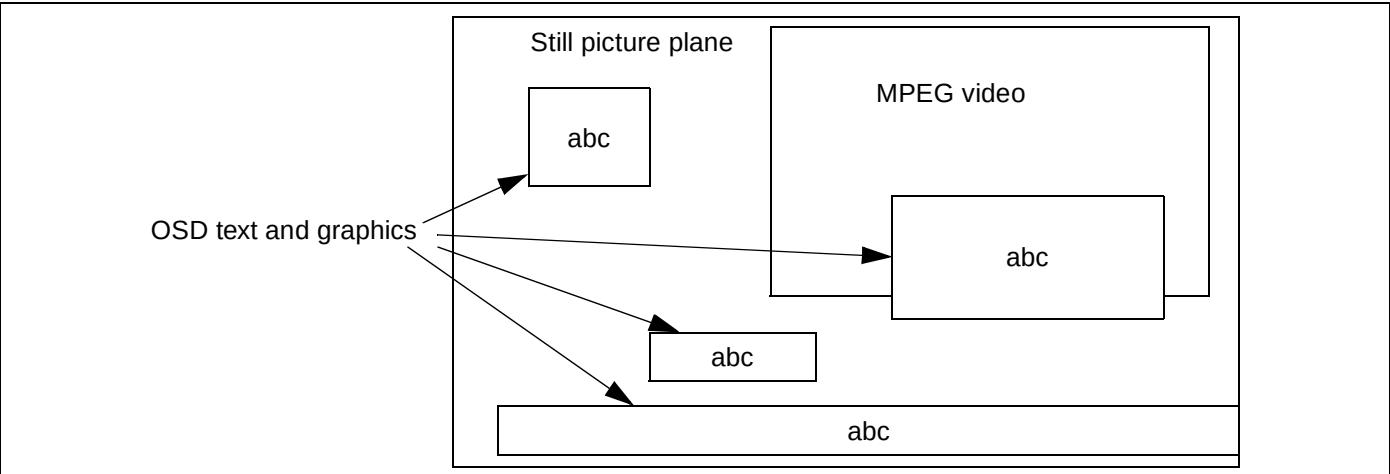


Figure 84 Use of the still picture plane

21.3.1 Still pictures

The still picture is stored in 4:2:2 raster scan format, ordered either as format A or format B in the figure below. The format is selected by register bit TDL_DCF[4]

Format A has alternating luma and chroma data types; format B has grouped luma and chroma data types like the video format. Using format B speeds-up the software conversion of the video format to the still picture 4:2:2 raster format, as the data can be copied in blocks of 32 bits.

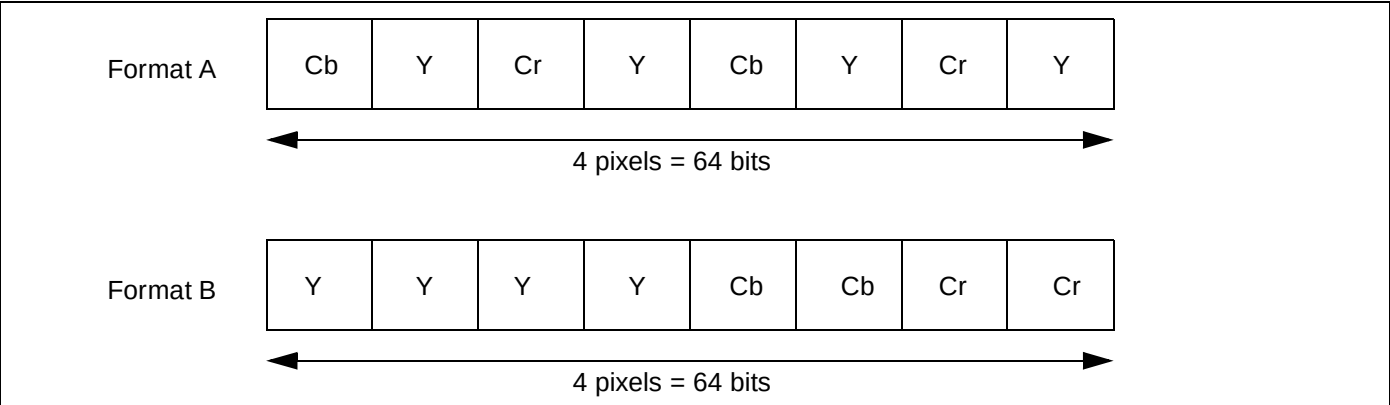


Figure 85 Formats of the still picture plane in memory

The still picture plane pipeline is a duplicate of the video plane pipeline, without the block-to-row converter or the vertical filter. The following features operate in the same way as the video plane:

- Horizontal sample rate conversion;
- Picture start X/Y co-ordinates;
- Programmable size

The start point of the displayed picture with respect to the decoded picture (the picture-start X/Y co-ordinate or pan/scan vector) is programmed by the register TDL_SCN. The vertical resolution is by line, and the horizontal resolution is by 1/8 pixel. Any part of the decoded picture that falls above or to the left of these coordinates is cut off.

The displayed still picture location and size is defined by registers TDL_XDO, TDL_XDS, TDL_YDO and TDL_YDS. Register TDL_TDW gives the number of macroblocks displayed in one row. The picture width must be stored as a multiple of 64 pixels.

21.3.2 Up and down scaling

Although there is no vertical filter, any part of the decoded still-picture can be displayed with scaling - by a factor of 2 or 0.5 in the vertical and horizontal directions. The vertical scaling is enabled by setting TDL_DCF.UDS. The field line repeat mode for up sampling is selected by setting TDL_DCF.UND and line dropping mode for down sampling is selected by clearing TDL_DCF.UND. Horizontal scaling is performed by the sample rate converter using TDL_LSR. The starting point, location and size of the display are programmed as for unscaled pictures.

21.3.3 Picture unrolling

When a new still picture is needed, it may be progressively unrolled onto the screen to replace an existing displayed still picture. This effect is shown in the figure below.

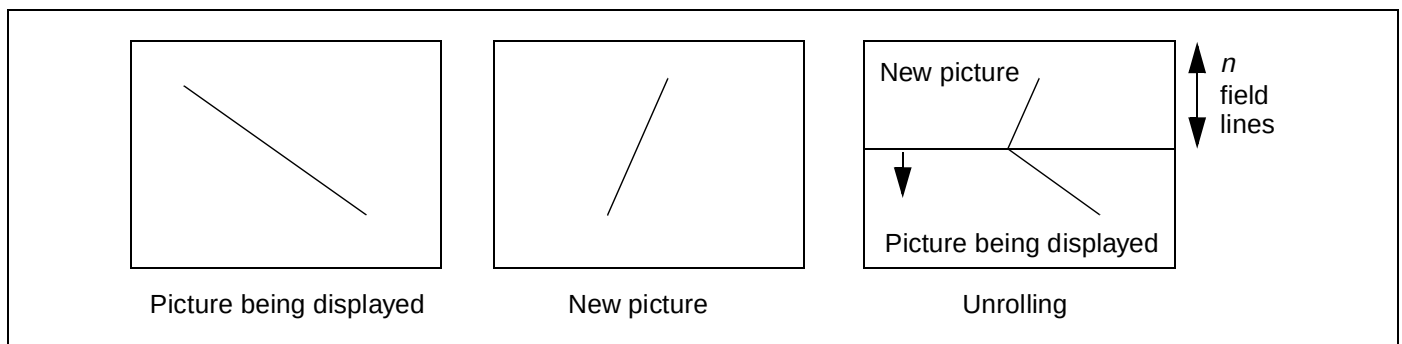


Figure 86 Still picture unrolling

The two still pictures to be displayed must be stored in memory and must have the same size and the same resolution. The displayed still picture is addressed by TDL_TOP and TDL_TEP, and the new one by TDL_TOP2 and TDL_TEP2. When the unrolling is enabled, the still picture display controller switches automatically from the new picture to the old picture when the programmed number of lines have been displayed. The number of lines before the switch (n in Figure 86) is set by software, so the software can control the speed of the unrolling. The number of lines is held in TDL_SWT, and when this value is greater than zero the unrolling starts.

21.3.4 Registers

The still picture processor uses a similar set of registers to the video plane, as listed in [Table 85](#).

Register	Bits	Function
TDL_DCF	5	Still picture display configuration
TDL_TOP	17	Buffer pointer for the displayed odd still picture
TDL_TEP	17	Buffer pointer for the displayed even still picture
TDL_TOP2	17	Buffer pointer for the displayed odd still picture 2
TDL_TEP2	17	Buffer pointer for the displayed even still picture 2
TDL_SWT	9	Picture switch line number for unrolling
TDL_SCN	15	Still picture scan vector
TDL_XDO	10	Still picture display start X offset
TDL_XDS	10	Still picture display X end
TDL_YDO	9	Still picture display start Y offset
TDL_YDS	9	Still picture display Y end
TDL_LSR	9	Still picture SRC luma/chroma resolution
TDL_TDW	6	Still picture width in macroblocks

Table 85 Still picture plane control registers

21.4 MPEG video plane

21.4.1 Introduction

The MPEG video plane displays a moving image decoded from an MPEG video stream by the MPEG video decoder. The MPEG video is displayed as the third layer, below sub-picture and OSD, and above background color and still-picture plane. The picture data is received either from the display frame-buffer area of the external memory, or directly from the MPEG video decoder (in the case of B frames in memory reduction mode).

The data is passed as an MPEG macroblock into the block-to-row converter, which generates a line-based raster from the frame store. It also performs pan/scan operation and vertical post-processing of the decoded video. The block-to-row converter is described in [MPEG video block-to-row converter \(vertical filtering\) on page 160](#).

The output of the block-to-row converter is fed to the horizontal sample rate converter (SRC), which is an 8-tap filter with two functions:

- up and down scaling of pixel data when the displayed line length is greater or smaller than the decoded picture width;
- implementation of the fractional part of the pan-scan horizontal offset.

The SRC outputs are upsampled lines each having equal numbers of luminance and chrominance samples. The SRC can be bypassed if necessary. The sample rate converter is described in [MPEG video sample-rate converter \(horizontal filtering\) on page 154](#).

The figure below illustrates the MPEG video plane pipeline.

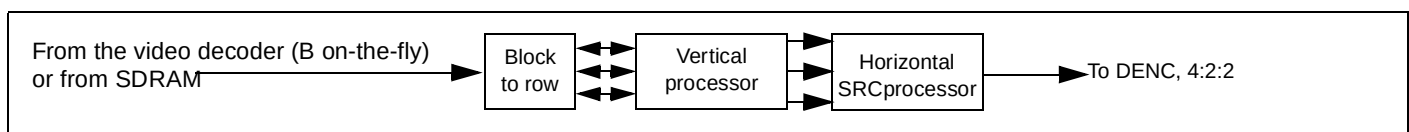


Figure 87 MPEG video plane pipeline

Setting-up an MPEG video display

The VID_DEP and VID_DFC registers must be set up with the base address of the buffer containing the picture to be displayed. This register is double-buffered; when a new value is written it is taken into account on the occurrence of a VSYNC. Therefore, it is possible to write a new value for this pointer every field, although it would normally be updated only once per frame.

The picture stored in the buffer is always treated as a frame. If ever no display is required, bit VID_DCF.EVD may be reset and a constant black value is output. The size and location of the display window is defined by registers VID_XDO, VID_XDS, VID_YDO and VID_YDS. The register values define the horizontal and vertical boundaries of the displayed picture, as shown in Figure 88.

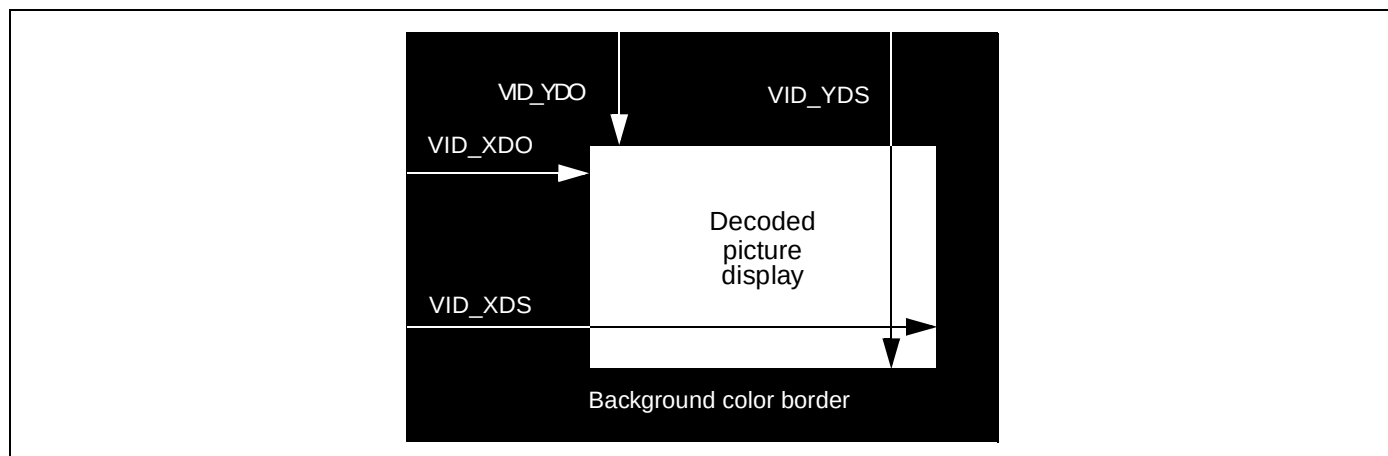


Figure 88 Display window positioning

21.4.2 MPEG video sample-rate converter (horizontal filtering)

The sample rate converter upsamples or downsamples picture data to increase or decrease the number of horizontal samples in a line.

- Upsampling is required if the horizontal size of the display is greater than the decoded picture width. For example, to display a 720 pixel-width 16:9 source image on a 720 pixel-width 4:3 display, then 540 pixels selected from each source line must be upsampled to 720.
- Downsampling is required when the resolution of the display is less than that of the decoded image. For example when square pixels are required for an NTSC image the 720 pixel wide image decoded must be downsampled to 640 pixels.

To enable the SRC, register bit VID_DCF.DSR must be reset to zero; if this bit is set to 1, the SRC is bypassed and the horizontal resolution of the decoded picture is not changed. The sample rate converter can change the sampling rate by a programmable factor. The upsampling ratio is limited to 8 and the downsampling to less than or equal to a factor of 2. The same filter is used both for upsampling and downsampling. If either extreme limits is approached, artifacts may appear in the displayed image. The SRC operates by directly interpolating samples required for the new sampling rate

by using those of the decoded picture data read from the display buffer. This is performed by an 8-tap interpolation filter with the structure shown in Figure 89.

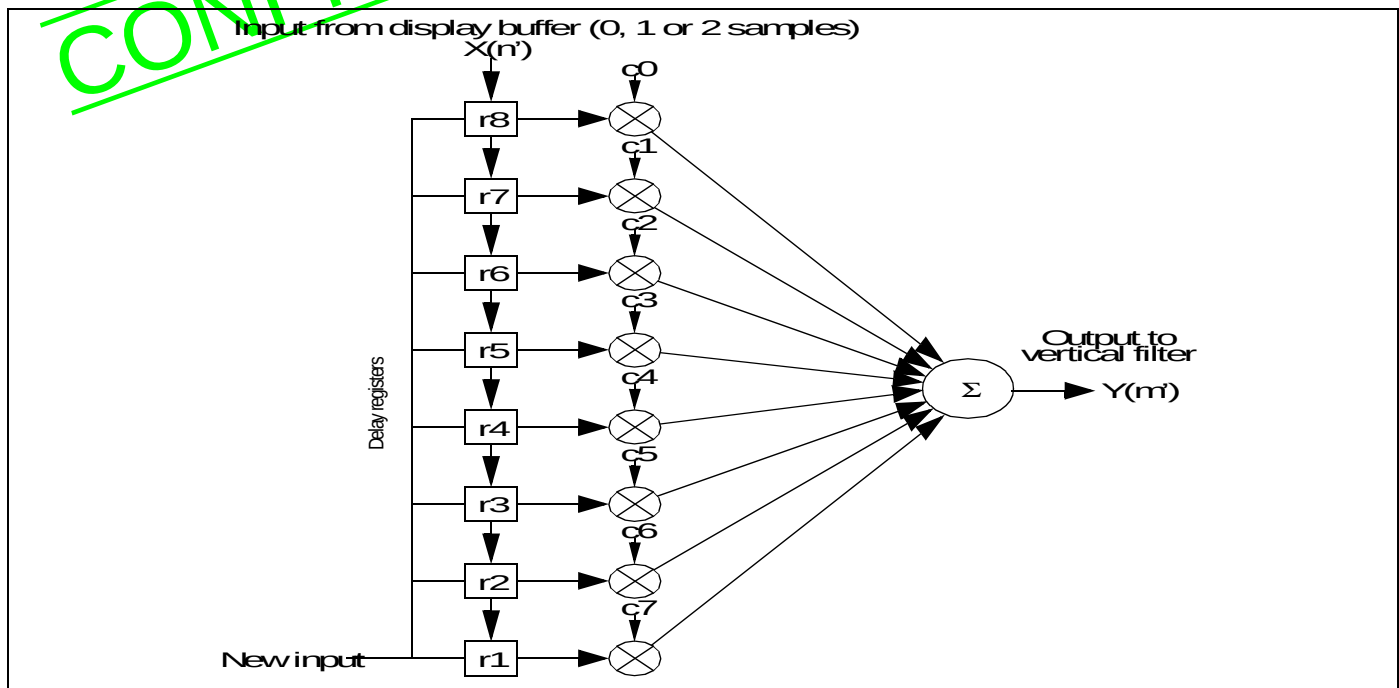


Figure 89 8-tap interpolation filter

The filter has three sets of delay registers multiplexed between the Y, C_B and C_R samples. It has 8 sets of coefficients, each set defining one of 8 sub-pixel interpolation positions. Consider an upsampling example, for sub-pixel position 0, the output is aligned with stored sample “r4”, for sub-pixel position 1, the output corresponds to an interpolated pixel position one eighth of the distance from sample “r4” to sample “r5”, and so on. The number of inputs clocked into the SRC is equal to the number of samples used in each line of the source image, and the number of outputs generated is equal to the number of samples displayed. Thus the rate of generation of outputs will be greater than the input data rate in the case of upsampling and less in the case of downsampling.

MPEG video SRC operation

The sample rate converter works in the following manner: The SRC takes block of M samples of the input signal denoted as $x(n')$, $n' = 0, 1, 2, 3, \dots, M-1$. and computes a block of L output samples $y(m')$, $m' = 0, 1, 2, \dots, L-1$.

For each output sample time m' , $m' = 0, 1, 2, \dots, L-1$ the 8 samples in the filter are multiplied with one of the 8 sets of filter coefficients the products are accumulated to give the output $y(m')$. Each time the quantity $m'M/L$ increases by one, one sample from the input buffer is shifted into the filter.

The coefficient set used will depend on the position of the sample being generated relative to the original samples of the source image. Thus after L output values are computed M input samples have been shifted into the filter delay registers.

The SRC up/down sampling factor is set up in the **VID_LSR** register. The re-sampling factors for the luminance and chrominance components are exactly the same. The resampling factor is equal to L/M . The value programmed into

VID_LSR is $256 \times M/L$. This value is used to determine both the rate of input of data into the filters and the sequence of sub-pixel interpolation positions. The mechanism by which this is achieved is shown in the figure below.

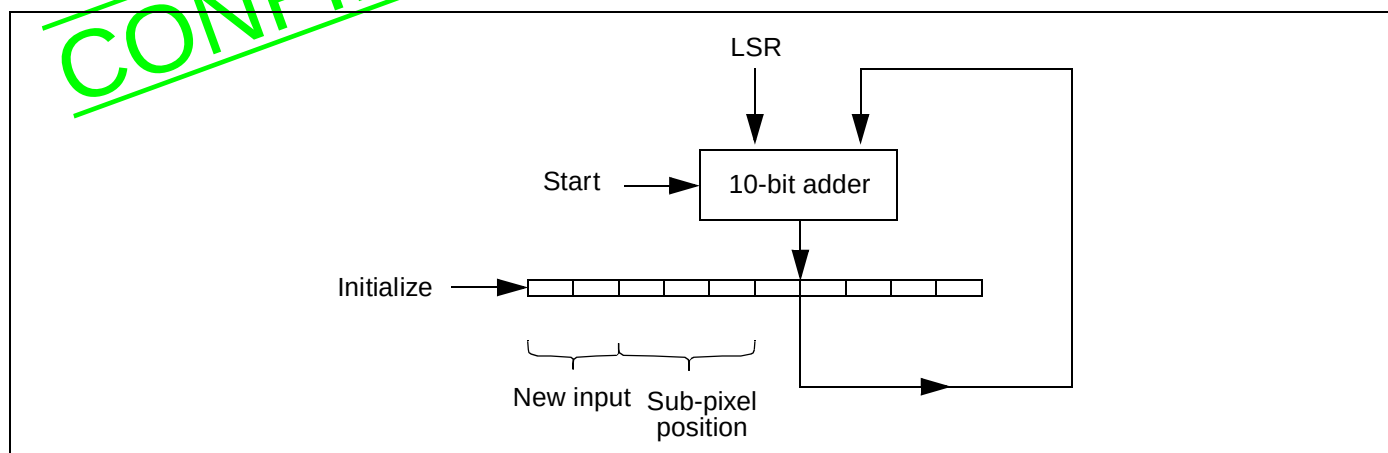


Figure 90 Up/down sampling filter control

MPEG video upsampling example

The example in [Figure 91](#) below, illustrates the operation of the sample rate converter when the upsampling ratio is 8:7. For every 8 samples clocked out of the filters, 7 samples are clocked in.

To illustrate the interpolation positions, at the right of [Figure 91](#) are shown the outputs which would occur with a simple linear interpolation (i.e. a 2-tap filter). The actual SRC output values are the 8-tap filter outputs with coefficients appropriate to sub-pixel positions 0, 7, 6, 5, 4, 3, 2, 1, 0 etc.

The SRC output is limited to lie within the range [1,254], so the codes 0x00 and 0xFF are never output, giving compatibility with ITU-R 656.

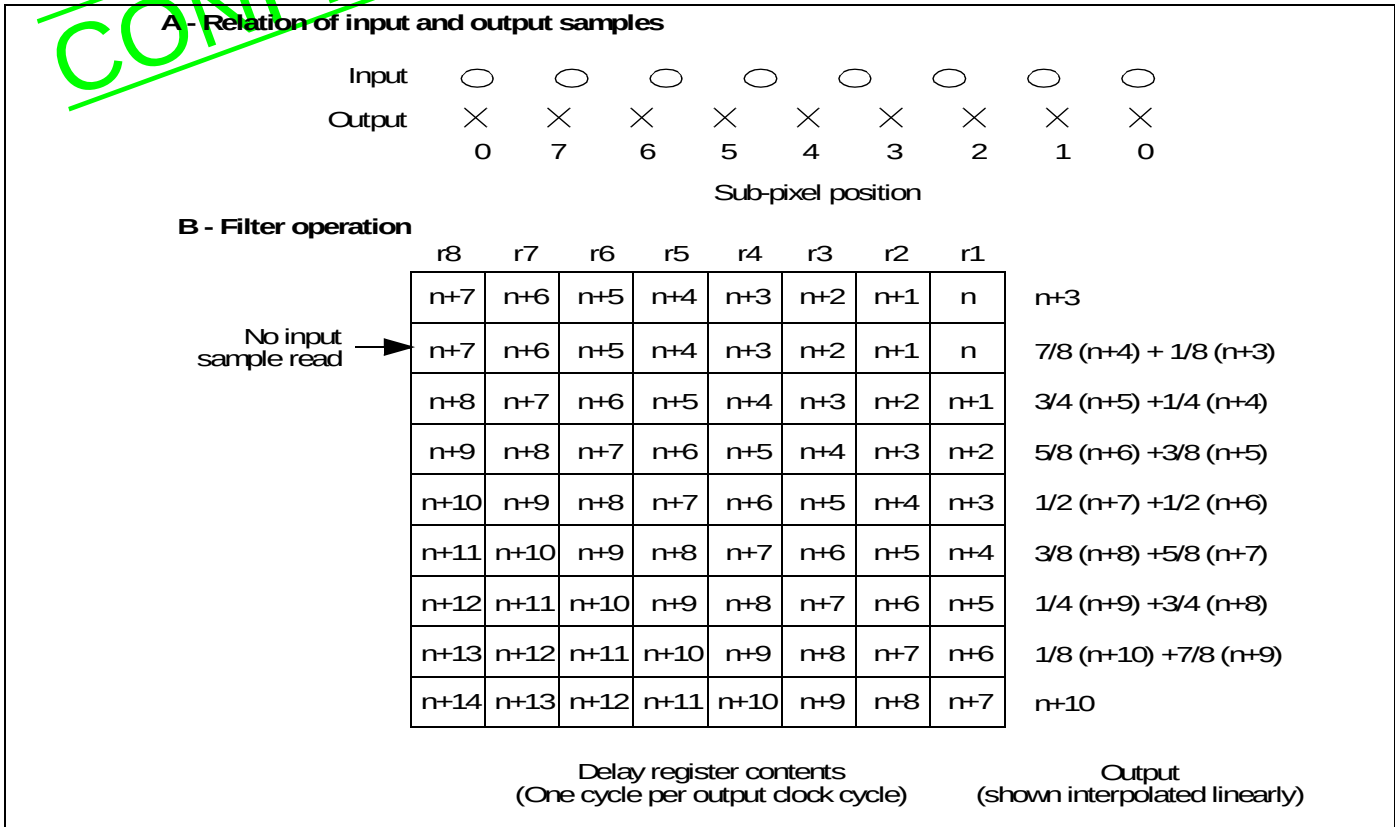


Figure 91 SRC example for 8:7 upsampling

The VID_LSR value is added into an accumulator register at a rate equal to the filter output rate. The top two bits indicate how many new inputs are to be loaded into the filter (0,1 or 2). The next three bits of the accumulator register are used to select the sub-pixel position. For example, with an upsampling factor of 8:7, the VID_LSR value is $(256/8) \times 7 = 224$. The sequence of values in the accumulator register will be as shown in Table 86, assuming that it is initialized to zero.

Accumulator register	New input	Sub-pixel position
0	yes	0
224	no	7
192	yes	6
160	yes	5
128	yes	4
96	yes	3
64	yes	2
32	yes	1
0	yes	0

Table 86 Accumulator register sequence for upsampling example

The VID_LSR value thus defines a cycle of sub-pixel positions as well as the rate of data input. If a value of less than 32 is loaded into VID_LSR, i.e. an upsampling ratio of greater than 8 is defined, there could be repeated values in the filter output. This may cause unacceptable display artifacts.

MPEG video downsampling example

The example shown in Figure 92 below, illustrates the operation of the sample rate converter when the downsampling ratio is 9:8 (720:640).

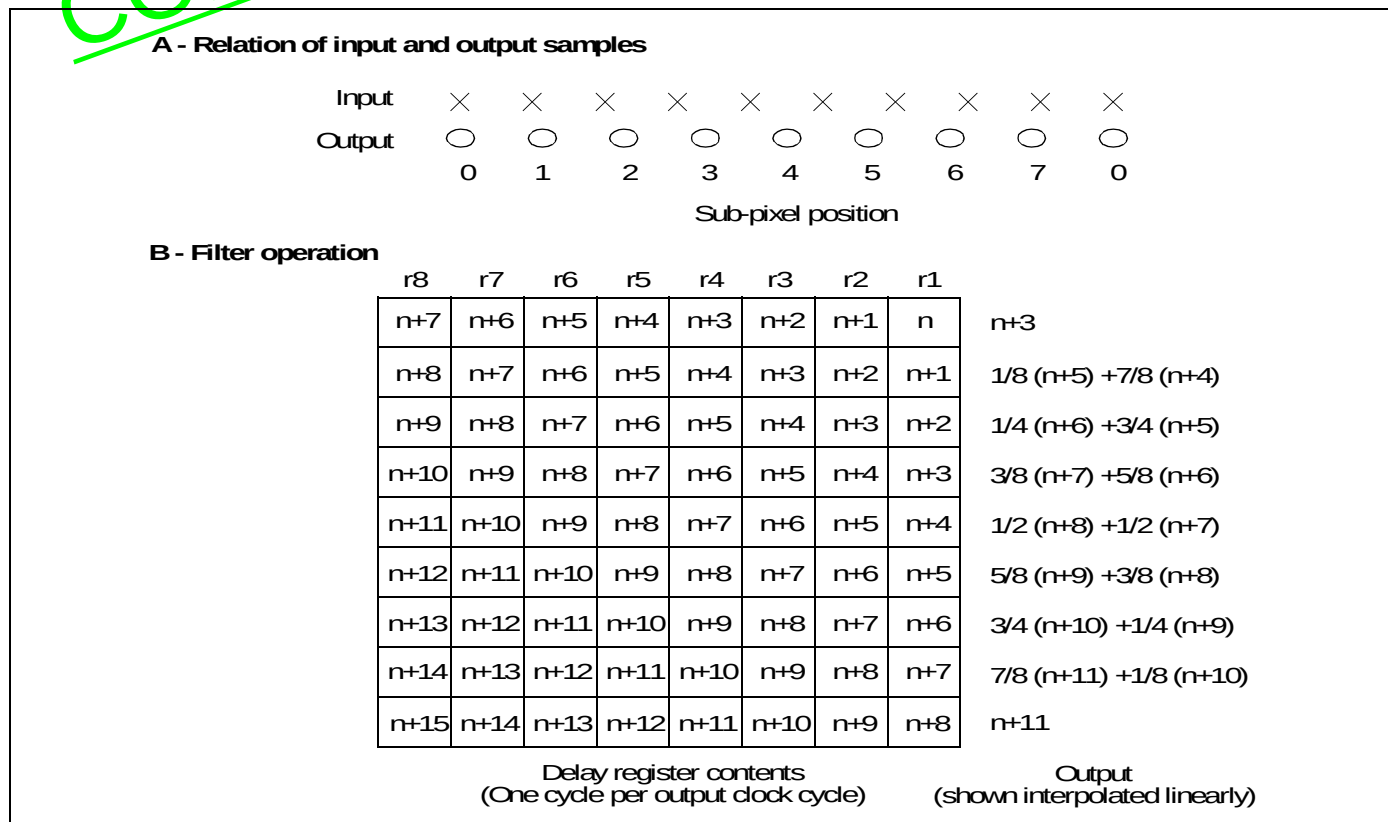


Figure 92 SRC example for 9:8 downsampling

The **VID_LSR** value required for a downsampling ratio of 9:8 is $256 \times 9 / 8 = 288$.

Accumulator register	Number of inputs	Sub-pixel position
0	1	0
288	1	1
576	1	2
864	1	3
128	1	4
416	1	5
704	1	6
992	1	7
0	2	0

Table 87 Accumulator register sequence for downsampling example

At the start of a line, the 3 sets of delay registers r1, r2 and r3 are loaded with the black value (Y=16, C_B=C_R=128).

The first output is thus derived from the inputs stored in registers r4 to r8. At the end of a line, the last eight input samples are stored in registers r1 to r8.

The last valid interpolation is between the samples stored in r4 and r5. Correct Interpolation is not possible beyond this except in the case where the next output is in sub-pixel position 0. This output is valid since coefficient C0 is zero for this position and the invalid sample beyond the end of the line is ignored.

There is thus no valid interpolation possible between the last four input samples. This is illustrated in Figure 93 below, in which 544 pixels are upsampled to 721, in which the upsampling ratio is 4:3. The VID_LSR register would be loaded with the value 192.

The number of valid outputs generated can be calculated as follows:

The ratio between the number of input and output samples is 256:VID_LSR. Given that the last output sample cannot occupy a position beyond the fourth-last input sample, the following inequality is always true:

$$VID_LSR (N-1) \leq 256 (M-4)$$

where N is the number of output samples and M is the number of input samples. The value of N is thus given by:

$$N = \lfloor 256 (M-4) / VID_LSR + 1 \rfloor$$

where $\lfloor x \rfloor$ indicates the integer part of x.

The value programmed into the VID_XDS register must be such that all samples beyond the last valid one are masked.

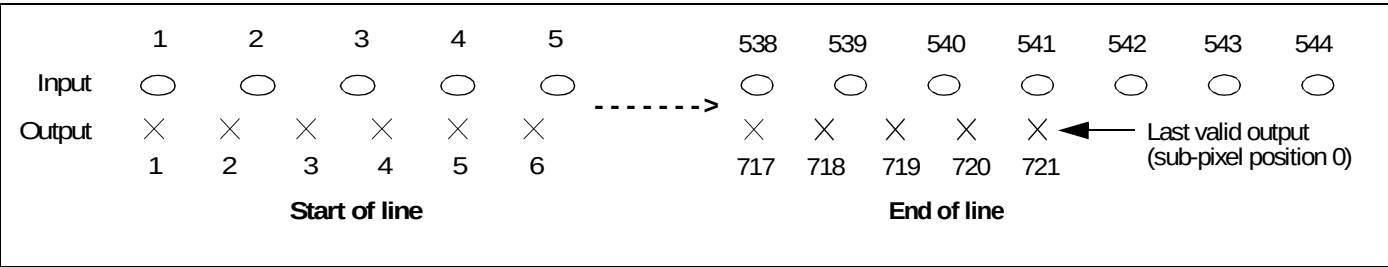


Figure 93 Upsampling from 544 to 720

21.4.3 MPEG video block-to-row converter (vertical filtering)

The block-to-row converter generates a line-based raster scan of individual video components (YCbCr) from an MPEG macroblock-organized frame store.

The block-to-row converter also performs:

- the pan/scan operation;
- vertical post-processing of decoded video.

MPEG video pan/scan vectors

The pan/scan vectors define the top-left-hand corner of the display area, with respect to the decoded picture area. Any picture that falls above or to the left of this coordinate is cut off. The displayed picture size and location is defined by registers VID_XDO, VID_XDS, VID_YDO and VID_YDS.

- Register VID_SCN defines the vertical starting point of the displayed picture in macroblocks (2 fields of 8 lines), and register VID_YDO.SCN[2:0] defines the line position within the field.
- Registers VID_PAN, VID_LSO and VID_CSO program the horizontal component with 1/8 pixel resolution.

These registers are double-buffered; when a new value is written it is taken into account on the occurrence of a VSYNC. Thus it is possible to write a new value of the pan/scan vector for every field.

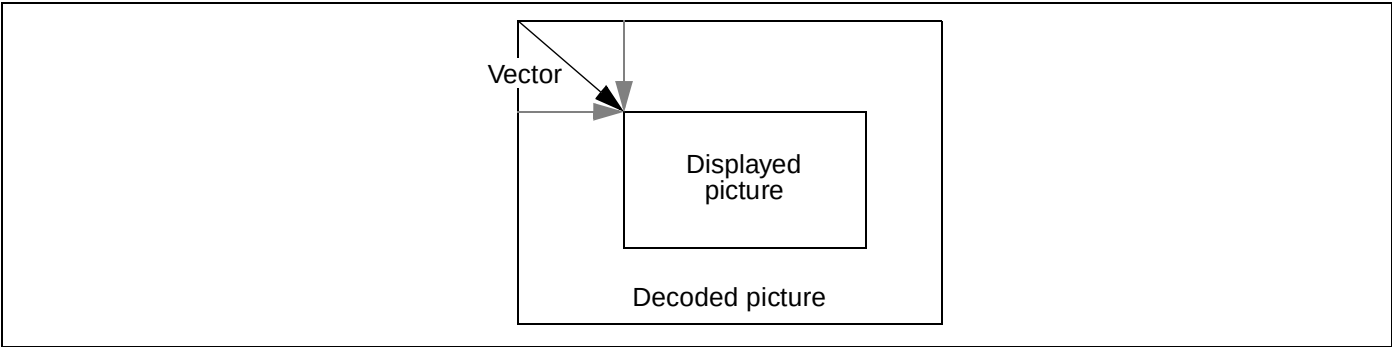


Figure 94 Pan/scan vector

The integer part of the horizontal component of the pan/scan vector is loaded into the VID_PAN register, and the fractional part defines the contents of the VID_LSO and VID_CSO registers. The relationship between these vectors is illustrated below.

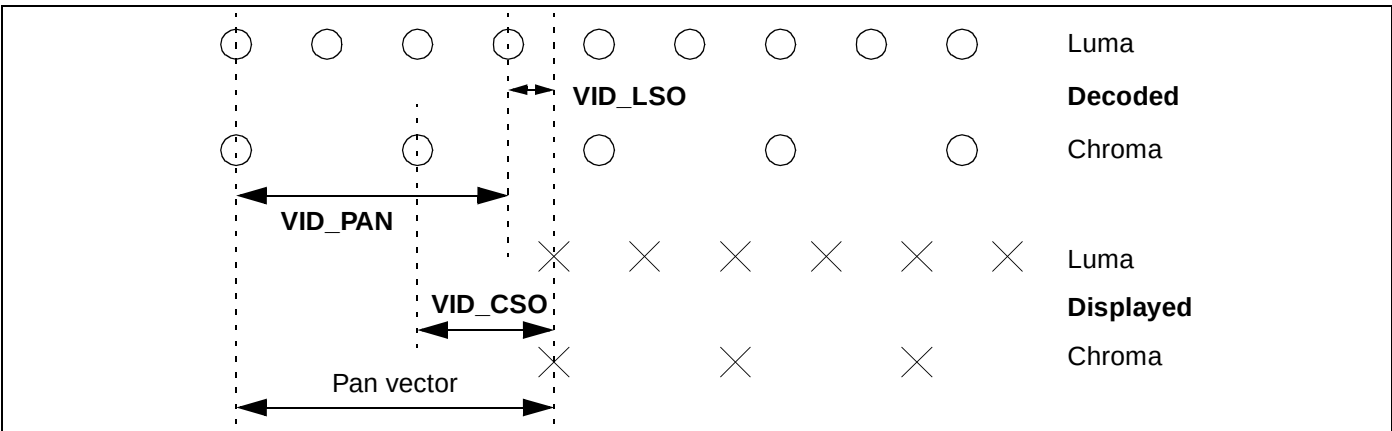


Figure 95 Components of the pan/scan vector



The numbers loaded into the VID_LSO and VID_CSO registers initialize the luminance and chrominance upsampling control registers at the start of every line. VID_LSO is set-up directly with the value of the fractional part of the pan/scan vector horizontal component. VID_CSO is set up with half of this number, plus 128 if the integer part is an odd number. The resolution to which the horizontal component can be defined is 1/8 pixel.

MPEG video vertical filter

The block-to-row converter has an output filter for vertical post-processing of the video. The vertical filter performs the chroma reconstruction from 4:2:0 to 4:2:2 format. In addition, the vertical filter upsamples and downsamples luma and chroma components for the following functions:

- zoom in by 2;
- zoom out by 2, 3 or 4;
- 16:9 and 14:9 letter box filtering.

The vertical filter operates on either field-based data (filtering odd and even lines separately as two fields), or on frame-based data (where rows are filtered sequentially, in odd/even/odd/even... sequence). The filtering mode is selected independently for luma and chroma data by register bits VID_DIS.FBS

The vertical filter implements the filter modes described and illustrated in [Table 88](#), [Figure 96](#) and [Figure 97](#). The source can be full or half resolution, interlaced or progressive. Two filter operations can be active at any one time, one for the luma data and one for the chroma data. The filtering modes are numbered and each operation is determined by programming mode numbers in the registers: VID_VFC for the chroma filter, VID_VFL for the luma filter and VID_DIS to set frame-based or field-based filtering.

MPEG video horizontal compression

To achieve the effect of a zoom out by three or four, a frame store must not only be compressed vertically by three or four, but also horizontally by three or four. The SRC is only able to downsample up to a factor of two.

In order to provide zoom out by three or four horizontally the block-to-row converter must be able to preprocess its output data for the SRC. Four modes (28 to 31) are provided for this purpose. These modes perform the same operations as modes 0 to 3, but with x2 horizontal downsampling.

MPEG video filter modes

The table below lists the filter modes set by registers VID_VFC and VID_VFL. [Figure 96](#) and [Figure 97](#) show these modes graphically. In each case the top field (odd lines) is shown on the left and the bottom field (even lines) on the right.

Some mode-pairs, such as 1 and 2, or 14 and 15, appear identical. However, in one mode the top field is filtered first, and in the other mode the bottom field is filtered first.

Mode: set by registers VID_VFC and VID_VFL	Vertical scaling factor	Filtered or repeated	Notes
0	1/4	Filtered	Vertical downsizing by more than 3/4 when in memory reduced (B on-the-fly) mode, is forbidden.
1	3/8	Filtered	
2	3/8	Filtered	
3	1/2	Filtered	
4	3/4	Filtered	
5	3/4	Filtered	
6	7/8	Filtered	
7	1	Repeated	
8	1	Filtered	Vertical offset of 1/4 sample height
9	1	Filtered	Vertical offset of 1/4 sample height
10	3/2	Filtered	
11	7/4	Filtered	
12	2	Repeated	
13	2	Filtered	
14	2	Filtered	Vertical offset of 1/4 sample height
15	2	Filtered	Vertical offset of 1/4 sample height
16	2	Filtered	Vertical offset of 1/2 sample height on odd fields
17	2	Filtered	Vertical offset of 1/2 sample height on odd fields
18	4	Repeated	
19	4	Filtered	
20	4	Filtered	
21 to 26	Reserved		
27	3/4	Filtered	As mode 4 but horizontally downsampled by 2
28	1/2	Filtered	As mode 3 but horizontally downsampled by 2
29	3/8	Filtered	As mode 2 but horizontally downsampled by 2
30	3/8	Filtered	As mode 1 but horizontally downsampled by 2
31	1/4	Filtered	As mode 0 but horizontally downsampled by 2

Table 88 Vertical filter modes

CONFIDENTIAL

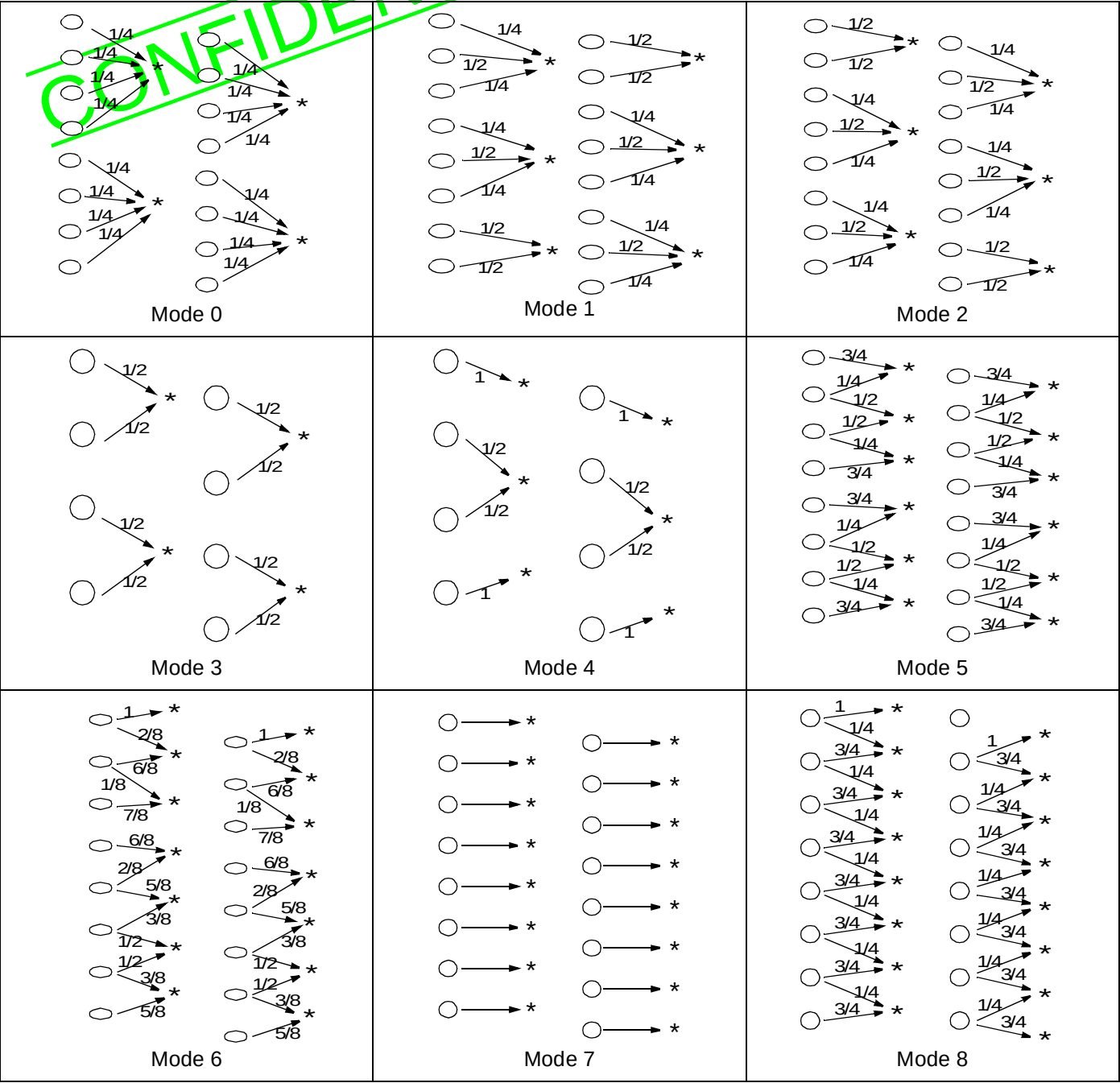


Figure 96 Block-to-row filtering modes for luma and chroma data

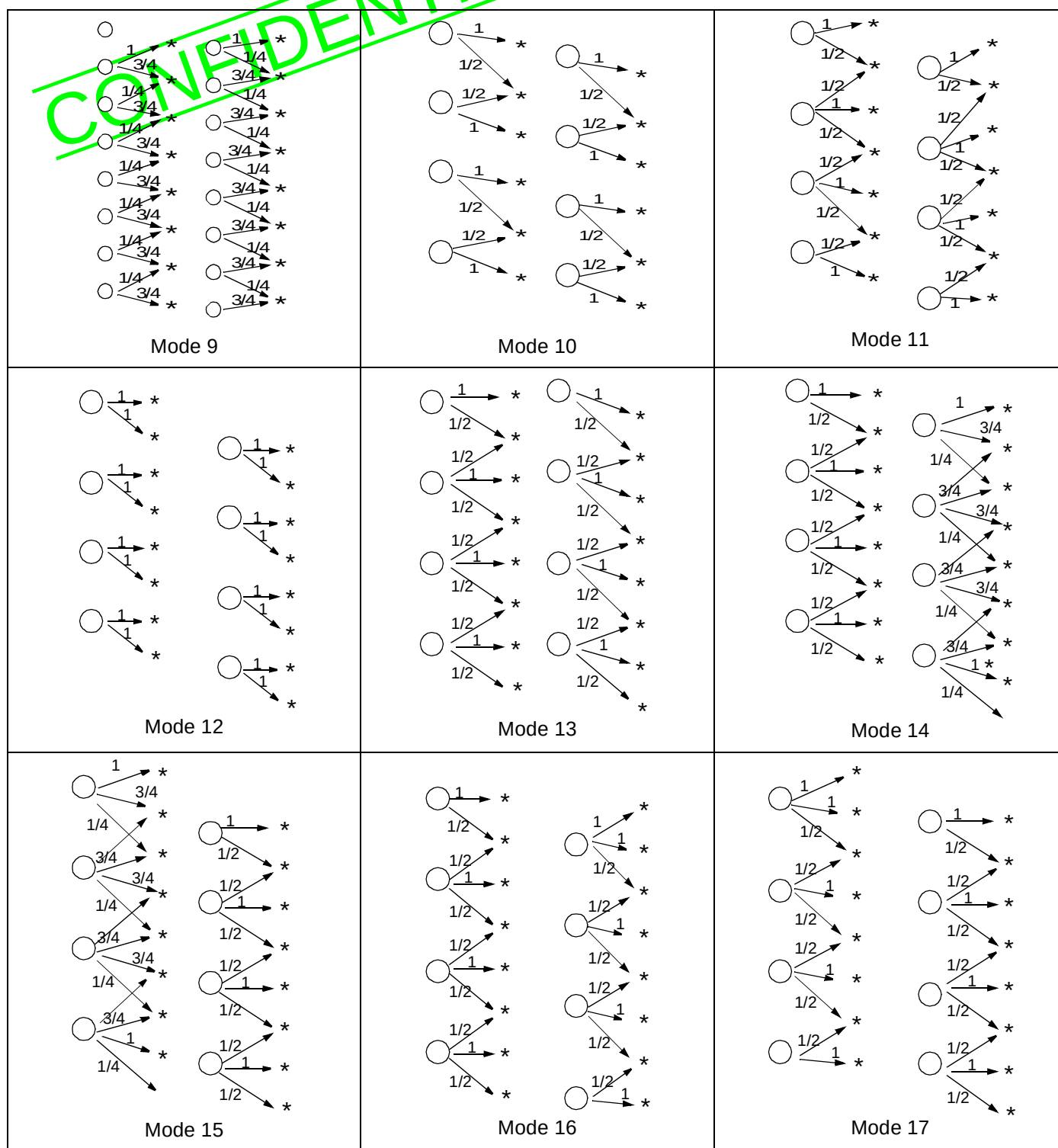


Figure 96 Block-to-row filtering modes for luma and chroma data

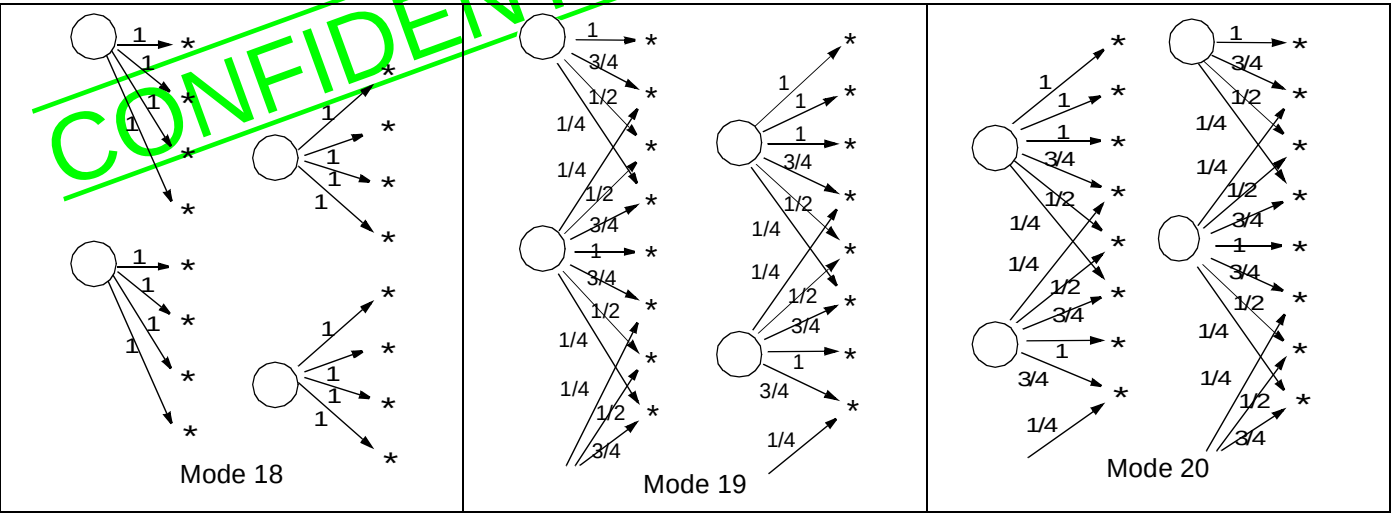
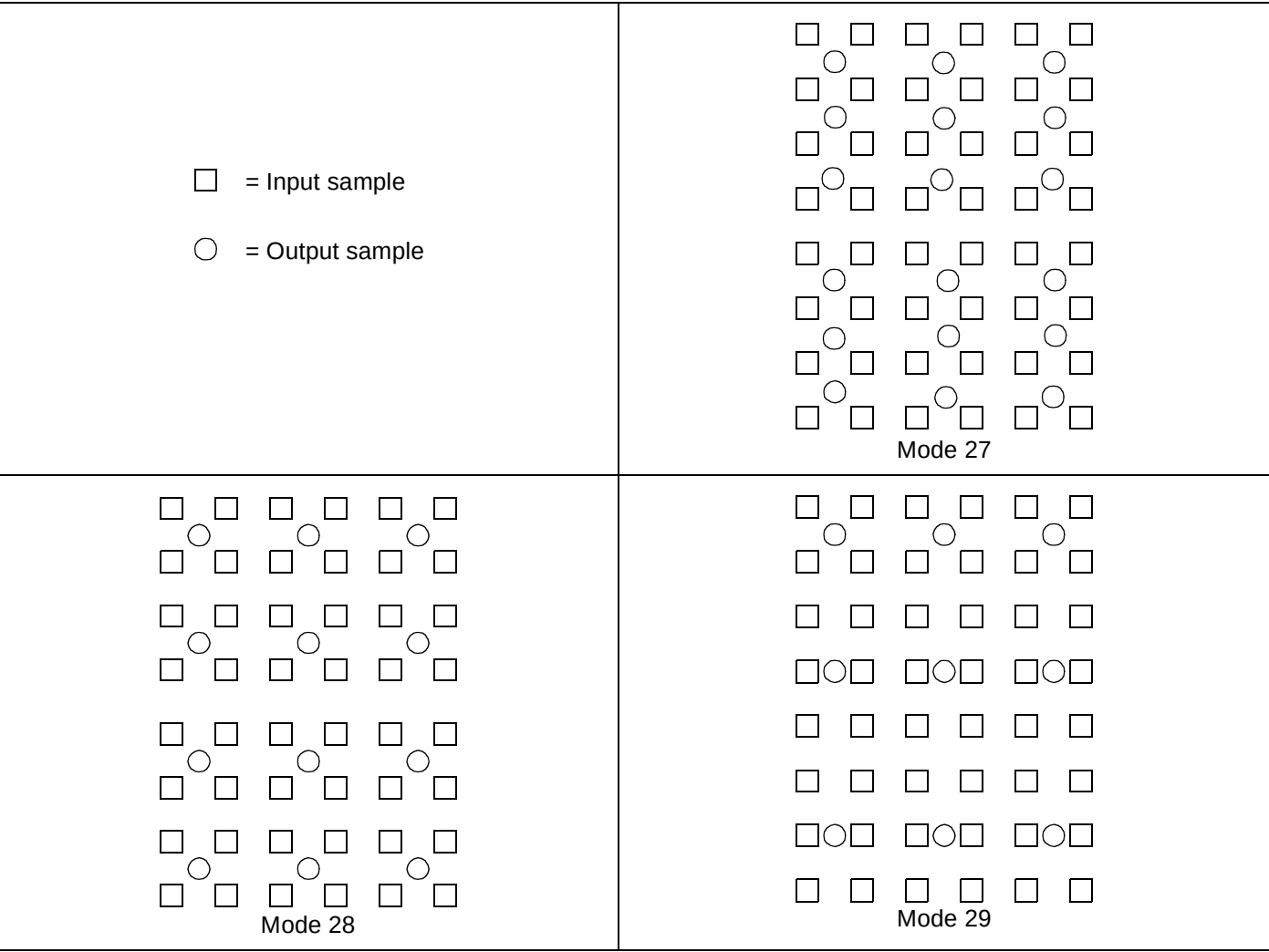


Figure 96 Block-to-row filtering modes for luma and chroma data

Horizontal downsampling is achieved in modes 27 to 31 by averaging adjacent samples as shown below.



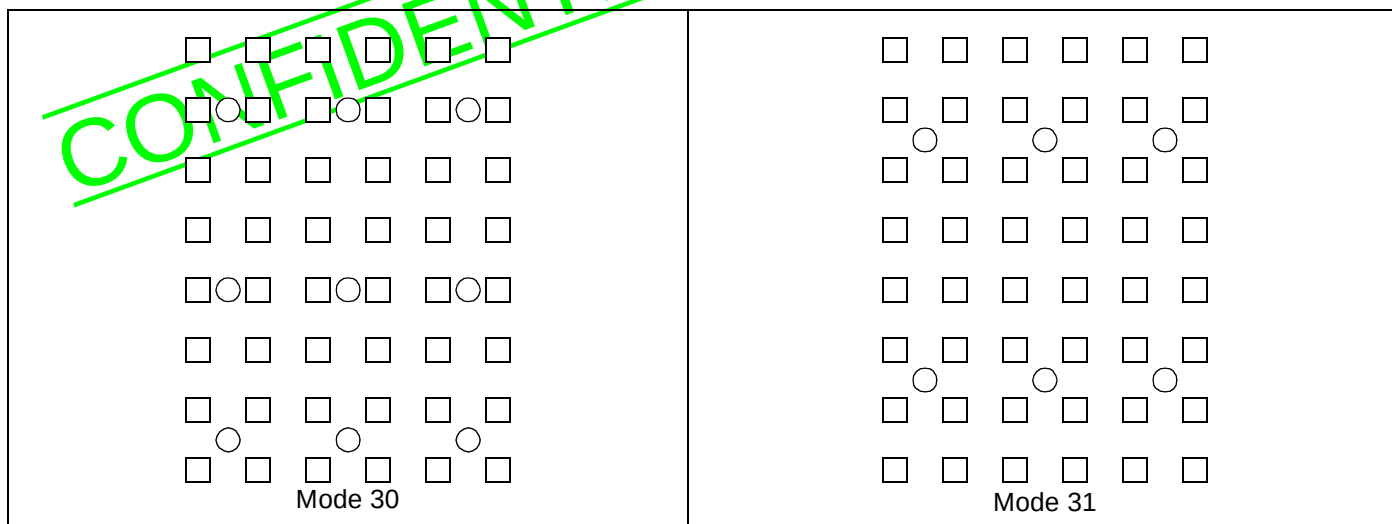


Figure 97 Filtering modes incorporating horizontal downsampling

The table below lists the suggested filter modes to use for different video processing requirements.

Display	Source		Suggested filter modes		Field or frame based filter VID_DIS.FBS
	Resolution	Temporal resolution	Luma VID_VFL	Chroma VID_VFC	
Full screen	Full	Interlaced or progressive	7	14	00
	Full	Interlaced or progressive	7	12	00
	Full	Progressive	7	8	01
	Half	Interlaced or progressive	13	18	00
	Half	Progressive	8	15	11
16:9 letter box	Full	Interlaced or progressive	5	10	00
	Full	Progressive	1	5	11
	Half	Progressive	5	10	11
14:9 letter box	Full	Interlaced (field based)	6	11	00
14:9 letterbox	Half	Progressive	6	11	11
Zoom in x 2	Full	Interlaced or progressive	14	19	00
	Full	Progressive	8	15	11
Zoom in x2	half	Progressive	15	19	00
Zoom out x 2	Full	Interlaced/progressive	3	7	00
	Half	Interlaced	7	14	00
	Half	Progressive	7	8	01
Zoom out x 3	Full	Interlaced or progressive	29	27	00
	Half	Interlaced	5	10	00
	Half	Progressive	2	4	00
Zoom out x 4 ¹	Full	Interlaced or progressive	28	3	00
	Half	Interlaced	3	7	00
	Half	Progressive	3	7	00

Table 89 Vertical filter mode selection

1. There are two ways to prevent increased bandwidth and consequent picture degradation in zoom outX4 mode:
 - For the STi5510 (or STi5512), set register bit VID_DCF.STP_DEC to 1 to prevent the MPEG video decoder from performing reconstruction accesses to the SDRAM during the video picture display.
 - Alternatively, for the STi5512 only, set register bit VID_DIS.ZoomOutX2 to 1. This activates a line drop, and only alternate lines are read into the memory. This increases the zoom-out by X2 without increasing the bandwidth. If this mode is used, register bit VID_DCF.STP_DEC cannot be used.

For CIF pictures (352x288) register bit VID_DFS.CIF enables a whole frame to be sent on both fields. This artificially doubles the vertical size of the picture (352x576). This function is forbidden in “on-the-fly” mode.

21.4.4 Degradation mode

Sometimes the device is used in a configuration where the available bandwidth on the SDRAM interface is limited. The possible reasons for this include a low clock frequency to use cheaper SDRAMs, or the processor making heavy use of the SDRAM. MPEG decode and display, being a real-time process and also a heavy user of SDRAM memory bandwidth will then require a graceful degradation mode.

A small piece of hardware is implemented in the decoder to measure the effective distance (in pixels) between the display process and the decode process. Under conditions of limited bandwidth the decoder will become late and therefore may get caught by the real-time limited display process.

Degradation mode can be enabled and disabled using the register VID_PTH. A threshold or minimum allowable distance between the decode and display processes can also be set. If this threshold is crossed the decoder will automatically insure that any bidirectionally predicted macroblock access will result in only a single prediction access to external memory thus reducing the bandwidth required by the decoder and allowing recovery.

21.5 On-screen display (OSD)

21.5.1 Introduction

The integrated On-Screen Display (OSD) unit overlays the video picture with software-generated graphics. The display priority puts the OSD in front of the MPEG video. It can be configured to be either in front of or behind the sub-picture plane by clearing or setting VID_OUT.SPO respectively. The OSD can be enabled by setting OSD_CFG.ENA. The OSD bit-map is defined with respect to the display area and is independent of the decoded picture size and any pan/scan offset.

Warning: The OSD region can NOT be defined with an odd number of pixels; it MUST be defined with an even number of pixels. The output from the OSD is in 4:4:4 format/width or height.

Register bit VID_DCF[2] controls the polarity of the Enot0 signal going to the OSD block.

The OSD has the following features:

- Linked list memory management mode or STi3500A compatibility mode;
- Selectable 2, 4 or 8-bits per pixel palette modes giving 4, 16 or 256 palette colors;
- Either 6-bit luma resolution and 4-bit chroma resolution per component or 8-bit luma resolution and 8-bit chroma resolution;
- Programmable 4-bit mixing factor for each OSD region to blend the video plane and OSD data;
- When anti-aliasing is enabled, each color in an OSD region can be assigned a separate 6-bit mixing factor for mixing with video;

- Optional anti-flicker and anti-flutter filters,
- Half resolution mode.

These features are described in the following sections.

The OSD unit uses color *look-up-tables* (LUTs), also called *palettes*, with 2-bit, 4-bit or 8-bit input. A 2-bit LUT means that four colors can be used at once, and each pixel of the bit-map occupies only two bits of memory. A 4-bit LUT gives 16 colors and an 8-bit LUT gives 256 colors. The palette of 4, 16 or 256 predefined colors is loaded into the SDRAM by software using the shared memory interface. The palette modes are described in [Section 21.5.6: Color palette](#) on page 174.

The output from the LUT can be 14-bit pixels (6-bit Y, 4-bit Cb, 4-bit Cr) or 24-bit pixels (8-bit Y, 8-bit Cb, 8-bit Cr) plus one bit or six bits for transparency control. The color modes are described in [Section 21.5.6: Color palette](#) on page 174.

The OSD can consist of a number of display regions, each with its own palette and characteristics. The number of OSD regions resident in memory at any time is limited only by the amount of memory available. Each region has a specification, stored in memory, which contains a header, possibly including a palette, and a bit-map. The specifications for the regions are linked in a list structure. The bit-map data in each specification is contiguous with the palette information, as shown in [Figure 103: OSD specification on page 172](#). The bit-map refers to the 2-, 4- or 8-bit color definitions in the palette to create the required picture.

During the display of an image a small state machine first picks up the palette from SDRAM and loads it into the LUT then the OSD region start and stop addresses are read. When the display reaches the OSD start position (defined in the bit-map) the bit-map is sent pixel by pixel to the LUT and the display switches from video to the output of the LUT or a mixture of both.

This process continues until the defined stop position. Thus, for the defined OSD region, the video display is overlaid by the colors which are defined by a combination of the LUT and the bit-map.

21.5.2 Using the OSD

The OSD is enabled if register bit OSD_CFG.ENA is set. The OSD top-field start address is defined by register OSD_TOP, and the bottom-field by register OSD_BOT.

The line numbers, used to define the top and bottom of an OSD region, are the internal (field) line numbers illustrated below. It is therefore possible to share the same OSD specification for both fields of a frame. In this case, OSD_TOP and OSD_BOT are loaded with the same address.

OSD specifications can be written into the SDRAM using the ST20 or the block move DMA. They can be rapidly moved within SDRAM using the SDRAM block move function.

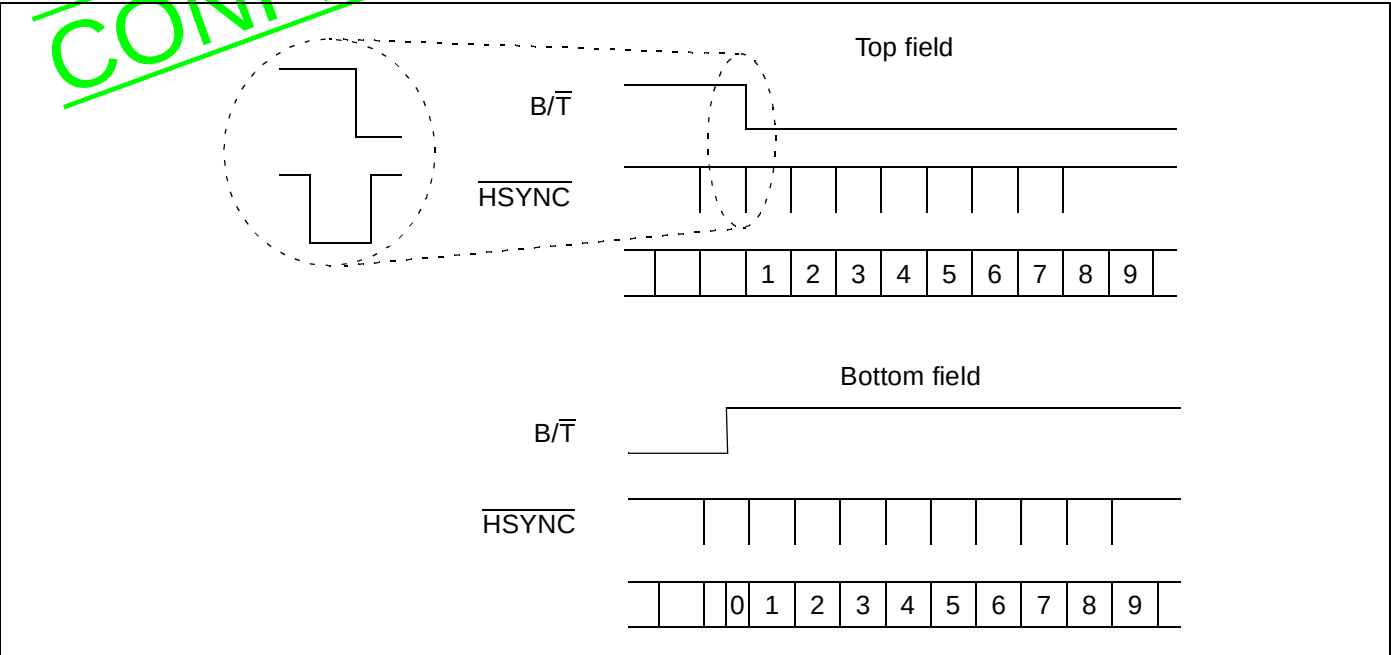


Figure 98 Internal line numbering

21.5.3 OSD regions

The OSD function can be used to display a user-defined bit-map over any part of the displayable (i.e. non-blanked) screen, independent of the size and location of the active video area (defined by VID_XDO, VID_XDS, VID_YDO, VID_YDS). This bit-map can be defined independently for each field.

The OSD consists of one or more regions in the display. Each region is a rectangle, and can have its own palette and other properties. The figure below shows examples of OSD regions. Region 3 shows that the OSD can be outside the active video area.

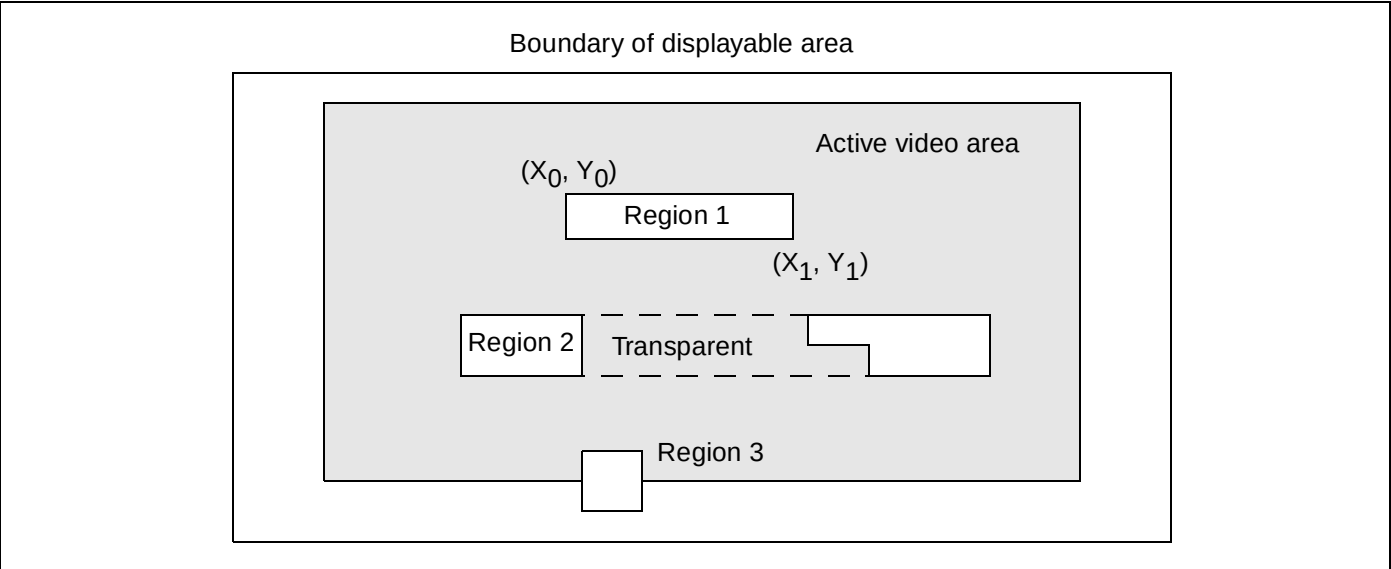


Figure 99 OSD regions

No display line can be included in more than one active OSD region, so only one OSD region can be active on a line. If two areas of OSD are required which include the same display line then one region must be defined which includes both areas. For example, the figure below shows two areas, marked A and B, with some display lines used in both areas. If these areas are to be active at the same time then one region, marked C, must be defined, which includes both areas. The area of C outside A and B can be defined as transparent.

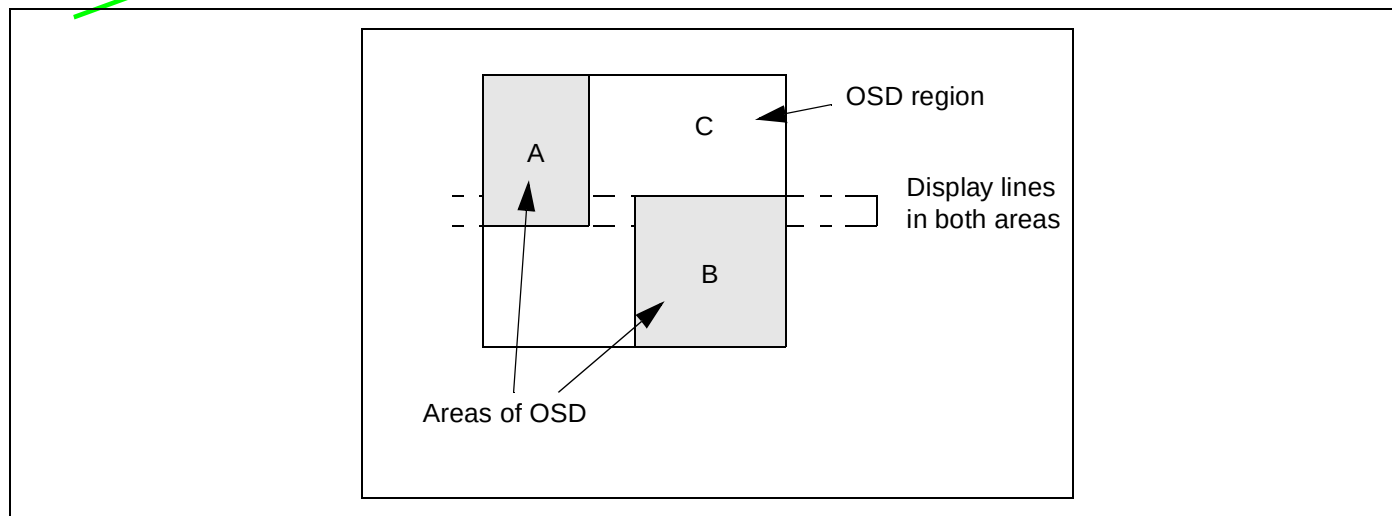


Figure 100 Two display areas using the same display lines

21.5.4 OSD specification

An OSD specification is two lists of blocks of 64-bit words, stored in SDRAM. One list is for the top field and one for the bottom. Generally the lists are linked lists, as shown in [Figure 101](#). The order of the blocks in the list is the order of the regions from top to bottom of the display. The last block in an OSD specification must point to an invalid header, which gives a starting line beyond the displayable area (example 0xFFFFFFFFFFFFFFFF). This invalid header will end the OSD.

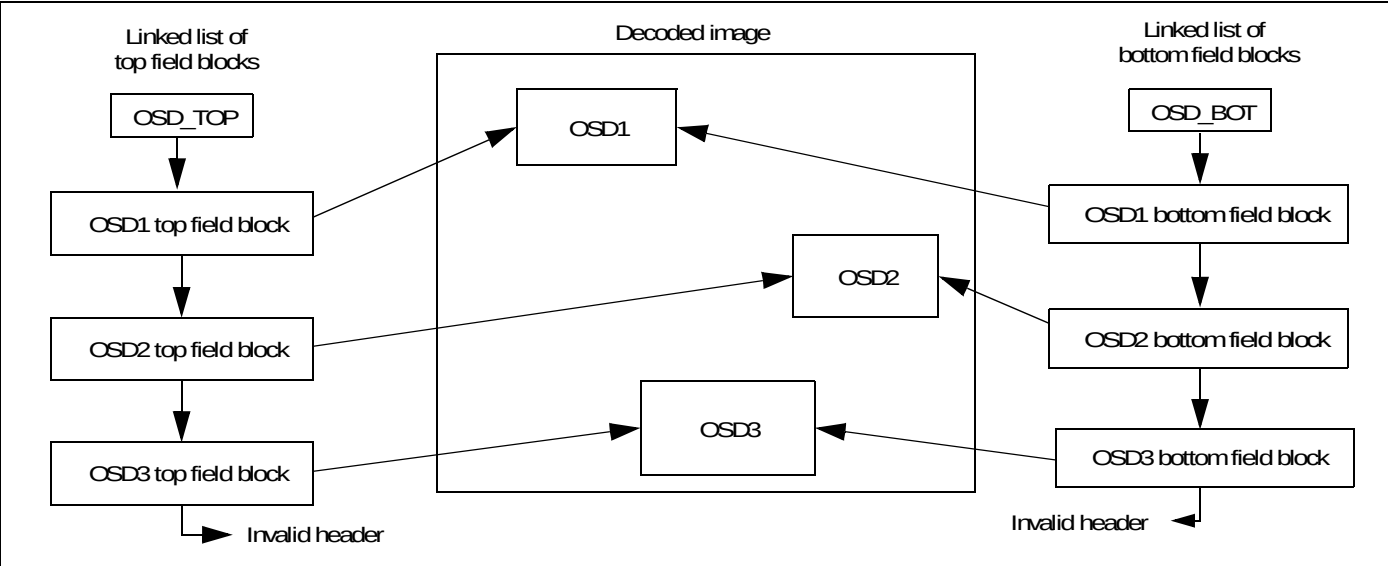


Figure 101 Linked list structure for OSD data

The only case in which linked lists are not used is when the palette mode is 2 bits for 2 pixels, i.e. the palette mode flags are set to M=1, Q=1, E=0, as described in [Section 21.5.6: Color palette](#) on page 174. In this case the blocks must be contiguous in memory, so the start of each block must immediately follow in memory the end of the previous block, as shown in [Figure 102](#). No linked list is allowed after a 2 bits for 2 pixels zone.

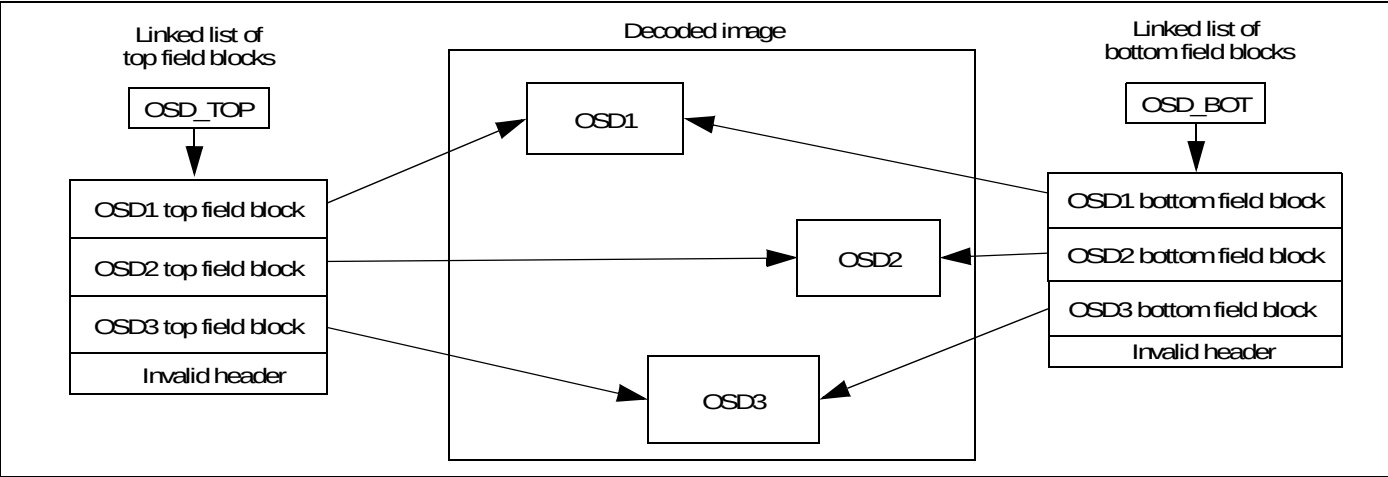


Figure 102 Block structure for OSD data in 2 bits per 2 pixels mode

Each block defines one field of one region and includes a header, an optional palette and a bit-map. Each block must be aligned on a 64-bit boundary and the first block of each field must be aligned on a 256-bit boundary. [Figure 103](#) shows a linked list of two OSD blocks.

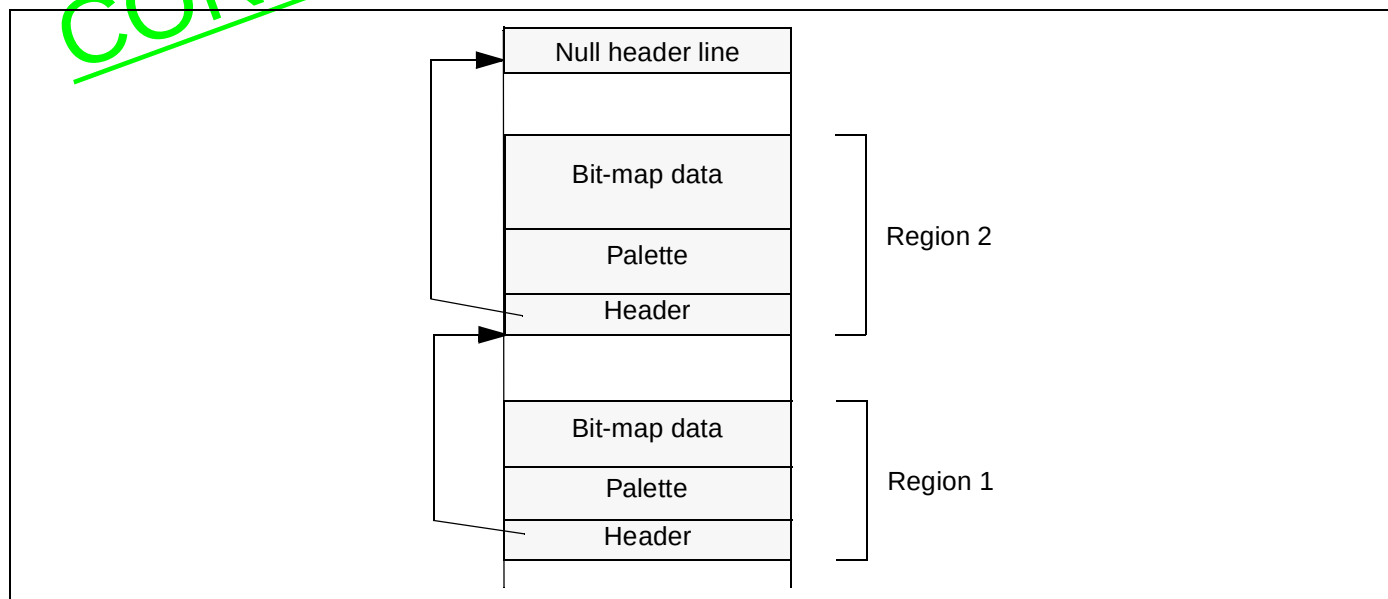


Figure 103 OSD specification

Each region has associated with it a palette defining 4, 16 or 256 colors, used by the bit-map. If required, one of these colors can be “transparent”, allowing the background to show through. Each region may have its own palette, or if a sequence of regions uses the same palette then the palette need only be defined in the first region of the sequence.

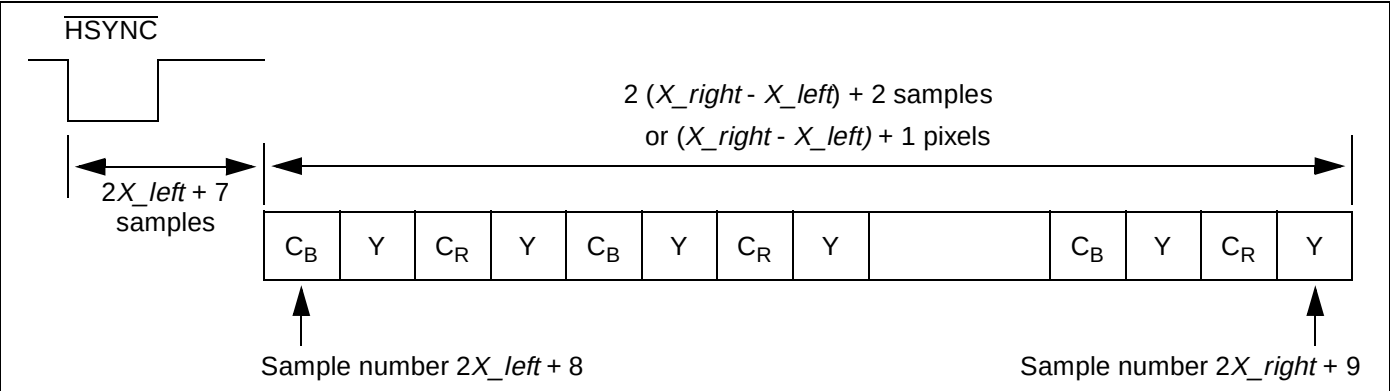
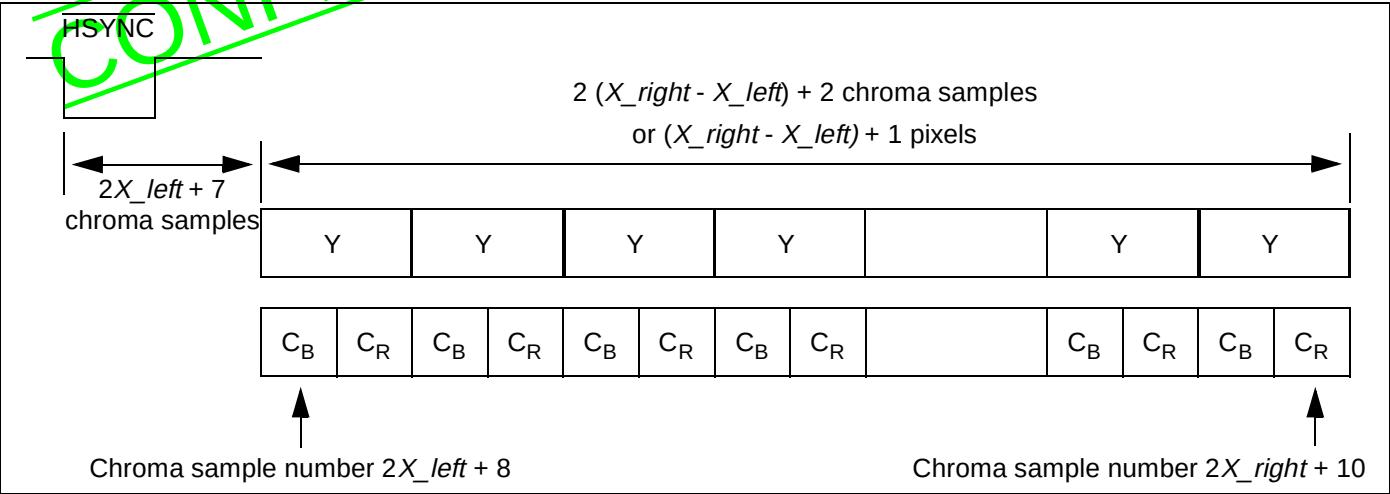
The header of each block contains a definition of the boundaries of the region, a pointer to the next region and other control information. The format of the palette depends on the palette mode, as described in [Color palette on page 174](#). The formats are given in [OSD block header format on page 176](#).

21.5.5 OSD region position

The position of each region of the OSD is defined in the header of the specification block. The positions of the left and right edge samples of an OSD region are defined as follows, in units of PIXCLK cycles from the falling edge of $\overline{\text{HSYNC}}$:

- *left edge position* = $(2 \times X_{\text{left}}) + 8$
- *right edge position* = $(2 \times X_{\text{right}}) + 9$

where X_left and X_right are the values defined in the header of the OSD region specification. This is illustrated in the following two figures. The first sample output in an OSD region is always a C_B value.



The top and bottom of the regions are defined by the values Y_top and Y_bottom , which are also in the block header. These values are specified in units of display lines. The top line specified in the first word of an OSD region specification must be greater than or equal to 3.

21.5.6 Color palette

Each specification block after the first can either define a new palette or use the same palette as the preceding region. If a new palette is defined then it is held in SDRAM immediately after the header and before the bit-map. The P flag in the header defines whether the palette follows the header; when P=0 the palette for the region is immediately after header, when P=1 the palette is the same as for the preceding region.

Palette modes

The palette mode defines the bits per pixel in the bit-map and the pixel resolution. The palette mode can be different for each OSD region, and is defined by the M, Q and E flags in the OSD region specification header. Q defines the pixel resolution, allowing half resolution modes to save memory while retaining the color resolution. The meaning of each combination of these flags is given in [Table 90](#).

M	Q	E	Bits per pixel	No. of colors	Resolution
0	0	0	2	4	1 pixel
1	1	0	2	4	2 pixels
1	0	0	4	16	1 pixel
0	1	0	4	16	2 pixels
0	0	1	8	256	1 pixel
1	1	1	8	256	2 pixels
1	0	1	Reserved		
0	1	1	Reserved		

Table 90 M, Q and E palette mode header flags

To reduce the size of the bit maps while retaining the color resolution, a half resolution mode is provided, as shown in [Table 90](#). In half resolution, each pixel in the bit-map defines the color of two adjacent pixels in the same line in the display.

Color modes

The pixel color mode defines the format of the output from the palette. Three pixel color modes are supported, as listed in [Table 91](#). This table shows how many bits are used for each color element and how many bits for the mixing factor MixWeight, which determines the effect of overlaying the picture. Anti-aliasing is supported only with 24-bit color.

24-bit or 14-bit color for the region field is selected by the bit Tc in the header of the specification block. Anti-aliasing is selected by the bit AA in the header.

Mode	Color resolution	Mixing	Reference
24-bit color with anti-aliasing	Y[7:0], Cb[7:0], Cr[7:0]	MixWeight[5:0] defined for each color.	Table 92
24-bit color	Y[7:0], Cb[7:0], Cr[7:0]	MixWeight[3:0] defined for the region.	Table 93
14-bit color	Y[5:0], Cb[3:0], Cr[3:0]	MixWeight[3:0] defined for the region.	Table 94

Table 91 OSD color modes

The format of each line of the palette depends on the color mode. [Table 92](#) to [Table 94](#) show the format of the palette lines for each color mode.

Field	Bits	Description
Cr[7:0]	7:0	Cr chroma value
Cb[7:0]	15:8	Cb chroma value

Table 92 Palette line format in 24-bit color with anti-aliasing

Field	Bits	Description
Y[7:0]	23:16	Y luma value
W[5:0]	29:24	Mix weight. See Section 21.5.10: Mixing OSD with video on page 180.
Reserved	31:30	Reserved. Write 0.

Table 92 Palette line format in 24-bit color with anti-aliasing

Field	Bits	Description
Cr[7:0]	7:0	Cr chroma value
Cb[7:0]	15:8	Cb chroma value
Y[7:0]	23:16	Y luma value
T	24	Transparency 0: Do NOT blend video with OSD for this color. 1: Blend video with OSD for this color using the mix weight.
Reserved	31:25	Reserved. Write 0

Table 93 Palette line format in 24-bit color without anti-aliasing

Field	Bits	Description
Cr[3:0]	3:0	Cr chroma value
Cb[3:0]	7:4	Cb chroma value
T	8	Transparency: 0: Do NOT blend video with OSD for this color. 1: Blend video with OSD for this color using the mix weight.
Reserved	9	Reserved. Write 0.
Y[5:0]	15:10	Y luma value

Table 94 Palette line format in 14-bit color mode

Standard colors

Table 95 shows the 14-bit Y, C_R and C_B values nearest to the standard color-bar colors.

Standard color	Y	C _R	C _B
White	0x3B	0x8	0x8
Black	0x04	0x8	0x8
Red	0x10	0xD	0x6
Green	0x1C	0x4	0x4
Blue	0x9	0x7	0xD
Yellow	0x28	0x9	0x3
Cyan	0x22	0x3	0xA
Magenta	0x15	0xC	0xD

Table 95 Standard colors in 14-bit color

Table 96 shows the 24-bit Y, C_R and C_B values nearest to the standard color bar colors.

Standard color	Y	C _R	C _B
White	0xEC	0x80	0x80
Black	0x10	0x80	0x80

Table 96 Standard colors in 24-bit color

Standard color	Y	C _R	C _B
Red	0x40	0xD4	0x64
Green	0x70	0x40	0x48
Blue	0x24	0x74	0xD4
Yellow	0xA0	0x8C	0x2C
Cyan	0x88	0x2C	0x9C
Magenta	0x54	0xC8	0xB8

Table 96 Standard colors in 24-bit color

21.5.7 OSD bit-map

The bit-map for an OSD region follows the palette if defined, or the header if no palette is defined. The bit-map defines the OSD pixels, in left to right order within lines, and follows the lines in top to bottom order. The number of bits per pixel is 2, 4 or 8 depending on the palette mode. The value for each pixel gives the line of the palette, which defines the color for the pixel. As all the different sources are mixed in 4:4:4 format, there is no risk of miscolored boundary between OSD, video, still picture and sub picture. An homogeneous decimation is applied to avoid the problem on the 4:2:2 format.

21.5.8 OSD block header format

Table 97 shows the layout of the header, which occupies one 64-bit word. Table 98 shows the layout in graphical form, with each line representing a quarter of a 64-bit word.

Field	Size	Bits	Meaning	Reference
M	1	63	Palette mode.	Table 90 .
Q	1	62		
E	1	61		
Tc	1	60	1: Select 24-bit color mode, 0: Select 14-bit color mode	Table 91
P	1	59	0: A new palette follows the header. 1: The palette is the same as the previous region.	
AA	1	58	Select anti-aliasing. Anti-aliasing can only be used with 24-bit color.	Section 21.5.6
S	1	57	OSD region not included in CVBS output in dual output mode.	Below
Y_top	9	56:48	Position of the top of the OSD region.	Section 21.5.5
MixWeight	4	47:44	Mixing weight $\alpha 4$ with planes behind when anti-aliasing is disabled. When anti-aliasing is enabled, write 0.	Section 21.5.10
OSDp[6:4]	3	43:41	Pointer to the next region specification.	Below
Y_bottom	9	40:32	Position of the bottom of the OSD region.	Section 21.5.5
OSDp[12:7]	6	31:26	Pointer to the next region specification.	Below
X_left	10	25:16	Position of the left of the OSD region.	Section 21.5.5
OSDp[18:13]	6	15:10	Pointer to the next region specification.	Below
X_right	10	9:0	Position of the right of the OSD region.	Section 21.5.5

Table 97 OSD block header format

M	Q	E	Tc	P	AA	S	Y_top
MixWeight				OSDp[6:4]			Y_bottom
OSDp[12:7]						X_left	
OSDp[18:13]						X_right	

Table 98 OSD region specification header

The header contains the OSDp[18:0] pointer. This pointer defines the address of the next block in the linked list to load from memory, as described in [OSD specification on page 171](#). If OSDp[18:0]=0, then the next OSD block is read contiguously from memory, after the current OSD block. This is STi3500 compatibility mode.

An OSD region can be placed anywhere in the whole 64Mbit SDRAM area by setting register bit **OSD_CFG.LLG**:

- When OSD_CFG.LLG=0, the OSD linked-list pointer can point to *any* word in the lower 32Mbits of the 64Mbit SDRAM area;
- When OSD_CFG.LLG=1, the OSD linked-list pointer can point to any alternate word in *all* of the 64Mbit SDRAM area. This effectively doubles the available area by halving the placement precision.

The block must be 16-word aligned, so the OSDp[18:0] pointer must be a multiple of 16. Therefore, the least significant 4 bits of OSDp are always zero, and are not included in the header.

The OSD specification block location is defined by the OSD_BOT or OSD_TOP registers, in units of 256 bytes. This means that the full address of the first block must be a multiple of 64.

Bit S in [Table 98](#) controls the presence of the OSD on a region basis for the 4:2:2 path to the Digital Encoder, from which YC and CVBS signals are generated. If this bit is 1, the OSD region will not be present; if it is set to 0, the OSD region will be present provided the OSD is included in the 4:2:2 output, as defined by VID_OUT. This is to allow selective recording of OSD regions. This bit does not affect the 4:4:4 path to the Digital Encoder from which the RGB and YCbCr signals are generated.

21.5.9 OSD specification block examples

This section shows the format for some complete specification blocks.

[Table 99](#) shows a specification using 2 bits per pixel in the bit-map with 1 pixel resolution and 14-bit color. Only the first 8 pixels of the bit-map are shown. The palette occupies one 64-bit word, and the bit-map occupies one 64-bit word for every 32 pixels.

Bits within a 16-bit quarter-word																Description
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
M=0	Q=0	E=0	Tc=0	P=0	A=0	S	Y_top									Word 0
MixWeight				OSDp[6:4]			Y_bottom									
OSDp[12:7]						X_left										
OSDp[18:13]						X_right										
Palette0 Y						0	T0	Palette0 Cb				Palette0 Cr				
Palette1 Y						0	T1	Palette1 Cb				Palette1 Cr				Word 1
Palette2 Y						0	T2	Palette2 Cb				Palette2 Cr				
Palette3 Y						0	T3	Palette3 Cb				Palette3 Cr				
Bit-map for 8 OSD pixels																

Table 99 2 bits per pixel, 14-bit color OSD region specification

[Table 100](#) shows a specification using 4 bits per pixel in the bit-map with 2 pixel resolution and 14-bit color. Only the first 4 pixels of the bit-map are shown. Each entry in the bit-map uses 4 bits, but defines two display pixels of the same color.

The palette occupies four 64-bit words, and the bit-map occupies one 64-bit word for every 16 bit-map pixels.

Bits within a 16-bit quarter-word															Description	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
M=0	Q=1	E=0	Tc=0	P=0	AA=0	S	Y_top								Word 0	
MixWeight				OSDp[6:4]			Y_bottom									
OSDp[12:7]						X_left										
OSDp[18:13]						X_right										
Palette0 Y						0	T0	Palette0 Cb				Palette0 Cr				Word 1
Palette1 Y						0	T1	Palette1 Cb				Palette1 Cr				
Palette2 Y						0	T2	Palette2 Cb				Palette2 Cr				
Palette3 Y						0	T3	Palette3 Cb				Palette3 Cr				
Palette4 Y						0	T4	Palette4 Cb				Palette4 Cr				Word 3
Palette5 Y						0	T5	Palette5 Cb				Palette5 Cr				
Palette6 Y						0	T6	Palette6 Cb				Palette6 Cr				
Palette7 Y						0	T7	Palette7 Cb				Palette7 Cr				
Palette8 Y						0	T8	Palette8 Cb				Palette8 Cr				Word 4
Palette9 Y						0	T9	Palette9 Cb				Palette9 Cr				
Palette10 Y						0	T10	Palette10 Cb				Palette10 Cr				
Palette11 Y						0	T11	Palette11 Cb				Palette11 Cr				
Palette12 Y						0	T12	Palette12 Cb				Palette12 Cr				Word 5
Palette13 Y						0	T13	Palette13 Cb				Palette13 Cr				
Palette14 Y						0	T14	Palette14 Cb				Palette14 Cr				
Palette15 Y						0	T15	Palette15 Cb				Palette15 Cr				
Bit-map for 4 OSD pixels															Bit-map word	

Table 100 4 bits per pixel, 2-pixel resolution 14-bit color OSD region specification

Table 101 shows a specification using 8 bits per pixel in the bit-map with full resolution and 14-bit color. Only the first 2 pixels of the bit-map are shown. Each pixel in the bit-map uses 8 bits.

The palette occupies 64 64-bit words (i.e. 512 bytes), and the bit-map occupies one 64-bit word for every 8 bit-map pixels.

Bits within a 16-bit quarter-word																Description	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
M=0	Q=0	E=1	Tc=0	P=0	AA=0	S	Y_top									Word 0	
MixWeight				OSDp[6:4]			Y_bottom										
OSDp[12:7]						X_left											
OSDp[18:13]						X_right											
Palette0 Y						0	T0	Palette0 Cb				Palette0 Cr				Word 1	
Palette1 Y						0	T1	Palette1 Cb				Palette1 Cr					
Palette2 Y						0	T2	Palette2 Cb				Palette2 Cr					
Palette3 Y						0	T3	Palette3 Cb				Palette3 Cr					
Palette4 Y						0	T4	Palette4 Cb				Palette4 Cr				Word 2	
...								...				...				...	
Palette252 Y						0	T12	Palette252 Cb				Palette252 Cr				Word 64	
Palette253 Y						0	T13	Palette253 Cb				Palette253 Cr					
Palette254 Y						0	T14	Palette254 Cb				Palette254 Cr					
Palette255 Y						0	T15	Palette255 Cb				Palette255 Cr					
Bit-map for 2 OSD pixels with 8 bits per pixel or 8 bits per 2 pixel																Bit-map Word	

Table 101 8 bits per pixel, 14-bit color OSD region specification

Table 102 shows a specification using 2 bits per pixel in the bit-map with full resolution and 24-bit color, without anti-aliasing. Only the first 8 pixels of the bit-map are shown. Each entry in the bit-map uses 2 bits. The palette occupies two 64-bit words, and the bit-map occupies one 64-bit word for every 32 bit-map pixels.

The palette for 4-bit and 8-bit colors would be similar, but with 16 or 256 color lines in the palette instead of 4.

Bits within a 16-bit quarter-word																Description
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
M=0	Q=0	E=0	Tc=1	P=0	AA=0	S	Y_top									Word 0
MixWeight				OSDp[6:4]			Y_bottom									
OSDp[12:7]						X_left										
OSDp[18:13]						X_right										
0 (reserved)							T0		Palette0 Y							
Palette0 Cb								Palette0 Cr								Word 1
0 (reserved)							T1		Palette1 Y							
Palette1 Cb								Palette1 Cr								
0 (reserved)							T2		Palette2 Y							
Palette2 Cb								Palette2 Cr								Word 2
0 (reserved)							T3		Palette3 Y							
Palette3 Cb								Palette3 Cr								
Bit-map for 8 OSD pixels																

Table 102 2 bits per pixel 24-bit color without anti-aliasing OSD region specification

Table 103 shows a specification using 2 bits per pixel in the bit-map with full resolution and 24-bit color with anti-aliasing. Only the first 8 pixels of the bit-map are shown. Each entry in the bit-map uses 2 bits. The palette and bit-map occupy the same memory as without anti-aliasing. The palette for 4-bit and 8-bit colors would be similar, but with 16 or 256 color lines in the palette instead of 4.

Bits within a 16-bit quarter-word																Description
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
M=0	Q=0	E=0	Tc=1	P=0	AA=1	S	Y_top									Word 0
0 (reserved)				OSDp[6:4]			Y_bottom									
OSDp[12:7]						X_left										
OSDp[18:13]						X_right										
0		MixWeight0						Palette0 Y								Word 1
Palette0 Cb								Palette0 Cr								
0		MixWeight1						Palette1 Y								
Palette1 Cb								Palette1 Cr								
0		MixWeight2						Palette2 Y								Word 2
Palette2 Cb								Palette2 Cr								
0		MixWeight3						Palette3 Y								
Palette3 Cb								Palette3 Cr								
Bit-map for 8 OSD pixels																Bit-map word

Table 103 2 bits per pixel 24-bit color with anti-aliasing OSD region specification

21.5.10 Mixing OSD with video

The mixing function allows each OSD pixel to be blended with the corresponding pixel generated by the planes behind the OSD. This is shown as α_4 in Figure 108 and Figure 109 on page 184. The mix weight is a programmable parameter and can be set for each OSD region, or for each color when anti-aliasing is enabled.

When anti-aliasing is disabled, the mix weight is set for each region in the region header. The mix weight is set by the equations below :

$$\chi = \left(1 - \frac{n}{16}\right)\chi_1 + \frac{n}{16}\chi_2 \quad \text{for } 0 \leq n < 15$$

and

$$\chi = \chi_2 \quad \text{for } n=15$$

Where χ is the resulting pixel, χ_2 is the OSD pixel and χ_1 is the video pixel.

The resulting OSD pixel can be completely transparent ($n=0$) or can completely cover the video ($n=15$). Each individual color in the palette can be specified to be used with or without mixing by setting the transparency (T) bit of the palette. A T bit equal to 0 means no mixing for the particular color, and a T of 1 means that mixing should be used.

When anti-aliasing is enabled, a mix weight is defined for each color in each region in the LUT. The mix weight is a 6-bit number allowing mixing ratios from 0 to 1 with a resolution of 1/63. The resulting pixel can be completely transparent (weighting of 0/63) or can completely cover the video (weighting of 63/63).

Palette color zero can also be set to be transparent by setting the Y, Cb and Cr values to zero. Only palette color zero can be used in this way.

21.5.11 Anti-flicker and anti-flutter filters

Flicker and flutter effects are visual problems due to interlaced television displays. Flutter effects can occur if the same OSD image is displayed in both fields within one frame. Anti-flicker and anti-flutter filtering are provided as part of the display unit.

At every VSYNC pulse the values in the register **OSD_CFG** will be taken in account. Normal mode means no filter modes are active. If **OSD_CFG.NOR** is 0 no filter modes will be active; if **OSD_CFG.NOR** is 1 the filter mode given by **OSD_CFG.FIL** will be active. Anti-flicker filtering is applied if bit **OSD_CFG.FIL** is 0; if the same bit is 1 the anti-flutter filter will be active.

For flicker or flutter filtering the OSD boundary weight can be specified in the register **OSD_BDW**. This weight will be taken into account at OSD top and bottom borders in filter modes.

The anti-flicker filter can be described by the following expressions, where T is a top-field pixel, B is a bottom field pixel and n is the line number:

$$T = \frac{(B[n-1] + 2T[n] + B[n])}{4} \quad B = \frac{(T[n] + 2B[n] + T[n+1])}{4}$$

The anti-flutter filter can be described by the following expression, where T is a top-field pixel, B is a bottom field pixel and n is the line number:

$$T = T[n] \quad B = \frac{(T[n] + T[n+1])}{2}$$

21.5.12 OSD active signal

The OSD active signal can be used in two modes. The mode is controlled using **OSD_ACT.OAM**. In the first mode the OSD active signal is configured as an output. In this mode the OSD active signal denotes when an active OSD pixel (non transparent) is on the YC output bus, as in [Figure 106](#) below. The signal, in this mode, has a programmable delay controlled by **OSD_ACT.OAD**.

This delay can be set such that the OSD active signal is set as much as two clocks before or 1 to 64 clocks after the actual pixel.

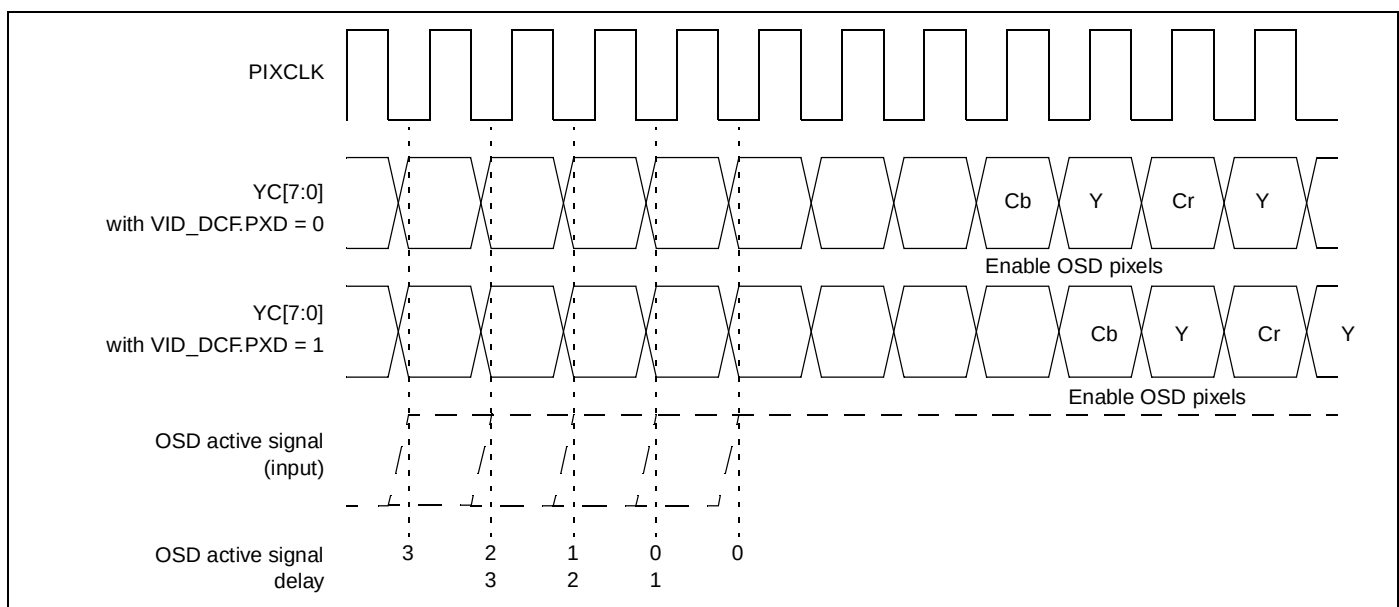


Figure 106 OSD active timing when **OSD_ACT.OAM** = 1

In the second mode of operation, the OSD active signal is configured as an input and is used to disable the OSD. When the signal goes high, the OSD will be placed on the YC bus if the OSD is enabled. When this signal is low then no OSD will be placed on the bus even if OSD is enabled. The programmable delay is used in the same way as for the input signal.

OSD active mode	OSD active signal	Meaning
0	0	Signal is an output. Video Pixels only on display bus.
0	1	Signal is an output. OSD Pixels on the display bus.
1	0	Signal is an input. Disable the OSD output.
1	1	Signal is an input. Enable OSD output if available.

Table 104 OSD active signal operation

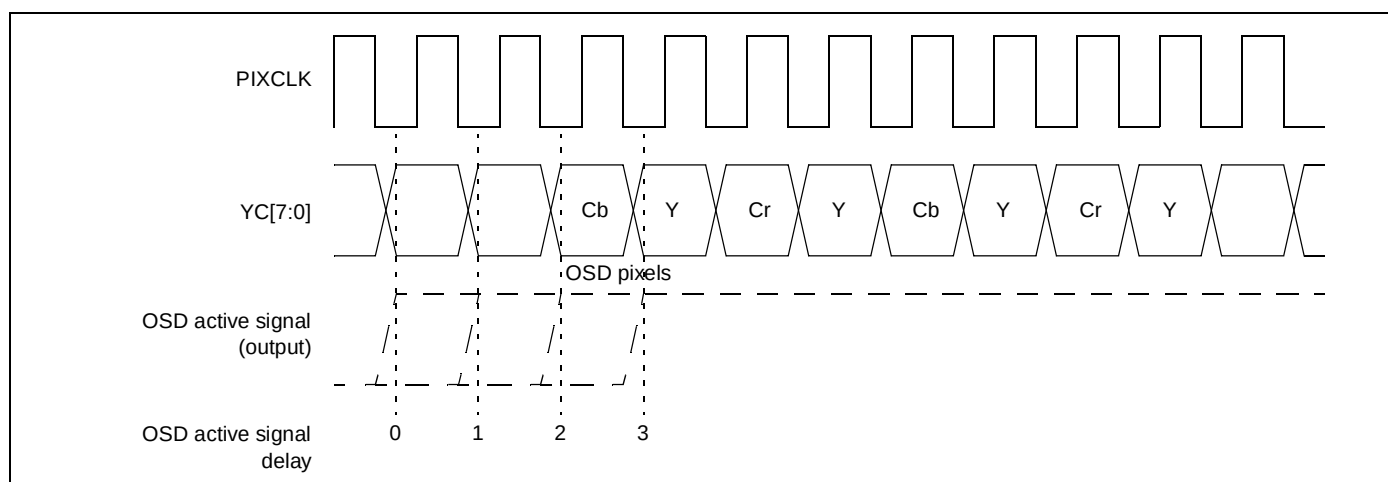


Figure 107 OSD active timing when OSD_ACT.OAM = 0

21.6 Sub-picture or cursor plane

The sub-picture or cursor plane displays the output from the sub-picture decoder, described in [Sub-picture decoder on page 144](#). The sub-picture can be either in front of or behind the OSD, depending on whether the bit **VID_OUT.SPO** is 1 or 0 respectively.

The sub-picture decoder can also be used as a hardware cursor unit. The sub-picture should be configured to be in front of the OSD. A cursor can be defined using an optionally compressed (run-length encoded) bitmap stored in external SDRAM. The bitmap can be any size up to a full screen. Per-pixel alpha-blending factors can be defined for each cursor to provide anti-aliasing with the background. The cursor is then moved around using register writes into X and Y coordinate registers.

21.7 Mixing display planes

The blending of the elements of the final picture is performed by a mixing unit, which is shown in [Figure 108](#) and [Figure 109 on page 184](#). The mixing of the display planes is controlled by five mix weights, α_1 to α_5 . The mix weights α_1 to α_3 values control the mixing of the back three planes, the background color, the still picture plane and the video plane.

These are 8-bit values held in registers, as shown in Table 105. The three back planes are generated in 4:2:2 format and are mixed in that format and then converted to 4:4:4 for mixing with the sub-picture and OSD.

Factor	Register	Mixed planes	Plane displayed when mix weight is 0	Plane displayed when mix weight is 0xFF
$\alpha 1$	VID_MWS	The background color and the still picture plane.	Still picture plane	Background color
$\alpha 2$	VID_MWV	The background color and MPEG video.	MPEG video	Background color
$\alpha 3$	VID_MWSV	The video and the still picture plane.	MPEG video	Still picture plane

Table 105 Control of mixing factors $\alpha 1$ to $\alpha 3$

The resultant of mixing the back three planes can be blended in turn with the sub-picture and OSD. The mixing formula is: $display=(1-\alpha) \times source1 + \alpha \times source2$. The order of this mixing depends on whether the sub-picture is in front of or behind the OSD. The OSD mix weight is $\alpha 4$ and is defined in the OSD palette. If anti-aliasing is disabled, then the mix weight is a 4-bit value, defined for each OSD region, and mixing can be enabled or disabled for each color. If anti-aliasing is enabled, then the mix weight is a 6-bit value defined for each color. Blending the OSD is described in Section 21.5.10: *Mixing OSD with video* on page 180.

The sub-picture mix weight is $\alpha 5$, which is a 4-bit value defined per pixel type. These values can be controlled by the sub-picture bit stream except in the highlight area which is controlled by SPD_HCN.

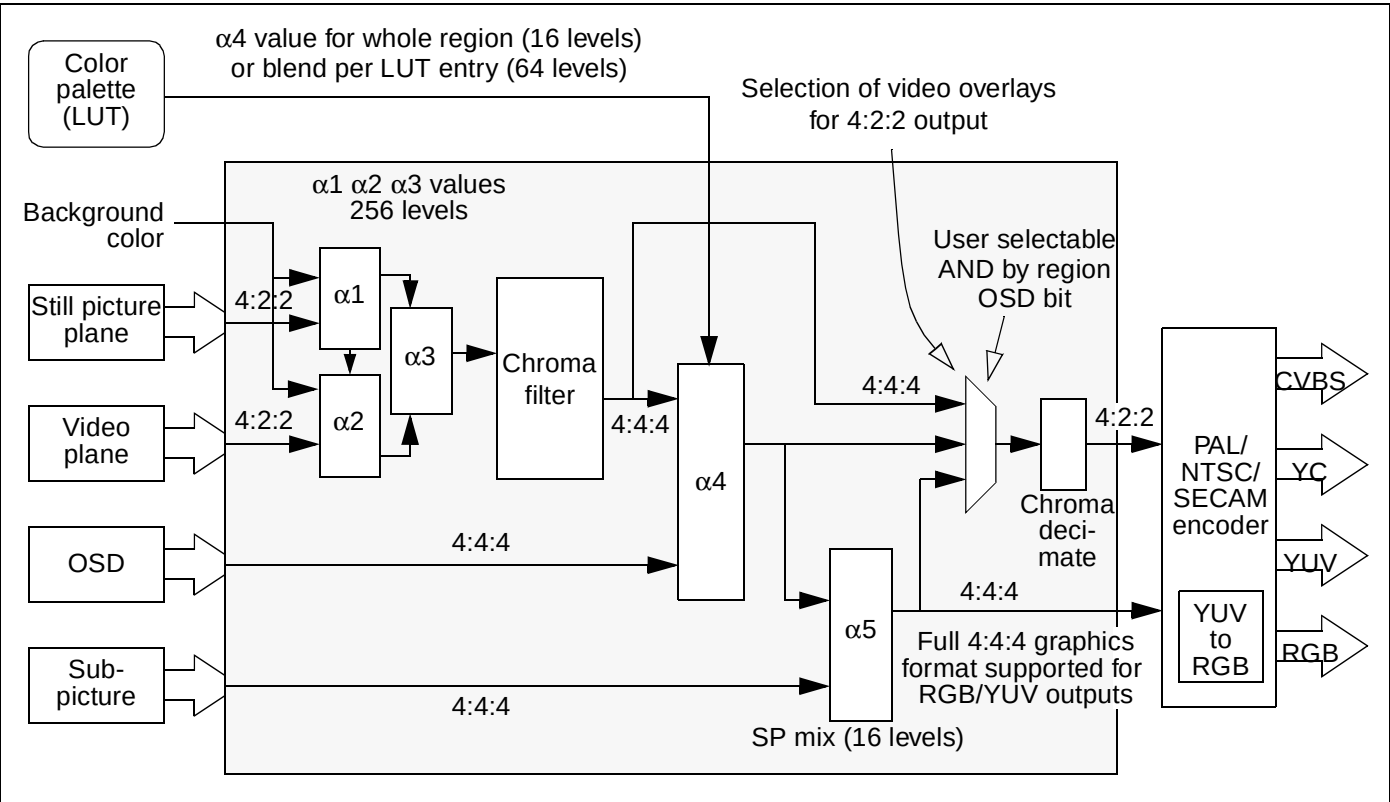


Figure 108 Mixing with the sub-picture in front

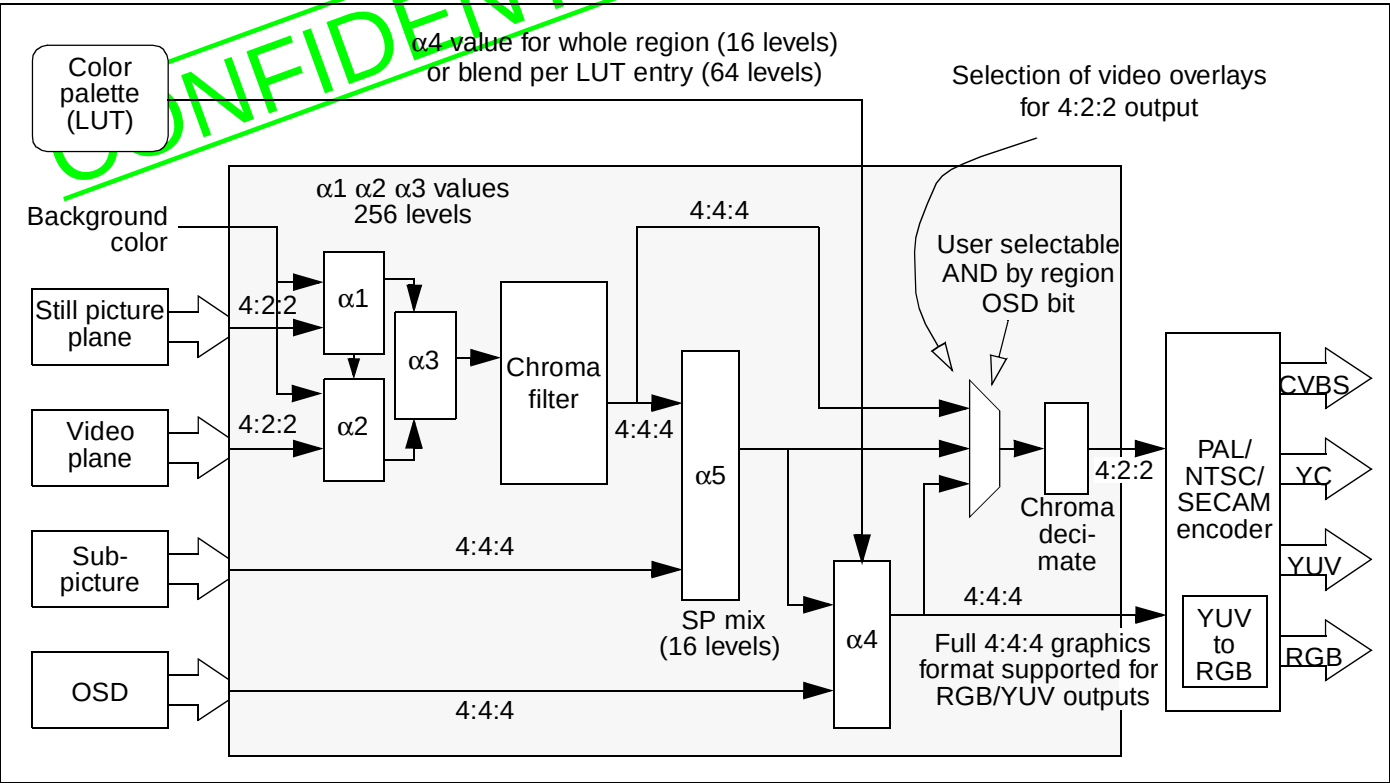


Figure 109 Mixing with the OSD in front

21.7.1 4:2:2 Output control

The 4:2:2 output is used by the DENC to generate CVBS and YC output, and is generally used for recording by VCR. The OSD and sub-picture can be omitted from this output, as controlled by the register VID_OUT and by the S bits in the OSD region headers. The combined actions of these fields depend on whether the OSD is behind or in front of the sub-picture. The 4:4:4 output is unaffected.

VID_OUT.LAY defines the number of planes in front of the video which are initially included. VID_OUT.NOS turns on or off the OSD, as shown in Table 106. If VID_OUT.NOS=0 (OSD present) then the S bit in an OSD region header can alternatively be used to turn off that OSD region. When the OSD and Sub-picture planes are present, VID_OUT.SPO defines which plane is in front. The combination of bit functions is given in the table below.

LAY	NOS	Sub-picture in front (SP0=1)	OSD in front (SP0=0)
00-01	Any	Video only	Video only
10	0	Video + OSD	Video + sub-picture
	1	Video only	Video + sub-picture
11	0	Video + OSD + sub-picture	Video + sub-picture + OSD
	1	Video + sub-picture	Video + sub-picture

Table 106 Encoding of LAY and NOS fields of VID_OUT



22 2D block move

This module copies rectangular blocks of data from one area in display memory to another. The width and height of the destination block must be the same as the width and height of the source block. If the source area overlaps the destination area, then the result depends on the copy direction. To copy correctly, the copy direction must first be setup by the user (See [Figure 111: How to handle overlaps on page 186](#)). The module cannot access devices other than the MPEG/Graphics SDRAM.

The copying is done by DMA engines, so the CPU can perform other tasks during the copying. The interface between the CPU and the 2D block move module is provided using a set of registers and an interrupt to signal when a copy has completed.

22.1 Copying blocks of data

To perform a 2D block move, the module must first be initialized with the source and destination addresses and the dimensions of the block to be copied. The terminology used for 2D block moves is shown below.

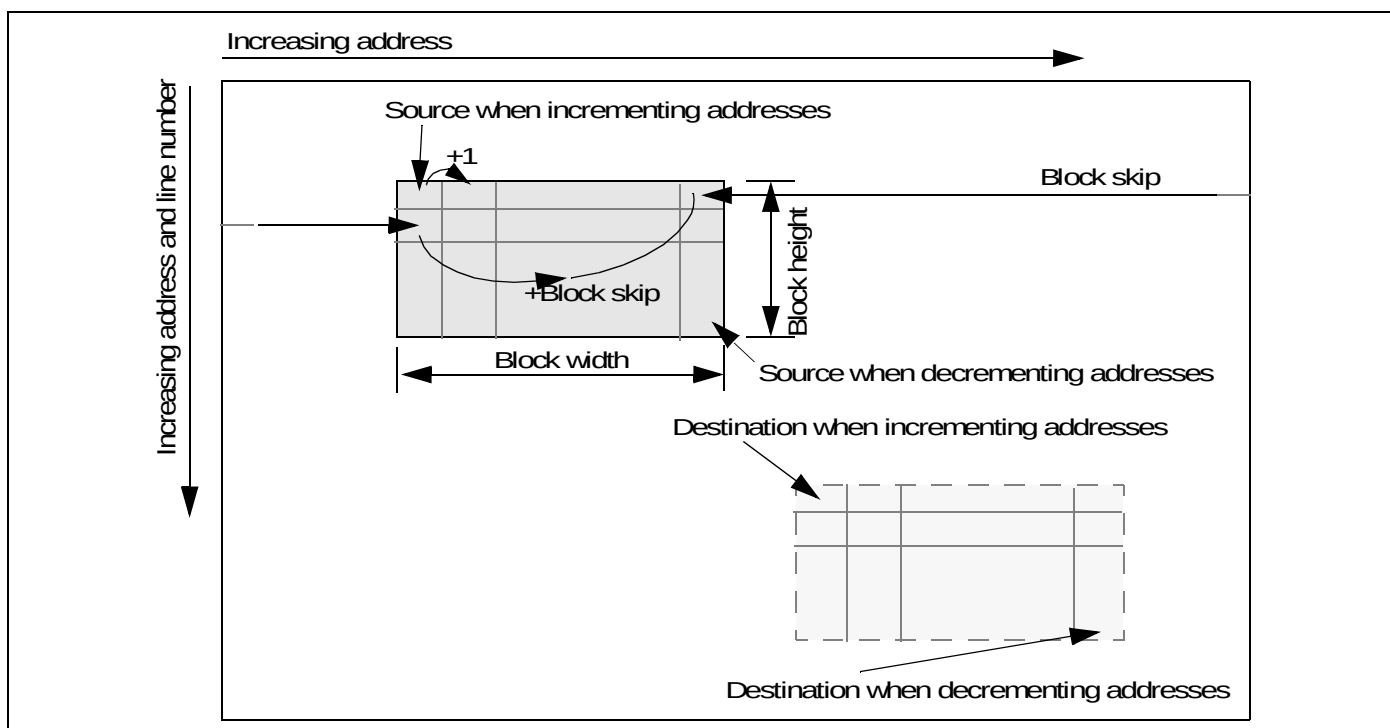


Figure 110 2D block move

The source and destination addresses are the start of the source and destination areas and must be 64-bit aligned. Normally the addresses are incremented at each step, so the top left hand corner of the source and destination blocks should be given. If the addresses are set to decrement by writing 1 to USD_BMC.D_A then the bottom right hand corners must be given.

The source and destination addresses are indicated by the initial value of the registers USD_BRP and USD_BWP. These registers give the offsets in 64-bit words from the base of the SDRAM, *SDRAM_base*. Thus if the full source and destination addresses are *source_address* and *dest_address* then the following values should be written in the following registers:

$$\text{USD_BRP} = (\text{source_address} - \text{SDRAM_base}) / 8$$

$$\text{USD_BWP} = (\text{dest_address} - \text{SDRAM_base}) / 8$$

The block width is held in USD_BMW. It is given in 64-bit words, and can be any value in the range of 1 to 65535. The block height is held in USD_BMH and is given in lines. It can be any value in the range of 1 to 1023 lines.

The block skip is defined as the number of 64-bit words from the end of one line of the block to the beginning of the next, i.e. the width of the displayed picture on the screen, less than the width of the block to be copied, plus or minus one pixel, depending on bit D_A as shown in Figure 111. The block skip is held in register USD_BSK.

To start the block move, a 1 is written to USD_BMC.EXE. The 2D block move engine can be set up to start on various types of events, depending on the value of fields in the control register USD_BMC.

- the start of display of the odd field,
- the start of display of the even field,
- after the display of a given line number.

Setting USD_BMC.CPA causes the pattern in register USD_PAT to be copied to the whole of the destination rectangular block as a fill function.

At the end of the block move operation an interrupt is generated.

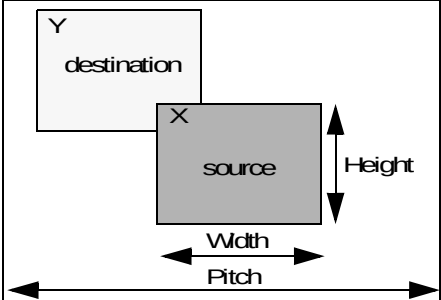
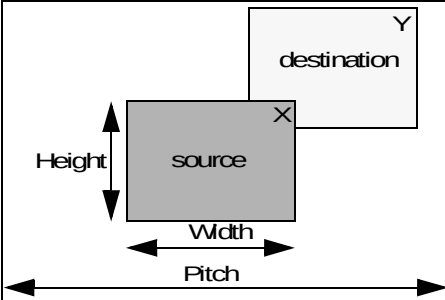
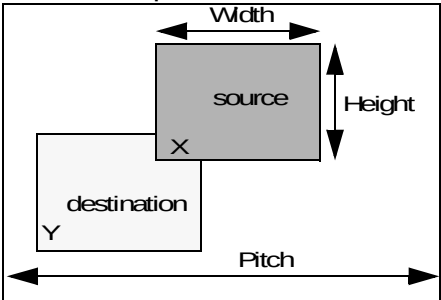
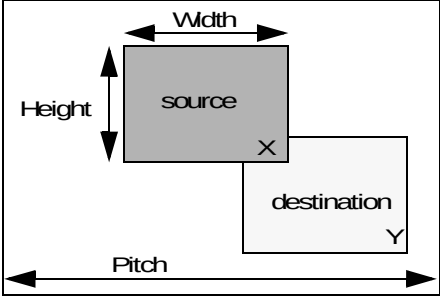
	Decrement address (USD_BMC.D_A == 1)	Increment address (USD_BMC.D_A == 0)
block skip > 0	block skip = Pitch - Width + 1  Read left-to-right and top-to-bottom from point X, line by line. Copy to the destination in the same way, starting at point Y	block skip = Pitch + Width - 1  Read right-to-left and top-to-bottom from point X, line by line. Copy to the destination in the same way, starting at point Y
block skip < 0	block skip = - Pitch - Width + 1  Read left-to-right and bottom-to-top from point X, line by line. Copy to the destination in the same way, starting at point Y	block skip = - Pitch + Width - 1  Read from right-to-left and bottom-to-top, from point X, line by line. Copy to the destination in the same way, starting at point Y

Figure 111 How to handle overlaps



23 Teletext interface

The STi5512 has a teletext interface (**TtxtInt**) which interfaces to a teletext peripheral. It translates teletext data to or from memory. It has two modes of operation, determined by the setting of the **TtxtMode** register:

- Teletext data output
- Teletext data input

In teletext data output mode, the teletext interface uses DMA to retrieve teletext data from memory, and serializes the data for transmission to a composite video encoder.

In teletext data input mode, teletext data can be extracted from a composite video signal and fed into the teletext interface as a serial stream. The teletext interface assembles the data and uses DMA to pass this data to memory.

The interface between the CPU and the teletext interface is based on an interrupt mechanism.

23.1 Teletext interface pins

For teletext output mode, these signals are internal and generated by the on-chip PAL/NTSC/SECAM encoder. For teletext input mode, they are derived from input pins.

Pin	In/Out	Function
TtxtData	in/out	Teletext serial data.
TtxtEvennotOdd	in	Teletext even not odd.
TtxtRequest/TtxtHsync	in	Teletext serial data request input. TtxtRequest becomes the TtxtHsync (<i>Hsync</i>) signal when the teletext interface is operating in the input mode.
TtxtClockIn	in	27 MHz teletext clock.

Table 107 Teletext interface pins

23.2 Teletext data output

In this mode, the teletext interface uses DMA to retrieve teletext data from memory, and serializes the data for transmission to a composite video encoder. Clock run-in bits are added to the start of the serial stream, as defined in the EBU specification¹.

The CPU is responsible for assuring the correct programming of the video encoder. The encoder must be programmed such that it makes requests for teletext data only on pre-specified lines.

The **TtxtEvennotOdd** input from the encoder is used to interrupt the CPU, allowing software control of the teletext output DMA initialization.

The CPU initiates the output of a number of lines of teletext data. These lines are output when suitable requests are made from the video encoder.

1. Document SPB492, 'Teletext Specification'. European Broadcasting Union, Geneva, December 1992.

23.2.1 Output line format

One teletext line is output as a stream of 360 bits, at an average frequency of 6.9375 MHz. The format of the data in memory is assumed to be the same as the contents of the PES data packet as defined in the ETSI specification. The DMA is designed to read multiples of 46 bytes and output from this lines of 45 bytes. Each output line is composed of two bytes (16 bits) of clock run-in, followed by the data extracted from the transport packet. The data field consists of

- The framing code
- The magazine and packet address
- Data block fields

These three fields provide the block of teletext data. The clock run-in is composed of two bytes, each with the hexadecimal value #AA, which has the bit pattern '10101010'. The framing code, which is extracted from the data field, should be a single byte of #E4¹. One line of teletext output will be composed as in the figure below. The data will be transmitted from least significant bit (LSB) to most significant bit (MSB) of each byte in memory.

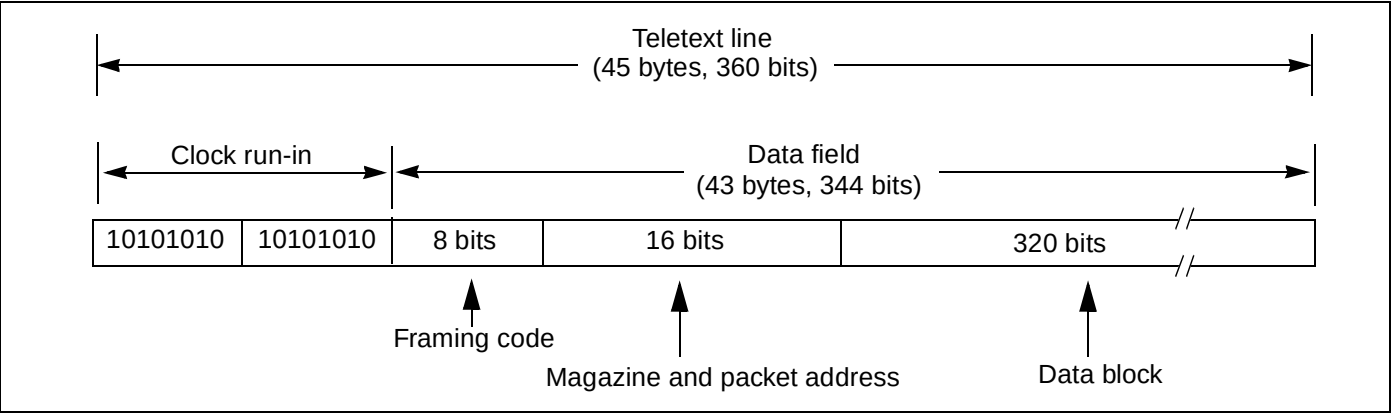


Figure 112 Line output

The 360 bits of output data are defined to be nine 37-bit sequences, ending with one 27-bit sequence. Within each sequence, each bit is transmitted in four 27 MHz cycles, except bits 10, 19, 28 and 37, which are transmitted in three 27 MHz cycles, as shown in the figure below.

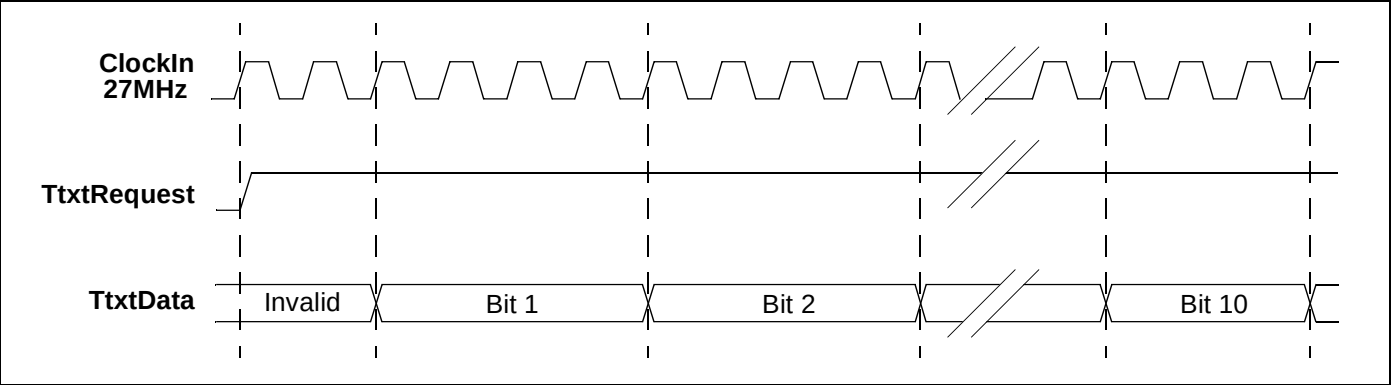


Figure 113 Output data

1. Specification for conveying ITU-R Systems B Teletext in Digital Video Broadcasting (DVB) bit-streams.



23.3 Teletext data input

Teletext data is extracted from the composite video signal. This data is fed into the teletext interface as a serial stream. The teletext interface assembles the data and uses DMA to pass this data to memory.

Horizontal and vertical sync information is extracted from a composite video signal. This defines the field and line positions.

An event on **TtxtEvennotOdd** causes the line counter to reset. Every successive hsync pulse increments this counter. When the current line is equal to that specified in the **TtxtInStartLine** register, the line is input as teletext data. In order to ignore color-burst data etc., both the **TtxtData** input and the teletext clock in signals will be gated off for a number of 27 MHz clock cycles after hsync, where the number of cycles is specified in the **TtxtInCbDelay** register. After the color burst blanking, the data on **TtxtData** will be shifted in on the rising edge of the teletext clock input. A valid teletext line will be determined on the first occurrence of the framing code contained within the shift register. Only at this point will the line be considered valid for writing to memory.

23.4 Teletext interrupt control

The teletext interrupts can be programmed, using the **TtxtIntEnable** register, to interrupt the CPU whenever one of the following occurs:

- A teletext data input or output data transfer completes,
- The current video frame toggles odd to even or even to odd.

The interrupt status contained within the **TtxtIntStatus** register is masked with the **TtxtIntEnable** register. The interrupt bits are reset when the CPU writes to the specific acknowledgment register, or when a DMA operation completes.

24 Digital encoder

24.1 Introduction

This The final stage of the video pipeline of the device is a high performance PAL/SECAM/NTSC digital encoder referred to as the DENC. The DENC converts the 4:2:2 digital video stream into a standard analog baseband PAL/SECAM/NTSC signal and into RGB and UV analog components. It can also convert a 4:4:4 digital video stream into RGB and UV analog components.

The DENC can handle interlaced mode (in all standards) and non-interlaced mode in PAL and NTSC. It can perform Closed-Captions, CGMS, WSS, Teletext and VPS encoding and allows Macrovision™ 7.01/ 6.1 copy protection. Six analog output pins are available, on which it is possible to output either (S-VHS(Y/C) + CVBS + RGB) or (S-VHS(Y/C)+ CVBS + V + Y + U) or (Y1 + C1 + CVBS1 + C2 + Y2 + CVBS2).

Six analog output pins are available on which it is possible to output CVBS, S-VHS (Y/C), RGB and YUV formats.

The encoder can operate in master mode or in one of several slave modes, where it locks onto incoming sync signals. An autotest mode is also provided.

The main functions are controlled using an 8-bit register interface with the CPU. The registers are listed at the end of this chapter and fully described in the Register Manual.

24.2 Video timing

The burst sequences are internally generated, subcarrier generation being performed numerically with CKREF as reference. 4-frame bursts are generated for PAL or 2-frame bursts for NTSC. Rise and fall times of synchronization tips and burst envelope are internally controlled according to the relevant ITU-R and SMPTE recommendations. The 6-frame sub-carrier phase sequence is generated in SECAM (see [Subcarrier insertion \(SECAM\) on page 204](#)).

[Figure 116](#) , [Figure 117](#) , [Figure 118](#), [Figure 119](#) and [Figure 120](#) depict typical VBI waveforms.

It is possible to allow encoding of incoming YCrCb data on those lines of the VBI that do not bear line sync pulses or pre/post-equalization pulses (see Figures 116, 117, 118, 119 and 120). This mode of operation is referred to as *partial blanking* and is the default set-up. It allows the encoded waveform to keep any VBI data present in digitized form in the incoming YCrCb stream (e.g. supplementary Closed-Caption lines or StarSight data, etc.). Alternatively, the complete VBI may be *fully blanked*, so no incoming YCrCb data is encoded on these lines. Full or partial blanking is set by register bit DEN_CFG1.blkli.

For 525/60 systems, with the SMPTE line numbering convention:

- complete VBI consists of lines 1 to 19 and the second half of lines 263 to 282;
- partial VBI consists of lines 1 to 9 and the second half of lines 263 to 272;
- line 282 is either fully blanked or fully active.

For 625/50 systems, with the CCIR line numbering convention:

- complete VBI consists of the second half of lines 623 to 22 and lines 311 to 335;
- partial VBI consists of the second half of lines 623 to 5 and lines 311 to 318;
- line 23 is always fully active.

In an ITU-R656-compliant digital TV line, the active portion of the digital line is the portion included between the SAV (Start of Active Video) and EAV (End of Active Video) words. However, this digital active line starts somewhat earlier and may end slightly later than the active line usually defined by analog standards.

The DENC allows two approaches:

- Encodes the full digital line (720 pixels / 1440 clock cycles). In this case, the output waveform will reflect the full YCrCb stream included between SAV and EAV.
- Drops some YCrCb samples at the extremities of the digital line so that the encoded analog line fits within the analog ITU-R/SMPTE specifications.

In all cases, the transitions between horizontal blanking and active video are shaped to avoid too steep edges within the active video. Figure 122 gives typical timings concerning the horizontal blanking interval and the active video interval.

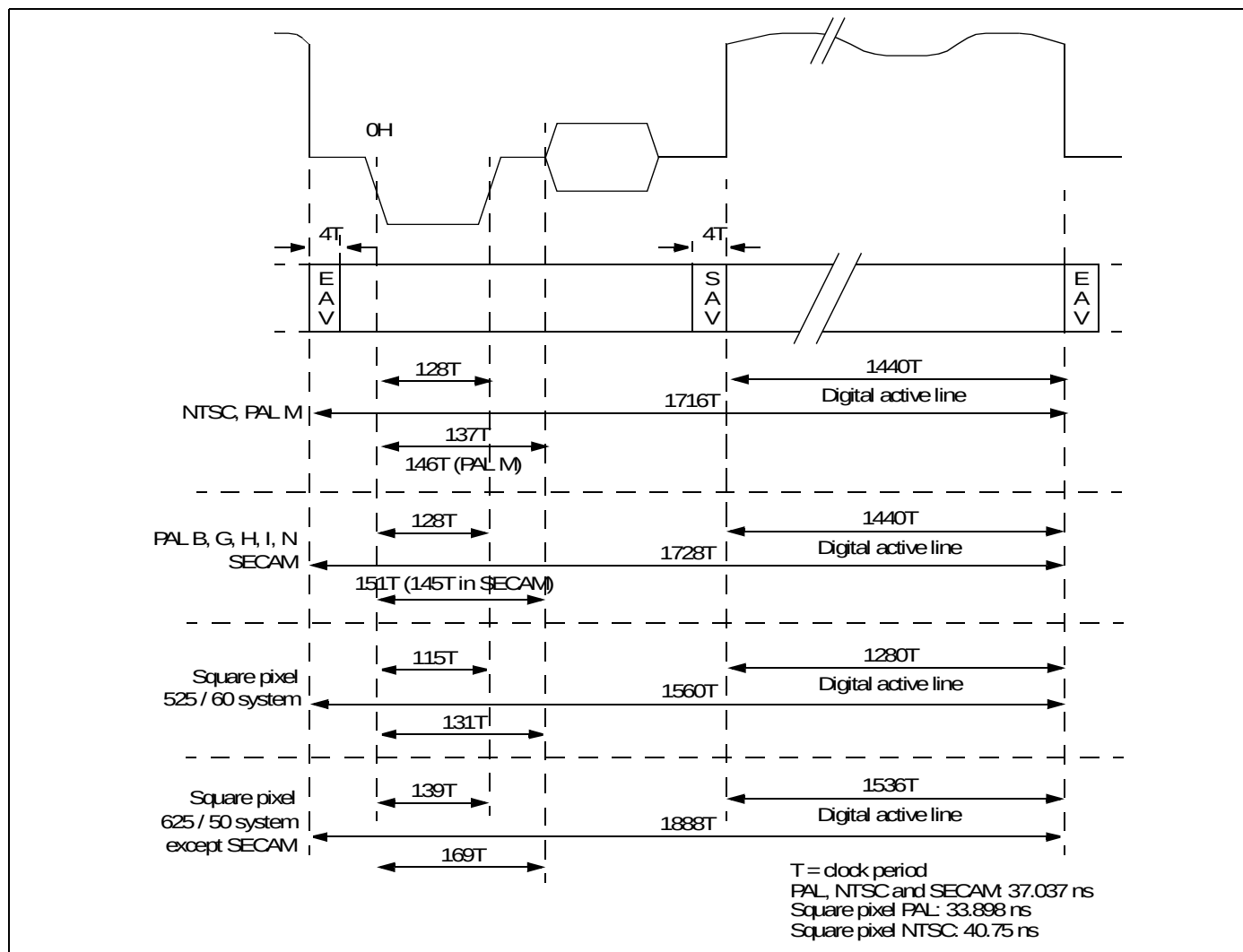


Figure 114 Input data format (ITU-R656 /D1 4:2:2)

Note The burst envelope shown here indicates the location from which the first subcarrier positive zero crossing is sought (with respect to the 0_{μ} reference). The normal burst always starts with such a positive zero crossing.

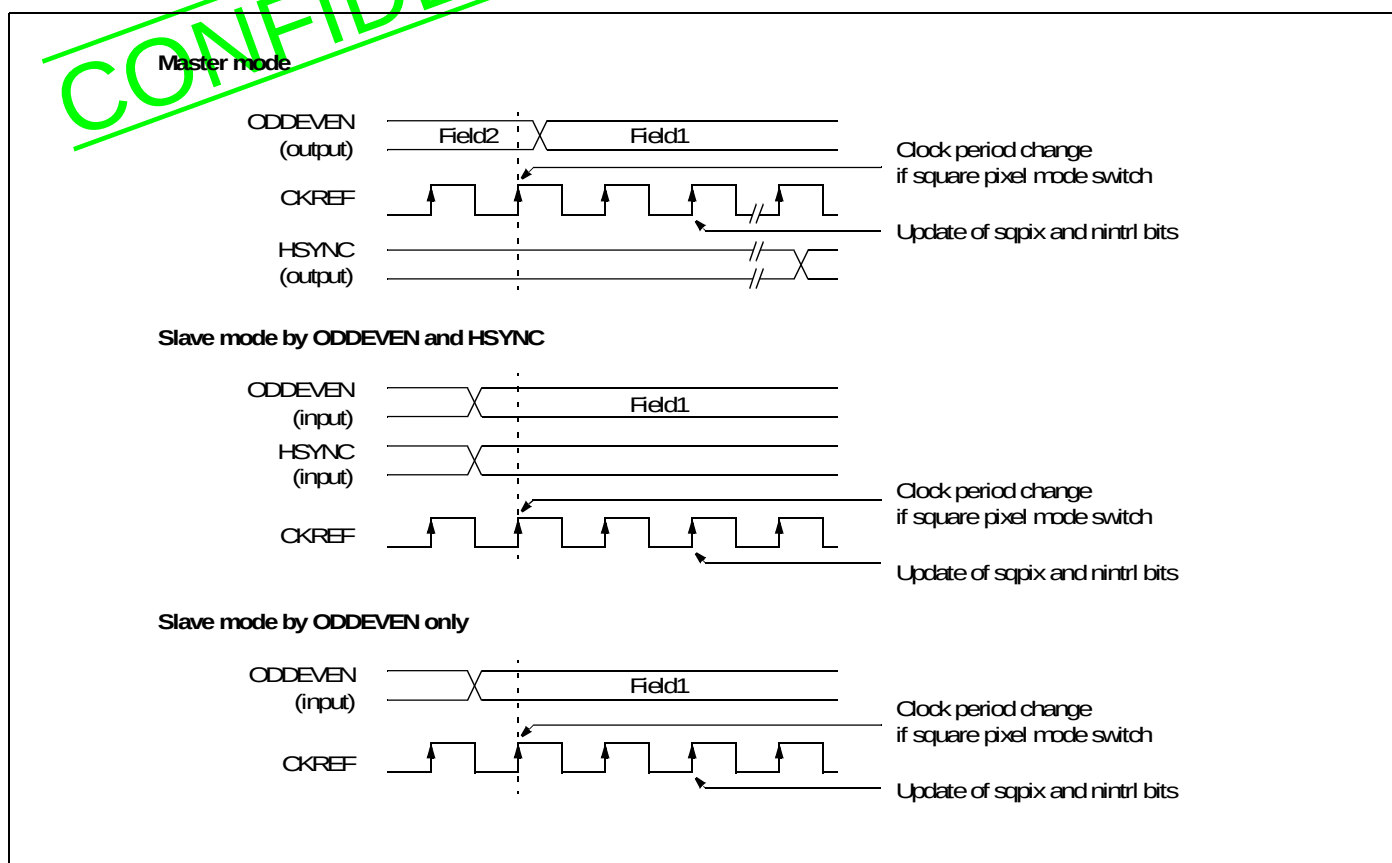


Figure 115 Square pixel and/or non-interlaced mode switch

- Note**
1. These diagrams are valid with contents of "delay" and "synchro-delay" register fields equal to the default values.
 2. If on-the-fly format changing is required, clock switching must be synchronized onto the start of frame as shown in the above waveform. Internally, **sqpix** and **nintrl** bits update is taken into account on the beginning of a new frame.

CONFIDENTIAL

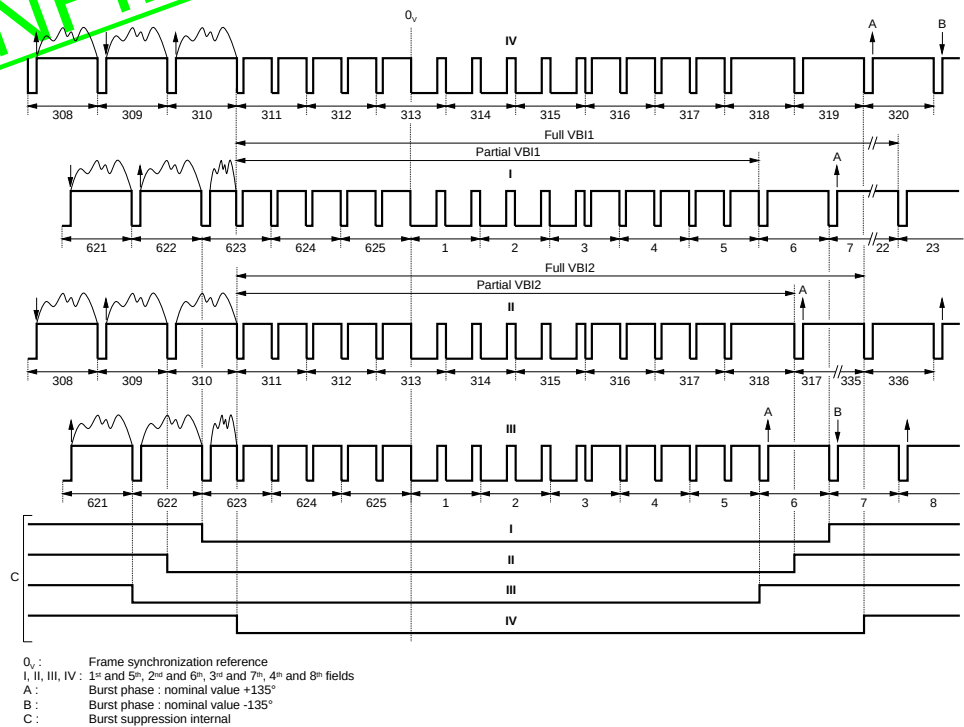


Figure 116 PAL-BDGHI, PAL-N typical VBI waveform, interlaced mode (ITU-R625 line numbering)

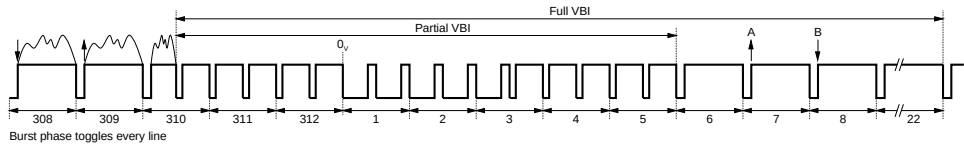


Figure 117 PAL-BDGHI, PAL-N typical VBI waveform, non-interlaced mode ('CCIR-like' line numbering)

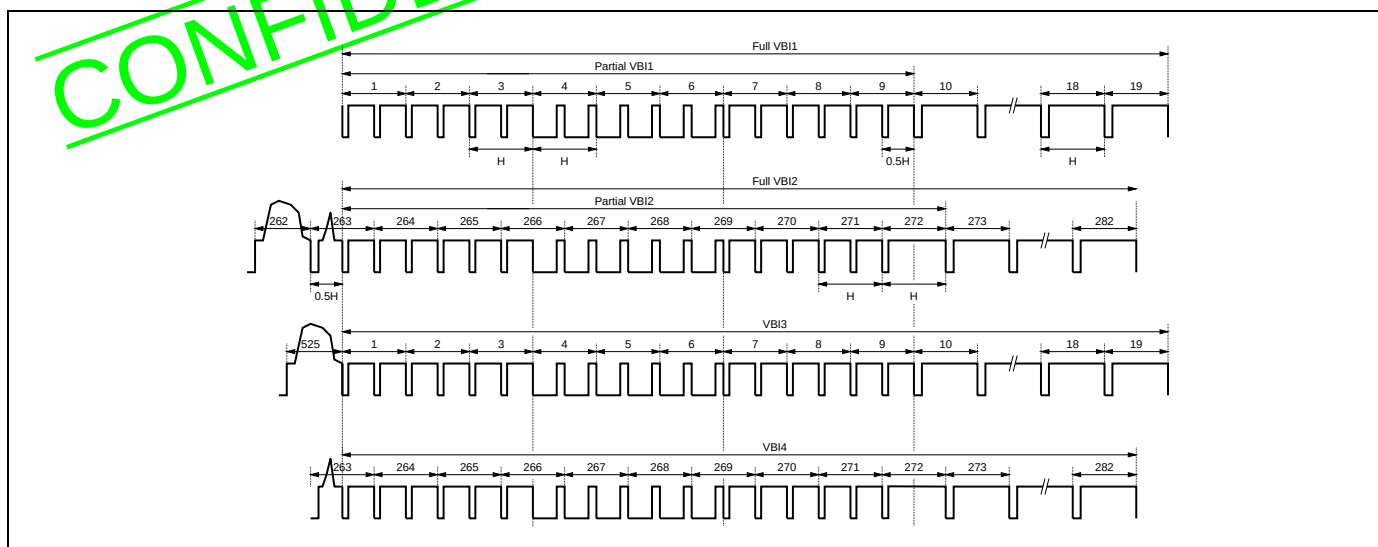


Figure 118 NTSC-M typical VBI waveforms, interlaced mode (SMPTE-525 line numbering)

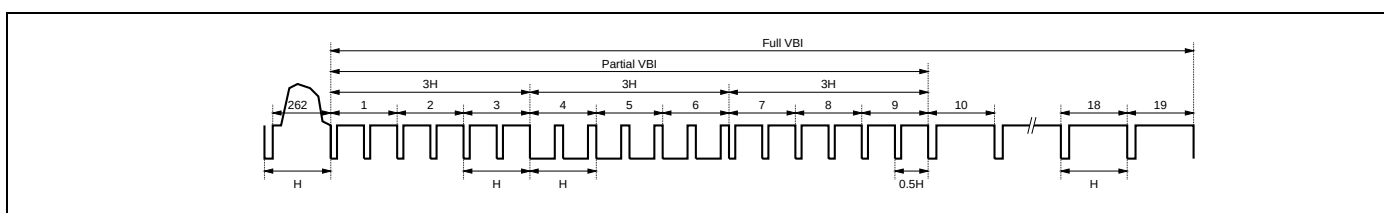


Figure 119 NTSC-M typical VBI waveforms, non-interlaced mode (SMPTE-like line numbering)

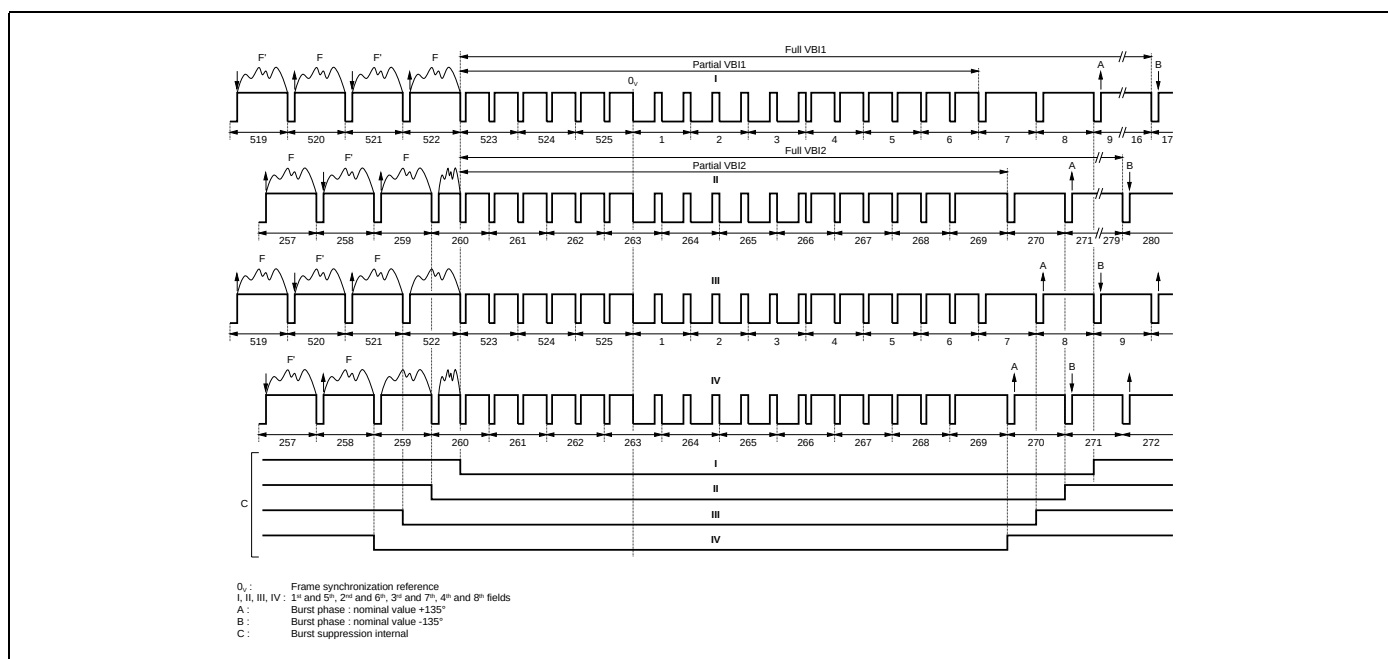


Figure 120 PAL-M typical VBI waveforms, interlaced mode (ITU-R/CCIR-525 line numbering)

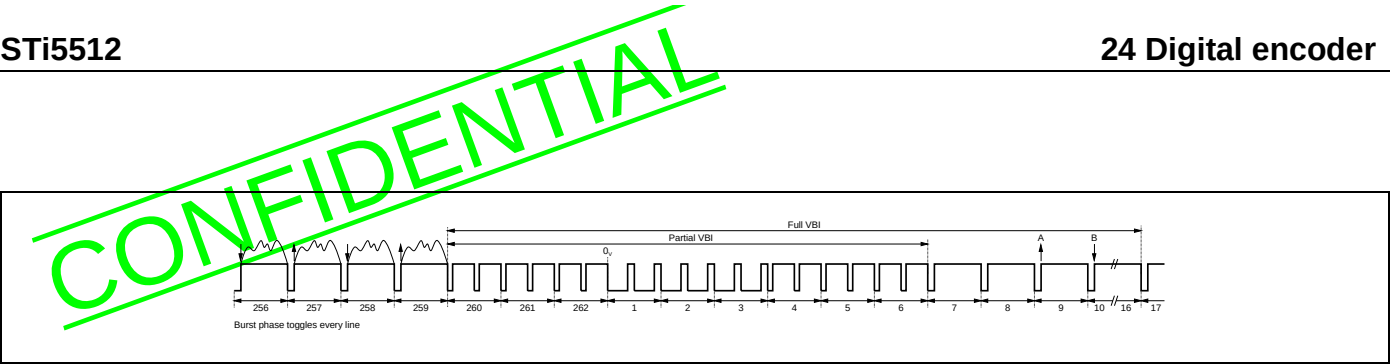


Figure 121 PAL-M typical VBI waveforms, non-interlaced mode (ITU-R/CCIR-like line numbering)

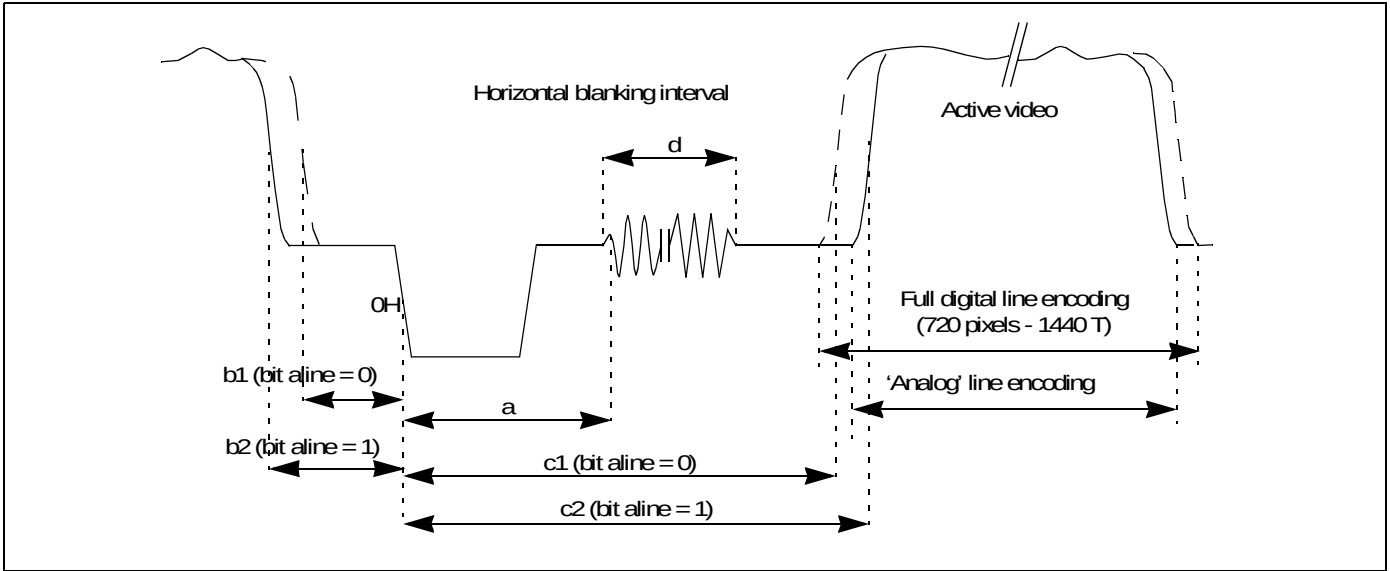


Figure 122 Horizontal blanking interval and active video timings

	NTSC-M	PAL-BDGIH	PAL-N	PAL-M	SECAM
a ¹	5.38 μs (even lines) 5.52 μs (odd lines)	5.54 μs (A-type) 5.66 μs (B-type)	5.54 μs (A-type) 5.66 μs (B-type)	5.73 μs (A-type) 5.87 μs (B-type)	5.37 μs
b1	1.56 μs	1.3 μs	1.3 μs	1.56 μs	1.0 μs
b2	1.56 μs	1.52 μs	1.52 μs	1.56 μs	1.52 μs
c1	8.8 μs	9.6 μs	9.6 μs	8.8 μs	9.9 μs
c2	9.41 μs	10.48 μs	10.48 μs	9.41 μs	10.48 μs
d	9 cycles of 3.58MHz	10 cycles of 4.43MHz	9 cycles of 3.58MHz	9 cycles of 3.58MHz	-

Table 108 Typical timing values from Figure 121

1. These are typical values, actual values will depend of the static offset programmed for subcarrier generation.

24.3 Reset procedure

A hardware reset sets the DENC in HSYNC+ODDEVEN (line-locked) slave mode, for NTSC-M, interlaced ITU-R601 encoding Closed-captioning, WSS, VPS, Teletext and CGMS encoding are all disabled.

The configuration can then be customized by writing into the appropriate registers. A few registers are never reset, their contents are unknown until the first loading (see the STi5512 Register Manual).

It is also possible to perform a software reset by setting the 7th bit in the **DEN_CFG6** register. The device responds in a similar way as after a hardware reset except that the configuration registers and a few other registers are not altered.

24.4 Slave modes

24.4.1 Introduction

The following slave modes are available:

- ODDEVEN(VSYNC) + HSYNC based (line-based sync),
- ODDEVEN(VSYNC)-only based (frame-based sync),
- sync-in-data based (line locked or frame locked).

ODDEVEN refers to an odd/even field flag, also known as BottomTop. HSYNC is a line sync signal and VSYNC is a vertical sync signal. Their waveforms are depicted in [Figure 123](#). The polarities of HSYNC and ODDEVEN(VSYNC) are independently programmable in all slave modes. In all slave modes, ODDEVEN(VSYNC) and/or HSYNC signals must be related to **PIXCLK2X**, the principal DENC clock. In other words, there is *NO* genlocking performed by the DENC.

24.4.2 Line-based synchronization

ODDEVEN+HSYNC based synchronization

Synchronization is performed on a line-by-line basis by locking onto incoming ODDEVEN and HSYNC signals. Refer to [Figure 123](#) for waveforms and timings. The polarities of the active edges of HSYNC and ODDEVEN are programmable and independent. The first active edge of ODDEVEN initializes the internal line counter but encoding of the first line does not start until an HSYNC active edge is detected (at the earliest, an HSYNC transition may be at the same time as ODDEVEN). At that point, the internal sample counter is initialized and encoding of the first line starts. Then, encoding of each subsequent line is individually triggered by HSYNC active edges. The phase relationship between HSYNC and the incoming YCrCb data is normally such that the first clock rising edge following the HSYNC active edge samples Cb (i.e. a blue chroma sample within the YCrCb stream). It is however possible to internally delay the incoming sync signals (HSYNC+ODDEVEN) by up to 3 clock cycles to cope with different data/sync phases, using configuration bits **syncin_ad** in **DEN_CFG4**. The DENC is thus fully slaved to the HSYNC signal, which means that lines may contain more or less samples than usual.

- If the digital line is shorter than its nominal value, the sample counter is re-initialized when the “early” HSYNC arrives and all internal synchronization signals are re-initialized.
- If the digital line is longer than its nominal value, the sample counter is stopped when it reaches its nominal end-of-line value and waits for the “late” HSYNC before reinitializing.

The field counter is incremented on each ODDEVEN transition. The line counter is reset on the HSYNC following each active edge of ODDEVEN.

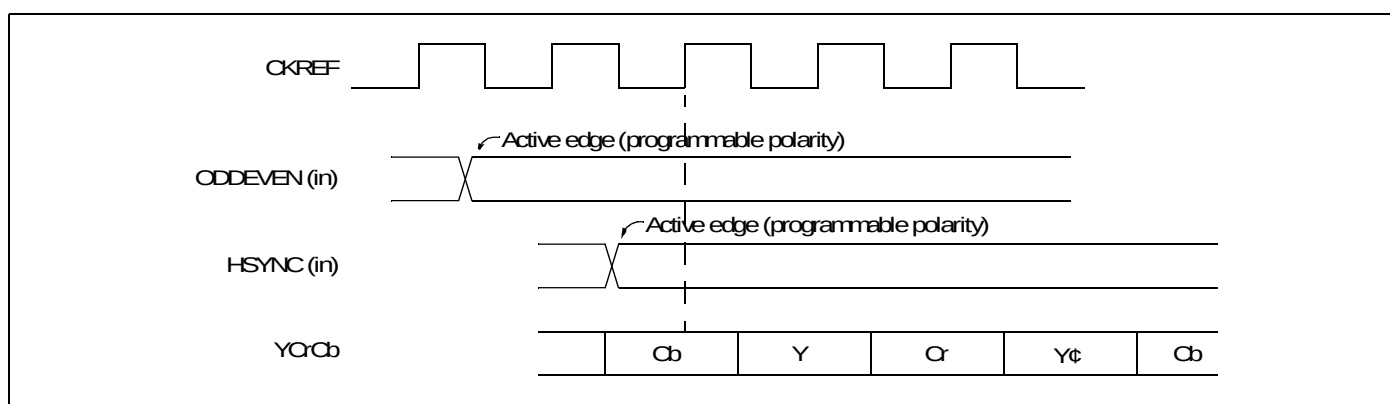


Figure 123 HSYNC + ODDEVEN based slave mode sync signals

Note This figure is valid for bits `syncin_ad[1:0]` = default

HSYNC+VSYNC based synchronization

Synchronization is performed on a line-by-line basis by locking onto incoming VSYNC and HSYNC signals. Refer to [Figure 125](#) for waveforms and timings. The polarities of HSYNC and VSYNC are programmable and independent.

The incoming VSYNC signal is immediately transformed into a waveform identical to the odd/even waveform of an ODDEVEN signal, therefore the behavior of the core is identical to that described above for ODDEVEN+HSYNC based synchronization. Again, the phase relationship between HSYNC and the incoming YCrCb data is normally such that the first clock rising edge following the HSYNC active edge samples "Cb" (i.e. a 'blue' chroma sample within the YCrCb stream). It is however possible to internally delay the incoming sync signals (HSYNC+VSYNC) by up to 3 clock cycles to cope with different data/sync phasing, using configuration bits `syncin_ad` (`DEN_CFG4`).

The field counter is incremented on each active edge of VSYNC.

24.4.3 Frame-based synchronization

Genlocking using external sync extraction

The genlock slave mode is used when the OSD (generated by the STi5512 and output via the DENC's RGB outputs) is mixed with analog video from which the vertical and horizontal synchronization signals are extracted by ROHM's BA7046.

A software-based control loop locks the 27MHz system clock to the analog video signal. The vertical sync signal from the ROHM sync extractor is connected to a capture port on the STi5512 and generates an equivalent of the PCR (programmable clock reference) for the analog signal. The ST20 controls an external VCXO through a PCM output port.

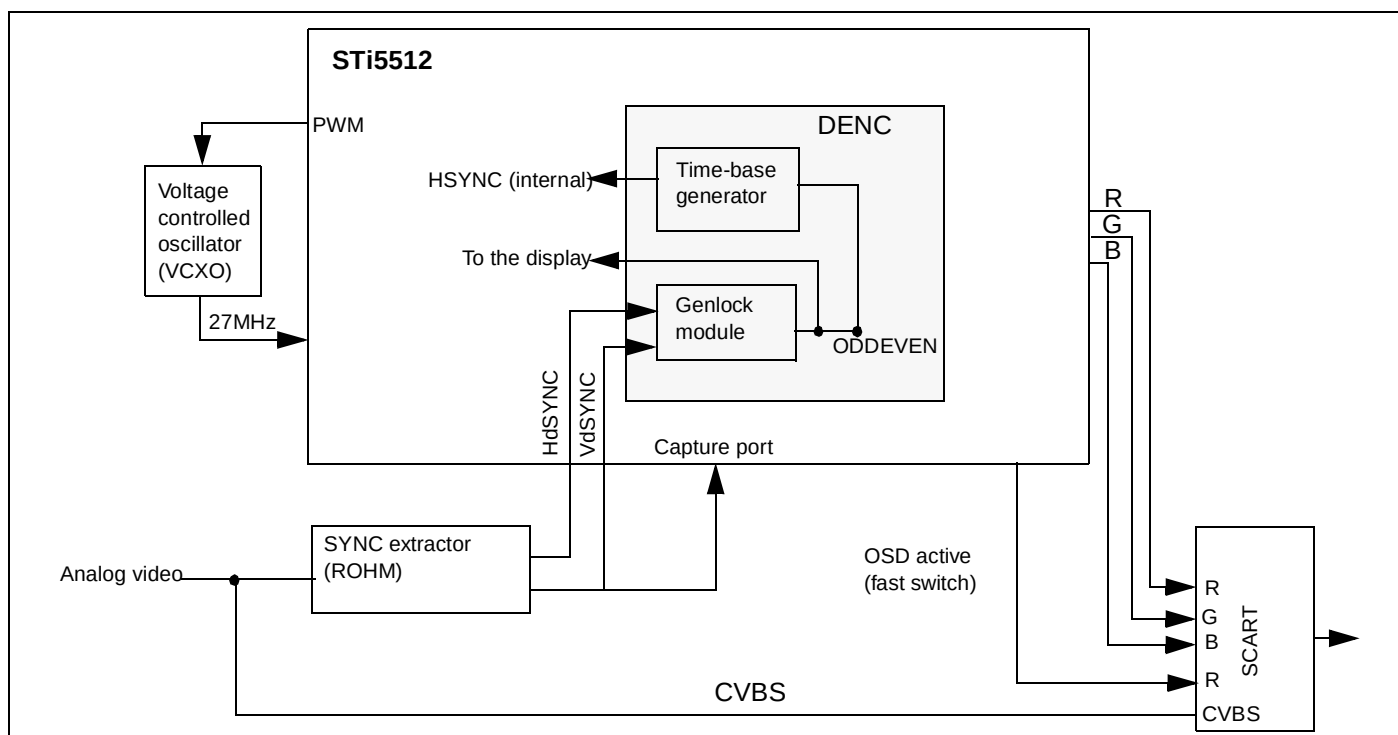


Figure 124 Genlock system using a SCART

VdSYNC and HdSYNC signals are provided by the external ROHM sync extractor (BA7046). The DENC regenerates the signal ODDEVEN (the device internal vertical synchronization signal) from these inputs by locking onto VdSYNC with the internal clock. HSYNC is generated internally by the sample counter and is not locked to HdSYNC. These HSYNC and ODDEVEN signals are then used inside the DENC and are also output to drive display and video parts of

the device. The DENC behaves in the same way as for *ODDEVEN-only based synchronization* on page 198. Register DEN_CFG0 should be programmed as for ODDEVEN-only based synchronization.

The polarities of HSYNC and ODDEVEN are programmable and independent. VdSYNC and HdSYNC are both falling edge active.

The DENC analog RGB output is delayed by one line vertically and by 25 pixels horizontally, compared to the analog video source. This must be taken into account for OSD, YDO and XDO programming. Refer to register DEN_CFG7 for genlock mode programming.

ODDEVEN-only based synchronization

Synchronization is performed on a frame-by-frame basis by locking onto an incoming ODDEVEN signal. A line sync signal is derived internally and is also issued to the outside as HSYNC. Refer to Figure 126 for waveforms and timings. The phase relationship between ODDEVEN and the incoming YCrCb data is normally such that the first clock rising edge following the ODDEVEN active edge samples “Cb” (i.e. a “blue” chroma sample within the YCrCb stream). It is however possible to internally delay the incoming ODDEVEN signal by up to 3 clock cycles to cope with different data/ sync phasing, using configuration bits **syncin_ad** in **DEN_CFG4**.

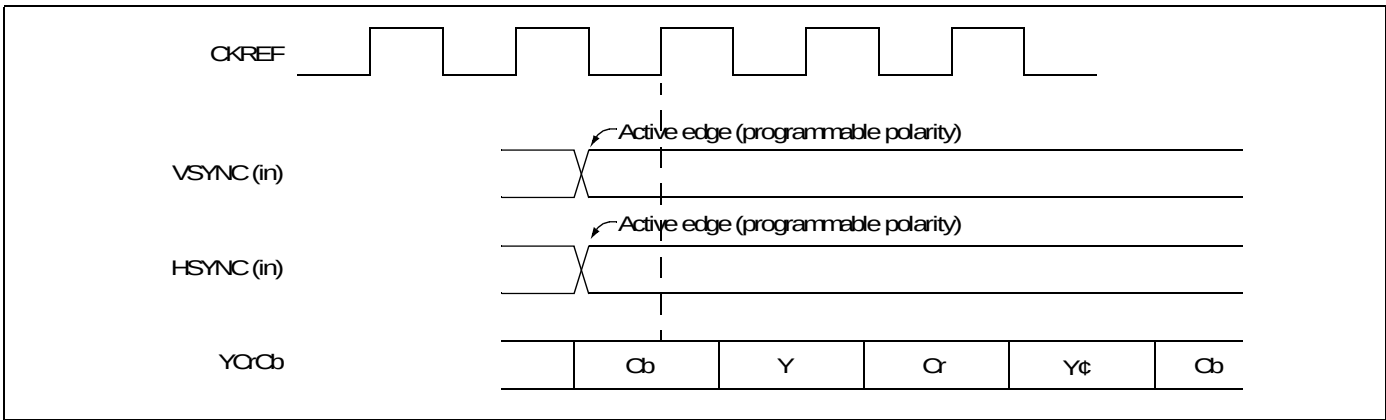


Figure 125 HSYNC + VSYNC based slave mode sync signals

- Note
- 1. This figure is valid for bits “syncin_ad[1:0]” = default.
 - 2. The active edges of HSYNC and VSYNC should normally be simultaneous. It is permissible that HSYNC transitions before VSYNC, but VSYNC must not transition before HSYNC.

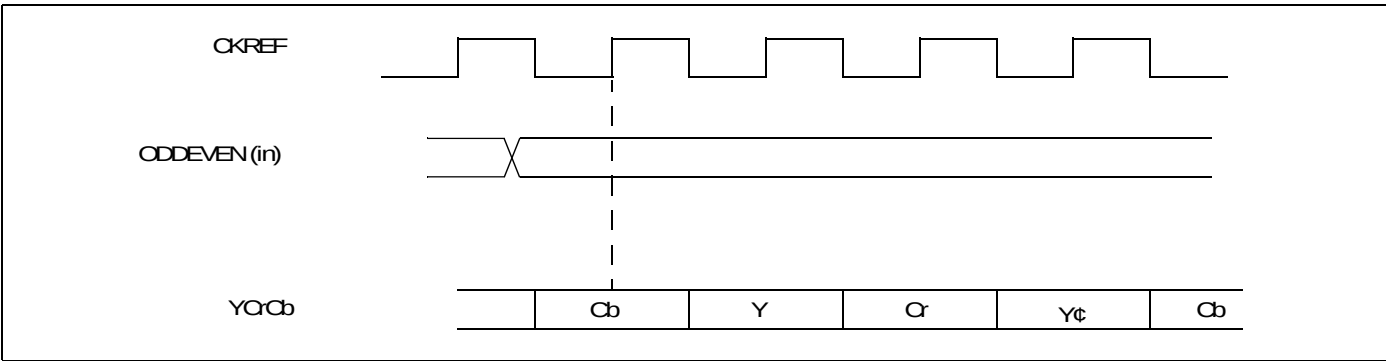


Figure 126 ODDEVEN based slave mode sync signals

Note This figure is valid for bits **syncin_ad[1:0]** = default

The first active edge of ODDEVEN triggers generation of the analog sync signals and encoding of the incoming video data. Frames being supposed to be of constant duration, the next ODDEVEN active transition is expected at a precise time after the last ODDEVEN detected.



So, once an active ODDEVEN edge has been detected, checks that the following ODDEVEN are present at the expected instants are performed.

Encoding and analog sync generation carry on unless three successive fails of these checks occur.

In that case, three behaviors are possible, according to the configuration programmed in registers **DEN_CFG1-2**:

- If **freerun** is enabled, the DENC carries on outputting the digital line sync HSYNC and generating analog video just as though the expected ODDEVEN edge had been present. However, it will re-synchronize onto the next ODDEVEN active edge detected, whatever its location.
- If **freerun** is disabled but bit **syncok** is set in the configuration registers, the DENC sets the active portion of the TV line to black level but carries on outputting the analog sync tips (on Ys and CVBS) and the digital line sync signal HSYNC. When programmed, Macrovision™ pseudo-sync pulses and AGC pulses are also present in the analog sync waveform.
- If **freerun** is disabled and bit **syncok** is not set, all analog video is at black level and neither analog sync tips nor digital line sync are output.

This mode is a frame-based sync mode, as opposed to a field-based sync mode. This means that only one type of edge (rising or falling, according to programming) is of interest to the DENC; the other one is ignored.

VSNC-only based synchronization

Synchronization is performed on a frame-by-frame basis by locking onto an incoming VSYNC signal.

An auxiliary line sync signal HSYNC must also be fed to the DENC, which uses it to reconstruct from VSYNC and HSYNC information an internal odd/ even waveform identical to that of an ODDEV signal. Therefore the behavior of the core is identical to that described above for ODDEV-only based synchronization (except that nothing is output on HSYNC pin since it is an input port in this mode).

Note that HSYNC is an input but has no other use than allowing the DENC to decide whether an incoming VSYNC pulse flags an odd or an even field. In other words, the DENC does not lock onto HSYNC in this mode since this is NOT a line-locked mode.

The phase relationship between VSYNC and the incoming YCrCb data is normally such that the first clock rising edge following the VSYNC active edge samples "Cb" (i.e. a 'blue' chroma sample within the YCrCb stream). It is however possible to internally delay the incoming sync signals (VSYNC+HSYNC) by up to 3 clock cycles to cope with different data/sync phasing, using configuration bits Syncin_ad (DEN_CFG4).

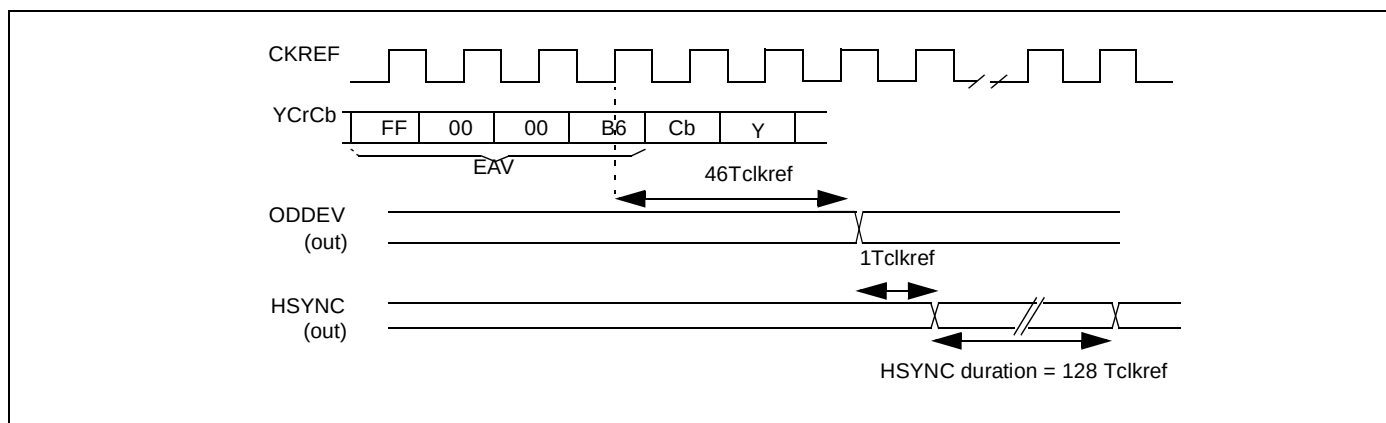


Figure 127 Data (EAV) based slave mode sync signals

24.4.4 Sync-in-data based synchronization

"End-of-frame" word-based synchronization

Synchronization is performed by extracting the 1-to-0 transitions of the 'F' flag (end-of-frame) from the 'EAV' (End-of-Active Video) sequence embedded within ITU-R656 / D1 compliant digital video streams. Both a frame sync signal and a line sync signal are derived and are made available externally as ODDEV and HSYNC. Refer to [Figure 127](#) for waveforms and timings.

The first successful detection of the 'F' flag triggers generation of the analog sync signals and encoding of the incoming video data. Frames being supposed to be of constant duration, the next EAV word containing the 'F' flag is expected at a precise time after the latest detection.

So, once an active 'F' flag has been detected, checks that the following flags are present within the incoming video stream at the expected times are performed.

Encoding and analog sync generation carry on unless there are three successive fails of these checks. Then, depending on the programmed configuration, one of the following three events occurs:

- If **'free-run'** is enabled, the DENC carries on generating the digital frame and line syncs (ODDEV and HSYNC) and generating analog video just as though the expected 'F' flag had been present. However, it will re-synchronize onto the next 'F' flag detected within the incoming ITU-R656/ D1 video stream.
- If **'free-run'** is disabled but the bit **'syncok'** is set in the configuration registers, the DENC sets the active portion of the TV line to black level but carries on outputting the analog sync tips (on Ys and CVBS) and the digital frame and line sync signals ODDEV and HSYNC. (When programmed, Macrovision™ pseudo-sync pulses and AGC pulses are also present in the analog sync waveform).
- If **'free-run'** is disabled and the bit **'syncok'** is not set, all analog video is at black level and neither analog sync tips nor digital frame/line sync are output.

The SAV and EAV words are Hamming-decoded.

After detection of two successive errors, a bit is set in the status register to inform the micro-controller of the poor transmission quality.

'End-of-line' word-based synchronization

Synchronization is performed by extracting the 'F' and 'H' flags from the 'SAV' (Start of Active Video) and 'EAV' (End of Active Video) words embedded within ITU-R656/D1 compliant digital video streams. A line sync signal and a frame sync signal are derived internally from these flags and are issued to the outside on the HSYNC and ODDEVEN/VSNC pins in output mode. These signals are also exploited by the core of the circuit which treats them like it treats incoming ODDEVEN and HSYNC signals in HSYNC+ODDEVEN based synchronization.

Autotest mode

An autotest mode is available, which causes the DENC to produce a color bar pattern, in the appropriate standard, independently from the video input.

The autotest mode is started by setting to 7 the 3-bit field **sync** in the register **DEN_CFG0**. As this mode sets the DENC in master mode, VSYNC/ODDEVEN and HSYNC signals are in output mode. In table below, the decimal values of Y, Cr and Cb are shown corresponding to the autotest color bar.

	Y	Cr	Cb
Black	16	128	128
Blue	36	116	212
Red	64	212	100

Table 109 Autotest colors

CONFIDENTIAL

	Y	Cr	Cb
Magenta	84	200	184
Green	112	56	72
Cyan	136	44	156
Yellow	160	140	44
White	236	128	128

Table 109 Autotest colors

The corresponding decimal output values just before the DACs are shown graphically in Figure 129 and Figure 131 . Both figures show the static values corresponding to the input values in Table 109 .

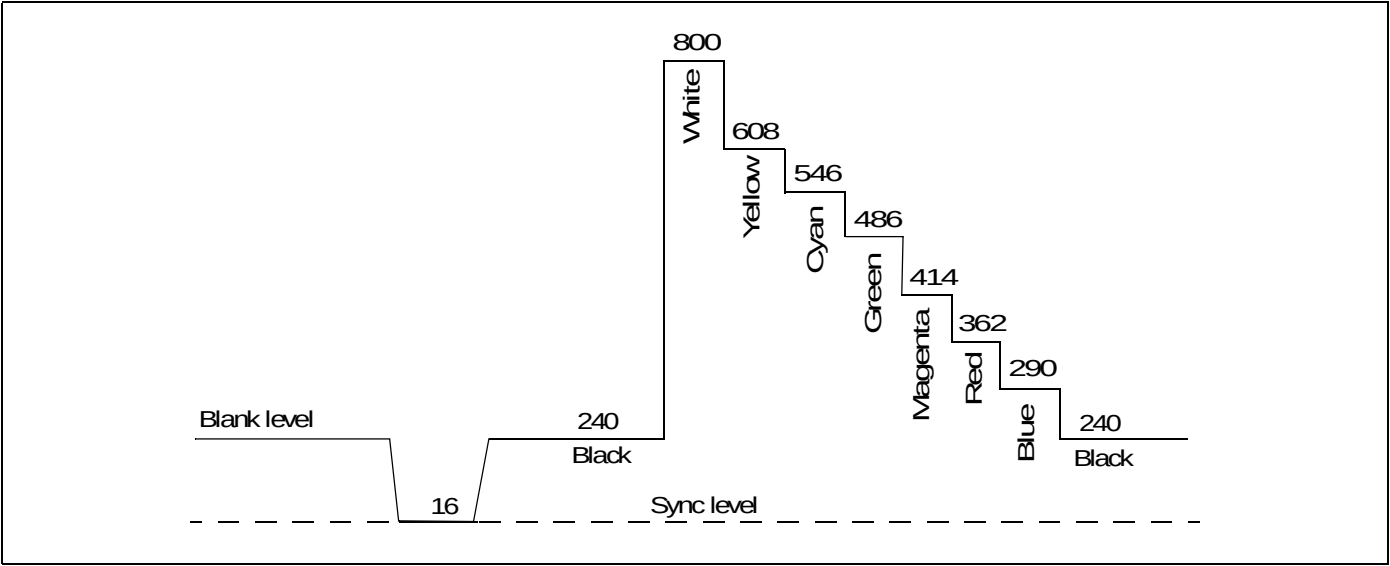


Figure 128 Luminance output levels in autotest for NTSC without set-up

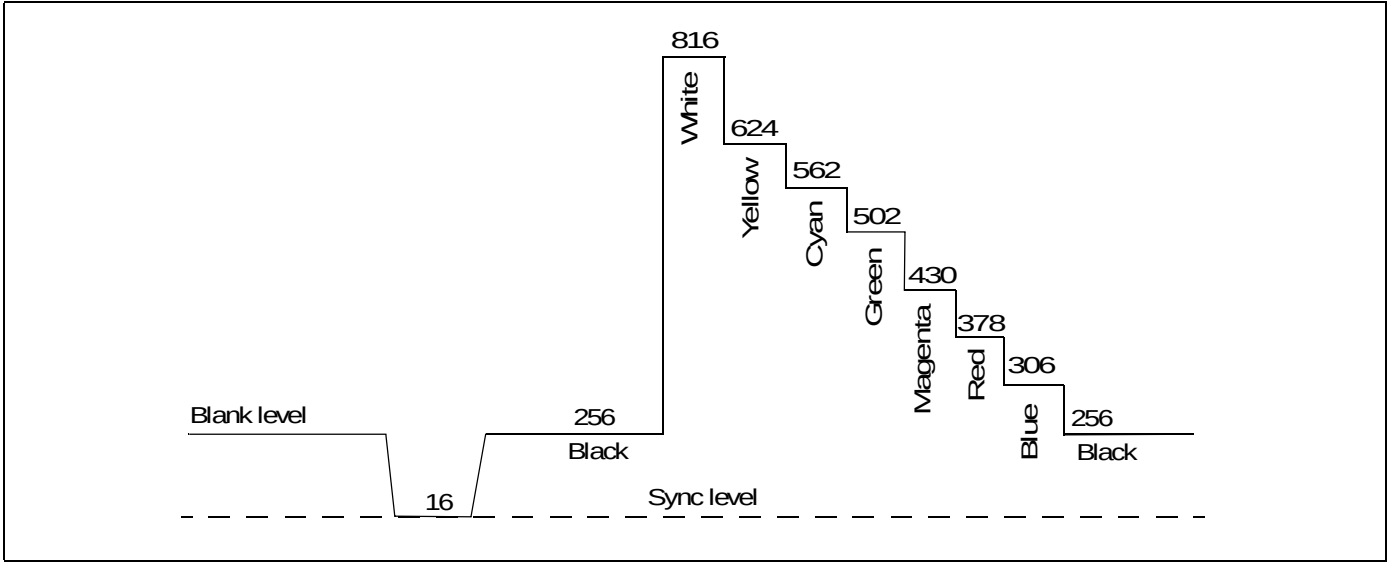


Figure 129 Luminance output levels in autotest for PAL (BGHI) and SECAM

24.5 Input demultiplexor

The incoming YCrCb data, as well as Y4 and CrCb in 4:4:4 mode, is demultiplexed into a “blue-difference” chroma information stream, a “red-difference” chroma information stream and a luma information stream. Incoming data bits are treated as blue, red or luma samples according to their relative position with respect to the sync signals in use and the contents of configuration bits **syncin_ad** (slave modes) or **syncout_ad** (master mode). Brightness, saturation and contrast are then performed on demultiplexed data, refer to the Register Manual registers **DEN_REG_69**, **DEN_REG_70** and **DEN_REG_71**.

The ITU-R601 recommendation defines the black luma level as Y=16 and the maximum white luma level as Y = 235. Similarly, it defines 225 quantification levels for the color difference components (Cr, Cb), centered around 128. After the saturation, brightness and contrast stage, the incoming YCrCb samples can be saturated in the input multiplexer with the following rules:

- For Cr or Cb samples: Cr,Cb > 240 => Cr,Cb saturated at 240
Cr,Cb < 16 => Cr,Cb saturated at 16
- For Y samples: Y > 235 => Y saturated at 235
Y < 16 => Y saturated at 16

This avoids having to heavily saturate the composite video codes before digital-to-analog conversion in case erroneous or unrealistic YCrCb samples are input to the encoder (there may otherwise be overflow errors in the codes driving the DACs), and therefore avoids generating a distorted output waveform.

However, in some applications, it may be desirable to let “extreme” YCrCb codes pass through the demultiplexor. This is controlled using bit **maxdyn** in register **DEN_CFG6**. In this case, only codes 0x00 and 0xFF are overridden; if such codes are found in the active video samples, they are forced to 0x01 and 0xFE.

In any case, the YCrCb codes are not overridden for EAV/SAV decoder.

24.6 Sub-carrier generation

A Direct Digital Frequency Synthesizer (DDFS) generates the required color sub-carrier frequency using a 24-bit phase accumulator. This oscillator feeds a quadrature modulator which modulates the base-band chrominance components.

The sub-carrier frequency is obtained from the following equation:

$$F_{sc} = (Increment_Word / 2^{24}) \times CKREF$$

where *Increment_Word* is a 24-bit value.

Hard-wired *Increment_Word* values are available for each standard and can be automatically selected. Alternatively in PAL and NTSC (according to bit **selrst_inc** in **DEN_CFG5**), the frequency can be fully customized by programming other values into a dedicated *Increment_Word* register, **DEN_IDFS**. This allows, for instance, the encoding of NTSC-4.43 or PAL-M-4.43.

This is done with the following procedure:

- Program the required increment in **DEN_IDFS**.
- Set bit **selrst_inc** to “1” in register **DEN_CFG5**.
- Perform a software reset using register **DEN_CFG6**. This sets all bits in all DENC registers except **DEN_CFGn** to their default value.
Alternatively, set **DEN_CFG8** bits **ph_rst_mode[1:0]** to “01”. Then the frequency (and phase) update is done on the beginning of the next video line.

Warning: if a standard change occurs after the software reset, the increment value is automatically re-initialized with the hard wired or loaded value according to bit selrst

The reset phase of the color sub-carrier can also be software-controlled by register **DEN_PDFS**.

The sub-carrier phase can be periodically reset to its nominal value to compensate for any drift introduced by the finite accuracy of the calculations. In PAL and NTSC sub-carrier phase adjustment can be performed every line, every eight fields, every four fields, or every two fields (**DEN_CFG2** bits **valrst[1:0]**). If SECAM is performed, the sub-carrier phase is reset every line. When performing external genlocking, the reference frequency of the regenerated clock may slightly deviate depending on the line length measurement. To prevent this drift from corrupting the colors, the "Color Frequency Control" line (CFC pin) can be used to update the 24-bit increment of the DDFS and keep the color sub-carrier stable (see Figure 130). Note, if this feature is not used, the CFC pin must be grounded. The DENC can also detect phase reset information coming up to 10 clock cycles after transmission of the CFC's LSB. (see **DEN_CFG8** bits **ph_rst_mode[1:0]**). It can also automatically reset the phase after CFC loading (see **DEN_CFG8** description).

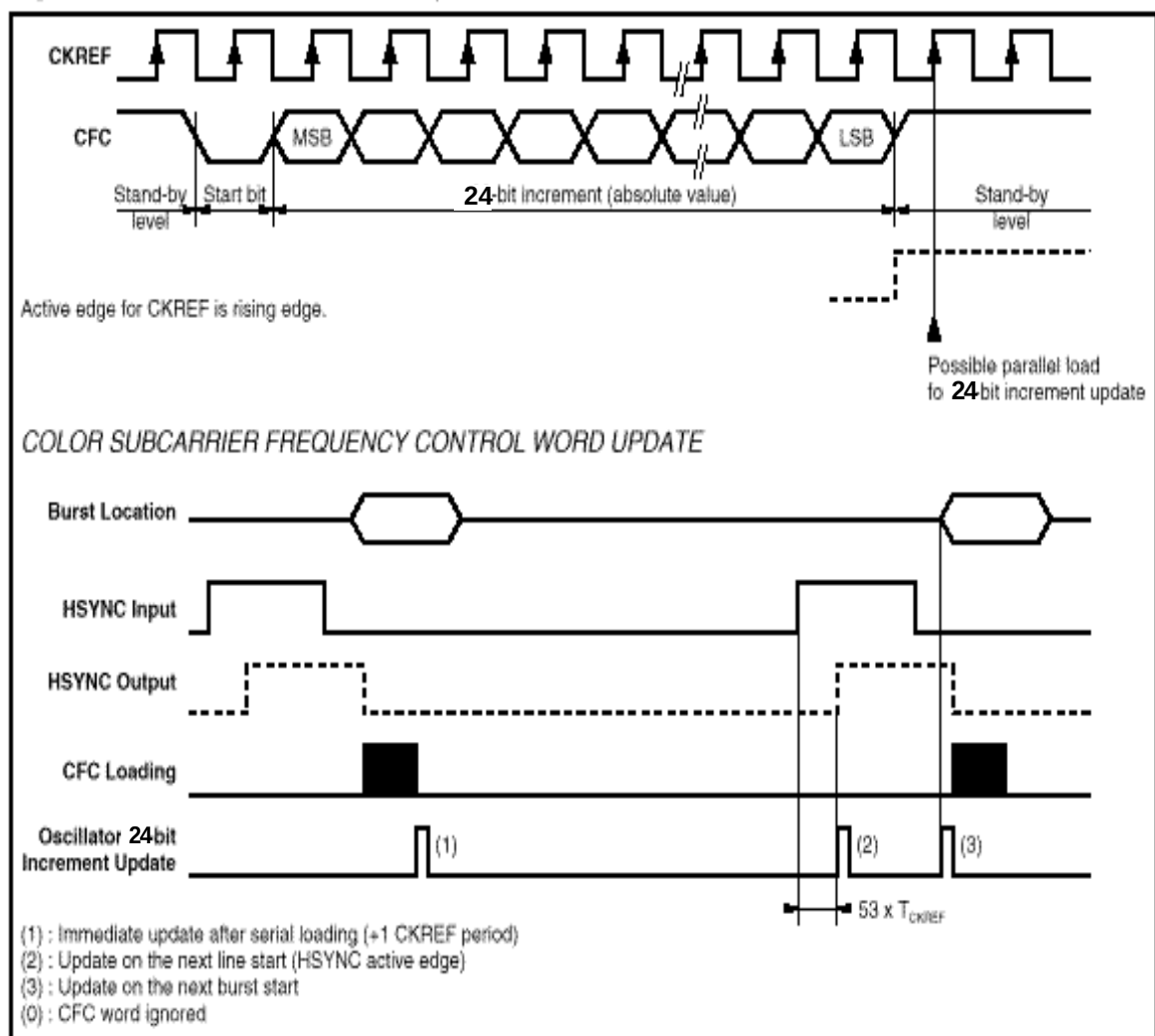


Figure 130 Color sub-carrier frequency control (CFC) transmission format

24.7 Burst insertion (PAL and NTSC)

The color reference burst is inserted so as to always start with a positive zero crossing of the subcarrier sine wave. The first and last half-cycles have a reduced amplitude so that the burst envelope starts and ends smoothly.

The burst contains 9 or 10 sine cycles of 4.43361875MHz or 3.579545MHz (depending on the standard programmed in the register **DEN_CFG0**) as follows:

NTSC-M	9	cycles of	3.579545	MHz
PAL-BDGH	10	cycles of	4.43361875	MHz
PAL-M	9	cycles of	3.579545	MHz
PAL-N	9	cycles of	3.579545	MHz

The burst can be turned off (no burst insertion) by setting **DEN_CFG2** configuration bit **bursten** to 0.

Burst insertion is performed by always starting the burst with a positive-going zero crossing. This guarantees a smooth start and end of burst with a maximum of undistorted burst cycles and can only be beneficial to chroma decoders.

This avoids an uncontrolled initial burst phase, and guarantees a start on a positive-going zero crossing with the consequence that two burst start locations are visible over successive lines, according to the line parity. This is normal and explained below.

In NTSC, the relation between subcarrier frequency and line length creates a 180° subcarrier phase difference (*with respect to the horizontal sync*) from one line to the next according to the line parity. So if the burst always starts with the same phase (positive-going zero crossing), this means the burst will be inserted at time X or at time $X + T_{NTSC}/2$ after the horizontal sync tip according to the line parity, where T_{NTSC} is the duration of one cycle of the NTSC burst.

With PAL, a similar rationale holds, and again there will be two possible burst start locations. The subcarrier phase difference (*with respect to the horizontal sync*) from one line to the next in that case is either 0 or 180° with the following series: A-A-B-B-A-A-...-etc. where A denotes "A-type" bursts and B denotes "B-type" bursts, A-type and B-type being 180° out of phase with respect to the horizontal sync. So two locations are possible, one for A-type, the other for B-type.

This assumes a periodic reset of the subcarrier is automatically performed (see bits **valrst[1:0]** in **DEN_CFG2**). Otherwise, over several frames, the start of burst will drift within an interval of half a subcarrier's cycle. THIS IS NORMAL and means the burst is correctly locked to the colors encoded. The equivalent effect with a gated burst approach would be the following: the start location would be fixed but the phase with which the burst starts (*with respect to the horizontal sync*) would be drifting.

24.8 Subcarrier insertion (SECAM)

Subcarrier frequency in SECAM mode depends on Cr and Cb values (frequency modulation). The color subcarrier frequency is 4 250 000 Hz for Cb=128 (on blue lines) and 4 406 249 Hz for Cr=128 (on red lines). Frequency clipping values are 3 900 000 Hz and 4 756 250 Hz.

The insertion point of the non-modulated sub-carrier is shown in the figure below.

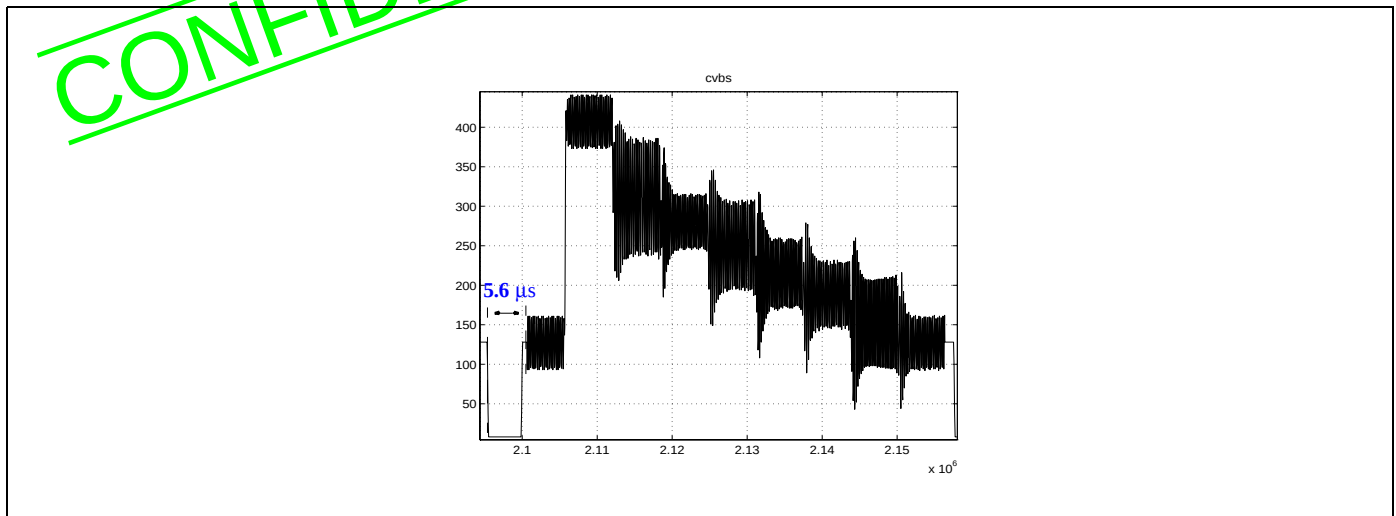


Figure 131 SECAM color bar pattern (blue line)

In odd fields the phase of sub-carrier follows the sequence: 0, 0, π , 0, 0, π , 0, 0, π , ... comparing to a sine wave starting at the same point - 5.6 μ s after horizontal synch pulse (inverted on one line out of every three and also at each frame). This sequence begins from line 1 or line 23 of the first field (see **gen_secam** bit of register **DEN_CFG7**). **DEN_CFG7** bit **inv_phi_secam** allows the inversion of this sequence (π , π , 0, π , π , 0, ... instead of 0, 0, π , 0, 0, π , ...), in odd fields. In even fields the sequence of sub-carrier is always inverted with respect to the odd field one.

To enable SECAM mode, program a "1" in **DEN_CFG7.7** (MSB) and then perform a soft-reset or loading of **DEN_CFG0**.

24.9 Luminance encoding

The demultiplexed Y samples are band-limited and interpolated at CKREF clock rate. The resulting luminance signal is properly scaled before insertion of any Closed-captions, CGMS, VPS, Teletext or WSS data and synchronization pulses.

The interpolation filter compensates for the $\sin(x)/x$ attenuation inherent in D/A conversion and greatly simplifies the output stage filter. See Figures 132, 133 and 134 for characteristic curves.

In addition, the luminance that is added to the chrominance to create the composite CVBS signal can be trap-filtered at 3.58 MHz (NTSC) or 4.43 MHz (PAL). This supports applications oriented towards low-end TV sets which are subject to cross-color if the digital source has a wide luminance bandwidth (e.g. some DVD sources). Note that the trap filter does not affect the S-VHS luminance output nor the RGB outputs. If SECAM is performed, enable the trap filter with 4.43 MHz cut-off frequency on the luma part of the CVBS signal (see **DEN_CFG1** bits **entrap** and **trap_4.43**).

A 7.5 IRE pedestal can be programmed if needed with all standards (see registers **DEN_CFG1** and **DEN_CFG7**). This allows in particular to encode Argentinian and non-Argentinian PAL-N, or Japanese NTSC (NTSC with no set-up).

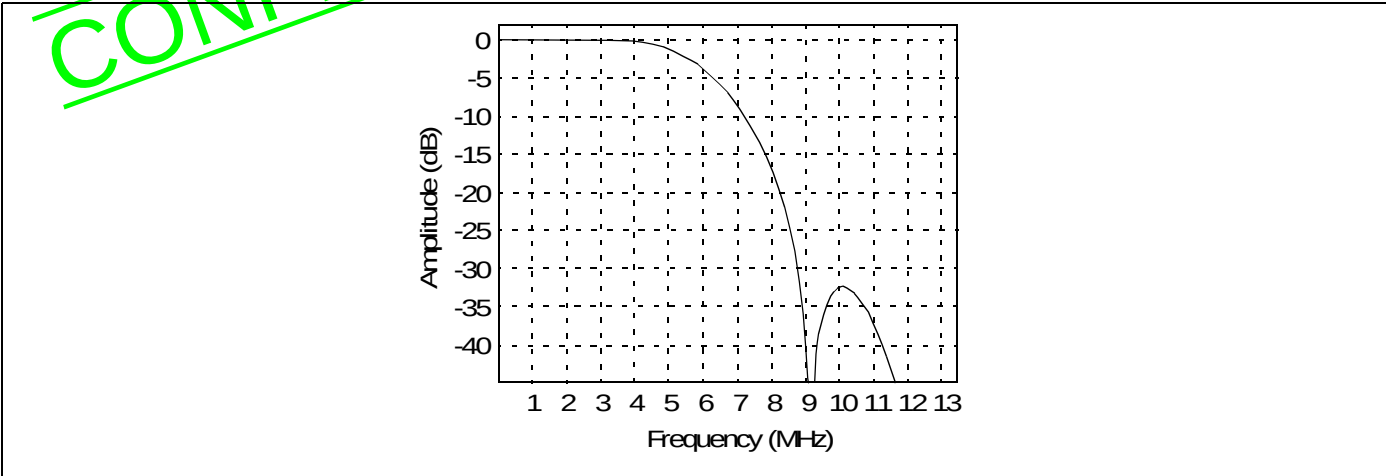


Figure 132 Luma filtering including DAC attenuation

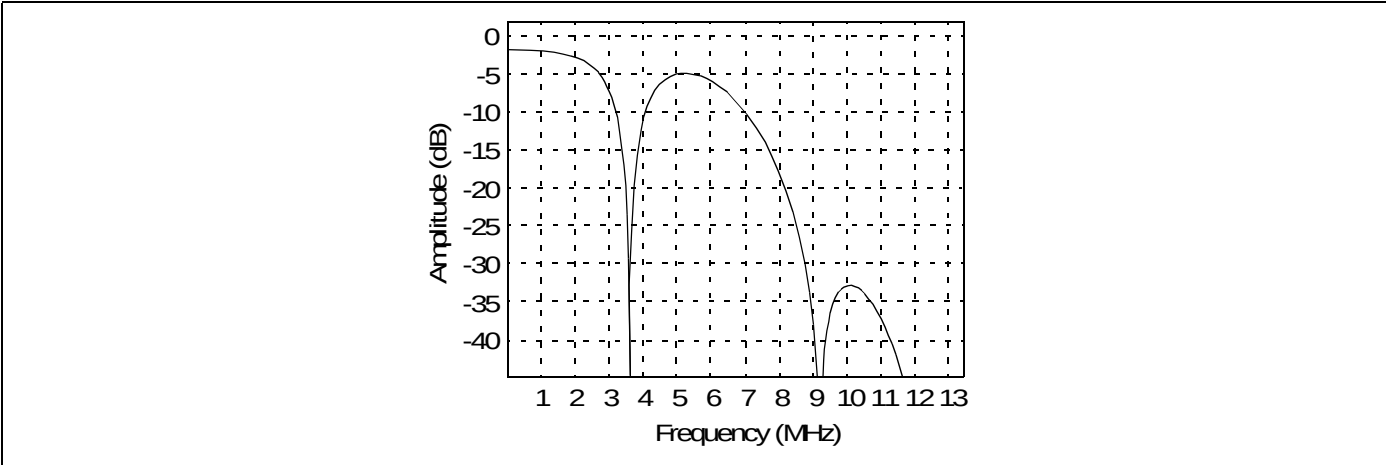


Figure 133 Luma filtering with 3.58MHz trap, including DAC attenuation

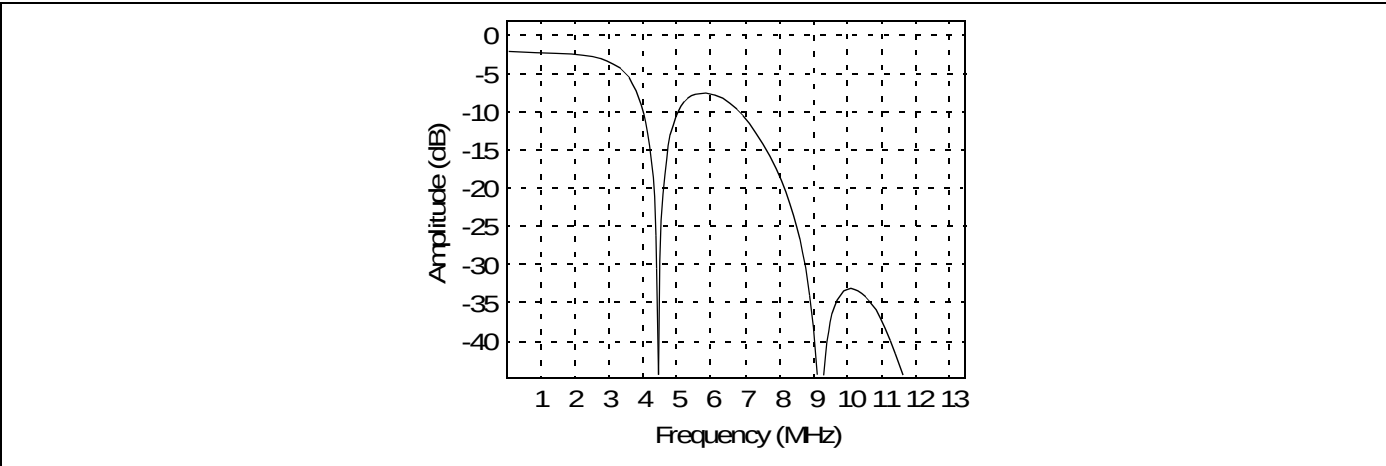


Figure 134 Luma filtering with 4.43MHz trap, including DAC attenuation

A programmable delay can be inserted on the luminance path to offset any chroma/luma delay introduced by off-chip filtering (chroma and luma transitions being coincident at the DAC output with default delay) (see register **DEN_CFG3**).

The luma processing as well as line and field timings in SECAM mode are identical to PAL BDGHI ones.

24.10 Chrominance encoding

U, V (PAL and NTSC) and Dr, Db (SECAM) chroma components are computed from demultiplexed Cb, Cr samples.

Before modulating the subcarrier, these are band-limited and interpolated at CKREF clock rate.

This processing eases the filtering following D/A conversion and allows more accurate encoding.

A set of 4 different filters is available in PAL and NTSC for chroma filtering to fit a wide variety of applications in the different standards and include filters recommended by ITU-R 624-4 and SMPTE170-M.

The available 3-dB bandwidths are 1.1, 1.3, 1.6 or 1.9 MHz.

See Figures 137, 138, 139, 140 and 141 for the various frequency responses and register **DEN_CFG1** for programming.

The narrower bandwidths are useful against cross-luminance artifacts, the wider bandwidths allow higher chroma contents.

In SECAM, 1.3 MHz low-pass and pre-emphasis filtering are performed on Dr and Db chroma components, before the frequency modulation, according to ITU-R Rec624-4.

Refer to [Figure 135](#) for frequency response of these filters. Bell filtering is performed at the end of frequency modulation stage.

Peak to peak amplitude of modulated chrominance signal at the central frequency (4 279.7 kHz) is 22,88% of the black-white interval (22.88 IRE).

Refer to [Figure 136](#) for frequency response of bell filter with sub-carrier frequencies and clipping values.

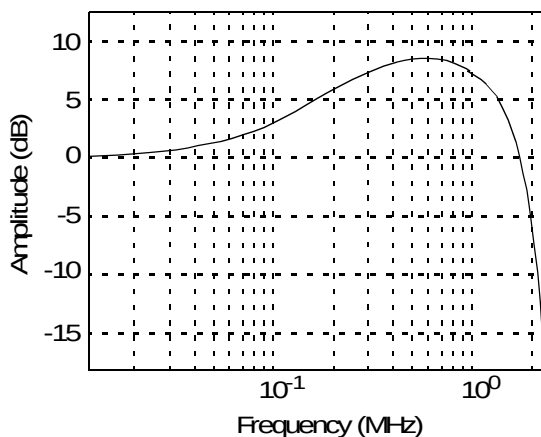


Figure 135 SECAM chroma filtering (pre-emphasis and 1.3 MHz low pass filtering)

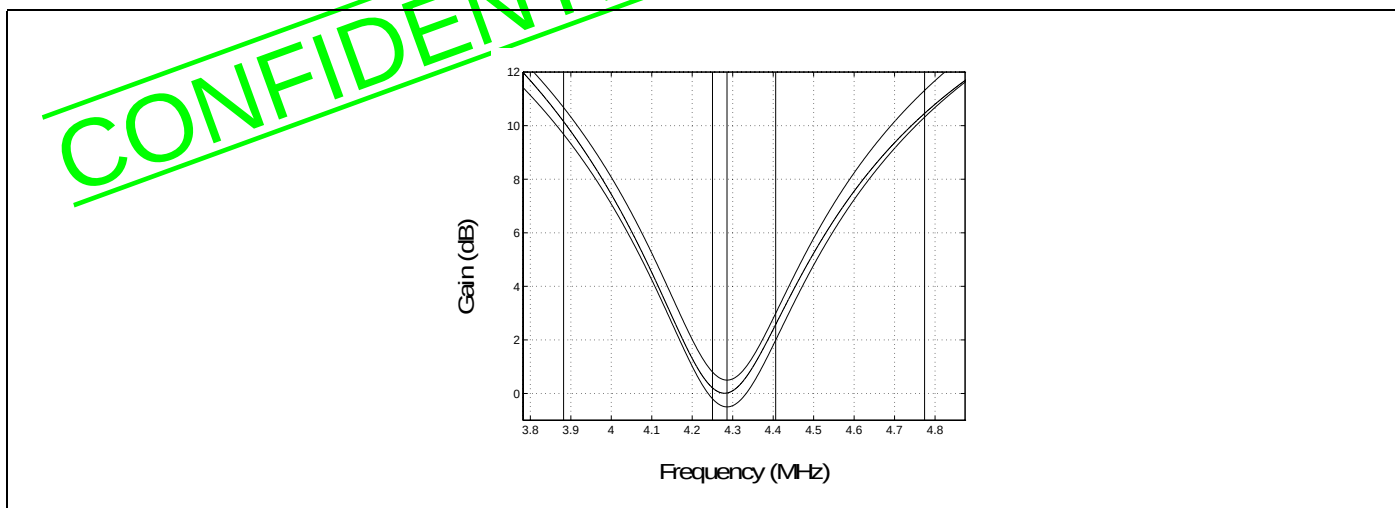


Figure 136 SECAM high-frequency subcarrier pre-emphasis (Bell filtering), including DAC attenuation

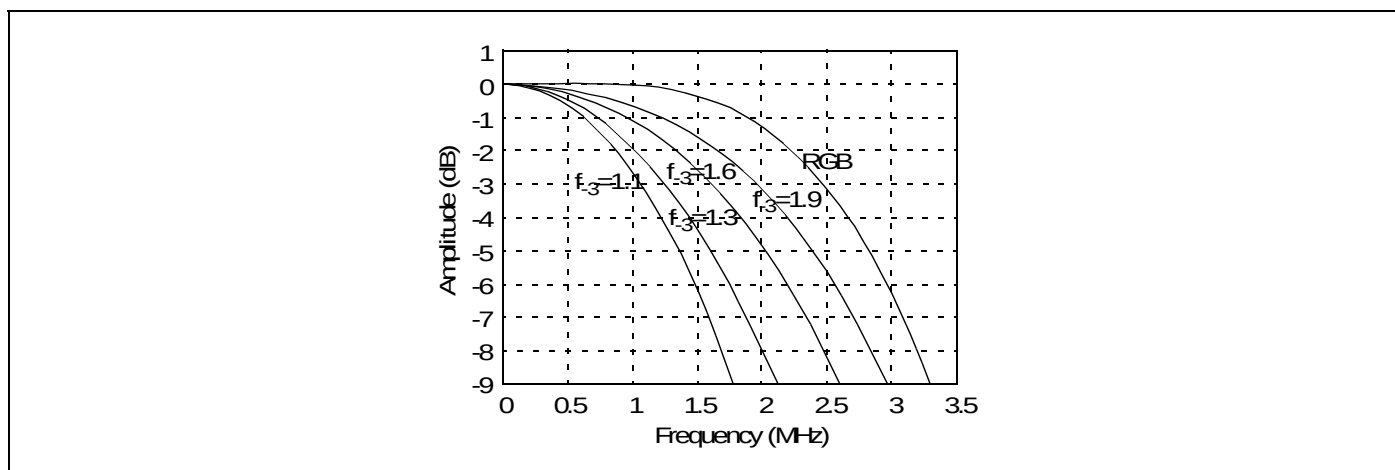


Figure 137 Various chroma filters available and RGB filter

24.11 Composite video signal generation

The composite video signal is created by adding the luminance (after trap filtering - optional in PAL and NTSC, see register **DEN_CFG3**) and the chrominance components. A saturation function is included in the adder to avoid overflow errors should extreme luminance levels be modulated with highly saturated colors. This does not occur with natural colors but may be generated by computers or graphics engines.

A “color killing” function is available, whereby the composite signal contains no chrominance, i.e. replicates the trap-filtered luminance. This function does not suppress the chrominance on the S/VHS outputs, but suppressing the S-VHS chrominance is possible using bit `bkdac $n$` in `DEN_CFG5`, where the chrominance signal is outputted on DAC n .

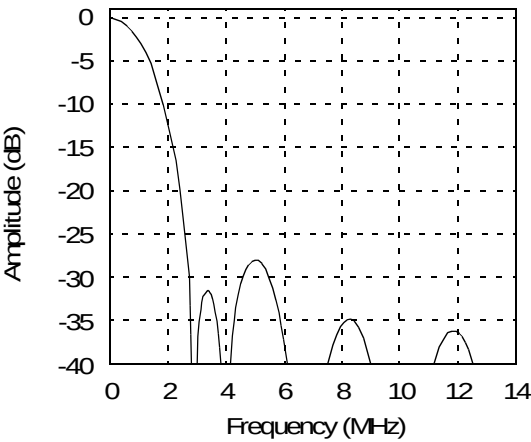


Figure 138 1.1 MHz chroma filter

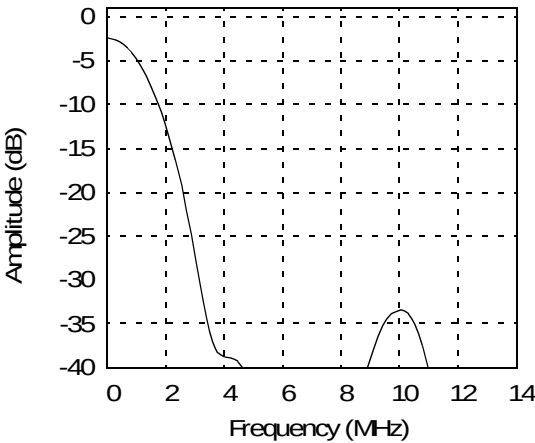


Figure 139 1.3 MHz chroma filter

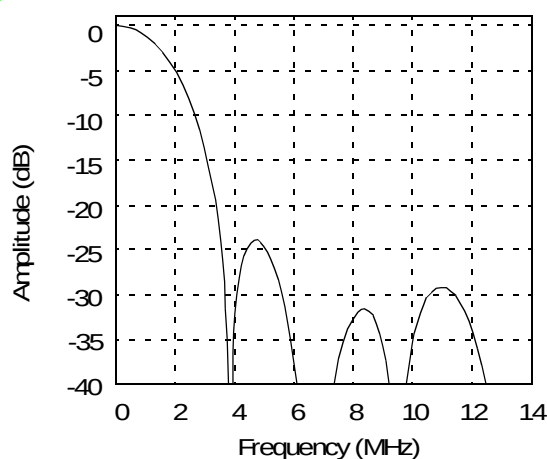
CONFIDENTIAL

Figure 140 1.6 MHz chroma filter

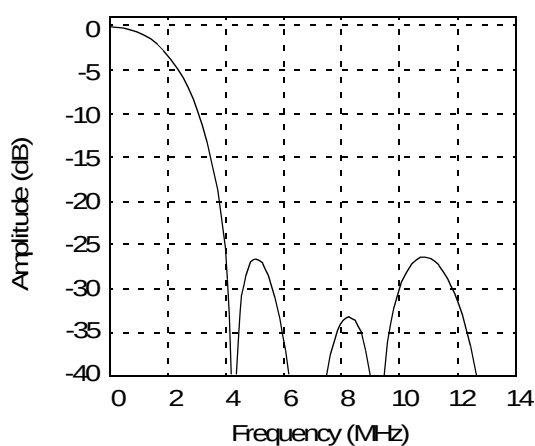


Figure 141 1.9 MHz chroma filter

24.12 RGB and UV encoding

After demultiplexing, the Cr and Cb samples feed a 4 times interpolation filter. The resulting base-band chroma signal has a 2.45 MHz bandwidth ([Figure 142](#)) and is combined with the filtered luma component to generate R,G,B or U,V samples at 27 MHz.

If Y4 and CrCb inputs are used, the filtering identical to luma filtering (see Figure 132) is performed on all components (Y4, Cr and Cb). In this case DAC5 output data encoded from Y4 input if YUV configuration is used (see DEN_CFG8 bits **conf_out1** and **conf_out0**).

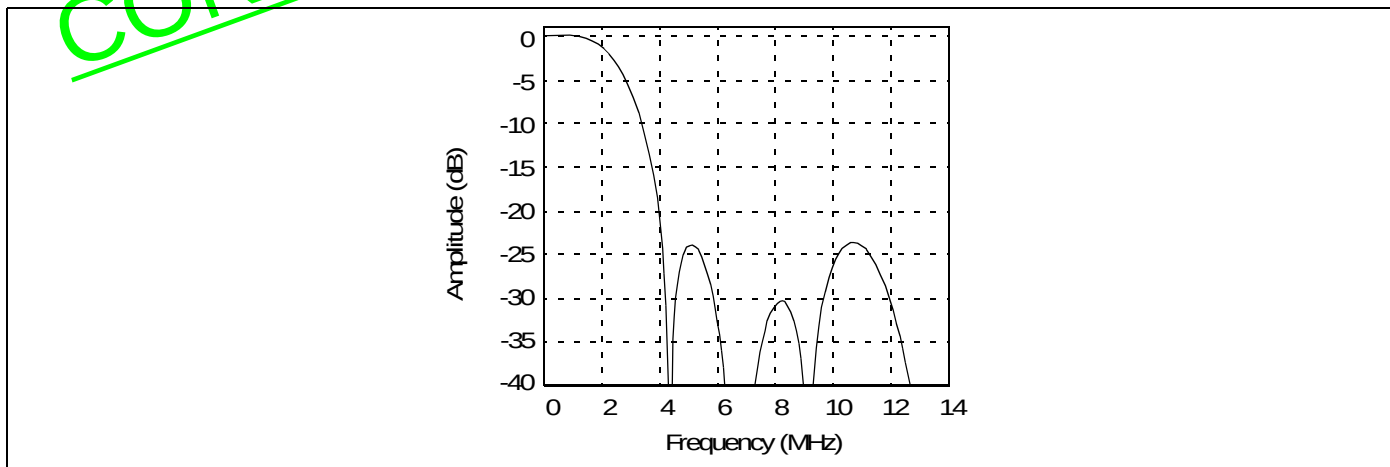


Figure 142 RGB - chroma filtering

24.13 Closed captioning

Closed-captions (or data from an Extended Data Service as defined by the Closed-Captions specification) can be encoded by the circuit. The closed caption data is delivered to the circuit through the register interface. Two dedicated pairs of bytes (two bytes per field), each pair preceded by a clock run-in and a start bit can be encoded and inserted on the luminance path on a selected TV line. The Clock Run-In and Start code are generated by the DENC.

Closed-caption data registers are double-buffered so that loading can be performed anytime, even during line 21/284 or any other selected line.

User register **DEN_CCF1** and **DEN_CCF2** each contain the first and second byte to send (LSB first) after the start bit on the appropriate TV line, where **DEN_CCF1** refers to field 1 and **DEN_CCF2** to field 2. The TV line number where data is to be encoded is programmable using registers **DEN_CLF1** and **DEN_CLF2**. Lines that may be selected include those used by the StarSight data broadcast system. Closed-caption data has priority over any CGMS signals programmed for the same line.

The internal Clock Run-In generator is based on a Direct Digital Frequency Synthesizer. The nominal instantaneous data rate is 503.496 KHz (i.e. 32 times the NTSC line rate). Data LOW corresponds nominally to 0 IRE, data HIGH corresponds to 50 IRE at the DAC outputs.

When closed-captioning is on (bits **cc1** and **cc2** in **DEN_CFG1**), the CPU should load the relevant registers (**DEN_CCF1** or **DEN_CCF2**) once every frame at most (although there is in fact some margin due to the double-

buffering). Two bits are set in the **DEN_STA** register in case of attempts to load the closed-caption data registers too frequently; these can be used to regulate the loading rate.

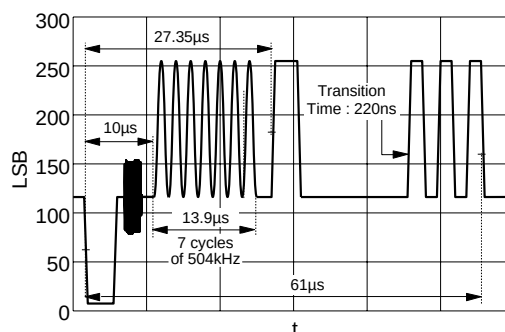


Figure 143 Example of closed-caption waveform

The closed caption encoder considers that closed caption data has been loaded and is valid on completion of the write operation into **DEN_CCF1** for field1, or **DEN_CCF2** for field 2. If closed caption encoding has been enabled and no new data bytes have been written into the closed caption data registers when the closed caption window starts on the appropriate TV line, then the circuit outputs two US-ASCII NULL characters with odd parity after the start bit.

24.14 CGMS encoding

CGMS stands for *Copy Generation Management System*, and is also known as VBI-D and described by standard *CPX-1204* of *EIAJ*. CGMS data can be encoded by the digital encoder.

Three bytes, containing 20 significant bits, are delivered to the chip via the register interface. Two reference bits (1 then 0) are encoded first, followed by 20 bits of CGMS data. This includes a Cyclic Redundancy Check sequence, which is not computed by the device but is supplied to it as part of the 20 data bits. The reference bits are generated locally by the DENC. Figure 144 shows a typical CGMS waveform.

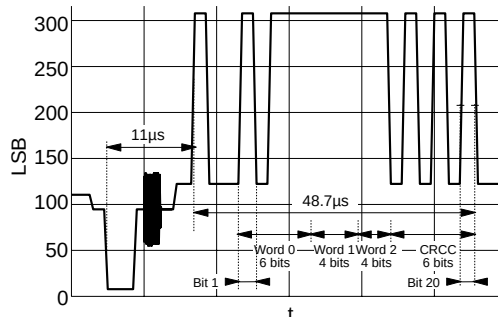


Figure 144 Example of CGMS waveform

CGMS encoding is enabled by setting bit **encgms** in register **DEN_CFG3**. When enabled, the CGMS waveform is present once in each field, on lines 20 and 283 (SMPTE-525 line numbering).

The CGMS data register is double-buffered, which means that it can be loaded at any time (even during line 20/283) without any risk of corrupting CGMS data that could be in the process of being encoded. The CGMS encoder considers that new CGMS data has been loaded and is valid on completion of the write operation into register **DEN_CGMS**.

24.15 WSS encoding

The digital encoder allows WSS (Wide Screen Signalling) in 625-line format, complying with the ETS 300 294 standard. Two bytes are delivered to the circuit through the register interface into two dedicated registers (see register **DEN_WSS**).

WSS encoding is enabled using bit **enwss** in register **DEN_CFG3**. When WSS encoding is enabled, a waveform is present on the first half of line 23 of each frame. Data is preceded by a run-in sequence and a start code generated locally by the DENC.

24.16 VPS encoding

VPS data encoding is defined by *ETS 300 231 communication, June 1993*. VPS data can be encoded by the DENC on line 16 (CCIR) for 625-line PAL and SECAM television systems. The VPS data is delivered to the circuit using registers **DEN_VPS**. The data transmission is preceded by a clock run-in and a start code generated by the DENC. The clock frequency is 5MHz. This feature is enabled by setting the **envps** bit of register **DEN_CFG7**. [Figure 145](#) shows an example of VPS waveform.

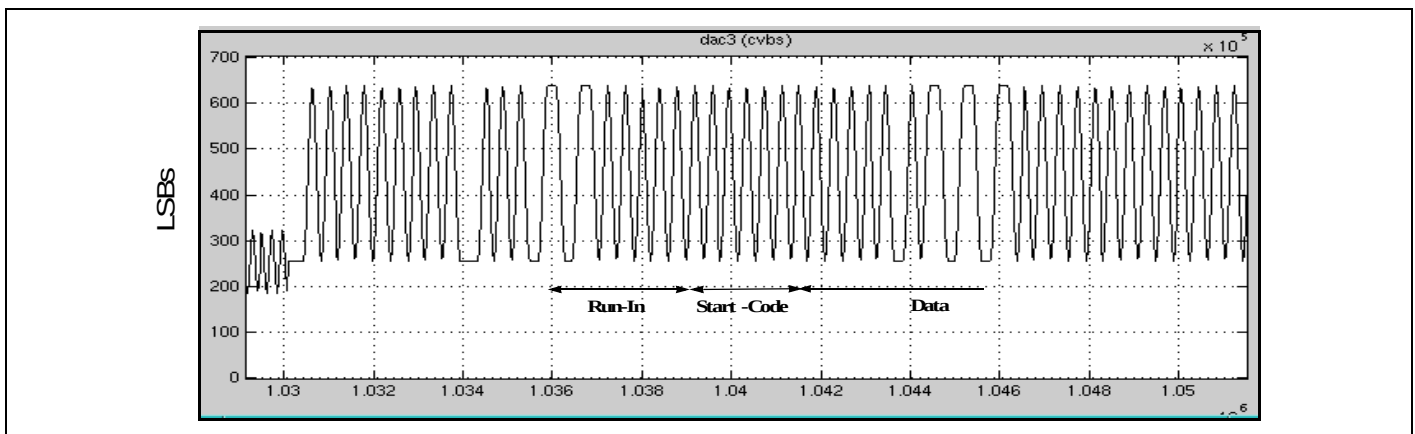


Figure 145 Example of VPS waveform

24.17 Teletext encoding

The DENC can encode Teletext according to the "CCIR/ITU-R Broadcast Teletext System B" specification (also known as World System Teletext), and NABTS (North American Basic Teletext Specification) EIA-516.

In DVB applications, Teletext data is embedded within DVB streams as MPEG data packets. The Transport Layer Processing IC (ST20) sorts incoming data packets and stores Teletext packets in a buffer. It then passes them to the DENC on request.

Signal exchange

The DENC and the Teletext buffer exchange 2 signals: TTXS (Teletext Synchronization) going from the DENC to the Teletext Buffer and TTXD (Teletext Data) going from the Teletext Buffer to the DENC.

The TTXS signal is a request signal generated on selected lines. In response to this signal, the Teletext buffer is expected to send Teletext bits to the DENC for insertion of a Teletext line into the analog video signal. The number of Teletext bits sent, depends on the Teletext system being used (selected by register bit **DEN_REG_64.txt_abcd**) 360 bits are sent for Teletext B - WST in PAL, or 288 for Teletext C - NABTS in NTSC.

The duration of the TTXS window corresponds to the number of bits being sent (see Transmission Protocol below).

- For Teletext B and 625 line systems, the TTXS window duration is 1402 reference clock periods (corresponding to 360 bits).
- For Teletext C and 525 line systems (NABTS), this duration is 1121 master clock periods.

Following the TTXS rising edge, the encoder expects data from the Teletext buffer after a programmable number (2 to 9) of 27MHz master clock periods. Data is transmitted synchronously with the master clock at an average rate of 6.9375 Mbit/s according to the protocol described below. In order of transmission, it consists of: 16 Clock Run-In bits, 8 Framing Code bits and one Teletext packet of 336 or 228 bits (depending on the Teletext system being used). If more than one packet of bits (336 or 228) are transmitted, they are ignored by the DENC. By default, register bit **DEN_REG_65.ttx_mask_off** masks the two bits of Teletext framing code, allowing the code to be set by the DENC according to the selected Teletext standard.

Transmission protocol

In order to transmit the Teletext data bits at an average rate of 6.9375 Mbit/s, which is about 1 / 3.89 times the master clock frequency, the following scheme is adopted:

The 360-bit packet is regarded as nine 37-bit sequences plus one 27-bit sequence. In every sequence, each Teletext data bit is transmitted as a succession of *four* identical samples at 27 Msample/s, except for the *10th*, *19th*, *28th* and *37th* bits of the sequence which are transmitted as a succession of *three* identical samples.

Programming “TTXS rising” to “first valid sample”

The encoder expects the Teletext buffer to clock-out the first Teletext data sample on the $(2+N)^{\text{th}}$ rising edge of the master clock following the rising edge of TTXS. Figure 146 depicts this graphically for $N=0$.

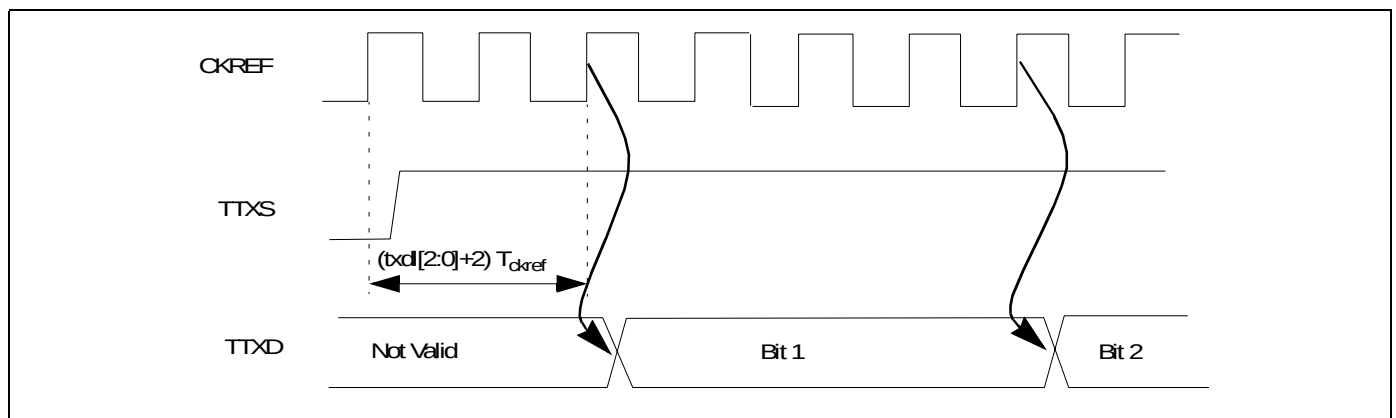


Figure 146 TTXS Rising to First Valid Sample delay for $txdl[2:0] = 0$

N is programmable from 0 to 7 by register bits **DEN_TTX1.ttxdel[2:0]**. The value written in **txdl[2:0]** is 2 less than the overall delay in 27MHz cycles, so a value of 0 for **txdl[2:0]** corresponds to an overall delay of 2 cycles, and a value of 7 corresponds to a delay of 9 cycles.

Programming Teletext line selection

Five dedicated registers program Teletext encoding in various areas of the Vertical Blanking Interval (VBI) of each field. 4 areas (i.e. blocks of contiguous Teletext lines) can *independently* be defined within the two VBIs of one frame (e.g. 2 blocks in each VBI, or 3 blocks in field1 VBI and one in field2 VBI, etc.). Under certain circumstances, it is possible to define up to 4 areas in each VBI.

Teletext block definition registers **DEN_TTX1-4** and a teletext block mapping register **DEN_TTXM** are used for programming. Refer to the STi5512 Register Manual for details.

Full-page teletext encoding is set by register bit **DEN_TTX1.fp_ttxt**. Teletext is encoded on lines 7 to 311 and 320 to 623 (ITU-R line numbering). If bit **fp_ttxt_all** is set, teletext encoding is also enabled on lines 6, 318 and 319. When full page teletext is performed, no video data is encoded (YCrCb, Y4 and CrCb input streams are ignored).

Programming Teletext pulse shape

The shape and amplitude of a single Teletext pulse is shown in Figure 147 . Its relative power spectral density is shown in Figure 148 and Figure 149 It is zero at frequencies above 5 MHz, as required by the World System Teletext specification.

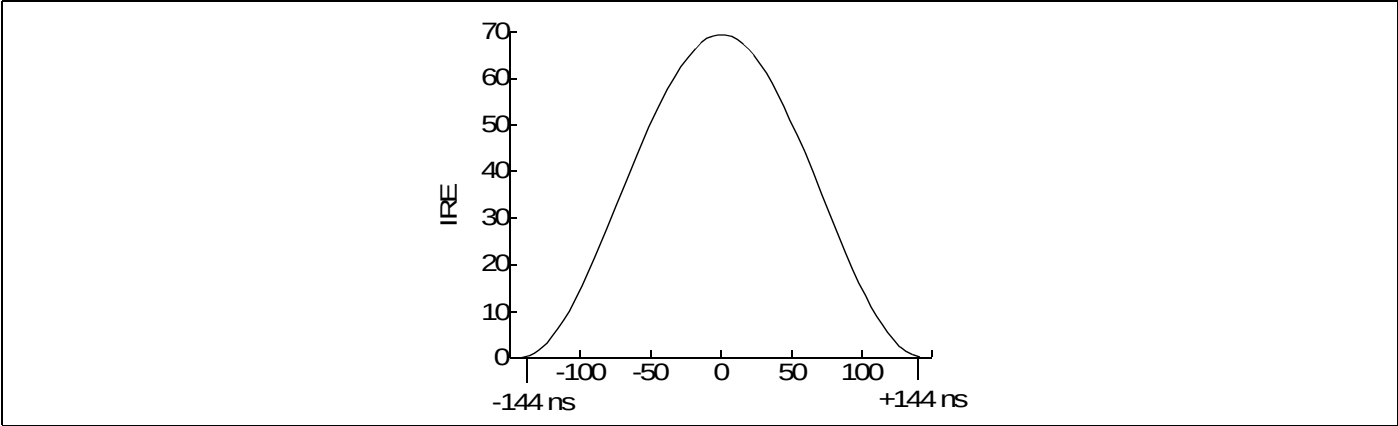


Figure 147 Shape and amplitude of a single teletext symbol

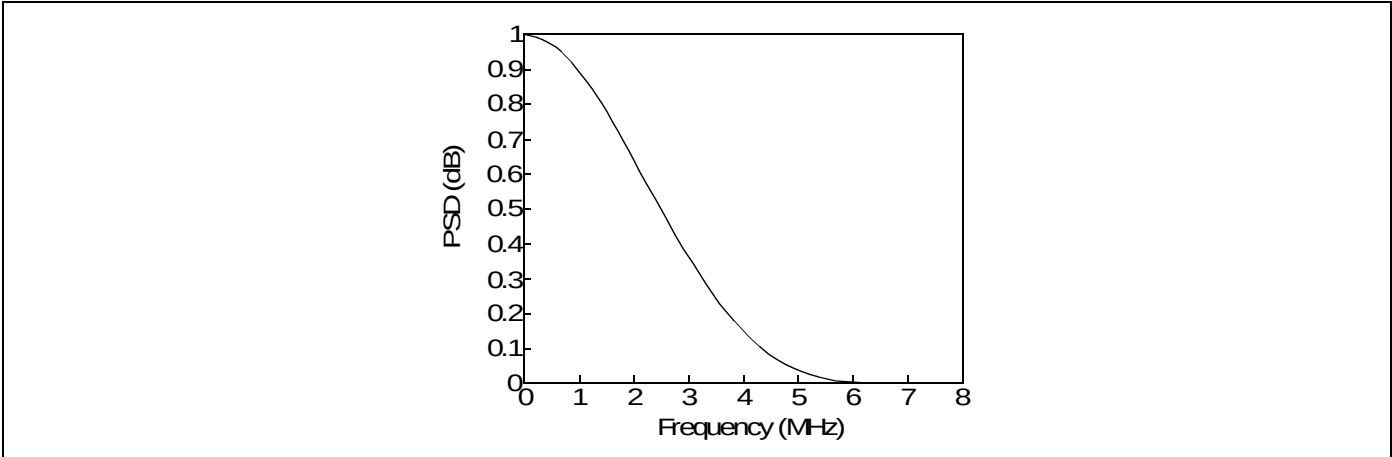


Figure 148 Linear PSD scale

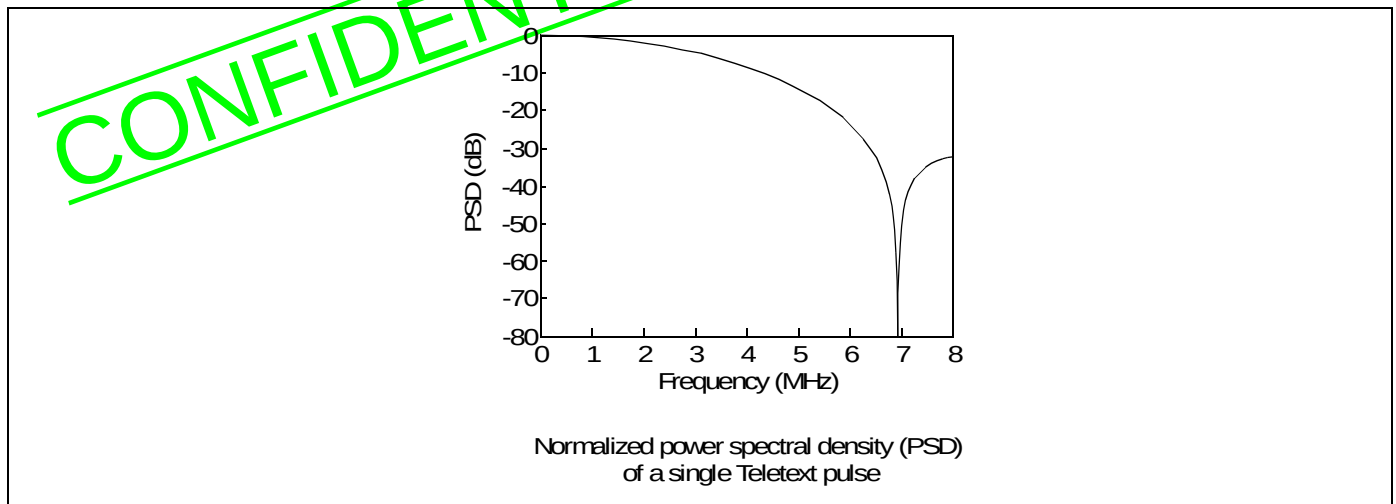


Figure 149 Logarithmic PSD scale

24.18 Line skip and line insert capability

This patented feature of the DENC offers the possibility to cut the cost of the application by suppressing the need for a VCXO.

Ideally, the master clock used on the application board and fed to the MPEG decoding IC would have exactly same frequency as the clock that was used when the MPEG data was encoded. Obviously this is not realistic; up to now a solution commonly used was to dynamically adjust the clock on the board as close to the “ideal” clock as possible with the help of time stamps embedded within the MPEG stream. Such a kind of tracking often involves the use of a VCXO: when the MPEG data buffer fills up to more than some threshold the clock frequency is increased, when it empties down to some other threshold the clock frequency is lowered.

The DENC offers an alternative, cost-saving solution: by programming two bits in register **DEN_CFG6**, the DENC is able to reduce or increase the length of some frames in a way that will not introduce visible artifacts (even if comb-filtering is used). These bits should be set according to the level of the MPEG data buffer.

Operation with the DENC as sync master is as follows:

- If the MPEG data buffers fills up too much, set bit **jump** to 1 and bit **dec_ninc** to 1. The DENC will reduce the length of the current frame. Bit **jump** will then automatically reset to 0.
- If the MPEG data buffers empties too much, set bit **jump** to 1 and bit **dec_ninc** to 0. The DENC will increase the length of the current frame. Bit **jump** will then automatically reset to 0.

These operations can be repeated until the MPEG data buffer is inside its fixed limits.

It is also possible to use the line skip/repeat capability in non-interlaced mode.

Line skip and line insert can be used in slave mode; the sync signals supplied to the DENC must be in accordance with the programmed frame lengths.

24.19 CVBS, S-VHS, RGB and UV outputs

Six out of eight video signals can be directed to six analog output pins through a 10-bit D/A converters operating at the reference clock frequency. The available combinations are:

S-VHS (Y/C) + CVBS + RGB, or S-VHS (Y/C) + CVBS + U + Y2 + V, or Y1 + C1 + CVBS1 + C2 + Y2 + CVBS2.

These combinations are controlled by bits **conf_out1** and **conf_out0** in register **DEN_CFG8**, as shown in the table below;

conf_out1	conf_out0	dac1	dac2	dac3	dac4	dac5	dac6	Notes
0	0	Y	C	CVBS	C	Y	CVBS	
0	1	Y	C	CVBS	V	Y	U	
1	x	Y	C	CVBS	R	G	B	Default

Table 110 Encoding of conf_out

The C to Y peak to peak amplitude ratio can be modified in both CVBS and VHS (Y/C) outputs (see **mult_rgb_c**).

Default peak to peak amplitude of UV and RGB outputs is set to 70% of Y or CVBS peak to peak amplitude, for 100/0/100/0 color bar pattern, and can be modified using the multiplying factors in registers **DEN_DAC45** and **DEN_DAC6C**.

If **DEN_CFG7[2]** bit **uv_lev** is 0 (default value) U and V outputs have 0.7V peak to peak amplitude if 100/0/100/0 color bar pattern is inputted. If this bit is "1" U and V outputs are those defined by ITU-R 624-4 for PAL and NTSC standards ($V_{pp}/U_{pp} = 1.4$). In that case U peak to peak amplitude is 0.61V (0.57V if **DEN_CFG7** bit **setupYUV** is set) and V peak to peak amplitude is 0.86 V (0.80 V if register **DEN_CFG7** bit **setupYUV** is set). In all these cases UV outputs can be multiplied by 0.75 to 1.22 factor according to register bits **DEN_DAC45.dac4_mult** and **DEN_DAC6C.dac6_mult**.

A single external analog power supply pair is used for all DACs, but two independent pairs of current and voltage references are needed. An 11K resistor must be connected between each **I_REF** and **V_REF** pins.

The internal current sources are independent from the positive supply, and the consumption of the DACs is constant whatever the codes converted.

Any unused DAC may be independently disabled by software, in which case its output is at "neutral" level (blanking for luma and composite outputs, no color for chroma output, black for RGB and UV outputs). For applications where a single CVBS output is required, the RGB/CVBS+S-VHS/UV Triple DAC should be disabled, pin **I_REF_DAC_RGB** should be tied to analog power supply and pin **V_REF_DAC_RGB** should be left unconnected.

Due to the 3.3 V power supply used, the output swing of the DACs is about 1Vp-p. Therefore some external gain may be required, which, combined with the recommended output filtering stage, means active filtering. For this active filtering stage to be very simple, it is possible to "invert" the DAC outputs by programming a bit of **DEN_CFG5**. Code "N" becomes code "1024-N", i.e. the resulting waveform undergoes a symmetry around the mid-swing code.

24.20 DENC registers

The control registers are in a block in the peripheral space in the address map. The addresses of the registers are given in this chapter as offsets from the base of this block. The base of the block is named **DENCBaseAddress**, and its value is given in the *STi5512 Register Manual*.

CONFIDENTIAL

25 Triple video DACs

25.1 Description

There are two on-chip 3X10-bit Digital to Analog converters (triple DACs) used for video output. One provides output signals in CVBS, Y, C, and the other in RGB. The figure below shows the DAC schematic.

An external reference resistor is associated with the bandgap voltage to generate a reference current. This resistor is connected between the V_REF pin of the bandgap and a dedicated I_REF pin (equivalent to ground, but distinct from analog ground) to achieve a higher noise immunity.

The global segmented architecture is presented in the figure below. Current-sources provide an output range of 1.5V with good linearity. Sampled data are available on video outputs after 1 clock period (on the next rising clock edge).

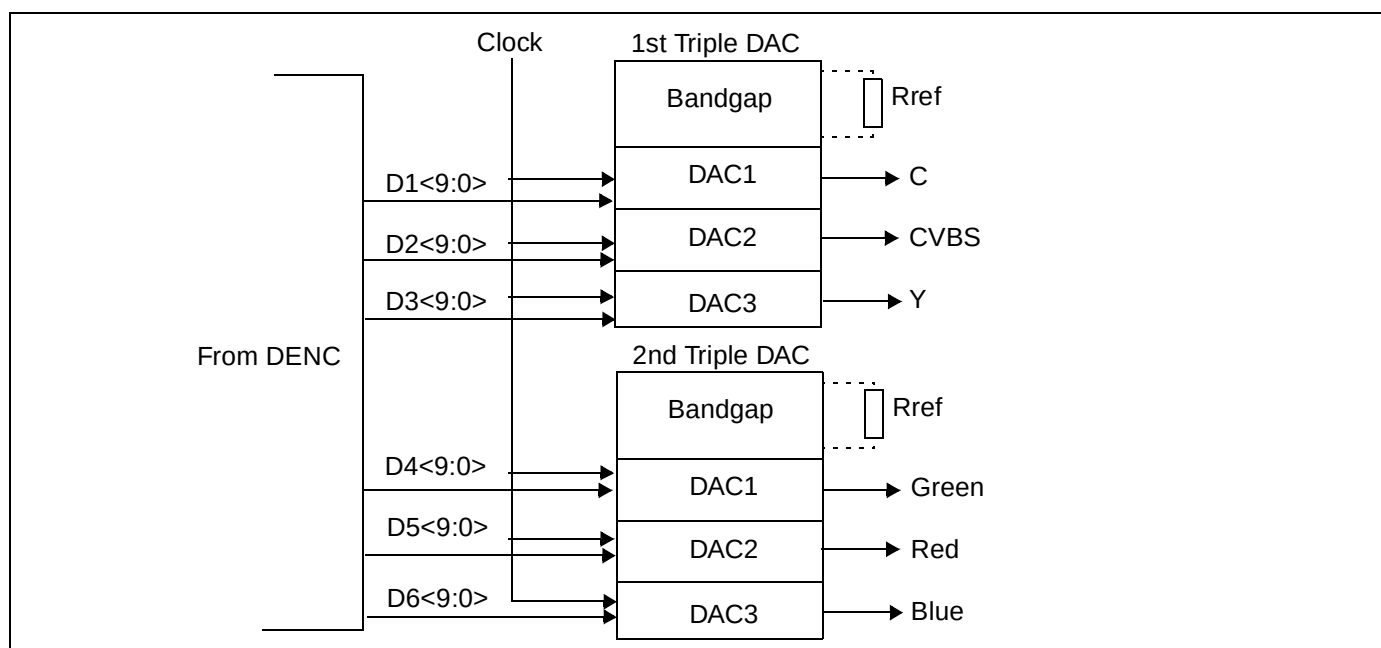


Figure 150 Triple video DAC schematic

The triple video DAC has its own 3.3V power and ground supplies for noise reduction. To guarantee good frequency response at high frequencies, these power and ground supplies are not connected to digital power supplies inside the chip.

25.2 Input codes for video application

The table below lists the reference input codes generated by the DENC depending on the configuration for the DACS and the standard.

	Y-PAL/SECAM	Y-NTSC	RGB
WHITE(235)	816.00	802.00	602.00
BLACK(16)	256.00	240.00	41.00
SYNC TIP	16.00	16.00	n.a.

Table 111 Reference input codes

Note that CVBS = Y+C, so chrominance component has no effect on CVBS signal (C is null)

25.3 Video output voltage level

The resistor Rref connected to the bandgap has a direct effect on the output current which flows from the DAC's outputs. For the maximum code (1023 in decimal):

$$I_{out} (max) = (1.25 / R_{ref}) * 49$$

For example, with a typical value of Rref = 11kohms, Iout (max) = 5.57mA for each DAC.

The value of Rref must be carefully chosen: **Iout should always be lower than 10mA**, otherwise, DAC linearity is not guaranteed.

Because of the sensitive relationship between the DAC and Rref, the tolerance on the Rref value must be small. Typically, the Rref resistor must be a 1% resistor.

The output voltage on the RGB output pins depends on the external load resistor Rload (connected between the DAC output and ground):

$$V_{out} (max) = R_{load} * I_{out} (max)$$

For example, with a typical load value Rload = 200ohms and Iout(max) = 5.57mA, Vout (max) = 1.11V.

Because of the previous constraint on Iout, Vout will always be lower than 1.5V.

For any given digital input code, the output voltage of the DAC will be given by the following formula:

$$V_{out} = D_{in} / 1023 * V_{out} (max) = (D_{in} * R_{load} * 49 * 1.25) / (R_{ref} * 1023)$$

$$V_{out} = D_{in} * R_{load} * 0.06 / R_{ref}$$

For example, with Rload = 200ohms, Rref = 11kohms and Din = 526, Vout = 0.57V

25.4 Video specifications and DAC setup

In Y-PAL/SECAM, the output video range between code 16 (synchronization level) and code 816 (white level) should be: $V_{out}(816) - V_{out}(16) = 1V$. The output video range between code 256 (black level) and code 816 (white level) should be 700mV. This must be respected for all applications.

The value of the Rref resistor must be chosen according to the value of Rload and the previous formula, to achieve the standard output video range. The minimum value for Rref is 6.13kohms.

According to video specifications ITU-R BT 601, the nominal sampling frequency is 27MHz. The clock for the DACs is the same as the one for the DENC but buffered. This clock is usually generated externally by a VCXO. It must be as clean as possible to achieve a good signal-to-noise ratio.

25.5 Output-stage adaptation and amplification

The purpose of the output filter depends on the application and the targeted ratio of price/performance. The output filter can connect directly to a scart connector or to other components, in which case the characteristics of the output signal may be different (level, output impedance).

The amplifier gain must be in accordance with the one of the tri-DACs (defined by Rload and Rref). If the amplifier gain cannot be set to the standard output video range by Rref and Rload, it can be tuned. In common applications (with Rload=200ohms), a video amplifier gain of 2 should be adequate. The ideal input impedance of the filter should be greater than Rload.

The tri-dacs have no cut-off frequency, therefore, a low-pass filter (around 6MHz) must be applied to remove harmonics (of mainly 27MHz). If additional attenuation is applied by the filter due to imperfection of the amplifier (generally degrading the C/L ratio), correction must be applied to preserve a good performance. Also, to guarantee a good frequency behavior at high frequency, the analog power supply must be separate from the digital power supply. If this is not the case, an additional correction may be required.

26 MPEG audio decoder with AC-3 interface

The audio decoder receives compressed data from an audio bit-buffer which is integrated in the external SDRAM. When the external AC-3 interface is used, the compressed/PCM data also goes through the audio bit-buffer. The audio bit-buffer is memory mapped into the register/compressed data address space in the same way as the video and sub-picture decoders. Data is transferred either using a compressed data DMA engine or CPU writes. The audio decoder is completely autonomous, needing no interaction from software during decoding, apart from exceptions such as error conditions and ancillary data in the audio stream.

26.1 PCM output

26.1.1 Interface and output formats

The decoded audio data is output in serial PCM format. The interface consists of the following signals:

- PCMDATA - PCM serial data output
- SCLK - PCM clock output
- LRCLK - Left/right channel select output
- PCMCLK - PCM clock input.

Output precision is selectable to be either 16 bits/word or 18 bits/word by setting the output precision select register, AUD_P18. In 16-bit mode, data may be output either with the most significant bit first or least significant bit first, selected by the output order select register, AUD_ORD. When 18-bit data is selected, 32 bits are output for each channel. The data-in-front register, AUD_DIF, is used to position the 18 data bits either at the beginning or at the end of each 32-bit frame. The AUD_FOR register is used to select standard or I2S-compatible format when 18-bit precision is selected. The figure below shows the five different output formats which are possible. AUD_ORD only has significance in 16-bit mode, while AUD_DIF only has significance in 18-bit mode. AUD_FOR only has significance in 18-bit mode and when AUD_DIF = 1. The last option shown below is compatible with the I²S format.

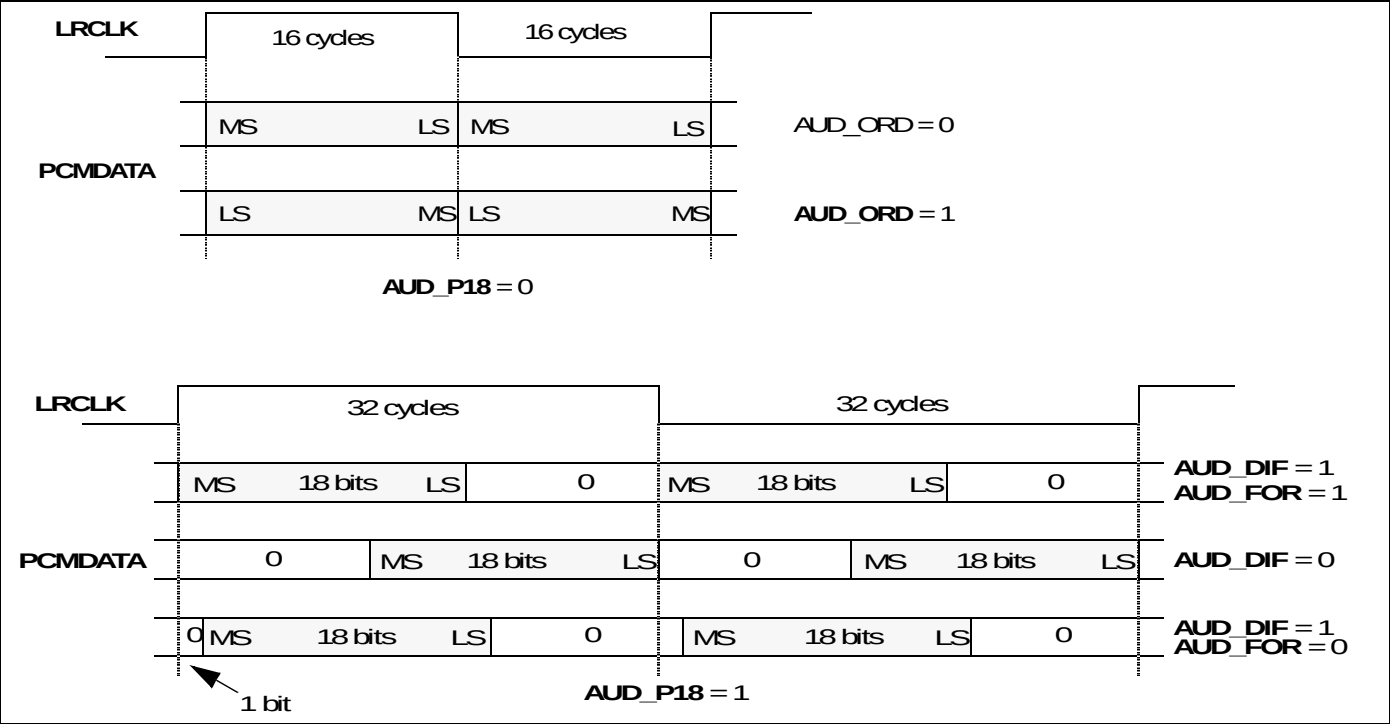


Figure 151 PCM output formats

The polarity of the PCM serial output clock, SCLK, and the left/right channel selection, LRCLK, are selected by the 1-bit registers **AUD_SCP** and **AUD_LRP** respectively.

Figure 152 shows the two polarities of SCLK. Normally, the DAC will sample LRCLK and PCMDATA on the rising edge of SCLK in the first case, and on the falling edge of SCLK in the second. The first option (**AUD_SCP** = 0) is the one normally used in I²S systems.

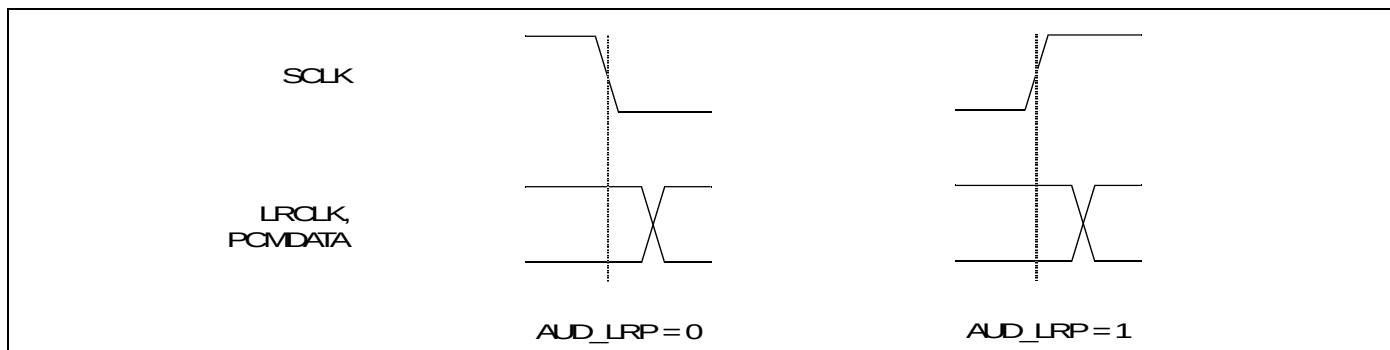


Figure 152 SCLK polarity

Figure 153 shows how the polarity of LRCLK is selected. The second option (**AUD_LRP** = 1) is compatible with the I²S format.

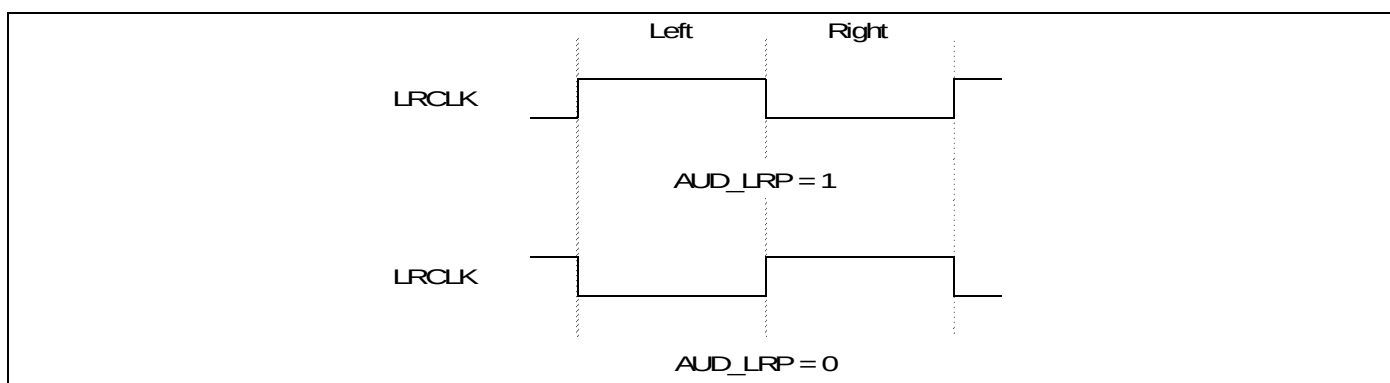


Figure 153 LRCLK polarity

26.1.2 Interrupts associated with PCM output

There are two interrupts associated with the PCM output. These are:

- Interrupt 8, indicating PCM buffer underflow. This is generated (unless masked) when a new output sample is required and the PCM buffer is empty. The PCM buffer, which contains up to 64 samples (i.e. 64 word-pairs in stereo), receives the decoded outputs from the DSP core. If the buffer is empty the output sample will have the value zero, but decoding will not stop. If the PCM buffer becomes full, decoding will stop, but PCM output will not be affected.
- Interrupt 14, indicating output of a new frame. This is generated (unless masked) whenever the first bit of a frame appears at the PCM output.

26.2 Audio decoder control

26.2.1 Play and mute

Once initialized and configured, decoding and output of PCM data is controlled by the commands PLAY and MUTE. The actions of the PLAY and MUTE commands are specified in [Table 154](#).

MUTE	PLAY	Function
de-asserted	de-asserted	No output or decoding. SCLOCK, LRCLK, PCMDATA all move into their inactive state. LRCLK completes its current cycle and stops, SCLK completes its last cycle in the second LRCLK frame and stops. Decoding stops when all internal buffers are full.
de-asserted	asserted	Normal decoding and PCM output. When PLAY is re-asserted, PCMDATA resumes where it left off, without data loss.
asserted	de-asserted	PCM clocks only, no decoding. PCMDATA becomes low after the output of the last complete sample. LRCLK and SCLK are not stopped. Decoding stops when all internal buffers are full. When PLAY is re-asserted, PCMDATA resumes where it left off, without data loss.
asserted	asserted	Decoding and muted output (soft mute). PCMDATA gradually decays to zero. Decoding continues normally. Data consumed as if output were playing.

Figure 154 Mute and play functions

26.2.2 Restart

The restart procedure is invoked when it is required to flush all buffers and restart decoding immediately. Restart is initiated by writing 0 or 1 to the **AUD_RST** register, after which it is automatically restored to the 0 state. A restart initiates the following actions:

- The **AUD_ITR** and **AUD_ITN** registers are cleared.
- All data buffers are cleared.
- The **AUD_MUT**, **AUD_PLY** and all others registers (except those mentioned above) remain in their existing state.
- Register access is not disabled. However, compressed data input may be interrupted
- The **AUD_RST** register is cleared; compressed data input can restart.

26.2.3 Bit stream synchronization

The compressed input bit stream must be synchronized before the decompression step may begin. This is done by looking for synchronization words inserted into the data stream at encoding. Synchronization must be done both at the audio frame and at the system packet layer if present.

At the packet level, the audio decoder will look for a valid start code, doing a bit by bit search. Once an audio packet is found, the decoder extracts the presentation time stamp (PTS) if present and starts the audio synchronization described below. At the audio frame level, there is a non-unique sync word at the beginning of the header. The STi5512 attempts to find this sync word by doing a bit by bit search. When found the action taken depends on the contents of the **AUD_LCK** and **AUD_LAT** registers.

26.2.4 Packet level synchronization

The complete algorithm is given in [Figure 155 on page 226](#).

To help the synchronization algorithm ignore an emulated packet synchronization word, it is possible to extend the packet start code to be matched. Depending to the content of registers **AUD_SYN**, **AUD_SID** and **AUD_IDE**, synchronization can be made on the 24-bit *packet_start_code_prefix* or can be extended to the *stream_id* field.

Synchronization mode depends on the type of packets received by the STi5512. The decoder can receive either:

- Multiplexed audio/video bitstream (AUD_SYN = 0)

In this case the STi5512 can receive both video and audio streams multiplexed together. Packet synchronization is possible only on the 24-bit start code.

All packets are used by the synchronization algorithm but all non-audio packets and, if AUD_IDE is set, all audio packets which have a stream_id which does not match the AUD_SID register value, are not decoded.

- Multiplexed audio bitstream (AUD_SYN = 1)

In this case, the STi5512 expects to receive only multiplexed audio streams. Synchronization is performed on 27 bits (24 bits packet_start_code_prefix + 3 first bits of stream_id).

All packets are used by the synchronization algorithm but if AUD_IDE is set, all audio packets that have a stream_id which does not match the AUD_SID register value are not decoded.

- Single audio bitstream (AUD_SYN = 2, AUD_IDE = 1)

Synchronization is performed on 32 bits.

All packets are used by the synchronization algorithm, and all audio packets that have a stream_id which matches the AUD_SID register value are decoded.

The AUD_SCN register is also taken into account in the global synchronization algorithm.

If AUD_SCN = 1, after the first packet synchronization word is found the STi5512 is considered to be synchronized. If AUD_SCN = 0, after the first packet synchronization word is found, the STi5512 must read the packet length and confirm synchronization by finding the next synchronization word in the correct position.

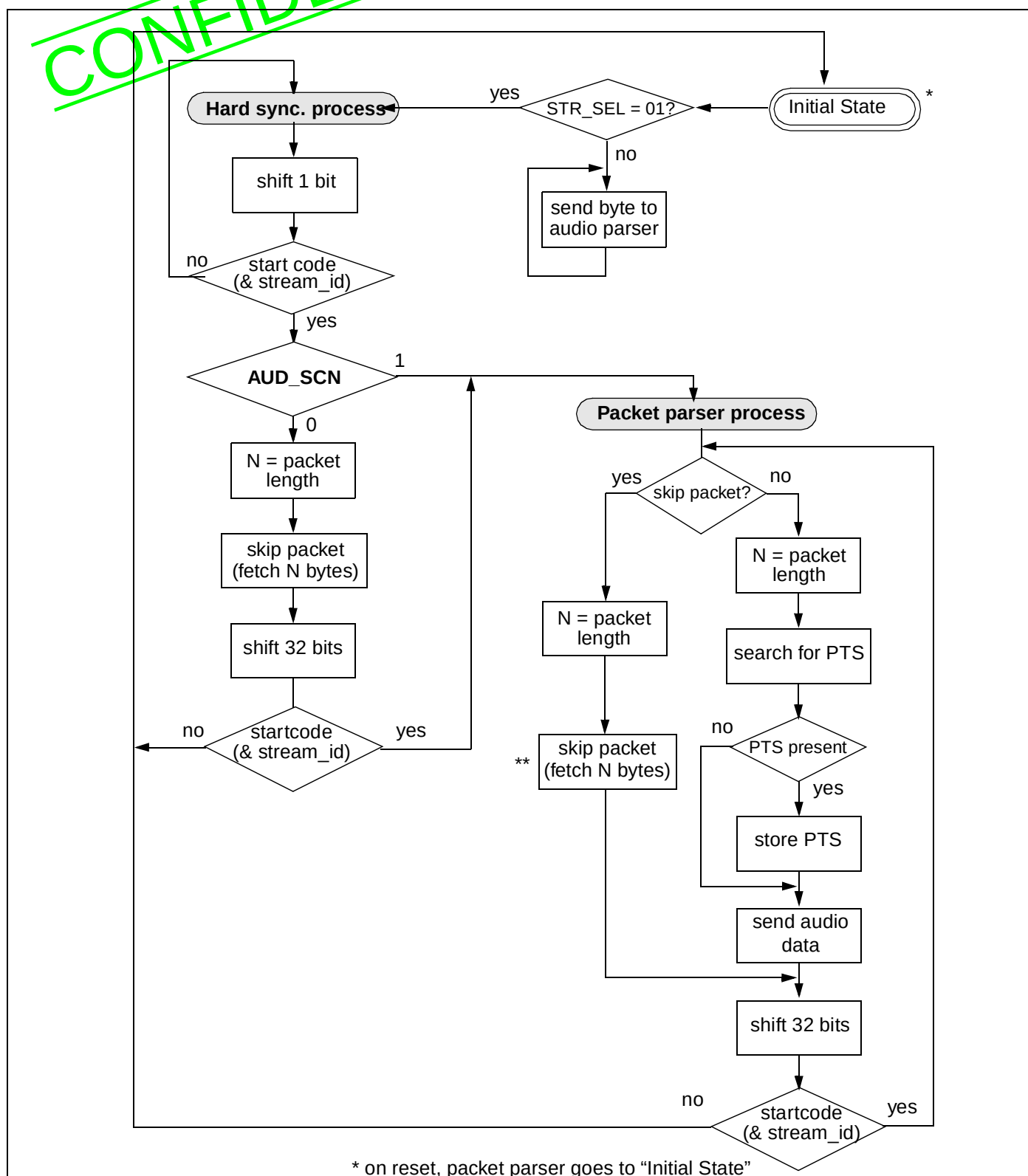


Figure 155 Packet synchronization algorithm

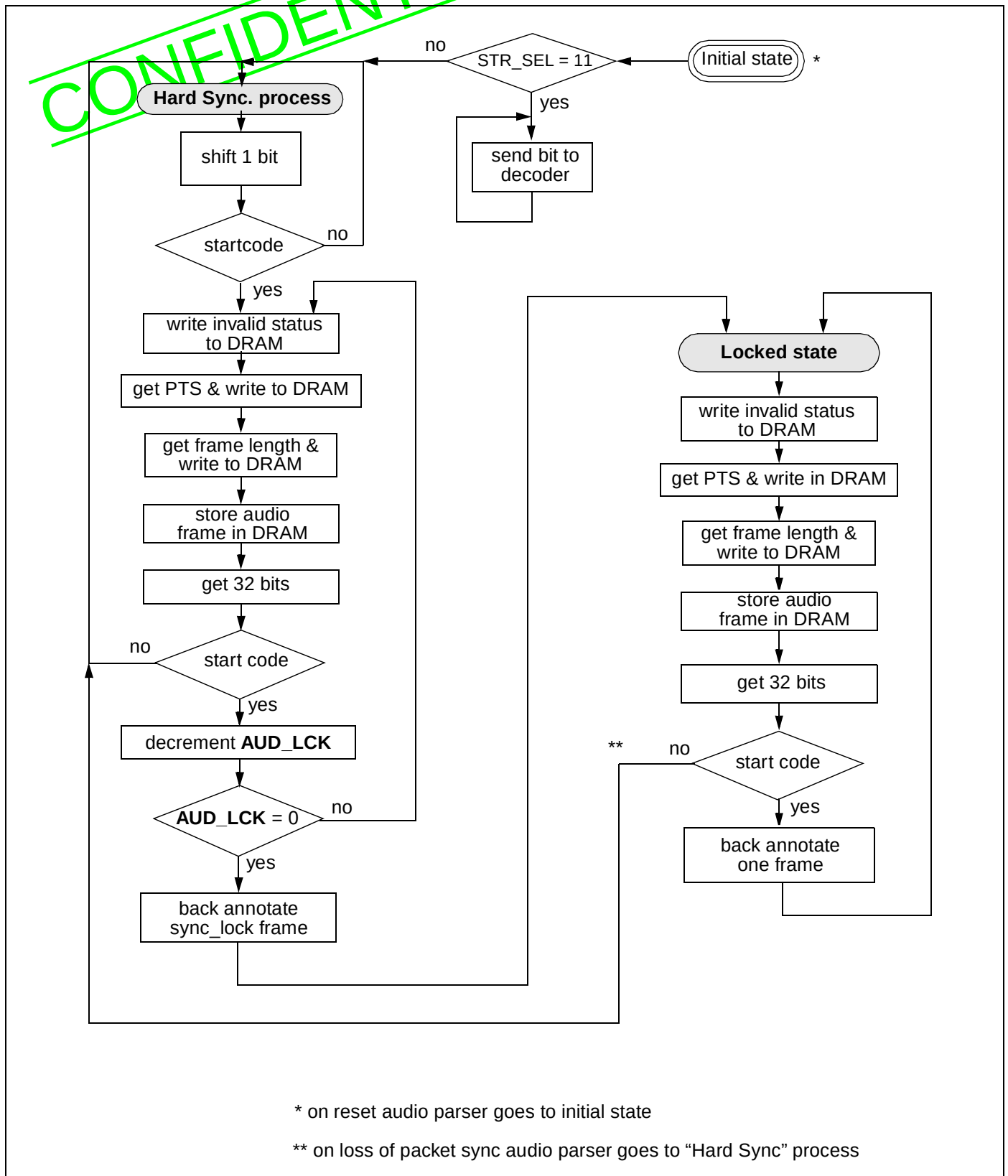


Figure 156 Audio frame synchronization algorithm

26.2.5 Audio frame synchronization

The synchronization algorithm is given in [Figure 156 on page 227](#).

As the audio *syncword* can be emulated in the bit stream, it is useful to extend this audio start code to avoid the detection of a false sync word. Each time the STi5512 detects a false sync word during the synchronization process, the delay to reach the locked state increases.

The AUD_SYE register is used for this purpose. When no field of the AUD_SYE register is enabled, the STi5512 saves the layer and sampling frequency information after synchronization is achieved. This aids the task of resynchronization, should synchronization be lost owing to an error in the audio data or the system layer. This internal register is disabled on AUD_RST or RESTART and will not be reinitialized until the audio parser is synchronized.

The AUD_LCK register specifies how many valid synchronization words after the initial one have to be found before entering the locked state. The highest value of AUD_LCK (i.e. 3) is assumed when the AUD_SYE register has its default value. The definition of a valid synchronization word depends on the AUD_LAT register value.

A valid synchronization word is a sequence of bits matching the expected word.

In free-format mode one additional register (AUD_FFL) can be used. The AUD_FFL register is a way of specifying the length of an audio frame in free-format mode. This register is 16 bits long and contains the length of the frame in bits.

26.2.6 Error recovery and concealment

The STi5512 audio decoder is able to recover from certain detectable errors. For this purpose it has a number of user-selectable error concealment modes.

Detectable errors may be caused by a bad audio frame CRC or by loss of synchronization. Concealment is similar, but may be selected independently by setting the AUD_CRC and AUD_SEM registers.

The register AUD_CRC defines the action which will be taken upon detection of a CRC error in an input frame.

Value	Meaning
00	Disable CRC detection and error concealment
01	Mute on detection of CRC error
10	Illegal
11	Skip invalid frame

Table 112 AUD_CRC register coding

The register AUD_SEM defines the action which will be taken upon detection of a synchronization error, using the coding in [Table 113](#).

Value	Meaning
00	Ignore error
01	Mute on detection of synchronization error
10	Illegal
11	Skip invalid frame

Table 113 AUD_SEM register coding

26.2.7 Ancillary data extraction

The ancillary data which may be held at the end of audio frames can be extracted and read from the AUD_ANC register. This register constitutes a 32-bit FIFO. The first bit of ancillary data received is stored in bit AUD_ANC[0].

The extraction of ancillary data in AUD_ANC is started by enabling interrupt 7. An interrupt 7 is generated when either:

- 32 bits of ancillary data have been received from the bit stream and written into AUD_ANC, i.e. when it is full, or
- The end of a frame is reached.

Register AUD_ADA holds the number of bits available in the ancillary data buffer, AUD_ANC[31:0].

When AUD_ANC[31:24] is read, interrupt 7 is cleared, ANC_ADA is cleared and the ancillary data buffer is reinitialized.

Decoding stops if the STi5512 tries to write data into AUD_ANC when it is full. The normal response would be to read AUD_ADA and then AUD_ANC. However, if interrupt 7 is disabled (by resetting bit AUD_ITN[7]), decoding will continue and the registers AUD_ANC and AUD_ADA will retain their contents until AUD_ANC[31:24] is read.

If AUD_ANC is not read at the end of the frame, and it is not full, ancillary data bits in the next frame will be appended.

26.3 AC-3 interface

The interface to the off-chip AC-3 Audio Decoder is composed of two serial buses. The I²C bus is used for control and a synchronous serial interface for compressed data transfer.

26.3.1 Input / output description

The external AC-3 interface uses six signals. Four of these signals are multiplexed with the internal MPEG-1 decoder output signals. The signals are described in [Table 114](#).

Signal name/AC-3	Signal name MPEG-1	Type	Description
A_C_DATA	PCM_DATA	Out	AC-3 packet data or PCM serial data
A_C_STB	SCLK	Out	AC-3 packet strobe or PCM clock
notA_C_REQ		In	AC-3 data request
A_WORD_CLK	LRCLK	Out	AC-3 word clock or PCM L/R clock
notA_PTS_STB		In	AC-3 PTS strobe
	PCM_CLK	In/Out	PCM clock input from VCXO

Table 114 External AC-3 decoder interface

A schematic of the interface along with timing is shown in [Figure 157](#).

notA_C_REQ is active when the AC-3 decoder is capable of accepting data and A_C_STB is used to strobe the data into the audio decoder on the rising edge. The signal A_WORD_CLK is the A_C_STB signal divided by 32. It is phased so that the transition coincides with a byte boundary. This signal can be used as a framing signal for certain AC-3 decoders.

notA_PTS_STB is used to latch the value of a free running timer in the STi5512 for clock recovery.

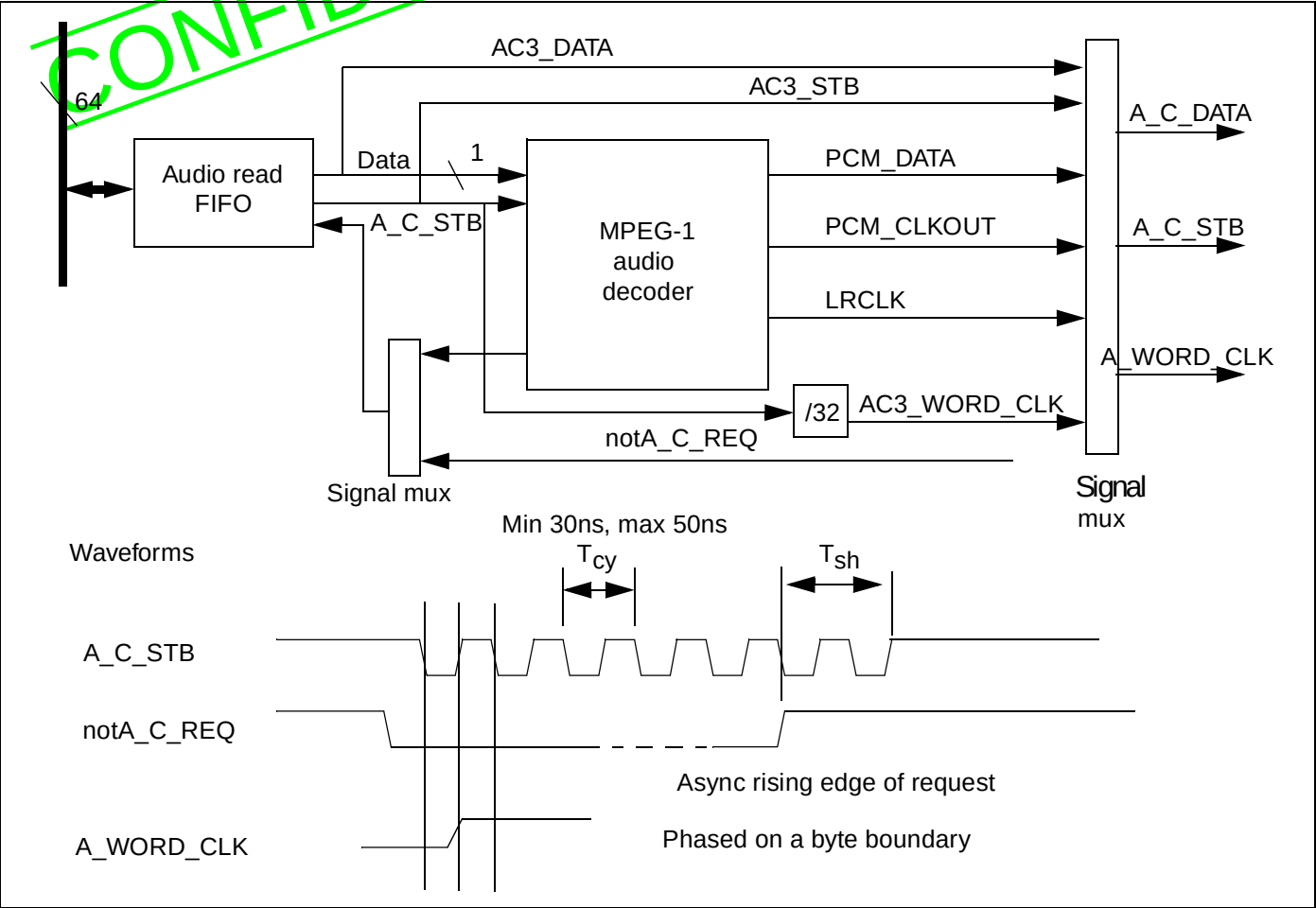


Figure 157 AC-3 interface detail

Parameter	Minimum	Maximum	Units	Configuration
Tcy	30	50	ns	cfg_gcf.sck = 1
Tcy	66	100	ns	cfg_gcf.sck = 0
Tsh	0	2	cycles	

Table 115 AC-3 interface timing

27 Clocks and power-down mode

This chapter describes the various clocks provided on the STi5512 for clocking various subsystems and for system and programmer use. Most of these clocks can be stopped to reduce power consumption, as described in [Power-down mode on page 234](#).

The figure below shows the configuration of all the PLLs, with nominal clock speeds, and the clock distribution within the device. Actual clock speeds are given in [Table 116 on page 232](#).

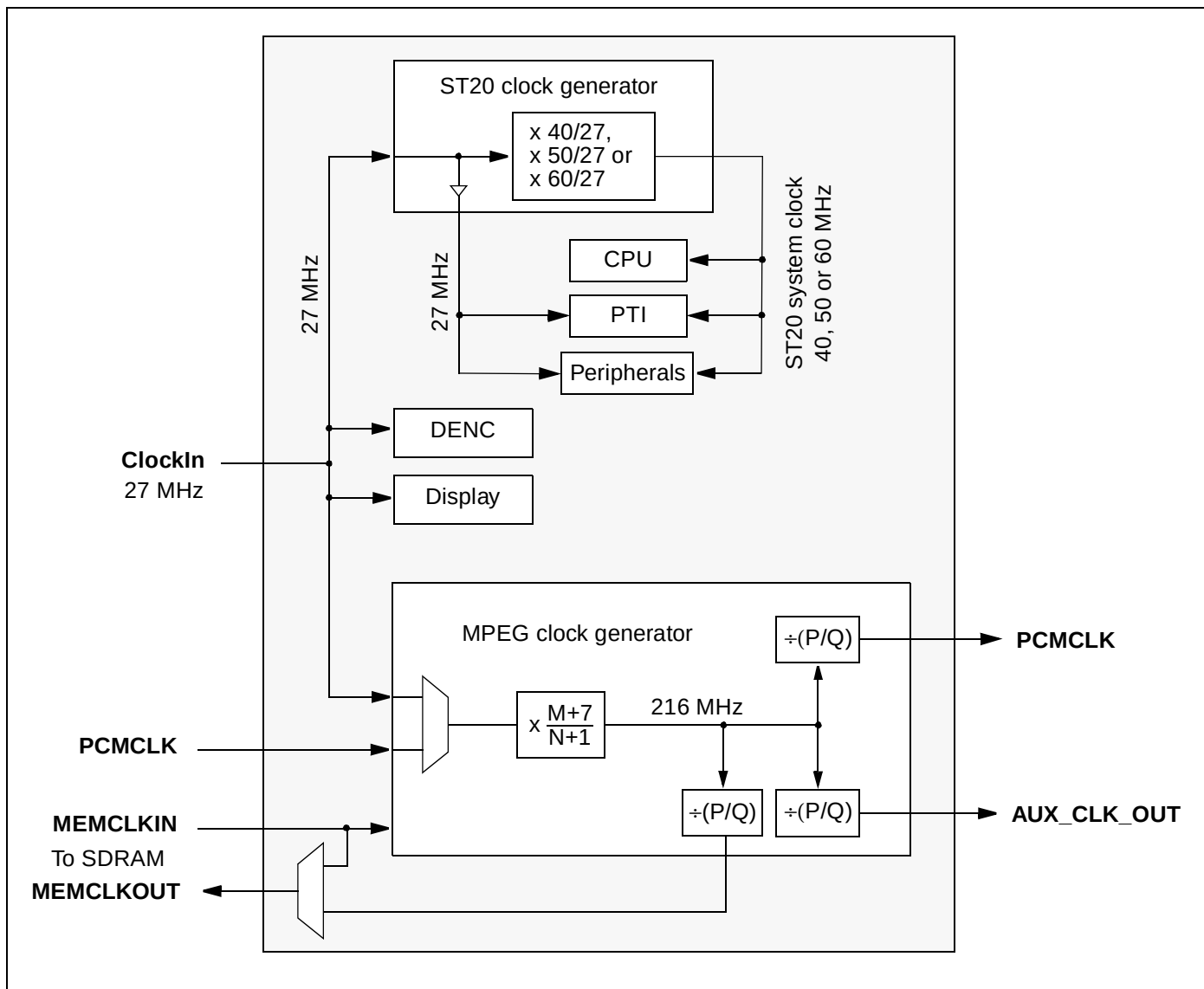


Figure 158 STi5512 clocks generation

The clock input pin **PCMLCK** can be configured as an input clock to the MPEG clock generator or as an output, using **ClockIn** as the input.

27.1 CPU and related clocks

The STi5512 has on-chip phase locked loops to provide the following high frequency internal clocks for the CPU and related functions:

- Processor clock;
- OS-Link clock;
- Low and high priority timer clocks.

These clocks are derived from the clock input (**ClockIn**) either by use of a phase-locked loop (PLL) (*normal mode*) or directly (*times one mode*).

- Normal mode: the PLL generates all internal clocks. The frequency of the clock input (**ClockIn**) must be 27 Mhz.
- Times one mode: in this mode the PLL is by-passed. The input clock must be in the range 0 to 50 Mhz. It should also nominally have a 50/50 mark space ratio.

A *power-down* mode is available where the internally generated clocks can be turned off.

27.1.1 Processor speed select

The speed of the internal processor clock is variable in discrete steps. The clock rate at which the STi5512 runs is determined by the logic levels applied on the two speed select pins **SpeedSelect0-1**. Table 116 shows possible processor speeds for a **ClockIn** input clock running at 27 MHz. The presence of a speed in this table does not necessarily mean that parts of that speed will be manufactured.

Speed Select1:0	Processor clock speed (MHz)	Processor cycle time ns	High priority timer (MHz)	Low priority timer (MHz)	Link speed Mb/s
00	Times one mode				
01	60.0	16.67	1.0	0.015625	19.20
10	39.9	25.06	0.9975	0.015586	19.95
11	49.875	20.05	0.9975	0.015586	19.95

Table 116 Processor speed selection with 27MHz ClockIn

Clock duty cycle is 50% for all speed selections.

27.2 Video and audio clocks

The STi5512 has an on-chip phase locked loop, called the MPEG/system PLL, to generate all the required clocks for video device operation from a single 27MHz input clock. It can be used to generate many different clocks from the 27MHz by fractional division. It also provides the audio decoder system clock.

The MPEG/system clock generation consists of a patented frequency synthesizer circuit and fractional dividers which derive all of the required system clocks from a single selectable input, thus eliminating the need for external dividers and PLL circuitry. The reference input frequency can be the incoming 27MHz clock or the PCM clock.

The selected reference clock frequency is multiplied by a programmable integrated PLL and the output of the PLL is steered to a bank of programmable fractional dividers to generate each of the following output clocks:

- The internal MPEG audio decoder clock,
- The SDRAM memory clock,
- A general purpose auxiliary clock which can be used on the board.

The internal video decoder clocks are generated from the SDRAM memory clock by division.

Each of the dividers is controlled by a dedicated register and there are two general configuration registers. These registers are described in detail in the register map and are summarized in [Table 117](#).

Register	Type	Address	Function
CKG_PLL	R/W	0x30	PLL parameters.
CKG_CFG	R/W	0x31	Clock generator configuration.
CKG_PCM	R/W	0xD4 - D7	Audio clock divider.
CKG_MCK	R/W	0xD8 - DB	SDRAM clock divider.
CKG-AUX	R/W	0xDC - DF	Auxiliary clock divider.

Table 117 Clock generation registers

The fractional dividers each perform a division by P/Q. This can alternatively be expressed as:

$$\frac{P}{Q} = P_0 + 1 + \frac{P_r}{Q}$$

where:

$$5 \leq \frac{P}{Q} \leq 17$$

$$2 \leq P_0 \leq 15$$

$$0 \leq P_r \leq 1023$$

$$0 \leq Q \leq 2047$$

$$P_r \leq Q$$

$$\frac{P_r}{Q} = 1, Q = 0$$

Thus, the output frequency of each fractional divider, f_{OUT} , is calculated as:

$$f_{OUT} = \frac{f_{VCO}}{P_0 + 1 + \frac{P_r}{Q}}$$

or

$$f_{OUT} = \frac{f_{IN} \times \frac{M+7}{N+1}}{P_0 + 1 + \frac{P_r}{Q}}$$

The values for M and N are programmed in register **CKG_PLL**, with M in the range 0-7 and N in the range 0 or 1. P_0 , P_r and Q are programmable for each of the fractional dividers and are stored in clock generator registers. For more detailed information refer to the *STi3520A PLL Application Note* (AN879).

27.3 Real-time counter

This timer keeps track of real time, even when the internal clocks are stopped. The timer is a 64-bit counter, powered from a separate Vdd (**RTCVD**) allowing it to operate even when the rest of the chip is not powered.

The timer must be clocked at all times by one of the following clocking sources even if the real-time counter or other low-power features are not being used, otherwise the CPU might fail to boot:

- An external clock input (**LPClockIn**): the frequency for this clock must not be higher than 150 kHz. In this case the **LPClockOsc** pin should not be connected on the board.
- A watch crystal, in the circuit shown in below.

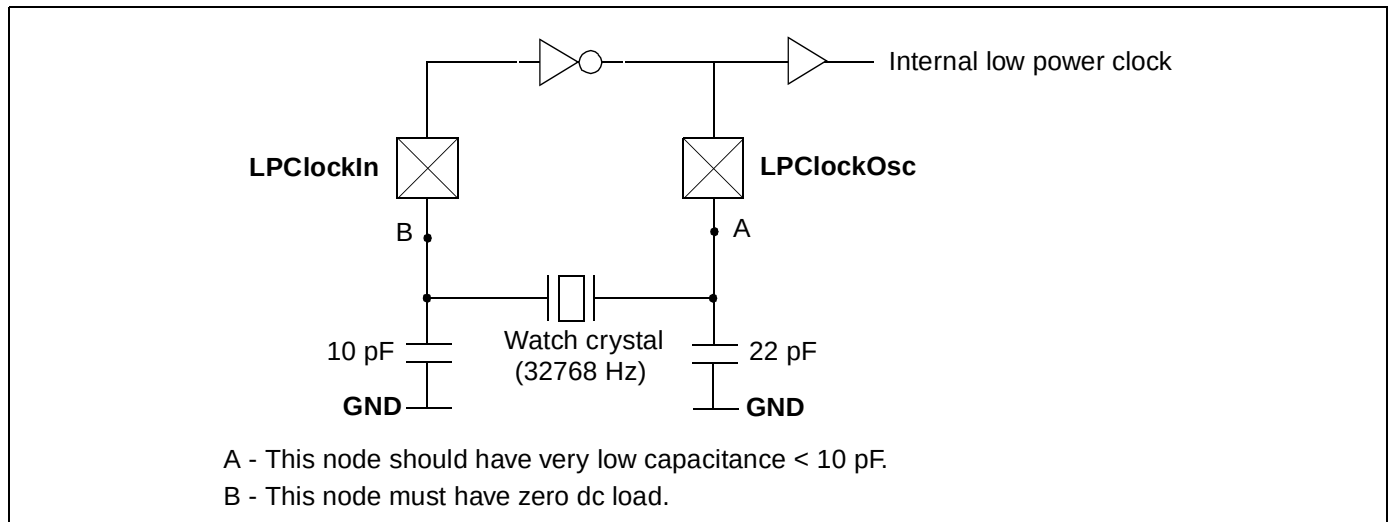


Figure 159 Watch crystal clocking source

27.4 Watchdog counter

The low power alarm counter can be used as a watchdog timer if bit 0 of the register **WdEnable** is set. Setting bit 0 of **WdEnable** disables the entering of low power mode when starting the low power alarm counter.

To trigger off watchdog functionality, the low power alarm is programmed and started as normal. When the low power alarm counts down to the value #1, the circuit resets. The **WdFlag** register is set when a watchdog reset occurs.

27.5 Power-down mode

The STi5512 is manufactured using 0.35 micron 3.3V CMOS technology, which provides reduced power consumption internally and allows the use of low power peripherals. A power-down mode is available, in which the internal clocks are turned off to further reduce power consumption. In power-down mode the processor and all peripherals are stopped, including the external memory controller and optionally the PLL. Effectively the internal clock is stopped and functional operation is stalled. On restart, the clock is restarted and the chip resumes normal functional operation.

Entering and leaving power-down mode

The STi5512 enters power-down when:

- the low power alarm counter is programmed and started, providing there are no external interrupts active.

The STi5512 exits power-down when:

- an enabled external interrupt becomes active;

- the low power alarm counter reaches zero.

Low power alarm

The low power alarm counter is a 40-bit counter which when started triggers off power-down mode. A write to the **LPAlarmStart** register starts the low power alarm counter and the STi5512 enters low power mode. When the counter has counted down to zero and assuming no other valid wake-up sources occur first, the STi5512 exits low power mode and the global clocks are turned back on.

PLL operation in power-down mode

In power-down mode the ST20 PLL can either be left running, partially turned off (power and reference still on) or turned off completely. This is determined by the value programmed in the register **LPSysPll**. The MPEG PLL can be turned off if required during power-down mode.

28 Block move DMA

This module copies blocks of data from one byte address to another in memory. The module cannot access devices other than memory.

A source address, a destination address and a count of the number of bytes to be transferred must be specified.

The interface between the CPU and the block move module is provided using a set of registers and an interrupt to signal when a DMA transfer has completed.

28.1 Moving blocks of data

To perform a DMA block move from one memory buffer to another, the block move module must first be initialized with the source and destination addresses and then a byte count written to the **BMDMACount** register to specify the amount of data to transfer and start the DMA operation.

The source and destination addresses are the bases of the source and destination areas and can be any byte addresses. The transfer size can be any value in the range of 1 to 65535 bytes. If the source area overlaps with the destination area, then the result is undefined.

At the end of the block move operation the **BMDMAStatus** register will signal that an interrupt is pending. If the interrupt enable bit of the **BMDMAIntEn** register is set to 1, this will cause an interrupt. The interrupt pending bit must be reset by software by writing to the **BMDMAIntAck** register before any further block move operations can be performed.

A DMA block move can be aborted by writing to the **BMDMAAbort** register.

29 PWM and counter module

This module provides three PWM encoder outputs, three PWM decoder (capture) inputs and four programmable timers. Each capture input can be programmed to detect rising edge, falling edge, both edges or neither edge (disabled). These facilities are clocked by two independent clocks, one for PWM outputs and one for capture inputs/timers.

In the STi5512, not all the facilities in the module can be used, since some of the interface pins of the module are not available as external pins of this device. In particular, the following interface pins are not available:

- PWMOut3
- CaptureIn3
- **CompareOut0, CompareOut1 and CompareOut3**

However the corresponding **CompareOut** facilities can be used to generate interrupts at programmable time intervals. Otherwise the corresponding registers should not be used.

The module is programmed by means of registers described in the individual sections.

The module generates a single interrupt signal. The exact event which caused an interrupt can be determined by reading status bits in a register, which can then be cleared.

29.1 External interface.

Name	In/Out	Function
PWMOut0, PWMOut1, PWMOut2	out	PWM outputs
CaptureIn0, CaptureIn1, CaptureIn2	in	Capture trigger inputs
CompareOut2	out	Compare output

Table 118 PWM and counter pins

29.2 PWM outputs

There are four PWM outputs which share a common counter. The relative width (in counts) of the output pulse on pin PWMOutN is set between 1 and 256 by loading a value from 0 to 255 into the register PWMValN. The width cannot be less than 1, and if it is 256 the pin is continuously high. Pulses occur every 256 counts.

The counter is clocked by the 27MHz clock ClockIn divided by a prescaler. The prescaling factor, and therefore the period represented by one count, is determined by the value of field PWMClkValue in register Control. The factor can be from 1 to 16.

The counter (in register PWMCount) is enabled by setting the PWMEnable bit of the Control register to 1. When it is disabled (PWMEnable is 0), PWMOut is forced low. PWMCount is writable at any time but can have a synchronization latency.

When the PWM counter overflows, an interrupt is generated if the PWMIntEn bit of the PWMIntEnable register is set to 1. Bit PWMInt of register PWMIntStatus becomes 1, and can be reset by writing 1 to bit PWMIntAck of register PWMIntAck.

29.3 Capture inputs

There are four capture inputs which share a common counter with four compare facilities.

What constitutes an event on input CaptureInN is defined by the code in register CaptureEdgeN. Possible events are rising edge, falling edge, both or neither (in other words, disabled).

When an input event occurs on input CaptureInN, the value of the counter (in register PWMCaptureCount) at that time is captured in register PWMCaptureValN. The value can be 0x00000000 to 0xFFFFFFFF.

When an input event occurs, an interrupt is generated provided the CaptureNIntEn bit of the PWMIntEnable register is set to 1. Bit CaptureIntN of register PWMIntStatus becomes 1, and can be reset by writing 1 to bit CaptureIntAckN of register PWMIntAck.

The counter is not stopped nor reset by any of these events. See [Section 29.5](#) below for details.

29.4 Compare (programmable timer) facilities

There are four programmable timer facilities which share a common counter with four capture inputs. Each of four compare registers PWMCompareValN in the module can be set to a value 0x00000000 to 0xFFFFFFFF.

When the counter in register PWMCaptureCount reaches the value of register PWMCompareValN, two things happen:

- An interrupt is generated provided the PWMCompareIntEnN bit of the PWMIntEnable register is set to 1. Bit PWMCompareIntN of register PWMIntStatus becomes 1, and can be reset by writing 1 to bit PWMCompareIntAckN of register PWMIntAck.
- Pin PWMCompareOutN takes on the value set in register PWMCompareOutValN.

The counter is not stopped nor reset by any of these events. See [Section 29.5](#) below for details of the counter.

29.5 Capture/compare counter, prescaling and clocking

The capture/compare counter is clocked from the prescaled system clock, and is common to all capture and compare functions. The prescaling factor, and therefore the period represented by one count, is determined by the value of field CaptureClkValue in register PWMControl. The factor can be from 1 to 32.

The counter (in register PWMCaptureCount) is enabled by setting the PWMCaptureEnable bit of the PWMControl register to 1. When it is disabled (PWMCaptureEnable is 0), none of the capture or compare functions work. PWMCaptureCount, like PWMCount, can be read or written at any time.

When the capture/compare counter reaches its maximum count of 0xFFFFFFFF, it wraps round to count up from zero again.

30 Asynchronous serial controller

The Asynchronous Serial Controller (ASC), also referred to as the UART interface, provides serial communication between the STi5512 and other microcontrollers, microprocessors or external peripherals. The STi5512 provides four ASCs, two of which are generally used by the SmartCard controllers.

Eight or nine bit data transfer, parity generation, and the number of stop bits are programmable. Parity, framing, and overrun error detection is provided to increase the reliability of data transfers. Transmission and reception of data can simply be double-buffered, or 16-deep FIFOs may be used. Handshaking is supported on both transmission and reception. For multiprocessor communication, a mechanism to distinguish the address from the data bytes is included. Testing is supported by a loop-back option. A dual mode 16-bit baud rate generator provides the ASC with a separate serial clock signal.

Each ASC supports full-duplex asynchronous communication, where both the transmitter and the receiver use the same data frame format and the same baud rate. Data is transmitted on the transmit data output pin **TxD** and received on the receive data input pin **RxD**.

Each ASC can be set to operate in SmartCard mode for use when interfacing to a SmartCard.

The registers for each ASC are grouped in a 4 Kbyte block, with the base of the block for ASC number *n* at the address *ASCnBaseAddress*. The value of each *ASCnBaseAddress* is given in the *STi5512 Register Manual*.

30.1 Control

The **ASC_nControl** register controls the operating mode of the ASC. It contains control and enable bits, error check selection bits, and status flags for error identification.

Serial data transmission or reception is only possible when the baud rate generator run bit (**Run**) is set to 1. When the **Run** bit is set to 0, **TxD** will be 1. Setting the **Run** bit to 0 will immediately freeze the state of the transmitter and receiver and should only be done when the ASC is idle.

Note: Programming the mode control field (**Mode**) to one of the reserved combinations may result in unpredictable behavior.

The ASC can be set to use either double-buffering or a 16-deep FIFO on transmission and reception.

30.1.1 Resetting the FIFOs

The 'registers' **ASC_nTxReset** and **ASC_nRxReset** have no actual storage associated with them. A write of any value to one of these registers resets the corresponding FIFO.

30.1.2 Transmission and reception

Serial data transmission or reception is only possible when the baud rate generator run bit (**Run**) is set to 1. A handshaking protocol is supported on both transmission and reception, using **CTS** and **RTS** signals.

A transmission is started by writing to the transmit buffer register **ASC_nTxBuffer**. Because data transmission is double-buffered or uses a FIFO (selectable in the **ASC_nControl** register), a new character may be written to the transmit buffer register before the transmission of the previous character is complete. This allows characters to be sent back-to-back without gaps.

Data reception is enabled by the receiver enable bit (**RxEnable**) in the control register. After reception of a character has been completed, the received data and, if provided by the selected operating mode, the parity error bit, can be read from the receive buffer register **ASC_nRxBuffer**.

Reception of a second character may begin before the received character has been read out of the receive buffer register. The overrun error status flag (**OverrunError**) in the status register **ASC_nStatus** will be set when the receive buffer register has not been read by the time reception of a second character is complete. The previously received

character in the receive buffer is overwritten, and the **ASC_n_Status** register is updated to reflect the reception of the new character.

The loop-back option (selected by the **LoopBack** bit) internally connects the output of the transmitter shift register to the input of the receiver shift register. This may be used to test serial communication routines at an early stage without having to provide an external network.

30.2 Data frames

Data frames may be 8-bit or 9-bit, with or without parity and with or without a wake-up bit. The data frame type is selected by the setting of the **Mode** bit field in the control register.

The transmitted data frame consists of three basic elements:

- The start bit;
- The data field (8 or 9 bits, least significant bit (LSB) first, including a parity bit or wake-up bit, if selected);
- The stop bits (0.5, 1, 1.5 or 2 stop bits).

30.2.1 8-bit data frames

Figure 160 illustrates a 8-bit transmitted data frame. 8-bit frames may use of one of the following formats:

- Eight data bits **D0-7** (**Mode** set to 001);
- Seven data bits **D0-6** plus an automatically generated parity bit (**Mode** set to 011).

Parity may be odd or even, depending on the **ParityOdd** bit in the **ASC_n_Control** register. If the modulo 2 sum of the seven data bits is 1, then the even parity bit will be set and the odd parity bit will be cleared.

In receive mode the parity error flag (**ParityError**) will be set if a wrong parity bit is received. The parity error flag is stored in the 8th bit (D7) of the **ASC_n_RxBuffer** register. The parity error bit is set high if there is a parity error.

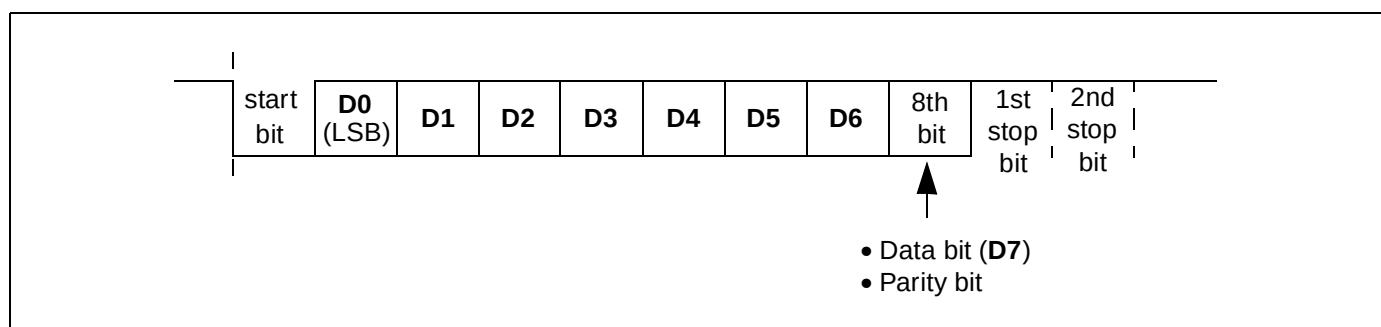


Figure 160 8-bit Tx data frame format

30.2.2 9-bit data frames

Figure 161 illustrates a 9-bit transmitted data frame. 9-bit data frames use of one of the following formats:

- Nine data bits **D0-8** (**Mode** set to 100);
- Eight data bits **D0-7** plus an automatically generated parity bit (**Mode** set to 111);
- Eight data bits **D0-7** plus a wake-up bit (**Mode** set to 101)

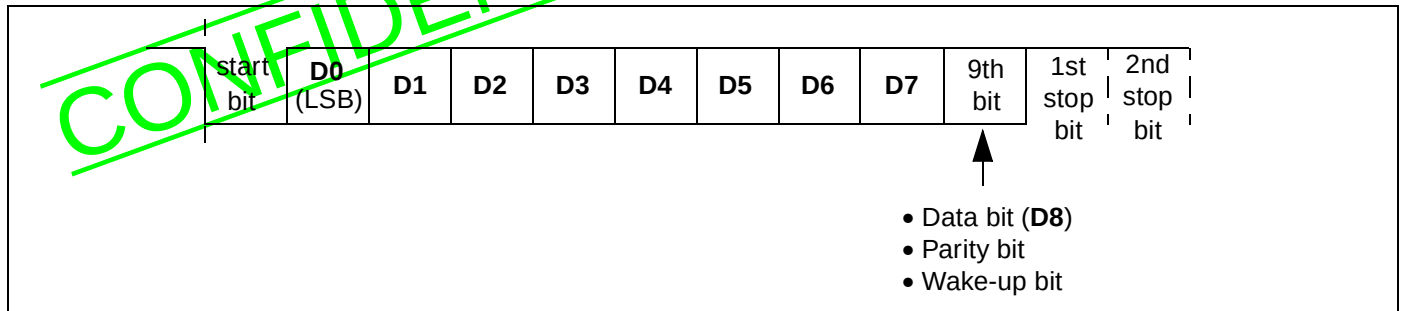


Figure 161 9-bit Tx data frame format

Parity may be odd or even, depending on the **ParityOdd** bit in the **ASC_n_Control** register. If the modulo 2 sum of the eight data bits is 1, then the even parity bit will be set and the odd parity bit will be cleared. The parity error flag (**ParityError**) will be set if a wrong parity bit is received. The parity error flag is stored in the 9th bit (D8) of the **ASC_n_RxBuffer** register. The parity error bit is set high if there is a parity error.

In wake-up mode, received frames are only transferred to the receive buffer register if the ninth bit (the wake-up bit) is 1. If this bit is 0, no receive interrupt request will be activated and no data will be transferred.

This feature may be used to control communication in multi-processor systems. When the master processor wants to transmit a block of data to one of several slaves, it first sends out an address byte which identifies the target slave. An address byte differs from a data byte in that the additional ninth bit is a 1 for an address byte and a 0 for a data byte, so no slave will be interrupted by a data byte. An address byte will interrupt all slaves (operating in *8-bit data plus wake-up bit* mode), so each slave can examine the 8 least significant bits (LSBs) of the received character, which is the address. The addressed slave will switch to 9-bit data mode, which enables it to receive the data bytes that will be coming (with the wake-up bit cleared). The slaves that are not being addressed remain in *8-bit data plus wake-up bit* mode, ignoring the data bytes which follow.

30.3 Transmission

Transmission begins at the next baud rate clock tick, provided that the **Run** bit is set and data has been loaded into the **ASC_n_TxBuffer**. If the **CTSEnable** bit is set in the **ASC_n_Control** register then transmission only occurs when **CTS** is high.

The transmitter empty flag (**TxEmpty**) indicates whether the output shift register is empty. It will be set at the beginning of the last data frame bit that is transmitted, i.e. during the first system clock cycle of the first stop bit shifted out of the transmit shift register.

The loop-back option (selected by the **LoopBack** bit of the **ASC_n_Control** register) internally connects the output of the transmitter shift register to the input of the receiver shift register. This may be used to test serial communication routines at an early stage without having to provide an external network.

30.3.1 Transmission with FIFOs enabled

The FIFOs are enabled by setting the **FifoEnable** bit of the **ASC_n_Control** register. The output FIFO is implemented as a 16-deep array of 9-bit vectors. Values to be transmitted are written to the output FIFO by writing to **ASC_n_TxBuffer**.

The **TxFull** bit of the **ASC_n_Status** register is set when the transmit FIFO is considered full, i.e. when it contains 16 characters. Further writes to **ASC_n_TxBuffer** will fail to overwrite the most recent entry in the output FIFO. The **TxHalfEmpty** bit of the **ASC_n_Status** register is set when the output FIFO contains 8 or fewer characters.

Values are shifted out of the bottom of the output FIFO into a 9-bit output shift register in order to be transmitted. If the transmitter is idle (i.e. the output shift register is empty) and something is written to the **ASC_n_TxBuffer** so that the

output FIFO becomes non-empty, the output shift register is immediately loaded from the output FIFO and transmission of the data in the output shift register begins at the next baud rate tick.

When the transmitter is just about to transmit the stop bits, and if the output FIFO is non-empty, the output shift register will be immediately loaded from the output FIFO, and the transmission of this new data will begin as soon as the current stop bit period is over (i.e. the next start bit will be transmitted immediately following the current stop bit period). If the output FIFO is empty at this point, the output shift register will become empty. Thus back-to-back transmission of data can take place. If the output FIFO is empty at this point, the output shift register will become empty. Writing anything to **ASC_n_TxReset** empties the output FIFO.

After changing the **FifoEnable** bit, it is important to reset the FIFO to empty (by writing to the **ASC_n_TxReset** register), or garbage may be transmitted.

30.3.2 Double-buffered transmission

Double buffering is enabled and the FIFOs disabled by writing 0 to the **FifoEnable** bit of the **ASC_n_Control** register. When the transmitter is idle, the transmit data written into the transmit buffer **ASC_n_TxBuffer** is immediately moved to the transmit shift register, thus freeing the transmit buffer for the next data to be sent. This is indicated by the transmit buffer empty flag (**TxHalfEmpty**) being set. The transmit buffer can be loaded with the next data while transmission of the previous data is still going on.

When the FIFOs are disabled, the **TxFull** bit is set when the buffer contains 1 character, and a write to **ASC_n_TxBuffer** in this situation will overwrite the contents. The **TxHalfEmpty** bit of the **ASC_n_Status** register is set when the output buffer is empty.

30.4 Reception

Reception is initiated by a falling edge on the data input pin **RxD**, provided that the **Run** and **RxEnable** bits of the **ASC_n_Control** register are set.

Controlled data transfer can be achieved using the **RTS** handshaking signal provided by the UART. The sender checks the **RTS** to ensure the UART is ready to receive data. In double-buffered reception **RTS** goes high when **ASC_n_RxBuffer** is empty, in FIFO-controlled operation it goes high when **RxHalfFull** is zero.

The **RxD** pin is sampled at 16 times the rate of the selected baud rate. A majority decision of the first, second and third samples of the start bit determines the effective bit value. This avoids erroneous results that may be caused by noise.

If the detected value of the first bit of a frame is not a 0, then the receive circuit is reset and waits for the next falling edge transition at the **RxD** pin. If the start bit is valid, i.e. is 0, the receive circuit continues sampling and shifts the incoming data frame into the receive shift register. For subsequent data and parity bits, the majority decision of the seventh, eighth and ninth samples in each bit time is used to determine the effective bit value. The effective values received on **RxD** are shifted into a 10-bit input shift register.

For 0.5 stop bits, the majority decision of the third, fourth, and fifth samples during the stop bit is used to determine the effective stop bit value. For 1 and 2 stop bits, the majority decision of the seventh, eighth, and ninth samples during the stop bits is used to determine the effective stop bit values. For 1.5 stop bits, the majority decision of the fifteenth, sixteenth, and seventeenth samples during the stop bits is used to determine the effective stop bit value.

Reception is stopped by clearing the **RxEnable** bit of **ASC_n_Control**. Any currently received frame is completed including the generation of the receive status flags. Start bits that follow this frame will not be recognized.

30.4.1 Hardware error detection

To improve the safety of serial data exchange, the ASC provides three error status flags in the **ASC_n_Status** register which indicate if an error has been detected during reception of the last data frame and associated stop bits.

- The parity error bit (**ParityError**) in the **ASC_n_Status** register is set when the parity check on the received data is incorrect. In FIFO operation parity errors on the buffers are OR-ed to yield a single parity error bit.

- The framing error bit (**FrameError**) in the **ASC_n_Status** register is set when the **RxD** pin is not a 1 during the programmed number of stop bit times (see section 30.4). In FIFO operation the bit remains set while at least one of the entries has a frame error.
- The overrun error bit (**OverrunError**) in the **ASC_n_Status** register is set when the input buffer is full and a character has not been read out of the **ASC_n_RxBuffer** register before reception of a new frame is complete.

These flags are updated simultaneously with the transfer of data to the receive input buffer.

30.4.1.1 Frame and parity errors

The most significant bit (bit 9 of 0-9) of each input entry records whether or not there was a frame error when that entry was received (i.e. one of the effective stop bit values was '0'). The **FrameError** bit of the **ASC_n_Status** register is set when the input buffer (double-buffered operation), or at least one of the valid entries in the input buffering (FIFO-controlled operation), has its most significant bit set.

If the mode is one where a parity bit is expected, then the next bit (bit 8 of 0-9) records whether there was a parity error when that entry was received. It does not contain the parity bit that was received. For 7-bit+parity data frames the parity error bit is set in both the eighth (bit 7 of 0-9) and the ninth (bit 8 of 0-9) bits. The **ParityError** bit of **ASC_n_Status** is set when the input buffer (double-buffered operation), or at least one of the valid entries in the input buffering (FIFO-controlled operation), has bit 8 set.

When receiving 8-bit data frames without parity (see section 30.2.1), the ninth bit of each input entry (bit 8 of 0-9) is undefined.

30.4.2 Input buffering modes

30.4.2.1 FIFO enabled reception

The FIFOs are enabled by setting the **FifoEnable** bit of the **ASC_n_Control** register. The input FIFO is implemented as a 16-deep array of 10-bit vectors (each 9 down to 0). If the input FIFO is empty i.e. no entries are present, the **RxBufFull** bit of the **ASC_n_Status** register is set to '0'. If one or more FIFO entries are present, the **RxBufFull** bit of the **ASC_n_Status** register is set to 1. If the input FIFO is not empty, a read from **ASC_n_RxBuffer** will get the oldest entry in the input FIFO.

The **RxHalfFull** bit of the **ASC_n_Status** register is set when the input FIFO contains more than 8 characters. Writing anything to **ASC_n_RxReset** empties the input FIFO. As soon as the effective value of the last stop bit has been determined, the content of the input shift register is transferred to the input FIFO (except during wake-up mode, in which case this happens only if the wake-up bit, bit 8, is a '1'). The receive circuit then waits for the next falling edge transition at the **RxD** pin.

The **OverrunError** bit of the **ASC_n_Status** register is set when the input FIFO is full and a character is loaded from the input shift register into the input FIFO. It is cleared when the **ASC_n_RxBuffer** register is read.

After changing the **FifoEnable** bit, it is important to reset the FIFO to empty by writing to the **ASC_n_RxReset** register; otherwise the state of the FIFO pointers may be garbage.

30.4.2.2 Double buffered reception

Double buffered operation is enabled and the FIFOs disabled by writing 0 to the **FifoEnable** bit of the **ASC_n_Control** register. This mode can be seen as equivalent to a FIFO-controlled operation with a FIFO of length 1 (the first FIFO vector is in fact used as the buffer). When the last stop bit has been received (at the end of the last programmed stop bit period) the content of the receive shift register is transferred to the receive data buffer register (**ASC_n_RxBuffer**). The receive buffer full flag (**RxBufFull**) is set, and the parity (**ParityError**) and framing error (**FrameError**) flags are updated at the same time, after the last stop bit has been received, i.e. at the end of the last stop bit programmed period. The flags are updated even if no valid stop bits have been received. The receive circuit then waits for the next falling edge transition at the **RxD** pin.

30.4.3 Time-out mechanism

The ASC contains an 8-bit time-out counter. This reloads from **ASC_n_Timeout** whenever one or more of the following is true:

- **ASC_n_RxBuffer** is read;
- the ASC is in the middle of receiving a character;
- **ASC_n_Timeout** is written to.

If none of these conditions hold the counter decrements towards 0 at every baud rate tick.

The **TimeoutNotEmpty** bit of the **ASC_n_Status** register is '1' when the input FIFO is not empty and the time-out counter is zero.

The **TimeoutIdle** bit of the **ASC_n_Status** register is '1' when the input FIFO is empty and the time-out counter is zero.

The effect of this is that whenever the input FIFO has got something in it, the time-out counter will decrement until something happens to the input FIFO. If nothing happens, and the time-out counter reaches zero, the **TimeoutNotEmpty** bit of the **ASC_n_Status** register will be set.

When the software has emptied the input FIFO, the time-out counter will reset and start decrementing. If no more characters arrive, when the counter reaches zero the **TimeoutIdle** bit of the **ASC_n_Status** register will be set.

30.5 Baud rate generation

Each ASC has its own dedicated 16-bit baud rate generator with 16-bit reload capability. The baud rate generator has two possible modes of operation.

The **ASC_n_BaudRate** register is the dual-function baud rate generator and reload value register. A read from this register returns the content of the counter or accumulator (depending on the mode of operation); writing to it updates the reload register.

If the **Run** bit of the control register is 1, then any value written in the **ASC_n_BaudRate** register is immediately copied to the counter/accumulator. However, if the **Run** bit is 0 when the register is written, then the counter/accumulator will not be reloaded until the first CPU clock cycle after the **Run** bit is 1.

The baud rate generator supports two modes of operation, offering a wide range of possible values. The mode is set via the **BaudMode** bit in the **ASC_n_Control** register. Mode 0 is a simple counter driven by the CPU clock whereas Mode 1 uses a loop-back accumulator. Mode 0 is recommended for low baud rates (below 19.2K baud), where its error deviation is low, and Mode 1 is recommended for baud rates above 19.2 K.

30.5.1 Baud rates

The baud rate generator provides an internal oversampling clock at 16 times the external baud rate. This clock only ticks if the **Run** bit of the **ASC_n_Control** register is set to 1. Setting this bit to 0 will immediately freeze the state of the ASCs transmitter and receiver.

30.5.1.1 Mode 0

When the **BaudMode** bit in the **ASC_n_Control** register is set to 0, the baud rate and the required reload value for a given baud rate can be determined by the following formulae:

$$\text{BaudRate} = \frac{f_{\text{CPU}}}{16 \times \text{ASCBaudRate}}$$

$$\text{ASCBaudRate} = \frac{f_{\text{CPU}}}{16 \times \text{BaudRate}}$$

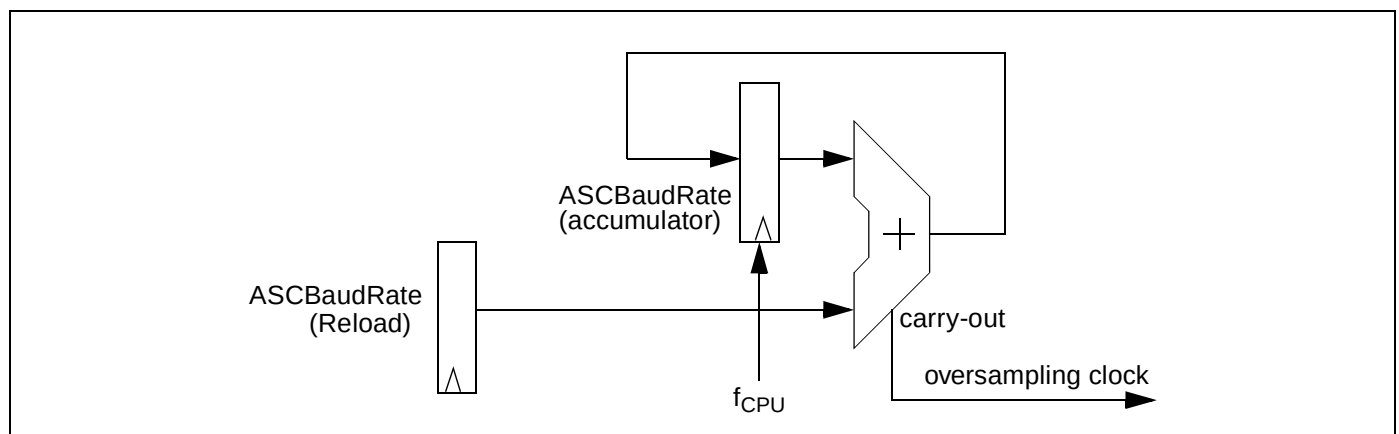
where: *ASCBaudRate* represents the content of the **ASC_n_BaudRate** reload value register, taken as an unsigned 16-bit integer and f_{CPU} is the frequency of the CPU.

The baud rate counter is clocked by the CPU clock. It counts downwards and can be started or stopped by the **Run** bit in the **ASC_n_Control** register. Each underflow of the timer provides one oversampling baud rate clock pulse. The counter is reloaded with the value stored in its 16-bit reload register each time it underflows.

Writes to the **ASC_n_BaudRate** register update the reload register value. Reads from the **ASC_n_BaudRate** register return the current value of the counter.

30.5.1.2 Mode 1

When the **BaudMode** bit in the **ASC_n_Control** register is set to 1, the baud rate is controlled by the following circuit:



Writes to **ASC_n_BaudRate** go to the reload register. Reads from **ASC_n_BaudRate** return the value in the accumulator register. Both registers are 16 bit wide and are clocked by the CPU clock.

If the system clock frequency is f_{CPU} , writing a value of *ASCBaudRate* to the **ASC_n_BaudRate** register results in an average oversampling clock frequency of:

$$\frac{\text{ASCBaudRate} \times f_{\text{CPU}}}{2^{16}}$$

so the baud rate is given by:

$$\text{BaudRate} = \frac{\text{ASCBaudRate} \times f_{\text{CPU}}}{16 \times 2^{16}}$$

This gives good granularity, and hence low baud rate deviation errors, at high baud rate frequencies.

30.6 Interrupt control

Each ASC contains two registers that are used to control interrupts, the status register (**ASC_n_Status**) and the interrupt enable register (**ASC_n_IntEnable**). The status bits in the **ASC_n_Status** register show the cause of any interrupt. The interrupt enable register allows certain interrupt causes to be masked. Interrupts will occur when a status bit is 1 (high) and the corresponding bit in the **ASC_n_IntEnable** register is 1.

The ASC interrupt signal is generated from the OR of all interrupt status bits after they have been ANDed with the corresponding enable bits in the **ASC_n_IntEnable** register, as shown in [Figure 162](#).

The status bits cannot be reset by software because the **ASC_n_Status** register cannot be written to directly. Status bits are reset by operations performed by the interrupt handler:

- Transmitter interrupt status bits (**TxEmpty**, **TxHalfEmpty**) are reset when a character is written to the transmitter buffer.
- Receiver interrupt status bit (**RxBufFull**) is reset when a character is read from the receive buffer.
- **ParityError** and **FrameError** status bits are reset when all characters containing errors have been read from the receive input buffer.
- The **OverrunError** status bit is reset when a character is read from **ASC_n_RxBuffer**.

30.6.1 Using the ASC interrupts when FIFOs are disabled (double-buffered operation)

The transmitter generates two interrupts; this provides advantages for the servicing software. For normal operation (i.e. other than the error interrupt) when FIFOs are disabled the ASC provides three interrupt requests to control data exchange via the serial channel:

- **TxHalfEmpty** is activated when data is moved from **ASC_n_TxBuffer** to the transmit shift register;
- **TxEmpty** is activated before the last bit of a frame is transmitted;
- **RxBufFull** is activated when the received frame is moved to **ASC_n_RxBuffer**.

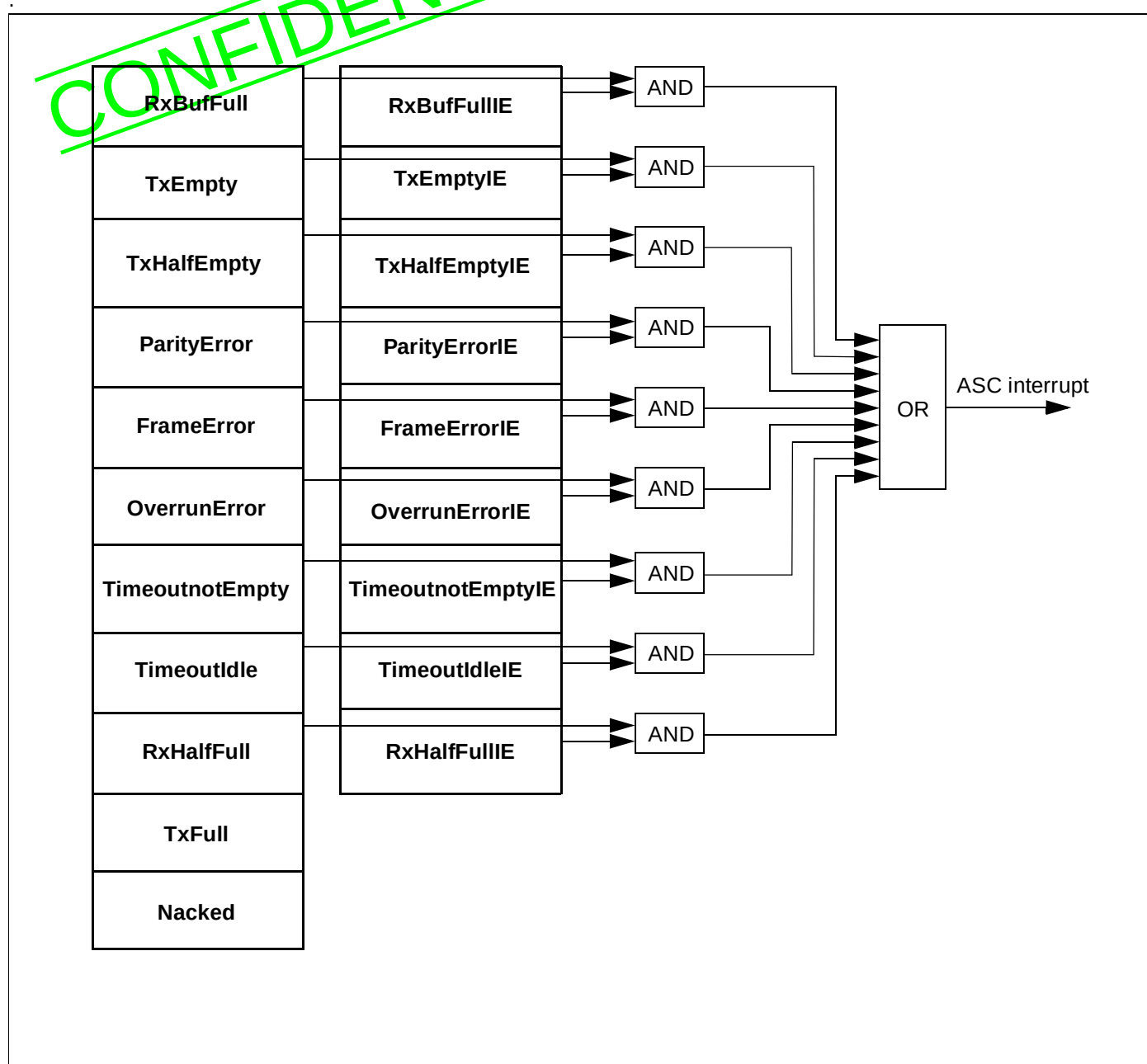


Figure 162 ASC status and interrupt registers

As shown in [Figure 163](#), **TxHalfEmpty** is an early trigger for the reload routine, while **TxEmpty** indicates the completed transmission of the data field of the frame. Therefore, software using handshake should rely on **TxEmpty** at the end of a data block to make sure that all data has really been transmitted.

For single transfers it is sufficient to use the transmitter interrupt (**TxEmpty**), which indicates that the previously loaded data has been transmitted, except for the last bit of a frame.

For multiple back-to-back transfers it is necessary to load the next data before the last bit of the previous frame has been transmitted. The use of **TxEmpty** alone would leave just one stop bit time for the handler to respond to the interrupt and initiate another transmission. Using the output buffer interrupt (**TxHalfEmpty**) to signal for more data allows the service routine to load a complete frame, as **ASC_n_TxBuffer** may be reloaded while the previous data is still being transmitted.

30.6.2 Using the ASC interrupts when FIFOs are enabled

To transmit a large number of characters back to back, the driver routine would initially write 16 characters to **ASC_n TxBuffer**. Then every time a **TxHalfEmpty** interrupt fired, it would write 8 more. When there is nothing more to send, a **TxEmpty** interrupt would tell the driver that everything has been transmitted.

When receiving, the driver could use **RxBufFull** to interrupt every time a character arrived. Alternatively, if data is coming in back-to-back, it could use **RxHalfFull** to interrupt it when there was more than 8 characters in the input FIFO to read. It would have as long as it takes to receive 8 characters to respond to this interrupt before data could overrun. If less than 8 characters streamed in, and no more were received for at least a time-out period, the driver could be woken up by one of the two time-out interrupts, **TimeoutNotEmpty** or **TimeoutIdle**.

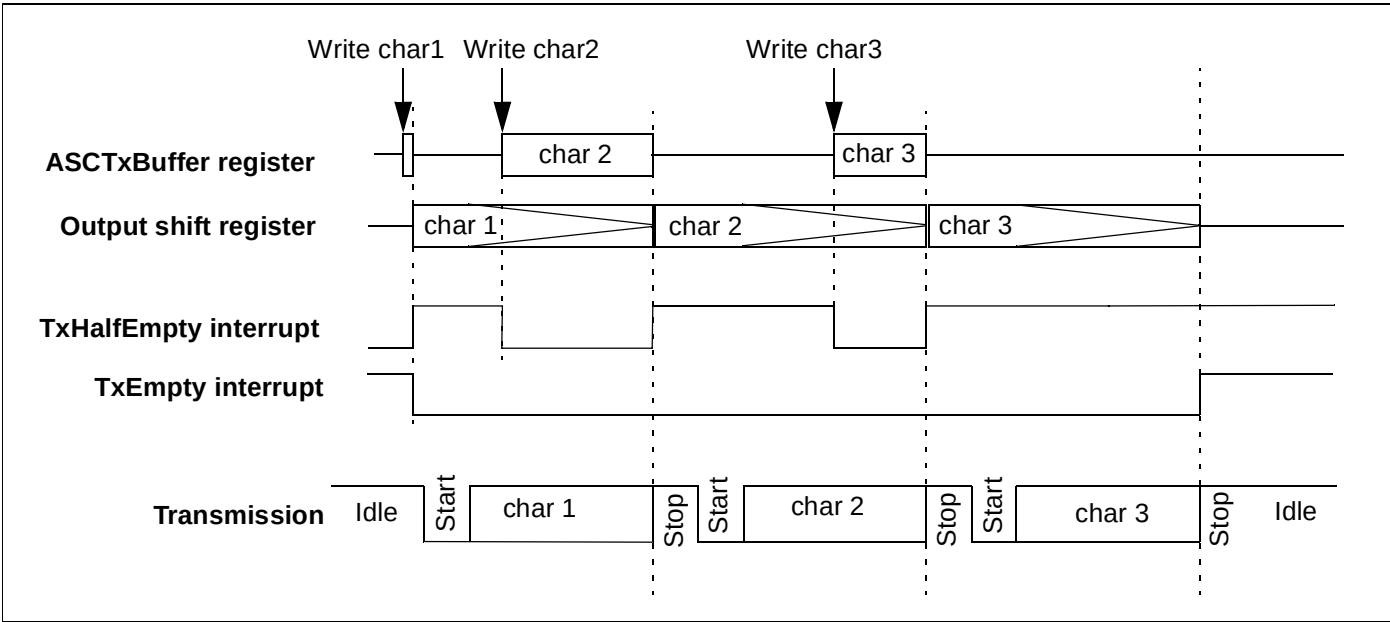


Figure 163 ASC transmission

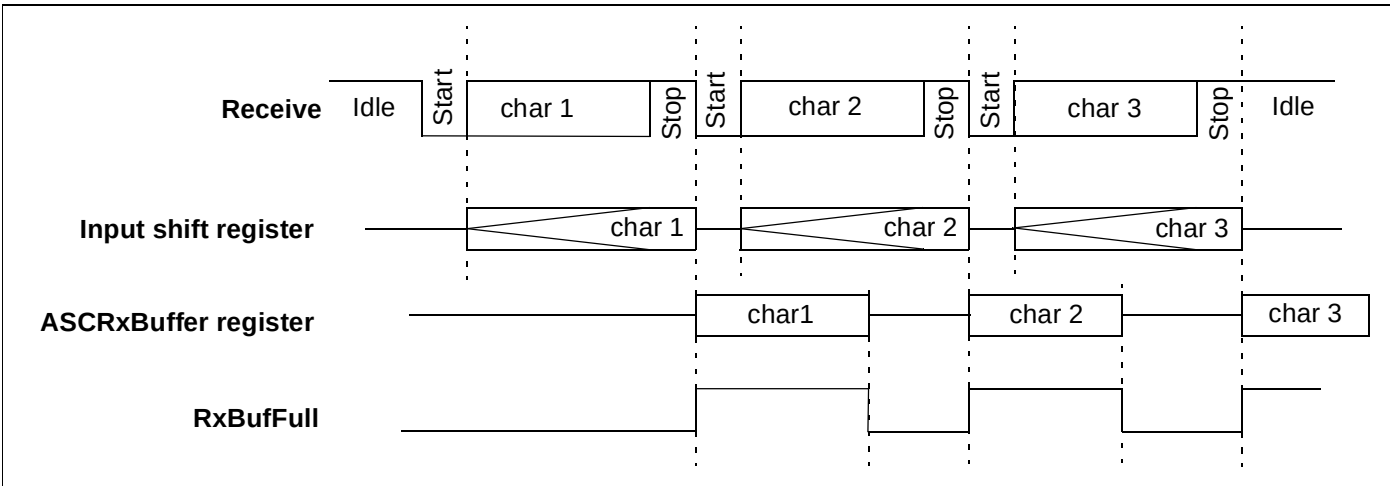


Figure 164 ASC reception



30.7 SmartCard operation

SmartCard mode is selected by setting the **SCEnable** bit in the **ASC_n_Control** register to 1. In SmartCard mode the **RxD** and **TxD** ports of the UART are both connected externally via a single bidirectional line to a smart card IO port. Characters are transferred to and from the smart card as 8-bit data frames with parity (see [Section 30.2](#)). Handshaking between the UART and the SmartCard ensures secure data transfer.

When the **SCEnable** bit in the **ASC_n_Control** register is set to 0, normal UART operation occurs.

SmartCard operation complies with the ISO SmartCard specification except where noted (see [Section 30.7.4](#)).

30.7.1 Control registers

ASC_n_Guardtime

A programmable 8-bit register **ASC_n_Guardtime** controls the time between transmitting the parity bit of a character and the start bit of any further bytes, or transmitting a 'nack' ('no acknowledge' signal, see [Section 30.7.2.1](#)). During the guardtime period the UART receiver is insensitive to possible start bits and the smart card is free to send 'nacks'.

Guardtime should always be set to at least 2.

ASC_n_Retries

A programmable 8-bit register **ASC_n_Retries** defines the number of times the UART will automatically try to send a 'nacked' character before giving up.

30.7.2 Transmission

In SmartCard mode FIFOs can be either enabled or disabled. If FIFOs are disabled, the UART transmission behaves according to NDC requirements.

30.7.2.1 Handshaking

When the UART is transmitting data to the smart card, the smart card can 'nack' ('not acknowledge') the transmission by pulling the line low 0.5 baud clock period into the Guardtime period and holding it low for at least 1 baud clock period. The UART should also be programmed in 1.5 stop bit mode, and since it receives what it transmits, nacks will be detected as receive framing errors.

30.7.2.2 Behavior with FIFOs enabled

At about 1 baud clock period into the Guardtime period, the UART knows whether or not the transmitted character has been 'nacked'. If no nack has been received and the Tx FIFO is not empty, the next character is transmitted after the guardtime period.

If a transmitted character is nacked by the receiving UART, the character is retransmitted as soon as the Guardtime period expires (or if Guardtime is 2, an extra baud clock period later), and retransmission is attempted up to the number of retries set in the **ASC_n_Retries** register. If the last retry is also 'nacked' the Tx FIFO is emptied, putting the transmitter into an idle state, and the **Nacked** bit is set in the **ASC_n_Status** register.

Emptying of the FIFO causes an interrupt, which can be handled by software. The **Nacked** bit in the **ASC_n_Status** register can be reset by writing to the **ASC_n_TxReset** register.

All 'un-nacked' (successfully transmitted) data is looped-back into the receive FIFO. This FIFO can be read by software to determine the status of the data transmission.

30.7.2.3 Behavior with FIFOs disabled

When the SmartCard mode bit is set to 1, the following operation occurs.

- Transmission of data from the transmit shift register is guaranteed to be delayed by a minimum of 1/2 baud clock. In normal operation a full transmit shift register will start shifting on the next baud clock edge. In SmartCard mode this transmission is further delayed by a guaranteed 1/2 baud clock.
- If a parity error is detected during reception of a frame programmed with a 1/2 stop bit period, the transmit line is pulled low for a baud clock period after the completion of the receive frame, i.e. at the end of the 1/2 stop bit period. This is to indicate to the SmartCard that the data transmitted to the UART has not been correctly received.
- The assertion of the **TxEmpty** interrupt can be delayed by programming the **ASC_n_GuardTime** register. In normal operation, **TxEmpty** is asserted when the transmit shift register is empty and no further transmit requests are outstanding.
- The receiver enable bit in the control register is automatically reset after a character has been transmitted. This avoids the receiver detecting a 'nack' from the SmartCard as a start bit.

In SmartCard mode an empty transmit shift register triggers the guardtime counter to count up to the programmed value in the **ASC_n_GuardTime** register. **TxEmpty** is forced low during this time. When the guard time counter reaches the programmed value **TxEmpty** is asserted high.

The de-assertion of **TxEmpty** is unaffected by SmartCard mode.

30.7.3 Reception

Reception can be done with FIFOs either enabled or disabled. The behavior is the same as in normal (non-smartcard) mode except that if a parity error occurs, then providing the transmitter is idle, the UART will transmit a nack on the **TxD** for 1 ETU from the end of the received stop bit. **RxD** is masked when transmitting a nack, since **TxD** is tied to **RxD** and a nack must not be seen as a start bit.

30.7.4 Divergence from ISO SmartCard specification

This UART does not support guardtimes of 0 or 1, and does not have any special behavior for a guardtime of 255.

31 SmartCard interface

The SmartCard interface is designed to support only asynchronous protocol SmartCards as defined in the *ISO7816-3* standard. Limited support for synchronous SmartCards can be provided in software by using PIO bits to provide the clock, reset, and I/O functions on the interface to the card. Two SmartCard interfaces are supported on the STi5512.

The UART function of the SmartCard interface is provided by a UART (ASC). UART ASC0 can be used by SmartCard0 and ASC2 can be used by SmartCard1.

Each ASC used by a SmartCard interface must be configured as eight data bits plus parity, 0.5 or 1.5 stop bits, with SmartCard mode enabled. A 16-bit counter, the SmartCard clock generator, divides down either the CPU clock, or an external clock connected to a pin shared with a PIO bit, to provide the clock to the SmartCard. PIO bits in conjunction with software are used to provide the rest of the functions required to interface to the SmartCard. The inverse signalling convention, as defined in *ISO7816-3*, is handled in software, inverted data and most significant bit first. See [Asynchronous serial controller on page 239](#) for details of the ASC and [Parallel input/output on page 269](#) for details of the PIO ports.

31.1 External interface

The signals required by the SmartCard are given in this table.

Pin	Function
Clk	Clock for SmartCard.
I/O	Input or output serial data. Open drain drive at both ends.
RST	Reset to card.
Vcc	Supply voltage.
Vpp	Programming voltage.

Table 119 SmartCard pins

The signals provided on the STi5512 are given in this table.

Pin	In/Out	Function
ScClk	Out, open drain for 5V cards.	Clock for SmartCard.
ScClkGenExtClk	In.	External clock input to SmartCard clock divider.
ScDataOut	Out, open drain driver.	Serial data output. Open drain drive.
ScDataIn	In.	Serial data input.
ScRST	Out, open drain.	Reset to card.
ScCmdVcc	Out.	Supply voltage enable/disable.
ScCmdVpp	Out.	Programming voltage enable/disable.
ScDetect	In.	SmartCard detection.

Table 120 SmartCard interface pins

The **ScRST**, **ScCmdVpp**, **ScCmdVcc**, and **ScDetect** signals are provided by pins of the PIO ports. Programming the PIO pins of the port for alternative function modes connects the ASC **TXD** data signal to the **ScDataOut** pin with the correct driver type and the clock generator to the **ScClk** pin. Details of the PIO pin assignments can be found in [Table 17: Alternative function of PIO pins on page 24](#).

The ISO standard defines the bit times for the asynchronous protocol in terms of a time unit called an ETU which is related to the clock frequency received by the card. One bit time is of length one ETU.

The ASC transmitter output and receiver input need to be connected together externally. For the transmission of data from the STi5512 to the SmartCard, the ASC will need to be set up in SmartCard mode.

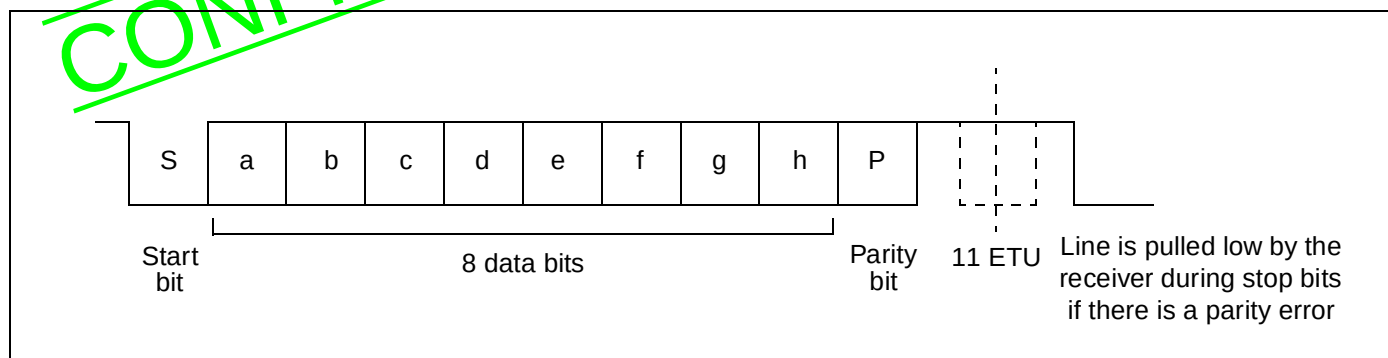


Figure 165 ISO 7816-3 asynchronous protocol

31.2 SmartCard clock generator

The SmartCard clock generator provides a clock signal to the connected SmartCard. The SmartCard uses this clock to derive the baud rate clock for the serial I/O between the SmartCard and another UART. The clock is also used for the CPU in the card, if present.

Operation of the SmartCard interface requires that the clock rate to the card is adjusted while the CPU in the card is running code, so that the baud rate can be changed or the performance of the card can be increased. The protocols that govern the negotiation of these clock rates and the altering of the clock rate are detailed in the *ISO7816-3* standard. The clock is used as the CPU clock for the SmartCard, so updates to the clock rate must be synchronized with the clock (**Clk**) to the SmartCard. This means the clock high or low pulse widths must not be shorter than either the old or new programmed value.

The clock generator clock source can be set to be either the system clock or an external pin. Two registers control the period of the clock and the running of the clock.

The **SclkVal** register determines the SmartCard clock frequency. The value given in the register is multiplied by 2 to give the division factor of the input clock frequency. The divider is updated with the new value for the divider ratio on the next rising or falling edge of the output clock.

The **SclkCon** register controls the source of the clock and determines whether the SmartCard clock output is enabled. The programmable divider and the output are reset when the enable bit is set to 0.

32 Synchronous serial controller

32.1 Introduction

The Synchronous Serial Controller (SSC) is a high-speed interface which can be used to communicate with a wide variety of serial memories, remote control receivers and other microcontrollers. There are a number of serial interface standards for these. Two SSCs are provided on the STi5512. The SSC supports all the features of the Serial Peripheral Interface (SPI) bus and also includes additional functions for the full support of the I²C bus. The general programmable features should also allow interface to other serial bus standards.

The SSC shares pins with the parallel input/output (PIO) ports. It supports full-duplex and half-duplex synchronous communication when used in conjunction with the PIO configuration.

The SSC uses three pins:

- serial clock **SClk**;
- serial data in/out **MRST**;
- serial data out/in **MTSR**.

To set the SSC PIOs to their alternate functions follow this sequence:

- 1 set SCL and MTSR as open drain bidirectional;
- 2 set MRST as input;
- 3 set SCL and MTSR to logic high;
- 4 set all SSC registers to slave mode;
- 5 only now, when the software is ready to accept data from the master, reprogram the PIO pins to their alternate output functions.

For I²C operation, **MRST** and **MTSR** can either be externally wired together, or just the **MTSR** pin can be used. These pins are connected to the SSC clock and data interface pins in a configuration which allows their direction to be changed when in master or slave mode (see [Pin connection and control on page 255](#)). The serial clock signal is either generated by the SSC (in master mode) or received from an external master (in slave mode). The input and output data are synchronized to the serial clock.

The following features are programmable: baud rate, data width, shift direction (heading control), clock polarity and clock phase. These features allow communications with SPI compatible devices.

In the SPI standard, the device can be used as a bus master, a bus slave, or can arbitrate in a multi-master environment for control of the bus. Many of these features require software support.

The SSC also fully supports the I²C bus standard and contains additional hardware (beyond the SPI standard) to achieve this. The extra I²C features include:

- multi-master arbitration;
- acknowledge generation;
- start and stop condition generation and detection;
- clock stretching.

These allow software to fully implement all aspects of the standard, such as master and slave mode, multi-master mode, 10-bit addressing and fast mode.

32.2 Basic operation

Control of the direction, either as input, output or bidirectional, of the **SClk**, **MTSR** and **MRST** pins is performed in software by configuring the PIO.

The serial clock output signal is programmable in master mode for baud rate, polarity and phase. This is described in [Clock generation on page 256](#).

The SSC works by taking the data frame (2 to 16 bits) from a transmission buffer and placing it into a shift register. It then shifts the data at the serial clock frequency out of the output pin and synchronously shifts in data coming from the input pin. The number of bits and the direction of shifting (MSB or LSB first) are programmable. This is described in [Shift Register on page 258](#).

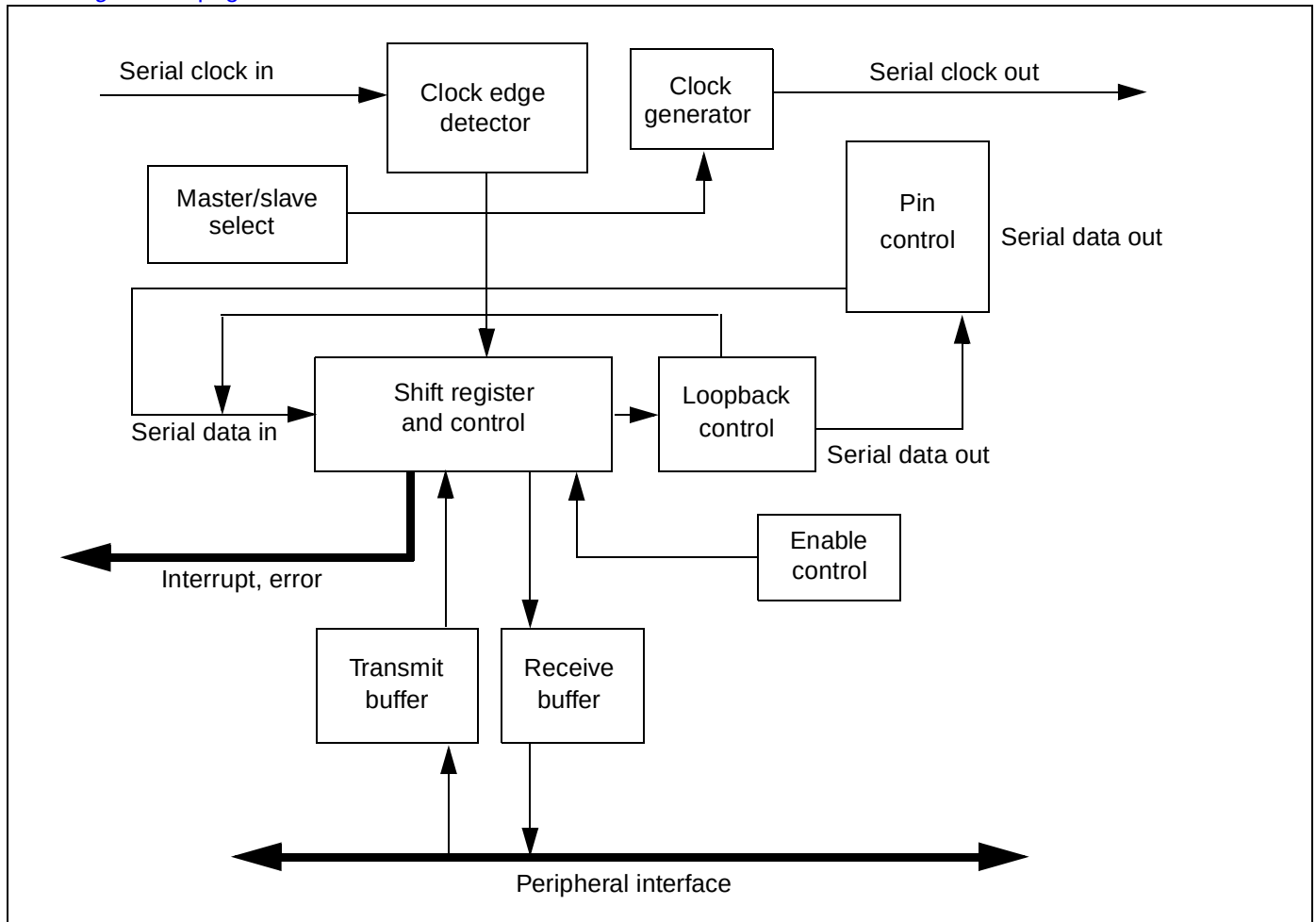


Figure 166 SSC architecture

After the data frame has been completely shifted out of the shift register, it transfers the received data frame into the receive buffer. The transmit and receive buffers are described in [Transmit and receive buffers on page 259](#). The SSC is therefore double buffered. This allows back-to-back transmission and reception of data frames up to the speed that interrupts can be serviced.

The SSC can also be configured to loop the serial data output back to serial data input in order to test the device without any external connections. This is described in [Loopback Mode on page 260](#).

The SSC can be turned on and off by setting the enable control. This is described in [Enabling Operation on page 260](#). It can also be set to operate as a bus master or as a bus slave device. This is described in [Master/Slave Operation on page 260](#).

The SSC generates interrupts in a variety of situations:

- when the transmission buffer is empty,
- when the receive buffer is full and
- when an error occurs. A number of error conditions are detected. These are described in [Error Detection on page 260](#).

There are additional hardware features which can be independently enabled in order to fully support the I²C bus standard when used in conjunction with a suitable software driver. The additional I²C hardware is described in [I²C operation on page 262](#).

32.2.1 Pin connection and control

To fully support the SPI standard, the interface presented at the pins is:

- a single clock pin, **SClk**, which is both an input and an output and
- two data pins, **MTSR**, and **MRST**, which are either inputs or outputs depending on whether the SSC is in slave or master mode.

In I²C mode only, the **MTSR** pin will be used as an input and output. This means only the **MTSR** pad needs to be used on the I²C data line. However, for backward compatibility, it is still possible to short **MTSR** and **MRST** data pins externally and achieve the same function (the **MRST** data output will be permanently driven to a high logic value and its input will be ignored).

These pads are provided by 3 bits of a standard PIO block. Their directions (input, output or bidirectional) can therefore be configured in software using the appropriate PIO settings. Consequently the SSC does not need to provide automatic control of data pad directions and does not need to provide a bidirectional clock port.

The connections between the SSC ports and the relevant PIO pins are illustrated in the figure below.

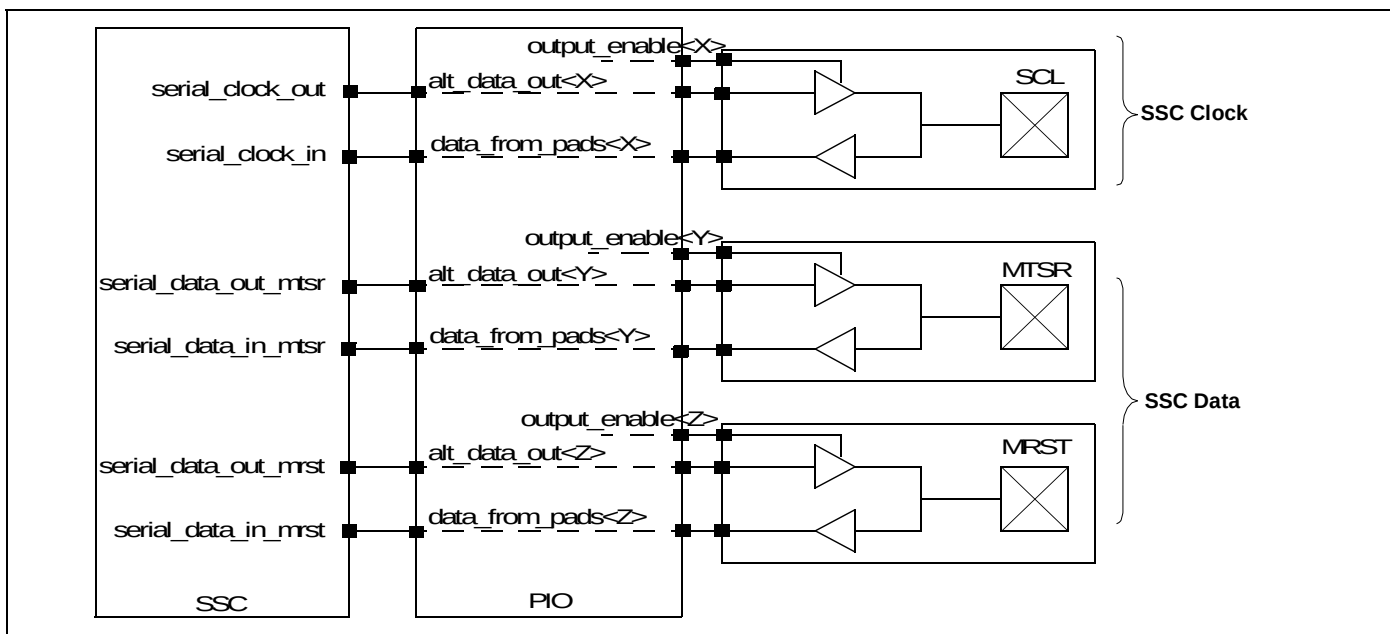


Figure 167 SSC to PIO connections

The pad control block inside the SSC determines which of the serial data input ports is used to read data from (depending on the master or slave mode). It also determines which of the serial data output ports to write data to (depending on the master or slave mode).

The deselected serial data output port is driven to ground (except in I²C mode when it is driven high). Therefore the user must ensure that the relevant PIO pad output enable is turned off depending on the master/slave status of the SSC.

It is up to the user to ensure that the PIO pads are configured correctly for direction and output driver type (e.g. push/pull or open drain).

Throughout the rest of this document, the data in and out ports will be referred to as **serial_data_out** and **serial_data_in**, where this is assumed to be the correct pair of pins dependent on the master or slave mode of the SSC.

32.2.2 Clock generation

If the SSC is configured to be the bus master, then it will generate a serial clock signal on the serial clock output port.

The clock signal can be controlled for polarity and phase and its period (baud rate) can be set to a variety of frequencies.

For I²C operation there are a number of additional clocking features. These are described in [I²C operation on page 262](#).

Clock Control

In master mode, the serial clock **SClk**, is generated by the SSC according to the setting of the phase bit **PH** and polarity bit **PO** in the control register **SSCnCon**.

The polarity bit **PO** defines the logic level the clock idles at i.e. when the SSC is in master mode but is between transactions. A polarity bit of 1 indicates an idle level of logic 1, 0 indicates idle of logic 0.

The phase bit **PH** indicates whether a pulse is generated in the first or second half of the cycle. This is a pulse relative to the idle state of the clock line; i.e. if the polarity is 0 then the pulse is positive going; if the polarity is 1 then the pulse will be negative going. A phase setting of 0 causes the pulse to be in the second half of the cycle while a setting of 1 causes the pulse to occur in the first half of the cycle.

The different combinations of polarity and phase are shown in the figure below.

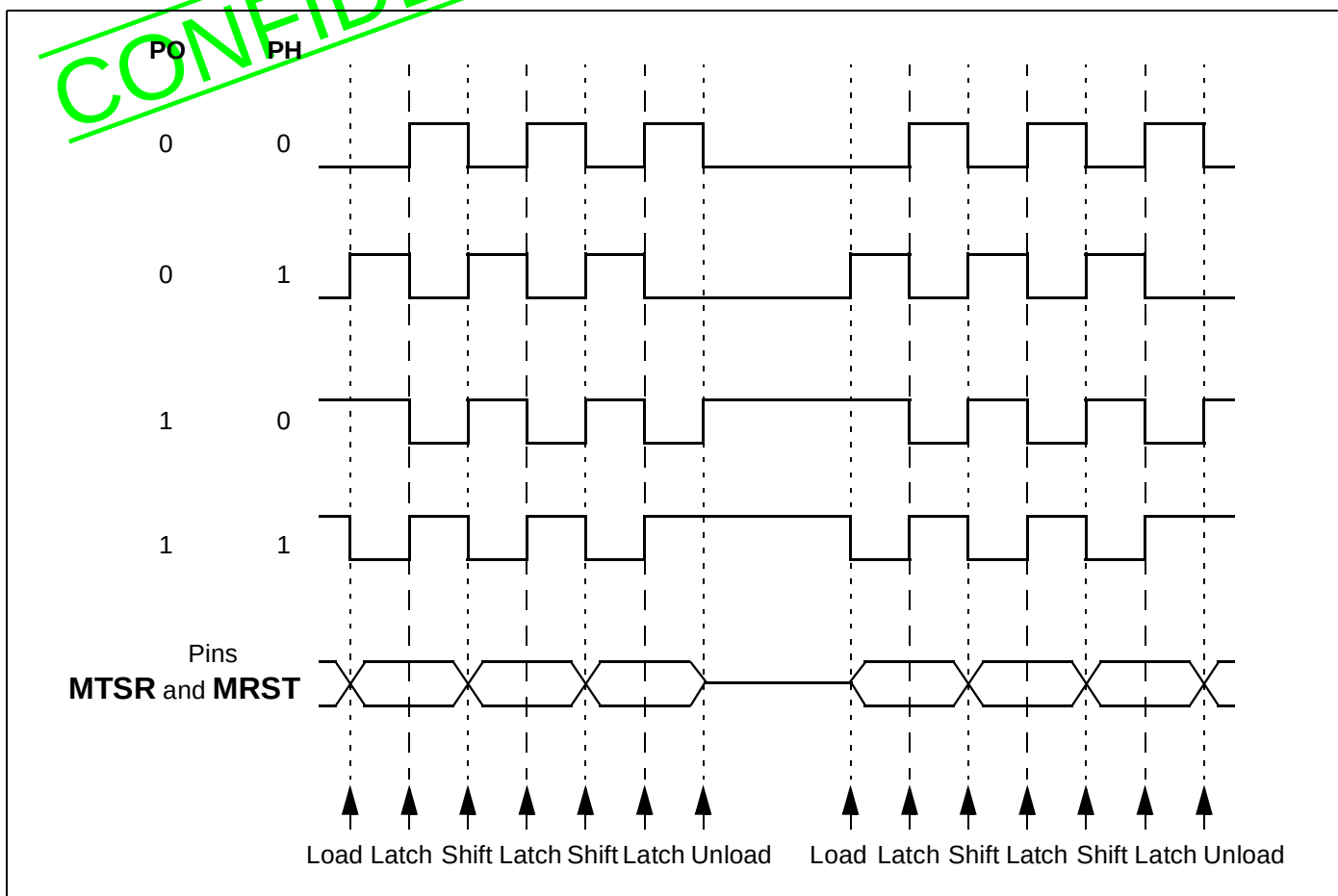


Figure 168 Polarity and phase combinations

The SSC will always latch incoming data in the middle of the clock period at the point shown in the diagram. With the different combinations of polarity and phase it is possible to generate or not generate a clock pulse before the first data bit is latched.

Shifting out of data occurs at the end of the clock period. At the start of the first clock period the shift register is loaded. At the end of the last clock period, the shift register is unloaded into the receive buffer.

32.2.3 Baud Rate Generation

The SSC can generate a range of different baud rate clocks in master mode. These are set up by programming the baud rate generator register **SSCBRG**.

In write mode this register is set up to program the baud rate as defined by the following formulae:

$$\text{Baudrate} = \frac{f_{\text{CPU}}}{2 \times \text{SSCBRG}} \quad \text{SSCBRG} = \frac{f_{\text{CPU}}}{2 \times \text{Baudrate}}$$

where **SSCBRG** represents the content of the baud rate generator register, as an unsigned 16-bit integer, and f_{CPU} represents the CPU clock frequency.

At a CPU clock frequency of 50 MHz the following baud rates can be generated as shown in the table below.

Baud Rate	Bit Time	Reload Value
Reserved. Use a reload value > 0	-	0000H
5 MBaud	200 ns	0005H
3.3 MBaud	300 ns	0007H
2.5 MBaud	400 ns	000AH
2.0 MBaud	500 ns	000CH
1.0 MBaud	1 us	0019H
100 KBaud	10 us	00FAH
10 KBaud	100 us	09C4H
1.0 KBaud	1 ms	61A8H

Table 121 Baud rates and bit times for different SSCBRG reload values

The value in **SSCnBRG** is used to load a counter at the start of each clock cycle. The counter counts down until it reaches 1 and then flips the clock to the opposite logic value. Consequently, the clock produced is twice the **SSCnBRG** number of CPU clock cycles.

In read mode the **SSCnBRG** register will return the current count value. This can be used to determine how far into each half cycle the counter is.

32.2.4 Shift Register

The shift register is loaded with the data in the transmit buffer at the start of a data frame. It then shifts data out of the serial output port and data in from the serial input port.

The shift register can shift out LSB first or MSB first. This is programmed by the heading control bit **HB** in the control register **SSCnCon**. A logic 1 indicates that the MSB will be shifted out first and a logic 0 that the LSB will shift first.

The width of a data frame is also programmable from 2 bits to 16 bits. This is set by the **BM** bit field of the control register. A value of 0000 is not allowed. Subsequent values set the bit width to the value plus one; for example 0001 sets the frame width to 2 bits and 1111 sets it to 16 bits.

When shifting LSB first, data comes into the shift register at the MSB of the programmed frame width and is taken out of the LSB of the register. When shifting in MSB first, data is placed into the LSB of the register and taken out of the MSB of the programmed data width. This is shown for a 9 bit data frame in the figure below/

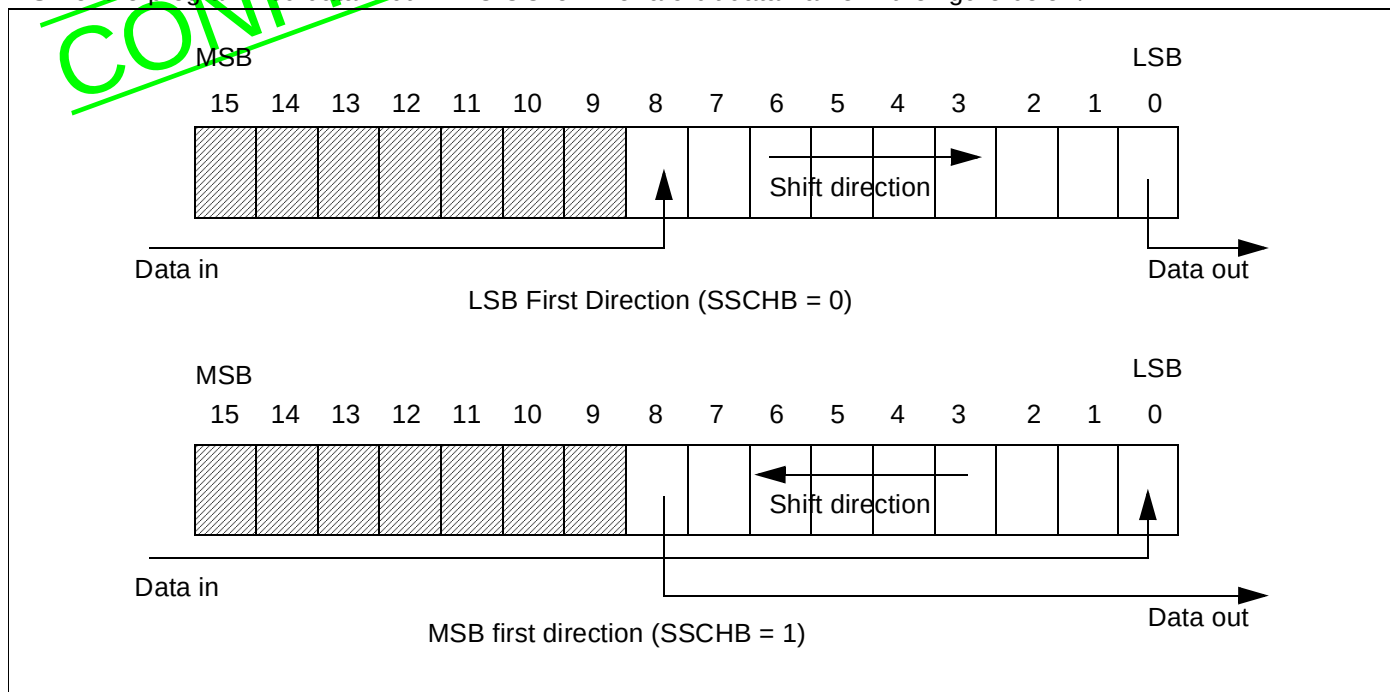


Figure 169 9-bit data frame shifting

The shift register shifts at the end of each clock cycle. The clock pulse for shifting is presented to it from the clock generator (see [Clock generation on page 256](#)). This is regardless of the polarity or phase of the clock.

When a complete data frame has been shifted, the contents of the shift register (i.e. all bits shifted into the register), is loaded into the receive buffer.

There are some additional controls required on the shifting operation to allow full support of the I²C bus standard. These are described in [I2C operation on page 262](#).

32.2.5 Receive Data Sampling

The data received by the SSC is sampled after the latching edge of the input clock, the latching edge being determined by the programming of the polarity and phase bits.

The data value which is finally latched is determined by taking 3 data samples at the 3rd, 4th and 5th system clock periods after the latching data edge. The data value is determined from the predominant data value in the 3 samples. This gives an element of spike suppression.

32.2.6 Transmit and receive buffers

The transmit and receive buffers are used to allow the SSC to do back-to-back transfers; i.e. continuous clock and data transmission.

The transmit buffer **SSCnTBuf** is written with the data to be sent out of the SSC. This is loaded into the shift register for transmission. Once this has been performed, the **SSCnTBuf** is available to be loaded again with a new data frame. This is indicated by the assertion of the transmit interrupt request status bit SSCTIR, which indicates that the transmit buffer is empty. This will cause an interrupt if the transmit buffer empty interrupt is enabled, by setting the **TIEN** bit in the interrupt enable register **SSCnIEn**.

A transmission is started in master mode by a write to the transmit buffer. This starts the clock generation circuit and loads the shift register with the new data.

Continuous transfers of data are therefore possible by reloading the transmit buffer whenever the interrupt is received. The software interrupt routine has the length of time for a complete data frame in order to refill the buffer before it is next emptied. If the transmit buffer is not reloaded in time when in slave mode, a transmit error condition **TE** (see [Error Detection on page 260](#)) is generated.

The number of bits to be loaded into the transmit buffer is determined by the frame data width selected in the control register bit **BM**. The unused bits are ignored.

The receive buffer **SSCnRBuf** is loaded from the shift register when a complete data frame has been shifted in. This is indicated by the assertion of the receive interrupt request status bit **RIR**, which indicates that the receive buffer is full. This will cause an interrupt if the receive buffer full interrupt is enabled, by setting the **RIEN** in the interrupt enable register.

The CPU should then read out the contents of this register before the next data frame has been received otherwise the buffer will be reloaded from the shift register over the top of the previous data. This will be indicated as a receive error condition **RE** (see [Error Detection on page 260](#)).

The number of bits which will be loaded into the receive buffer is determined by the frame data width selected in the control register **BM**. The unused bits are not valid and should be ignored.

32.2.7 Loopback Mode

A loopback mode is provided which connects the **serial_data_out** to **serial_data_in**. This allows software testing to be performed without the need for an external bus device. This mode is enabled by setting the **LPB** bit in the control register. A setting of logic 1 enables loopback, logic 0 puts the SSC into normal operation.

32.2.8 Enabling Operation

The transmission and reception of data by the SSC block can be enabled or disabled by setting the **EN** bit in the control register. A setting of logic 1 turns on the SSC block for transmission and reception. Logic 0 prevents the block from reading or writing data to the serial data input and output ports.

32.2.9 Master/Slave Operation

The control of a number of the features of the SSC depends on whether the block is in master or slave mode. For example, in master mode the SSC will generate the serial clock signal according to the setting of baud rate, polarity and phase. In slave mode, no clock is generated and instead the assumption is made that an external device is generating the serial clock.

Master or slave mode is set by the **MS** bit in the control register. A setting of logic 0 means the SSC is in slave mode, a setting of logic 1 puts the device into master mode.

32.2.10 Error Detection

A number of different error conditions can be detected by the SSC. These are related to the mode of operation (master or slave, or both).

On detection of any of these error conditions a status flag will be set in the status register, **SSCnStat**. Also, if the relevant enable bit is set in the interrupt enables register **SSCnIE**, then an error interrupt will be generated from the SSC.

The different error conditions are described below.

Transmit Error

A transmit error can only be generated in slave mode. It indicates that a transfer has been initiated by a remote master device *before* a new transmit data buffer value has been written in to the SSC.

In other words, the error occurs when old transmit data is going to be transmitted from the slave. (This could cause data corruption in the half-duplex open drain configuration.)

The error condition is indicated by the setting of the **TE** bit in the status register. An interrupt will be generated if the **TEEN** bit is set in the interrupt enables register.

The transmit error status bit (and the interrupt, if enabled) is cleared by the next write to the transmit buffer.

Receive Error

A receive error can be generated in both master and slave modes. It indicates that a new data frame has been completely received into the shift register and has been loaded into the receive buffer *before* the existing receive buffer contents have been read out. Consequently, the receive buffer has been overwritten with new data and the old data is lost.

The error condition is indicated by the setting of the **RE** bit in the status register **SSCnStat**. An interrupt will be generated if the **REEN** bit is set in the interrupt enables register.

The receive error status bit (and the interrupt, if enabled) is cleared by the next read from the receive buffer.

Phase Error

A phase error can be generated in master and slave modes. This indicates that the data received at the incoming data pin (**MRST** in master mode or **MTSR** in slave mode) has changed during the time from one sample before the latching clock edge and two samples after the edge.

The data at the incoming data pin is supposed to be stable around the time of the latching clock edge, hence the error condition. Each sample occurs at the CPU clock frequency. The sampling scheme is shown in the figure below.

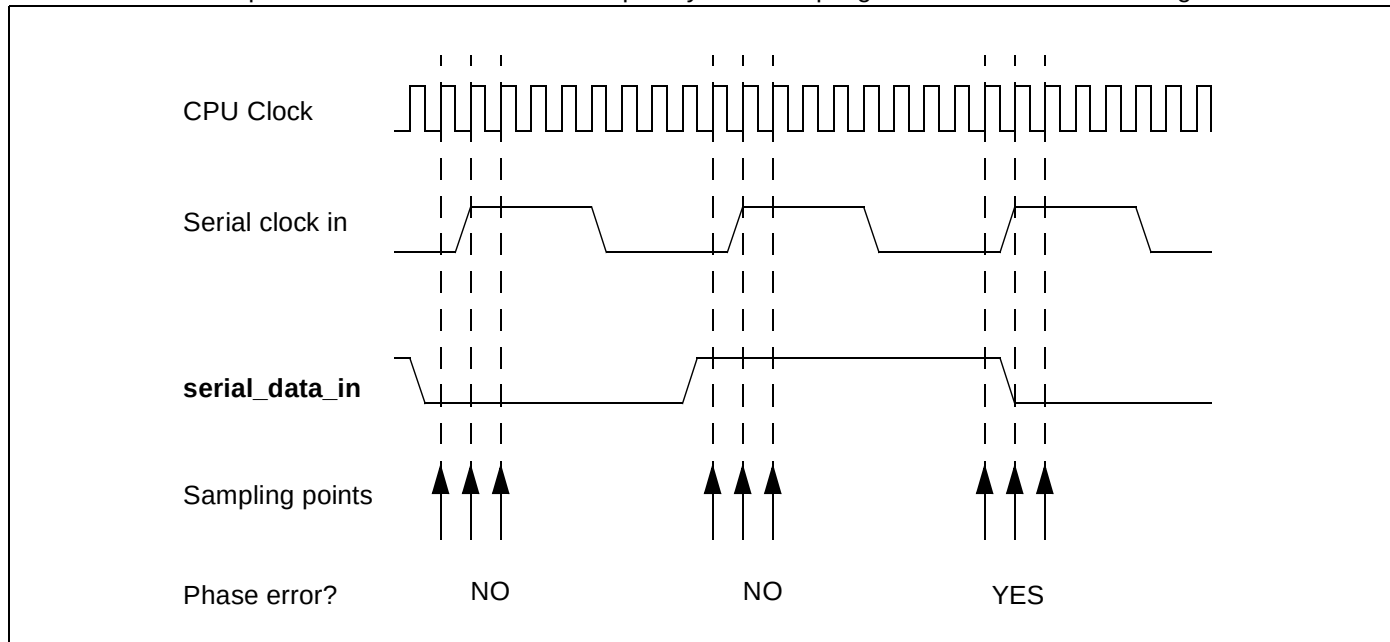


Figure 170 Sampling scheme

The error condition is indicated by the setting of the **PE** bit in the status register. An interrupt will be generated if the **PEEN** bit is set in the interrupt enables register. The phase error status bit (and the interrupt, if enabled) is cleared by the next read from the receive buffer.

32.2.11 Interrupt Mechanism

The SSC can generate a variety of different interrupts. They can all be enabled or disabled independently of each other. All the enabled interrupt conditions are OR-ed together to generate a global interrupt signal.

To determine which interrupt condition has occurred, a status register **SSCStat** is provided which includes a bit for each condition. This is independent of the interrupt enables register **SSC/IEn**, and determines whether the condition asserts one or more of the interrupt signals.

32.3 I²C operation

This section describes the additional hardware features which are implemented in order to allow full support for the I²C bus standard.

The architecture of the I²C including all the I²C hardware additions is shown in the figure below.

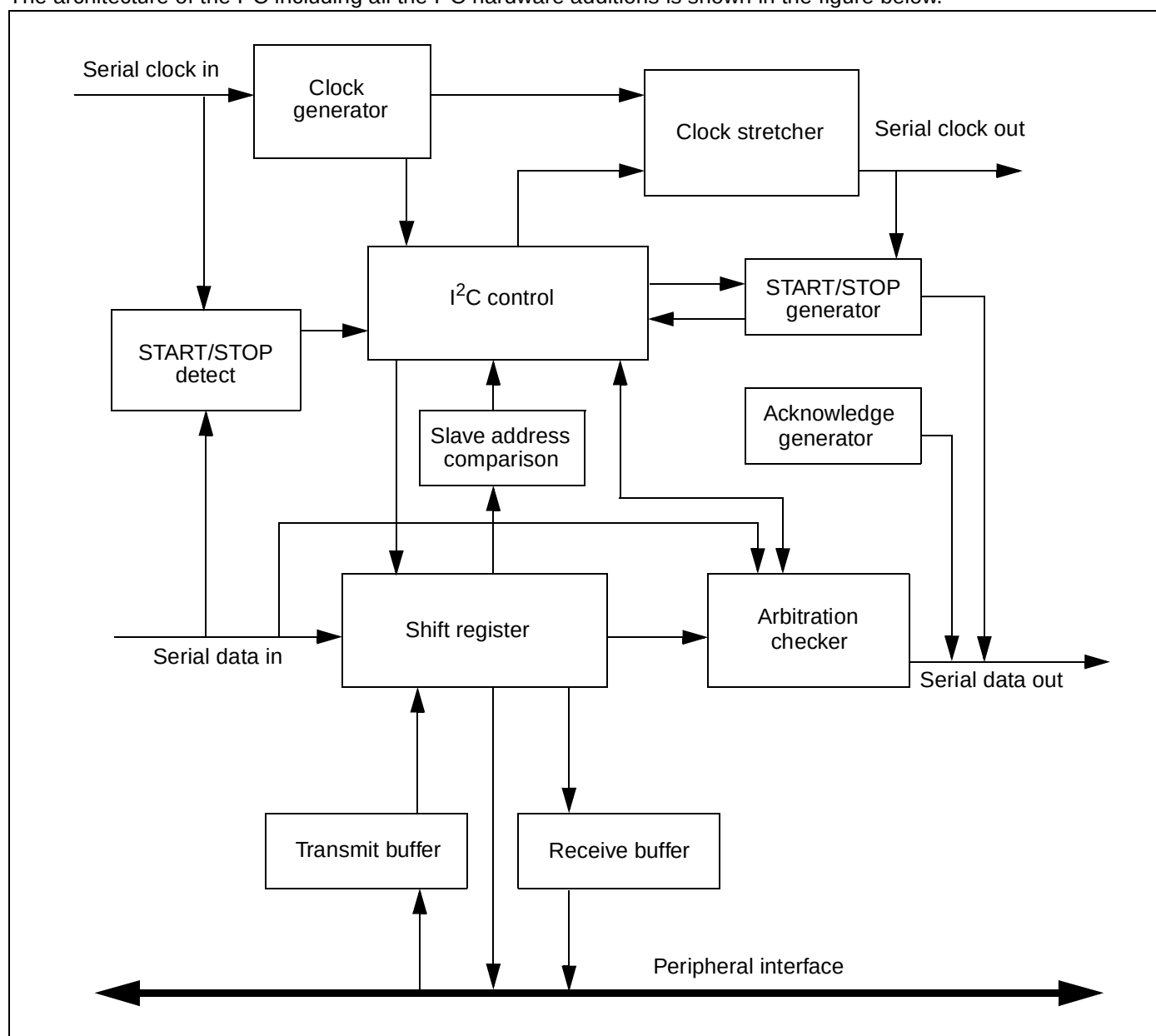


Figure 171 I²C architecture

32.3.1 I²C control

There are a number of features of the I²C-bus protocol which require special control:

- To allow slow slave devices to be accessed and to allow multiple master devices to generate a consistent clock signal, a clock synchronization mechanism is specified.
- START and STOP conditions must be recognized when in slave mode or multi-master mode. A START condition initiates the address comparison phase. A STOP condition indicates that a master has completed transmission and that the bus is now free.
- In slave mode (and in multi-master configurations), it is necessary to determine if the first byte received after a START condition is the address of the SSC. If it is, then an acknowledge must be generated in the 9th bit position.

Subsequently, an interrupt must be generated to inform the software that the SSC has been addressed as a slave device and therefore that it needs to either send data to the addressing master or to receive data from it.

In addition to normal 7 bit addressing, there is an extended 10 bit addressing mode where the address is spread over 2 bytes. In this mode, the SSC must compare 2 consecutive bytes with the incoming data after a START condition. It must also generate acknowledge bits for the first and second bytes automatically if the address matches.

The 10 bit addressing mode is further complicated by the fact that if the slave has been previously addressed for **writing** with the full 2 byte address, the master can issue a repeated START condition and then transmit just the first address byte for a **read**. The slave therefore must remember that it has already been addressed and must respond.

- In order for the software interrupt handler to have time to service interrupts, the SSC can hold the clock line LOW until the software releases it. This is called clock stretching.
- In master mode the SSC must begin a transmission by generating a START condition and must end transmission by generating a STOP condition. In multi-master configurations a START condition should not be generated if the bus is already busy; i.e. a START condition has already been received.
- When the SSC is receiving data from another device, it must generate acknowledge bits in the 9th bit position. However, when receiving data as a master, the last byte received must NOT be acknowledged. This only applies to data bytes; when operating as a slave device the SSC should always acknowledge a matching address byte; i.e. the first byte after a START condition.
- In multi-master configurations, arbitration must take place because it is not possible to determine if another master is also trying to transmit to the bus; i.e. the START conditions were generated within the allowed time frame.

Arbitration involves checking that the data being transmitted is the same as the data received. If this is not the case, then we have lost arbitration. The SSC must then continue to transmit a HIGH logic level for the rest of the byte to avoid corrupting the bus.

It is also possible that, having lost arbitration, we are being addressed as a slave device. So the SSC must then go into slave mode and compare the address in the normal fashion (and generate an acknowledge if we are addressed).

After the byte plus acknowledge the SSC must indicate to the software that we have lost arbitration by setting a flag.

All of these features are provided in the SSC design. They are controlled by the I²C control block which interacts with various other modules to perform the protocols.

In order to program for I²C mode, a separate control register **SSCnI2C** is provided. To perform any of the I²C hardware features, the I²C control bit **I2CM**, must be set in this register. When the I²C control bit is set, the clock synchronization mechanism is always enabled (see [Clock synchronization on page 264](#)). When the I²C control bit is set, the START and STOP condition detection is performed.

To program the slave address of the SSC the slave address register, **SSCnSIAd** must be written to with the address value. In the case of 7-bit addresses, only 7 bits should be written. For 10-bit addressing, the full 10 bits are written to. The SSC then uses this register to compare the slave address transmitted after a START condition (see [Slave Address](#)

Comparison on page 266). To perform 10-bit address comparison and address acknowledge generation, the 10-bit addressing mode bit **AD10** must be set in the **SSCnI2C** register (see *Slave Address Comparison on page 266*).

The clock stretching mechanism is enabled for various interrupt conditions when the I²C control enable bit **I2CM** is set (see *Clock Stretching on page 266*).

To generate a START condition, the I²C START condition generate bit **STRTG**, must be set (see *START/STOP condition generation on page 267*). To generate a STOP condition, the I²C STOP condition generate bit **STOPG**, must be set (see *START/STOP condition generation on page 267*).

To generate acknowledge bits (i.e. a LOW data bit), after each 8 bit data byte when receiving data, the acknowledge generation bit **ACKG**, must be set. When receiving data as a master, this bit must be reset to 0 before the final data byte is received, thereby signalling to the slave to stop transmitting (see *Acknowledge bit generation on page 267*).

To indicate to the software that various situations have arisen on the I²C-bus, a number of status bits are provided in the status register **SSCnStat**. In addition, some of these bits can generate interrupts if corresponding bits are set in the interrupt enable register **SSCnIEn**.

To indicate that the SSC has been accessed as a slave device, the addressed as slave bit **AAS**, is set. This will also cause an interrupt if the **AASEN** bit is set in the interrupt enable register.

The interrupt will occur after the SSC has generated the address acknowledge bit. In 10 bit addressing mode; the interrupt will occur after the second byte acknowledge bit, in the situations where 2 bytes of address are sent; or it will occur after the first byte acknowledge in the situation where only one byte is required.

Until the status bit is reset, the SSC will hold the clock line LOW (see *Clock Stretching on page 266*). This forces the master device to wait until the software has processed the interrupt.

The status bit and the interrupt are reset by reading from the receive buffer **SSCnRBuf**, when the slave is being sent data, and by writing to the transmit buffer **SSCnTBuf**, when the SSC needs to send data.

To indicate that a STOP condition has been received, when in slave mode, the STOP condition detected bit **STOP** is set. This will also cause an interrupt if the **STOPEN** bit is set in the interrupt enable register. The **STOP** interrupt and status bit will be reset by a read of the status register **SSCnStat**.

To indicate that the SSC has lost the arbitration process, when in a multi-master configuration, the arbitration lost bit **ARBL**, is set. This will also result in an interrupt if the **ARBLEN** bit is set in the interrupt enable register. The interrupt will occur immediately after the arbitration is lost.

Until the status bit is reset, the SSC will hold the clock line LOW at the end of the current data frame, (see *Clock Stretching on page 266*). This forces the winning master device to wait until the software has processed the interrupt.

The interrupt and status bit will be reset by a read of the status register **SSCnStat**.

To indicate that the I²C-bus is busy (i.e. between a START and a STOP condition), the I²C bus busy bit **BUSY** is set. This does not generate an interrupt.

32.3.2 Clock synchronization

The I²C standard defines how the serial clock signal can be stretched by slow slave devices and how a single synchronized clock is generated in a multi-master environment. The clock synchronization of all the devices is performed as follows.

All master devices start generating their low clock pulse when the external clock line goes low (this may or may not correspond with their own generated high to low transition).

They count out their low clock period and when finished attempt to pull the clock line high. However, if another master device is attempting to use a slower clock frequency, then it will be holding the clock line low, or if a slave device wants to, it can extend the clock period by deliberately holding the clock low.

As the output drive is open-drain, the slower clock will win and the external clock line will remain low until this device has finished counting its slow clock pulse, or until the slave device is ready to proceed.

In the mean time the faster master device will have detected a contradiction and will go into a wait state until the clock signal goes high again.

Once the external clock signal goes high, all the master devices will begin counting off their high clock pulse. In this case the first master to finish counting will attempt to pull the external clock line low and will win (because of the open drain line). The other master devices will detect this and will abort their high pulse count and switch to counting out their low clock pulse.

Consequently, the faster master device will determine the length of the high clock pulse and the slowest master or slave device will determine the length of the low clock pulse.

This results in a single synchronized clock signal which all master and slave devices then use to clock their shift registers.

The synchronization and stretching mechanism is shown in the figure below.

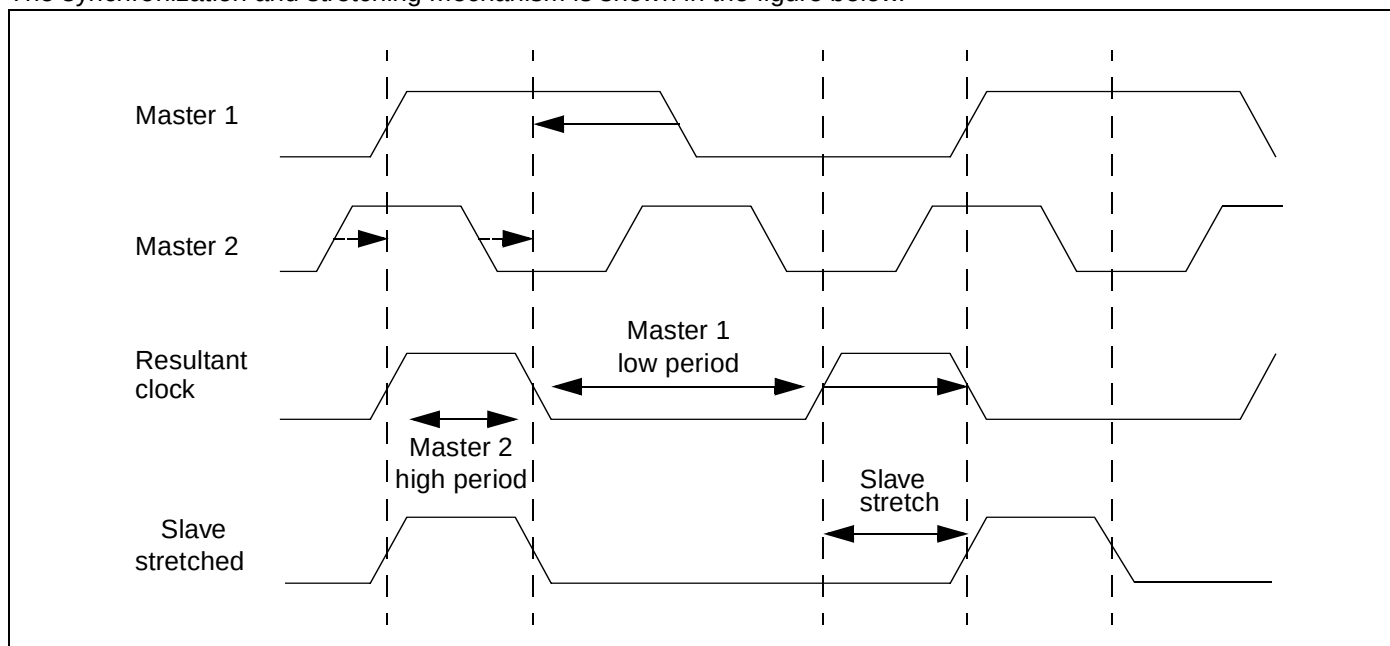


Figure 172 Synchronization and stretching

The SSC implements this clock synchronization mechanism when the I²C control bit SSCI2C, is enabled.

32.3.3 START/STOP condition detection

START/STOP conditions are only generated by a master device. A slave device must detect the START condition and expect the next byte (or 2 bytes in 10 bit addressing) to be a slave address. A STOP condition is used to signal when the bus is free.

A START condition occurs when the transmit/receive data line changes from high to low DURING the high period of the clock line. It indicates that a master device wants control of the bus. In a single master configuration, it will automatically get control. In a multi-master configuration, it begins to transmit as part of the arbitration procedure, and may or may not get control (see [Arbitration checking on page 268](#)).

A STOP condition occurs when the transmit/receive data line changes from low to high DURING the high period of the clock line. It indicates that a master device has relinquished control of the bus (the bus will be free a specified time after the stop condition).

An additional piece of hardware is provided on the SSC to detect START and STOP conditions. This is necessary in slave mode as detection cannot be performed in time merely by programming the PIO pads. This is because there is not sufficient time for a software interrupt between the end of the START condition and the beginning of the data transmitted by a remote master.

START and STOP conditions are detected by sampling the data line continuously when the clock line is high. If four consecutive samples are "1000" then a START condition is detected. If the four samples are "0111" then a STOP condition is detected. Four samples are required in order to suppress spikes from causing a false condition to be seen (fast-mode I²C requires a spike suppression of 50 ns to be incorporated).

The START and STOP condition detection is enabled when the I²C control bit **I2CM** is set in the I²C control register.

When a START condition is triggered, the SSC will inform the I²C control block which will then initiate the address comparison phase.

When a STOP condition is triggered, the SSC will set the **STOP** bit in the status register. It will also generate an interrupt if the **STOPDEN** bit is set in the interrupt enable register.

The interrupt and the status bit will be cleared when the status register is read.

32.3.4 Slave Address Comparison

After a START condition has been detected, the SSC goes into the address comparison phase.

It receives the first 8 bits of the next byte transmitted and compares the first 7 bits against the address stored in the slave address register **SSCnSIAd**. If they match, the address comparison block indicates this to the I²C control block.

This will generate an acknowledge bit in the next bit position and set the addressed as slave bit **AAS** in the status register. An interrupt will then be generated after the acknowledge bit if the addressed as slave enable bit **AASEN** is set in the interrupt enables register.

The 8th bit of the first byte indicates whether the SSC will be written to (LOW) or read from (HIGH). This will be used by the control block to determine if it needs to acknowledge the following data bytes (i.e. when receiving data).

When 10-bit addressing mode is selected by setting the 10 bit addressing bit **AD10**, the first 7 bits of the first data byte will be compared against "11110XX", where XX are the 2 most significant bits of the 10-bit address stored in the slave address register.

The read/write bit then determines what to do next.

If the read/write bit is LOW, indicating a write, an acknowledge must be generated for the byte. The addressed as slave status bit and interrupt however are not yet asserted so, instead, the address comparator will wait for the next data byte and will compare this against the 8 least significant bits of the slave address register.

If this also matches, then the SSC is being addressed, so the second byte is acknowledged and the addressed as slave bit is set. An interrupt also occurs after the acknowledge bit if the addressed as slave interrupt enable is set.

On the other hand if the first byte sent has the read/write bit HIGH, then the SSC will only acknowledge it if it has previously been addressed and a STOP condition has not yet occurred (i.e. the master has generated a repeated START condition). In this case the addressed as slave bit is set after the first byte plus acknowledge and an interrupt is generated if the interrupt enable is set. The second byte in this case will be sent by the SSC as this is a read operation.

In all cases if the address does not match, then the SSC ignores further data until a STOP condition is detected.

32.3.5 Clock Stretching

The I²C standard allows slave devices to hold the clock line low if they need more time to process the data being received (see [Clock synchronization on page 264](#)). The SSC takes advantage of this by inserting extended clock low periods. This is done to allow a software device driver to process the interrupt conditions when in slave mode.

The clock stretching mechanism is used in the following situations:

- When the SSC has been addressed as a slave device and the interrupt has been enabled. The clock stretch occurs immediately after the first byte with acknowledge, after a START condition has occurred (or in the case of 1-bit addressing this might occur after the second byte plus acknowledge). This gives the software interrupt routine time to initialize for transmission or reception of data. The clock stretch is cleared by writing 0x1FF to the transmit buffer register.
- When the SSC is in slave mode and is transmitting or receiving. The clock stretch occurs immediately after each data byte plus acknowledge. When transmitting, this allows the software interrupt routine to check that the master has acknowledged before writing the next data byte into the transmit buffer. If no acknowledge is received, then the software must stop transmitting bytes. When receiving, it allows the software to read the next data byte before the master starts to send the next one. The clock stretch is cleared by a write to the transmit buffer when transmitting and by a read from the receive buffer when receiving.
- When the SSC loses arbitration. The clock stretch occurs immediately after the current data byte and acknowledge have been performed. This gives the software time to abort its current transmission and prepare to retry after the next STOP condition. The clock stretch is cleared by a read from the receive buffer.

If a clock stretching event occurs but no relevant interrupt is enabled then the clock will be stretched indefinitely. Hence it is important that the correct interrupts are always enabled.

32.3.6 START/STOP condition generation

As a master device the SSC must generate a START condition before transmission of the first byte can start. It may also generate repeated START conditions. It must complete its access to the bus with a STOP condition.

Between STOP and START conditions, the bus is free and the clock and data lines must be held high. The I²C control block determines this and instructs the START/STOP generator to hold the lines high between transactions.

The START/STOP generator is controlled by the START condition generate bit **STRTG** and the STOP condition generate bit **STOPG** in the I²C control register.

The generator will pull the **serial_data_out** line LOW during the high period of the clock to produce a START condition. In the case of a STOP condition it will pull the data line HIGH.

However, a START condition will only be generated if the bus is currently free (i.e. the **BUSY** bit in the status register is LOW). This is to prevent the SSC from generating a START condition when another master has just generated one.

If a START condition cannot be generated because the bus is busy, then the generator will force the arbitration checker to generate an arbitration lost interrupt and prevent data from being transmitted for the next byte. The software interrupt handler is therefore informed of the aborted transmission when servicing the interrupt.

To properly generate the timing waveforms of the START and STOP conditions, the SSC contains a timing counter. This ensures the minimum setup and hold times are met with some additional margin.

32.3.7 Acknowledge bit generation

For I²C operation, it is required to both detect acknowledge bits when transmitting data, and to generate them when receiving data.

An acknowledge bit must be transmitted by the receiver at the end of every 8 bit data frame. The transmitter must verify that an acknowledge bit has been received before continuing.

An acknowledge bit will not be generated by a master receiver for the last byte it wishes to receive. This "not acknowledge" is used by the slave device to determine when to stop transmission.

The acknowledge bit is generated by the receiver after the 8 data bits have been transferred to it. In the 9th clock pulse, the transmitter holds the data line high and the receiver must pull the line low to acknowledge receipt. If the receiver is unable to acknowledge receipt, then the master will generate a stop condition to abort the transfer.

Acknowledge bits are generated by the SSC when the acknowledge generation bit, **ACK**, is set in the I²C control register. They are only generated when receiving data.

When in master mode and receiving data the **ACK** bit should be set to 0 before the last byte to be received. The SSC will automatically generate acknowledge bits when addressed as a slave device.

32.3.8 Arbitration checking

This situation only arises when two or more master devices generate a START condition within the minimum hold time of the bus standard. This generates a valid start condition on the bus with more than one master valid.

However, a master device cannot determine if two or more masters have generated a START condition, so arbitration is always enabled. The arbitration for who wins control of the bus is determined by which master is the first to transmit a low data bit on the data line when the other master wants to send a high bit. This master wins control of the bus. Therefore a master which detects a different data bit on its input to that which it transmitted must switch off its output stage for the rest of the 8 bit data byte, as it has lost the arbitration.

The arbitration scheme does not affect the data transmitted by the winning master. Consequently, arbitration proceeds concurrently with data transmission and valid data will have been received by the selected slave during the arbitration process. It is possible that the winning master is actually addressing the losing master and hence this device must respond as if it were a slave device.

Arbitration is implemented in hardware by comparing the transmitted and received data bits every cycle. Loss of arbitration is indicated by the setting of the **ARBL** arbitration lost error flag in the status register. An interrupt will also occur if the **ARBL**EN bit is set in the interrupt enables register.

Loss of arbitration also causes a clock stretch to be inserted. The interrupt and the clock stretch will occur immediately after the 8 bits plus acknowledge. The clock stretch is cleared when the software reads the receive buffer.

33 Parallel input/output

The STi5512 device has 40 bits of parallel input/output (PIO), configured in ports of eight bits. Each bit is programmable as an output, an input, or a bidirectional pin, or as an alternative function output pin. The alternative function connects signals from device peripherals to the pins of the device through the PIO. Details of the alternative function assignments can be found in [Chapter 4: Pin list](#) on page 19.

Each port of eight input bits can also be compared against a register and an interrupt generated when the value is not equal. Each of the ports operates as described in the rest of this chapter.

Output drivers for the PIO pins, both in PIO mode and the alternative function mode, can be programmed to be push-pull or open drain.

33.1 PIO registers

The PIO port can be controlled by registers which are mapped into the device address space. The registers for each port are grouped in a 4 Kbyte block, with the base of the block for port n at the address *PIOBaseAddress*. The value of *PIOBaseAddress* is given in the *Memory Map* chapter. The addresses of the registers are given in the tables as offsets from this address. During reset all the registers are reset to zero.

Each eight-bit PIO port has a set of eight-bit registers. Each of the eight bits of each register refers to the corresponding pin in the corresponding port. These registers hold:

- the output data for the port (**PnOut**)
- the input data read from the pin (**PnIn**)
- PIO bit configuration registers (**PnC0-2**)
- the two input compare function registers (**PnComp** and **PnMask**).

Each of the registers, except **PnIn**, is mapped onto two additional addresses so that bits can be set or cleared individually. The **Set_** register allows bits to be set individually. Writing a '1' in this register sets the corresponding bit in the associated register, a '0' leaves the bit unchanged. Similarly the **Clear_** register allows bits to be cleared individually. Writing a '1' in this register resets the corresponding bit in the associated register; a '0' leaves the bit unchanged.

34 IEEE 1284 port (PC parallel port)

The IEEE 1284 port 8 bit wide parallel interface supports a high speed data input/output to and from the set top receiver. It is capable of interfacing to a PC to the IEEE 1284 standard. The interface has a dedicated DMA controller to transfer data between memory and the port with little CPU overhead.

The IEEE 1284 specification¹ defines a standard for asynchronous, interlocked, bidirectional parallel communications between a host and a peripheral. The IEEE 1284 port supports all IEEE 1284 modes of communication (except EPP mode) with appropriate software control. The port has three additional non IEEE 1284 compliant modes to support transport stream output modes and allows software control of the port.

Data may be accessed and sourced from either internal registers or by a DMA transfer. DMA transfers are used where appropriate to increase throughput and decrease system load. DMA transfers are not word aligned and may transfer between 1 and 65535 bytes. The DMA may only operate in one direction at any one time.

An interrupt mechanism is used to indicate the port has completed a transfer or has an event which needs servicing.

The IEEE 1284 port supports the following main modes of operation:

- IEEE 1284 mode;
- transport stream mode;
- software control mode.

Each of these modes has associated modes. The modes are described in later sections of this chapter.

Note that the port only functions as a 1284 slave mode device in IEEE 1284 mode. The port then expects the other device to be a master mode device (I.E. a PC host). rather than another slave mode device, such as a printer. It is possible ,however to configure the port to be a master mode device, but this can only be used in software control mode.(Section 34.3.7 on page 276)

Power on, initialization and termination

The interface may be re-initialized at any time by the host; this produces an interrupt for the peripheral to respond to. The slave may request to terminate a communication, or request to interrupt the master, but will wait for acknowledgment when operating in IEEE 1284 mode.

1. IEEE Standard 1284-1994: IEEE Standard Signalling method for a Bidirectional Parallel Peripheral Interface for Personal Computers.

34.1 IEEE 1284 port pins

The port pins meet IEEE 1284 level 2 device requirements and are designed to directly drive an IEEE 1284 compliant cable with external matching resistors. The figure below shows how to interface the module to the cable.

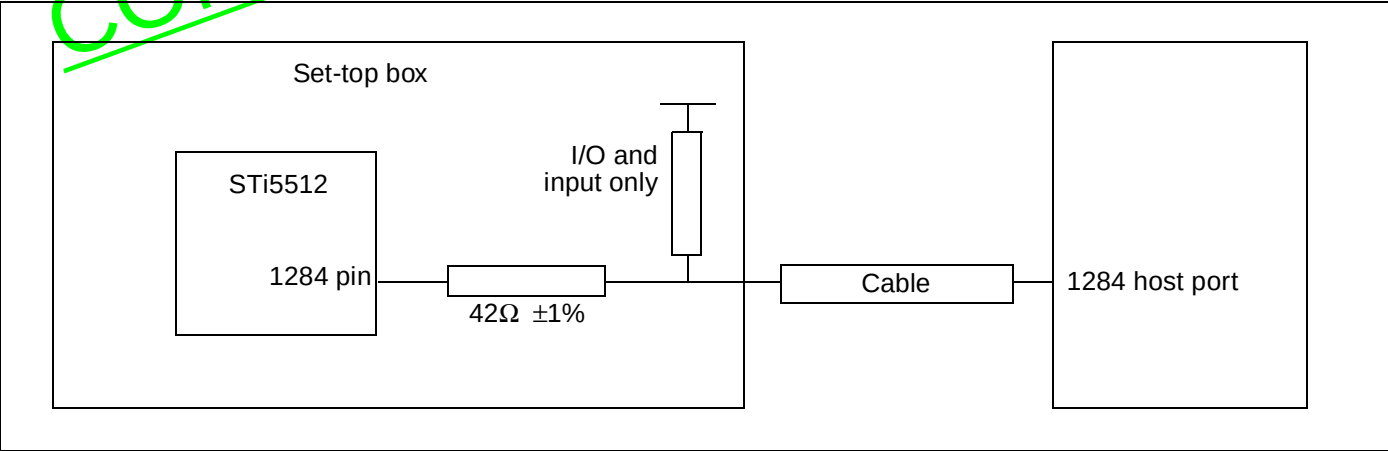


Figure 173 Interface to a cable

Pin	In/Out	Function
1284Data0-7	in/out	IEEE 1284 serial data.
1284notSelectIn	in	The function of these control pins is dependent on the mode of operation of the IEEE 1284 port, as shown in Table 123 below.
1284notInit	in	
1284notFault	out	
1284notAutoFd	in	
1284Select	out	
1284PErrror/ TSByteClkValid	out	
1284Busy/ TSPacketClk	out	
1284notAck/ TSByteClk	out	
1284notStrobe	in	
1284InnotOut (PIO3[7])	out*	IEEE 1284 data output enable for an external buffer.
1284PeriphLogicH (PIO4[3])	out*	Peripheral logic high.
1284HostLogicH (PIO4[4])	in*	Host logic high.

Table 122 IEEE 1284 port pins

* These pins have the PIO pin electrical characteristics and timings.

The nine control pins have different functions depending on the mode of operation of the port interface. The mapping of the IEEE 1284 port pins to the function of the pin in a specific mode is given in [Table 123](#) below. For full details of the IEEE 1284 signal functions in each mode refer to the *IEEE Standard 1284-1994*.

Pin	IEEE 1284 modes				Transport stream mode
	Compatible mode	Nibble mode	Byte mode	ECP mode	
1284notStrobe	nStrobe	HostClk	HostClk	HostClk	TSByteClk TSPacketClk TSByteClkValid
1284notAck	nAck	PtrClk	PtrClk	PeriphClk	
1284Busy	busy	PtrBusy	PtrBusy	PeriphAck	
1284PErr	pError	AckDataReq	AckDataReq	nAckReverse	
1284Select	select	XFlag	XFlag	XFlag	
1284notAutoFd	nAutoFD	HostBusy	HostBusy	HostAck	
1284notInit	nInit	high	high	nReverseRequest	
1284notFault	nFault	nDataAvail	nDataAvail	nPeriphRequest	
1284notSelectIn	nSelectIn	active	active	active	

Table 123 IEEE 1284 port control pin functions

34.2 IEEE 1284 mode

The IEEE 1284 port supports IEEE 1284 modes of communication, as defined below, with appropriate software control and use of DMA transfers where appropriate to increase throughput and decrease system load. For full details of the IEEE 1284 protocols and signal functions in each mode refer to the *IEEE Standard 1284-1994*.

Forward transfer means a transfer from the host to the peripheral; reverse transfer means a transfer from the peripheral to the host.

34.2.1 IEEE 1284 mode initialization

The **1284ModeEnable**, **1284PulseWidth** and **1284PinOut** registers must be set before entering any IEEE 1284 mode. The **1284PeriphLogicH** pin is forced high in all IEEE 1284 modes.

On entering the IEEE 1284 modes, the peripheral always completes an initialization sequence before starting in compatibility mode. If the **OverrideHostLogicH** bit in the **1284Control** register is not set then the part remains in this mode until the **1284HostLogicH** pin goes high.

It is the responsibility of the software driver to ensure that the **1284PeriphLogicH** pin setting is correct before entering the IEEE 1284 modes.

The status of the peripheral is indicated to the host using the values in the **1284PinOut** register. If the **Busy** bit is high then the peripheral will be busy on entering compatible mode and the values of the **1284PErr**, **1284Select** and **1284notFault** pins will reflect the values in the **1284PinOut** register. The value of the **1284Busy** and **1284notAck** pins are not under user control.

34.2.2 Compatibility mode

Compatibility mode supports forward transfers only. This mode is comparable to the 'Centronics Parallel Port' (CPP).

Compatibility mode is always enabled when IEEE 1284 mode is enabled. Following initialization, or reset by either the host or the peripheral, the port operates in this mode until the host negotiation allows the port to move to another mode. Following any protocol exceptions or termination requests the module returns to this mode.

The busy status of the peripheral in this mode is controlled by the **Busy** bit of the **1284PinOut** register. The peripheral becomes busy when a transfer occurs, or when the **Busy** bit of the **1284PinOut** register is set. If the **Busy** bit is high, the **1284PErr**, **1284Select** and **1284notFault** pins are driven to the value given in the **1284PinOut** register. The value of the **1284notAck** pin is not under user control.

34.2.3 Negotiation

The host may request that the IEEE 1284 compliant device change communication mode. This is done by placing an extensibility request on the data bus during negotiation mode. Negotiation may only be entered from compatibility mode. A negative response to a request will stall the port until the host terminates the transaction, and returns the port to compatibility mode.

The modes to which the module responds positively depends on the specific implementation and the **1284ModeEnable** register.

On entering this mode the **1284Busy** pin assumes the value in the **1284PinOut** register. The control of the other pins is dependent on the mode being entered, and whether data is available to be transferred.

34.2.4 Nibble mode

Nibble mode supports reverse transfers only. This is the most basic reverse transfer mode and is used as the reverse channel in conjunction with compatible mode. Nibble mode is always enabled if IEEE 1284 mode is enabled.

The data is transferred as 4-bit values on four of the IEEE 1284 control pins: **1284PErr**, **1284Busy**, **1284notFault**, **1284Select**. The **1284Busy** pin reflects the value in the **1284PinOut** register or the data value depending on the point in the transfer. The other pins are not under user control.

34.2.5 Byte mode

Byte mode supports reverse transfers only. It uses a similar protocol to nibble mode, but transfers the data as 8-bit values on the data bus (**1284Data0-7**).

The **1284Busy** pin reflects the value in the **1284PinOut** register. The other pins are not under user control.

34.2.6 ECP mode

ECP mode supports both forward and reverse transfers. The port supports run length encoding (RLE). The hardware allows access to channel and RLE data, and software support is provided. Expansion of incoming data using RLE encoding is supported in hardware and enabled using the **1284Control** register. All output RLE encoded data must be pre-encoded.

If channel or RLE information is passed to the DMA engines, a DMA error occurs.

The **1284notFault** pin reflects the value in the **1284PinOut** register and is expected to be used to trigger a host interrupt.

For the case when the IEEE 1284 port is not busy and a forward transfer is occurring, then the peripheral should ensure that when a token becomes available it is accepted from the IEEE 1284 port within 35 ms. Failure to do so may cause the host to signal a time-out error. If hardware RLE decode is enabled, the application should ensure that a complete decoded RLE sequence will be accepted within 35 ms.

The maximum RLE sequence length allowed by the IEEE 1284 standard is 128 bytes. The tokens may be accepted either by the DMA or register transfers.

34.2.7 Device identification

The peripheral asserts an interrupt to indicate a device identification request has occurred. Software will handle this request and return the device identification data stream. The protocol used to return the identification stream depends on the **1284ModeEnable** register.

34.2.8 Host reset

The interface may be re-initialized at any time by the host, this produces an interrupt for the peripheral to respond to. The slave may request to terminate a communication, or request to interrupt the master, but will wait for acknowledgment when operating in IEEE 1284 mode.

34.2.9 Termination

Following termination of a mode by the host, the peripheral will always return to compatible mode. The behavior of the **1284PErr**, **1284notFault** and **1284Select** pins is dependent on the value in the **1284PinOut** register, and will reflect the value in this register if the **Busy** bit is set.

If the **Busy** bit of the **1284PinOut** register is set, the peripheral will be busy on entering compatible mode. The peripheral will set the value of the **1284Busy** pin.

34.2.10 Data transfer rates

The data transfer rate in these modes is dependent on the host, operating mode and memory speed, and is expected to be limited by the host response time.

The DMA engine implements eight bytes of buffering for outgoing data, and four for incoming data.

34.3 Transport stream mode

The transport stream interface produces a byte wide output data stream compatible with the Link-IC protocol. The two alternate implementations of this output stream are defined below. The number of null byte transfers must be controlled by the driver software.

The following sections describe the pin and register functionality of the IEEE 1284 port in transport stream mode.

The value of the **1284PulseWidth**, **1284PinOut** and **1284PacketSize** registers must be set before entering transport mode.

34.3.1 TSByteClk

The data (**1284Data0-7**), **TSPacketClk** and **TSByteClkValid** are valid on the rising edge of this signal. The data, **TSPacketClk** and **TSByteClkValid** change on the falling edge of this clock. The clock is active when a valid data token is available on the data bus.

The minimum frequency of the byte clock depends on the value held in the **1284PulseWidth** register. This gives the delay in clock cycles between byte clock edge transitions. At 40 MHz, a value of 2 in this register produces a byte clock with a nominal 100 ns period.

In transport stream mode A, the byte clock is free running and **TSByteClkValid** going high indicates that the clock is active. The frequency of the clock is fixed, and in the case of memory stalls, **TSByteClkValid** going low indicates there is no data packet to transmit.

In transport stream mode B, a rising transition only occurs on this clock when valid information is available to transmit. The frequency of the clock may change in the event of a memory stall. At the end of a packet transfer the clock becomes free running until the next packet transfer is started.

34.3.2 TSByteClkValid

This validates the byte clock and indicates that the **TSByteClk** transition is valid.

34.3.3 TSPacketClk

This is high during a packet transfer. The length of a packet is defined by the **1284PacketSize** register. A packet transfer commences when valid data has been read from memory and is available on the data bus. It completes when the number of bytes defined by the **1284PacketSize** register have been transferred.

34.3.4 Packet size register

A write to the **1284PacketSize** register defines the number of bytes within a packet.

The IEEE 1284 packet size count is restarted after the required number of bytes have been transferred, and if a DMA transfer of greater than one packet is started, the second packet is transferred with a single null byte between packets. If a DMA transfer transfers an incomplete packet, the module will stall until more bytes become available.

The count may be restarted by writing to the **Reset** bit in the **1284Control** register or by writing to the **1284PacketSize** register.

34.3.5 Transfer stream mode A and B examples

Figure 174 and Figure 175 give examples of a single packet transfer in transport stream modes A and B respectively. The number of null bytes depends on the time taken to start a second DMA transfer.

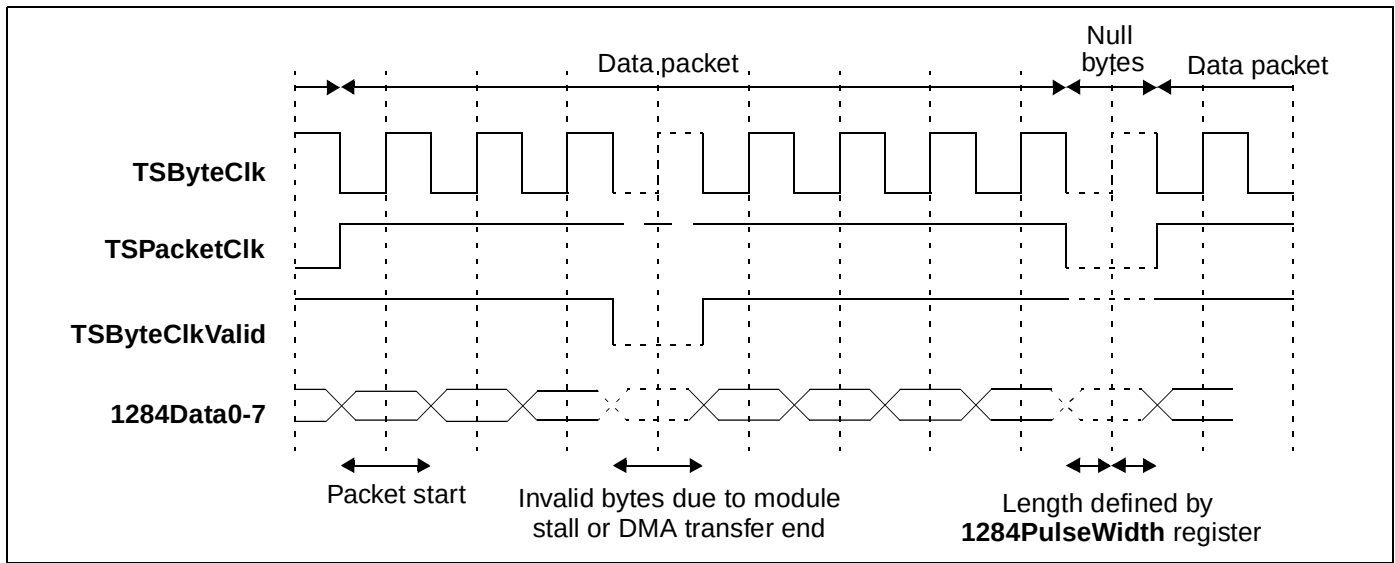


Figure 174 Packet transfer in transport stream mode A

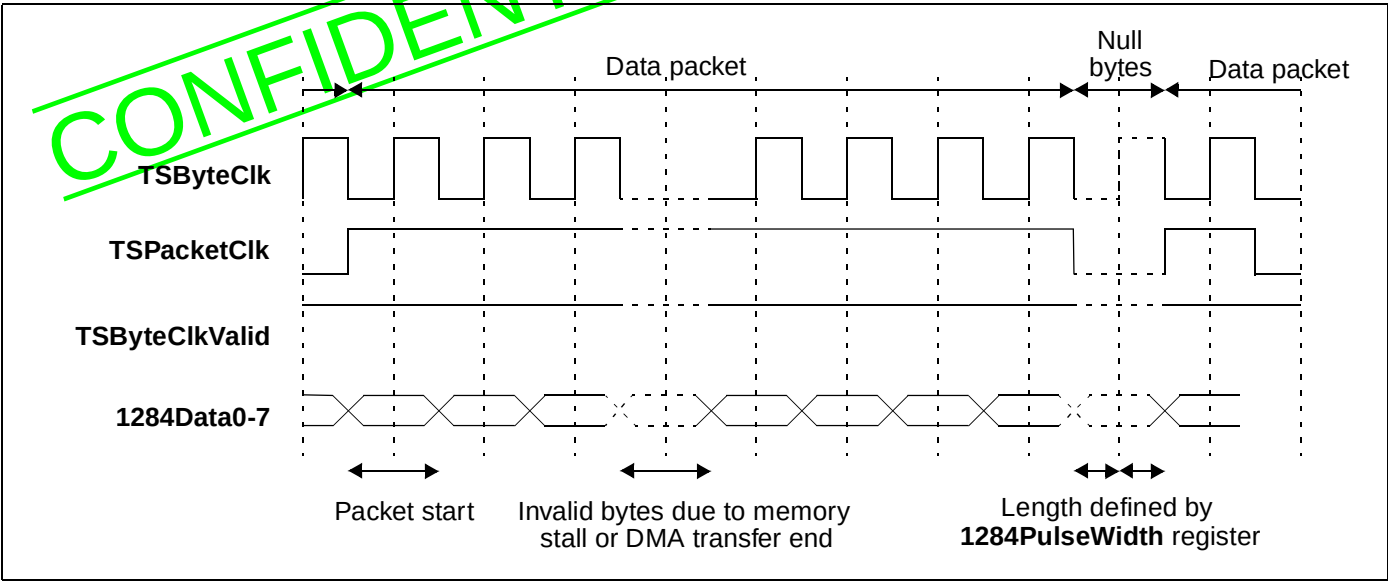


Figure 175 Packet transfer in transport stream mode B

34.3.6 Data rates

In transport mode the data throughput is a function of the memory speed, the byte clock rate and the packet size. Assuming an average memory speed of 12 cycles, a sustained data rate of 8 Mbytes/s can be maintained for word-aligned accesses for large packets.

34.3.7 Software mode

Software mode supports direct software control of the IEEE 1284 port, via the relevant control registers.

The peripheral may set the value of the output pins, control the value and direction of the data bus, and examine the input pins.

Interrupts may be set to occur if the input pins fail to match a pattern.

Data tokens may be transferred to and from the DMA engines.

In software mode the IEEE 1284 port can be configured to be a 1284 master mode device.

34.3.8 1284 master mode

This mode can only be used in software mode, as in [Section 34.3.7](#). The port can be programmed to be in master mode by writing a 1 to bit 15 of the **EMI_ConfigPadlogic** register. In this mode the function and direction of the IEEE 1284 port control signals are changed to support master mode. This is shown in [Table 124](#). Note the control signal pins must be directly controlled by software. The input control signals are input by reading the **1284PinIn** register and the output control signals are output by writing to the **1284PinOut** register.

Slave mode (default) pin name and function	Pin direction	Master mode pin function	Pin direction
1284notSelectIn	I	1284notFault	O
1284notFault	O	1284notSelectIn	I
1284notInit	I	1284Select	O
1284Select	O	1284notInit	I

Table 124 1284 master mode control signals



Slave mode (default) pin name and function	Pin direction	Master mode pin function	Pin direction
1284notSelectIn	I	1284notFault	O
1284notAutoFd	I	1284PError	O
1284PError	O	1284notAutoFd	I
1284HostLogicH	I/O (PIO pin input)	1284Busy	O
1284Busy	O	1284HostLogicH	I
1284notStrobe	I	1284notAck	O
1284notAck	O	1284notStrobe	I

Table 124 1284 master mode control signals

34.3.9 Signal filtering

All IEEE 1284 control inputs (i.e. all inputs except the data bus) have a digital filter to remove signal glitches less than the system clock period.

35 Electrical specifications

35.1 Absolute maximum ratings

Symbol	Parameter	Min	Max	Units
VDDmax	DC supply voltage		4.5	V
VImax	Voltage on input and bi-directional pins	GND-0.6	5.5	V
VOmax	Voltage on output pins	GND-0.6	VDD+0.6	V
IOmax	DC output current		25	mA
TSmax	Storage temperature (ambient)	-55	125	°C
TAmx	Temperature under bias (ambient)	-55	125	°C

Table 125 Absolute maximum ratings

Note Stresses greater than those listed under 'Absolute maximum ratings' may cause permanent damage to the device. This is a stress rating only and functional operation of the device at these or any other conditions above those indicated in the operating sections of this specification is not implied. Exposure to absolute maximum rating conditions for extended periods may affect reliability.

35.2 Operating conditions

Symbol	Parameter	Min	Max	Typical	Units	Notes
VI	Input voltage	0	5.5		V	1
CL	Load capacitance per pin		60		pF	2
CLD	Load capacitance per data pin		60		pF	
CLA	Load capacitance per address/strobe pin		100		pF	
CLP	Load capacitance per PIO pin		400		pF	
TA	Operating temperature (ambient)	0	70		°C	
PD	Typical power dissipation			1.6	W	3
	Maximum power dissipation at 3.6volts		2		W	3
PDlp	Power dissipation (low power mode)		400		mW	4
PDpd	Power dissipation (power down mode)		5		mW	5

Table 126 Operating conditions

Notes:

- Excursions beyond the supplies are permitted but not recommended.
- Excluding **LinkOut** load capacitance and EMI pin load capacitance.
- Measured on an application board at $f_{\max}$ 108MHz
- Power supplied to both RTCVdd and Vdd supplies and PLL still running, but internal clocks stopped.
- Power removed from Vdd but power remaining on RTCVdd to allow the real time clock to continue operating.

35.3 DC specifications

Symbol	Parameter	Min	Typical	Max	Units	Notes
VDD	Positive supply voltage	3.0	3.3	3.6	V	
VIH	Input logic 1 voltage	2.0		5.5	V	
VIL	Input logic 0 voltage	-0.5		0.8	V	
IIN	Input current (input pin)			±10	μA	1
IOZ	Off state digital output current			±50	μA	2
IOZPIO	Peak off state PIO input/output current			±200	μA	3
IOZEMI	Peak off state EMI input/output current			1	mA	3
IwpPIO	Input weak pull-up current on PIO pins			30	μA	4
VOH	Output logic 1 voltage	2.4			V	5
VOL	Output logic 0 voltage			0.4	V	5
CIN	Input capacitance (input pins)			10	pF	
CIO	Input capacitance (bi-directional pins)			15	pF	
COUT	Output capacitance			15	pF	

Table 127 DC specifications

Notes

- $0 \leq V_I \leq 5.5$
- $0 \leq V_I \leq V_{DD}$ and $4.5 < V_I < 5.5$
- $V_{DD} \leq V_I \leq 4.5\zeta$
- $0 \leq V_I \leq V_{DD}$
- $I_{load} = 14 \text{ mA}$ for IEEE1284, 4 mA for PIO, 2 mA for all other outputs

35.4 DAC specifications

Test	Value
Nominal voltage	3.3V
RL=200	Ohms
RIref= 11k	Ohms
DLE/ILE tests	1K point ramp (10bits) is acquired. 10 points per step are captured
THD tests	1MHz sinusoide with a DAC clock at 27MHz was used

Table 128 DAC test conditions

Symbol	Parameter	Min	Typ	Max	Unit
R Iref	Reference current resistor	6.13	11.0		kΩ
Vout	DAC output voltage from code 0 to code max	1.0		1.2	V
	DAC to DAC Vout mismatch (per tri-DAC)	-4		4	%
ILE	LF Integral non-linearity	-2		2	LSB
DLE	LF Differential non-linearity	-2		2	LSB
THD	Total harmonic distortion			-55	dB

Table 129 DAC specifications

36 Timing specifications

The timings are based on the following conditions unless otherwise stated:

- 1 Input rise and fall times of 3ns (10% -> 90%).
- 2 Output load = 30pF
- 3 Output threshold = 1.5V

36.1 SDRAM

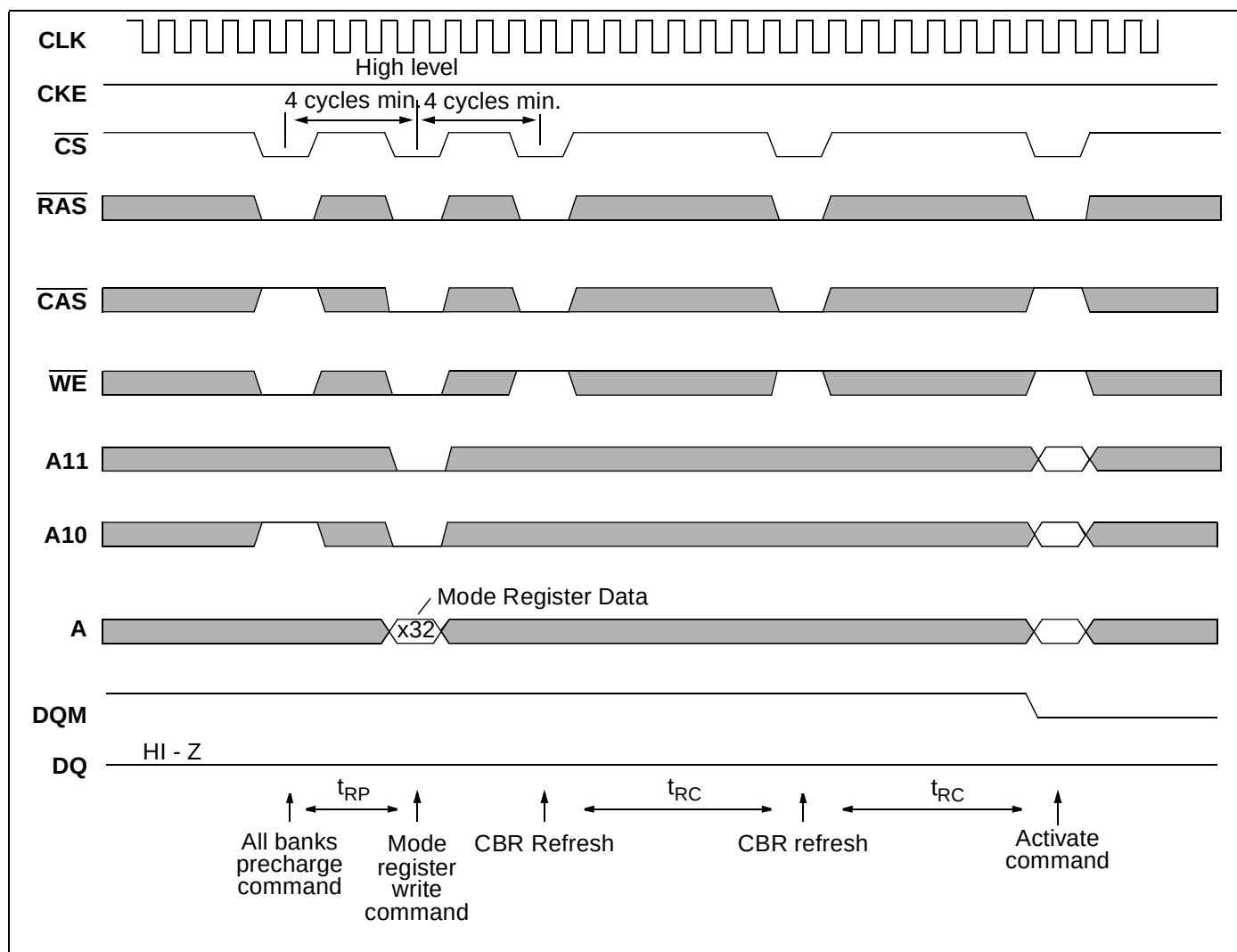


Figure 176 Synchronous DRAM power-on sequence

CONFIDENTIAL

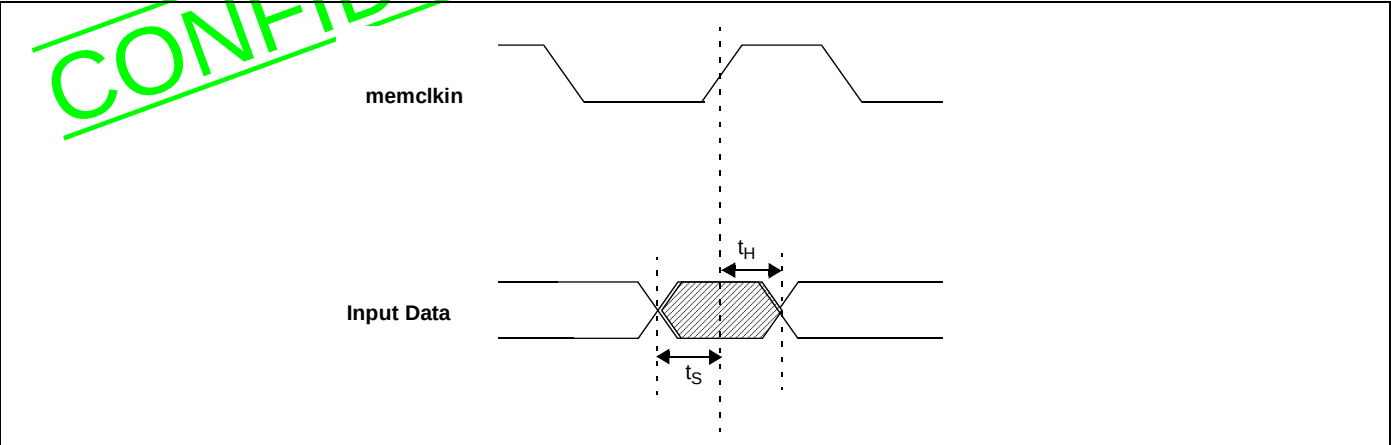


Figure 177 AC parameters for read and write (synchronous DRAM)

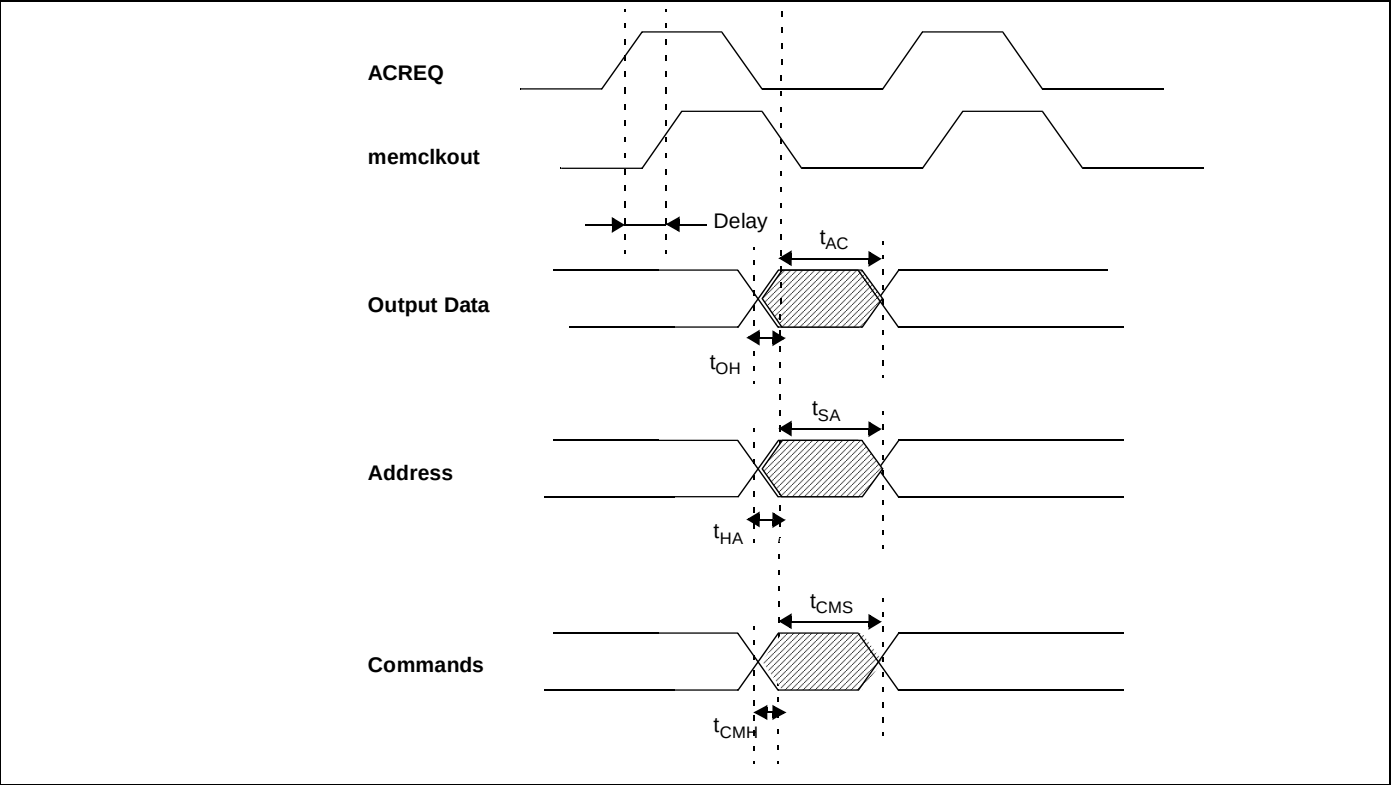


Figure 178 AC parameters for read and write (synchronous DRAM)

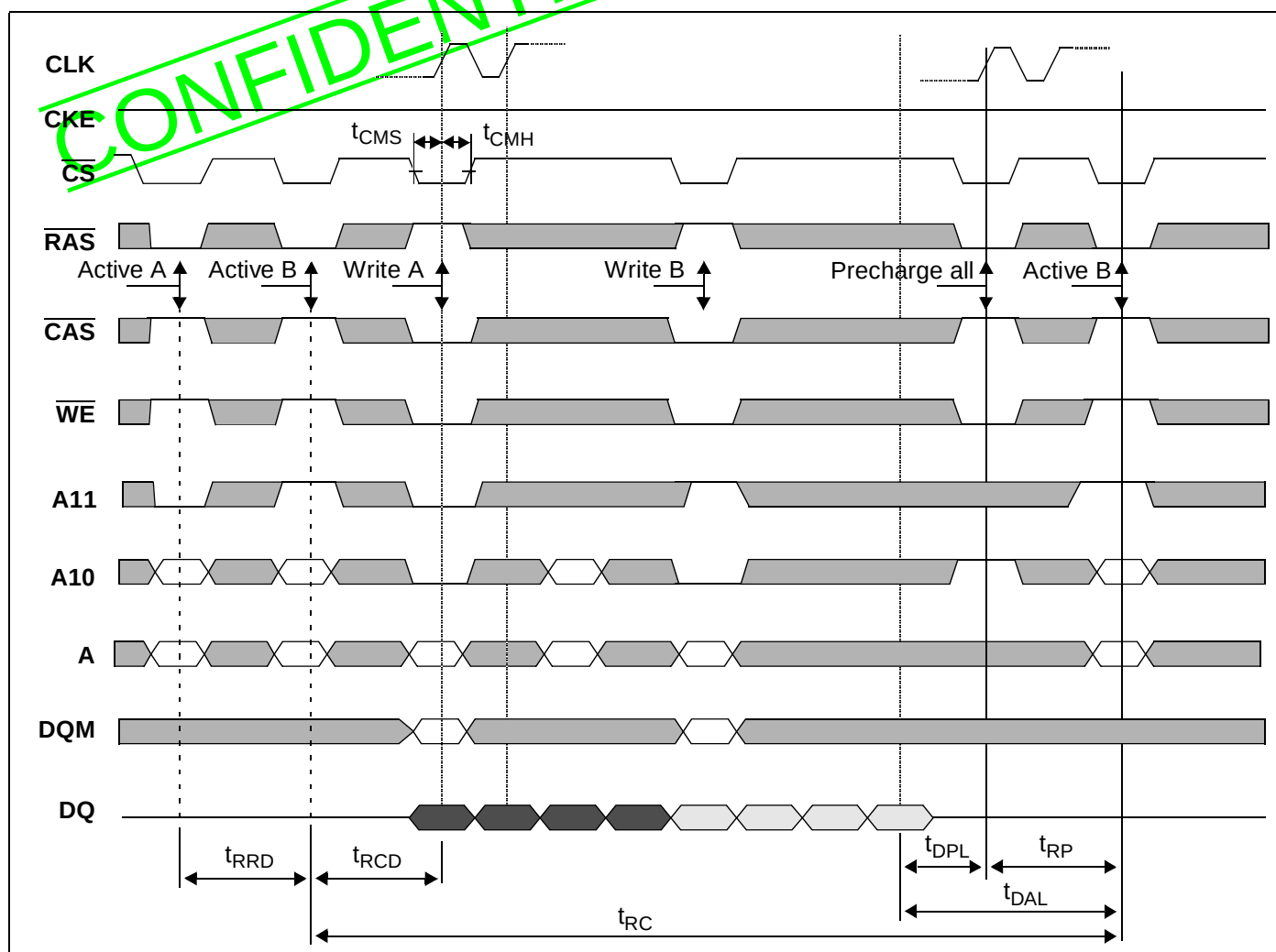


Figure 179 Synchronous DRAM write (burst length = 4, CAS latency = 3)

In the following table, the unit T is the period of the memory subsystem clock, which is typically 100MHz.

Symbol	Parameter	Min	Max	Units	Notes
t_{RP}	ACTIVE to PRE Command Period	3		T	
t_{RC}	REF to REF / ACTIVE Command Period	8		T	
t_{CK}	Clock Cycle time	9.25		ns	1
t_{CH}	Clock HIGH level width			ns	1, 2
t_{CL}	Clock LOW level width			ns	1, 2
t_S	Data input setup time	3		ns	
t_H	Data input hold time	2		ns	
t_{AC}	Output data access time		2.4	ns	
t_{OH}	Output data hold time	-3.6		ns	3
t_{SA}	Address access time		2.4	ns	
t_{HA}	Address hold time	-2.9		ns	3
t_{CMS}	Command ($\overline{CS}$, $\overline{RAS}$, $\overline{CAS}$, $\overline{WE}$, DQM) access time		2.4	ns	
t_{CMH}	Command ($\overline{CS}$, $\overline{RAS}$, $\overline{CAS}$, $\overline{WE}$, DQM) hold time	-3.6		ns	3
t_{RCD}	Delay time ACTIVE to READ / WRITE command	4		T	
t_{RRD}	ACTIVE(A) to ACTIVE(B) Command period	4		T	
t_{DAL}	Data-out to ACTIVE command period	5		T	
t_{DPL}	Data-out to Precharge Command period	2		T	
t_{RAS}	ACTIVE to PRECHARGE Command Period	9		T	

Table 130 Synchronous DRAM read and write

- 1 A 50% duty cycle is only obtained using the hard wired divide by 2, which is recommended.
- 2 A 50% duty cycle is recommended.
- 3 Negative values mean that the measured event is *before* the falling edge of **memclkout**.

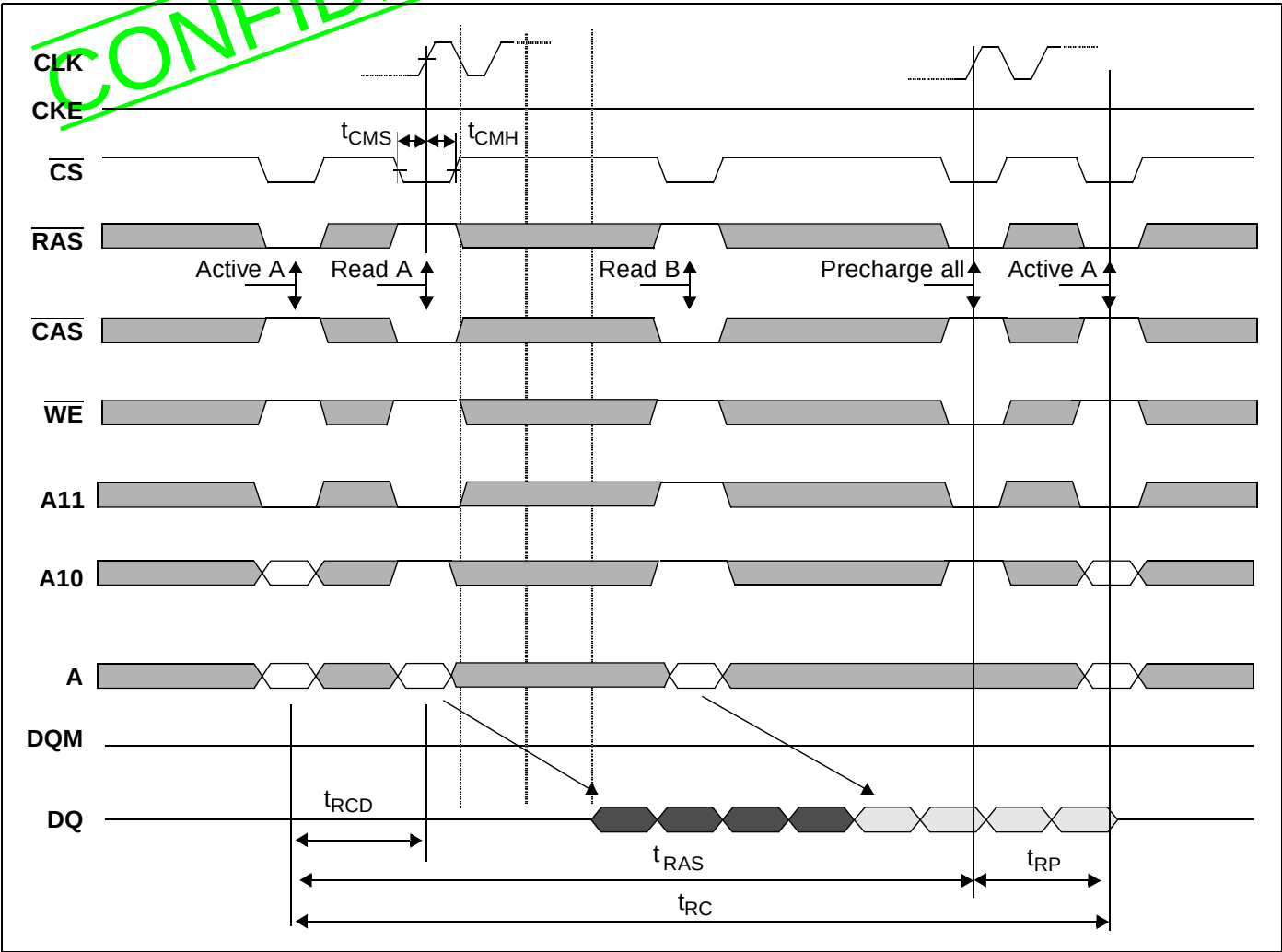


Figure 180 Synchronous DRAM read (burst length = 4, CAS latency = 3)



36.2 PCM/AC-3 decoder interface

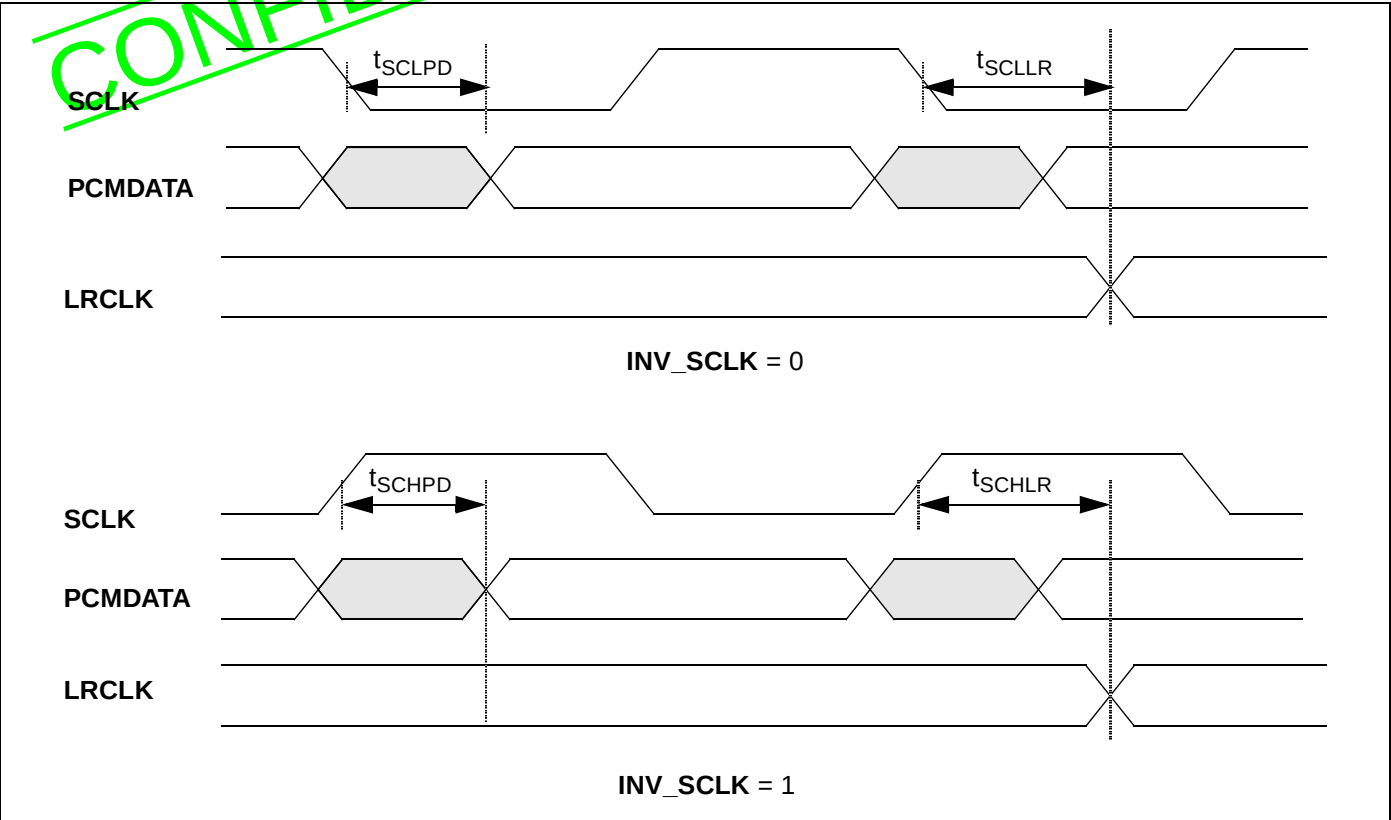


Figure 181 PCM data output

Symbol	Parameter	Min	Max	Units	Notes
t_{SCLPD}	SC low to PCMDATA valid		50	ns	
t_{SCLLR}	SC low to LRCLK		50	ns	
t_{SCHPD}	SC high to PCMDATA valid		50	ns	
t_{SCHLR}	SC high to LRCLK		50	ns	

Table 131 PCM data output

CONFIDENTIAL

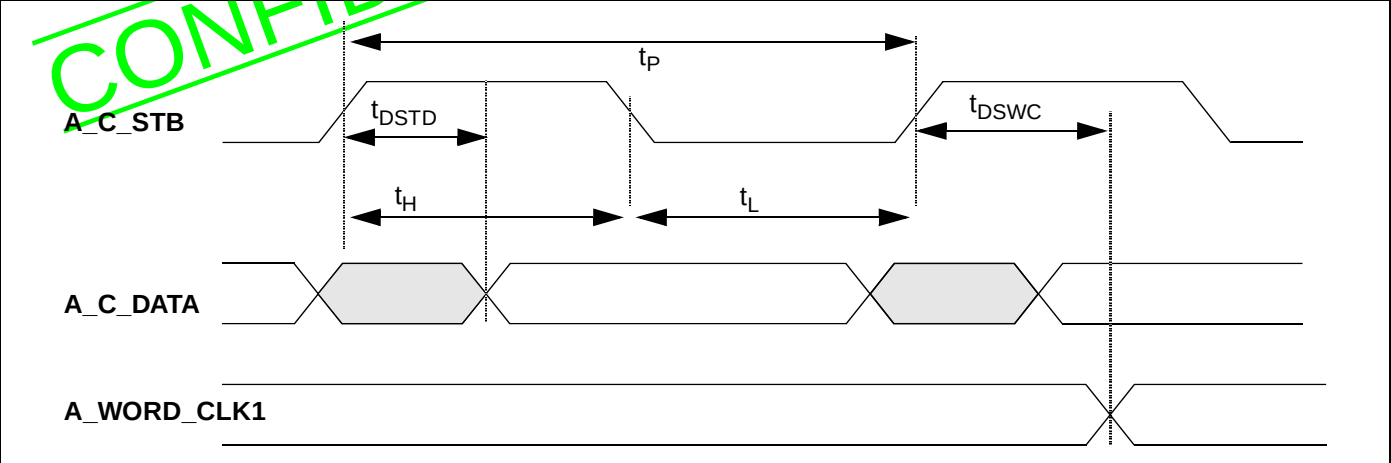


Figure 182 AC-3 decoder interface

Symbol	Parameter	Min	Max	Units	Notes
t_{DSTD}	Delay strobe to data	30	40	ns	
t_{DSWC}	Delay strobe to word clock	30	40	ns	
t_P	Strobe period	70		ns	
t_H	Strobe pulse width, high	30		ns	
t_L	Strobe pulse width, low	30		ns	

Table 132 AC-3 decoder interface



36.3 EMI timings

The *EMI Reference Clock* used in the EMI timings is a virtual clock and is defined as the point at which all positively edged EMI strobe and address outputs are valid. This removes process dependent skews from the datasheet description and highlights the dominant influence of address and strobe timings on memory system design.

Symbol	Parameter	Min	Max	Units	Note
tCHAV	Reference Clock high to Address valid	-8.0	0.0	ns	
tCLSV	Reference Clock low to Strobe valid	-8.0	3.0	ns	
tCHSV	Reference Clock high to Strobe valid	-8.0	0.0	ns	
tRDVCH	Read Data valid to Reference Clock high	13.0		ns	
tCHRDV	Read Data hold after Reference Clock high	-2.0		ns	
tSVRDX	Read Data hold after Strobe valid	0.0		ns	1
tCLWDV	Reference Clock low to Write Data valid	-8.0	7.0	ns	1
tCHWDV	Reference Clock high to Write Data valid	-8.0	6.0	ns	1
tCHWDZ	Reference Clock high to write data tristate	-8.0	6.0	ns	
tCHRSV	Reference Clock high to remaining Strobes valid	-8.0	3.0	ns	
tCHPH	Reference Clock high to ProcClkOut high	-8.0	0.0	ns	
tWVCH	MemWait valid to Reference Clock high	13.0		ns	
tRVCH	MemReq valid to Reference Clock high	13.0		ns	
tPHWX	MemWait hold after ProcClkOut high	0.0		ns	1
tPHRX	MemReq hold after ProcClkOut high	0.0		ns	1

Table 133 EMI cycle timings

Note 1 Minimum values are guaranteed by design.

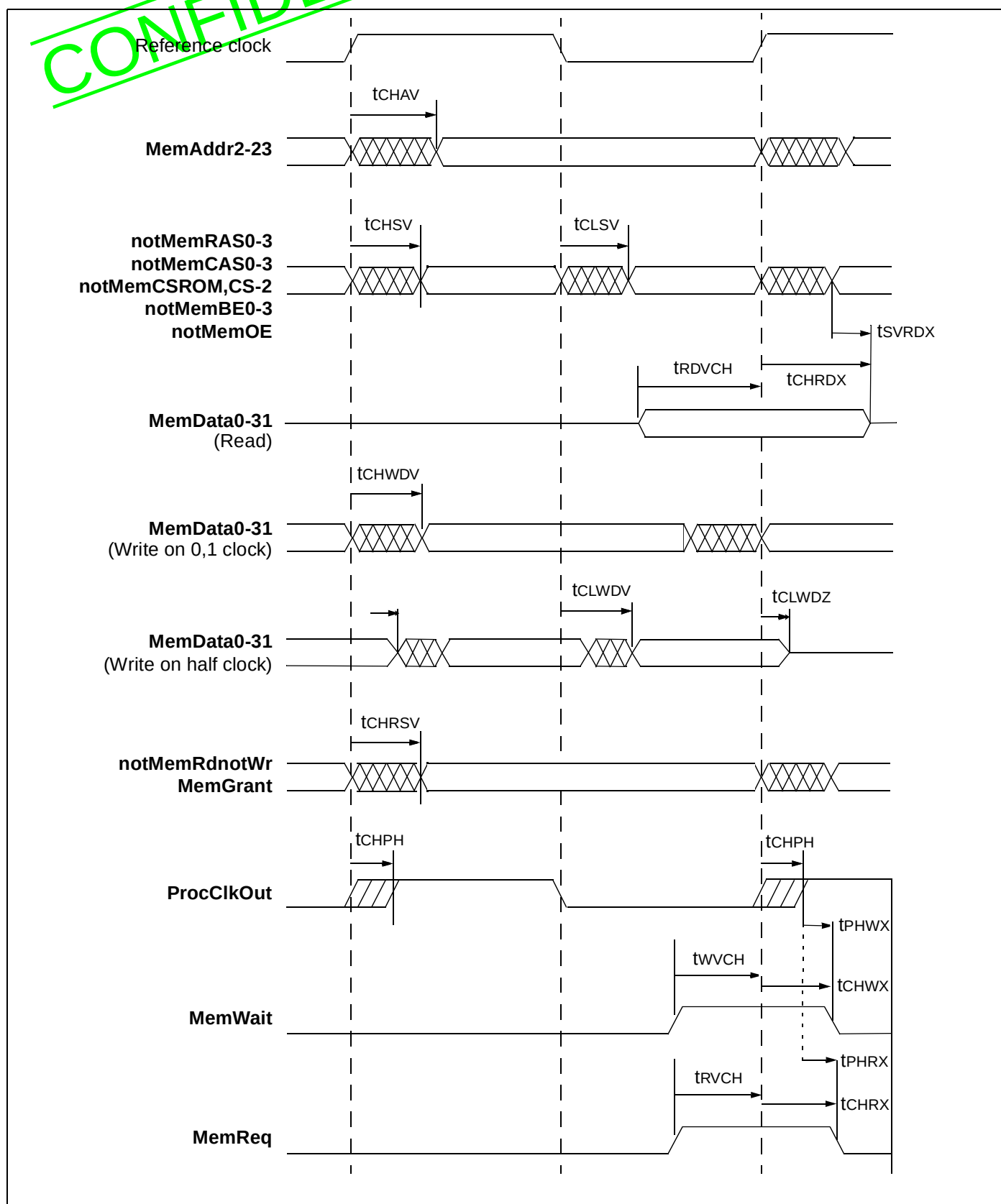


Figure 183 EMI timings

36.4 PIO timings

Reference clock in this case means the last transition of any PIO signal.

Symbol	Parameter	Min	Max	Units	Note
tPCHPOV	PIO_refclock high to PIO output valid	-15.0	0.0	ns	1
tPCHWDZ	PIO tristate after PIO_refclock high	-15.0	5.0	ns	1
tPIOr	Output rise time	7.0	30.0	ns	
tPIOf	Output fall time	7.0	30.0	ns	

Table 134 PIO timings

Note 1: These timings do not apply when a PIO pin is an output for the PWM outputs, since these are generated using a different clock.

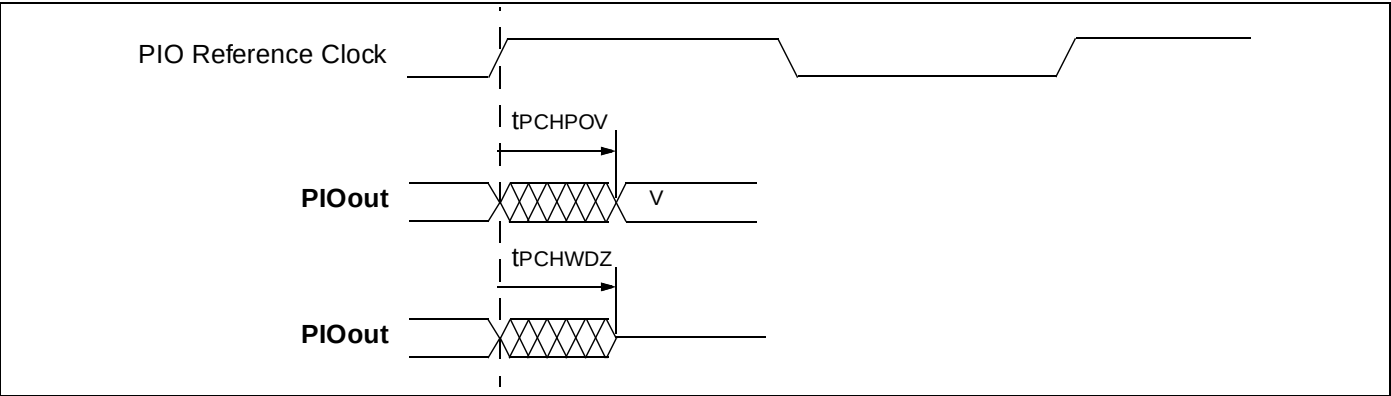


Figure 184 PIO timings

36.5 OS-Link timings

Symbol	Parameter	Min	Nom	Max	Units	Notes
tJQr	LinkOut rise time			20	ns	
tJQf	LinkOut fall time			10	ns	
tJDr	LinkIn rise time			20	ns	
tJDf	LinkIn fall time			20	ns	
tJQJD	Buffered edge delay	0			ns	
ΔtJB	Variation in tJQJD 20 Mbits/s			3	ns	1
	10 Mbits/s			10	ns	1
	5 Mbits/s			30	ns	1
CLIZ	LinkIn capacitance @ f=1MHz			10	pF	
CLL	LinkOut load capacitance			50	pF	

Table 135 OS-Link timings

Note This is the variation in the total delay through buffers, transmission lines, differential receivers etc, caused by such things as short term variation in supply voltages and differences in delays for rising and falling edges.

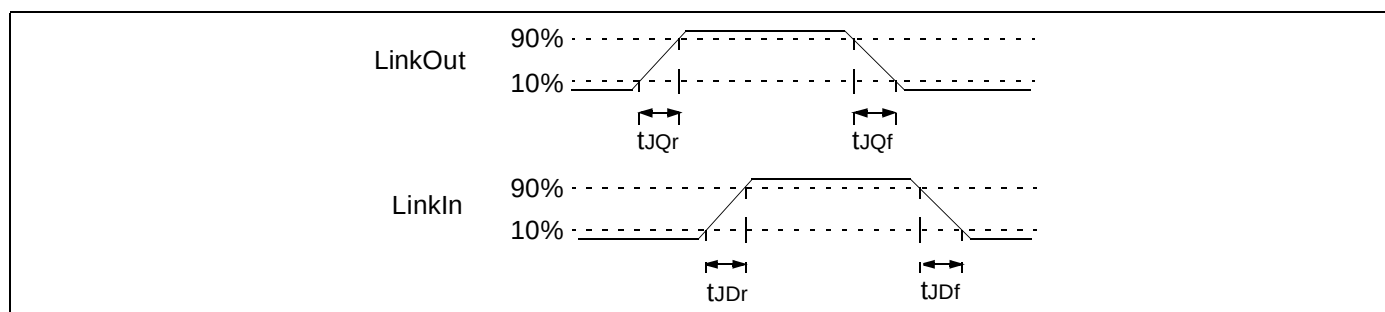


Figure 185 Link timings

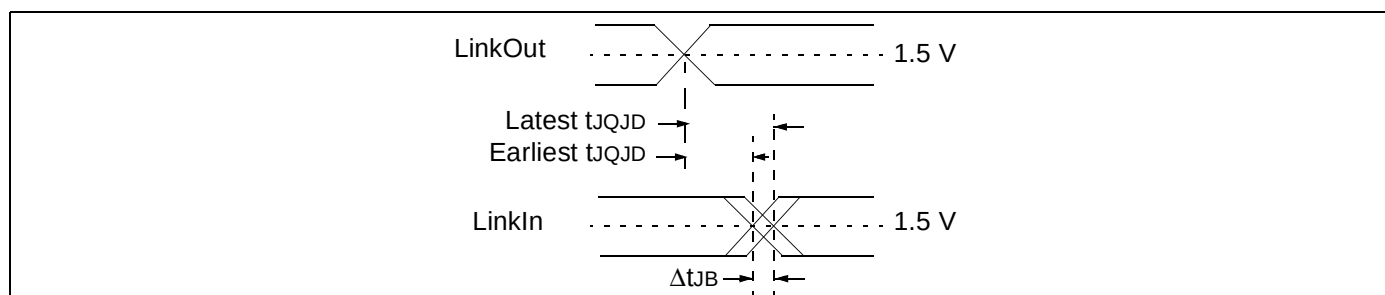


Figure 186 Buffered Link timings

36.6 Reset and analyze timings

The $t_{RSTHRSTL}$ value in the following figure and table are for power-on reset when the device is cold. Warm reset parameters when the clock and power supply are stable, will fall within these parameters. The $t_{RSTHRSTL}$ value should be used for both cases.

Symbol	Parameter	Min	Nom	Max	Units
$t_{RSTLRSTH}$	notRST pulse width low	2			ClockIn
t_{RHRL}	CPUReset pulse width high	1			ClockIn
t_{AHRH}	CPUAnalyse setup before CPUReset	3			ms
t_{RLAL}	CPUAnalyse hold after CPUReset end	1			ClockIn

Table 136 Reset and analyze timings

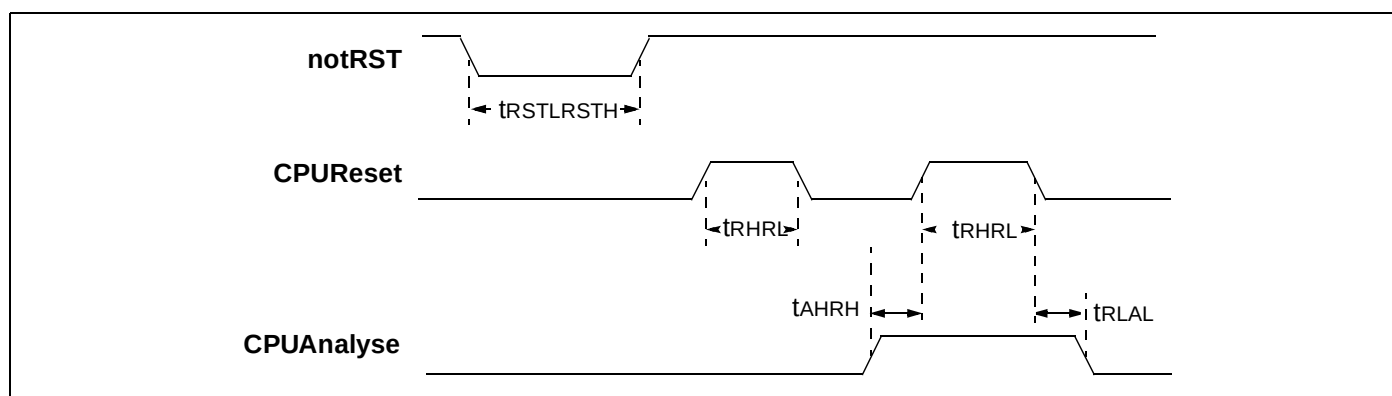


Figure 187 Reset and analyze timings

The $t_{RSTHRSTL}$ value specified in the following table and figure is for power-on reset when the device is cold. Warm reset parameters (when the clock and power supply are stable) will fall within these limits and the $t_{RSTHRSTL}$ value should be used for both cases.

Symbol	Parameter	Min	Nom	Max	Units	Notes
$t_{RSTHRSTL}$	notRST pulse width low with a stable VDD	10			ms	Takes temp, voltage and process variations into account, and provides guardband. Crystal oscillator start-up times can be long and may dominate the time from power-up to the end of reset.

Table 137 Power-on (hard) reset timings

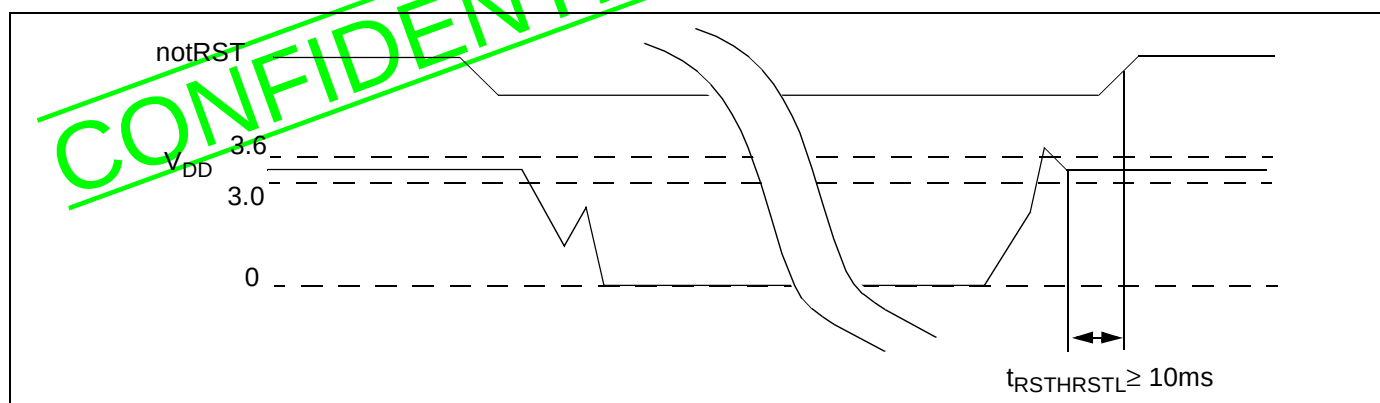


Figure 188 Reset timings

36.7 Clock timings

36.7.1 ClockIn timings

Symbol	Parameter	Min	Nom	Max	Units	Notes
tDCLDCH	ClockIn pulse width low	6			ns	
tDCHDCL	ClockIn pulse width high	10			ns	
tDCLDCL	ClockIn period		37		ns	1, 2
tDCr	ClockIn rise time			10	ns	3
tDCf	ClockIn fall time			10	ns	3

Table 138 ClockIn timings

Notes

- 1 Measured between corresponding points on consecutive falling edges.
- 2 Variation of individual falling edges from their nominal times.
- 3 Clock transitions must be monotonic within the range VIH to VIL.

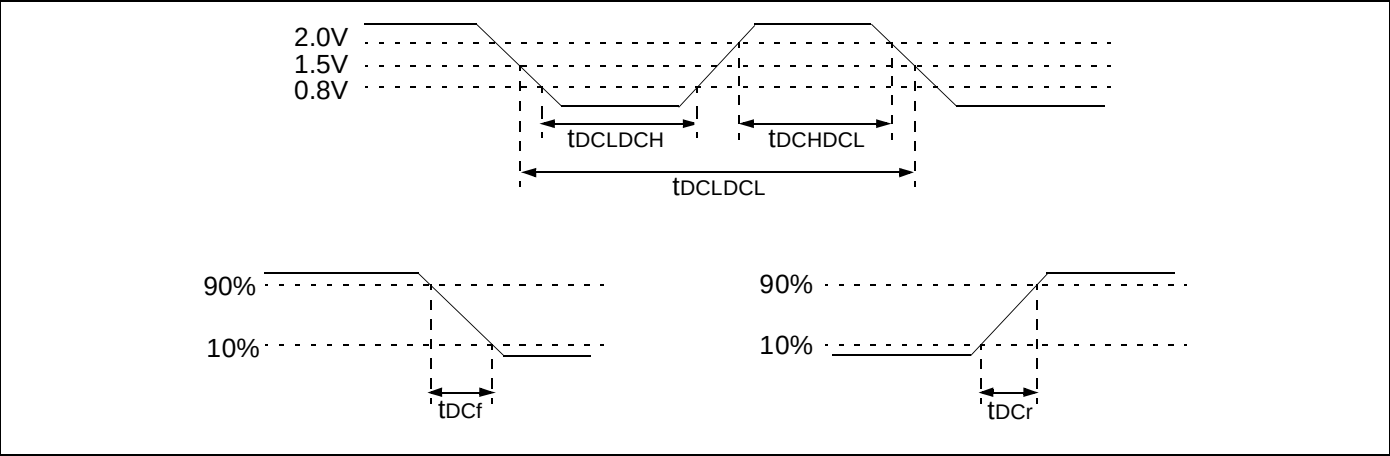


Figure 189 ClockIn timings

36.8 TAP timings

Symbol	Parameter	Min	Nom	Max	Units
t_{TCHTCH}	TCK period	50			ns
t_{TIVTCH}	TAP inputs valid to TCK high	10			ns
t_{TCHTIX}	TAP input hold after TCK high	10			ns
t_{TCHTOV}	TCK low to TAP output valid			50	ns

Table 139 TAP timings

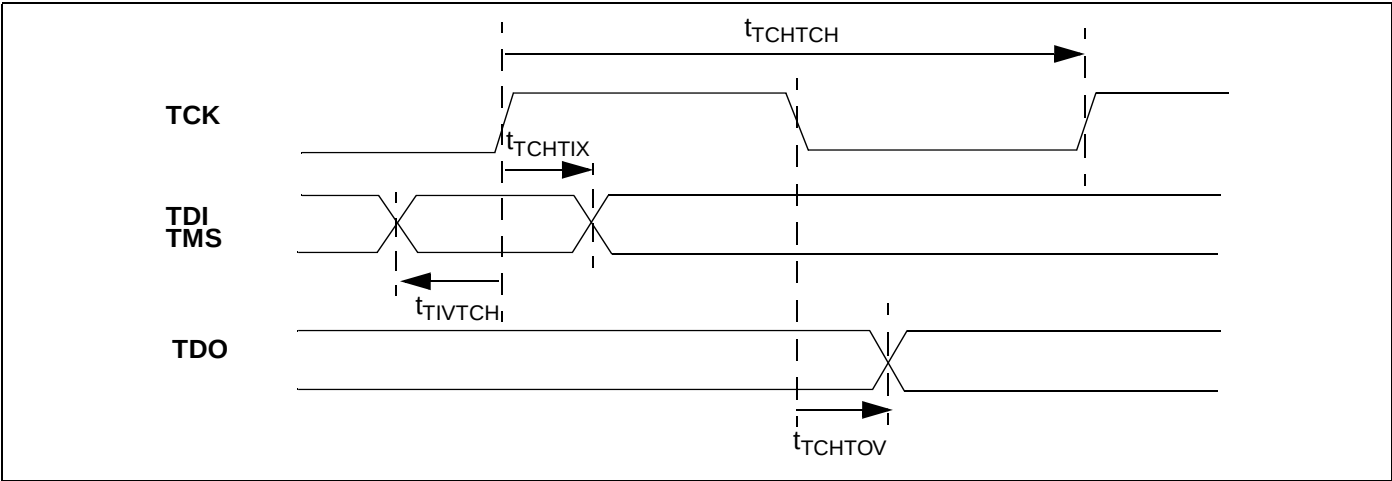


Figure 190 TAP timings



36.9 Transport stream timings

36.9.1 TSIn interface

Symbol	Parameter	Min	Nom	Max	Units	Notes
tLCLLCL	TSInByteClk period	100			ns	
tLCHLCL	TSInByteClk pulse width high	10			ns	
tLCLLCH	TSInByteClk pulse width low	10			ns	
tLDVLCH	TSIn signals valid to TSInByteClk high	10			ns	
tLCHLDX	TSIn signals hold after TSInByteClk high	0			ns	

Table 140 TSIn timings

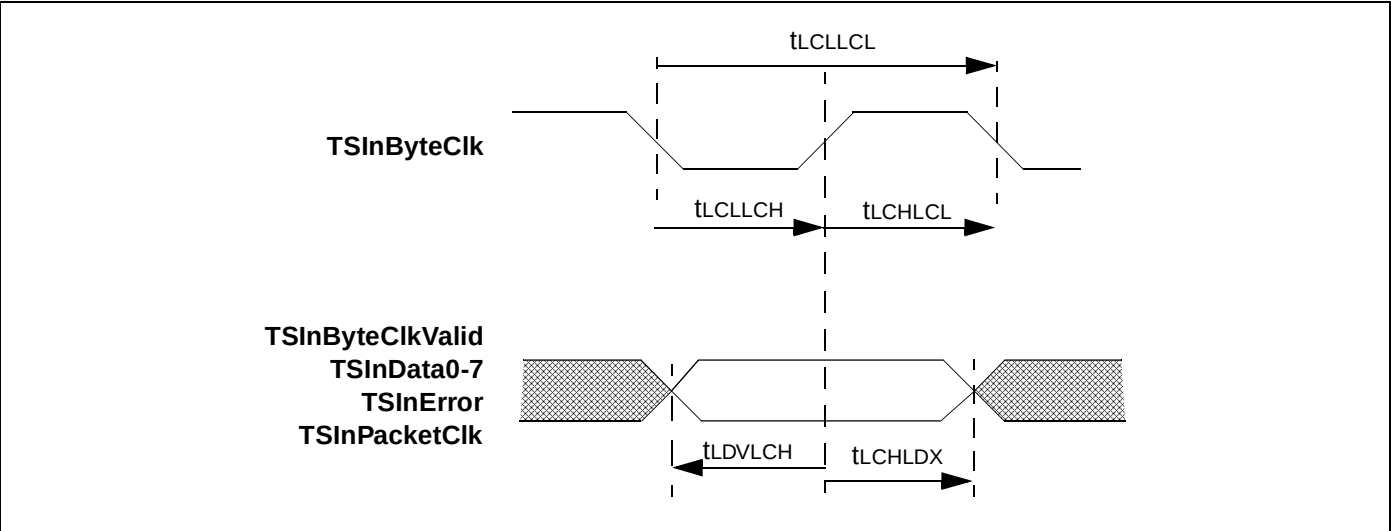


Figure 191 Transport stream input timings

36.9.2 AVStream interface on the LLI/1284 pins in AV mode

Symbol	Parameter	Min	Nom	Max	Units	Notes
t_{ACLACL}	AVByteClk period	100			ns	
t_{ACHACL}	AVByteClk pulse width high	40			ns	
t_{ACLLAH}	AVByteClk pulse width low	40			ns	
t_{ADVACH}	LinkIC inputs valid to AVByteClk or TSInByteClk high	10			ns	1
t_{ACHADX}	LinkIC input signal hold after AVByteClk high	0			ns	1
t_{ACLADV}	AVByteClk low to output data valid	0		20	ns	

Table 141 AV timings

Note 1: The inputs are refer to the clock that is selected by the **LLIControl** register.

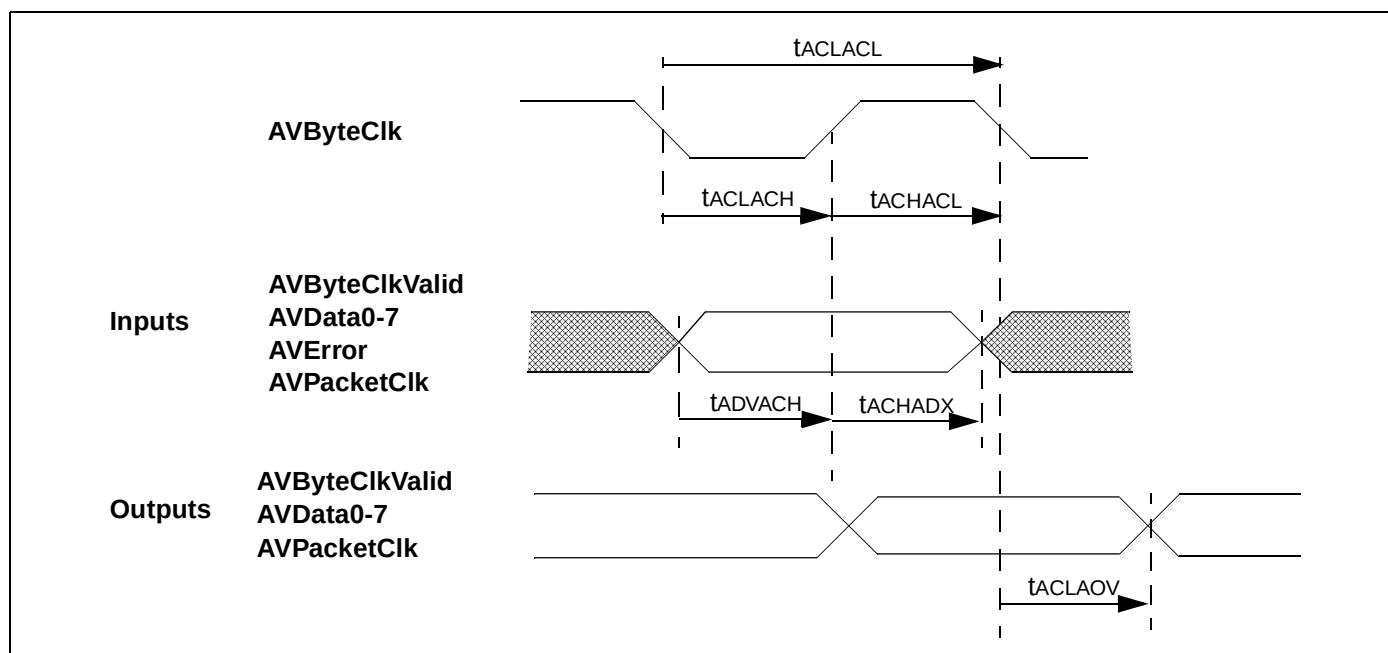


Figure 192 AV stream I/O timings

36.10 Teletext timings

36.10.1 Teletext Data Out

Symbol	Parameter	Min	Nom	Max	Units	Notes
tTRVCIH	TeletextRequest setup time to clockin high	10			ns	
tCIHTRX	TeletextRequest hold time from clockin high	0			ns	
tCIHTTOX	TeletextData output hold after clockin high	8			ns	
tCIHTTOV	Clockin high to Teletext data output valid			25	ns	

Table 142 Teletext Out

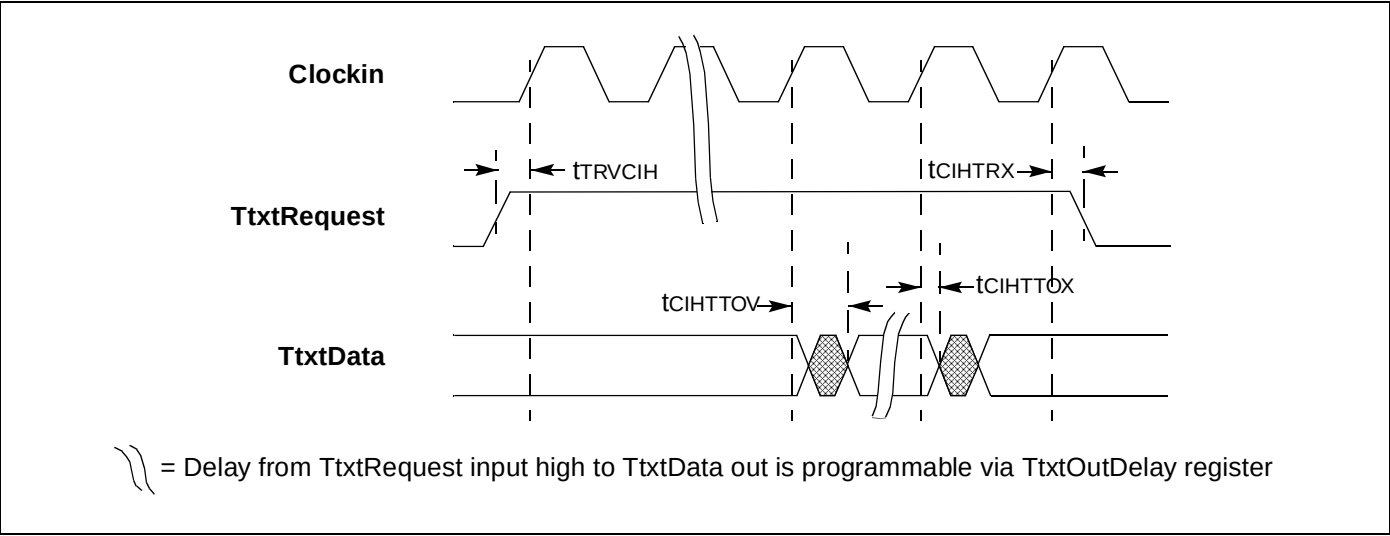


Figure 193 Teletext Out

36.10.2 Teletext Data In

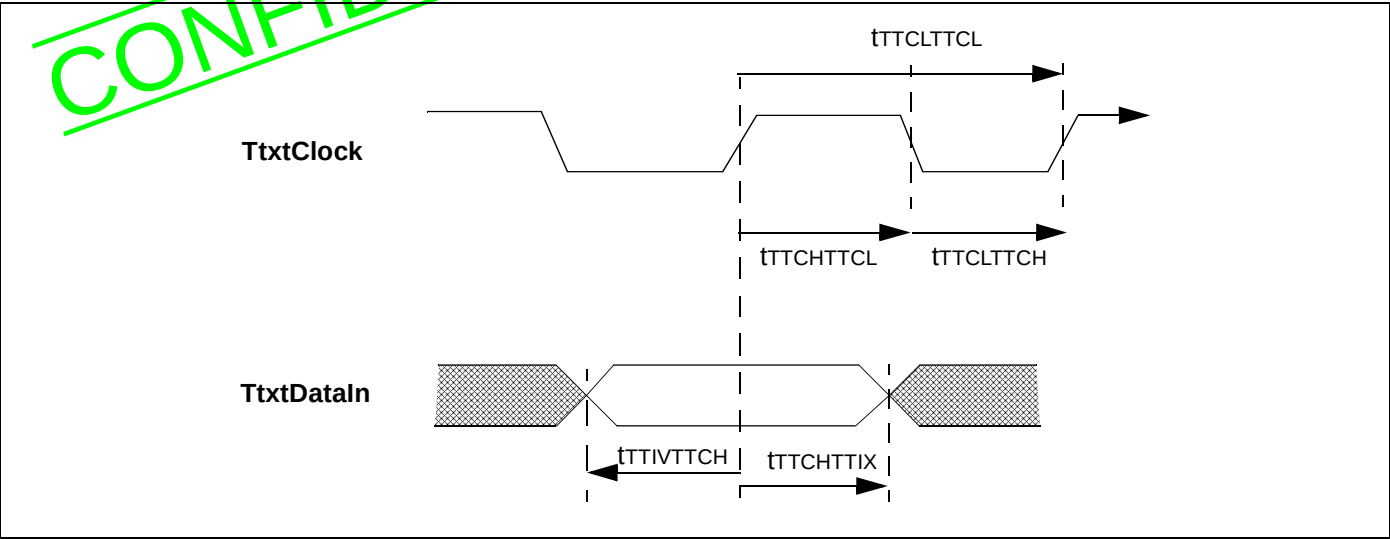


Figure 194 Teletext data in

Symbol	Parameter	Min	Nom	Max	Units	Notes
tTCHTTCH	TtxtClock period	140				
tTCHTTCL	TtxtClock pulse width high	30			ns	
tTCLTTCH	TtxtClock pulse width low	30			ns	
tTIVTTCH	Teletext data in setup time to teletext clock high	0			ns	
tTCHTTIX	Teletext data in hold time from teletext clock high			120	ns	1

Table 143 Teletext data in

Note 1: tTCHTTIX is expressed in clock cycles i.e. 111ns = 3 x 27MHz clock cycles.



36.11 IEEE 1284 timings

The reference clock shown below is a virtual clock and is defined as the point at which all output strobes are valid.

Symbol	Parameter	Min	Nom	Max	Units	Notes
tIRCHIOV	i1284_refclock high to 1284 outputs valid.	-15		0	ns	
tIIVIIX	i1284 inputs valid time.	120			ns	1
tIRCHIOV	1284Innotout low to data enable	0			ns	2
tIIVIIX	1284Innotout low to data tristate			15	ns	2

Table 144 IEEE 1284 Timings

Notes

- 1 tIIVIIX is expressed in clock cycles i.e. 111ns = 3 x 27MHz clock cycles.
- 2 xextBusDriection control bit set to 0

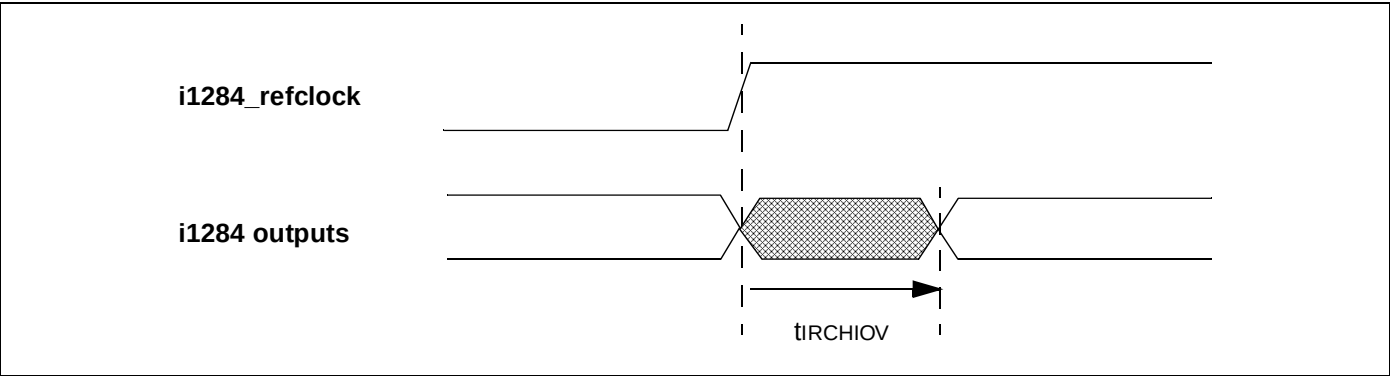


Figure 195 i1284 Output Skews

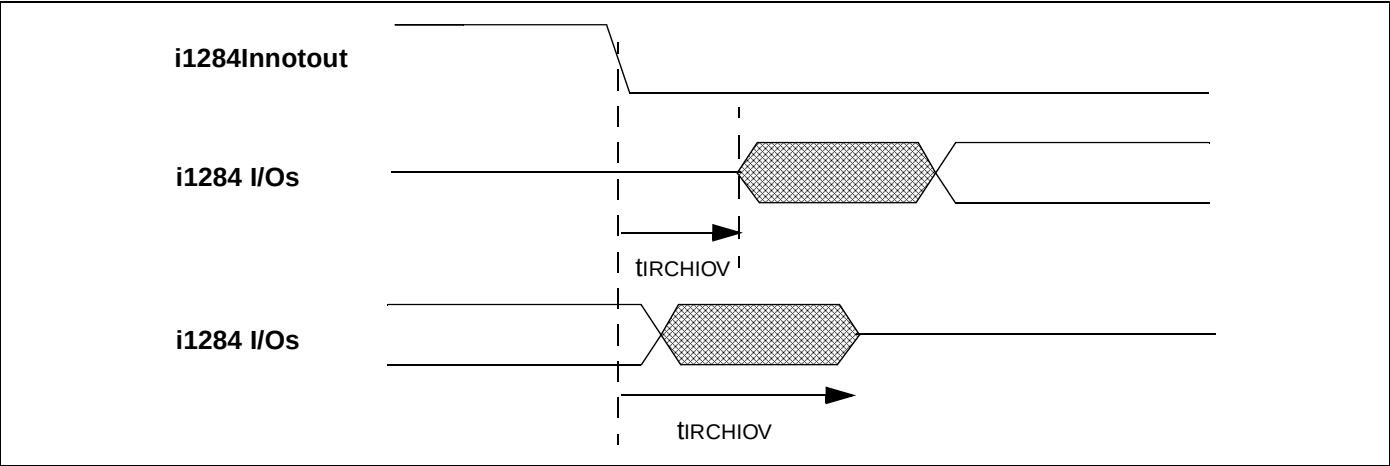


Figure 196 i1284inNotOut to i1284 I/O Enable & Disable

Symbol	Parameter	Min	Nom	Max	Units	Notes
T _{1284slew}	Output slew rate	0.05		0.4	V/ns	Cload =50pF

Table 145 i1284 Pad slew rate

37 Revision history

37.1 7155560A to 7145560B

Section	Change
1284 master mode	1284 master mode on page 276 : New subsection
DAC	Triple video DACs on page 219 : New chapter
Display	On-screen display (OSD) on page 167 and OSD region specification header on page 176: Added reference to OSD 3500A Compatibility Mode 4:2:2 Output control on page 184: Clarification of text and table. Block-to-row filtering modes for luma and chroma data on page 163: Mode 1 and 2 inverted. OSD block header format on page 176: Bit P definition inverted. Tc value inverted in tables Table 99, Table 100, Table 101, Table 102 and Table 103
Reset	Power-on (hard) reset on page 96: $T_{RSTHRSTL}$ timing information removed. Power-on (hard) reset on page 96: "LPClockIn clock source" bullet added. Note added to the "notTRST TAP reset pin" bullet.
2D block move	Figure 111: How to handle overlaps on page 186 : Figure changed for clarification. Clarification of text on page 185 and page 186
EMI	Chapter re-written for clarification.
Pin list	Power supply pins on page 19: Notes on VCLAMP Video output interface pins on page 19: Notes on 5V tolerance Table 8: STi5512 External memory interface pins on page 21: 6 lower pins in the table, previously omitted from version A, have been replaced. Table 5: AC-3/MPEG1 audio output interface pins on page 20 and Pin-out on page 25: Pins A_C_REQ and A_PTS_STB prefixed with "not" for active low.
UART	Asynchronous serial controller on page 239 : New chapter
Video	32-bit word addressing for the CPU on page 134 : 19-bit address corrected to 21-bit address
Electrical spec and timing	Table 125: Absolute maximum ratings on page 278: $V_{I\max}$ reduced from 5.75 to 5.5 changed. Table 126: Operating conditions on page 278: V_I reduced from 5.75 to 5.5, V_O removed. New power dissipation values added. DAC specifications on page 279: New section added and V_{IH} reduced from 5.75 to 5.5. Timing specifications on page 280: Sentence removed that explained that the part was under characterization. Figure 178: AC parameters for read and write (synchronous DRAM) on page 281: Timing diagram updated. Table 130: Synchronous DRAM read and write on page 283: Parameters updated and footnote 3 added. EMI timings on page 287, PIO timings on page 289, TAP timings on page 294 and IEEE 1284 timings on page 299: Rise and fall time diagrams removed because they were incomplete. Reset and analyze timings on page 291: Section re-written, timing diagram added.
Denc	Digital encoder (DEN) on page 66 : Serial R/W changed to R/W
	AC-3 interface on page 229 : Signal A_C_REQ changed to notA_C_REQ, and A_PTS_STB changed to notA_PTS_STB. Reference to A_IRQ removed.

Table 146 7155560A to 7145560B

37.2 STi5512-STi5510

The table below lists the changes that have been made to the STi5510 Data Sheet version 7112463A, to create this document. The changes listed below are hypertext-linked into the document where they are also identified by change bars.

Section	Change
Pin changes	Table 4: Video output interface pins on page 19: Pins YCO-7 and CFC added. Table 8: STi5512 External memory interface pins on page 21: all new pins Table 9: Shared SDRAM interface pins on page 21: all new pins Table 13: Transport stream input pins on page 22: all new pins Table 17: Alternative function of PIO pins on page 24: Alternate functions added to PIO port4[bit3] ASC3 CTS, PIO port4[bit4] ASC3 RTS. All pin changes are reflected in Pin-out on page 25.
64Mbit memory mode	Changes detailed in Figure 16: Memory map on page 52 , region 1 address space description on page 53 , and Table 51: STi5512 memory map on page 53 .
Memory cacheability	Changes detailed in: EMI accessible memory on page 56 and Cacheable and non-cacheable memory locations on page 60 .
EMI	Extensive re-write of chapter External memory interface on page 63 , particularly with respect to Section 11.5.2: SDRAM access on page 83.
Subpicture decoder	Explanation of register bits SPD_CTL1.bot_top, for field selection in Subpicture decoder operation on page 145 .
Display planes	Overview: Figure 83: Graphics and display subsystem on page 150: YCrCb digital output included in diagram and explained in the text below. Still picture plane: Formats of the still picture plane in memory on page 151: New format for storing the still picture plane in memory. Description of the TDL picture-start X/Y co-ordinate (pan/scan vector), set by register TDL_SCN. HSCN bits have been added to the register and horizontal resolution is to 1/8 pixel. Still pictures on page 151. MPEG video plane: Figure 87: MPEG video plane pipeline on page 153: added for clarification. MPEG video pan/scan vectors on page 160: MPEG video pan/scan is now defined by line and not field, by registers VID_SCN and VID_YDO.SCN. - Vertical filter selection: Figure 96 and Figure 97 on page 165 moved from the register manual to the datasheet, and new filter mode 27 included. MPEG video vertical filter on page 161 and Table 89 on page 166: Vertical filter can be set as field-based or frame based by register VID_DIS.FBS. OSD plane: page 177. Option to double the OSD linked list granularity, enabling the an OSD region to be placed anywhere in the whole 64Mbit SDRAM instead of just the bottom 32Mbit. Mixing OSD with video on page 180: re-written.
Digital Encoder	Chapter rewritten to improve clarity, describe support for placement of teletext, WSS and VPS on any line, VBI insertion in square pixel mode. and the Genlock feature described in Genlocking using external sync extraction on page 197 .
Electrical specification	Table 127: DC specifications on page 279 : V _{DD} 3.3v +/-5% instead of 3.3v +/-10%
MPEG audio	Figure 153: LRCLK polarity on page 223 : corrected. PCM clock generation section removed.

CONFIDENTIAL

Information furnished is believed to be accurate and reliable. However, STMicroelectronics assumes no responsibility for the consequences of use of such information nor for any infringement of patents or other rights of third parties which may result from its use. No license is granted by implication or otherwise under any patent or patent rights of STMicroelectronics. Specifications mentioned in this publication are subject to change without notice.

This publication supersedes and replaces all information previously supplied. STMicroelectronics products are not authorized for use as critical components in life support devices or systems without express written approval of STMicroelectronics.

The ST logo is a registered trademark of STMicroelectronics

© 1999 STMicroelectronics - All Rights Reserved

STMicroelectronics GROUP OF COMPANIES

Australia - Brazil - Canada - China - France - Germany - Italy - Japan - Korea - Malaysia - Malta - Mexico - Morocco
The Netherlands - Singapore - Spain - Sweden - Switzerland - Taiwan - Thailand - United Kingdom - U.S.A.

<http://www.st.com>